

The Technology Landscape

How Computers, Networks, AI, and Devices Create the Experiences We Depend
On

Edmund Gunn, Jr.

Table of contents

1 The Technology Landscape	1
1.1 Start here	1
2	3
Manuscript status	i
2.1 WORKING FULL-MANUSCRIPT DRAFT	i
Rights notice	iii
Know first	v
2.2 Non-claims (keep visible)	v
How to use this book	vii
2.3 Navigation hub	vii
2.4 Suggested first pass	vii
2.5 What “working draft” means here	vii
Reader pathways	ix
Device Quartet	xi
2.6 Device Quartet (learning laboratory)	xi
2.7 Integrity rules	xi
WAIKE	xiii
2.8 WAIKE explanation	xiii
2.9 What WAIKE is	xiii
2.10 What WAIKE is not	xiii
Evidence and status legend	xv
Preface	xvii
	xix
1 Chapter 1 — Technology Is a System, Not a Screen	1
1.1 1. The moment	1
1.2 2. What you notice	1
1.3 3. Exploded ecosystem	2
1.3.1 Human	3

1.3.2	Visible interface	3
1.3.3	Hidden local system	3
1.3.4	Optional network and remote service	3
1.3.5	Human-usable result	4
1.4	4. Follow the signal	4
1.4.1	Alternate paths (the honesty rule)	5
1.5	5. Component cards	6
1.5.1	Visible interface / UI	6
1.5.2	Application / runtime	6
1.5.3	OS services / scheduling	6
1.5.4	Storage	6
1.5.5	Network interface	6
1.5.6	Remote service	7
1.6	6. Stability contract	7
1.7	7. Try it	8
1.7.1	LAB-SYS-001 — Name the System Behind a Familiar “Open”	8
1.8	8. Build it	10
1.8.1	Explorer	10
1.8.2	Operator	10
1.8.3	Builder	10
1.8.4	Engineer	10
1.8.5	Researcher	11
1.9	9. Secure and include it	11
1.9.1	Security	11
1.9.2	Privacy	11
1.9.3	Accessibility	11
1.9.4	Equity	11
1.10	10. Career lens	11
1.11	11. Check understanding	12
1.12	References	13
1.13	12. Glossary links	13
1.14	Figure references (embedded above; accessibility metadata)	14
1.14.1	FIG-CH01-001 — First-minute ecosystem map	14
1.14.2	FIG-CH01-002 — Inputs → processing → state → outputs	14
1.14.3	FIG-CH01-003 — Why “the app” is not one thing	14
1.14.4	FIG-CH01-004 — Local-only vs network-dependent readiness	14
1.14.5	FIG-CH01-005 — Failure-domain map	14
1.14.6	FIG-CH01-006 — Exploded ecosystem preview	15
1.14.7	FIG-CH01-007 — Stability Contract preview	15
2	Chapter 2 — Follow One Tap Through the Entire Stack	17
2.1	1. The moment	17
2.2	2. What you notice	18
2.3	3. Exploded ecosystem	18
2.3.1	Human	18
2.3.2	Input hardware	20
2.3.3	Compute hardware	20
2.3.4	System software	20
2.3.5	Application	20
2.3.6	Networking (when applicable)	20
2.3.7	Output	20

2.3.8	Human feedback	21
2.4	4. Follow the signal	21
2.4.1	Alternate paths (the honesty rule)	22
2.5	5. Component cards	24
2.5.1	Touch digitizer	24
2.5.2	Device driver	24
2.5.3	Kernel	24
2.5.4	Scheduler	24
2.5.5	Event loop	24
2.5.6	RAM	25
2.5.7	GPU	25
2.5.8	Packet	25
2.6	6. Stability contract	25
2.7	7. Try it	27
2.7.1	LAB-TAP-001 — Observe a tap-to-response path on a device you already own	27
2.8	8. Build it	28
2.8.1	Explorer	28
2.8.2	Operator	29
2.8.3	Builder	29
2.8.4	Engineer	29
2.8.5	Researcher	29
2.9	9. Secure and include it	29
2.9.1	Security	29
2.9.2	Privacy	29
2.9.3	Accessibility	29
2.9.4	Equity	30
2.10	10. Career lens	30
2.11	11. Check understanding	31
2.12	References	31
2.13	12. Glossary links	31
2.14	Figure references (embedded above; accessibility metadata)	33
2.14.1	FIG-CH02-001 — One Tap: Human-to-System Overview	33
2.14.2	FIG-CH02-002 — Cross-Layer Tap Sequence	33
2.14.3	FIG-CH02-003 — Representative Device Exploded View	33
2.14.4	FIG-CH02-004 — Software Stack	33
2.14.5	FIG-CH02-005 — Local vs Network-Dependent Tap	34
2.14.6	FIG-CH02-006 — Tap Latency Budget	34
2.14.7	FIG-CH02-007 — Stability Contract	34
2.15	Claim footnotes used in this chapter (project-specific)	34
3	Chapter 3 — Performance: Why Technology Feels Fast, Slow, Smooth, or Unstable	37
3.1	1. The moment	37
3.2	2. What you notice	38
3.3	3. Exploded ecosystem	38
3.3.1	Human	38
3.3.2	Application and runtime	39
3.3.3	Operating system	39
3.3.4	Compute and memory hardware	39
3.3.5	Network (optional)	39

3.3.6	Power and thermal policy	39
3.3.7	Society and equity	39
3.4	4. Follow the signal	39
3.4.1	Four diagnostic axes (keep them separate)	40
3.4.2	Local contention with a healthy icon	40
3.5	5. Component cards	41
3.5.1	Perceived performance	41
3.5.2	Latency	41
3.5.3	Jitter	41
3.5.4	Throughput	41
3.5.5	Stall / hitch	41
3.5.6	Contention	42
3.5.7	Thermal / power limit (qualitative)	42
3.6	6. Stability contract	42
3.7	7. Try it	43
3.7.1	LAB-PERF-001 — Make Feel Visible	43
3.8	8. Build it	44
3.8.1	Explorer	44
3.8.2	Operator	44
3.8.3	Builder	45
3.8.4	Engineer	45
3.8.5	Researcher	45
3.9	9. Secure and include it	45
3.9.1	Security	45
3.9.2	Privacy	45
3.9.3	Accessibility	45
3.9.4	Equity	45
3.10	10. Career lens	46
3.11	11. Check understanding	46
3.12	References	47
3.13	12. Glossary links	47
3.14	Figure references (conceptual plates; accessibility metadata)	48
3.14.1	FIG-CH03-001 — Feel Words to Diagnostic Axes	48
3.14.2	FIG-CH03-002 — Action to Hitch Causal Flow	48
3.14.3	FIG-CH03-003 — Same Average, Different Jitter	48
3.14.4	FIG-CH03-004 — Local vs Network Contributors to Feel	48
3.14.5	the blocked CMS measured plate — Annotated commodity monitor snapshot (BLOCKED)	49
3.15	Claim footnotes used in this chapter	49
4	Chapter 4 — The Device Quartet as a Learning Laboratory	51
4.1	1. The moment	51
4.2	2. What you notice	52
4.3	3. Exploded ecosystem	52
4.3.1	Human	53
4.3.2	Desk clamshell lens (Student 14.5-inch)	53
4.3.3	Mobile/docked lens (Handheld Hybrid)	53
4.3.4	Learn-to-build lens (DS-XL Coder)	53
4.3.5	Embodied sensing lens (Edge IO Wearables)	53
4.3.6	Shared under-layers (every lens)	53
4.4	4. Follow the signal	54

4.4.1	Alternate honesty paths	54
4.5	5. Component cards	55
4.5.1	Learning laboratory	55
4.5.2	Research form factor	55
4.5.3	Student 14.5-inch	55
4.5.4	Handheld Hybrid	56
4.5.5	DS-XL Coder	56
4.5.6	Edge IO Wearables	56
4.5.7	Constraint contrast	56
4.5.8	PHYSICAL_PENDING	56
4.6	6. Stability contract	56
4.7	7. Try it	58
4.7.1	LAB-QUARTET-001 — Constraint contrast without Quartet hardware	58
4.8	8. Build it	59
4.8.1	Explorer	59
4.8.2	Operator	59
4.8.3	Builder	59
4.8.4	Engineer	59
4.8.5	Researcher	60
4.9	9. Secure and include it	60
4.9.1	Security	60
4.9.2	Privacy	60
4.9.3	Accessibility	60
4.9.4	Equity	60
4.10	10. Career lens	60
4.11	11. Check understanding	61
4.12	References	61
4.13	12. Glossary links	62
4.14	Figure references (embedded above; accessibility metadata)	62
4.14.1	FIG-CH04-001 — Device Quartet as a Learning Laboratory	63
4.14.2	FIG-CH04-002 — Same Task Across Form Factors	63
4.14.3	FIG-CH04-003 — Constraint Matrix (Conceptual)	63
4.15	Claim footnotes used in this chapter (project-specific)	63
5	Chapter 5 — Electricity, Signals, Clocks, and Logic	65
5.1	1. The moment	65
5.2	2. What you notice	66
5.3	3. Exploded ecosystem	66
5.3.1	Human	67
5.3.2	Transducer / input interface	67
5.3.3	Electrical medium	67
5.3.4	Signal	67
5.3.5	Clock (when the design is synchronous)	67
5.3.6	Logic	67
5.3.7	Effect	68
5.3.8	Optional software boundary	68
5.4	4. Follow the signal	68
5.5	5. Component cards	70
5.5.1	Electricity as useful energy and control	70
5.5.2	Signal	70
5.5.3	Analog versus digital	70

5.5.4	Clock	70
5.5.5	Logic gate / Boolean decision	71
5.5.6	Noise and integrity (qualitative)	71
5.6	6. Stability contract	71
5.7	7. Try it	72
5.7.1	LAB-SIG-001 — From Press to Logic	72
5.8	8. Build it	73
5.8.1	Explorer	73
5.8.2	Operator	73
5.8.3	Builder	73
5.8.4	Engineer	73
5.8.5	Researcher	74
5.9	9. Secure and include it	74
5.9.1	Electrical and lab safety	74
5.9.2	Security of the decision path	74
5.9.3	Privacy	74
5.9.4	Equity	74
5.9.5	Accessibility	74
5.10	10. Career lens	75
5.11	11. Check understanding	76
5.12	References	76
5.13	12. Glossary links	76
5.14	Figure references (embedded above; accessibility metadata)	77
5.14.1	FIG-CH05-001 — Press to Logic Flow	77
5.14.2	FIG-CH05-002 — Analog versus Digital	77
5.14.3	FIG-CH05-003 — Clock Edges Sequencing Decisions	77
5.14.4	FIG-CH05-004 — Logic Building Blocks	78
6	Chapter 6 — CPU, Instructions, and Parallel Work	79
6.1	1. The moment	79
6.2	2. What you notice	79
6.3	3. Exploded ecosystem	80
6.3.1	Human	80
6.3.2	Application	81
6.3.3	Process and threads	81
6.3.4	Scheduler and OS mediation	81
6.3.5	CPU cores	81
6.3.6	Accelerators (survey depth)	81
6.3.7	Adjacent layers (named, not encyclopedized)	82
6.4	4. Follow the signal	82
6.4.1	Parallelism without fairy tales	82
6.4.2	Honesty branches	83
6.5	5. Component cards	83
6.5.1	Instruction	83
6.5.2	CPU	83
6.5.3	Process / thread	84
6.5.4	Scheduler	84
6.5.5	Parallel work	84
6.5.6	Accelerator / GPU (survey)	84
6.5.7	CPU-bound vs waiting	84
6.6	6. Stability contract	85

6.7	7. Try it	86
6.7.1	LAB-CMS-001 — Make Local Slowness Visible	86
6.8	8. Build it	87
6.8.1	Explorer	87
6.8.2	Operator	87
6.8.3	Builder	87
6.8.4	Engineer	87
6.8.5	Researcher	88
6.9	9. Secure and include it	88
6.9.1	Security	88
6.9.2	Privacy	88
6.9.3	Accessibility	88
6.9.4	Equity	88
6.10	10. Career lens	89
6.11	11. Check understanding	89
6.11.1	Misconception probes	89
6.11.2	Teach-back (say it out loud)	90
6.11.3	Pathway self-check	90
6.12	References	90
6.13	12. Glossary links	90
6.14	Figure references (embedded above; accessibility metadata)	91
6.14.1	FIG-CH06-001 — App → process/threads → scheduler → cores/accelerator	91
6.14.2	FIG-CH06-002 — Single-thread vs parallelizable work	91
6.14.3	FIG-CH06-003 — CPU-bound vs waiting-on-I/O symptoms	91
6.15	Claim footnotes used in this chapter	91
7	Chapter 7 — Memory, Cache, and Storage	93
7.1	1. The moment	93
7.2	2. What you notice	93
7.3	3. Exploded ecosystem	94
7.3.1	Human	94
7.3.2	Application and processes	94
7.3.3	OS mediation	94
7.3.4	CPU (and optional accelerators)	94
7.3.5	Memory hierarchy	96
7.3.6	Storage and power	96
7.3.7	Optional network (often ruled out)	96
7.4	4. Follow the signal	96
7.5	5. Component cards	97
7.5.1	Registers	97
7.5.2	Cache	98
7.5.3	RAM (main memory)	98
7.5.4	Storage	98
7.5.5	Working set	98
7.5.6	Volatility vs durability	98
7.6	6. Stability contract	99
7.7	7. Try it	99
7.7.1	LAB-CMS-001 — Make Local Slowness Visible	99
7.8	8. Build it	101
7.8.1	Explorer	101
7.8.2	Operator	101

7.8.3	Builder	101
7.8.4	Engineer	101
7.8.5	Researcher	101
7.9	9. Secure and include it	101
7.9.1	Security	101
7.9.2	Privacy	102
7.9.3	Accessibility	102
7.9.4	Equity	102
7.10	10. Career lens	102
7.11	11. Check understanding	103
7.12	References	103
7.13	12. Glossary links	103
7.14	Figure references (embedded above; accessibility metadata)	104
7.14.1	FIG-CH07-001 — Memory hierarchy	104
7.14.2	FIG-CH07-002 — Open file path	104
7.14.3	FIG-CH07-003 — Healthy network icon with memory-pressure hitch	105
7.15	Claim footnotes used in this chapter (project-specific)	105
8	Chapter 8 — Graphics, Displays, Audio, Cameras, and Sensors	107
8.1	1. The moment	107
8.2	2. What you notice	107
8.3	3. Exploded ecosystem	108
8.3.1	Human	109
8.3.2	Sensors and transducers	109
8.3.3	Capture pipelines	109
8.3.4	Compute and composition	109
8.3.5	Presentation pipelines	109
8.3.6	Device Quartet (learning spine only)	109
8.4	4. Follow the signal	109
8.4.1	Path P — Present (stored or generated media → human)	109
8.4.2	Path C — Capture (world → representation)	110
8.4.3	Honesty rule	110
8.4.4	Contention rule	110
8.4.5	What this chapter does not invent	111
8.5	5. Component cards	111
8.5.1	Display pipeline	111
8.5.2	Frame	111
8.5.3	Audio pipeline	111
8.5.4	Camera pipeline	112
8.5.5	Sensor	112
8.5.6	Sampling	112
8.5.7	Compositor / GPU role (survey)	112
8.6	6. Stability contract	112
8.7	7. Try it	113
8.7.1	LAB-IO-001 — Map one media pipeline (permission-safe)	113
8.8	8. Build it	114
8.8.1	Explorer	114
8.8.2	Operator	115
8.8.3	Builder	115
8.8.4	Engineer	115
8.8.5	Researcher	115

8.9	9. Secure and include it	115
8.9.1	Security and privacy	115
8.9.2	Accessibility	115
8.9.3	Equity	116
8.10	10. Career lens	116
8.11	11. Check understanding	116
8.12	References	117
8.13	12. Glossary links	117
8.14	Figure references (embedded above; accessibility metadata)	118
8.14.1	FIG-CH08-001 — Media sensorium map	118
8.14.2	FIG-CH08-002 — Presentation timing and feel	118
8.14.3	FIG-CH08-003 — Sampling continuous to discrete	118
8.15	Claim footnotes used in this chapter	118
9	Chapter 9 — Power, Batteries, Thermals, and Mechanical Design	121
9.1	1. The moment	121
9.2	2. What you notice	121
9.3	3. Exploded ecosystem	122
9.3.1	Human and room	122
9.3.2	Energy sources	123
9.3.3	Conversion and distribution	123
9.3.4	Loads	123
9.3.5	Sensors and policy	123
9.3.6	Mechanical enclosure	123
9.4	4. Follow the signal	123
9.4.1	Alternate paths (honesty rule)	124
9.5	5. Component cards	125
9.5.1	Power budget	125
9.5.2	Battery	125
9.5.3	Thermal limit	125
9.5.4	Throttle	126
9.5.5	Mechanical design	126
9.5.6	Energy vs peak performance	126
9.6	6. Stability contract	126
9.7	7. Try it	127
9.7.1	LAB-PWR-001 — Budget Collapse Observation	127
9.8	8. Build it	128
9.8.1	Explorer	128
9.8.2	Operator	128
9.8.3	Builder	129
9.8.4	Engineer	129
9.8.5	Researcher	129
9.9	9. Secure and include it	129
9.9.1	Safety	129
9.9.2	Security	129
9.9.3	Privacy	129
9.9.4	Accessibility	130
9.9.5	Equity	130
9.10	10. Career lens	130
9.11	11. Check understanding	131
9.12	References	131

9.13	12. Glossary links	131
9.14	Figure references (embedded above; accessibility metadata)	132
9.14.1	FIG-CH09-001 — Energy Path with Throttle Feedback	132
9.14.2	FIG-CH09-002 — On Charger vs On Battery	132
9.14.3	FIG-CH09-003 — Enclosure Roles	132
9.15	Claim footnotes used in this chapter	133
10	Chapter 10 — Ports, Buses, Boards, Packaging, and Manufacturing	135
10.1	1. The moment	135
10.2	2. What you notice	136
10.3	3. Exploded ecosystem	136
10.3.1	Human	136
10.3.2	Port (the face you can see)	137
10.3.3	Cable / accessory packaging	137
10.3.4	Board (PCB)	137
10.3.5	Bus and protocol-on-a-wire	137
10.3.6	Packaging (chip / module / product)	137
10.3.7	Manufacturing / DFM	137
10.3.8	System software (preview)	137
10.4	4. Follow the signal	138
10.4.1	Alternate paths (the honesty rule)	138
10.4.2	Failure branch without drama	138
10.5	5. Component cards	138
10.5.1	Port	139
10.5.2	Bus	139
10.5.3	Board (PCB)	139
10.5.4	Packaging	139
10.5.5	Manufacturing / DFM	140
10.5.6	Protocol on a wire	140
10.6	6. Stability contract	140
10.7	7. Try it	141
10.7.1	LAB-BUS-001 — Name the Interconnect	141
10.8	8. Build it	142
10.8.1	Explorer	142
10.8.2	Operator	142
10.8.3	Builder	142
10.8.4	Engineer	142
10.8.5	Researcher	142
10.9	9. Secure and include it	142
10.9.1	Security	142
10.9.2	Privacy	143
10.9.3	Accessibility	143
10.9.4	Equity	143
10.9.5	Safety	143
10.9.6	Ethics	143
10.10	10. Career lens	143
10.11	11. Check understanding	144
10.12	References	144
10.13	12. Glossary links	144
10.14	Figure references (planned embeds; accessibility metadata)	145
10.14.1	FIG-CH10-001 — Board buses ports world	145

10.14.2	FIG-CH10-002 — Host request → bus transaction → device response . .	145
10.14.3	FIG-CH10-003 — Electrical layer vs protocol layer	145
10.14.4	FIG-CH10-004 — Packaging / manufacturing stages (conceptual)	146
10.15	Claim footnotes used in this chapter (project-specific)	146
11	Chapter 11 — Firmware, Boot, and Trust	147
11.1	1. The moment	147
11.2	2. What you notice	148
11.3	3. Exploded ecosystem	148
11.3.1	Human	148
11.3.2	Firmware	148
11.3.3	Root of trust	149
11.3.4	Bootloader	149
11.3.5	Operating system kernel and userspace	149
11.3.6	Trust policy vs user proof	149
11.3.7	Updates and recovery (preview)	149
11.4	4. Follow the signal	150
11.4.1	Alternate paths (the honesty rule)	151
11.4.2	Failure branch without drama	151
11.5	5. Component cards	151
11.5.1	Firmware	151
11.5.2	Boot sequence	151
11.5.3	Bootloader	152
11.5.4	Root of trust	152
11.5.5	Secure boot (intent)	152
11.5.6	Measured boot and attestation (concepts)	152
11.5.7	Recovery / fail-safe path	152
11.6	6. Stability contract	153
11.7	7. Try it	154
11.7.1	LAB-BOOT-OBS-001 — Observe Boot and Wake (publication-owned, IMPLEMENTED_DIGITAL)	154
11.8	8. Build it	155
11.8.1	Explorer	155
11.8.2	Operator	155
11.8.3	Builder	155
11.8.4	Engineer	155
11.8.5	Researcher	155
11.9	9. Secure and include it	156
11.9.1	Security — authentication vs authorization vs boot authenticity	156
11.9.2	Privacy	156
11.9.3	Accessibility	156
11.9.4	Equity	156
11.9.5	Safety	157
11.9.6	Ethics	157
11.10	10. Career lens	157
11.11	11. Check understanding	157
11.12	References	158
11.13	12. Glossary links	158
11.14	Figure references (embedded; registered SVG + ally)	159
11.14.1	FIG-CH11-001 — Power → firmware → bootloader → kernel → lock screen	159
11.14.2	FIG-CH11-002 — Root of trust → verified stages	159

11.14.3	FIG-CH11-003 — Lock screen vs secure boot	159
11.14.4	FIG-CH11-004 — Update failure → recovery → outcomes	161
11.15	Claim footnotes used in this chapter	161
12	Chapter 12 — Operating Systems, Processes, Threads, and Scheduling	163
12.1	1. The moment	163
12.2	2. What you notice	164
12.3	3. Exploded ecosystem	164
12.3.1	Human	164
12.3.2	Application	165
12.3.3	Process	165
12.3.4	Thread	165
12.3.5	Scheduler and OS mediation	165
12.3.6	CPU cores	165
12.3.7	Adjacent layers (named, not encyclopedized)	166
12.4	4. Follow the signal	166
12.4.1	Concurrency without fairy tales	166
12.4.2	Honesty branches	168
12.5	5. Component cards	168
12.5.1	Operating system	168
12.5.2	Process	168
12.5.3	Thread	168
12.5.4	Scheduler	168
12.5.5	Context switch	169
12.5.6	Process isolation	169
12.5.7	System call	169
12.5.8	Concurrency vs parallelism	169
12.6	6. Stability contract	169
12.7	7. Try it	171
12.7.1	LAB-CMS-001 — Make Local Slowness Visible	171
12.8	8. Build it	172
12.8.1	Explorer	172
12.8.2	Operator	172
12.8.3	Builder	172
12.8.4	Engineer	173
12.8.5	Researcher	173
12.9	9. Secure and include it	173
12.9.1	Security	173
12.9.2	Privacy	173
12.9.3	Accessibility	173
12.9.4	Equity	173
12.10	10. Career lens	174
12.11	11. Check understanding	174
12.11.1	Misconception probes	174
12.11.2	Teach-back (say it out loud)	175
12.11.3	Pathway self-check	175
12.12	References	175
12.13	12. Glossary links	175
12.14	Figure references (embedded above; accessibility metadata)	176
12.14.1	FIG-CH12-001 — Apps → processes/threads → scheduler → cores	176
12.14.2	FIG-CH12-002 — Context-switch swimlane	176

12.14.3	FIG-CH12-003 — Concurrency vs parallelism	176
12.14.4	FIG-CH12-004 — OS-monitor before/during (fixture)	176
12.15	Claim footnotes used in this chapter	177
13	Chapter 13 — Files, Databases, and Data Lifecycles	179
13.1	1. The moment	179
13.2	2. What you notice	180
13.3	3. Exploded ecosystem	180
13.3.1	Human	180
13.3.2	Application buffer / working set	180
13.3.3	File and filesystem	181
13.3.4	Caches and write-back	181
13.3.5	Device media	181
13.3.6	Structured stores (qualitative)	181
13.3.7	Sync and replicas (qualitative)	182
13.3.8	Lifecycle policy	182
13.3.9	System software	183
13.4	4. Follow the signal	183
13.4.1	Alternate paths (honesty rule)	183
13.5	5. Component cards	184
13.5.1	File	184
13.5.2	Filesystem	184
13.5.3	Cache vs durable store	184
13.5.4	Durability	185
13.5.5	Database (intro, qualitative)	185
13.5.6	Consistency model (intro, qualitative)	185
13.5.7	Data lifecycle	185
13.5.8	Deletion / redaction (qualitative)	185
13.6	6. Stability contract	186
13.7	7. Try it	186
13.7.1	LAB-CMS-001 — Experience B adjacency (save / quit / reopen)	186
13.8	8. Build it	188
13.8.1	Explorer	188
13.8.2	Operator	188
13.8.3	Builder	188
13.8.4	Engineer	188
13.8.5	Researcher	189
13.9	9. Secure and include it	189
13.9.1	Security	189
13.9.2	Privacy	189
13.9.3	Accessibility	189
13.9.4	Equity	189
13.9.5	Safety and ethics	189
13.10	10. Career lens	190
13.11	11. Check understanding	190
13.12	References	191
13.13	12. Glossary links	191
13.14	Figure references (embedded; registered SVG + ally)	192
13.14.1	FIG-CH13-001 — Data lifecycle (create → use → retain → share → delete/redact)	192

13.14.2	FIG-CH13-002 — App buffer → filesystem → device media → optional cloud replica	192
13.14.3	FIG-CH13-003 — Save click vs durability point	192
13.14.4	FIG-CH13-004 — File vs database responsibilities	193
13.15	Claim footnotes used in this chapter	193
14	Chapter 14 — Applications, APIs, Runtimes, and User Interfaces	195
14.1	1. The moment	195
14.2	2. What you notice	196
14.3	3. Exploded ecosystem	196
14.3.1	Human	196
14.3.2	User interface (UI)	197
14.3.3	Application logic	197
14.3.4	Runtime	197
14.3.5	Libraries and frameworks	197
14.3.6	OS services (local APIs)	197
14.3.7	Optional remote APIs	198
14.3.8	Human-usable result	198
14.4	4. Follow the signal	198
14.4.1	Alternate paths (the honesty rule)	199
14.5	5. Component cards	199
14.5.1	User interface	199
14.5.2	Application / feature logic	200
14.5.3	Runtime / event loop	200
14.5.4	Library / framework	200
14.5.5	API (contract)	200
14.5.6	Local vs remote API	200
14.5.7	Accessibility API / inclusive status	200
14.6	6. Stability contract	201
14.7	7. Try it	201
14.7.1	LAB-SYS-001 — Name the system behind a familiar “open” (CE-1 adjacency)	201
14.7.2	LAB-TAP-001 — Trace one interaction (CH02 method adjacency)	202
14.7.3	LAB-PKT-001 — When the API path leaves the device (CE-4 adjacency)	202
14.7.4	Shared interpretation rule	202
14.7.5	Optional operator extension (not a new lab ID)	202
14.8	8. Build it	202
14.8.1	Explorer	203
14.8.2	Operator	203
14.8.3	Builder	203
14.8.4	Engineer	203
14.8.5	Researcher	203
14.8.6	Educator	203
14.9	9. Secure and include it	203
14.9.1	Security	203
14.9.2	Privacy	204
14.9.3	Accessibility	204
14.9.4	Equity	204
14.9.5	Safety and ethics	204
14.10	10. Career lens	204
14.11	11. Check understanding	205

14.12	References	205
14.13	12. Glossary links	206
14.14	Figure references (embedded; registered SVG + ally)	206
14.14.1	FIG-CH14-001 — UI → logic → runtime → libraries → OS → optional remote API	206
14.14.2	FIG-CH14-002 — Event → handler → API → state → render	207
14.14.3	FIG-CH14-003 — Local API vs remote API failure domains	207
14.15	Claim footnotes used in this chapter (project-specific)	207
15	Chapter 15 — Containers, Virtualization, Cloud, and Edge Computing	209
15.1	1. The moment	209
15.2	2. What you notice	210
15.3	3. Exploded ecosystem	210
15.3.1	Human	210
15.3.2	Local device and OS	210
15.3.3	Isolation / packaging layer	210
15.3.4	Access network (not placement)	212
15.3.5	Path / Internet (survey)	212
15.3.6	Edge placement	212
15.3.7	Cloud placement	212
15.3.8	Orchestration (intro only)	212
15.3.9	Multitenancy	212
15.4	4. Follow the signal	213
15.4.1	Alternate paths (the honesty rule)	213
15.4.2	Failure domains without drama	213
15.5	5. Component cards	214
15.5.1	Virtual machine (VM)	214
15.5.2	Hypervisor	214
15.5.3	Container	214
15.5.4	Image / packaging	214
15.5.5	Cloud (placement)	215
15.5.6	Edge (placement)	215
15.5.7	Multitenancy	215
15.5.8	Orchestration (intro)	215
15.6	6. Stability contract	216
15.6.1	Concurrent conditions (qualitative; no invented numeric budgets)	216
15.6.2	Failure domains	216
15.6.3	Measurements we can seek (when available)	217
15.6.4	Measurements we cannot invent	217
15.7	7. Try it	217
15.7.1	Baseline (all pathways)	217
15.7.2	Pathway notes	217
15.8	8. Build it	218
15.8.1	Explorer	218
15.8.2	Operator	218
15.8.3	Builder	218
15.8.4	Engineer	218
15.8.5	Researcher	218
15.9	9. Secure and include it	219
15.9.1	Security	219
15.9.2	Privacy	219

15.9.3	Accessibility	219
15.9.4	Equity	219
15.9.5	Safety	219
15.9.6	Ethics	219
15.10	10. Career lens	219
15.11	11. Check understanding	220
15.12	References	221
15.13	12. Glossary links	221
15.14	Figure references (working embeds; accessibility metadata)	222
15.14.1	FIG-CH15-001 — Hardware → hypervisor/VM vs shared kernel → containers	222
15.14.2	FIG-CH15-002 — User → access network → edge vs regional cloud	222
15.14.3	FIG-CH15-003 — Same app: local vs container vs cloud URL	222
15.15	Claim footnotes used in this chapter	222
16	Chapter 16 — Packets, Protocols, Routing, and the Internet	223
16.1	1. The moment	223
16.2	2. What you notice	224
16.3	3. Exploded ecosystem	224
16.3.1	Human	224
16.3.2	Device (local scope)	224
16.3.3	Access on-ramp (not “the Internet”)	224
16.3.4	LAN scope	225
16.3.5	DNS (dependency, not the whole path)	225
16.3.6	Internet path / routing	225
16.3.7	Transport and protocols	225
16.3.8	Remote service (placement)	226
16.3.9	System software preview	226
16.4	4. Follow the signal	226
16.4.1	Failure domains without drama	227
16.5	5. Component cards	228
16.5.1	Packet	228
16.5.2	Encapsulation	228
16.5.3	Addressing	228
16.5.4	Routing	228
16.5.5	Protocol	229
16.5.6	Transport (intro)	229
16.5.7	DNS	229
16.5.8	Local vs LAN vs Internet	229
16.6	6. Stability contract	229
16.7	7. Try it	230
16.7.1	LAB-PKT-001 — Trace One Connected Action Across Path and Access	230
16.8	8. Build it	231
16.8.1	Explorer	232
16.8.2	Operator	232
16.8.3	Builder	232
16.8.4	Engineer	232
16.8.5	Researcher	232
16.9	9. Secure and include it	232
16.9.1	Security	232
16.9.2	Privacy	232

16.9.3	Accessibility	233
16.9.4	Equity	233
16.9.5	Safety	233
16.9.6	Ethics	233
16.10	10. Career lens	233
16.11	11. Check understanding	234
16.12	12. References	234
16.13	13. Glossary links	234
16.14	14. Figure references (embeds above; accessibility metadata)	235
16.14.1	FIG-CH16-001 — Local / LAN / Internet scopes	235
16.14.2	FIG-CH16-002 — Encapsulation sticky notes	235
16.14.3	FIG-CH16-003 — DNS on the critical path	235
16.14.4	FIG-CH16-004 — LAB-PKT-001 timing phases	235
16.15	15. Claim footnotes used in this chapter	236
17	Chapter 17 — Wi-Fi, Cellular, 5G, and the Road to 6G	237
17.1	1. The moment	237
17.2	2. What you notice	237
17.3	3. Exploded ecosystem	238
17.3.1	Human	239
17.3.2	Device radios and policy	239
17.3.3	Wi-Fi access (local wireless LAN)	239
17.3.4	Cellular access (operator mobile)	239
17.3.5	Radio access network and core (cellular path)	239
17.3.6	Internet path and placement (not access)	239
17.3.7	Icons, bars, and badges	239
17.4	4. Follow the signal	239
17.4.1	Alternate paths (the honesty rule)	241
17.4.2	Failure branch without drama	241
17.5	5. Component cards	241
17.5.1	Wi-Fi (IEEE 802.11 family)	241
17.5.2	Cellular access	242
17.5.3	Radio access technology (RAT)	242
17.5.4	5G system (survey)	242
17.5.5	6G roadmap	242
17.5.6	Handover / mobility (intro)	243
17.5.7	Bars vs experience	243
17.6	6. Stability contract	243
17.7	7. Try it	244
17.7.1	LAB-PKT-001 — Trace One Connected Action Across Path and Access	244
17.8	8. Build it	245
17.8.1	Explorer	245
17.8.2	Operator	245
17.8.3	Builder	245
17.8.4	Engineer	245
17.8.5	Researcher	245
17.9	9. Secure and include it	245
17.9.1	Security	245
17.9.2	Privacy	246
17.9.3	Accessibility	246
17.9.4	Equity	246

17.9.5	Safety	246
17.9.6	Ethics	246
17.10	10. Career lens	246
17.11	11. Check understanding	247
17.12	References	247
17.13	12. Glossary links	247
17.14	Figure references (embedded; registered SVG + ally)	248
17.14.1	FIG-CH17-001 — Wi-Fi AP path vs cellular RAN path vs Internet	248
17.14.2	FIG-CH17-002 — Generations survey to 5G to 6G roadmap	248
17.14.3	FIG-CH17-003 — Wi-Fi to cellular transition during a walk	249
17.15	Claim footnotes used in this chapter (project-specific)	249
18 Chapter 18 — Spectrum, Antennas, Beams, MIMO, and Radio Conditions		251
18.1	1. The moment	251
18.2	2. What you notice	252
18.3	3. Exploded ecosystem	252
18.3.1	Human	252
18.3.2	Device radios and antennas	252
18.3.3	Obstacles and geometry	253
18.3.4	Spectrum vs channel	253
18.3.5	Access network edge	253
18.3.6	Network and service beyond	253
18.3.7	Measurement honesty overlay	253
18.4	4. Follow the signal	254
18.4.1	Alternate paths (the honesty rule)	255
18.4.2	Failure branch without drama	255
18.5	5. Component cards	256
18.5.1	Spectrum	256
18.5.2	Channel	256
18.5.3	Antenna	256
18.5.4	Path loss (intro)	256
18.5.5	Interference	257
18.5.6	MIMO (survey)	257
18.5.7	Beamforming / beams (survey)	257
18.5.8	Radio condition	257
18.6	6. Stability contract	258
18.7	7. Try it	258
18.7.1	LAB-RADIO-OBS-001 — Observe Radio Conditions Without Transmitting	258
18.8	8. Build it	259
18.8.1	Explorer	259
18.8.2	Operator	260
18.8.3	Builder	260
18.8.4	Engineer	260
18.8.5	Researcher	260
18.9	9. Secure and include it	260
18.9.1	Security	260
18.9.2	Privacy	260
18.9.3	Accessibility	260
18.9.4	Equity	261
18.9.5	Safety	261
18.9.6	Ethics	261

18.1010. Career lens	261
18.1111. Check understanding	261
18.12References	262
18.1312. Glossary links	262
18.14Figure references (embedded; registered SVG + ally)	263
18.14.1 FIG-CH18-001 — Device—obstacle—AP path loss cartoon	263
18.14.2 FIG-CH18-002 — Omni vs beam pattern intuition	263
18.14.3 FIG-CH18-003 — MIMO spatial streams metaphor	263
18.14.4 FIG-CH18-004 — Quartet antenna placement with PHYSICAL_PENDING overlay	263
18.15Claim footnotes used in this chapter (project-specific)	264
19 Chapter 19 — NTN and Service Continuity Across Ground, Air, and Space	265
19.1 1. The moment	265
19.2 2. What you notice	266
19.3 3. Exploded ecosystem	266
19.3.1 Human	266
19.3.2 Device and local stack	266
19.3.3 Terrestrial path class	267
19.3.4 Air / space path class (NTN)	267
19.3.5 Core, edge, and service placement	267
19.3.6 Operator policy and capability class	267
19.3.7 Evidence boundary (project)	267
19.4 4. Follow the signal	268
19.4.1 Alternate paths (the honesty rule)	268
19.4.2 Failure branch without drama	268
19.5 5. Component cards	269
19.5.1 Non-terrestrial network (NTN)	269
19.5.2 Terrestrial network	269
19.5.3 Service continuity	269
19.5.4 Coverage gap	269
19.5.5 Delay regime	269
19.5.6 Multi-path continuity	269
19.6 6. Stability contract	270
19.7 7. Try it	271
19.7.1 Continuity inheritance — LAB-PKT-001 and LAB-CE06-001	271
19.8 8. Build it	272
19.8.1 Explorer	272
19.8.2 Operator	272
19.8.3 Builder	272
19.8.4 Engineer	272
19.8.5 Researcher	272
19.9 9. Secure and include it	273
19.9.1 Security	273
19.9.2 Privacy	273
19.9.3 Accessibility	273
19.9.4 Equity	273
19.9.5 Safety	273
19.9.6 Ethics	273
19.1010. Career lens	273
19.1111. Check understanding	274

- 19.12References 274
- 19.1312. Glossary links 275
- 19.14Figure references (embedded; registered SVG + ally) 275
 - 19.14.1 FIG-CH19-001 — Ground / air / space path classes to a device 275
 - 19.14.2 FIG-CH19-002 — Continuity vs icon-lit 275
 - 19.14.3 FIG-CH19-003 — Delay regime comparison (illustrative) 276
- 19.15Claim footnotes used in this chapter 276

- 20 Chapter 20 — Latency, Reliability, QoE, and the Stability Contract 277**
- 20.1 1. The moment 277
- 20.2 2. What you notice 278
- 20.3 3. Exploded ecosystem 278
 - 20.3.1 Human and context 278
 - 20.3.2 Input and interaction path 278
 - 20.3.3 Application / code / event loop 279
 - 20.3.4 Memory, storage, and local state 279
 - 20.3.5 Network path (if remote work is required) 279
 - 20.3.6 Remote service / authorization 279
 - 20.3.7 Render / display feedback 279
 - 20.3.8 Power / thermal policy 280
 - 20.3.9 Trust / permissions 280
 - 20.3.10 Accessibility / alternate path 280
 - 20.3.11 Total delay and variability 280
- 20.4 4. Follow the signal 280
 - 20.4.1 Metric families without collapse 281
 - 20.4.2 Evidence hierarchy (climb only as far as tools and ethics allow) 281
 - 20.4.3 Failure branch without drama 282
- 20.5 5. Component cards 282
 - 20.5.1 Latency 282
 - 20.5.2 Reliability 282
 - 20.5.3 Throughput 283
 - 20.5.4 Quality of Experience (QoE) 283
 - 20.5.5 Quality of Service (QoS) 283
 - 20.5.6 Ping / probe 283
 - 20.5.7 Stability Contract (teaching model) 284
- 20.6 6. Stability contract 284
 - 20.6.1 How to teach and apply it (eight moves) 284
 - 20.6.2 Concurrent conditions (qualitative anchor) 284
 - 20.6.3 CE-6 + LAB-CE06-001 linkage 285
- 20.7 7. Try it 285
 - 20.7.1 LAB-CE06-001 — Explain, Measure, Improve, and Teach 285
- 20.8 8. Build it 286
 - 20.8.1 Explorer 286
 - 20.8.2 Operator 287
 - 20.8.3 Builder 287
 - 20.8.4 Engineer 287
 - 20.8.5 Researcher 287
- 20.9 9. Secure and include it 287
 - 20.9.1 Security 287
 - 20.9.2 Privacy 287
 - 20.9.3 Accessibility 287

20.9.4	Equity	288
20.9.5	Safety	288
20.9.6	Ethics	288
20.10	10. Career lens	288
20.11	11. Check understanding	289
20.12	References	289
20.13	12. Glossary links	289
20.14	Figure references (embedded; registered SVG + ally)	290
20.14.1	FIG-CH20-001 — Stability Contract concurrent conditions	290
20.14.2	FIG-CH20-002 — Latency vs reliability vs throughput	290
20.14.3	FIG-CH20-003 — QoE vs QoS vs ping	291
20.14.4	FIG-CH20-004 — Learner portfolio timings labeled n=1	291
20.15	Claim footnotes used in this chapter (project-specific)	291
21	Chapter 21 — Data, Machine Learning, and Generative AI	293
21.1	1. The moment	293
21.2	2. What you notice	294
21.3	3. Exploded ecosystem	294
21.3.1	Human and context	294
21.3.2	Input and interaction path	294
21.3.3	Application / prompt assembly	295
21.3.4	Model parameters and runtime	295
21.3.5	Local vs remote placement	295
21.3.6	Network and service (when remote)	295
21.3.7	Logging, retention, and side effects	295
21.3.8	Render / announce	295
21.3.9	Society and equity	295
21.4	4. Follow the signal	295
21.4.1	Generative systems without mythology	296
21.4.2	Local versus cloud as a path fork	296
21.4.3	Uncertainty families without collapse	296
21.5	5. Component cards	297
21.5.1	Training and input data	297
21.5.2	Model parameters	297
21.5.3	Inference runtime / output path	297
21.5.4	Retrieval / tool adapters (optional)	297
21.5.5	Logging and disclosure store	298
21.5.6	Identity and authorization adjacency	298
21.6	6. Stability contract	298
21.7	7. Try it	299
21.7.1	Procedure spine (Explorer baseline)	299
21.7.2	Pathway extensions	300
21.8	8. Build it	300
21.8.1	Explorer	300
21.8.2	Operator	300
21.8.3	Builder	300
21.8.4	Engineer	300
21.8.5	Researcher	300
21.9	9. Secure and include it	301
21.9.1	Security (UX-linked, not a scare-list)	301
21.9.2	Privacy	301

21.9.3	Accessibility	301
21.9.4	Equity	301
21.9.5	Safety	301
21.9.6	Ethics and responsible use	301
21.10	10. Career lens	302
21.11	11. Check understanding	302
21.12	References	303
21.13	12. Glossary links	303
21.14	Figure references (planned embeds; accessibility metadata)	304
21.14.1	FIG-CH21-001 — Data → model → inference → output	304
21.14.2	FIG-CH21-002 — Local vs cloud AI boundaries	304
21.14.3	FIG-CH21-003 — Fluency vs correctness vs evaluation evidence	304
21.15	Claim footnotes used in this chapter (project-specific)	305
22	Chapter 22 — Edge AI, Sensors, and Embodied Interaction	307
22.1	1. The moment	307
22.2	2. What you notice	308
22.3	3. Exploded ecosystem	308
22.3.1	Human and context	308
22.3.2	Sensors and local capture path	308
22.3.3	Features and on-device / edge inference	309
22.3.4	Actuators, haptics, and UI feedback	309
22.3.5	Budgets and thermal/power managers	309
22.3.6	Policy, identity, and privacy boundary	309
22.3.7	Evidence boundary (Device Quartet)	309
22.4	4. Follow the signal	309
22.4.1	Alternate paths (the honesty rule)	310
22.4.2	Failure branch without drama	310
22.5	5. Component cards	310
22.5.1	Embodied sensing	310
22.5.2	Edge / on-device inference	310
22.5.3	Edge resource budget	310
22.5.4	Embodied interaction loop	310
22.5.5	Sensing privacy boundary	311
22.5.6	Edge / cloud fallback	311
22.6	6. Stability contract	311
22.7	7. Try it	311
22.7.1	Local-path adjacency — LAB-TRUST-001	311
22.8	8. Build it	313
22.8.1	Explorer	313
22.8.2	Operator	313
22.8.3	Builder	313
22.8.4	Engineer	313
22.8.5	Researcher	313
22.9	9. Secure and include it	313
22.9.1	Security	313
22.9.2	Privacy	314
22.9.3	Accessibility	314
22.9.4	Equity	314
22.9.5	Safety	314
22.9.6	Ethics	314

22.1010. Career lens	314
22.1111. Check understanding	315
22.12References	315
22.1312. Glossary links	316
22.14Figure references (planned embeds; draft-blocked until SVG + ally land) . . .	316
22.14.1 FIG-CH22-001 — Sense → feature → edge inference → haptic/UI feedback	316
22.14.2 FIG-CH22-002 — Edge resource budget bands (illustrative)	316
22.14.3 FIG-CH22-003 — Edge IO Wearables learning context	317
22.15Claim footnotes used in this chapter	317
23 Chapter 23 — Cybersecurity from Chip to Cloud	319
23.1 1. The moment	319
23.2 2. What you notice	320
23.3 3. Exploded ecosystem	320
23.3.1 Human	321
23.3.2 Chip / boot (conceptual)	321
23.3.3 Firmware / OS	321
23.3.4 App / session	321
23.3.5 Network	321
23.3.6 Cloud / identity / policy	322
23.3.7 Society / recovery equity	322
23.4 4. Follow the signal	322
23.5 5. Component cards	323
23.5.1 Identity	323
23.5.2 Authentication (AuthN)	323
23.5.3 Authorization (AuthZ)	323
23.5.4 Session / recovery	324
23.5.5 Encryption boundary	324
23.5.6 Chip-to-cloud trust path	324
23.6 6. Stability contract	325
23.7 7. Try it	325
23.7.1 LAB-TRUST-001 — Authn/authz literacy + trust restore adjacency (link)	325
23.8 8. Build it	327
23.8.1 Explorer	327
23.8.2 Operator	327
23.8.3 Builder	327
23.8.4 Engineer	327
23.8.5 Researcher	327
23.8.6 Educator	328
23.9 9. Secure and include it	328
23.9.1 Security	328
23.9.2 Privacy	328
23.9.3 Accessibility	328
23.9.4 Equity	328
23.1010. Career lens	328
23.1111. Check understanding	329
23.11.1 Misconception probes	329
23.11.2 Teach-back (say it out loud)	330
23.11.3 Pathway self-check	330
23.12References	330
23.1312. Glossary links	330

23.14	Figure references (embedded above; accessibility metadata)	331
23.15	Claim footnotes used in this chapter	331
24	Chapter 24 — Privacy, Identity, Safety, Accessibility, and Ethics	333
24.1	1. The moment	333
24.2	2. What you notice	334
24.3	3. Exploded ecosystem	334
24.3.1	Human and context	335
24.3.2	Interaction and alternate path	335
24.3.3	Application / policy / consent UI	335
24.3.4	Identity provider and session state	335
24.3.5	Data stores and logs	335
24.3.6	Inference / remote services (when AI is in play)	335
24.3.7	Network and crypto boundary	335
24.3.8	Organizational / societal layer	335
24.4	4. Follow the signal	335
24.4.1	Lifecycle stages without collapse	336
24.4.2	Evidence hierarchy (climb only as far as tools and ethics allow)	336
24.4.3	Failure branch without drama	336
24.5	5. Component cards	337
24.5.1	Privacy / data lifecycle	337
24.5.2	Identity (human systems)	337
24.5.3	Authentication vs authorization	337
24.5.4	Safety constraints	337
24.5.5	Accessibility path	338
24.5.6	Ethics ladder	338
24.5.7	Consent / trust card	338
24.6	6. Stability contract	338
24.6.1	Concurrent conditions (qualitative anchor)	338
24.6.2	CE-5 + LAB-TRUST-001 linkage	339
24.7	7. Try it	339
24.7.1	LAB-TRUST-001 — Compare local vs remote AI paths and write a consent/trust card	339
24.8	8. Build it	340
24.8.1	Explorer	340
24.8.2	Operator	341
24.8.3	Builder	341
24.8.4	Engineer	341
24.8.5	Researcher	341
24.9	9. Secure and include it	341
24.9.1	Security (UX-linked)	341
24.9.2	Privacy	341
24.9.3	Identity	341
24.9.4	Accessibility	342
24.9.5	Equity	342
24.9.6	Safety	342
24.9.7	Ethics	342
24.10	10. Career lens	342
24.11	11. Check understanding	343
24.12	References	343
24.13	12. Glossary links	343

24.14	Figure references (planned embeds; draft-blocked until SVG + ally land)	344
24.14.1	FIG-CH24-001 — Privacy data lifecycle wheel with owners	344
24.14.2	FIG-CH24-002 — Accessible vs blocked recovery/auth paths	345
24.14.3	FIG-CH24-003 — Ethics ladder (observation → inference → disclosure)	345
25	Chapter 25 — Digital Equity: Who Benefits, Who Is Excluded, and What We Can Measure	347
25.1	1. The moment	347
25.2	2. What you notice	348
25.3	3. Exploded ecosystem	348
25.3.1	Human and context	348
25.3.2	Device and form factor	349
25.3.3	Cost and ownership	349
25.3.4	Network path	349
25.3.5	Application / service assumptions	349
25.3.6	Accessibility / alternate path	349
25.3.7	Evidence and governance	349
25.4	4. Follow the signal	349
25.4.1	Evidence labels for equity claims	350
25.4.2	Failure branch without humiliation	350
25.5	5. Component cards	350
25.5.1	Digital equity	350
25.5.2	Exclusion mechanism	351
25.5.3	Measurable inclusion signals	351
25.5.4	Constrained completion route	351
25.5.5	Honest evidence labeling	351
25.5.6	Accessibility path (contract condition)	351
25.6	6. Stability contract	352
25.6.1	Equity-specific contract conditions (chapter focus)	352
25.6.2	CE-6 + LAB-CE06-001 linkage	352
25.7	7. Try it	353
25.7.1	LAB-CE06-001 — Equity fields inside EMIT (inherit)	353
25.8	8. Build it	354
25.8.1	Explorer	354
25.8.2	Operator	354
25.8.3	Builder	354
25.8.4	Engineer	354
25.8.5	Researcher	354
25.9	9. Secure and include it	355
25.9.1	Security	355
25.9.2	Privacy	355
25.9.3	Accessibility	355
25.9.4	Equity (primary depth)	355
25.9.5	Safety	355
25.9.6	Ethics	355
25.10	10. Career lens	356
25.11	11. Check understanding	356
25.12	References	357
25.13	12. Glossary links	357
25.14	Figure references (planned embeds; draft-blocked until SVG + ally land)	358
25.14.1	FIG-CH25-001 — Same task; divergent completion	358

25.14.2	FIG-CH25-002 — Evidence labels for equity claims	358
25.14.3	FIG-CH25-003 — Exclusion mechanism cards	358
25.15	Claim footnotes used in this chapter (project-specific)	358
26	Chapter 26 — Software Development and Version Control	361
26.1	1. The moment	361
26.2	2. What you notice	362
26.3	3. Exploded ecosystem	362
26.3.1	Human and intent	362
26.3.2	Working tree	362
26.3.3	Staging / index (when used)	362
26.3.4	Committed history	363
26.3.5	Message and identity metadata	363
26.3.6	Branch / integration path	363
26.3.7	Review surface	363
26.3.8	Build-test loop	363
26.3.9	Secrets boundary	363
26.3.10	Project evidence neighbors (adjacency only)	363
26.4	4. Follow the signal	363
26.4.1	Alternate paths stay labeled	364
26.4.2	Version numbers when APIs are public	364
26.5	5. Component cards	364
26.5.1	Version control	364
26.5.2	Commit	364
26.5.3	Diff	365
26.5.4	Branch / integration	365
26.5.5	Code review	365
26.5.6	Secrets hygiene	365
26.5.7	Build-test loop	365
26.6	6. Stability contract	366
26.7	7. Try it	366
26.7.1	Explorer baseline (about 40–50 minutes)	366
26.7.2	Operator extension	367
26.7.3	Builder extension	367
26.7.4	Engineer extension	367
26.7.5	Researcher extension	367
26.7.6	Safety / privacy / accessibility	367
26.8	8. Build it	367
26.9	9. Secure and include it	368
26.9.1	Secrets	368
26.9.2	Inclusive tooling	368
26.9.3	Collaboration ethics	368
26.9.4	Device and hardware honesty	368
26.9.5	Evidence ethics	368
26.10	10. Career lens	368
26.11	11. Check understanding	369
26.12	References	369
26.13	12. Glossary links	369
26.14	Figure references (planned embeds; draft-blocked until SVG + ally land) . . .	370
26.14.1	FIG-CH26-001 — Edit → status → diff → commit → review	370
26.14.2	FIG-CH26-002 — Working tree vs committed history	370

26.14.3 FIG-CH26-003 — Secrets must not enter the repository 371

26.15 Claim footnotes used in this chapter (project-specific) 371

27 Chapter 27 — Testing, Observability, and Evidence 373

27.1 1. The moment 373

27.2 2. What you notice 374

27.3 3. Exploded ecosystem 374

27.3.1 Human experience and claim 375

27.3.2 Deliberate cases (tests) 375

27.3.3 Oracles 375

27.3.4 Runtime signals 375

27.3.5 Aggregation and views 375

27.3.6 Simulation / fixture / replay 375

27.3.7 Inference and narrative 375

27.3.8 Redaction and sharing 375

27.4 4. Follow the signal 375

27.4.1 Signal families without collapse 376

27.4.2 Evidence hierarchy (climb only as far as tools and ethics allow) 376

27.4.3 Adjacent practice (not invented CH27 WAIKE modules) 376

27.5 5. Component cards 377

27.5.1 Testing 377

27.5.2 Test oracle 377

27.5.3 Observability 377

27.5.4 Evidence hierarchy 377

27.5.5 Observation vs inference 378

27.5.6 Evidence redaction 378

27.6 6. Stability contract 378

27.6.1 How to teach and apply it (eight moves) 379

27.6.2 LAB-CE06-001 linkage 379

27.7 7. Try it 379

27.7.1 LAB-CE06-001 — evidence practice (inherit) 379

27.7.2 Proposed worksheet (not a WAIKE ID) 380

27.8 8. Build it 380

27.8.1 Explorer 380

27.8.2 Operator 380

27.8.3 Builder 380

27.8.4 Engineer 380

27.8.5 Researcher 381

27.9 9. Secure and include it 381

27.9.1 Security 381

27.9.2 Privacy 381

27.9.3 Accessibility 381

27.9.4 Equity 381

27.9.5 Safety 381

27.9.6 Ethics 381

27.10 10. Career lens 381

27.11 11. Check understanding 382

27.12 References 383

27.13 12. Glossary links 383

27.14 Figure references (planned embeds; **draft-blocked** until SVG + ally land) . . . 384

27.14.1 FIG-CH27-001 — Evidence hierarchy ladder 384

27.14.2 FIG-CH27-002 — Test pass vs usable experience 384

27.14.3 FIG-CH27-003 — Signal → observation → inference gate 384

27.15 Claim footnotes used in this chapter (project-specific) 384

28 Chapter 28 — Digital Twins, Simulation, and Reproducible Research 385

28.1 1. The moment 385

28.2 2. What you notice 386

28.3 3. Exploded ecosystem 386

28.3.1 Human and decision context 387

28.3.2 Question and claim 387

28.3.3 Model / simulation 387

28.3.4 Twin pairing and update path (when claimed) 387

28.3.5 Inputs, fixtures, and seeds 387

28.3.6 Compute and runtime environment 387

28.3.7 Observation and instrumentation 387

28.3.8 Peer regeneration path 387

28.3.9 Society / equity / access 388

28.4 4. Follow the signal 388

28.4.1 Stand-in families without collapse 388

28.4.2 Evidence hierarchy (climb only as far as tools and ethics allow) 389

28.5 5. Component cards 389

28.5.1 Simulation 389

28.5.2 Digital twin (teaching sense) 389

28.5.3 Validity bounds 389

28.5.4 Fixture 390

28.5.5 Reproducible research (teaching sense) 390

28.5.6 Reproducibility card 390

28.5.7 Observation vs inference 390

28.6 6. Stability contract 391

28.6.1 Concurrent conditions (qualitative; no invented twin fidelity) 391

28.6.2 Failure domains 391

28.6.3 Measurements we can seek (when available) 391

28.6.4 Measurements we cannot invent 392

28.7 7. Try it 392

28.7.1 Baseline (all pathways) 392

28.7.2 Pathway notes 392

28.8 8. Build it 393

28.8.1 Minimum build 393

28.8.2 Stretch (optional) 393

28.8.3 What not to build 393

28.9 9. Secure and include it 394

28.9.1 Secure 394

28.9.2 Include 394

28.9.3 Failure symptoms to watch 394

28.10 10. Career lens 394

28.11 11. Check understanding 395

28.11.1 Misconceptions to retire 395

28.11.2 Quick checks 395

28.12 References 396

28.13 12. Glossary links 396

28.14 Figure references (planned embeds; **draft-blocked** until SVG + ally land) . . . 397

28.14.1 FIG-CH28-001 — Simulation/twin vs measured world 397

28.14.2 FIG-CH28-002 — Reproducibility card fields 397

28.14.3 FIG-CH28-003 — Validity bounds annulus 397

28.15 Claim footnotes used in this chapter (project-specific) 397

29 Chapter 29 — Designing a Complete Technology Product 399

29.1 1. The moment 399

29.2 2. What you notice 400

29.3 3. Exploded ecosystem 400

29.3.1 Human experience and context 400

29.3.2 Interaction and feedback path 401

29.3.3 Application / code / local state 401

29.3.4 Network and placement (if remote work is required) 401

29.3.5 Identity, authorization, and privacy lifecycle 401

29.3.6 Evidence and observability 401

29.3.7 Society and responsibility 401

29.4 4. Follow the signal 401

29.4.1 Product assembly without marketing collapse 402

29.4.2 Failure branch without drama 402

29.5 5. Component cards 402

29.5.1 Complete technology product 403

29.5.2 Design gates 403

29.5.3 Evidence plan 403

29.5.4 Explicit tradeoffs 403

29.5.5 Claim boundary 403

29.6 6. Stability contract 404

29.6.1 How to teach and apply it (eight moves) 404

29.6.2 Concurrent conditions (qualitative product anchor) 404

29.6.3 CE + lab linkage (link — do not duplicate depth) 405

29.7 7. Try it 405

29.7.1 `INLINE_ACTIVITY` — Product one-pager + evidence/inclusion gates 405

29.8 8. Build it 406

29.8.1 Explorer 406

29.8.2 Operator 407

29.8.3 Builder 407

29.8.4 Engineer 407

29.8.5 Researcher 407

29.9 9. Secure and include it 407

29.9.1 Security 407

29.9.2 Privacy 407

29.9.3 Accessibility 407

29.9.4 Equity 408

29.9.5 Safety 408

29.9.6 Ethics 408

29.10 10. Career lens 408

29.11 11. Check understanding 408

29.12 References 409

29.13 12. Glossary links 409

29.14 Figure references (planned embeds; accessibility metadata) 410

29.14.1 FIG-CH29-001 — Complete product layers under Stability Contract 410

29.14.2 FIG-CH29-002 — Design gates including secure/include 410

29.14.3 FIG-CH29-003 — One-pager evidence fields (claim-boundary support) . . 411

29.15 Claim footnotes used in this chapter (project-specific) 411

30 Chapter 30 — Career Maps and Portfolio Proof 413

30.1 1. The moment 413

30.2 2. What you notice 414

30.3 3. Exploded ecosystem 414

30.3.1 Human interest and context 414

30.3.2 Role family (not job title promise) 414

30.3.3 Skill and chapter work 415

30.3.4 Artifact / portfolio proof 415

30.3.5 Claim boundary 415

30.3.6 Review criteria 415

30.3.7 Index and front door 415

30.3.8 Sharing and accessibility surface 415

30.3.9 Employment non-guarantee boundary 415

30.4 4. Follow the signal 415

30.4.1 Portfolio index fields (minimum) 416

30.4.2 Evidence honesty ladder (portfolio edition) 416

30.5 5. Component cards 417

30.5.1 Role family 417

30.5.2 Portfolio proof 417

30.5.3 Review criteria 417

30.5.4 Employment non-guarantee 417

30.5.5 Portfolio index 417

30.6 6. Stability contract 418

30.6.1 Portfolio Stability Contract (CH30) 418

30.7 7. Try it 418

30.7.1 Route A — Commodity assemble (preferred) 418

30.7.2 Route F — Fixture / no-specialized-hardware 419

30.7.3 Operator checklist 419

30.8 8. Build it 419

30.8.1 Small extension options (pick one) 419

30.8.2 Done means 419

30.9 9. Secure and include it 420

30.9.1 Privacy and safety 420

30.9.2 Accessibility 420

30.9.3 Equity 420

30.9.4 Ethics 420

30.10 10. Career lens 420

30.11 11. Check understanding 421

30.12 References 421

30.13 12. Glossary links 422

30.14 Figure references (planned embeds; accessibility metadata) 422

30.14.1 FIG-CH30-001 — Role family → artifact → review criteria 422

30.14.2 FIG-CH30-002 — Portfolio index fields + claim boundary 423

30.14.3 FIG-CH30-003 — Learning proof employment guarantee 423

31 Chapter 31 — Capstone: Explain, Measure, Improve, and Teach the Ecosystem 425

31.1 1. The moment 425

31.2	2. What you notice	426
31.3	3. Exploded ecosystem	426
31.3.1	Human and context	426
31.3.2	Input and interaction path	427
31.3.3	Application / code / event loop	427
31.3.4	Memory, storage, and local state	427
31.3.5	Network path (if remote work is required)	427
31.3.6	Remote service / authorization	427
31.3.7	Render / display feedback	427
31.3.8	Power / thermal, trust / permissions, accessibility path	427
31.3.9	Society / equity boundary	427
31.4	4. Follow the signal	427
31.5	5. Component cards	428
31.5.1	Failure-domain cards	428
31.5.2	Capstone portfolio field cards (required set)	429
31.6	6. Stability contract	429
31.6.1	Concurrent conditions (teaching list)	430
31.6.2	CE-6 + LAB-CE06-001 linkage	430
31.6.3	Evidence firewall (non-negotiable)	430
31.7	7. Try it	430
31.7.1	LAB-CE06-001 — Explain, Measure, Improve, and Teach	430
31.8	8. Build it	431
31.8.1	Explorer	432
31.8.2	Operator	432
31.8.3	Builder	432
31.8.4	Engineer	432
31.8.5	Researcher	432
31.9	9. Secure and include it	432
31.9.1	Security	432
31.9.2	Privacy	432
31.9.3	Accessibility	433
31.9.4	Equity	433
31.9.5	AI / uncertainty (CE-5 synthesis)	433
31.9.6	Ethics	433
31.10	10. Career lens	433
31.11	11. Check understanding	434
31.12	12. References	434
31.13	13. Glossary links	435
31.14	14. Figure references (planned embeds; accessibility metadata)	435
31.14.1	FIG-CH31-001 — Stability Contract concurrent conditions	435
31.14.2	FIG-CH31-002 — EMIT cycle with evidence gates	436
31.14.3	FIG-CH31-003 — Fixture / illustrative vs human evidence wall	436
31.15	15. Claim footnotes used in this chapter (project-specific)	436
32		437
Parts and chapters		439
32.0.1	Part I — Read the experience	439
32.0.2	Part II — Open the device	439
32.0.3	Part III — Make hardware useful	440
32.0.4	Part IV — Connect everything	440

32.0.5 Part V — Intelligence, security, and responsibility 440

32.0.6 Part VI — Build, prove, and contribute 440

32.1 Subject index (editorial) 441

Glossary **443**

References **447**

Key standards **449**

Lab index **451**

Figure index **453**

Career and role map **459**

Errata **461**

32.2 How to report 461

32.3 Maintainer workflow 461

Acknowledgments **463**

Chapter 1

The Technology Landscape

! WORKING FULL-MANUSCRIPT DRAFT

Human reader validation pending.
Technical/editorial revision pending.
Not publication-ready.

Welcome to the full-book working manuscript preview.

Teaching model: **Human experience** → **system** → **component** → **code** → **network** → **society**

Gate posture remains `GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING`. Historical `CH02-REVIEW-R1` is chapter-prototype evidence only; full-book human validation is deferred until all 31 working drafts exist.

1.1 Start here

Need	Page
Know first	Know first
Status / rights	Manuscript status · Rights notice
How to use / pathways	How to use · Pathways
Where am I?	Parts and chapters
Glossary / labs / figures	Glossary · Labs · Figures
References / standards / roles / errata	References · Key standards · Roles · Errata

Chapter 2

Manuscript status

2.1 WORKING FULL-MANUSCRIPT DRAFT

Human reader validation pending.

Technical/editorial revision pending.

Not publication-ready.

This build is a **working full-manuscript draft** of *The Technology Landscape*. It is intended for authoring, internal technical review preparation, and toolchain verification.

- Gate posture remains: `GATE_3_IN_PROGRESS` - `READER_EVIDENCE_PENDING`.
- Historical chapter-prototype review CH02-REVIEW-R1 is **not** full-manuscript validation.
- Full-book human validation is **deferred until** the entire 31-chapter working draft exists (see `publication/full131/VALIDATION_SEQUENCE_DECISION.md`).
- Successful HTML/PDF/EPUB rendering does **not** mean publication-ready.
- Rights: All Rights Reserved © 2026 Edmund Gunn Jr. for manuscript/artwork/instructor materials; MIT only where explicitly scoped for code/tooling; upstream retains upstream. See Rights notice.

Rights notice

Domain	Right	Notes
Manuscript prose	All Rights Reserved © 2026 Edmund Gunn Jr.	Development-stage; not Creative Commons
Original artwork / figures	All Rights Reserved © 2026 Edmund Gunn Jr.	Educational diagrams remain rights-reserved
Instructor-only assets	All Rights Reserved © 2026 Edmund Gunn Jr.	Closed unless later opened by explicit decision
Publication scripts / CI / Makefile	MIT (software)	Infrastructure only
Runnable lab / code samples	MIT where files explicitly identify as code samples	Does not license surrounding prose
Upstream WAIKE / gunnchOS / third-party works	Upstream retains upstream	Cite SHAs; fair citation only

Machine-readable decision records: `publication/BOOK_LICENSE_DECISION.md`, `publication/RIGHTS_A`

No blanket Creative Commons license for this working draft. Successful HTML/PDF/EPUB rendering does not change rights or publication readiness.

Know first

Before deep reading, lock these navigation truths:

Need	Go here
Where am I in the book?	Parts and chapters
What is this draft’s status?	Manuscript status
How should I read?	How to use this book · Reader pathways
What do evidence labels mean?	Evidence and status legend
Device Quartet / WAIKE boundaries	Device Quartet · WAIKE
Glossary / labs / figures / roles / standards	Back-matter indexes below
Report an error	Errata

2.2 Non-claims (keep visible)

- **Not publication-ready.** Gate posture: `GATE_3_IN_PROGRESS` - `READER_EVIDENCE_PENDING`.
- Historical `CH02-REVIEW-R1` is chapter-prototype evidence only — not full-manuscript validation.
- Acknowledgments remain a **placeholder**; no invented reviewers, endorsements, or testimonials.
- Rights: manuscript/artwork/instructor materials are **All Rights Reserved © 2026 Edmund Gunn Jr.**; code/tooling is **MIT only where explicitly scoped**; upstream keeps upstream. **No blanket CC.**

How to use this book

This publication teaches technology as a lived system: **human experience** → **system** → **component** → **code** → **network** → **society**.

2.3 Navigation hub

Question	Destination
Where am I?	Parts and chapters
What should I know first?	Know first
Which pathway fits me?	Reader pathways
Glossary	Glossary
Labs	Lab index
Figures	Figure index
References / standards	References · Key standards
Career / role map	Career and role map
Errata	Errata

2.4 Suggested first pass

1. Read Know first and Manuscript status so expectations stay honest.
2. Skim Reader pathways and pick the lane that matches your starting point.
3. Read Chapter 2 (*Follow One Tap Through the Entire Stack*) as the method prototype.
4. Use Device Quartet and WAIKE as orientation — not as purchase or credential claims.
5. Follow labs only on devices and accounts you already control.

2.5 What “working draft” means here

Prose, figures, and labs may still move. Claims that require physical Device Quartet measurements remain `PHYSICAL_PENDING`. Do not treat scaffolds or unfinished chapters as finished teaching units.

Reader pathways

Six pathways share the same chapter anatomy; they differ in starting evidence and depth.

Pathway	Starting point	Evidence expectation
Explorer	little/no technical background	experience map, major components, ordinary-language path, teach-back
Operator	regular device/software user	guided test, status/logs/measurement, likely failure domain
Builder	student/maker/career changer	modify code/config, construct small system, document tradeoffs
Engineer	undergraduate/practitioner	measure, diagnose, architecture reasoning, alternatives
Researcher	graduate/investigator	hypothesis, variables, experiment, uncertainty, limitations
Educator	teacher/mentor/family guide	facilitation, lab adaptation, misconception detection

You may change pathways between chapters. The book does not rank pathways by prestige.

Device Quartet

2.6 Device Quartet (learning laboratory)

The Device Quartet is a set of **research form factors** used as a recurring learning laboratory across the book:

ID	Name	Learning role
student-14-5	Student 14.5-inch	Sustained desk compute for learning and work sessions
handheld-hybrid	Handheld Hybrid	Mobile and docked compute
ds-xl-coder	DS-XL Coder	Strongest learn-to-build device lens
edge-io-wearables	Edge IO Wearables	Embodied compute and safety-critical interaction

2.7 Integrity rules

- Form factors are **learning benchmarks**, not commercial product promises.
- Physical fabrication / EVT measurement claims remain **PHYSICAL_PENDING** until dated evidence exists.
- Chapters must not pressure readers to purchase hardware.
- Canonical machine-readable definitions live in `devices/quartet.yaml`.

WAIKE

2.8 WAIKE explanation

WAIKE is the learning-system alignment layer used with this publication: courses, assessments, and portfolio-style evidence maps that connect chapter outcomes to practice.

2.9 What WAIKE is

- A crosswalk between book concepts and existing learning objects where adjacency is honest.
- A place to record **exact**, **adjacent**, **proposed**, or **no_map** relationships without inventing course IDs.

2.10 What WAIKE is not

- Not a claim that every chapter already has a finished assessed module.
- Not employment credentialing or a guaranteed certification.
- Not Gate 3 reader evidence.

Working maps live under `waike/` and per-chapter `WAIKE_CROSSWALK.md` packets in `publication/full131/chapters/`.

Evidence and status legend

Label	Meaning
WORKING FULL-MANUSCRIPT DRAFT	Full-book prose in progress; not publication-ready
SCAFFOLD / outline	Placeholder structure only — not teachable final prose
DRAFT_COMPLETE / draft	Working draft prose exists for that chapter
PHYSICAL_PENDING	Requires measured Device Quartet / hardware evidence not yet available
ILLUSTRATIVE_ONLY / illustrative / conceptual	Teaching illustration; not field measurement
SOURCE_NEEDED	Claim still needs a citable source
IMPLEMENTED_DIGITAL / DRAFT_DIGITAL / FIXTURE_VALIDATED	Lab/toolchain maturity labels — not human reader validation
GATE_3_IN_PROGRESS -	Gate 3 open; CH02 prototype review
READER_EVIDENCE_PENDING	historical; full-book human validation deferred
DEFERRED_UNTIL_FULL_MANUSCRIPT_DRAFT	Full-book reader recruitment waits for all 31 working drafts
PUBLICATION_READY	Not claimed for this working draft

Successful rendering of HTML/PDF/EPUB is **toolchain evidence**, not human validation and not a release decision. Synthetic fixtures must never be described as Gate 3 PASS.

Preface

This publication begins from human experience and works backward through the systems that make everyday technology possible.

WORKING FULL-MANUSCRIPT DRAFT. Human reader validation pending. Technical/editorial revision pending. Not publication-ready.

Start with **Know first** and **How to use this book** if you need orientation; acknowledgments remain a placeholder until a publication-ready release.

Chapter 1

Chapter 1 — Technology Is a System, Not a Screen

Status: draft · Chapter ID: CH01

Author: Edmund Gunn, Jr.

1.1 1. The moment

You unlock a familiar phone, tablet, or laptop. You open something you already use—messages, a document, a map, a school portal. The chrome appears quickly: the icon, the title, the navigation bar, the skeleton layout that looks almost ready.

Then you wait.

The list is empty. The tiles stay gray. A spinner turns. Yesterday’s content sits there while a refresh fails to finish. From your seat it still feels like *one app*. Underneath, several cooperating parts may still be starting, restoring state, reading storage, talking to an optional remote service, or failing quietly while the surface looks polite.

This chapter’s promise is simple:

After this chapter, you can look at an ordinary device experience and name the hidden cooperating parts—not just the colorful surface—and explain why “the app” is usually not one thing.

Chapter 2 will later prove a method by following one tap through the stack. Chapter 1 teaches the **system lens** first: visible interface versus hidden work, local readiness versus network-dependent readiness, and the honesty of naming a **failure domain** before blaming a vague villain. When one domain limits the whole path, later chapters will call that limiting resource a **bottleneck**—here it is enough to name the domain before inventing a cause.

1.2 2. What you notice

Before jargon, notice the human contract you already expect when you open something familiar.

You expect the open action to be recognized—whether you used a finger, trackpad, keyboard, switch, or voice. You expect the shell of the experience to appear soon enough that the device feels awake. You expect usable content to arrive, or a failure you can understand. You expect the interface not to seize up. You expect “connected” status—when it matters—to relate to the service you need, not merely to a radio icon. You also expect battery and heat to stay within reason for something so ordinary.

Those expectations are not decorations around technology. From the person’s point of view, they *are* the product.

Chrome visible is not the same event as content usable.

The title bar can look finished while the body is still empty. A skeleton can look “alive” while a sync has already failed. Stale content can look complete while a refresh never returns. Your nervous system may stitch those timelines into one story—“the app opened”—or into another—“something feels wrong even though the screen is there.”

Optional comparison available on almost any device you already own: open a photo, note, or file that already lives on the device. Then open something that must fetch fresh remote content. The first often finishes inside the device. The second may add waiting that has nothing to do with how hard you pressed the glass.

Accessibility belongs in this noticing, not later as a disclaimer. If the only “busy” cue is a colored spinner, someone who does not see that color—or someone using a screen reader that never hears a ready announcement—experiences a different system than the designer imagined (W3C 2023). Inclusive readiness is part of the human contract.

1.3 3. Exploded ecosystem

An ordinary open is not a single object. It is a path through a **technology ecosystem**: people, device, software, optional network, and a usable (or failed) result. Figure 1.1 is the first-minute map. Figure 1.2 opens the everyday word *app* into cooperating cards. Both are **conceptual / Representative educational architecture**—not measured teardowns of a specific manufactured revision.

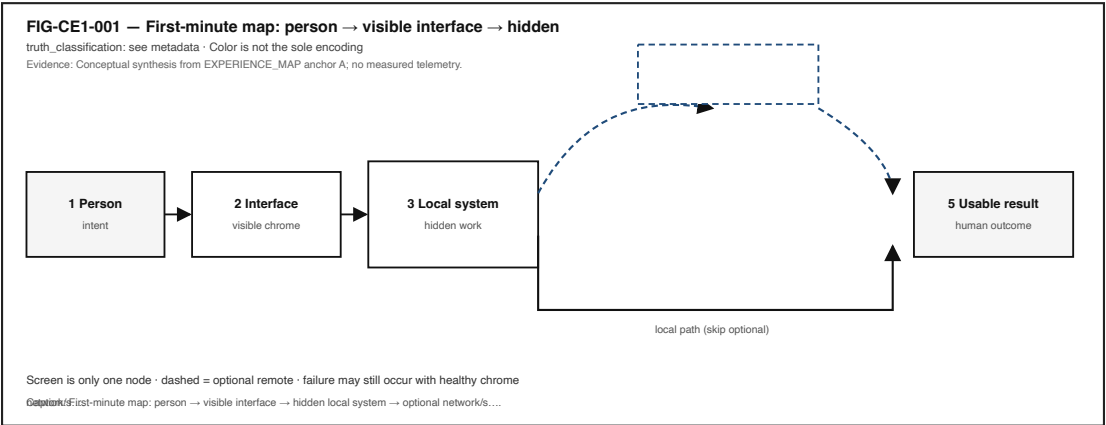


Figure 1.1: Person to visible interface to hidden local system to optional network/service to human-usable result.

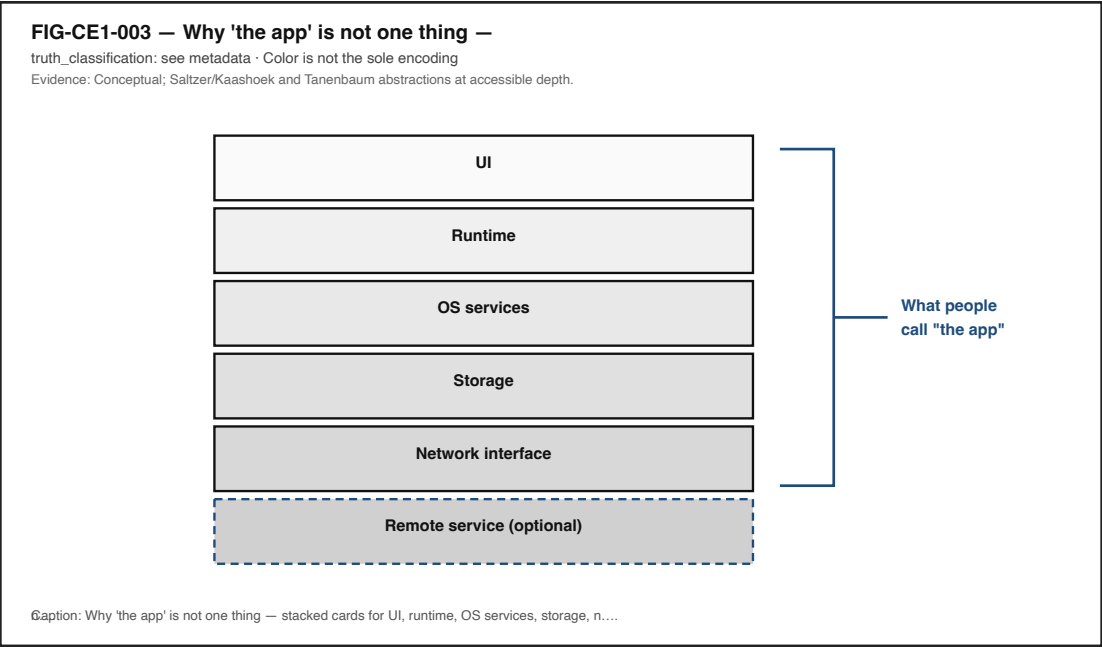


Figure 1.2: Stacked cards for UI, runtime, OS services, storage, network interface, and remote service grouped as everyday ‘the app’.

Walk the layers in ordinary language. Keep the same layers when vocabulary deepens later.

1.3.1 Human

You form intent: open this portal, resume this draft, check this map. Muscles, voice, or assistive controls issue the open. Later, eyes, ears, and hands judge whether the result matches what you meant.

1.3.2 Visible interface

What you can see or hear on the surface—chrome, skeleton, spinners, lists, error text, spoken status. The **visible interface** is a cooperating part, not the whole story (Saltzer and Kaashoek 2009).

1.3.3 Hidden local system

Application runtime, operating-system services, working memory, durable storage, power and thermal headroom, and the local path that can schedule work soon enough to show chrome and continue. Computer systems are designed as layered cooperating abstractions; treating the glass as the entire machine collapses those layers into one vague object (Saltzer and Kaashoek 2009; Tanenbaum and Bos 2022).

1.3.4 Optional network and remote service

When the experience needs fresh remote content, a network interface, access network, and remote service may enter the path. Packet transport commonly rests on IP and TCP foundations (Postel

1981; Eddy 2022). Many useful opens never take this branch. Teaching the word **optional** is part of the chapter’s honesty rule.

1.3.5 Human-usable result

Content you can act on—or a clear failure. The ecosystem succeeds only when the person can finish the job they came to do.

Figure 1.3 foreshadows device subsystems that support readiness (display, compute, memory/storage, radio, power) as **Representative educational architecture**. The Device Quartet—Student 14.5-inch, Handheld Hybrid, DS-XL Coder, and Edge IO Wearables—appears here only as a future shared learning-laboratory spine of research form factors. Physical fabrication remains pending (**PHYSICAL_PENDING**); Chapter 1 labs run on commodity devices you already own (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

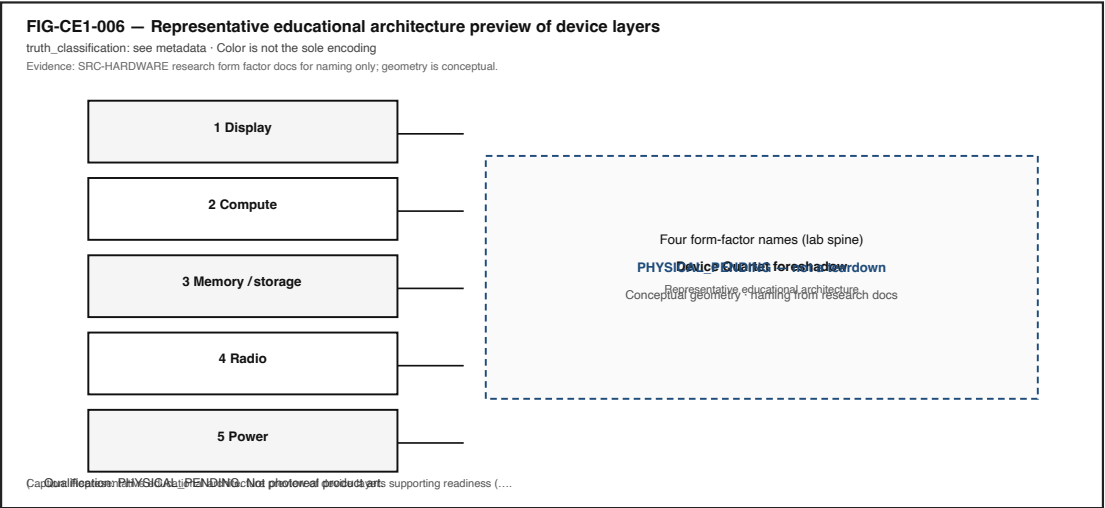


Figure 1.3: Representative educational device blocks with Device Quartet form-factor names as future lab spine.

Equity belongs in the map: always-online assumptions exclude learners on metered plans, shared devices, captive portals, or intermittent links. Low-cost devices remain first-class learning instruments, not lesser technology.

1.4 4. Follow the signal

Everyday interactive software commonly waits for inputs, processes them, updates remembered **state**, and presents outputs (WHATWG 2026b, 2026a). Figure 1.4 shows that teaching path for the chrome-before-content moment. Read it as a logical story, not as a claim that every platform uses one identical event-loop implementation or that steps never overlap.

A useful reading of the open moment:

1. **Input arrives** — unlock/open intent becomes an event the software can interpret.
2. **Local processing begins** — the application or runtime starts (or wakes) enough work to draw chrome.

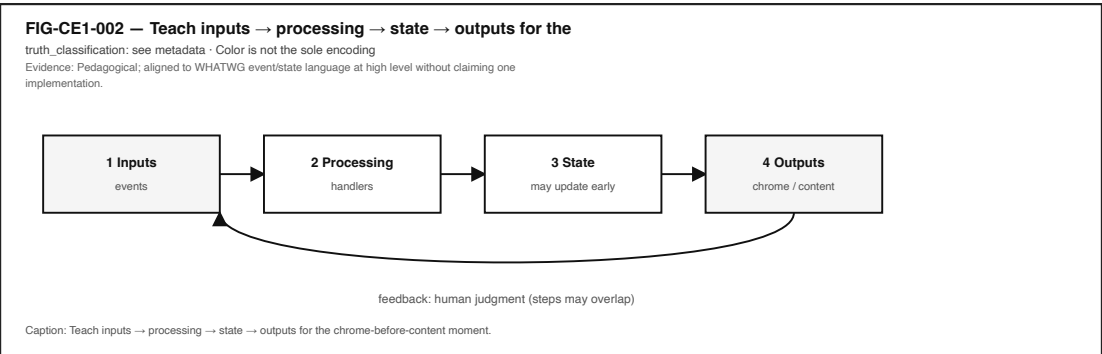


Figure 1.4: Inputs to processing to state to outputs with feedback to human judgment.

- 3. **State restore** — session, cache, or local files are consulted; validity matters.
- 4. **Optional remote branch** — if needed, a request may leave the device toward a service.
- 5. **Outputs update** — chrome first, then content—or a failure message.
- 6. **Human judgment** — ready enough to use, still busy, or failed.

Figure 1.5 forks after chrome appears into a **local-only** path and an **optional remote** path that rejoin at “content usable or failed.” Same starting human moment; different dependency sets. The fork is **illustrative** teaching geometry for observation—not a benchmark of any brand’s launch times.

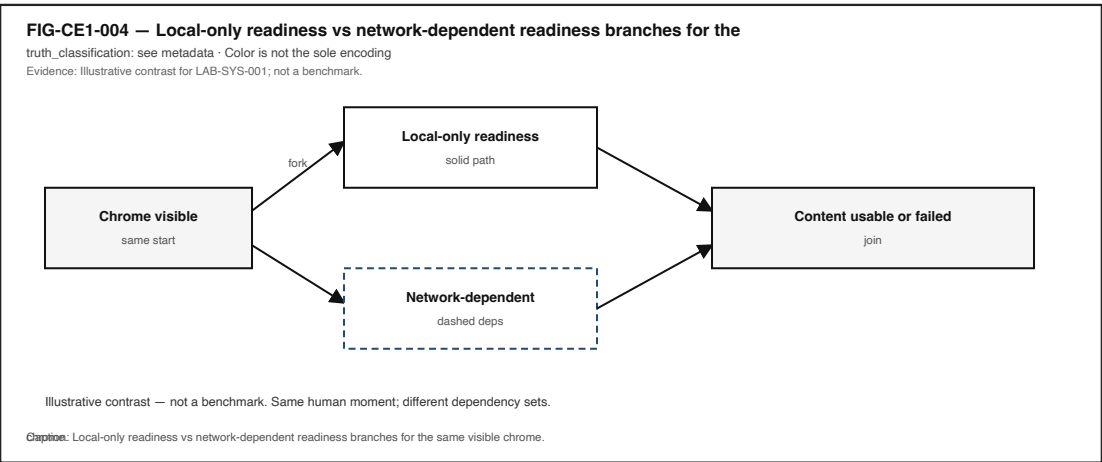


Figure 1.5: Fork after app chrome visible into local-only and optional remote readiness paths.

1.4.1 Alternate paths (the honesty rule)

- A notes app opening a local draft may finish without any remote call.
- A streaming video may show player chrome while buffers and licenses still travel.
- A portal may show a healthy connectivity icon while DNS, authentication, or the application API still fails.

Illustrative teaching point (not a measured universal product fact): a connectivity indicator is related to, but not identical with, application success. Bars (or a “connected” label) report something about a link or association. They do not certify that *this* service, for *this*

account, right now, is healthy.

RAM and durable storage play different jobs when an open feels slow: working memory holds what is being used now; storage holds what must survive restarts (Patterson and Hennessy 2020b). Naming the difference is enough here; later chapters deepen the hierarchy without inventing timings for this chapter.

1.5 5. Component cards

These cards are orientation tools—not a complete bill of materials. Use them to name parts when something feels wrong.

1.5.1 Visible interface / UI

What it is. The surface the person can see or hear: chrome, content, status text, spoken updates.

Why it matters. It can look finished while hidden work is incomplete.

Common misconception. “If I can see the screen, the system is ready.”

Failure hint. Forever spinner; empty skeleton; missing ready announcement.

1.5.2 Application / runtime

What it is. The local program and runtime that wait for events, update state, and ask for outputs (WHATWG 2026b, 2026a).

Why it matters. Chrome can render from one code path while content waits on another.

Common misconception. “The app” is a single indivisible object.

Failure hint. Crash after chrome; frozen shell; endless redirect that still shows a title.

1.5.3 OS services / scheduling

What it is. Operating-system abstractions that let software run, share resources, and talk to devices (Tanenbaum and Bos 2022). Scheduling decides when runnable work receives CPU time soon enough to continue (The Linux Kernel Documentation 2026b).

Why it matters. Extreme load can delay opens that still “look” like an app problem.

Common misconception. Only the colorful app process matters.

Failure hint. Everything on the device feels laggy, not only one title.

1.5.4 Storage

What it is. Durable holding for files, caches, and offline copies—distinct from RAM’s working role (Patterson and Hennessy 2020b).

Why it matters. Offline success and online failure often split here.

Common misconception. Slow open always means “bad Wi-Fi.”

Failure hint. Missing offline file; corrupt cache; local item works while remote item fails.

1.5.5 Network interface

What it is. The local radio or wired path and stack that can carry packets when the remote branch is taken (Postel 1981; Eddy 2022).

Why it matters. Association usable experience.

Common misconception. Connected icon proves the needed service.
Failure hint. Icon healthy; target tiles never fill.

1.5.6 Remote service

What it is. A provider of data or computation off-device—edge or cloud in later chapters’ vocabulary.
Why it matters. UI can survive while the service does not.
Common misconception. “My phone is broken” whenever a portal fails.
Failure hint. Other local apps work; one remote experience fails.

Figure 1.6 maps everyday symptoms to failure-domain buckets without claiming root cause. Use it as an operator triage aid: symptom → guessed domain → evidence still needed.

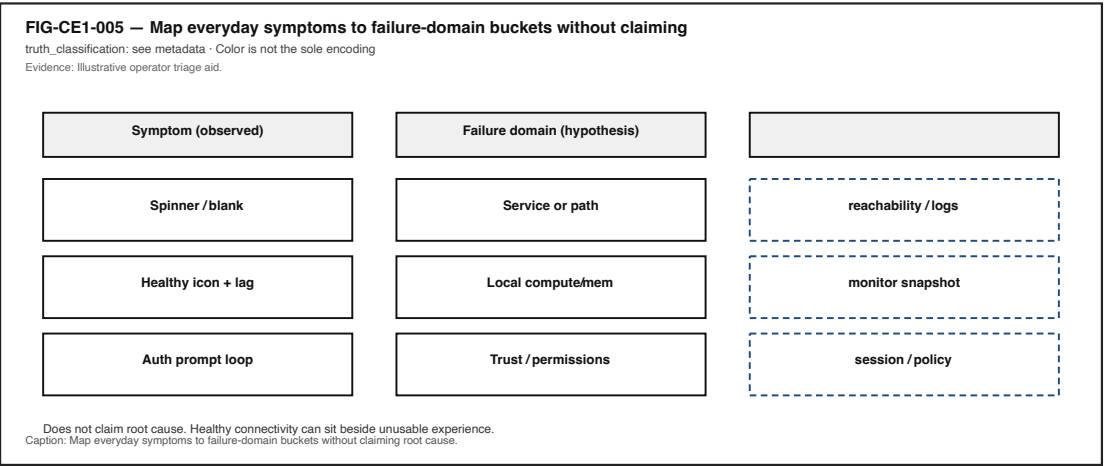


Figure 1.6: Symptom to failure-domain to evidence-still-needed columns.

1.6 6. Stability contract

The **Stability Contract** is a signature idea across this book:

A person experiences a usable open only while multiple hidden conditions remain within acceptable bounds.

Chapter 1 needs the preview, not invented numeric budgets. For the chrome-before-content anchor, conditions typically include:

- input path available (touch, keyboard, switch, voice),
- application/runtime schedulable soon enough to show chrome and continue,
- state/session usable—or failure communicated,
- storage readable when the path is local,
- optional network/service path healthy end-to-end *when* remote content is required (association DNS route auth API),
- output path able to present updates the person can perceive,
- power/thermal headroom sufficient that needed work is not silently deferred beyond human patience (qualitative only).

Figure 1.7 shows concurrent conditions feeding a single “human usable?” gate. Multiple conditions can look acceptable while one critical dependency still fails the experience.

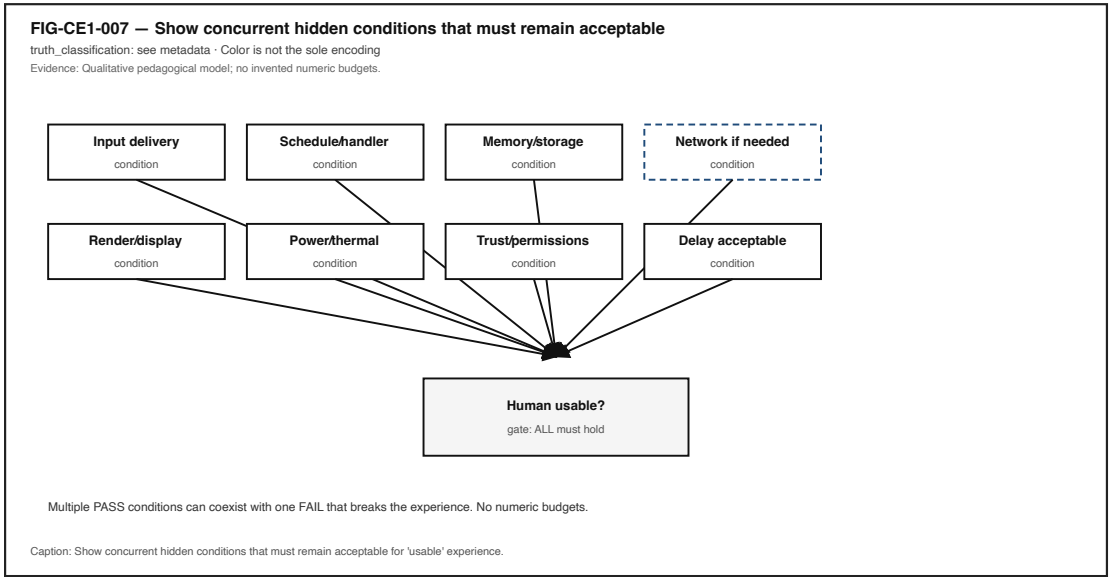


Figure 1.7: Parallel hidden conditions feeding a human-usable gate.

Three separations matter:

1. The interface can look **alive** after the usable result has already failed—or before it is ready.
2. A connectivity indicator can look healthy while the application service you need is unreachable (**illustrative** operator lesson, not a measured RF law).
3. Local dependencies are not optional for on-device opens; network dependencies are optional branches.

When the contract breaks, people lose time, trust, and sometimes access to school, work, or care tasks. Mis-naming the failure domain—“the phone is broken” versus “the service is unreachable”—leads to wrong fixes and unfair blame, especially on shared or low-bandwidth networks. Later chapters deepen the contract; here, refuse false certainty.

1.7 7. Try it

1.7.1 LAB-SYS-001 — Name the System Behind a Familiar “Open”

Observable question. When I open something I already use, what becomes visible first, what becomes usable later, and which hidden parts might still be working?

Why this lab (vs Chapter 2). LAB-TAP-001 times a tap-to-response path. LAB-SYS-001 teaches the system lens: readiness, layers, dependencies, and failure domains—without duplicating one-tap instrumentation.

WAIKE alignment note. WAIKE accepted **main** hosts file-backed curriculum packages that can support adjacent systems-thinking competencies (for example observation/inference and ticket triage). It does **not** currently ship a course module literally titled like this chapter

(gunnchOS3k 2026). **LAB-SYS-001** is therefore a **publication-owned** commodity lab with competency neighbors—not a renamed false WAIKE lab ID.

Prerequisites. A phone, tablet, or laptop you may use for learning; a modern browser; optional Python for a large-print observation sheet helper. No Device Quartet hardware. No root or jailbreak.

Safety and privacy. Do not capture passwords, tokens, private chats, ID photos, or classmate personal information. Prefer the supplied fixture or public demo pages in shared spaces. No packet capture of others’ traffic. Do not disable safety features required for your context. Scrub identifiers before saving portfolio evidence.

Time estimate. About 45 minutes for the Explorer baseline.

1.7.1.1 Prediction

Before you open anything, write one sentence: do you expect **chrome** (shell/nav/title) or **content** (usable body) to become ready first, and why?

1.7.1.2 Route A — Browser demo (baseline)

Open the lab’s browser readiness demo. Watch once without changing settings. Record wall-clock (or on-page) times for **chrome visible** and **content usable** (or failed). List at least three visible cues and at least three guessed hidden parts—mark guesses as inference.

1.7.1.3 Route B — Local observation sheet (baseline)

Run the lab’s local observation-sheet helper to print a large-print structured sheet. Fill it while opening a familiar experience, or while using the fixture. Keyboard, switch, and voice opens count.

1.7.1.4 Route C — Offline fixture (required fallback)

Open the lab’s offline readiness fixture. Use the simulate-offline control to contrast chrome-still-visible versus content failure. Or use the written scenario card labeled **fixture / illustrative** for Explorer/Educator work when a live open is unavailable.

1.7.1.5 Evidence (minimum)

- readiness observation table (chrome vs usable; online vs offline if attempted),
- labeled ecosystem map,
- observation versus inference paragraph,
- scrubbed notes.

1.7.1.6 Interpretation

Allowed as observation	Inference until more evidence
Chrome appeared at time T1	“Network is bad”
Content usable/failed at T2	“Storage is dying”
Airplane mode was on/off	“DNS failed”
Icon showed connected	“Server is down”

Learner wall-clock timings are **your** observations on **your** commodity device or fixture—not publication benchmarks and not touch-to-photon laboratory results.

1.7.1.7 Limits (say them out loud)

- One run is not a benchmark.
- Fixture delays are illustrative teaching aids, not proof of a specific commercial app’s architecture.
- Software clocks do not measure every physical stage from intention to photon.
- Missing kernel-level visibility is a limit, not a failure of curiosity.

1.7.1.8 Portfolio output

Produce a small packet with a short README (question, method, limits), observation table, ecosystem map, scrubbed evidence note, and a teach-back paragraph. Completion means a claim is supported by an artifact—not that “a page loaded.”

1.8 8. Build it

Use the same open story at the depth that matches your pathway. Your artifact is a **personal technology-ecosystem map** plus a readiness checklist tailored to one familiar experience.

1.8.1 Explorer

Draw or list two columns: visible cues versus guessed hidden parts (3 each). Write one short paragraph explaining why “the app” is usually not a single object—UI, local software, and at least one optional remote dependency—without drowning a nontechnical listener in jargon.

1.8.2 Operator

Repeat the open once online and once offline/airplane (if safe), or use the fixture’s offline toggle. Fill the observation table. Assign a failure-domain guess for any failure and mark it as inference. Note connectivity-icon state as a separate observation from usability.

1.8.3 Builder

Fill an ecosystem-map template with named layers and a dashed optional remote branch. Adapt the readiness checklist to your chosen experience. Document one tradeoff (example: richer notes versus avoiding personal content in a shared classroom).

1.8.4 Engineer

Write a diagnosis plan that separates observation, interpretation, and causal claim. State what additional evidence would be required before blaming network versus app versus storage. Optionally use browser Performance or Network panels only on a non-sensitive page; scrub secrets (MDN Web Docs 2026b). Map at least one Chapter 1 layer to a cited abstraction (process, event, packet, or storage hierarchy).

1.8.5 Researcher

State a testable hypothesis such as: “For experience X, offline mode increases content-usable delay or causes failure while chrome still appears.” Define a small number of runs, environment notes, and explicit limits. Document what evidence would be required to upgrade an illustrative claim to a measured claim. Do not publish student timings as universal launch laws.

Educators can facilitate the ten-to-fifteen-minute misconception probe in Section 11 and adapt **LAB-SYS-001** with the fixture/scenario fallback when networks or personal apps are unavailable.

1.9 9. Secure and include it

1.9.1 Security

Chapter 1 teaches structure, not offensive technique. Opens can involve sessions, permissions, and privileged data. Treat **identity/session** as a first-class failure domain: an expired login can look like a “broken app.” Capture only evidence you are allowed to keep. Prefer fixtures and public demos over production accounts with sensitive data.

1.9.2 Privacy

Screenshots and logs are evidence and risk. Do not save passwords, tokens, private message bodies, or classmates’ identifying details. Classroom mode should default to fixture content. Redact before sharing portfolio packets.

1.9.3 Accessibility

“Open” includes keyboard, switch, voice, and assistive pointer paths as first-class actions. Readiness states (busy versus ready) should be communicable beyond color alone; WCAG 2.2 guidance frames why status must not depend on a single sensory channel (W3C 2023). Figures in this chapter carry alt text and reading-order descriptions; labs accept audio notes and structured text tables instead of screenshots when needed.

1.9.4 Equity

Do not assume a high-end phone, unlimited data, or Device Quartet hardware. Offline/fixture fallback is mandatory for **LAB-SYS-001**. “Bars look fine but the service fails” is also an equity story: shared networks, captive portals, throttled plans. Designing only for ideal always-online office conditions silently excludes many learners and workers.

1.10 10. Career lens

One familiar open crosses many ownership domains. No table promises employment; roles vary by organization. Completing Chapter 1 or **LAB-SYS-001** does not qualify anyone for a job. Your artifacts resemble early professional evidence in miniature.

Layer / concern	Example role family	Lab resemblance
Perceived readiness	UX / product design	Annotated chrome-vs-usable observations
Chrome vs data paths	Application / frontend engineering	Ecosystem map with UI/runtime/state cards
Symptom triage	IT support / operations	Two symptom→domain tickets labeled inference
Service dependency	SRE / backend / cloud	Online vs offline comparison notes
Link vs experience	Network engineering	Experience analysis without invented RF numbers
Inclusive status	Accessibility engineering	Notes on non-color busy/ready cues
Layered explanation	Systems / OS engineering	Mapping cards to textbook abstractions with citations
Evidence hygiene	Security / identity (introductory)	Privacy scrub checklist on portfolio files
Facilitation	Technical educator / mentorship	Probe questions for screen-as-surface misconceptions

When Device Quartet form factors appear in later chapters, they remain research/learning spines—not mascots and not fabricated shipping SKUs (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Commodity devices remain first-class citizens for Chapter 1 evidence.

1.11 11. Check understanding

Answer with reasoning. Prefer short paragraphs over one-word guesses.

1. Why might an app’s chrome appear quickly while the usable list stays empty?
2. Which parts of an open can still succeed when the Internet is unavailable?
3. Why is a connectivity indicator insufficient proof that a specific portal or sync service is healthy?
4. What evidence would you need before blaming the network for a slow or failed open?
5. How can expired login/session problems look like “the app is broken”?
6. Why is “the app” usually not one object? Name at least three cooperating parts.
7. How could a keyboard or switch open enter a similar readiness story after the hardware differs?
8. **Teach-back.** Explain the system lens to a family member **without** using the words *kernel*, *API*, *packet*, or *runtime*. Then introduce those terms one at a time, tying each to something already understood.

Educator note: successful teach-backs show visible surface versus hidden cooperating parts and at least one alternate path (local-only versus network-dependent), not memorized vocabulary lists. Gate 3 chapter-prototype reader evidence for Chapter 2 remains historical; this chapter does not claim completed full-manuscript human validation.

1.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography. Project-specific repository evidence is cited with keys such as `gunnchOS3k` (2026) and “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026) and remains distinct from external literature.

Inline citations used in this chapter include Saltzer and Kaashoek (2009), Tanenbaum and Bos (2022), WHATWG (2026b), WHATWG (2026a), Patterson and Hennessy (2020b), Postel (1981), Eddy (2022), The Linux Kernel Documentation (2026b), W3C (2023), and MDN Web Docs (2026b).

1.13 12. Glossary links

Terms introduced or relied on as formal vocabulary in this chapter should resolve in the living glossary registry as entries mature. Candidates below are for linking—not a dump of free-standing encyclopedia definitions.

Term	Role in this chapter
Visible interface	Surface the person can see or hear
Hidden layers	Cooperating parts under the interface
Technology ecosystem	Human + device + software + optional network producing usability
Dependency	Condition or component needed for a usable result
Failure domain	Bucket for where trouble may live
Inputs → processing → state → outputs	Teaching path for everyday interactive software
Event / event loop	Wait-and-dispatch structure in many interactive programs
State	Application’s current remembered condition
RAM	Fast working memory
Storage	Durable data holding
Packet / protocol	Network data unit and agreed exchange rules (optional branch)
Service	Provider of remote or local functions
Stability Contract	Experience depends on hidden conditions staying acceptable
Device Quartet	Research form-factor learning laboratory (PHYSICAL_PENDING)

Deeper entries, analogies labeled as analogies, and “not the same as” warnings live in the glossary network. Prefer following a link when stuck over inventing a private definition.

1.14 Figure references (embedded above; accessibility metadata)

All figures below are **conceptual** or **illustrative** unless a future revision cites a specific validated hardware release. Source preference: editable SVG in the publication repository. Reused from Concept Edition CE-1 preproduction art.

1.14.1 FIG-CH01-001 — First-minute ecosystem map

- **File:** figures/ecosystem/fig-ch01-001-ecosystem-map.svg (reuse of FIG-CE1-001)
- **Truth classification:** conceptual
- **Alt / reading order:** Person → Visible interface → Hidden local system → Optional network/service → Human-usable result; state conceptual educational architecture
- **Color-independent encoding:** Shape + label + dashed vs solid stroke for optional branch

1.14.2 FIG-CH01-002 — Inputs → processing → state → outputs

- **File:** figures/sequence/fig-ch01-002-inputs-state-outputs.svg (reuse of FIG-CE1-002)
- **Truth classification:** conceptual
- **Alt / reading order:** Numbered steps Inputs, Processing, State, Outputs, with feedback to human judgment; steps may overlap
- **Color-independent encoding:** Numbers + arrows

1.14.3 FIG-CH01-003 — Why “the app” is not one thing

- **File:** figures/architecture/fig-ch01-003-app-not-one-thing.svg (reuse of FIG-CE1-003)
- **Truth classification:** conceptual
- **Alt / reading order:** UI → Runtime → OS services → Storage → Network interface → Remote service; brace groups everyday “the app”
- **Color-independent encoding:** Labeled layers + brace

1.14.4 FIG-CH01-004 — Local-only vs network-dependent readiness

- **File:** figures/ecosystem/fig-ch01-004-local-vs-network.svg (reuse of FIG-CE1-004)
- **Truth classification:** illustrative
- **Alt / reading order:** Fork after chrome visible into Local path and Optional remote path; join at content usable or failed
- **Color-independent encoding:** Solid vs dashed branch labels

1.14.5 FIG-CH01-005 — Failure-domain map

- **File:** figures/architecture/fig-ch01-005-failure-domains.svg (reuse of FIG-CE1-005)
- **Truth classification:** illustrative
- **Alt / reading order:** Symptom → domain → evidence still needed
- **Color-independent encoding:** Icons + text labels; status words not color alone

1.14.6 FIG-CH01-006 — Exploded ecosystem preview

- **File:** figures/exploded-views/fig-ch01-006-ecosystem-preview.svg (reuse of FIG-CE1-006)
- **Truth classification:** conceptual; Device Quartet callout **PHYSICAL_PENDING**
- **Alt / reading order:** Device subsystem blocks + Quartet form-factor names as future lab spine; not a teardown photo
- **Color-independent encoding:** Leader lines + labels

1.14.7 FIG-CH01-007 — Stability Contract preview

- **File:** figures/architecture/fig-ch01-007-stability-preview.svg (reuse of FIG-CE1-007)
- **Truth classification:** conceptual
- **Alt / reading order:** List each concurrent condition and the human-usable gate outcome
- **Color-independent encoding:** Pass/fail words + icons; not green/red alone

Chapter 2

Chapter 2 — Follow One Tap Through the Entire Stack

Status: draft · Chapter ID: CH02

Author: Edmund Gunn, Jr.

2.1 1. The moment

You open an app you already know. Maybe it is a school portal, a messaging thread, a map, or a weather page. Your finger finds a familiar control—a button that says *Refresh*, *Send*, *Open*, or *Show more*—and you tap.

The button changes almost at once. It darkens, lifts, or flashes a tiny highlight. Something about the screen tells you: *I heard you*. A fraction of a second later—sometimes less than a blink, sometimes long enough that you notice—new content arrives. A list fills. A message status flips. A spinner disappears and a paragraph takes its place.

From your seat, it feels like one action. You asked; the device answered.

Underneath that feeling, many systems may have cooperated. Some of them live in the glass and metal under your finger. Some live in software that never shows itself. Some may live on another computer far away. Some may never leave your device at all.

This chapter follows that ordinary moment until it becomes visible, audible, or physical again. The governing question is simple and honest:

What actually happens between my finger touching a screen and the system responding?

We will not pretend every tap travels the same road. A tap that opens a local menu is not the same journey as a tap that fetches a remote document. Immediate feedback and later content are related, but they are not the same timeline. Learning to tell those stories apart is the first real skill of systems thinking.

2.2 2. What you notice

Before names like *kernel* or *packet* enter the conversation, notice the human contract you already expect.

When you tap, you expect the touch to be recognized. You expect feedback soon enough that the control feels alive. You expect the right action—not the neighboring button, not yesterday’s screen, not a frozen half-state. You expect the interface not to seize up. You expect the same tap not to fire twice by accident if your finger lingered. If the action depends on a network, you expect the remote part to arrive eventually, or to fail in a way you can understand. You also expect the device not to become unusably hot, and battery use to stay within reason for something so small.

Those expectations are not decorations around technology. They *are* the product, from the person’s point of view.

Responsiveness is a human perception produced by multiple technical timelines.

The button’s quick visual change may complete while a remote request is still traveling. The remote answer may arrive while the display is still finishing another frame. The radio may report “connected” while the specific service you need is unreachable. Your nervous system stitches those timelines into one story—“I tapped and it worked”—or into another—“I tapped and something felt wrong.”

Later chapters measure delay, jitter, and throughput with more machinery. Here, keep the human observation first: something felt immediate; something arrived later; something might have failed quietly while the interface still looked polite.

Optional comparison, available on almost any device you already own: tap a control that only changes local appearance (a menu open, a theme toggle, a local checkbox). Then tap a control that must fetch remote content. The first often finishes inside the device. The second may add waiting that has nothing to do with how hard you pressed the glass.

2.3 3. Exploded ecosystem

A tap is not a single object. It is a path through an ecosystem. Figure 2.1 is the first-minute map: person → device → optional network → result. Figure 2.2 opens the box conceptually. Both are **conceptual / Representative educational architecture**—not a claim that any specific manufactured revision looks exactly like the diagram. The Device Quartet used elsewhere in this series—Student 14.5-inch, Handheld Hybrid, DS-XL Coder, and Edge IO Wearables—are research form factors and learning benchmarks defined in the hardware industrial-design source of truth; physical fabrication remains pending (**PHYSICAL_PENDING**) (CLM-0003).

Walk the layers in ordinary language, then keep the same layers when vocabulary deepens.

2.3.1 Human

You form intent: refresh this page, send this note, open this file. Muscles move. Skin contacts glass or a trackpad. Later, eyes, ears, and hands judge whether the result matches what you meant.

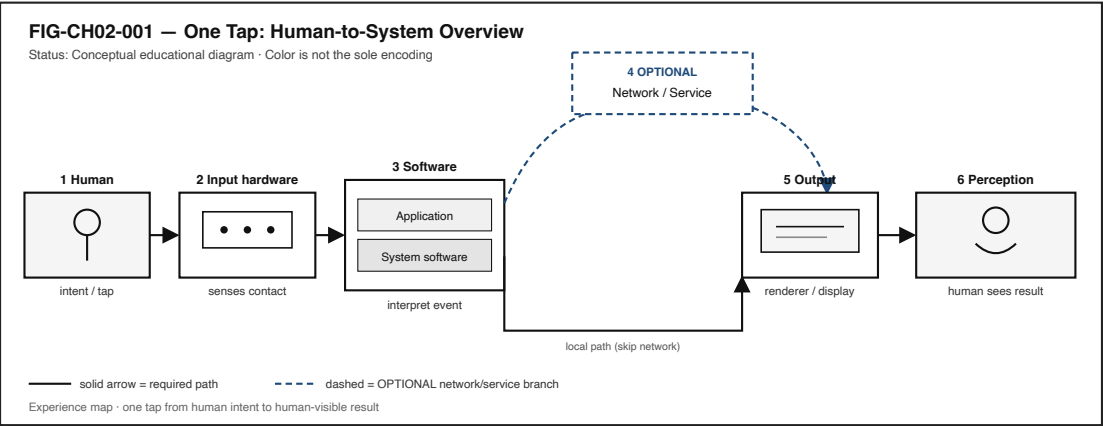


Figure 2.1: End-to-end map from human tap through input, software, optional network, output, and perception.

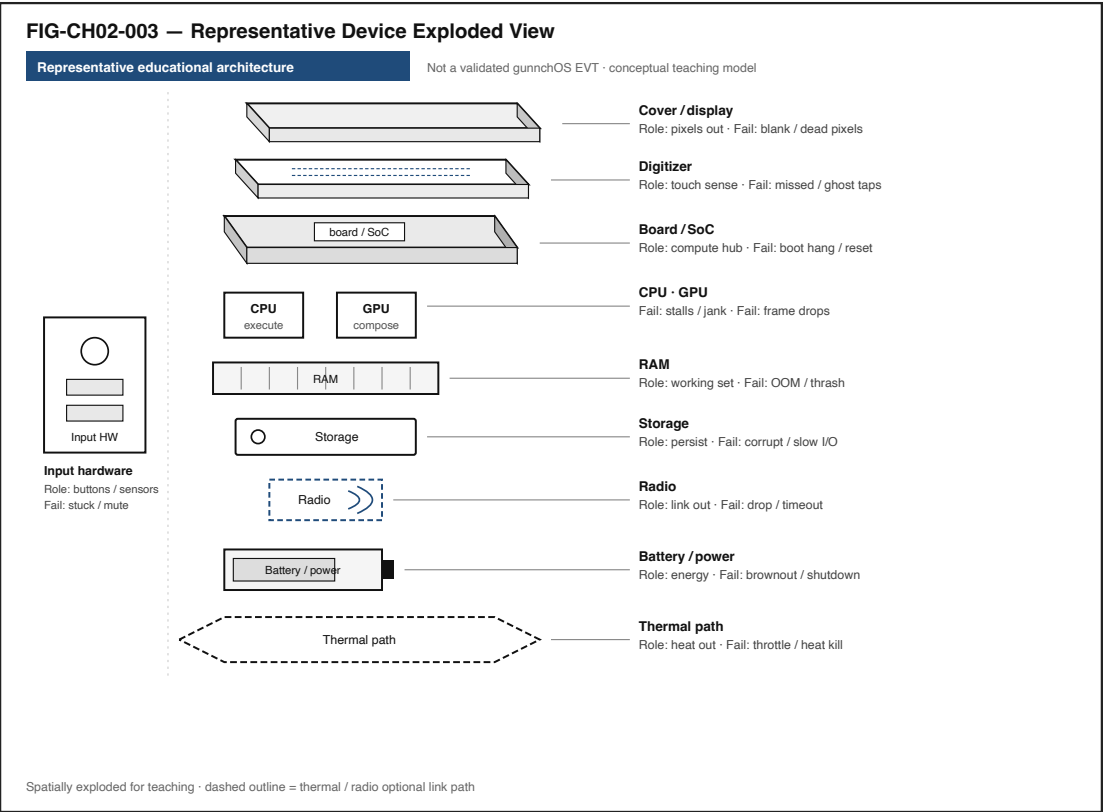


Figure 2.2: Exploded educational device view with display, digitizer, SoC, memory, storage, radio, battery, and thermal path.

2.3.2 Input hardware

Where a touchscreen is involved, cover glass, a touch-sensing layer (often called a **touch digitizer**), and input electronics convert contact into electrical signals. Controllers package those signals into digital reports—coordinates, contact state, sometimes pressure or size. On the web platform, related user-input event behavior is standardized in UI Events and Pointer Events (W3C 2026b, 2026a). Not every device uses the same sensing method, and not every interaction is a finger on glass. Keyboards, switches, voice, and assistive pointers enter through different hardware but must still become events software can interpret.

2.3.3 Compute hardware

A system-on-chip (**SoC**) typically gathers a **CPU** for general work, a **GPU** for highly parallel graphics and similar tasks, **RAM** as fast working memory, and storage for durable data (Patterson and Hennessy 2020b). Radios, batteries, power regulation, and thermal paths sit nearby. When the book shows these parts for learning devices, treat exploded views as **Representative educational architecture** unless a dated, revision-specific hardware evidence package is cited—and for gunnchOS hardware sources audited for this edition, that physical EVT evidence is not yet available (CLM-0003).

2.3.4 System software

Firmware may initialize hardware. A **kernel** mediates access to devices and resources. **Device drivers** translate between hardware reports and operating-system abstractions. An input subsystem, compositor or window system, and runtime or framework sit above, so applications do not personally micromanage every sensor.

On the gunnchOS device OS accepted **main**, what exists today is an **alpha / digital** experience layer with **SOFTWARE_SIMULATED** input routing for sources such as touch, controller, keyboard/mouse, and ring adapters—not a shipping OS bootable on reference hardware, and not a claim of physical ring silicon in the field (CLM-0002). When this chapter mentions that project, those labels travel with the sentence.

2.3.5 Application

Your app runs an **event loop** or equivalent structure: wait for events, dispatch handlers, update **state**, maybe read local storage, maybe call a service (WHATWG 2026b, 2026a). Local computation and remote calls are choices the application makes after it understands the event.

2.3.6 Networking (when applicable)

A socket or higher-level API, a network stack, a Wi-Fi or cellular interface, an access network, and the wider Internet may carry a request to an **edge** or **cloud** service. Packet transport over the Internet commonly rests on IP and TCP foundations (Postel 1981; Eddy 2022). Many taps never enter this layer. Teaching that “optional” word is part of the chapter’s honesty rule.

2.3.7 Output

A renderer prepares pixels (and sometimes audio or haptic patterns). The GPU and display pipeline present frames. Speakers and vibration motors may add confirmation.

2.3.8 Human feedback

Visual change, sound, vibration, and the felt sense of “alive” close the loop. The person does not experience layers; the person experiences the combined result.

Figure 2.3 stacks Application → Runtime/framework → Libraries/system services → Kernel → Drivers → Firmware → Hardware as a conceptual ladder. Names differ across platforms; the ladder’s purpose is orientation, not brand loyalty.

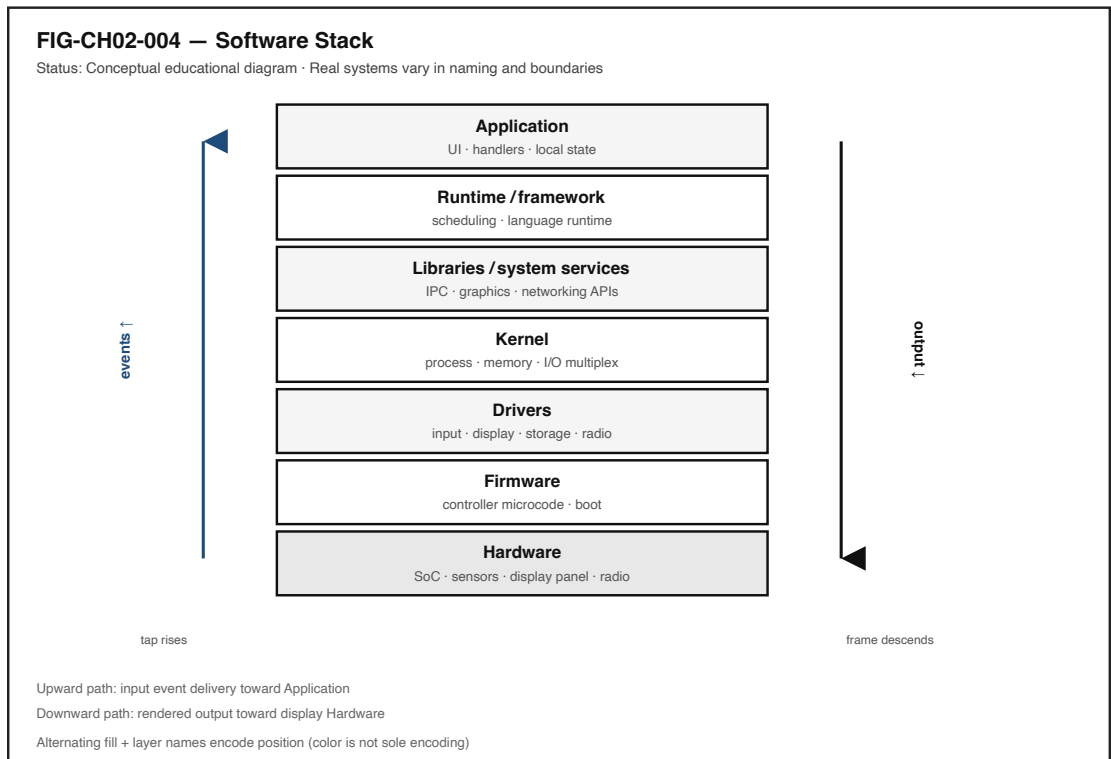


Figure 2.3: Vertical software stack from application down to hardware.

2.4 4. Follow the signal

Figure 2.4 shows a numbered path with optional branches and a failure branch. Read it as a logical story, not as a claim that a CPU executes exactly one step at a time with no overlap.

1. **Human intent.** You decide to act.
2. **Finger movement.** Contact begins.
3. **Sensing.** Digitizer and controller detect contact (or another input path activates).
4. **Event creation.** Coordinates and contact state become a digital touch (or key, switch, voice) event.
5. **Driver / input subsystem.** Hardware reports become OS-level input events (The Linux Kernel Documentation 2026c).
6. **Scheduler / process availability.** The operating system's **scheduler** decides when runnable software receives CPU time so the app can notice the event soon enough (The

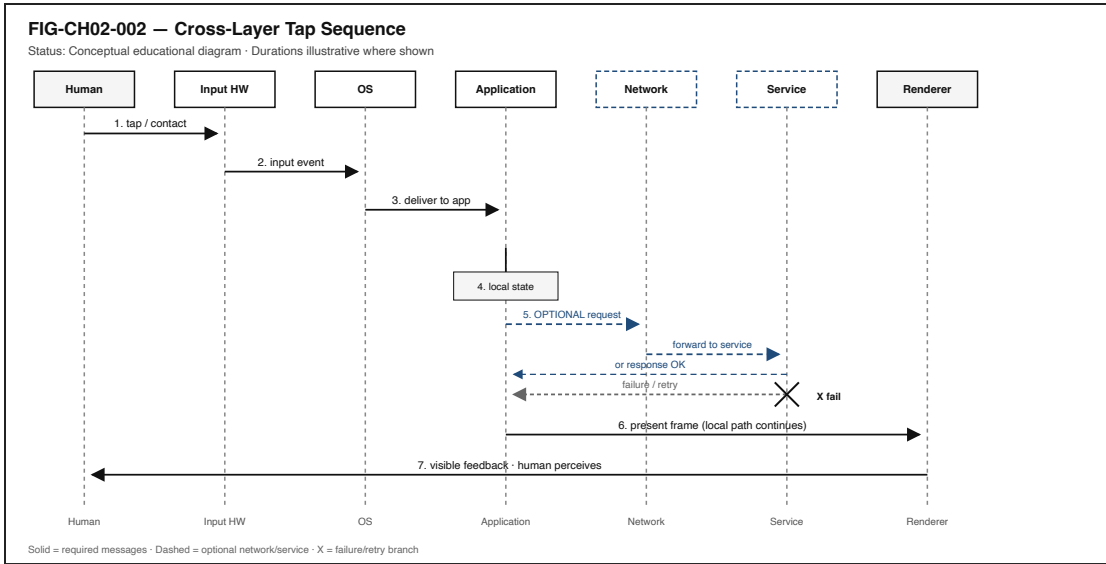


Figure 2.4: Sequence across Human, Input HW, OS, Application, Network, Service, and Renderer.

Linux Kernel Documentation 2026b; Tanenbaum and Bos 2022).

7. **Application event dispatch.** The runtime delivers the event toward the control under your finger.
8. **Handler executes.** Application code runs for that event.
9. **Local state changes.** The button’s pressed appearance, a flag, a list selection—something in memory flips.
10. **Optional storage / data access.** A local database, file, or cache may be consulted.
11. **Optional network request.** Only if the handler needs remote work.
12. **Network transmission.** Data moves as **packets** across interfaces and networks.
13. **Optional remote service processing.** An edge or cloud service may compute or fetch.
14. **Response arrives.** Bytes return—or fail, time out, or retry.
15. **Application state updates again.** New content merges into the interface model.
16. **Renderer prepares output.** Layout and paint work produce a frame description.
17. **GPU / display pipeline.** The frame becomes light on the screen (and optionally sound or haptics).
18. **Human perceives the result.** Eyes and brain close the experience.

Several of these may overlap. Sensing can continue while earlier events are still being handled. Rendering can proceed while a network request is in flight. A cache hit can skip remote processing entirely. A blocked UI thread can stall steps 7–9 even on a fast network.

2.4.1 Alternate paths (the honesty rule)

Not every tap goes to the cloud.

Path	Everyday example	What travels
Local-only	Open a menu; toggle a local theme; move a local game piece	Sensing → OS → app → render. No remote service required.
Local service	Change an OS setting; write a local file; query an on-device database	App talks to local services/storage. Still no Internet requirement.
LAN	Send a command to a printer or classroom device on the same network	Packets stay on the local network.
Internet / cloud	Fetch remote data; submit a form; invoke a hosted API	Request leaves the premises toward a distant service.
Edge	Request handled at a nearby edge service	Still networked, but closer than a far data center may be.
Cache hit	Content already available locally or nearby	Skip or shorten remote fetch.
Cache miss	Needed data is absent	Retrieve or compute elsewhere.

Figure 2.5 places these side by side. Immediate local feedback can succeed on the local-only path while a later remote path is still pending—or has already failed. That split is not a trick of one brand’s UI; it is a structural fact of modern interfaces.

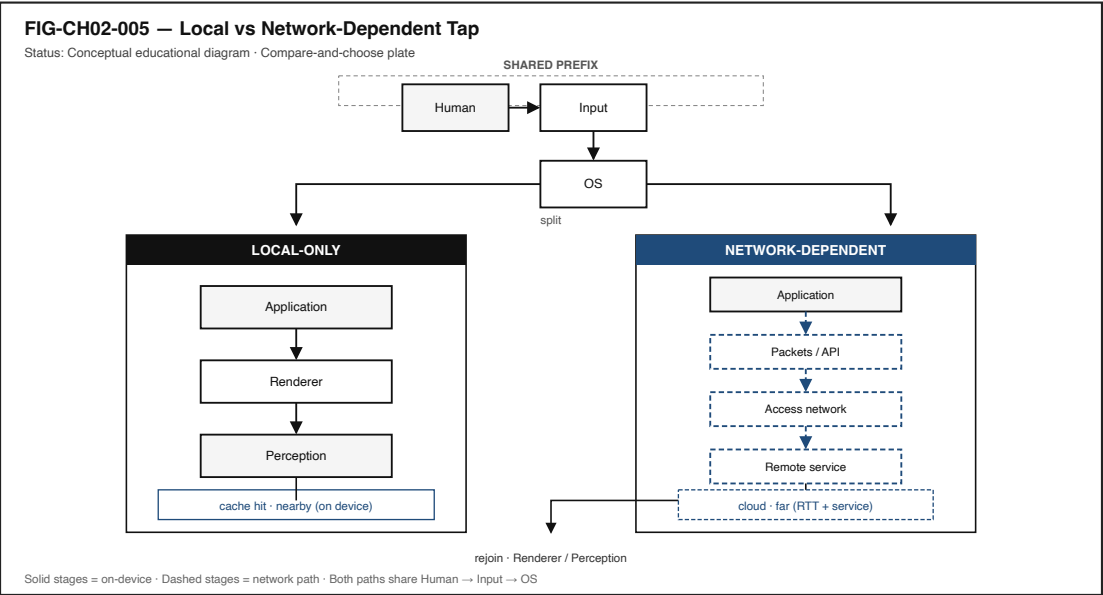


Figure 2.5: Side-by-side local-only and network-dependent execution paths.

2.5 5. Component cards

Component cards answer three questions: What is it? What does it do for the person? What fails when it misbehaves? Treat platform-specific names as aliases of these roles.

2.5.1 Touch digitizer

Plain definition. The sensing layer that detects touch position and movement on a touch surface.

Experience benefit. Touch becomes usable input instead of anonymous pressure on glass.

Failure symptom. Missed taps, ghost touches, delay, or inaccurate coordinates.

2.5.2 Device driver

Plain definition. Software that lets the operating system control hardware or receive information from it.

Experience benefit. Hardware reports become usable by the rest of the software stack.

Failure symptom. Device unavailable, unreliable, or incorrectly interpreted.

2.5.3 Kernel

Plain definition. The central part of an operating system that manages hardware access and system resources.

Experience benefit. Applications can use devices and memory without each app personally controlling every chip.

Failure symptom. Severe instability, unavailable resources, or system failure.

2.5.4 Scheduler

Plain definition. The operating-system mechanism that decides when runnable software receives CPU time.

Experience benefit. Important work—including input handling—gets compute soon enough to feel responsive.

Failure symptom. Input lag, stutter, audio glitches, general sluggishness even when “the network is fine.”

2.5.5 Event loop

Plain definition. A program structure that waits for events and dispatches work in response.

Experience benefit. The application reacts to input and other events instead of ignoring them.

Failure symptom. Frozen interface or delayed action when long work blocks the loop.

2.5.6 RAM

Plain definition. Fast working memory used by running software.

Experience benefit. Active data stays quickly available.

Failure symptom. Memory pressure, slowdowns, app termination, thrashing to storage.

2.5.7 GPU

Plain definition. A processor specialized for highly parallel work such as graphics rendering.

Experience benefit. Smooth visual output when frames complete on time.

Failure symptom. Missed frames, stutter, delayed rendering.

2.5.8 Packet

Plain definition. A formatted unit of data transmitted across a network.

Experience benefit. Digital information can move between systems.

Failure symptom. Loss, retries, delay, incomplete responses.

These cards are not a complete bill of materials. They are the first toolkit for naming failure domains when a tap feels wrong.

2.6 6. Stability contract

The **Stability Contract** is a signature idea across this book:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For Chapter 2, a successful tap-to-response experience may require all of the following to stay “good enough” at once:

- touch (or equivalent input) recognized,
- input event delivered into the software stack,
- application scheduled to run,
- event handler completes without hanging the UI path,
- memory available for working state,
- UI thread / event loop not blocked by unrelated heavy work,
- network path available *if* the action needs the network,
- remote service responds *if* the action depends on it,
- renderer completes a coherent update,
- frame reaches the display (and optional audio/haptics fire),
- total delay remains acceptable to the person.

Figure 2.6 shows these as concurrent conditions, not a vanity checklist.

Three separations matter:

1. The interface can look **alive** while a network request has already failed.
2. The network can report **connected** while the application is blocked on the CPU or waiting on storage.

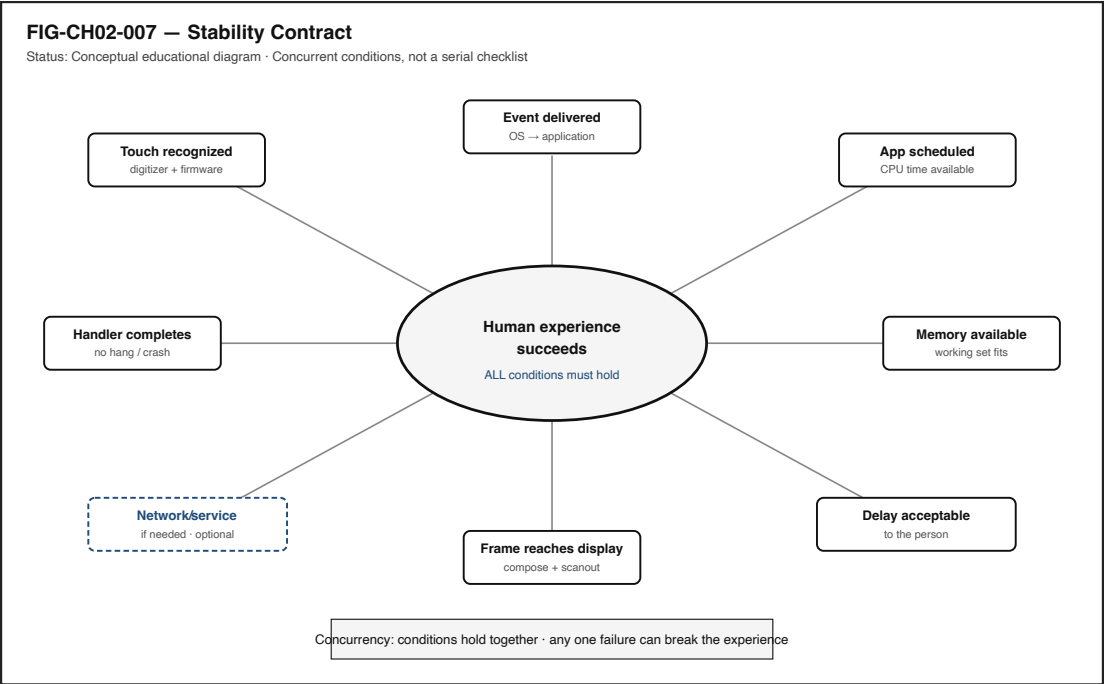


Figure 2.6: Hub-and-spoke diagram of concurrent Stability Contract conditions.

3. The application can finish its work while the **frame** has not yet reached the display.

You experience the combined result. Blaming “the Wi-Fi” or “the app” without evidence collapses those domains into one vague villain. Section 11 asks you to practice better blame.

Figure 2.7 sketches where time can accumulate. Segments shown there are labeled **illustrative** teaching aids (not measured gunnchOS benchmarks). This chapter does not invent hardware scores. Commodity lab timings you collect in **LAB-TAP-001** are *your* measured evidence for *your* device and browser—not a universal score.

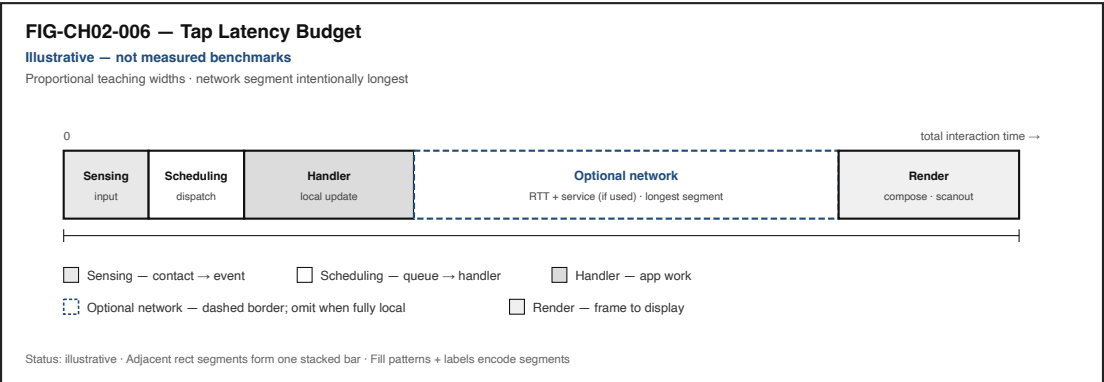


Figure 2.7: Horizontal stacked bar of illustrative latency segments.

2.7 7. Try it

2.7.1 LAB-TAP-001 — Observe a tap-to-response path on a device you already own

Observable question. How much of a tap-to-response path can I directly observe on a device I already own?

WAIKE alignment note. WAIKE (accepted `main`) maintains file-backed curriculum packages and lab validators; it does **not** currently ship a literal “trace one UI tap” module ID (CLM-0001). **LAB-TAP-001** is therefore a **publication-owned commodity lab**. Competency neighbors in WAIKE include input-action practice, networking datapath thinking, and embedded latency-budget fixtures—useful analogies, not renamed false lab IDs.

Prerequisites. A computer or phone you may use for learning; a modern browser; optional local Python if you choose Route B.

Safety. Do not capture passwords, tokens, private messages, or personal identifiers in screen-shots or logs. Prefer a local HTML file or a harmless public demo endpoint. No device rooting, no unsafe hardware modification.

Time estimate. About 45–90 minutes for the baseline browser route, including write-up.

Equipment / software. Browser developer tools and timing interfaces such as the Performance API where available (MDN Web Docs 2026b). Optional: a small local GUI toolkit for Route B. Optional: Android logging tools for Route C without root.

No-specialized-hardware route. Route A below.

2.7.1.1 Prediction

Before you measure, write one sentence: which portion do you expect to take longest—local handler work, network wait, or visible paint? Predictions make later surprises educational instead of merely annoying.

2.7.1.2 Route A — Browser (baseline)

Create or open a simple page with:

- a button,
- an event handler,
- a visible local state update (for example, change the button label immediately),
- an optional `fetch` to a benign URL.

Using developer tools, capture what you can of:

- input-related timestamps where the tools expose them,
- handler start and end,
- request and response timing,
- paint or performance entries where feasible.

2.7.1.3 Route B — Local application

Build a small Python or other local GUI that logs:

- event receipt,
- processing start,

- processing end,
- output update time.

Compare a local-only button to one that performs a short network request.

2.7.1.4 Route C — Android-compatible extension (optional)

Where available, collect application logs, frame timing, network timing, and input-related evidence **without root**. Treat missing kernel-level visibility as a limit, not a failure of curiosity.

2.7.1.5 Evidence (minimum)

- one screenshot or log excerpt (scrubbed),
- a timestamp table,
- a short explanation separating observation from inference.

2.7.1.6 Interpretation

Label each row in your table:

- **Directly observed** (tool reported this timestamp),
- **Inferred** (you believe this internal step occurred between observations).

2.7.1.7 Limits (say them out loud)

- Software timestamps do not measure every physical stage from skin to photon.
- Clocks and logging themselves add overhead.
- Browser instrumentation is not kernel-level measurement.
- One run is not a benchmark; it is a single story under one set of conditions.

2.7.1.8 Portfolio output

Produce a small folder containing:

1. README.md (question, method, limits),
2. one diagram of your observed path,
3. one result table,
4. one evidence artifact,
5. one reflection,
6. one “teach it to a nontechnical person” paragraph.

Completion means a claim is supported by an artifact—not that “the command ran.”

2.8 8. Build it

Use the same tap story at the depth that matches your pathway.

2.8.1 Explorer

Change the button’s text or local visual state. Explain, in ordinary language, what changed on the screen and what you think stayed inside the device.

2.8.2 Operator

Using developer tools, compare a local-only action with a network-dependent action. Note status codes, timing panels, and whether “Wi-Fi connected” predicted success.

2.8.3 Builder

Add timing instrumentation: log handler start/end; log request start/end; log when you update the DOM or GUI. Keep secrets out of logs.

2.8.4 Engineer

Break an interaction’s latency budget into **measured** segments (from your tools) and **inferred** segments (between measurements). Identify at least two failure domains that could produce the same human complaint (“it felt slow”).

2.8.5 Researcher

Design a repeated experiment: number of repetitions, environment controls (same device, same network class, quiet background load), a summary statistic, a variability measure, and an explicit limitations list. Separate correlation (“slow when on cellular”) from causation (what additional evidence would you need?).

Educators can facilitate teach-backs from Section 11 and adapt LAB-TAP-001 for classroom bandwidth and device constraints.

2.9 9. Secure and include it

2.9.1 Security

A tap may trigger privileged behavior: sending a message, paying a fee, changing a setting, granting a permission. Relevant ideas here—without turning this chapter into a full cybersecurity treatise—include:

- **permissions** (what the app is allowed to do),
- **authentication** (who is acting),
- **trusted UI** (can the person tell which control they actually pressed?),
- **secure transport** when data leaves the device,
- **validation** on the receiving service so crafted input cannot become unintended action.

2.9.2 Privacy

Touch and input telemetry can become sensitive when logged, stored, or correlated with identity and location. LAB-TAP-001 logs must not capture secrets. Prefer synthetic pages and scrubbed artifacts.

2.9.3 Accessibility

Not everyone interacts through touch. Equivalent paths include keyboard, switch control, voice, assistive technology, and alternative pointing devices. The system-level constant remains:

Human intent must become an input event that software can interpret.

If your lab device supports it, trigger the same local state change once by pointer and once by keyboard. Notice that different hardware can converge on similar application handlers.

On gunnchOS device OS accepted **main**, accessibility-related contracts and managers exist as part of the alpha digital layer; automated WCAG certification is **not** claimed (CLM-0002). Upstream WAIKE documents emphasize phone-first, low-cost, and offline-friendly learning intent; checklist completion and certification evidence remain incomplete (CLM-0001). Publication figures in this book carry their own alt text and text equivalents regardless.

2.9.4 Equity

A network-dependent tap may behave differently under weak connectivity, high latency, data caps, low-cost hardware, or older devices. Local-first feedback and honest failure states matter more, not less, when the network is fragile. Designing only for ideal office Wi-Fi silently excludes many learners and workers.

2.10 10. Career lens

One tap crosses many ownership domains. No table promises employment; roles vary by organization. Your LAB-TAP-001 artifacts resemble early professional evidence in miniature.

Layer	Example role	Professional artifact	Lab resemblance
Interaction	UX / HCI practitioner	Interaction specification	Written prediction + teach-back of expected feedback
Input hardware	Embedded engineer	Hardware/firmware interface notes	Description of sensing vs software event boundary
Driver	Systems engineer	Driver code / validation plan	Notes on what user-space never sees directly
Kernel	OS engineer	Scheduler / input-subsystem change write-up	Reasoning about scheduling under load
Application	App developer	Event-handling implementation	Instrumented handler and state update
Performance	Performance engineer	Trace / profile bundle	Timestamp table with measured vs inferred labels
Network	Network engineer	Packet / latency analysis	Request timing and loss/retry hypotheses
Wireless	Wireless engineer	Radio-performance analysis	Comparing “connected” vs service reachability
Service	Backend / cloud engineer	API / service implementation notes	Interpreting response success/failure independently of UI chrome

Layer	Example role	Professional artifact	Lab resemblance
Security	Security engineer	Threat model / control checklist	Permission and secret-scrub review of lab logs
Accessibility	Accessibility engineer	Conformance / accessibility review	Keyboard/switch/voice equivalent path check

When Device Quartet form factors appear in later labs—desk compute, handheld hybrid, learn-to-build coder, embodied wearables—they remain research/learning spines, not mascots and not fabricated shipping SKUs (CLM-0003). Commodity devices remain first-class citizens for Chapter 2 evidence.

2.11 11. Check understanding

Answer with reasoning. Prefer short paragraphs over one-word guesses.

1. Why might a button visually depress immediately even if the requested remote data takes about one second to arrive?
2. Which parts of the path can still work when the Internet is unavailable?
3. Why is “Wi-Fi connected” insufficient proof that the requested service is reachable?
4. What evidence would you need before blaming the network for a slow interaction?
5. Why might a heavily loaded device create input lag even on a fast network?
6. Why is a touch event not the same thing as an application action?
7. How could an accessible keyboard interaction enter a similar software path after the hardware differs?
8. **Teach-back.** Explain the entire interaction to a family member **without** using the words *kernel*, *interrupt*, *API*, or *packet*. Then introduce those four terms one at a time, tying each to something already understood in the teach-back.

Educator note: successful teach-backs show causal sequence and at least one alternate path (local-only vs network-dependent), not memorized vocabulary lists.

2.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Project-specific repository evidence remains in `evidence/claim_registry.yaml` and is cited by claim ID (for example, CLM-0002), separately from external literature.

Inline citations used in this chapter include W3C (2026b), WHATWG (2026a), Eddy (2022), The Linux Kernel Documentation (2026b), and Patterson and Hennessy (2020b).

2.13 12. Glossary links

Terms introduced or relied on as formal vocabulary in this chapter should resolve in the living glossary registry. This section lists them for linking—not as a dump of forty free-standing definitions.

Term	Role in this chapter
Touch digitizer	Sensing layer for touch position/movement
Device driver	Hardware OS translation software
Kernel	OS core mediating hardware and resources
Scheduler	Decides when runnable software gets CPU time
Event / input event	Digitized report of human action
Event loop	Wait-and-dispatch structure in applications
Handler	Code that runs in response to an event
State	Application's current remembered condition
CPU	General-purpose processor
GPU	Parallel processor often used for rendering
RAM	Fast working memory
Storage	Durable data holding
SoC	System-on-chip integrating major compute blocks
Packet	Formatted network data unit
Protocol	Agreed rules for exchanging data
Latency	Delay along a path (label numbers carefully)
Jitter	Variation in delay
Cache	Nearby stored copy that may satisfy a request
Cache hit / cache miss	Found locally vs must fetch/compute elsewhere
Edge computing	Processing nearer the user than a far core cloud
Cloud computing	Remote hosted compute/storage/services
Rendering / frame	Preparing and presenting visual output
Haptic feedback	Touch-based output such as vibration
Stability Contract	Experience depends on hidden conditions staying acceptable
Interrupt	(Introduce in teach-back stage two; do not imply every touch surfaces identically to every app)
API	Program interface between software layers/services
Process / thread	Units of running work scheduled by the OS
Radio	Wireless interface hardware/path
Router	Forwards packets between networks
Service	Running provider of remote or local functions

Deeper entries, analogies labeled as analogies, and “not the same as” warnings live in the glossary network. Prefer following a link when stuck over inventing a private definition.

2.14 Figure references (embedded above; accessibility metadata)

All figures below are **conceptual** unless a future revision cites a specific validated hardware release. Source preference: editable SVG in the publication repository. Reviewer and version fields fill during art production.

2.14.1 FIG-CH02-001 — One Tap: Human-to-System Overview

- **Caption.** A first-minute map from human intent through device layers to optional network and back to human perception.
- **Alt text.** Diagram linking a person tapping a device to optional remote services and a visible result.
- **Text equivalent / reading order.** (1) Human intent and tap → (2) Device input and compute → (3) Optional network/service → (4) Output on device → (5) Human perceives result.
- **Status.** Conceptual educational diagram.
- **Source.** Publication-owned original.

2.14.2 FIG-CH02-002 — Cross-Layer Tap Sequence

- **Caption.** Numbered cross-layer sequence with optional network branch and failure branch.
- **Alt text.** Sequence diagram of actors Human, Input hardware, OS, Application, Network, Service, Renderer returning to Human.
- **Text equivalent / reading order.** Follow steps 1–18 in Section 4; mark optional network steps; show failure returning an error state to the application and UI.
- **Status.** Conceptual.
- **Source.** Publication-owned original.

2.14.3 FIG-CH02-003 — Representative Device Exploded View

- **Caption.** Representative educational architecture showing display, digitizer, SoC (CPU/GPU), RAM, storage, radio, battery, power, thermal path, and input hardware.
- **Alt text.** Exploded educational device diagram of major hardware blocks.
- **Label required on art.** Representative educational architecture.
- **Status.** Conceptual; not a validated physical EVT of a gunnchOS SKU.
- **Source.** Publication-owned original informed by research form-factor roles (CLM-0003).

2.14.4 FIG-CH02-004 — Software Stack

- **Caption.** Conceptual ladder from application down through runtime, libraries/services, kernel, drivers, and firmware to hardware.
- **Alt text.** Vertical software stack diagram above hardware.
- **Reading order.** Top (application) to bottom (hardware).
- **Status.** Conceptual abstraction; platform names vary.
- **Source.** Publication-owned original.

2.14.5 FIG-CH02-005 — Local vs Network-Dependent Tap

- **Caption.** Comparison of a local-only tap path and a network-dependent tap path, including cache hit/miss and edge/cloud options.
- **Alt text.** Two-path comparison diagram for local versus networked taps.
- **Reading order.** Local path complete end-to-end; then network path with optional edge/cloud and cache branches.
- **Status.** Conceptual.
- **Source.** Publication-owned original.

2.14.6 FIG-CH02-006 — Tap Latency Budget

- **Caption.** Where time can accumulate along a tap-to-response path; numeric bands labeled illustrative, measured, or inferred—not fabricated product benchmarks.
- **Alt text.** Latency budget bar from input through optional network to display.
- **Status.** Conceptual teaching aid; replace illustrative bands with learner-measured values from LAB-TAP-001 where possible.
- **Source.** Publication-owned original.

2.14.7 FIG-CH02-007 — Stability Contract

- **Caption.** Hidden conditions that must remain good enough for a tap experience to succeed.
- **Alt text.** Checklist-style diagram of concurrent technical conditions behind a successful tap.
- **Reading order.** Input delivery → scheduling/handler → memory/UI thread → optional network/service → render/display → acceptable total delay.
- **Status.** Conceptual.
- **Source.** Publication-owned original.

2.15 Claim footnotes used in this chapter (project-specific)

Claim ID	Approved gist	Classification
CLM-0001	WAIKE accepted <code>main</code> provides file-backed curriculum packages and validators; no literal UI tap-trace lab module ID	repository-implemented (curriculum ops) + explicit non-claim for tap lab
CLM-0002	gunnchOS device OS accepted <code>main</code> exposes alpha digital experience with <code>SOFTWARE_SIMULATED</code> input routing; not shipping / not physical ring validated	repository-implemented (digital) with required qualifiers

Claim ID	Approved gist	Classification
CLM-0003	Device Quartet defined as research form factors / learning benchmarks; PHYSICAL_PENDING; educational exploded views are representative	repository-documented research form factors

General statements about RAM, packets, schedulers, and event loops are treated as general technical knowledge and are not rewritten as repository claims. Latency numbers, when shown in figures or learner tables, must carry **illustrative**, **measured**, or **inferred** labels.

End of Chapter 2 draft manuscript.

Chapter 3

Chapter 3 — Performance: Why Technology Feels Fast, Slow, Smooth, or Unstable

Status: WORKING_DRAFT_COMPLETE · Manuscript review: PENDING_FULL_MANUSCRIPT_REVIEW
· Publication readiness: NOT_PUBLICATION_READY

Chapter ID: CH03 · Author: Edmund Gunn, Jr.

Gate posture: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does not claim Gate 3 PASS)

3.1 1. The moment

You scroll a feed. You scrub a video scrubber. You switch apps. Sometimes motion is continuous enough that your attention stays on the content. Sometimes the same gesture hitches, freezes, then recovers unevenly—even when the network icon still looks fine, or the device is relatively new.

From your seat, the complaint is ordinary language: *fast, slow, smooth, janky, unstable*. Those words are not decorations around the product. They *are* the product’s first report card.

Chapter 2 followed one tap through layers and taught an honesty rule: immediate feedback and later content are related timelines, not one magic clock (W3C 2026b; MDN Web Docs 2026b). This chapter stays with the human report card and asks a sharper question:

What system behaviors make an ordinary experience feel fast, slow, smooth, or unstable—and how can I separate observation from guessed cause without inventing numbers?

We will not invent Device Quartet performance budgets, EVT thermal curves, or fleet-wide SLOs. Numeric hardware claims for those research form factors remain **PHYSICAL_PENDING** (CLM-CH03-004) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Classroom timings you collect later are *your* evidence for *your* conditions—not universal product law (CLM-CH03-005).

3.2 2. What you notice

Before jargon, name the feel.

Fast usually means a response arrives soon enough that waiting does not become the story. **Slow** means waiting *is* the story—progress indicators linger, key echoes trail fingers, lists fill late. **Smooth** means motion and updates feel continuous enough that your attention stays on intent. **Unstable** means the same action is sometimes fine and sometimes not: hitch, stall, recovery, surprise.

Those words mix several technical stories. A device can move a lot of data per second (**throughput**) while still feeling unresponsive if the *first* useful response is delayed (**latency**) or if delay wanders unpredictably (**jitter**) (Saltzer and Kaashoek 2009). A screen can look “busy” while the interactive path is starved. A connectivity icon can look healthy while local contention is the real seat problem (Tanenbaum and Bos 2022).

Perceived performance is a human judgment produced by multiple measurable behaviors—not a single score (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023). **Responsiveness** is that seat judgment for interactive work: whether latency (and its variability) stayed tolerable—not a synonym for peak throughput.

Notice three traps early:

1. Treating peak throughput (“this is a fast chip / fast link”) as proof of a responsive seat experience.
2. Treating “Wi-Fi connected” as proof that slowness is not local.
3. Treating one lucky run—or one CH02 lab timing—as a universal SLO for every chapter and device (CLM-CH03-005).

Optional comparison on a device you already own: scroll a mostly local document, then scrub a video that must fetch remote segments, then switch apps while several tabs or documents are open. Do not yet claim root cause. Only write what you felt and what visible clues appeared (spinner, hitch, fan sound, warmth, monitor percentages if you open them).

3.3 3. Exploded ecosystem

Feel is not a single object. It is a path through an ecosystem. **FIG-CH03-001** maps everyday feel words onto diagnostic axes: latency, jitter, throughput, stall/hitch, and availability. Read it as orientation, not as a measured scoreboard.

Walk the teaching model used across this book:

Human experience → system → component → code → network → society

3.3.1 Human

You intend a continuous action: keep scrolling, keep scrubbing, keep switching. Eyes and hands judge whether the system kept up. Vestibular and cognitive load rise when motion stutters; that is already an accessibility concern, not only an aesthetics note.

3.3.2 Application and runtime

An app updates state, lays out content, asks for frames, and may start local or remote work. An **event loop** that blocks on heavy work can make a fast network look slow (WHATWG 2026b; MDN Web Docs 2026b).

3.3.3 Operating system

A **scheduler** decides when runnable work gets CPU time (The Linux Kernel Documentation 2026b; Tanenbaum and Bos 2022). Memory managers reclaim under pressure. Storage stacks complete (or delay) durable writes. The OS mediates; it does not replace the app’s computation.

3.3.4 Compute and memory hardware

CPU, optional GPU/accelerators, caches, and RAM form a hierarchy with different roles (Patterson and Hennessy 2020b). Storage is the ordinary durable home for files—not interchangeable with RAM. Contended bandwidth or a working set that no longer fits can turn “simple” motion into hitch.

3.3.5 Network (optional)

Packets, queues, loss, and retransmission matter when the action depends on remote bytes. Many feel failures never enter this layer. Teaching that optional branch is part of the honesty rule from Chapter 2.

3.3.6 Power and thermal policy

Sustained work can reduce available performance to protect the device and stay within energy budgets (The Linux Kernel Documentation 2026a). Heat and slowdown can be policy, not a mysterious software “bug.”

3.3.7 Society and equity

Fragile networks, shared school devices, older hardware, and locked-down monitors change what “try it” can mean. Fixture routes exist so diagnosis literacy is not gated on admin rights or high-end laptops.

Device Quartet silhouettes, when shown elsewhere, remain **representative educational architecture** / learning benchmarks—not shipping SKUs and not fabricated timing tables (CLM-CH03-004) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

3.4 4. Follow the signal

FIG-CH03-002 is the causal spine for this chapter:

Action → work → contention or wait → perceptible hitch or delay → human judgment

Read the numbered story as logic, not as a claim that hardware executes exactly one step at a time with no overlap.

1. **Human action.** Scroll, scrub, switch, save, export.

- 2. **Input becomes events.** Pointer, keyboard, or assistive path enters software (W3C 2026a).
- 3. **Interactive work is scheduled.** Threads become runnable; the scheduler allocates time (The Linux Kernel Documentation 2026b).
- 4. **Computation and data movement.** Instructions execute; caches and RAM serve a working set (Patterson and Hennessy 2020b).
- 5. **Optional storage I/O.** Saves, opens, indexes, or write-back compete for the disk path (Tanenbaum and Bos 2022).
- 6. **Optional network I/O.** Remote bytes are requested; waiting begins if needed (MDN Web Docs 2026c).
- 7. **Rendering / composition.** Frames must complete often enough for motion to feel continuous.
- 8. **Power/thermal feedback.** Governors may reduce clocks under sustained load (The Linux Kernel Documentation 2026a).
- 9. **Perception.** You experience combined latency, variability, stalls, and progress—not layers in isolation.

3.4.1 Four diagnostic axes (keep them separate)

Axis	Plain question	Feel word it often colors
Latency	How long until a useful response?	Fast / slow
Jitter	How much does that timing wander?	Smooth / unstable
Throughput	How much work completes per time?	“Powerful” yet still sluggish if latency dominates
Availability / errors	Did the needed path work at all?	Broken / unreliable (distinct from “merely slow”)

Quality models for systems and software treat characteristics such as performance efficiency and reliability as related but not identical vocabulary (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023). Telecommunications vocabulary likewise separates performance, quality of service, and quality of experience as related families of terms—not one interchangeable slogan (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.). This chapter uses that separation pedagogically: **do not collapse every complaint into “the network” or “the CPU.”**

FIG-CH03-003 shows the teaching point: two traces can share a similar *average* latency while one has calm spacing and the other has clusters of long gaps. The second often feels unstable even when a single average looks “fine.” Labels on that plate remain **illustrative**—not measured gunnchOS benchmarks.

FIG-CH03-004 contrasts local contributors (CPU/memory/storage/scheduler/thermal) with optional network contributors. Use it to ask “which branch has evidence?”—not to declare a villain from habit.

3.4.2 Local contention with a healthy icon

Local resource contention—CPU time, memory pressure, storage I/O wait, scheduler competition—can produce “slowness” while connectivity still looks fine (CLM-CH03-003)

(Tanenbaum and Bos 2022). That is the adjacent vocabulary inherited carefully from Concept Edition CE-3 (which maps primarily to later full-book systems chapters, not as a title twin of CH03). Chapter 3 owns *feel measure*; CE-3/CH06–CH12 deepen the inside-the-device machine.

3.5 5. Component cards

Component cards answer: What is it? What does it do for the person? What fails when it misbehaves?

3.5.1 Perceived performance

Plain definition. How fast, smooth, and stable the experience feels to a person.

Experience benefit. Trust that the system is keeping up with intent.

Failure symptom. Frustration, repeated taps, abandoned tasks, wrong-layer blame.

3.5.2 Latency

Plain definition. Time from an action to a perceptible useful response (Saltzer and Kaashoek 2009; MDN Web Docs 2026b).

Experience benefit. Waiting does not become the main story.

Failure symptom. Spinners dominate; echoes trail; “did it hear me?”

3.5.3 Jitter

Plain definition. Variability in timing; uneven gaps between responses or frames.

Experience benefit. Motion and turn-taking feel predictable enough to trust.

Failure symptom. Hitching, surging, “sometimes fine, sometimes awful.”

3.5.4 Throughput

Plain definition. How much work completes per unit time (Saltzer and Kaashoek 2009; Patterson and Hennessy 2020b).

Experience benefit. Large jobs finish; bulk transfers progress.

Failure symptom. Big exports crawl—or, conversely, high throughput with poor interactive latency (“busy but unresponsive”).

3.5.5 Stall / hitch

Plain definition. A visible interruption of expected continuous motion or progress.

Experience benefit. Continuity supports comprehension and motor planning.

Failure symptom. Frozen frames, queued clicks, beachballs, scrubber jumps.

3.5.6 Contention

Plain definition. Competing work for limited CPU, memory, storage, GPU, or radio budget (Tanenbaum and Bos 2022).

Experience benefit. Sharing resources fairly enough that interactive paths stay usable.

Failure symptom. Everything feels heavy under mild multitasking; monitors show crowded run queues or disk storms.

3.5.7 Thermal / power limit (qualitative)

Plain definition. Energy and heat constraints that can reduce available performance (The Linux Kernel Documentation 2026a).

Experience benefit. The device survives sustained work without unsafe heat.

Failure symptom. The same action feels slower after prolonged load; fans rise; progress decelerates without a new “bug” appearing in the UI copy.

These cards are a diagnosis toolkit, not a complete bill of materials.

3.6 6. Stability contract

The **Stability Contract** is a signature idea across this book:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For Chapter 3’s performance lens, a smooth usable feel may require several conditions at once:

- interactive work becomes runnable soon enough,
- latency to first useful feedback stays within what the person will tolerate for that task,
- timing variability stays calm enough that motion does not feel random,
- needed throughput for the foreground job remains adequate,
- memory and storage paths do not thrash the interactive thread into waits,
- optional network/service dependencies succeed *if* the action needs them,
- rendering completes coherent frames often enough,
- power/thermal policy still permits the needed performance,
- errors remain recoverable instead of silent half-states.

Qualitative wording is intentional. This chapter does **not** mint universal numeric SLOs for all apps and devices. CH02’s latency-budget figure and LAB-TAP-001 timings are method precedent and prototype evidence—not product law for Chapter 3 (CLM-CH03-005).

Three separations matter here:

1. High throughput can coexist with poor interactive latency.
2. Healthy connectivity can coexist with local contention.
3. A polite spinner can coexist with a failed request or a blocked UI thread.

You experience the combined result. Later chapters (including formal QoE/stability synthesis) deepen measurement design; here the contract restores agency: **inspect before you accuse.**

3.7 7. Try it

3.7.1 LAB-PERF-001 — Make Feel Visible

Observable question. Under two load conditions on a device I already own, what wall-clock and commodity observations can I record so that “felt slow / felt smooth” becomes evidence instead of vibes?

Relationship to other labs. Chapter 2’s **LAB-TAP-001** traces one tap path. Concept Edition CE-3’s **LAB-CMS-001** makes *local* bottleneck symptoms visible with OS monitors. **LAB-PERF-001** is the publication-owned feel→measure lab for this chapter: same honesty rules, different emphasis (feel words axes), with LAB-CMS-001 as the preferred deep local-diagnosis neighbor.

WAIKE alignment note. WAIKE accepted **main** offers adjacent competencies (observability habits, storage triage, playtest metrics, embedded latency-budget thinking)—not a literal CH03 performance module ID. Do not invent one.

Prerequisites. A computer or phone you may use for learning; a modern browser and/or built-in OS monitor; optional local Python.

Safety. Do not capture passwords, tokens, private messages, or personal document contents. Prefer synthetic pages and redacted filenames. Mild load only. Stop if the device warns about heat or becomes uncomfortably hot. No rooting, no untrusted “optimizer” tools, no disabling security software.

Time estimate. About 45–90 minutes including write-up.

3.7.1.1 Prediction

Before measuring, write which axis you expect to dominate under load: latency, jitter/stalls, throughput limits, or availability/errors—and whether you expect the cause branch to look local, network, or unclear.

3.7.1.2 Route A — Browser feel timeline (baseline)

Use a simple local page or benign demo:

- perform one continuous interaction (scroll a long local page, or repeatedly trigger a local visual update),
- then add a mild second load (extra tabs or a second local task),
- capture what browser performance tools expose (user timestamps, long tasks, resource timing where applicable) (MDN Web Docs 2026b, 2026c),
- record wall-clock duration for a fixed action count (for example, “time to finish 10 deliberate scrolls”) with a phone timer if needed.

3.7.1.3 Route B — OS monitor neighbor (LAB-CMS-001)

Follow **LAB-CMS-001** Route A or fixture fallback: before/during CPU, memory, and disk snapshots during one controlled local action. Use this when the feel problem appears while connectivity looks fine.

3.7.1.4 Route C — Offline / fixture fallback

If monitors are inaccessible, use LAB-CMS-001 fixtures for local-diagnosis literacy. Treat fixture numbers as **teaching illustrations**, not measurements of your device. The measured

annotated CMS monitor plate remains **BLOCKED_EVIDENCE_REQUIRED** until qualifying evidence exists; synthetic fixtures labeled with that ID do **not** unblock the measured figure.

3.7.1.5 Evidence (minimum)

- a feel log (fast/slow/smooth/unstable words + timestamps),
- a two-condition observation table (idle/light vs mild load),
- one scrubbed screenshot, log excerpt, or fixture ID,
- labels: **directly observed** vs **inferred**.

3.7.1.6 Interpretation rules

Allowed observation	Inference that must be labeled
Wall-clock for action X rose from A to B	“Root cause is CPU”
Long gaps between frames in a tool	Exact physical display deadline miss without more evidence
Resource timing shows waiting on network	“Wi-Fi is broken” without path evidence
Device felt warmer	Specific throttle temperature

3.7.1.7 Limits (say them out loud)

- One run is not a benchmark.
- Software timestamps do not measure every physical stage from muscle to photon.
- Browser instrumentation is not kernel-level truth.
- CH02 / LAB-TAP-001 timings are not universal SLOs for this chapter (CLM-CH03-005).

3.7.1.8 Portfolio output

1. README.md (question, method, limits),
2. feel→axis map for your experience,
3. observation table,
4. one evidence artifact,
5. reflection separating observation from inference,
6. teach-back paragraph for a nontechnical person.

3.8 8. Build it

Use the same feel story at the depth that matches your pathway.

3.8.1 Explorer

Name the feel words you noticed. Point to at least three possible contributing factors (for example: waiting on content, local hitch, thermal slowdown) **without** claiming root cause.

3.8.2 Operator

Record wall-clock and simple OS/browser observations under two load conditions. Keep observation and inference columns honest. Prefer LAB-PERF-001 Route A plus LAB-CMS-001 when the icon looks fine.

3.8.3 Builder

Build a labeled **feel** → **candidate cause** map for one experience. Show local vs optional network branches. Change one variable (fewer tabs, airplane mode for a local-only task, shorter scroll distance) and note what changed.

3.8.4 Engineer

Separate latency, jitter, throughput, and availability as distinct diagnostic axes for one incident write-up. List next evidence you would need before upgrading a hypothesis to a cause claim.

3.8.5 Researcher

Propose a small controlled comparison: repetitions, environment controls, a summary statistic, a variability measure, and explicit instrument limits. State what would falsify your leading hypothesis. Do not fabricate significance from a classroom N.

Educators can facilitate misconception probes from Section 11 and use LAB-CMS-001 fixtures when live monitors are inequitable across a classroom.

3.9 9. Secure and include it

3.9.1 Security

Performance panic is a social-engineering surface. “PC cleaner” and miracle optimizer downloads often market themselves at exactly the symptoms this chapter names. Teach inspection with built-in tools before trust of unknown utilities. Labs must not require disabling security software, loading untrusted kernel modules, or capturing privileged sessions.

3.9.2 Privacy

Timing logs, monitor screenshots, and portfolio artifacts can leak filenames, account names, thumbnails, and location-correlated habits. Redact. Prefer synthetic workloads. Do not require cloud sync of lab evidence.

3.9.3 Accessibility

Stutter and lag are accessibility failures for many motor, cognitive, and vestibular profiles—not merely polish defects. Motion is not the only feedback channel: provide text status, keyboard-operable controls, and non-color encodings in figures and lab sheets. Equivalent input paths (keyboard, switch, assistive pointer) must still become schedulable work the interactive path can handle (W3C 2024).

3.9.4 Equity

Low-cost devices, shared machines, data caps, and locked-down school images are normal learning contexts. “Buy more RAM” is not the moral of the chapter. Diagnosis, workload control, and fixture routes come first. Device Quartet form factors are learning benchmarks, not purchase requirements (CLM-CH03-004).

3.10 10. Career lens

Performance work crosses ownership domains. No table promises employment; roles vary by organization. LAB-PERF-001 and LAB-CMS-001 artifacts resemble early professional evidence in miniature.

Concern	Example role	Professional artifact	Lab resemblance
Feel language metrics	UX researcher / HCI practitioner	Study notes separating subjective report from instrument	Feel log + axis labels
Interactive latency	Performance engineer	Trace bundle with measured vs inferred spans	Timestamp table with honesty labels
Local contention	OS / systems engineer	Scheduler / memory / I/O analysis notes	LAB-CMS-001 observation vs inference columns
Runtime health	Application engineer	Main-thread / event-loop profile	Blocked-loop hypothesis with evidence plan
Capacity vs speed	SRE / reliability engineer	Error budget and latency percentile discussion	Availability separated from “merely slow”
Sustained load	Embedded / power engineer	Thermal/power policy notes (qualitative or measured)	Warmth as labeled inference, not fake °C claims
Inclusive performance	Accessibility specialist	Motion/status alternatives review	Keyboard path + text equivalent checks
Classroom diagnosis	Educator / lab facilitator	Facilitation sheet with fixture fallback	LAB-CMS-001 facilitation artifacts

When later chapters deepen QoE, networking, or hardware budgets, keep the same discipline: **evidence scope travels with the sentence.**

3.11 11. Check understanding

Answer with reasoning. Prefer short paragraphs over one-word guesses.

1. Why can a system with high throughput still feel slow at the seat?
2. What is the difference between latency and jitter in everyday feel language?
3. Why is “Wi-Fi connected” insufficient proof that a hitch is a network problem?
4. Give one example where local contention could produce instability while a connectivity icon looks healthy.
5. What evidence would you need before blaming thermal limits—and what must stay labeled inference without sensors?
6. Why are CH02 / LAB-TAP-001 timings not universal product SLOs for this chapter?

7. How should a classroom treat the blocked CMS measured plate while it remains blocked?
8. **Teach-back.** Explain to a family member why “fast chip” and “feels fast” are not the same sentence—without using the words *throughput*, *jitter*, or *scheduler*. Then introduce those three terms one at a time, tying each to something already understood.

Educator note: successful teach-backs show at least two axes and one alternate cause branch (local vs network), not memorized vocabulary lists.

3.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`), with working Full31 harvest keys aligned to `publication/full131/WORKING_BIBLIOGRAPHY.bib`. Project-specific repository evidence is cited by claim ID where needed and kept distinct from external literature.

Inline citations used in this chapter include Saltzer and Kaashoek (2009), “Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” (2023), “Vocabulary for Performance, Quality of Service and Quality of Experience” (n.d.), Tanenbaum and Bos (2022), Patterson and Hennessy (2020b), The Linux Kernel Documentation (2026b), The Linux Kernel Documentation (2026a), MDN Web Docs (2026b), MDN Web Docs (2026c), “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026), and W3C (2024).

Unresolved evidence gaps from the chapter packet (including classic HCI response-time category thresholds pending exact edition binding) are **omitted** from reader prose rather than filled with invented numbers.

3.13 12. Glossary links

Terms introduced or relied on as formal vocabulary in this chapter should resolve in the living glossary registry. This section lists them for linking—not as a dump of free-standing encyclopedia entries.

Term	Role in this chapter
Perceived performance	How the experience feels to a person
Latency	Time to a perceptible useful response
Jitter	Variability in timing
Throughput	Work completed per time
Stall / hitch	Visible break in expected continuity
Contention	Competition for limited resources
Availability	Whether a needed path works at all
Scheduler	OS mechanism allocating CPU time
Event loop	Wait-and-dispatch structure in apps
RAM	Fast working memory
Storage	Durable data holding
Frame / rendering	Preparing and presenting visual output
Thermal / power limit	Energy/heat policy that can reduce performance

Term	Role in this chapter
Quality of experience (QoE)	Experience-centered quality vocabulary (pointer; deeper later)
Stability Contract	Experience depends on hidden conditions staying acceptable

Deeper entries and “not the same as” warnings live in the glossary network.

3.14 Figure references (conceptual plates; accessibility metadata)

All figures below are **conceptual** or **illustrative** unless a future revision cites a dated measurement bundle. SVG production may follow; absence of a rendered file does not authorize invented telemetry. Source preference: editable SVG in the publication repository.

3.14.1 FIG-CH03-001 — Feel Words to Diagnostic Axes

- **Caption.** Everyday feel words mapped to latency, jitter, throughput, stall/hitch, and availability axes.
- **Alt text.** Diagram linking fast/slow/smooth/unstable language to distinct diagnostic axes; conceptual, not measured.
- **Reading order.** Feel words → axis definitions → reminder that one feel word can involve multiple axes.
- **Status.** Conceptual educational diagram (planned).
- **Source.** Publication-owned original (preproduction intent).

3.14.2 FIG-CH03-002 — Action to Hitch Causal Flow

- **Caption.** Action → work → contention/wait → perceptible hitch → human judgment.
- **Alt text.** Left-to-right causal flow from human action to perceived hitch with optional local and network wait branches.
- **Reading order.** Follow Section 4 steps; mark optional network and thermal feedback.
- **Status.** Conceptual.
- **Source.** Publication-owned original (preproduction intent).

3.14.3 FIG-CH03-003 — Same Average, Different Jitter

- **Caption.** Two illustrative timelines with similar average latency and different gap variability.
- **Alt text.** Twin timelines showing calm versus clustered delays; labeled illustrative teaching aid.
- **Status.** Illustrative—not measured gunnchOS benchmarks.
- **Source.** Publication-owned original (preproduction intent).

3.14.4 FIG-CH03-004 — Local vs Network Contributors to Feel

- **Caption.** Side-by-side local and network contributor map for performance feel without deep network-chapter detail.

- **Alt text.** Comparison plate of local contention domains versus optional network/service domains.
- **Reading order.** Local column complete; then optional network column; then “evidence?” checkpoint.
- **Status.** Conceptual.
- **Source.** Publication-owned original (preproduction intent).

3.14.5 the blocked CMS measured plate — Annotated commodity monitor snapshot (BLOCKED)

- **Status.** BLOCKED_EVIDENCE_REQUIRED.
- **Note.** LAB-CMS-001 ships synthetic teaching fixtures that may carry this ID for offline literacy. Those fixtures do **not** unblock a measured figure. Do not present fixture numbers as Device Quartet or fleet evidence.

3.15 Claim footnotes used in this chapter

Claim ID	Approved gist	Status / class
CLM-CH03-001	Perceived responsiveness depends on latency and variability, not only peak throughput	SOURCE_IDENTIFIED (general technical)
CLM-CH03-002	Latency, reliability/availability, and throughput are distinct failure/feel axes	SOURCE_IDENTIFIED (general technical)
CLM-CH03-003	Local contention can produce slowness while connectivity looks healthy	SOURCE_IDENTIFIED (general technical)
CLM-CH03-004	Numeric Device Quartet performance budgets remain PHYSICAL_PENDING	PHYSICAL_PENDING (project-specific)
CLM-CH03-005	CH02 / LAB-TAP-001 timings are prototype evidence, not universal product SLOs	SOURCE_IDENTIFIED (publication internal)

General statements about schedulers, memory hierarchy, and browser timing APIs are treated as general technical knowledge scoped to the cited works. Latency numbers, when shown in learner tables, must carry **illustrative**, **measured**, or **inferred** labels.

End of Chapter 3 working draft manuscript. Pending full-manuscript review. Not publication-ready. Gate 3 remains in progress with reader evidence pending.

Chapter 4

Chapter 4 — The Device Quartet as a Learning Laboratory

Status: draft · Chapter ID: CH04
Author: Edmund Gunn, Jr.
Gate posture: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (not Gate 3 PASS)
Physical Device Quartet claims: PHYSICAL_PENDING

4.1 1. The moment

You imagine one ordinary goal—write a short report, measure how long a page takes to become usable, sense whether a device is with you, or carry a work session across a day—and then you ask a dishonestly simple question:

What if the *same* goal had to live on four different bodies of technology?

On a clamshell at a desk, the task has a keyboard, a stable posture, and room to spread windows. On a hybrid handheld that can also dock, the same task must survive pockets, one-handed moments, and sudden keyboard availability. On a larger coding-and-display form, the task becomes a learn-to-build loop: edit, run, inspect, revise. On a wearable or edge I/O device near the body, the task is no longer only about screens; it is about sensing, proximity, consent, and what it means for compute to move with you.

From the seat of imagination, the goal is identical. The failure modes are not.

This chapter introduces the book’s **Device Quartet** as a **learning laboratory**: four recurring research form factors used to expose different constraints, not to sell shipping SKUs (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026; Gunn 2026a). The four names you will meet again are:

Research form factor	Learning role in this book
Student 14.5-inch	Sustained desk compute for full learning and work sessions
Handheld Hybrid	Mobile and docked compute

Research form factor	Learning role in this book
DS-XL Coder	Strongest learn-to-build device lens
Edge IO Wearables	Embodied sensing and body-proximate interaction

Those roles come from the publication device index and the hardware industrial-design research docs audited for this edition. Physical fabrication and engineering validation testing (EVT) measurements remain pending—**PHYSICAL_PENDING**—so this chapter refuses invented battery capacities, wattage curves, thermal °C plots, FPS scores, RF field numbers, latency benchmarks, or shipping claims (CLM-0003; CLM-CH04-002).

You do not need Quartet hardware to learn here. Commodity devices you already own are enough for **LAB-QUARTET-001**.

4.2 2. What you notice

Before catalogs and SKUs, notice what changes when the *body* of the computer changes.

On a desk clamshell, you notice sustained attention: two hands, a stable seat, a surface that invites long editing. Interruptions are often social or network interruptions, not “I put the computer in my pocket.”

On a hybrid handheld, you notice mode switching: walking versus sitting; touch versus docked keyboard; bright outdoor light versus indoor desk light; the way “connected” and “usable” can diverge when you move between networks.

On a learn-to-build form, you notice workflow thickness: terminals, logs, local builds, side-by-side references. The limiting resource may be screen real estate for inspection, not pocketability.

On a wearable or edge I/O path, you notice embodiment: sensors near skin, haptics, glanceable cues, and a sharper privacy boundary because the device may know where a body is and what it is doing.

Constraint contrast is the chapter’s central observation: the same human task can remain the “same” in intention while different technical conditions become the first ones to break.

You may also notice a temptation: to fill the empty measurement cells with impressive numbers. Resist it. Empty cells labeled **PHYSICAL_PENDING** are more scientific than invented confidence.

Optional commodity comparison (no Quartet required): take one task you already do—draft a paragraph, open a shared document, start a timer, check a calendar—and try it once at a desk keyboard and once on a phone. Do not invent Quartet scores from that trial. Simply notice which constraint domain moved first: input, mobility, attention, or network.

4.3 3. Exploded ecosystem

A learning laboratory is not a store shelf. Figure 4.1 shows the Quartet as four educational lenses. Treat the diagram as **conceptual**—representative educational architecture for teaching roles—not a photograph of validated manufactured units.

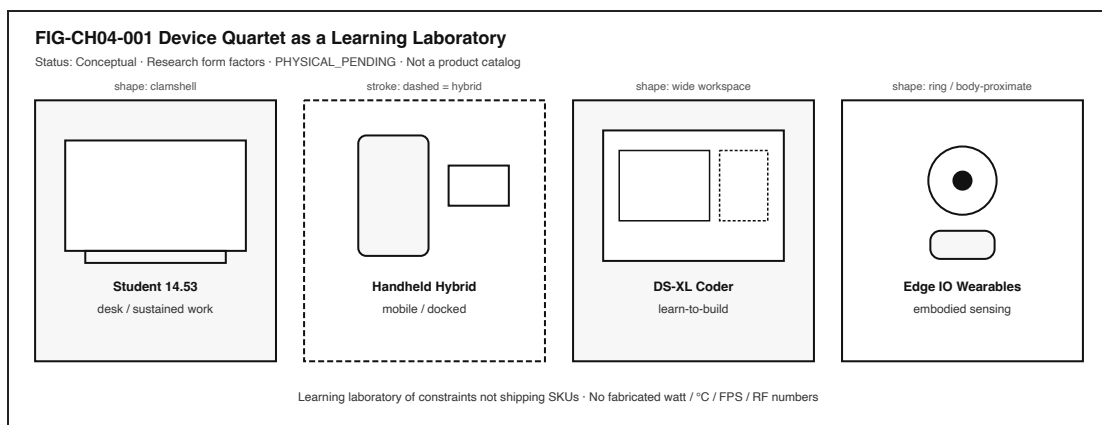


Figure 4.1: Four research form factors as a learning laboratory, not a product catalog.

4.3.1 Human

You bring intent, attention, posture, and consent. On wearables, consent is not a footer checkbox; it is part of the experience boundary.

4.3.2 Desk clamshell lens (Student 14.5-inch)

A sustained learning/work session form: keyboard-forward input, larger visual workspace, peripherals that can stay attached. The ecosystem emphasis is **stationary productivity** and long sessions (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

4.3.3 Mobile/docked lens (Handheld Hybrid)

A form that must remain useful while carried and become deeper when docked. The ecosystem emphasis is **mode transition**—not entertainment-only portability (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

4.3.4 Learn-to-build lens (DS-XL Coder)

A form that privileges local build-test-deploy learning: room to see code, logs, and references together. The ecosystem emphasis is **inspection bandwidth** for makers and students (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

4.3.5 Embodied sensing lens (Edge IO Wearables)

A form where sensing and interaction sit near the body. The ecosystem emphasis is **proximity, safety-critical interaction, and privacy**, not novelty accessories (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

4.3.6 Shared under-layers (every lens)

Across all four, familiar stacks still exist: input hardware, SoC/CPU/GPU/RAM/storage classes, radios when networked, batteries and power regulation, thermal paths, system software, applications, and optional network services (Patterson and Hennessy 2020b; Tanenbaum and Bos 2022). Chapter 2 taught you to follow one tap through those layers. Chapter 4 asks a different question: **which layers become the bottleneck when the enclosure and posture change?**

Publication machine-readable roles live in `devices/quartet.yaml` (Gunn 2026a). Concept Edition foreshadowing of the Quartet (CE-1) may have introduced the names; this chapter makes them a laboratory. Note carefully: Concept Edition module **CE-4** is about packets, access networks, edge, and cloud (later full-book chapters)—it is **not** a title match for Chapter 4.

4.4 4. Follow the signal

Figure 4.2 keeps one task in the center and fans constraint callouts outward. Read it as a logical story about *where pressure appears*, not as a measured EVT report.

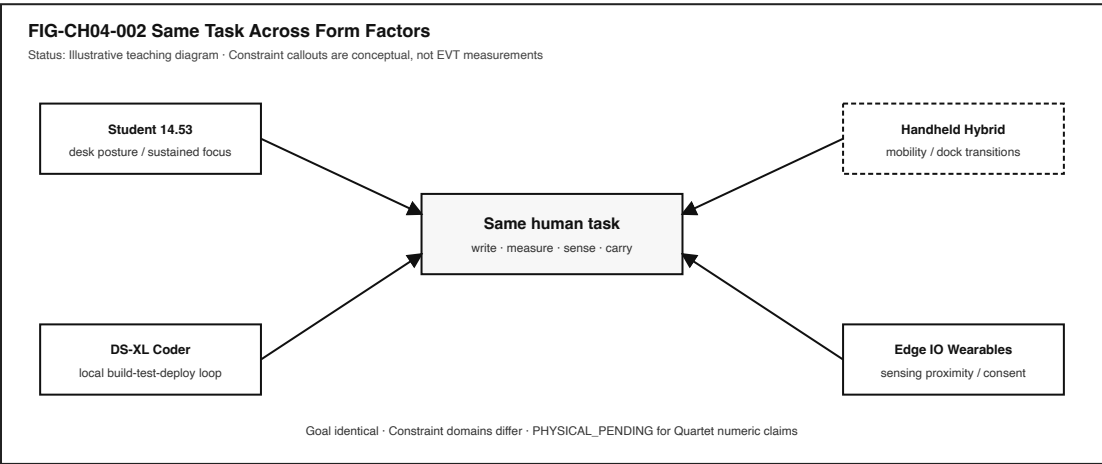


Figure 4.2: Same human task with different constraint callouts by form factor.

Walk one example task—“finish and submit a short write-up”—across the four lenses:

1. **Intent forms.** You decide the write-up must be completed.
2. **Posture selected.** Desk seat, walking commute, lab bench, or on-body glance.
3. **Input path engages.** Keyboard and pointer; touch; docked keyboard; or wearable confirmation plus companion screen.
4. **Working state loads.** Document opens; cache hit or miss; optional network sync.
5. **Attention budget appears.** Notifications, motion, ambient light, and multitasking compete differently by form.
6. **Compute and storage work.** Editing, autosave, local preview, optional compile if the “write-up” includes a small program on the coder lens.
7. **Power/thermal class becomes relevant qualitatively.** Desk sessions, pocket heat, sustained local builds, and skin-comfort budgets are different *classes* of constraint—even before any Wh or °C number exists.
8. **Network path (optional).** Sync, submit, or fetch references; “connected” may not mean “submitted.”
9. **Output and confirmation.** Screen text, haptic tap, audio cue, or docked display confirmation.
10. **Human judgment.** Done, stalled, or falsely marked done.

4.4.1 Alternate honesty paths

Path	What you are allowed to claim today
Published research roles	The four form factors exist as documented learning benchmarks (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026)
Commodity analogy	Your laptop/phone/tablet/band can stand in as <i>analogy devices</i> for labs
Measured commodity trial	Numbers you collect apply to <i>your</i> device and conditions only
Quartet EVT comparison	PHYSICAL_PENDING — do not invent

The signal to follow in this chapter is not a single interrupt line. It is the migration of the bottleneck as posture and enclosure change.

4.5 5. Component cards

These cards name laboratory roles and the failure domains they teach. They are not a bill of materials and not a price list.

4.5.1 Learning laboratory

Plain definition. A recurring set of form factors used to teach constraints by contrast.

Experience benefit. Readers practice systems thinking without waiting for a single “perfect” device.

Failure symptom. Treating the laboratory as a shopping list, or inventing measurements to make the lab feel “more real.”

4.5.2 Research form factor

Plain definition. An educational/design benchmark describing a class of device experience; not a claim that a finished commercial SKU has shipped.

Experience benefit. Shared vocabulary across chapters.

Failure symptom. Marketing language (“buy this”) replacing constraint language (“this posture stresses X”).

4.5.3 Student 14.5-inch

Plain definition. Clamshell-class research form factor for sustained desk learning and work (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Experience benefit. Teaches long-session compute, keyboard-first input, and desk ergonomics as system conditions.

Failure symptom. Assuming desk comfort equals universal usability on mobile or wearable paths.

4.5.4 Handheld Hybrid

Plain definition. Portable/hybrid research form factor emphasizing mobility and docked depth (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Experience benefit. Teaches mode transitions and carry constraints.

Failure symptom. Designing only for the docked mode, or only for the pocket mode, and calling either “the” product.

4.5.5 DS-XL Coder

Plain definition. Larger coding/display-oriented research form factor for local learn-to-build workflows (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Experience benefit. Teaches inspection bandwidth: seeing build, test, and logs together.

Failure symptom. Confusing “more screen” with “measured faster compiles” without evidence.

4.5.6 Edge IO Wearables

Plain definition. Wearable/edge I/O research form factor emphasizing embodied sensing, proximity, and safety-critical interaction (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Experience benefit. Teaches that some computing is body-adjacent, with sharper privacy and consent stakes.

Failure symptom. Treating wearables as optional gadgets, or capturing other people’s biometric streams “for the lab.”

4.5.7 Constraint contrast

Plain definition. Same task, different limiting conditions by form factor.

Experience benefit. Better failure-domain hypotheses (“mobility,” “input,” “sensing consent”) instead of one vague villain.

Failure symptom. Declaring a universal winner device from qualitative imagination alone.

4.5.8 PHYSICAL_PENDING

Plain definition. Physical fabrication/EVT evidence is not yet available; numeric hardware claims stay unmarked or explicitly pending.

Experience benefit. Protects scientific honesty and reader trust.

Failure symptom. Filling tables with plausible-looking invented wattage, thermals, FPS, RF, or latency figures.

4.6 6. Stability contract

The book’s **Stability Contract** still holds:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For the Quartet laboratory, learning-lab usability adds a publication rule:

Comparisons stay qualitative until EVT exists; readers are never blocked by missing Quartet hardware.

Concurrent conditions for a fair Chapter 4 learning experience include:

- the task goal is stated clearly,
- at least two form-factor lenses are compared,
- constraint language is used instead of invented SKU scores,
- commodity fallback routes remain available,
- privacy boundaries hold for embodied/sensing scenarios,
- accessibility alternatives are accepted as first-class input paths,
- observation is separated from inference,
- PHYSICAL_PENDING labels travel with any quantitative Quartet claim.

Figure 4.3 makes the matrix idea visible. Cells show **role classes**, not measured EVT values. Color is not required to read it; labels carry the meaning.

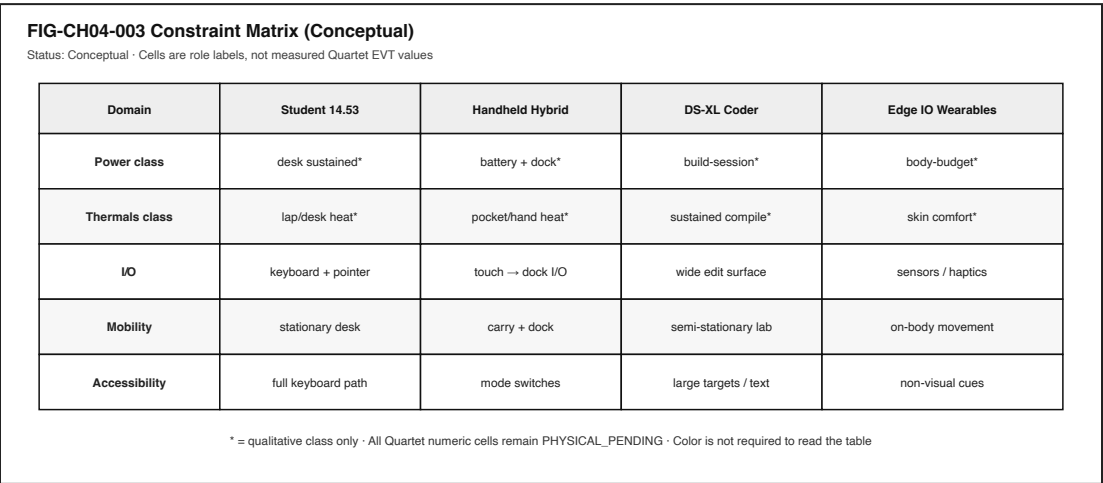


Figure 4.3: Conceptual constraint matrix across Quartet lenses.

Three separations matter here:

1. A desk session can feel “stable” while the same task on a handheld is already failing for mobility reasons.
2. A coder form can expose build/inspection failures that never appear in a short phone edit.
3. A wearable path can keep radios “fine” while consent or sensing proximity has already broken the human contract.

You experience the combined result. Blaming “the battery” or “the OS” without evidence collapses those domains. Section 11 practices better blame—and better restraint.

4.7 7. Try it

4.7.1 LAB-QUARTET-001 — Constraint contrast without Quartet hardware

Observable question. For one familiar task, which constraint domains change across desk, mobile/docked, learn-to-build, and embodied form-factor lenses—without inventing physical Device Quartet measurements?

WAIKE alignment note. WAIKE accepted `main` (audited SHA recorded in the Chapter 4 packet) offers adjacent product-charter, hardware-triage, and compatibility-matrix practice labs. It does **not** currently ship an exact Quartet physical module. **LAB-QUARTET-001** is therefore a **publication-owned** commodity/paper lab. Do not mint false WAIKE Quartet IDs.

Prerequisites. A text editor or spreadsheet; optional Python 3 for the blank CSV helper. No Quartet hardware. No purchase required.

Safety / privacy. No unsafe electrical, battery, RF, or thermal abuse. Do not capture passwords, tokens, private messages, or other people’s biometric streams. Prefer synthetic or public tasks.

Time estimate. About 45–90 minutes including write-up.

Equipment / software. None specialized (`equipment: []`). Fixtures live under `labs/LAB-QUARTET-001/`.

4.7.1.1 Prediction

Before filling the matrix, write one sentence: which Quartet lens do you expect to stress **mobility** most for your chosen task, and why?

4.7.1.2 Route A — Paper / digital matrix (baseline)

1. Choose one familiar task.
2. Open `fixtures/constraint_matrix_template.md`.
3. Fill qualitative cells only for the four lenses.
4. Leave Quartet numeric EVT fields empty or marked `PHYSICAL_PENDING`.

4.7.1.3 Route B — Commodity analogy (required companion)

1. Open `fixtures/commodity_analogy_card.md`.
2. Map two devices you already own onto two lenses by analogy.
3. Record what you **observed** vs what you only **inferred**.

4.7.1.4 Route C — Optional local helper

Run `python labs/LAB-QUARTET-001/local/matrix_sheet.py` to print a blank CSV skeleton. It invents nothing; it only prints headers.

4.7.1.5 Evidence (minimum)

- completed constraint matrix,
- observation vs inference paragraph,
- one-page `PHYSICAL_PENDING` measurement plan (what EVT evidence would be required later),

- teach-back paragraph.

4.7.1.6 Interpretation

Label claims:

- **Directly observed** on a commodity device you used,
- **Analogical** (mapped to a Quartet lens for learning),
- **PHYSICAL_PENDING** (would require Quartet fabrication/EVT).

4.7.1.7 Limits (say them out loud)

- Analogies are not product equivalence.
- One informal trial is not a benchmark.
- Qualitative class labels are not Wh, °C, FPS, dBm, or ms.
- Fixture completion is not Gate 3 PASS and not human full-manuscript validation.

4.7.1.8 Portfolio output

1. README.md (question, method, limits),
2. constraint matrix,
3. observation vs inference note,
4. PHYSICAL_PENDING plan,
5. teach-back,
6. scrubbed evidence note.

Completion means a claim is supported by an artifact—not that a command ran.

4.8 8. Build it

Use the same laboratory at the depth that matches your pathway.

4.8.1 Explorer

Name the four form factors and one constraint each exposes. Use ordinary language. No invented specs.

4.8.2 Operator

Map one familiar task onto two lenses. Predict different failure domains (for example, mobility vs inspection bandwidth). Check your prediction against a commodity trial.

4.8.3 Builder

Produce the comparison matrix: task $\times$ form factor $\times$ likely constraint. Keep cells qualitative unless you measured *your* commodity device. Document tradeoffs without purchase pressure.

4.8.4 Engineer

State which claims would require physical EVT evidence before quantitative comparison. Draft a measurement plan that lists instruments, revision identifiers, and honesty labels—without fabricating results.

4.8.5 Researcher

Design a controlled comparison protocol for a future date when physical units exist: hypotheses, variables, environment controls, uncertainty, and stop-rules against overclaim. Until then, keep status PHYSICAL_PENDING.

Educators can facilitate teach-backs from Section 11 and adapt LAB-QUARTET-001 for class-rooms that only have phones and shared laptops.

4.9 9. Secure and include it

4.9.1 Security

Form-factor changes can change the attack and mistake surface: docked sessions may trust peripherals differently; handhelds increase loss/theft exposure; wearables add on-body capture risk. Relevant ideas here include permissions, authentication at mode transitions, and not treating a haptic buzz as proof of a privileged action.

4.9.2 Privacy

Embodied sensing is not a toy layer. LAB-QUARTET-001 forbids capturing other people’s biometric or body-proximate streams. Prefer imagined constraints or your own consented commodity sensors, with scrubbed artifacts.

4.9.3 Accessibility

Not every learner uses the same input path. Desk keyboards, touch, switches, voice, and assistive pointers must remain first-class ways to complete the laboratory task. Comparison materials should not rely on color alone; Figure 4.1 through Figure 4.3 use shape and label encodings.

4.9.4 Equity

Readers must not be blocked by missing Quartet hardware or by purchase pressure. Commodity fallback is a requirement, not a consolation prize (CLM-CH04-004). Designing labs only for ideal desks silently excludes learners who work on phones, shared machines, or intermittent networks.

4.10 10. Career lens

Constraint-based laboratories show up in real work. No table promises employment; roles vary by organization.

Concern	Example role	Professional artifact	Lab resemblance
Form-factor strategy	Industrial designer	Research form-factor brief	Quartet role table without fake SKUs
Hardware systems	Hardware systems engineer	Constraint and interface notes	Matrix of I/O and thermal <i>classes</i>

Concern	Example role	Professional artifact	Lab resemblance
Product management	Product manager	Problem framing across postures	Same-task / different-failure write-up
Education design	Learning designer / educator	Lab that works without specialty gear	LAB-QUARTET-001 commodity routes
Accessibility	Accessibility specialist	Mode-equivalent interaction review	Keyboard/switch/voice acceptance
Privacy / trust	Privacy engineer	On-body sensing consent model	Wearable boundary rules in the lab
Validation	Test / EVT engineer	Revision-dated measurement package	PHYSICAL_PENDING plan (results empty until real)

When later chapters reuse these lenses—performance, sensors, edge AI, product design—they remain research/learning spines, not mascots and not fabricated shipping products (CLM-0003).

4.11 11. Check understanding

Answer with reasoning. Prefer short paragraphs over one-word guesses.

1. Why is a learning laboratory of four form factors more honest than a single “ideal student device” story?
2. Name one constraint each of Student 14.5-inch, Handheld Hybrid, DS-XL Coder, and Edge IO Wearables is meant to expose.
3. Why must Quartet battery, wattage, thermal, FPS, RF, and latency figures stay `PHYSICAL_PENDING` in this edition?
4. How can a commodity phone-and-laptop comparison teach Quartet ideas without claiming product equivalence?
5. Why might “the device is connected” be true while an embodied or mobile task has already failed for a person?
6. What evidence would you need before claiming one form factor is “faster” or “cooler” than another?
7. How should privacy rules change when a laboratory involves body-proximate sensing?
8. **Teach-back.** Explain the Device Quartet to a family member **without** using the words *EVT*, *SoC*, or *form factor*. Then introduce those three terms one at a time, tying each to something already understood.

Educator note: successful teach-backs show constraint contrast and the no-purchase rule, not memorized product slogans.

4.12 References

Selected sources for this chapter:

- Device Quartet research docs and manufacturing-readiness honesty labels (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026)
- Publication device index and learning-lab posture (Gunn 2026a)
- General computer organization vocabulary for shared under-layers (Patterson and Hennessy 2020b)
- Operating-system and systems vocabulary for software under-layers (Tanenbaum and Bos 2022)

Project-specific claim IDs (for example, CLM-0003, CLM-CH04-001-004) live in `evidence/claim_registry`. and the Chapter 4 claim plan. They are not substitutes for Gate 3 human reader evidence. Full-book human validation remains deferred until the working manuscript is complete; this chapter does **not** claim Gate 3 PASS.

4.13 12. Glossary links

Term	Role in this chapter
Device Quartet	Four research form factors used as a learning laboratory
Learning laboratory	Recurring constraint-teaching set, not a store shelf
Research form factor	Educational/design benchmark; not a shipping SKU claim
Student 14.5-inch	Desk / sustained-session lens
Handheld Hybrid	Mobile and docked lens
DS-XL Coder	Learn-to-build lens
Edge IO Wearables	Embodied sensing / body-proximate lens
Constraint contrast	Same task, different limiting conditions
PHYSICAL_PENDING	Physical/EVT evidence not yet available
Stability Contract	Experience depends on hidden conditions staying acceptable
Commodity fallback	Lab route that uses devices learners already own
Observation	What was directly seen or recorded
Inference	What is believed between observations
EVT	Engineering validation testing (evidence class—not claimed complete here)

Deeper entries and “not the same as” warnings belong in the living glossary network. Prefer following a link when stuck over inventing a private definition.

4.14 Figure references (embedded above; accessibility metadata)

All figures below are **conceptual** or **illustrative** unless a future revision cites a dated, revision-specific hardware evidence package.

4.14.1 FIG-CH04-001 — Device Quartet as a Learning Laboratory

- **Caption.** Four research form factors as a learning laboratory, not a product catalog.
- **Alt text.** Four panels: Student 14.5 desk; Handheld Hybrid mobile/docked; DS-XL Coder learn-to-build; Edge IO Wearables embodied sensing.
- **Status.** Conceptual.
- **Source.** Publication-owned original informed by research form-factor roles (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

4.14.2 FIG-CH04-002 — Same Task Across Form Factors

- **Caption.** One task with different constraint callouts by lens.
- **Alt text.** Center task node with four constraint branches.
- **Status.** Illustrative teaching diagram; not measured EVT.
- **Source.** Publication-owned original.

4.14.3 FIG-CH04-003 — Constraint Matrix (Conceptual)

- **Caption.** Power class, thermals class, I/O, mobility, and accessibility across four lenses.
- **Alt text.** Conceptual table; numeric Quartet EVT cells pending.
- **Status.** Conceptual; PHYSICAL_PENDING for quantitative Quartet claims.
- **Source.** Publication-owned original.

4.15 Claim footnotes used in this chapter (project-specific)

Claim ID	Approved gist	Classification
CLM-0003	Device Quartet defined as research form factors / learning benchmarks; PHYSICAL_PENDING	repository-documented research form factors
CLM-CH04-001	Quartet docs define research form factors / educational learning benchmarks	SOURCE_IDENTIFIED (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026) + devices/quartet.yaml)
CLM-CH04-002	Physical Quartet units and EVT measurements remain PHYSICAL_PENDING	PHYSICAL_PENDING
CLM-CH04-003	Different form factors expose different ecosystem constraints for the same task	ILLUSTRATIVE_ONLY teaching model
CLM-CH04-004	Chapter 4 labs must not require purchase or possession of Quartet hardware	publication policy / Wave-1 lab posture

General statements about CPUs, memory classes, and operating-system layers are treated as general technical knowledge and cited to textbooks where helpful. Quartet numeric hardware claims, when shown at all, must carry **PHYSICAL_PENDING**, **illustrative**, **measured (commodity)**, or **inferred** labels—and must never be fabricated.

End of Chapter 4 working draft manuscript.

Chapter 5

Chapter 5 — Electricity, Signals, Clocks, and Logic

Status: WORKING_DRAFT_COMPLETE · Chapter ID: CH05

Author: Edmund Gunn, Jr.

Part II: Hardware and electrical fundamentals (not a parts catalog)

5.1 1. The moment

You press a button—or touch a screen that *behaves* like a button. The control darkens. A light turns on. A menu opens. A lock clicks. Something you can see, hear, or feel has changed.

From your seat, it is tempting to credit the glass, the plastic, or the logo. Those surfaces matter for comfort and trust, but they are not where the decision lives. Underneath the ordinary moment, voltages change, a time-varying quantity is treated as information, a shared timing reference keeps order, and logic decides what should happen next.

This chapter’s governing question is simple:

How does an everyday device turn a human action into an electrical story that logic can decide—and turn that decision back into something a person notices?

Part II starts here on purpose. Later chapters name processors, memory, radios, and firmware. Those systems still rest on electricity, signals over time, clocks, and logic. Chapter 2 already followed a tap through a stack and lightly named sensors and interrupts; this chapter does **not** replay that story. It zooms under the stack into the physical substrate—without becoming an encyclopedia of every resistor and chip package.

If you remember only one sentence after the first reading, remember this: **glass shows; electricity and timed logic decide**. Everything else in Part II elaborates that sentence with more honest detail.

5.2 2. What you notice

Before formulas, notice the human contract you already enforce with frustration.

When you press, you expect recognition. When nothing happens, you may blame a “dead button,” a drained battery, a flaky cable, or “the software.” Those everyday diagnoses are not always wrong—but they collapse several domains into one complaint. A button that never changes voltage cannot deliver a signal. A signal that arrives while logic is confused about *when* to look may be ignored. Power that is present for a display and missing for an input controller can look like “the app froze” even though the glass still glows.

You also notice timing. A light that flips instantly feels different from a light that hesitates. A keyboard that repeats characters feels different from one that drops them. Those feelings are not decorations around engineering. They are the product, from the person’s point of view.

Devices move and decide using electrical signals and timed logic.

That sentence is enough for the Explorer pathway. Operators and Builders will add failure domains. Engineers will separate continuous physical quantities from discrete abstractions. Researchers will state what would be required to *measure* a real waveform instead of inventing one.

Optional comparison available on almost any device you already own: press a physical power or volume rocker, then tap an on-screen control. Both can produce a perceptible effect. The hardware paths differ. The shared idea does not: human action → physical change → interpreted signal → decision → effect.

5.3 3. Exploded ecosystem

Figure 5.1 is the first-minute map for this chapter: press → electrical change → signal → clocked logic → perceptible effect. It is **conceptual**—not a claim that any particular manufactured revision wires those stages the same way, and not a measured Device Quartet waveform. Device Quartet form factors used elsewhere in this series remain research/learning spines; physical fabrication and EVT electrical measurements stay **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

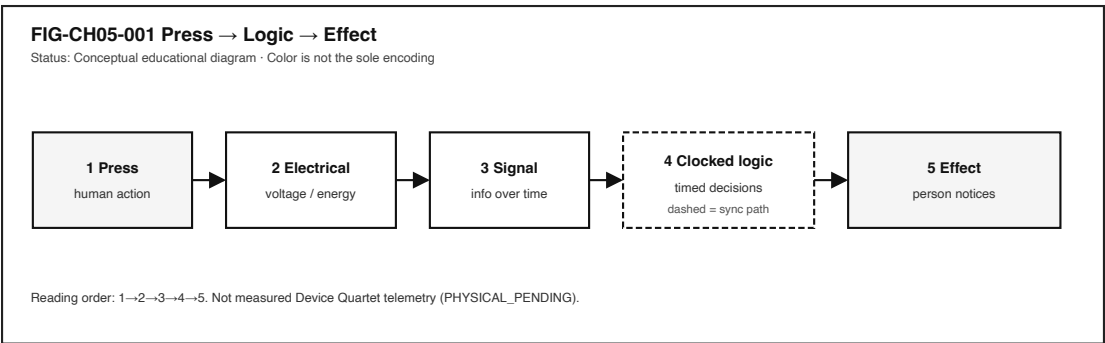


Figure 5.1: Causal flow from human press through electrical change, signal, clocked logic, and perceptible effect.

Walk the ecosystem in ordinary language. Keep the same layers when vocabulary deepens. Do **not** treat this as a bill of materials.

5.3.1 Human

You form intent: turn something on, open a menu, confirm a choice. Muscles move. Skin contacts a switch, a key, or a sensing surface. Later, eyes, ears, and hands judge whether the result matches what you meant.

5.3.2 Transducer / input interface

Something converts the action into an electrical change. A mechanical switch opens or closes a path. A touch-sensing layer alters capacitance or another measurable property. A microphone membrane moves. The exact physics differ; the educational role is the same: human action becomes a change the electronics can sense.

5.3.3 Electrical medium

Three ordinary electrical quantities stay distinct in this chapter. **Voltage** is a potential difference—the “pressure” that can drive charge when a path exists. **Current** is charge in motion along that path. **Power** is how fast energy is delivered or converted (qualitatively: when both voltage and current are present in a useful path, work can be done and heat can appear). You do not need a full circuits course to hold the separation: energy and control arrive as electrical quantities that must be present, limited, and safe enough for the intended parts. Later sections stay qualitative; this book will not invent precision meter readings for research hardware that has not been measured.

5.3.4 Signal

A **signal** is a physical quantity varying in time that carries information. The press is not the signal. The changing voltage (or related quantity) *is*—once a designer decides how to interpret it. Noise, bounce, and incomplete connections can corrupt that interpretation without erasing the human’s intent.

5.3.5 Clock (when the design is synchronous)

Many digital systems share a **clock**: a timing reference that sequences when values are sampled and updated. Not every useful circuit is synchronous; asynchronous and continuous analog paths exist. Still, for the digital behavior most people meet in phones, laptops, and controllers, a clock is the drumbeat that keeps decisions orderly (Harris and Harris 2021; Patterson and Hennessy 2020b).

5.3.6 Logic

Logic implements decision rules. Simple elements—often taught as gates realizing Boolean functions—combine into larger digital behavior: “if the button is down *and* the unlock condition is true, then release the latch” (Harris and Harris 2021). Software later names the same decisions as **if** statements; underneath, the substrate remains electrical interpretation against rules.

5.3.7 Effect

An actuator, display pixel path, speaker, motor driver, or status LED turns the decision into something perceptible. The person experiences the effect, not the intermediate abstractions.

5.3.8 Optional software boundary

Once digitized, the same story may continue as an input event delivered into an operating-system input subsystem and application handlers (The Linux Kernel Documentation 2026c). Chapter 2 already walked that upper path. Here, keep the boundary honest: software events rest on earlier electrical recognition.

5.4 4. Follow the signal

Read the following as a logical story, not as a claim that every device executes exactly one step at a time with no overlap.

1. **Intent.** You decide to press.
2. **Contact or proximity.** Skin meets a control surface, or a finger approaches a capacitive sensor.
3. **Transduction.** The interface changes an electrical quantity.
4. **Conditioning (often invisible).** Designers may debounce, filter, amplify, or threshold so that a messy physical change becomes a cleaner decision input. Commodity devices hide this; educational kits sometimes expose it.
5. **Interpretation as levels.** Continuous variation is often mapped to discrete levels—commonly taught as logic 0 and 1 bands with forbidden or uncertain regions between them (Harris and Harris 2021). Figure 5.2 compares continuous analog variation with digital level bands.
6. **Timing reference.** In synchronous designs, a clock edge (or related timing event) says *when* to trust a sample. Figure 5.3 is an **illustrative** timing sketch—no invented gigahertz claims, no Device Quartet scope captures.
7. **Logic decision.** Gates and sequential elements combine present inputs (and remembered state) into next outputs (Harris and Harris 2021). Figure 5.4 shows tiny Boolean blocks composing larger behavior.
8. **Downstream action.** The decision enables a driver, updates a register that software will read, or changes a display path.
9. **Human feedback.** Light, motion, sound, or haptic change closes the loop.

Failure branches are part of honesty:

- **No power in the right place.** The display may still glow while the input path is starved.
- **Signal not interpretable.** Bounce, noise, or a broken cable yields chatter or silence.
- **Timing violated.** The value changes while logic is sampling; the system may see a metastable or simply wrong bit—explained here qualitatively, without invented lab numbers.
- **Logic correct, effect blocked.** The decision happened; the actuator path did not.

Separating those domains is an Operator skill. Claiming root cause without evidence is not.

A useful habit is to narrate the same press twice: once as a person (“I pushed; the light should turn on”) and once as a path (“contact → electrical change → sample → decision → drive → light”). When the stories disagree, you have located a teaching moment instead of a vague

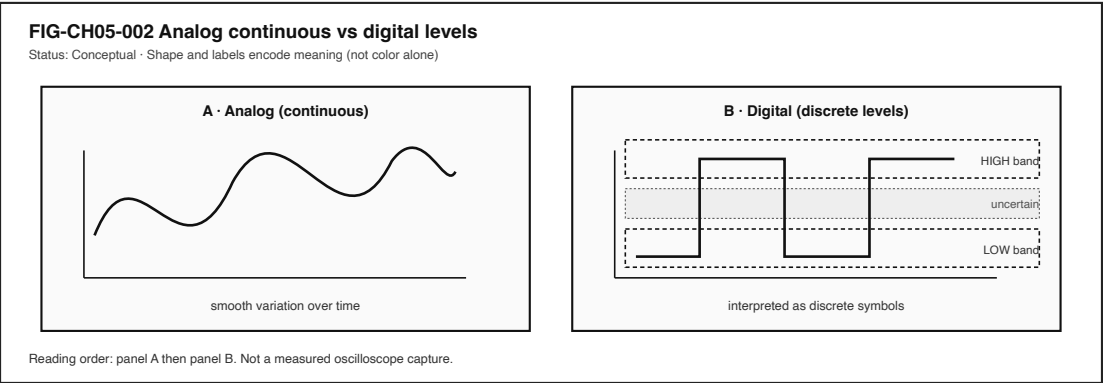


Figure 5.2: Side-by-side comparison of a continuous analog waveform and discrete digital logic levels.

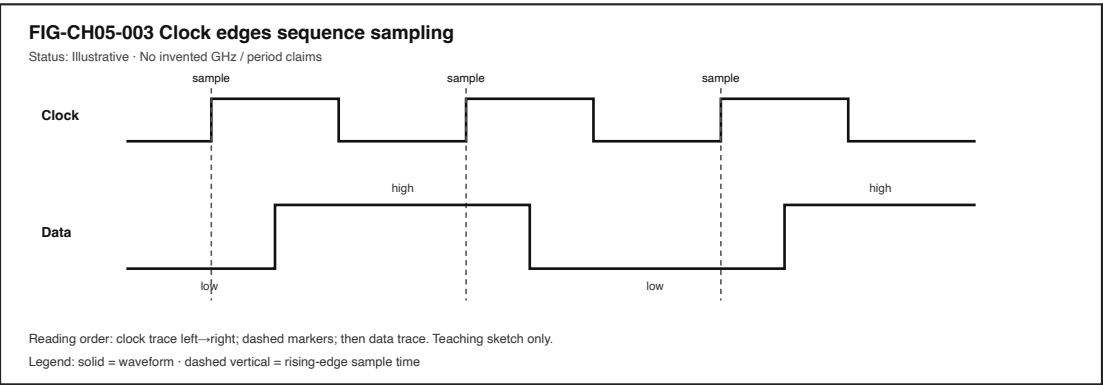


Figure 5.3: Illustrative clock edges sequencing sampling of a digital signal.

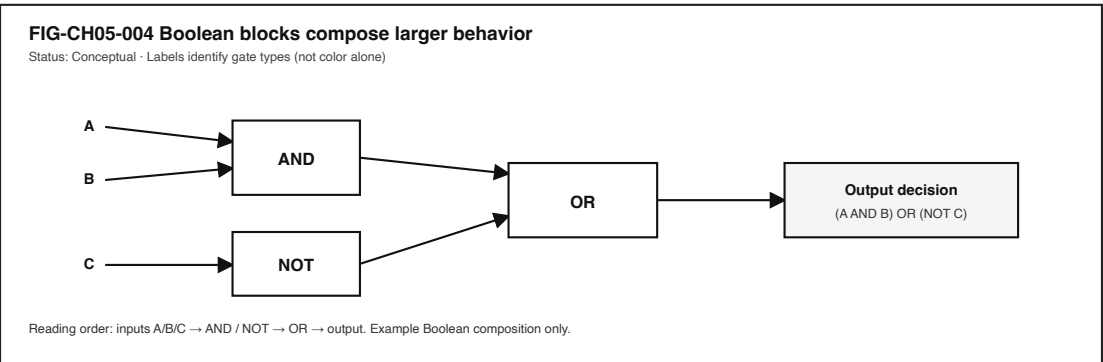


Figure 5.4: Small logic gates composing a larger Boolean decision path.

villain. Chapter 6 and later hardware chapters will add richer component vocabulary; they still depend on this path discipline. Do not replay Chapter 2’s full tap stack here—keep the zoom under the stack, on the substrate that makes later stacks possible.

5.5 5. Component cards

These cards are teaching tools, not a catalog of SKUs. If removing a name still leaves the human story intact, the name was trivia.

5.5.1 Electricity as useful energy and control

- **What it is.** Voltage (potential difference), current (charge flow), and power (energy delivery rate) used together to do work or convey control—three related quantities, not three names for one thing.
- **What it does.** Powers sensors, logic, radios, and actuators; carries many of the signals this chapter cares about.
- **When it works.** Enough energy arrives where it is needed, within limits the parts can tolerate.
- **When it fails.** Brownout, open path, short, or wrong domain powered—often experienced as “dead” behavior without a helpful error message.
- **Misconception to drop.** Seeing a glowing screen (some power domain alive) does not prove the input path has voltage, current, and power where the decision needs them.

5.5.2 Signal

- **What it is.** A time-varying physical quantity treated as information.
- **What it does.** Carries the press, the sensor reading, the bit stream, the clock itself.
- **When it works.** Variation stays within the interpreter’s expectations.
- **When it fails.** Noise, attenuation, disconnection, or mis-wiring corrupt meaning.

5.5.3 Analog versus digital

- **What it is.** Continuous physical quantities versus discrete symbolic levels (Harris and Harris 2021).
- **What it does.** Analog paths can track smooth change; digital abstractions tolerate some mess by snapping to levels.
- **Misconception to drop.** “Digital” does not mean “perfect” or “non-physical.” Digits still ride on voltages (or other media) that must be interpreted.

5.5.4 Clock

- **What it is.** A shared timing reference that sequences synchronous digital work (Harris and Harris 2021; Patterson and Hennessy 2020b).
- **What it does.** Coordinates when state updates, so cooperating parts agree on “now.”
- **Misconception to drop.** Not all electronics are clocked the same way; analog and asynchronous designs exist. A clock is not magic speed—it is agreement about time.

5.5.5 Logic gate / Boolean decision

- **What it is.** A simple decision element (AND, OR, NOT, and kin) realizing a Boolean function (Harris and Harris 2021).
- **What it does.** Builds larger digital behavior from small, composable rules.
- **Misconception to drop.** Software `if` statements are not a different universe; they are a higher description of decisions that must eventually become physical.

5.5.6 Noise and integrity (qualitative)

- **What it is.** Unwanted disturbance that can corrupt interpretation.
- **What it does.** Turns a clean story into chatter, dropouts, or wrong bits.
- **When teaching.** Stay qualitative here. Measuring integrity on a real board requires fixtures, probes, and stated methods—Researcher territory, not invented numbers.

5.6 6. Stability contract

The **Stability Contract** is a signature idea across this book:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For Chapter 5, a successful press-to-effect experience may require all of the following to stay “good enough” at once—stated **qualitatively**, with no fabricated thresholds:

- power present in the domains that sense, decide, and actuate,
- a signal that remains interpretable after transduction and conditioning,
- a timing reference stable enough for the intended synchronous logic (when the design relies on one),
- noise not overwhelming the interpretation margins,
- logic rules matching the designer’s intent (including remembered state),
- an effect path actually able to change something the person can notice,
- safety limits respected so “it worked once” does not become a hazard.

Three separations matter:

1. The interface can look **alive** while the input electrical path is already dead.
2. Logic can **decide correctly** while the actuator never receives drive.
3. A cable can be **mechanically seated** while the signal integrity is already insufficient.

You experience the combined result. Blaming “the button” or “the battery” without evidence collapses domains. Section 11 practices better blame.

Think of the Stability Contract as concurrent conditions, not a single score. Power can be “fine” in one domain and missing in another. A clock can be present yet unsuitable for the intended logic if it is unstable in ways the design cannot tolerate—stated here only as a qualitative warning, not as a numeric jitter budget. Noise can be harmless until it pushes a signal into the uncertain middle region on Figure 5.2. The contract fails when *any* required condition leaves the acceptable region for long enough that the person notices.

Device Quartet electrical measurements remain **PHYSICAL_PENDING**; do not treat conceptual figures as scope captures (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

5.7 7. Try it

5.7.1 LAB-SIG-001 — From Press to Logic

Observable question. Can I label a press-to-effect path—human action → transducer → signal → logic → effect—using a simulation, offline fixture, or commodity kit without unsafe electrical work?

WAIKE alignment note. WAIKE (accepted main audit SHA recorded in the chapter packet) includes adjacent `HARDWARE_ENGINEERING` labs such as logic-gate and RC-transient practice. Those are **competency neighbors**, not renamed false IDs for this publication lab. **LAB-SIG-001** is publication-owned.

Prerequisites. A computer you may use for learning; ability to open a local HTML/Markdown fixture or a browser-based logic simulation. Optional commodity LED/button learning kit only if your setting already allows supervised low-voltage educational use.

Safety (non-negotiable).

- Do **not** open mains-powered equipment, cut into power cords, defeat safety covers, or improvise high-voltage experiments.
- Do **not** abuse batteries (no crushing, heating, shorting, or “see what happens” modifications).
- Do **not** transmit on radio frequencies you are not authorized to use.
- Prefer **simulation and offline fixtures**. If a supervised educational kit is used, stay within the kit’s documented low-voltage intent and local lab rules.
- Do not capture passwords, tokens, or private identifiers in screenshots.

Time estimate. About 40–75 minutes for the baseline simulation/fixture route, including write-up.

Equipment / software. Baseline: browser or local viewer for the supplied fixture. Optional: a classroom logic simulator. Optional supervised kit is an extension, never a requirement.

No-specialized-hardware route. Route A below.

5.7.1.1 Prediction

Before you look at the fixture, write one sentence: where do you expect the story to fail first if the button “does nothing”—power, transduction, interpretation, timing, logic, or effect?

5.7.1.2 Route A — Simulation / offline fixture (baseline)

Open the LAB-SIG-001 fixture (see `labs/LAB-SIG-001/`). It presents a labeled path with stages you can mark as **observed**, **inferred**, or **not evidenced**.

Complete:

1. Name each stage in ordinary language.
2. Mark which stages the fixture lets you *observe* directly versus only *infer*.
3. Introduce one deliberate failure in the fixture’s controls (for example, “noise high,” “clock paused,” or “effect disconnected”) and record which human complaint it mimics.
4. Write two sentences separating observation from interpretation.

5.7.1.3 Route B — Commodity kit (optional, supervised only)

Only where policy allows: use a documented educational button→LED kit at the kit’s intended low voltage. Map the same five-stage path. Do not invent oscilloscope numbers you did not capture with a method you can state. If you cannot measure, say so.

5.7.1.4 Evidence to keep

- Your prediction sentence.
- A filled stage table (observed / inferred / not evidenced).
- One failure-mode note tied to a Stability Contract condition.
- A teach-back paragraph (Section 11).

5.7.1.5 Limits

One fixture run is not a characterization of a phone or a Quartet revision. Software timestamps and cartoon waveforms are not physical scope captures. Commodity kit results are *your* evidence for *that* kit—not universal device truth.

If your classroom cannot open HTML files, use the Markdown fixture and a printed stage table. The learning target is the labeled path and the observation/inference split—not a particular file format. Educators may substitute an equivalent simulator as long as safety rules stay intact and learners still produce the same artifacts.

5.8 8. Build it

Use the press-to-logic story at the depth that matches your pathway.

5.8.1 Explorer

Trace Figure 5.1 with a finger and retell it aloud without using the words *voltage*, *Boolean*, or *synchronous*. Then add those three words back, one at a time, tied to something already understood.

5.8.2 Operator

Given a familiar complaint—“dead button,” “flaky cable,” “random double presses”—list at least two electrical/logic domains that could produce the same human report. Do not claim root cause without a test.

5.8.3 Builder

Construct a labeled signal-path card: human action → transducer → signal → logic → effect. Annotate where software events might begin (The Linux Kernel Documentation 2026c). Keep the card free of SKU marketing language.

5.8.4 Engineer

Distinguish analog continuous quantities from digital abstractions on Figure 5.2. Name the clock as a sequencing reference on Figure 5.3, and state one reason a design might still include asynchronous or analog paths (Harris and Harris 2021).

5.8.5 Researcher

State what would be required to measure a real press-related waveform: probe points, ground reference, instrument class, sampling intent, and uncertainty notes. Keep Device Quartet claims **PHYSICAL_PENDING** until a dated evidence package exists (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Educators can facilitate teach-backs from Section 11 and adapt LAB-SIG-001 for classrooms that have only simulation access—equity of route is part of the design, not an afterthought.

5.9 9. Secure and include it

5.9.1 Electrical and lab safety

Curiosity about electricity is healthy; improvisation with mains power, damaged batteries, or unknown wiring is not. Publication labs for this chapter prefer simulation and fixtures precisely so learning does not require hazardous setups. Supervised educational kits remain optional extensions with documented limits.

5.9.2 Security of the decision path

A press may unlock a door, approve a payment, or send a message. Relevant ideas—without turning this chapter into a full security course—include:

- **trusted sensing** (is the press authentic to the intended control?),
- **debounce and filtering** as integrity helpers that can also be abused if poorly designed,
- **privilege** of the effect (what can this decision enable?),
- **no unauthorized RF** experiments while “testing signals.”

5.9.3 Privacy

Input timing and sensor traces can become sensitive when logged and correlated with identity. LAB-SIG-001 artifacts should use synthetic fixtures and scrubbed notes.

5.9.4 Equity

Not every learner has a hardware kit. The baseline route is simulation/fixture on ordinary computers. Kit routes must not be framed as the only “real” learning. Assessment should accept high-quality fixture reasoning.

5.9.5 Accessibility

Timing diagrams and logic figures must remain readable without color alone: labels, shapes, and stroke patterns carry meaning (Figure 5.2, Figure 5.3, Figure 5.4). Equivalent input paths—keyboard, switch control, voice, assistive pointers—still require the same underlying honesty: human intent must become an interpretable signal. Describe timing relationships in text equivalents, not only in pictures.

Inclusion also means pacing. Readers who are new to electricity should be able to finish Section 1–3 and LAB-SIG-001 Route A without being told they are “not technical enough.” Readers with prior circuits coursework should find enough precision in the Engineer/Researcher prompts

to stay honest—especially the ban on invented measurements. The chapter’s middle path is intentional: fundamentals tied to experience, not a dump of catalog pages.

5.10 10. Career lens

Electricity, signals, clocks, and logic cross many ownership domains. No table promises employment; roles vary by organization. LAB-SIG-001 artifacts resemble early professional evidence in miniature: clear stage labels, observation-versus-inference discipline, and safety-aware procedures.

Domain	Example role	Professional artifact	Lab resemblance
Circuits fundamentals	Electrical engineering student / technician	Annotated circuit explanation	Stage card with power and signal domains named
Digital design	Digital / FPGA engineer	Timing diagram + logic rationale	Clock-edge sketch with qualitative constraints
Embedded	Embedded systems engineer	GPIO / input contract notes	Builder path card including software boundary
Hardware test	Test engineer	Fixture procedure + uncertainty notes	Researcher measurement plan without invented data
Hardware bring-up	Board bring-up engineer	Power-rail and reset checklist	Stability Contract mapping for “dead board” complaints
Reliability	Hardware reliability engineer	Noise / integrity investigation plan	Failure-mode note tied to interpretation margins
Accessibility engineering	Accessibility engineer	Text equivalent for timing figures	Alt-text / reading-order check on chapter figures
Educator / mentor	Teacher, lab mentor	Scaffolded simulation lab	Facilitating Route A when kits are unavailable

When Device Quartet form factors appear nearby in the series, treat them as learning spines with **PHYSICAL_PENDING** electrical evidence—not as shipping SKUs and not as sources of invented measurements (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Career growth in this space often looks like better questions, not louder confidence: *Which domain failed? What did I observe? What would a fair test require?* Those questions travel from a classroom fixture to a bring-up lab to a field failure review.

5.11 11. Check understanding

Answer with reasoning. Prefer short paragraphs over one-word guesses.

1. Why is it incomplete to say “the screen did it” when a button press changes what you see?
2. What is the difference between a human press and a signal?
3. Why might a device look powered on while a particular button still does nothing?
4. In ordinary language, what job does a clock do in a synchronous digital design?
5. Why can analog and digital descriptions both be true of the same physical voltage over time?
6. Name two Stability Contract conditions for a press-to-effect experience and explain how each could fail independently.
7. What evidence would you need before blaming “noise” for a flaky button?
8. **Teach-back.** Explain electricity → signal → clock → logic → effect to a family member **without** using the words *Boolean*, *synchronous*, or *transducer*. Then introduce those three terms one at a time, tying each to something already understood.

Educator note: successful teach-backs show causal sequence and at least one failure branch, not a memorized parts list.

5.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Project-specific repository evidence for Device Quartet physical status remains labeled **PHYSICAL_PENDING** and is cited separately from textbook foundations.

Inline citations used in this chapter include Harris and Harris (2021), Patterson and Hennessy (2020b), The Linux Kernel Documentation (2026c), and “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026).

5.13 12. Glossary links

Term	Role in this chapter
Electricity (energy/control)	Medium for power and many control signals
Voltage	Potential difference that can drive charge when a path exists
Current	Charge in motion along a path
Power	Rate of energy delivery or conversion (not a synonym for voltage)
Signal	Time-varying physical quantity carrying information
Analog	Continuous physical representation
Digital	Discrete symbolic levels interpreted from physical media
Clock	Timing reference sequencing synchronous digital work
Logic / logic gate	Decision rules and Boolean building blocks
Boolean function	Truth mapping realized by digital logic

Term	Role in this chapter
Noise	Unwanted disturbance affecting interpretation
Signal integrity (qualitative)	Whether a signal remains interpretable
Transducer / input interface	Converts human/physical action into an electrical change
Debounce / conditioning	Making a messy press more interpretable (conceptual)
Stability Contract	Experience depends on hidden conditions staying acceptable
PHYSICAL_PENDING	Label for unmeasured Quartet/EVT electrical claims

Deeper entries, analogies labeled as analogies, and “not the same as” warnings belong in the glossary network. Prefer following a link when stuck over inventing a private definition.

5.14 Figure references (embedded above; accessibility metadata)

All figures below are **conceptual** or **illustrative** unless a future revision cites a dated, method-stated measurement package. Source preference: editable SVG in the publication repository.

5.14.1 FIG-CH05-001 — Press to Logic Flow

- **Caption.** Human press through electrical change, signal, clocked logic, and perceptible effect.
- **Alt text.** Left-to-right causal flow: Press; Electrical change; Signal; Clocked logic; Effect.
- **Text equivalent / reading order.** (1) Press → (2) Electrical change → (3) Signal → (4) Clocked logic → (5) Perceptible effect.
- **Status.** Conceptual educational diagram.
- **Source.** Publication-owned original.

5.14.2 FIG-CH05-002 — Analog versus Digital

- **Caption.** Continuous analog variation beside discrete digital level bands.
- **Alt text.** Two panels: smooth continuous waveform; stepped high/low digital levels with labeled bands.
- **Reading order.** Analog panel first, digital panel second; labels for high, low, and uncertain middle region.
- **Status.** Conceptual.
- **Source.** Publication-owned original.

5.14.3 FIG-CH05-003 — Clock Edges Sequencing Decisions

- **Caption.** Illustrative clock edges marking when a digital signal is sampled.
- **Alt text.** Timing diagram with a clock waveform and a data waveform; arrows at selected edges mark sample times.
- **Status.** Illustrative teaching sketch; not a measured frequency claim.
- **Source.** Publication-owned original.

5.14.4 FIG-CH05-004 — Logic Building Blocks

- **Caption.** Small Boolean gates composing a larger decision.
- **Alt text.** AND, OR, and NOT blocks feeding a labeled output decision.
- **Reading order.** Inputs left; gates center; composed output right.
- **Status.** Conceptual.
- **Source.** Publication-owned original.

Chapter 6

Chapter 6 — CPU, Instructions, and Parallel Work

Status: working_draft · Chapter ID: CH06

Author: Edmund Gunn, Jr.

Inheritance: CE-3 CPU / instruction / parallel slices (not a full OS encyclopedia)

Gate posture: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS)

6.1 1. The moment

You open a local app you already know—notes, a photo editor, an IDE, a spreadsheet—while other work is already running. The connectivity icon still looks fine. The window still draws. Scroll still tries. Yet the interface stutters, a save hangs a beat too long, fans spin up, or the chassis warms under your wrists.

From the seat it feels like “the CPU is bad,” or “this computer is dying,” or “something is broken.” Underneath, **instruction streams** are competing for limited processor time. An **operating system scheduler** is deciding who runs next. Some work is busy executing; some work is waiting. Extra cores may help—or may sit idle while a single dependent chain blocks the path your fingers care about.

This chapter stays with that inside-the-device story. It does not re-trace Chapter 2’s one-tap network path. It inherits Concept Edition CE-3’s compute focus and leaves memory hierarchy depth, file durability, and full OS internals for neighboring chapters. The governing question is ordinary and honest:

When my device feels slow while the network icon looks fine, what is the CPU actually doing—and why doesn’t “more cores” always fix the feeling?

6.2 2. What you notice

Before jargon, notice the human contract you already expect from local compute.

You expect typing and scrolling to stay responsive enough that the machine feels present. You expect a save or export to finish, or to fail in a way you can understand. You expect background downloads, sync, antivirus scans, or other apps not to erase the foreground for minutes without explanation. You expect the device not to become unusably hot for ordinary editing. You expect that a healthy Wi-Fi icon does not guarantee a smooth local experience—because local work was never only a network story.

Those expectations are not decorations. They *are* the product from the person’s point of view.

Responsiveness is a human perception produced by competing instruction streams under scheduling, memory, storage, and power constraints.

What you may notice, without opening a monitor yet:

- stuttering scroll or delayed key echoes,
- a spinning cursor while the window still redraws sometimes,
- fan noise or warmth during “simple” actions,
- one app lagging while another still feels snappy,
- recovery after you close heavy background work.

What those symptoms do **not** prove by themselves: a single root cause named “CPU.” High activity and poor feel can travel together or apart. A machine can feel sluggish while average CPU percent looks modest, if the interactive thread keeps missing its turn. It can also show a dramatic CPU spike during work that still finishes cleanly. Chapter 3 taught performance *feel*; here we name the compute machinery behind one family of causes—and practice separating observation from inference.

Optional comparison on almost any computer you already own: open a light text note and scroll. Then open a second heavy document, start a mild export or compile if you have one, and scroll the first note again. The first path often stays calm. The second often reveals contention. Write one sentence about what changed in feel before you look at any graph. That comparison is a teaching experience, not a universal benchmark—and not a Device Quartet EVT result.

6.3 3. Exploded ecosystem

Local lag is not a single object. It is a path through an inside-the-device ecosystem. Figure 6.1 is the first-minute map for this chapter: app experience → process/threads → scheduler → CPU cores → optional accelerator. Treat it as **conceptual / Representative educational architecture**—not a claim that any specific manufactured revision looks exactly like the diagram.

The Device Quartet used elsewhere in this series—Student 14.5-inch, Handheld Hybrid, DS-XL Coder, and Edge IO Wearables—are research form factors and learning benchmarks. Physical fabrication and EVT CPU measurements remain **PHYSICAL_PENDING**; do not treat comparison-matrix core counts or clock stories as shipping product facts (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Walk the layers in ordinary language.

6.3.1 Human

You form intent: scroll this page, export this image, save this file. Muscles move. Eyes and hands judge whether the result arrived soon enough to trust.

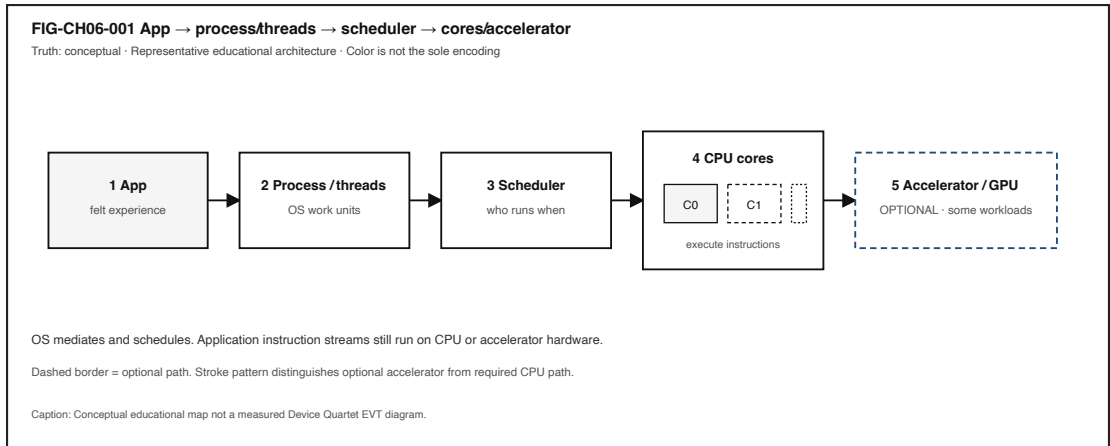


Figure 6.1: Conceptual map from app experience through process/threads and scheduler to CPU cores and optional accelerator.

6.3.2 Application

Your app holds state and code. When you act, software prepares work: update a document model, render a frame, encode media, search an index. That work will eventually become **instructions**—discrete operations a processor can execute (Patterson and Hennessy 2020a).

6.3.3 Process and threads

The operating system does not usually “become” your app. It creates and manages **processes** and **threads**: abstractions for concurrent work units with their own scheduling identity and (for processes) memory/address-space boundaries (Tanenbaum and Bos 2022). Your UI thread, a background save thread, and a helper process can all be “the app” from the seat while remaining distinct to the OS.

6.3.4 Scheduler and OS mediation

A **scheduler** decides which runnable thread obtains CPU time next (Tanenbaum and Bos 2022; The Linux Kernel Documentation 2026b). The OS mediates access to processors, memory, and devices. It provides services. It does **not** replace application computation: your logic still runs as instructions on hardware under that mediation (Tanenbaum and Bos 2022).

6.3.5 CPU cores

A **CPU** fetches, decodes, and executes instructions. Modern packages often expose multiple **cores**—independent execution engines on a chip that can progress different instruction streams when work is available (Patterson and Hennessy 2020a). Some cores also expose multiple hardware threads (survey depth only); that is not the same claim as “more software threads always feel faster.” “The CPU is at 90%” in a monitor is a coarse summary, not a biography of every instruction.

6.3.6 Accelerators (survey depth)

A **GPU** or other **accelerator** can execute specialized parallel workloads—graphics, some media, some ML kernels—while host software and OS/device interfaces still orchestrate submission and

sharing (Patterson and Hennessy 2020a; “Vulkan Overview” 2026). Accelerators are not mystical second brains; they are optional helpers for suited work.

6.3.7 Adjacent layers (named, not encyclopedized)

Working **memory**, **storage** I/O, and **power/thermal** policy sit beside compute. They can starve or throttle the same human moment. Chapter 7 and Chapter 9 deepen those layers; CE-3 already warned that local lag with a healthy network icon is a multi-domain hypothesis set, not a single villain.

6.4 4. Follow the signal

Follow one ordinary local action—say, scrolling a document while another export runs—as a logical story. Platforms differ; the teaching sequence is orientation, not brand loyalty.

1. **Intent becomes software work.** A scroll or key event reaches the application’s event path (Chapter 2 named that arrival). The app decides what to recompute or redraw.
2. **Work becomes runnable.** One or more threads become **runnable**: ready to execute if a core is free (Tanenbaum and Bos 2022).
3. **Scheduler chooses.** The OS scheduler selects among runnable entities on available cores (The Linux Kernel Documentation 2026b; Tanenbaum and Bos 2022). Interactive threads can be delayed when contention is high—even if they are “important” from your seat.
4. **Instructions execute.** On a chosen core, the CPU advances the instruction stream: arithmetic, branches, memory references, and calls into libraries or system services (Patterson and Hennessy 2020a).
5. **Waits appear.** A thread may stop being runnable while it waits for memory, storage, a lock, or a reply from another thread. Busy and waiting can both feel like “frozen” from the seat.
6. **Optional offload.** Some frames or kernels may be submitted to an accelerator; completion still has to rejoin the UI story (“Vulkan Overview” 2026).
7. **Feedback returns.** Pixels update, a progress indicator moves, or the cursor finally stops spinning. Your nervous system judges the combined timeline.

Figure 6.2 contrasts a mostly serial dependency chain with work that splits into independent chunks. Extra cores help the second pattern more than the first. The bar lengths are **illustrative** teaching aids—not measured speedups, IPC claims, or Device Quartet EVT results.

6.4.1 Parallelism without fairy tales

Computer organization teaching has long stressed that useful speedup depends on how much work can actually overlap, and on other bottlenecks that remain serial (Patterson and Hennessy 2020a). In plain language for this book:

- **More cores can improve throughput** when independent work is ready.
- **More cores do not guarantee lower latency** for every click, scroll, or save.
- Contention, waiting, thermal limits, and single-threaded critical paths can erase the brochure promise.

A practical seat-test: if one thread must finish step A before step B can start, a second core cannot teleport B into the past. If three photo thumbnails truly need independent decoding, three cores may finish the set sooner—until storage or memory bandwidth becomes the new line

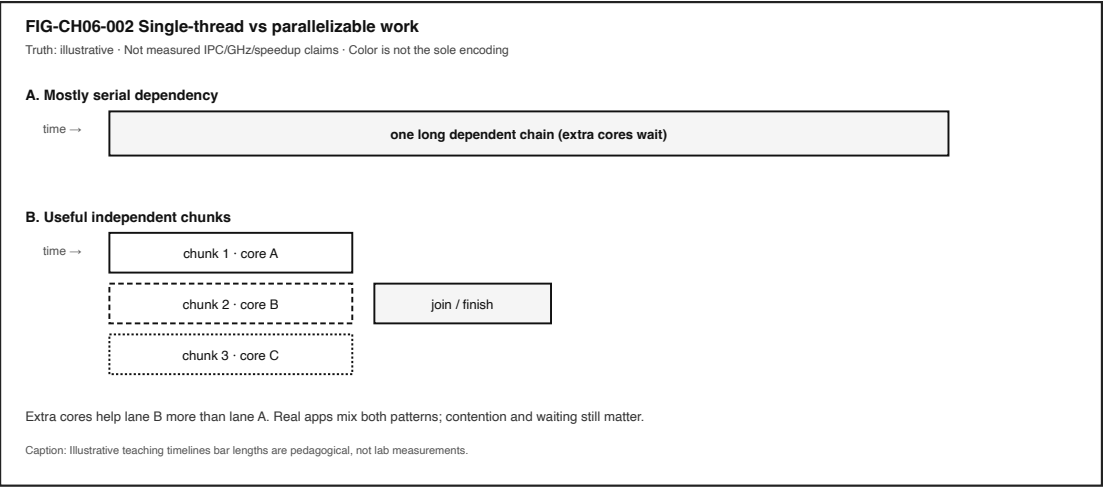


Figure 6.2: Illustrative timelines comparing a serial dependency chain with parallelizable chunks across cores.

at the door. Naming that shift is systems literacy; inventing a percentage speedup without a measurement bundle is not.

That is the CE-3 inheritance this chapter keeps central: parallel hardware is real; universal UX speedup is not. Full-book Chapter 12 will deepen OS policy; Chapter 7 will deepen hierarchy waits. Here we keep the compute promise small enough to inspect.

6.4.2 Honesty branches

- A UI can redraw while a background export still holds locks or fills disks.
- A monitor can show high CPU % while the interactive thread is mostly waiting.
- Closing a window may stop some work and leave other helper processes running.
- Airplane mode can rule out the network for a *local* workload without proving the CPU is healthy.

6.5 5. Component cards

These cards are a first toolkit—not a bill of materials for every SoC.

6.5.1 Instruction

Plain definition. A discrete operation a CPU (or similar processor) can execute.

Experience benefit. Software progress becomes physical work on silicon.

Failure symptom. Endless busy-work, livelock feelings, or progress that never reaches the UI.

6.5.2 CPU

Plain definition. Hardware that fetches, decodes, and executes instructions.

Experience benefit. General application logic can run.

Failure symptom. Saturation, contention, or thermal/power limits that shrink available performance (The Linux Kernel Documentation 2026a).

6.5.3 Process / thread

Plain definition. OS abstractions for concurrent work: a **process** typically owns an address-space boundary; a **thread** is a schedulable execution path inside a process (often what the scheduler actually picks next) (Tanenbaum and Bos 2022).

Experience benefit. Multiple activities can progress without being one single frozen program counter from the person’s point of view.

Failure symptom. Too many competing units; a stuck UI thread; unclear which process owns the lag.

Not the same as. A hardware **core** (execution engine). Extra cores help only when useful work is ready to overlap.

6.5.4 Scheduler

Plain definition. OS mechanism that allocates processor time among runnable work (The Linux Kernel Documentation 2026b; Tanenbaum and Bos 2022).

Experience benefit. Interactive work can share the machine with background tasks.

Failure symptom. Foreground starvation; fair-looking averages that still feel awful in the seat.

6.5.5 Parallel work

Plain definition. Overlapping execution across cores or accelerators when the workload allows (Patterson and Hennessy 2020a).

Experience benefit. Throughput rises for suited tasks (exports, builds, some renders).

Failure symptom. “I bought more cores and my typing still stutters”—often a serial or waiting path.

6.5.6 Accelerator / GPU (survey)

Plain definition. Specialized hardware for some parallel workloads, orchestrated by host software (Patterson and Hennessy 2020a; “Vulkan Overview” 2026).

Experience benefit. Frames, filters, or kernels finish without blocking every CPU core the same way.

Failure symptom. Submission stalls, driver waits, or assuming “GPU” means games only.

6.5.7 CPU-bound vs waiting

Plain definition. Busy executing useful instructions versus blocked on memory, I/O, locks, or scheduling.

Experience benefit. Diagnosis language that prevents buying the wrong upgrade.

Failure symptom. Treating a single CPU % number as a courtroom verdict (**illustrative teaching caution**—see Section 11 and LAB-CMS-001).

6.6 6. Stability contract

The **Stability Contract** is a signature idea across this book:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For Chapter 6, a smooth local interactive experience may require all of the following to stay “good enough” at once—qualitative bounds only; no invented numeric budgets:

- interactive threads become runnable and obtain CPU (or needed accelerator) time,
- competing background work stays bounded enough not to erase the foreground,
- needed memory and storage paths do not turn every action into a wait storm (deepened in Chapter 7),
- the scheduler continues to serve the interactive path under contention (The Linux Kernel Documentation 2026b),
- power/thermal policy still permits adequate performance rather than silently collapsing clocks (The Linux Kernel Documentation 2026a),
- the person can still tell progress from failure.

A device can remain **powered on and connected** while the **human experience has already failed**.

Figure 6.3 separates **CPU-bound** patterns from **waiting** patterns as conceptual diagnosis branches. High CPU percent in a monitor is not always proof the CPU is the root cause of poor feel—that caution is an **illustrative teaching claim** for lab practice, not a fleet-wide measured distribution.

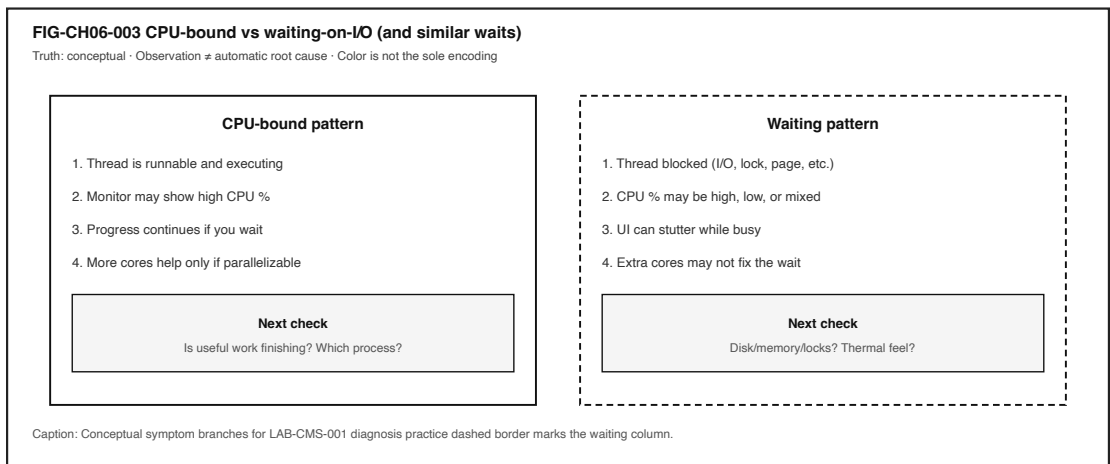


Figure 6.3: Conceptual comparison of CPU-bound versus waiting symptom checks.

Three separations matter here:

1. **Busy executing** versus **busy waiting** can look similar in the seat.
2. **More cores** versus **more useful parallel work** are different purchases.
3. **Healthy network icon** versus **healthy local Stability Contract** are different stories.

Commodity observations you collect in **LAB-CMS-001** are *your* evidence for *your* device and session—not universal EVT curves, and not Gate 3 reader validation.

6.7 7. Try it

6.7.1 LAB-CMS-001 — Make Local Slowness Visible

Observable question. When a familiar local app feels slow but the connectivity icon looks fine, what evidence can I gather—using only commodity tools—to separate **CPU**, **memory**, **storage**, and **scheduling/thermal** hypotheses?

This chapter’s Try It **inherits and links** the publication-owned CE-3 lab rather than inventing a duplicate CPU-focused lab package. Follow the full lab packet at `labs/LAB-CMS-001/` (README, routes, fixtures, portfolio templates). Focus your write-up on the **CPU / scheduler / parallel-work** columns of the diagnosis; neighboring chapters reuse the same lab for memory and storage emphasis.

WAIKE alignment note. WAIKE accepted `main` (audit SHA recorded in the CH06 packet) hosts adjacent labs on observability and “who runs when,” but there is **no** exact WAIKE module ID for this publication lab. Do not mint one.

Prerequisites. A computer you may use for learning; built-in OS monitor (Task Manager / Activity Monitor / `top` or equivalent); a familiar local app.

Safety.

- Do not capture personal document contents; redact filenames before sharing.
- Do not install “optimizer” tools, disable security software, or load kernel modules.
- Use **mild** load only. **Stop** if the device warns about heat or becomes uncomfortably hot.
- No Device Quartet hardware is required. No invented EVT numbers.

Time estimate. About 45 minutes for Explorer + Operator baseline.

6.7.1.1 Prediction

Before measuring, write which hidden part you expect to dominate during your controlled action: CPU activity, memory pressure, storage I/O, or scheduling/thermal contention.

6.7.1.2 Route A — Commodity OS monitor (baseline)

1. Record **before** CPU, memory, and disk/storage activity.
2. Perform one controlled local action (open a medium document, scroll, or save).
3. Record **during** readings and wall-clock feel.
4. Fill observation vs inference columns in the lab portfolio table.

6.7.1.3 Route B — Safe CLI snapshot (optional)

```
python3 labs/LAB-CMS-001/local_app/safe_snapshot.py
```

Read-only sampling where the OS exposes coarse stats. If a metric is unavailable, the script reports `unavailable`—never invent hardware claims.

6.7.1.4 Route C — Fixture fallback

Use `labs/LAB-CMS-001/fixtures/` when monitors are inaccessible or you must avoid personal screenshots. Fixtures teach concepts; they are not claims about your personal device.

6.7.1.5 Evidence (minimum)

- observation table,
- two snapshots or fixture IDs with timestamps,
- teach-back: “OS schedules; apps still compute,” plus one sentence on when more cores might not help.

6.7.1.6 Interpretation labels

Observation (allowed)	Inference (must label)
CPU % rose from A to B	“CPU-bound root cause”
Fans spun; device felt warmer	“Thermal throttle at X °C”
Scroll stuttered under extra load	“Need N more cores”

6.7.1.7 Limits (say them out loud)

- OS monitors are coarse; they are not silicon performance-counter labs.
- One run is not a benchmark.
- This lab does not close Gate 3 and does not validate Device Quartet EVT CPU claims (**PHYSICAL_PENDING**).

6.8 8. Build it

Use the same local-lag story at the depth that matches your pathway. Prefer commodity tools and LAB-CMS-001 artifacts.

6.8.1 Explorer

Draw Figure 6.1 from memory with three labels: process/thread, scheduler, CPU. Teach a nontechnical person: “The OS decides who runs; the app’s work still happens as instructions.”

6.8.2 Operator

Capture before/during monitor snapshots under idle vs mild load. Mark each row observation or inference. Note whether connectivity looked fine the whole time.

6.8.3 Builder

Change **one** variable (extra documents, a background export, fewer tabs). Re-run the observation table. Produce a process map: which processes/threads appeared, which core activity moved, what you did *not* measure.

6.8.4 Engineer

Order a diagnosis plan: connectivity rule-out (if the workload is local) → CPU activity → memory → disk → qualitative thermal/power. State what additional evidence would be required before claiming causation. Distinguish CPU-bound from waiting using Figure 6.3 language without inventing counter values.

6.8.5 Researcher

Propose a controlled load comparison: hypothesis, variables, planned runs, confounders (thermal state, battery mode, other processes), and uncertainty. Report that Device Quartet physical CPU claims remain **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Do not invent IPC, GHz product claims, or Gate 3 PASS language.

Educators can facilitate misconception probes from Section 11 and use LAB-CMS-001 fixtures for learners without admin rights.

6.9 9. Secure and include it

6.9.1 Security

Process isolation is an intent you can already name at survey depth: separate processes limit how much damage one buggy or malicious program can do to another’s memory and credentials (Tanenbaum and Bos 2022). Scheduling and privilege boundaries are part of that story. This chapter does not teach exploitation—only the literacy that “apps share a CPU” is not the same claim as “apps share all secrets.”

6.9.2 Privacy

Monitor screenshots can leak filenames, thumbnails, account names, and open document titles. Redact before sharing. Prefer fixtures when portfolio evidence must leave the device. LAB-CMS-001 forbids capturing personal document contents as a requirement.

6.9.3 Accessibility

Activity visualizations and color-coded monitor graphs are not equally readable. Prefer labeled columns, patterns, and text tables. Keyboard-reachable OS monitors matter; fixture transcripts exist when GUIs are hostile. Dictate observation tables if needed. Do not make “see the red spike” the only teaching channel—Figure 6.1 through Figure 6.3 encode meaning with shape, order, and stroke pattern as well as text.

6.9.4 Equity

Not every learner has a new multi-core laptop, admin rights, or a quiet thermal environment. Commodity routes and offline fixtures are the baseline. Device Quartet form factors are learning analogies here, not admission tickets (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Avoid shaming older hardware; teach diagnosis that works with the machine in front of you.

Shared family or school machines raise another equity edge: “just open Activity Monitor” may require permissions a learner does not hold. Fixture transcripts and verbal observation tables keep the pathway open without forcing privilege escalation. The Stability Contract fails for people first; pedagogy should not add a second failure by assuming identical tools.

6.10 10. Career lens

Literacy from this chapter shows up in several neighboring roles—without employment promises:

- **Performance engineering / SRE-adjacent local diagnosis** — separate CPU-bound from waiting; treat monitors as evidence, not folklore.
- **Operating systems and embedded** — processes, threads, schedulers, and interrupt-time vs process-time thinking.
- **Compiler-lite literacy** — instructions and parallelizable work as something compilers and runtimes try to expose to hardware (Patterson and Hennessy 2020a).
- **Graphics / media systems** — when accelerators help and when host orchestration still dominates (“Vulkan Overview” 2026).
- **Educator / mentor** — teach observation vs inference so people stop buying the wrong upgrade story.

Day-one habits transfer across those roles: write a prediction before you open a monitor; label every causal sentence; refuse product-brochure numbers without a method. Power and frequency scaling policies can shrink available performance to protect the device (The Linux Kernel Documentation 2026a)—so “the CPU got worse overnight” sometimes means policy and heat, not a mysterious hardware betrayal.

Career growth here means better questions: “Is this runnable work, waiting, or thermal policy?”—not a guaranteed job title.

6.11 11. Check understanding

6.11.1 Misconception probes

1. **“More CPU cores always make everything faster.”**
Counter: useful overlap and non-CPU bottlenecks decide; serial and waiting paths remain (Patterson and Hennessy 2020a).
2. **“The operating system is doing my app’s compute for me.”**
Counter: at CPU depth, keep instruction streams and parallel work visible; Chapter 12 deepens what the OS actually schedules versus what the app still computes (Tanenbaum and Bos 2022).
3. **“If Wi-Fi is connected, lag must be the network.”**
Counter: local contention can fail the Stability Contract while the icon stays polite (LAB-CMS-001 hypothesis set; illustrative until *you* measure).
4. **“High CPU % always means the CPU is the root cause.”**
Counter: waiting, sampling artifacts, and mixed threads confuse percent alone—label inferences (illustrative teaching caution).
5. **“GPUs only matter for games.”**
Counter: accelerators appear in UI composition, media, and other parallel kernels (Patterson and Hennessy 2020a).
6. **“Device Quartet core counts are measured shipping specs.”**
Counter: research form factors; physical EVT remains **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

6.11.2 Teach-back (say it out loud)

Programs become instruction streams. The CPU executes them. The OS decides who runs when. Extra cores help when work can overlap usefully. A healthy network icon does not prove local compute is fine.

6.11.3 Pathway self-check

Pathway	You can...
Explorer	Name instruction, CPU, process/thread, scheduler in one local lag story.
Operator	Capture before/during CPU activity and separate observation from inference.
Builder	Map one experience to process/threads/CPU (optional accelerator) without claiming root cause.
Engineer	Distinguish CPU-bound vs waiting; explain why more cores are not universal speedups.
Researcher	Propose a controlled load comparison; report uncertainty; avoid invented IPC/GHz claims.

6.12 References

Inline citations used in this chapter include Tanenbaum and Bos (2022), Patterson and Hennessy (2020a), The Linux Kernel Documentation (2026b), The Linux Kernel Documentation (2026a), “Vulkan Overview” (2026), and “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026).

CE-3 preproduction packets (`publication/preproduction/ce-03/`) supply the inherited claim and stability wording this chapter adapts. Full bibliography entries live in the active Quarto project bibliography (including `publication/full131/WORKING_BIBLIOGRAPHY.bib`).

6.13 12. Glossary links

Term	Plain link
instruction	Discrete CPU-executable operation
CPU	Hardware that executes instructions
process / thread	OS abstractions for concurrent work
scheduler	OS decision of what runs when
parallel work	Overlapping execution when the workload allows
accelerator	Specialized hardware for some workloads
CPU-bound vs waiting	Busy executing vs blocked on other resources
Stability Contract	Experience depends on hidden conditions staying acceptable

See also Chapter 2 (process/thread/scheduler naming), Chapter 3 (performance feel), Chapter 7 (memory/storage), Chapter 9 (power/thermal), and Chapter 12 (OS depth).

6.14 Figure references (embedded above; accessibility metadata)

6.14.1 FIG-CH06-001 — App → process/threads → scheduler → cores/accelerator

- **File:** figures/architecture/fig-ch06-001-app-to-cores.svg
- **Truth:** conceptual
- **A11y:** figures/preproduction/accessibility/fig-ch06-001.yaml

6.14.2 FIG-CH06-002 — Single-thread vs parallelizable work

- **File:** figures/architecture/fig-ch06-002-serial-vs-parallel.svg
- **Truth:** illustrative
- **A11y:** figures/preproduction/accessibility/fig-ch06-002.yaml

6.14.3 FIG-CH06-003 — CPU-bound vs waiting-on-I/O symptoms

- **File:** figures/architecture/fig-ch06-003-cpu-bound-vs-waiting.svg
- **Truth:** conceptual
- **A11y:** figures/preproduction/accessibility/fig-ch06-003.yaml

Related CE-3 maps (optional cross-read, not required embeds): figures/preproduction/ce-03/fig-ce-fig-ce3-003.svg.

6.15 Claim footnotes used in this chapter

Claim	Status handling in prose
CLM-CH06-001 — apps execute as instructions under OS mediation	Taught with Tanenbaum and Bos (2022) (and CE-3 CLM-CE3-001 inheritance)
CLM-CH06-002 — more cores help only when work parallelizes usefully	Taught via CE-3 CLM-CE3-005 inheritance with Patterson and Hennessy (2020a) (no invented sources)
CLM-CH06-003 — high CPU % automatic root cause	Framed illustrative + LAB-CMS-001 observation/inference practice
CLM-CH06-004 — Quartet CPU/core EVT claims	PHYSICAL_PENDING with “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026)

Explicit non-claims: Gate 3 PASS; measured Device Quartet CPU EVT; invented IPC/GHz product numbers; a minted WAIKE CPU lab ID as if published.

Chapter 7

Chapter 7 — Memory, Cache, and Storage

Status: draft · Chapter ID: CH07

Author: Edmund Gunn, Jr.

7.1 1. The moment

You reopen an app you already know—or a large file you edited yesterday. Sometimes the window returns as if it never left. Sometimes a spinner spins long enough that you notice. Sometimes the drive (or fan) makes a thrash-sound, the pointer hitch-steps, and the app reloads as if starting over.

Glance at the network icon. It may still look fine: connected bars, a calm Wi-Fi glyph, no angry offline badge. From the seat, the temptation is automatic—*the internet must be slow*—even when the work you are waiting on never left the device.

This chapter stays with that ordinary moment until hierarchy and persistence become visible. The governing questions are simple and honest:

Why does “memory” mean different things in everyday speech?

Why is RAM not the same as storage?

How do hierarchy misses and storage I/O change how technology *feels*?

Chapter 6 names the local compute machine. This chapter deepens the **memory hierarchy** and the **durability** story without becoming a filesystem or database encyclopedia (that depth belongs later, especially Chapter 13). Concept Edition module CE-3 supplies the shared teaching spine—local lag with a healthy connectivity icon—and the commodity lab you will run inherits as **LAB-CMS-001**.

7.2 2. What you notice

Before jargon like *working set* or *write-back*, notice the human contracts you already expect.

When you open a familiar file, you expect it to return. When you edit, you expect the screen to keep up with your hands. When you save, you expect the work to still be there after quit or restart. When something stalls, you expect a progress cue honest enough that you know whether to wait, cancel, or worry. You also expect the device not to become unusably warm for something that “should be simple,” and you expect a healthy-looking network icon not to gaslight you when the real wait is local.

Those expectations collide with ambiguous everyday language. People say *memory* for RAM stickers on a store shelf, for “how much storage the phone has,” for remembering a password, and for “the app is using too much memory.” The words collide. The layers do not.

What you feel is often a hierarchy miss or a durability wait dressed up as “the computer is bad.”

Optional comparison on a device you already own: open a small text note that is already warm in the app, then open a large local document after a fresh launch. The first often feels like the data was nearby. The second may pay a longer path from durable storage into working memory—even while Wi-Fi still smiles.

7.3 3. Exploded ecosystem

A reopen or open-file moment is not a single object. It is a path through cooperating layers. Figure 7.1 is the first-minute map of those layers as a **memory hierarchy**: registers, cache, RAM (main memory), and storage. It is **conceptual**—a teaching architecture, not a claim that your particular laptop silicon matches the diagram’s geometry.

Walk the ecosystem in ordinary language, then keep the same layers when vocabulary deepens.

7.3.1 Human

You form intent: reopen this project, open that photo, save these notes. Eyes and hands judge whether the result matches what you meant—and how long the wait felt.

7.3.2 Application and processes

The app holds on-screen state, open documents, undo history, and caches of its own. The operating system represents that work as one or more **processes** and **threads**—abstractions for concurrent execution, not mystical containers that “are” the file (Tanenbaum and Bos 2022).

7.3.3 OS mediation

The OS schedules who runs, mediates access to memory and files, and may reclaim RAM under pressure. It does **not** replace your application’s own computation; it allocates attention and enforces boundaries (Tanenbaum and Bos 2022).

7.3.4 CPU (and optional accelerators)

Processors execute instructions and touch data that must arrive from somewhere in the hierarchy. Accelerators may help with media or composition; they still depend on data movement and shared resource limits.

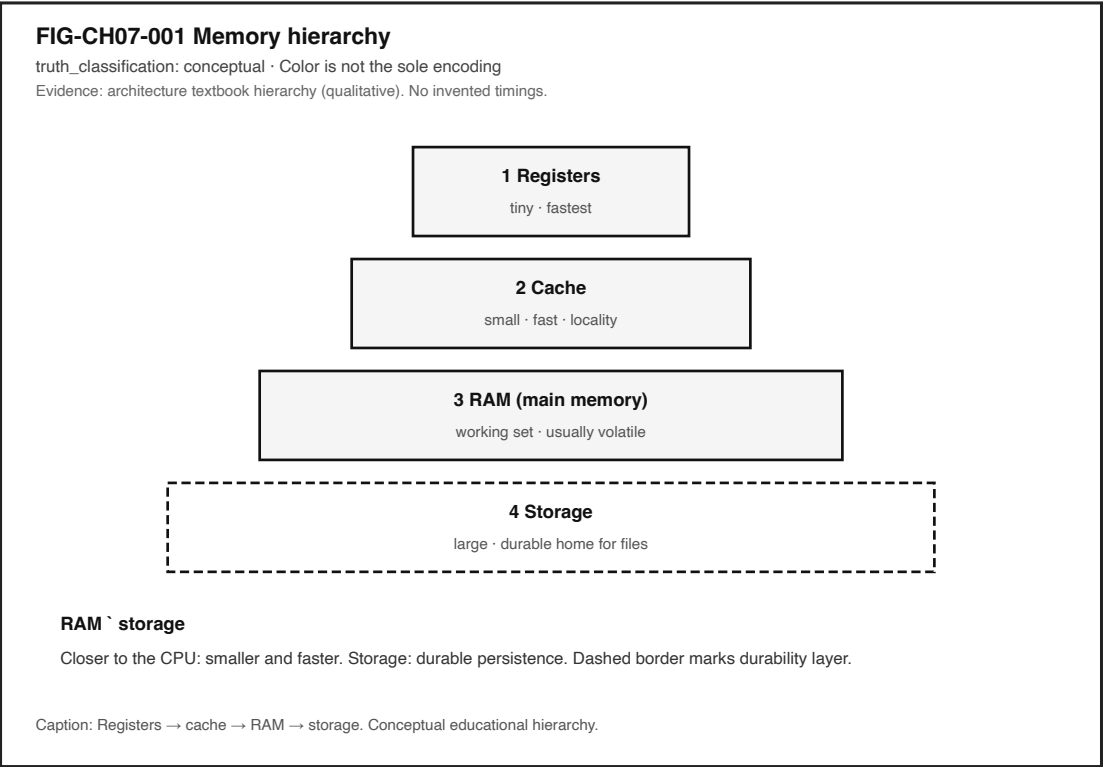


Figure 7.1: Conceptual pyramid from registers through cache and RAM to durable storage.

7.3.5 Memory hierarchy

Closer to the CPU, storage tends to be **smaller and faster**. Farther away, capacity grows and durability becomes the point. Architecture texts teach this capacity/latency/cost tradeoff as a hierarchy, not as four interchangeable synonyms for “memory” (Patterson and Hennessy 2020b).

7.3.6 Storage and power

Durable media hold files across quit and power-off in ordinary personal computing. Power and thermal policy can quietly reduce available performance during sustained work—another local domain that can look like “slowness” while connectivity still appears healthy.

7.3.7 Optional network (often ruled out)

Remote services may matter for other chapters. For this chapter’s anchor, a healthy connectivity icon is frequently a **ruled-out** explanation when the workload is local: open a local file, scroll a local editor, save to local disk.

Device Quartet form factors (desk, handheld hybrid, learn-to-build, wearables) remain **research / learning benchmarks**. Any RAM or storage *capacity figures* that would require physical validation stay **PHYSICAL_PENDING** (CLM-CH07-004). Commodity devices remain first-class evidence sources for the lab.

7.4 4. Follow the signal

Follow one open. Imagine a medium-sized local file on durable storage.

1. **Intent.** You choose *Open* (or the app restores a previous session).
2. **Request.** Software asks the OS for the file’s bytes—not for “some RAM,” not for “the Wi-Fi.”
3. **Storage read.** Durable media and the storage stack supply data. This step can dominate first-open feel.
4. **Into RAM.** A working copy (and related structures) lands in main memory so the CPU can use it without revisiting storage for every tiny access (Patterson and Hennessy 2020b); (Tanenbaum and Bos 2022).
5. **Into cache.** Hardware caches keep recently or likely-needed lines nearby. **Locality**—reusing the same or nearby data—is why caches reduce average access time; a **miss** costs a trip to a farther layer (Patterson and Hennessy 2020b).
6. **CPU work.** Instructions decode, edit, render previews, run spellcheck—whatever the app does.
7. **UI progress.** Frames update; a page appears; a spinner stops.
8. **Later save (optional second path).** Edited state in RAM is not automatically durable. Completing a save pushes bytes toward storage; buffers may delay true durability until the write path finishes (Tanenbaum and Bos 2022).

Figure 7.2 shows that left-to-right teaching path. Segments are conceptual—not a stopwatch claim about your device.

Three separations matter while you follow the signal:

1. **Warm reopen vs cold open.** Data already in RAM/cache can feel instant; a cold open may pay storage I/O.

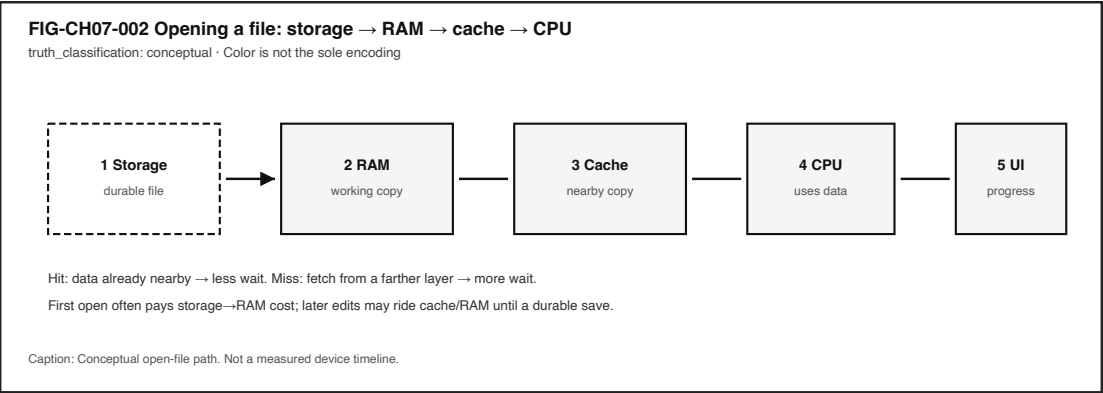


Figure 7.2: Conceptual open-file path from storage through RAM and cache to CPU and UI.

- 2. **On screen vs on disk.** Visible text can live in a volatile working set while the durable file is older—or missing—until a save completes.
- 3. **Connected icon vs local wait.** Memory pressure and disk backlog can hitch the UI while the network glyph stays polite (Tanenbaum and Bos 2022). Figure 7.3 sketches that contrast as an **illustrative** teaching aid.

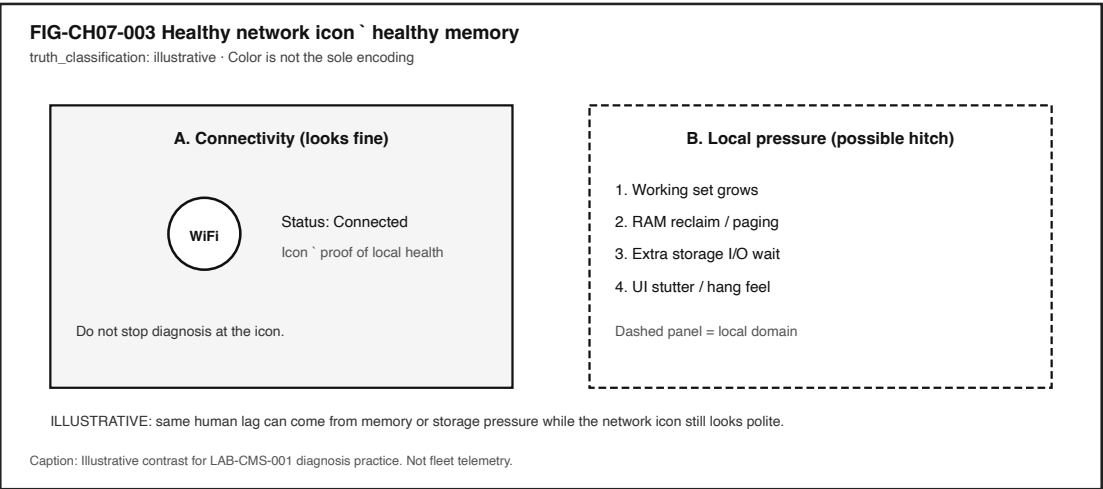


Figure 7.3: Illustrative two-panel contrast of a healthy network icon beside memory-pressure symptoms.

7.5 5. Component cards

These cards are a toolkit for naming layers—not a complete bill of materials.

7.5.1 Registers

Role. Tiny holding places inside the CPU for values in active use.

Human feel. Invisible; you notice only the aggregate responsiveness of computation.

Failure symptom (rare as a named user complaint). Extreme compute load, not “I ran out of registers” in everyday speech.

Not the same as. App “memory use” percentages in a monitor, RAM sticks, or durable storage.

7.5.2 Cache

Role. Smaller, faster memory holding recently or likely-needed data to exploit locality (Patterson and Hennessy 2020b).

Human feel. Repeated actions often feel snappier than first-time opens.

Failure symptom. Cold starts and large working-set jumps that miss nearby copies.

Not the same as. Browser “clear cache” buttons (related idea, different layer) or durable storage.

7.5.3 RAM (main memory)

Role. Volatile working space for running programs and live data under ordinary power-off conditions (Patterson and Hennessy 2020b); (Tanenbaum and Bos 2022).

Human feel. Smooth multitasking when the **working set**—what the workload actively needs in a time window—fits; hitching when it does not.

Failure symptom. Climbing memory use, reclaim activity, paging/swap pressure, apps reloading after backgrounding.

Not the same as. Disk/SSD capacity marketed as “storage.”

7.5.4 Storage

Role. Durable persistence for files and installed software—the ordinary home that survives quit and power-off (Tanenbaum and Bos 2022).

Human feel. First opens, saves, installs, and photo imports that wait on I/O.

Failure symptom. Lingering save dialogs, disk activity spikes, “recovered files” after a crash, full-disk warnings.

Not the same as. RAM. Confusing the two is the chapter’s primary misconception.

7.5.5 Working set

Role. The subset of memory a workload actively needs now—not the entire installed app size on disk.

Human feel. One heavy document can hurt more than ten idle icons.

Failure symptom. Thrashing-like feel when the system spends too much time moving data between RAM and storage instead of making progress (Tanenbaum and Bos 2022).

Not the same as. A single marketing gigabyte number on a retail box.

7.5.6 Volatility vs durability

Role. What disappears on power loss versus what is meant to persist.

Human feel. Unsaved buffers vanish; saved files return.

Failure symptom. Believing “I saw it on screen” equals “it is saved.”

Not the same as. Encryption or backup policy (related, later chapters).

7.6 6. Stability contract

The **Stability Contract** is a signature idea across this book:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For Chapter 7, a successful open/edit/save experience may require all of the following to stay “good enough” at once:

- the working set fits available RAM **or** the system degrades gracefully without pathological thrash,
- needed storage reads/writes complete (or fail with a clear report),
- storage I/O is not saturating the interactive path,
- the interactive threads still obtain CPU time under contention,
- hierarchy misses stay mild enough that “simple” actions do not feel randomly stuck,
- power/thermal policy still permits needed performance,
- durable saves that the person believes completed actually reach durable media—or the UI admits they did not.

A device can remain **powered on and connected** while the **human experience has already failed**.

Three separations matter:

1. The interface can look **alive** while a save has not finished durably.
2. The network can report **connected** while RAM reclaim or disk wait owns the hitch.
3. High CPU percent can mean useful work—or spinning while waiting on memory/I/O; monitors need paired columns, not a single villain number.

This chapter does not invent universal millisecond budgets or Device Quartet capacity measurements (CLM-CH07-004). Classroom observations in **LAB-CMS-001** are *your* evidence for *your* device and conditions—not a fleet benchmark.

7.7 7. Try it

7.7.1 LAB-CMS-001 — Make Local Slowness Visible

Observable question. When a familiar local app feels slow but the connectivity icon looks fine, what evidence can I gather—using only commodity tools—to separate **memory pressure**, **storage I/O**, and **cache/hierarchy misses** from a CPU-only story?

Inheritance note. LAB-CMS-001 is the publication-owned Concept Edition lab for CE-3; Chapter 7 reuses it as the hierarchy/persistence practice lab rather than inventing a duplicate WAIKE module ID. WAIKE accepted **main** hosts adjacent competencies (for example MCU memory-map and storage-triage labs) but no exact “CH07 memory hierarchy” course module ID—adjacency only, no invented titles.

Prerequisites. A computer you may use for learning; built-in OS monitor (Task Manager / Activity Monitor / **top**) or the lab’s fixture fallback; optional Python 3 for the safe snapshot helper.

Safety. Do not capture personal document contents; redact filenames before sharing. Do not install “PC cleaner” tools, disable security software, or load kernel modules. Use **mild** load

only. **Stop** if the device warns about heat or becomes uncomfortably hot. No Device Quartet hardware is required.

Time estimate. About 45 minutes for Explorer + Operator baseline.

7.7.1.1 Prediction

Before measuring, write which hidden part you expect to dominate during your action: CPU activity, memory pressure, persistent storage / disk I/O, or scheduling / thermal-power limits.

7.7.1.2 Route A — Commodity computer (baseline)

- 1. Open your built-in OS monitor.
- 2. Record **before** CPU, memory, and disk/storage activity.
- 3. Perform one controlled local action (open a medium document, scroll, or save).
- 4. Record **during** readings and wall-clock feel.
- 5. Optional Experience B: save → quit → reopen; note whether content returned.
- 6. Fill observation vs inference columns.

7.7.1.3 Route B — Safe CLI snapshot (optional)

```
python3 labs/LAB-CMS-001/local_app/safe_snapshot.py
```

Read-only sampling where the OS exposes coarse stats. If a metric is unavailable, the script reports `unavailable`—never invent hardware claims.

7.7.1.4 Route C — Fixture fallback

Use `labs/LAB-CMS-001/fixtures/` when monitors are inaccessible or you must avoid personal screenshots. Fixtures teach reading order; they are not measurements of your personal device.

7.7.1.5 Evidence (minimum)

- observation table,
- two monitor snapshots or fixture IDs with timestamps,
- teach-back: RAM vs storage; OS schedules while apps still compute.

7.7.1.6 Interpretation

Observation (allowed)	Inference (must label)
CPU % rose from A to B	“CPU-bound root cause”
Memory used rose; disk active	“Thrashing” without page-fault/swap evidence
Save completed; reopen succeeded	“All future autosaves are durable”
Device felt warmer	“Thermal throttle at X °C”

7.7.1.7 Limits

- Classroom N is small; no published benchmark claims.
- Monitor samples are coarse.
- Thermal/power effects stay qualitative unless you have disclosed sensors.
- No unsupported hardware timing budgets; no shipping-SKU marketing language.

7.7.1.8 Portfolio

Use labs/LAB-CMS-001/portfolio/ templates: observation table, teach-back, hierarchy map, diagnosis plan, hypothesis note.

7.8 8. Build it

Use the same open/save story at the depth that matches your pathway.

7.8.1 Explorer

Teach-back in ordinary language: what is RAM for, what is storage for, and why calling both “memory” confuses diagnosis? Point at Figure 7.1 while you talk.

7.8.2 Operator

Complete LAB-CMS-001 before/during snapshots for a controlled open **and** a controlled save. Note whether the connectivity icon changed. Label every causal sentence as observation or inference.

7.8.3 Builder

Draw a labeled hierarchy map for one experience: registers/cache/RAM/storage, plus the process that owns the document, plus where a durable file lives. Change one variable (extra documents open, larger file, or save vs scroll) and re-run the observation table.

7.8.4 Engineer

Relate roles to qualitative latency/capacity tradeoffs: why a cache miss costs more than a hit; why a cold storage read can dominate first open; what evidence would upgrade “disk looked busy” to a storage-bound claim (Patterson and Hennessy 2020b); (Tanenbaum and Bos 2022). Order inspections: connectivity rule-out → CPU → memory → disk → qualitative thermal/power.

7.8.5 Researcher

State a hypothesis about working-set size versus thrashing-like symptoms under two load conditions. List variables, planned repetitions, confounders (battery vs plugged, background updates, thermal state), and what would be required to publish a measured claim. Do **not** invent GB/s product numbers.

Educators can facilitate the RAM storage misconception check and the fixture route for classrooms without admin rights.

7.9 9. Secure and include it

7.9.1 Security

Memory and storage are trust boundaries, not only performance knobs. Process isolation exists partly so one app should not freely read another’s memory—teach the **intent** without claiming

every consumer OS is perfectly enforced (Tanenbaum and Bos 2022). Storage holds credentials, messages, and schoolwork; foreshadow encryption-at-rest and careful sharing without turning this chapter into a cryptography treatise. Warn that “RAM cleaner / PC booster” downloads marketed for memory symptoms are a common social-engineering path—LAB-CMS-001 forbids them.

7.9.2 Privacy

Monitor screenshots can leak filenames, account names, and thumbnails. Portfolio artifacts should redact personal paths. Do not require cloud sync of lab evidence. Prefer fixtures when evidence must leave the learner device.

7.9.3 Accessibility

Lag and stutter are accessibility failures: motor timing, cognitive load, and vestibular stress all rise when frames hitch. Long I/O needs progress that is perceivable without color alone—labels, text, and patterns. Provide keyboard paths to OS monitors where the platform allows them; accept dictated observation tables; keep fixture routes first-class.

7.9.4 Equity

Not every learner has a high-RAM laptop. Teaching must not shame low-end devices or treat “just buy more memory” as the moral of the chapter. Diagnosis, workload control, and honest save UI come first. Device Quartet form factors are learning benchmarks, not purchase requirements (CLM-CH07-004). Offline fixtures keep Explorer/Operator goals reachable on locked-down school images.

7.10 10. Career lens

No table promises employment; roles vary by organization. LAB-CMS-001 artifacts resemble early professional evidence in miniature—observation tables, hierarchy maps, and labeled inferences.

Layer	Example role	Professional artifact	Lab resemblance
Hierarchy teaching	Computer architecture / CS educator	Hierarchy lecture + misconception probes	Teach-back + Figure 7.1 map
Main memory systems	Systems / embedded engineer	Memory map / budget notes	Working-set vs capacity discussion
Storage stack	Storage / filesystem engineer	I/O path write-up	Save/quit/reopen checklist
Performance	Performance engineer	Profile bundle with claim labels	Before/during monitor table
Reliability	SRE / support engineer	Incident timeline separating domains	Healthy-icon + local-pressure contrast
Data lifecycle	Backup / data governance adjacent	Durability and retention notes	Volatility vs durability teach-back

Layer	Example role	Professional artifact	Lab resemblance
Security	Security engineer	Isolation / at-rest threat notes	Redacted screenshots; no cleaner tools
Accessibility	Accessibility engineer	Progress and stutter review	Progress during long I/O; fixture equity

When Device Quartet capacities appear in hardware-repo comparison materials, treat them as **representative educational targets**, not measured shipping specs (CLM-CH07-004).

7.11 11. Check understanding

Answer with reasoning. Prefer short paragraphs over one-word guesses.

1. Why might an app reopen instantly once, then feel slow after you open several large files—even though Wi-Fi still looks fine?
2. What is wrong with saying “the phone has 128 GB of memory” when you mean durable capacity?
3. How do caches use locality to reduce average access time, and what does a miss cost in human terms?
4. What evidence would you need before claiming thrashing rather than “disk looked busy once”?
5. Why can text visible on screen disappear after quit if a durable save never completed?
6. Name two Stability Contract conditions from Section 6 that can fail while the connectivity icon stays healthy.
7. Why is “buy more RAM” an incomplete default answer to every hitch?
8. **Teach-back.** Explain RAM vs storage to a family member **without** using the words *volatile*, *hierarchy*, or *paging*. Then introduce those three terms one at a time, tying each to something already understood.

Educator note: successful teach-backs show the reopen/open/save causal sequence and at least one mis-attribution (network icon vs local pressure).

7.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Project-specific Device Quartet boundaries remain claim-scoped (CLM-CH07-004) and **PHYSICAL_PENDING** for fabrication measurements.

Inline citations used in this chapter include Patterson and Hennessy (2020b) and Tanenbaum and Bos (2022).

7.13 12. Glossary links

Terms introduced or relied on as formal vocabulary in this chapter should resolve in the living glossary registry. This section lists them for linking—not as a dump of free-standing encyclopedia entries.

Term	Role in this chapter
Memory hierarchy	Speed/capacity layers from registers to storage
Register	Tiny CPU-local storage for active values
Cache	Smaller faster holding of likely-needed data
Cache hit / cache miss	Found nearby vs must fetch farther
Locality	Reuse of same or nearby data over time/space
RAM (main memory)	Volatile working memory for running programs
Storage	Durable persistence medium for files
Volatility vs durability	Disappears on power loss vs meant to persist
Working set	Memory a workload actively needs in a window
Thrashing (qualitative)	Excessive paging/I/O under memory pressure that destroys feel
Process / thread	OS abstractions for concurrent work
Scheduler	Allocates processor time among runnable work
File	OS-managed durable object (deeper in CH13)
Stability Contract	Experience depends on hidden conditions staying acceptable
Paging / reclaim	OS movement/recovery of memory under pressure

Deeper entries, analogies labeled as analogies, and “not the same as” warnings live in the glossary network. Prefer following a link when stuck over inventing a private definition.

7.14 Figure references (embedded above; accessibility metadata)

All figures below are **conceptual** or **illustrative** as labeled. Source preference: editable SVG in the publication repository.

7.14.1 FIG-CH07-001 — Memory hierarchy

- **Caption.** Registers → cache → RAM → storage with qualitative tradeoffs; RAM storage.
- **Alt text.** Pyramid diagram with four labeled layers from tiny/fastest to large/durable.
- **Text equivalent / reading order.** Registers → Cache → RAM → Storage → RAM storage callout.
- **Status.** Conceptual educational diagram.
- **Source.** Publication-owned original informed by architecture hierarchy teaching (Patterson and Hennessy 2020b).

7.14.2 FIG-CH07-002 — Open file path

- **Caption.** Storage → RAM → cache → CPU → UI for a conceptual open.
- **Alt text.** Five numbered boxes connected left to right.

- **Reading order.** Storage, RAM, Cache, CPU, UI.
- **Status.** Conceptual.
- **Source.** Publication-owned original.

7.14.3 FIG-CH07-003 — Healthy network icon with memory-pressure hitch

- **Caption.** Illustrative contrast of a connected icon beside local memory-pressure symptoms.
- **Alt text.** Two panels: connectivity looks fine; local pressure steps to UI stutter.
- **Reading order.** Panel A, then Panel B steps 1–4.
- **Status.** Illustrative teaching aid—not measured fleet evidence.
- **Source.** Publication-owned original for LAB-CMS-001 diagnosis practice.

7.15 Claim footnotes used in this chapter (project-specific)

Claim ID	Approved gist	Classification
CLM-CH07-001	Registers/cache/RAM/storage form a hierarchy; RAM durable storage	general technical (textbook-backed)
CLM-CH07-002	Caches exploit locality; misses cost time	general technical (textbook-backed)
CLM-CH07-003	Memory pressure can degrade experience while connectivity appears healthy	general technical / lab-illustrated
CLM-CH07-004	Device Quartet RAM/storage capacity figures requiring physical validation remain PHYSICAL_PENDING	project-specific

General statements about hierarchy, volatility, and files are treated as general technical knowledge scoped to the cited textbooks. Latency or throughput numbers, when shown later, must carry **illustrative**, **measured**, or **inferred** labels—never invented product scores.

Manuscript status: working draft. Human reader validation pending. Not publication-ready. Gate 3 reader evidence is not claimed here.

Chapter 8

Chapter 8 — Graphics, Displays, Audio, Cameras, and Sensors

Status: draft · Chapter ID: CH08

Author: Edmund Gunn, Jr.

Manuscript: working full-manuscript draft · human validation pending · not publication-ready

8.1 1. The moment

You watch a short video. A notification banner slides in from the top. Music from another app keeps playing. Somewhere in the same pocket of glass and metal, a microphone may be waiting for a wake word—or a camera preview may be one tap away if you decide to take a photo.

From your seat it feels like one device doing several friendly things at once. Underneath, several **pipelines**—paths that turn between the physical world and digital representations—share the same processors, memory, power, and thermal budget.

This chapter’s governing question is practical:

How do displays, audio, cameras, and sensors turn between the world and bits—and why do frames, sampling, and pipelines shape how the experience *feels*?

We will not catalog every gadget part number. Part II treats human I/O modalities as **system interfaces**. Chapter 2 already named rendering and frames on the way out of a tap; Concept Edition compute materials survey GPU roles beside the CPU. Here we stay with experience → system → component, then give you a permission-safe way to map one pipeline yourself.

8.2 2. What you notice

Before vocabulary like *compositor* or *ISP*, notice the human contracts you already expect.

You expect the picture to look continuous enough that motion does not feel broken. You expect sound not to crackle for no reason. You expect a photo button to respect your choice about

the camera. You expect a wake-word listener—if present—not to pretend it was off when it was on. You expect the phone not to become a hand warmer just because a banner animated over a video. You expect failure to be visible: a permission denied, a black preview with an honest message, a frozen frame with a spinner—not a silent wrong recording of someone else.

Those expectations are not decorations. They *are* the media product from the person’s point of view.

Media feel is produced by multiple pipelines sharing one device’s budgets.

A smooth-looking video can share the GPU with an animation. An audio path can glitch while the screen still looks polite. A camera can be optically fine while the software never received permission to open the sensor. Your nervous system stitches those timelines into “this feels fine” or “something is wrong,” often before you have a technical name for the fault.

Optional comparison on a device you already own: play a local video alone; then play it while opening a heavy settings animation or another media app. You are not collecting a product benchmark. You are noticing contention as an everyday fact (CLM-CH08-003) (Tanenbaum and Bos 2022).

A second optional notice: open a camera or microphone permission dialog in a context where you are allowed to experiment, then cancel or deny. The preview may never appear. That is still a complete lesson about gates sitting in front of sensors—without capturing anyone.

8.3 3. Exploded ecosystem

Figure 8.1 is the first-minute map for this chapter: physical world → sensors/cameras/mics → processing → display/speakers → human perception. It is **conceptual**—Representative educational architecture, not a claim that any specific manufactured revision wires exactly like the diagram.

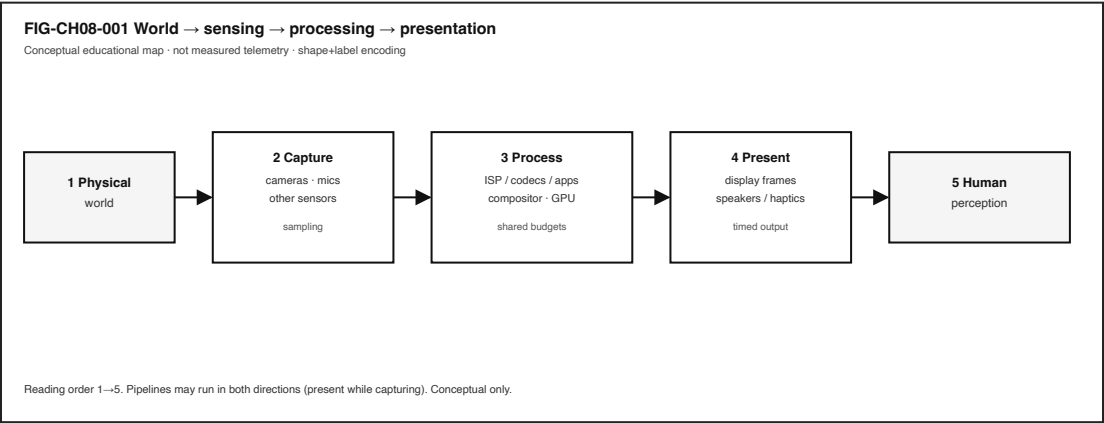


Figure 8.1: Conceptual map from physical world through capture and processing to presentation and perception.

Walk the layers in ordinary language.

8.3.1 Human

You watch, listen, speak, move, and decide. Eyes and ears close presentation loops. Intent to capture—“take a photo,” “start a call”—opens capture loops only when policy and permission allow.

8.3.2 Sensors and transducers

A **sensor** turns a physical quantity into a signal or data. Cameras and microphones are familiar; accelerometers, ambient light sensors, and touch digitizers are neighbors. They do not hand you “the world.” They hand you **samples**.

8.3.3 Capture pipelines

A **camera pipeline** (optics → sensor → image-signal processing → buffers) and an **audio pipeline** (microphone → conversion → buffers → optional encode) build digital media. On the web platform, camera and microphone access is explicitly modeled as mediated capture of media streams under a permission model—not as unmediated truth about reality (“Media Capture and Streams” 2025) (CLM-CH08-002).

8.3.4 Compute and composition

CPU work, **GPU** parallel work, codecs, and a **compositor** (the system that combines layers into a presentable image) share memory and time (Patterson and Hennessy 2020b; “Vulkan Overview” 2026). Operating systems schedule competing work rather than letting every app own the silicon alone (Tanenbaum and Bos 2022).

8.3.5 Presentation pipelines

A **display pipeline** turns framebuffer/compositor work into visible light patterns. Speakers and haptic actuators present other modalities. A **frame** is one image update in a timed presentation sequence—vocabulary Chapter 2 already used on the way out of a tap.

8.3.6 Device Quartet (learning spine only)

Where Edge IO Wearables or other Quartet form factors appear as sensing analogies, treat them as research/learning spines. Physical camera/sensor EVT claims remain **PHYSICAL_PENDING** (CLM-CH08-004) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Commodity devices remain first-class for labs.

8.4 4. Follow the signal

Two directions matter. Presentation moves bits toward light and sound. Capture moves light and sound toward bits. Many moments do both.

8.4.1 Path P — Present (stored or generated media → human)

1. **Source ready.** A file, stream, or generated UI layer exists in memory or storage.
2. **Decode / render.** Software (and often a GPU or media block) produces pixel and/or audio buffers (Patterson and Hennessy 2020b).

- 3. **Compose.** The compositor combines video, UI chrome, banners, and captions into a frame candidate.
- 4. **Present.** The display pipeline shows a frame; the audio pipeline emits samples.
- 5. **Perceive.** You judge smoothness, lip-sync, loudness, and meaning.

Figure 8.2 sketches steady versus uneven presentation *feel* as an **illustrative** teaching aid. It does not assert product hitch thresholds or a surveyed law of missed deadlines; that survey-depth evidence gap remains open as **CLM-CH08-001** (claim footnotes below). Read it as “rhythm you can notice,” not as a measured scoreboard.

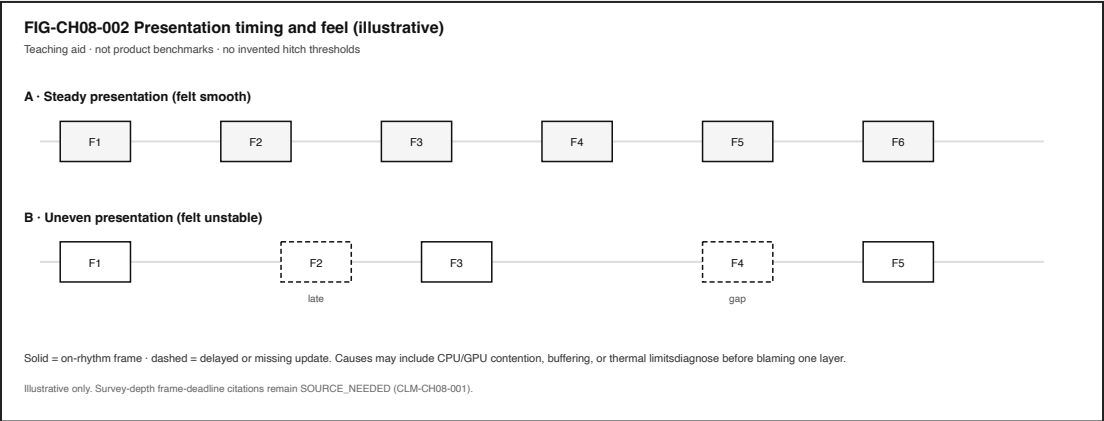


Figure 8.2: Illustrative timeline of steady versus uneven frame presentation feel.

8.4.2 Path C — Capture (world → representation)

- 1. **Consent / permission gate.** Policy asks whether this app may use the camera or microphone now (“Media Capture and Streams” 2025).
- 2. **Transduce.** Optics and sensors (or a microphone diaphragm and converters) produce electrical signals.
- 3. **Sample.** Continuous phenomena become discrete samples—pixels, PCM audio, IMU readings (Figure 8.3).
- 4. **Process.** Image-signal processing, noise filtering, encoding, or on-device inference may run.
- 5. **Buffer / store / send.** Representations land in memory, files, or network messages.
- 6. **Optional preview.** A presentation path may show what is being captured—still a representation.

8.4.3 Honesty rule

Digital media are representations, not the world itself (CLM-CH08-002). A beautiful preview can still be cropped, delayed, color-processed, or blocked by permission. A denied permission is not a broken lens; it is a policy outcome.

8.4.4 Contention rule

Multiple media pipelines contend for CPU, GPU, memory, and power on one device (CLM-CH08-003) (Tanenbaum and Bos 2022). When a banner animates over a video while audio plays, you are watching shared scheduling and shared accelerators, not three separate computers glued together.

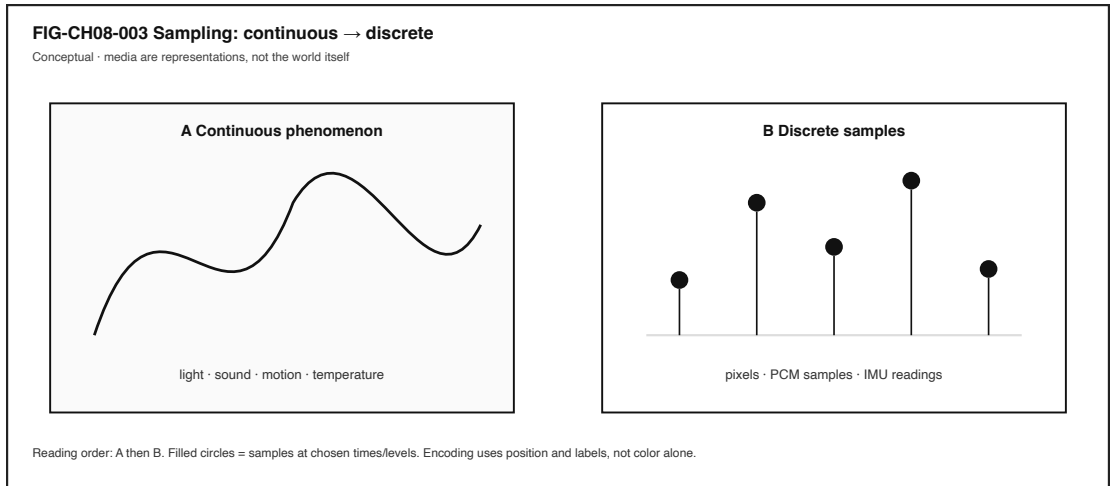


Figure 8.3: Conceptual comparison of a continuous phenomenon and discrete samples.

8.4.5 What this chapter does not invent

Frame-timing pedagogy is anchored to platform display-refresh docs (CLM-CH08-001 · SOURCE_IDENTIFIED via (“Window: requestAnimationFrame() Method” 2026; WHATWG 2026b)). This chapter still refuses invented hitch/tearing thresholds or product frame budgets. Learner-measured notes from LAB-IO-001 stay labeled as *your* observations on *your* device.

8.5 5. Component cards

Each card answers: What is it? What does it do for the person? What fails when it misbehaves?

8.5.1 Display pipeline

Plain definition. The path that turns framebuffer/compositor work into visible light patterns.

Experience benefit. You see coherent UI and video.

Failure symptom. Black screen, wrong layer order, frozen UI chrome, or uneven motion feel—diagnose before assuming “the panel is dead.”

8.5.2 Frame

Plain definition. One image update in a timed presentation sequence.

Experience benefit. Motion and UI change become watchable over time.

Failure symptom. Stuttery or uneven visual update; still may be compositor, app, GPU, or power—not only “refresh rate.”

8.5.3 Audio pipeline

Plain definition. Capture and/or render path for sound.

Experience benefit. Speech, music, and alerts become audible in sync with intent.

Failure symptom. Glitches, silence despite UI animation, wrong routing (earpiece vs speaker), or permission denial.

8.5.4 Camera pipeline

Plain definition. Optics $\rightarrow$ sensor $\rightarrow$ ISP/processing $\rightarrow$ image/video buffers.

Experience benefit. Scenes become images or video representations you can preview, save, or send.

Failure symptom. Black preview with permission errors, focus/exposure oddities, thermal throttling after long capture, or privacy blocks.

8.5.5 Sensor

Plain definition. A transducer that turns a physical quantity into a signal or data.

Experience benefit. The device can react to light, motion, orientation, touch, and more.

Failure symptom. Stuck readings, noisy values, missing calibration, or apps inventing meaning the sensor never provided.

8.5.6 Sampling

Plain definition. Measuring a continuous phenomenon at discrete times and/or levels.

Experience benefit. Computers can store and compute on media and measurements.

Failure symptom. Alias-looking artifacts, choppy motion from too-sparse samples, or over-confidence that samples equal reality.

8.5.7 Compositor / GPU role (survey)

Plain definition. Two cooperating jobs, not one synonym: the **compositor** combines layers into a presentable frame; a **GPU** (or similar accelerator) may speed rendering and related parallel work when software submits it through mediated APIs (“Vulkan Overview” 2026; Patterson and Hennessy 2020b).

Experience benefit. Smooth composition of video + UI + banners when budgets hold.

Failure symptom. Janky animations, dropped UI responsiveness, or heat while “nothing important” seemed to run.

Not the same as. The display panel itself—the pipeline that turns a composed frame into light can fail even when GPU work looked busy.

These cards are a failure-domain toolkit—not a shopping list.

8.6 6. Stability contract

The **Stability Contract** returns:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For media on one device, a “fine” experience may require all of the following to stay good enough at once:

- presentation path producing frames and/or audio samples the person can use,
- capture path only active when permission and consent allow,
- compositor ordering layers so the person sees the intended surface,
- CPU/GPU/memory headroom shared fairly enough that one pipeline does not silently starve another (Tanenbaum and Bos 2022),
- power/thermal conditions not forcing a sudden quality collapse without feedback,
- failures visible (permission denied, offline file missing, device busy)—not silent wrong capture,
- accessibility alternatives available when vision- or hearing-only cues would exclude someone (W3C 2024).

Three separations matter:

1. The screen can look busy while audio has already glitched.
2. A camera preview can look live while permission never authorized recording.
3. A sensor can report numbers while the app’s interpretation is wrong.

You experience the combined result. Blaming “the display” or “the mic” without evidence collapses domains into one vague villain.

Analogy (labeled as an analogy): shared pipelines on one device are a bit like several cooks sharing one stove—the meal can still succeed, but only while heat, space, and timing stay “good enough” together. The analogy must not replace measurement; it only names concurrency.

8.7 7. Try it

8.7.1 LAB-IO-001 — Map one media pipeline (permission-safe)

Observable question. For one familiar media experience, what is the capture → process → present path—and which parts can I observe without capturing other people?

WAIKE alignment note. WAIKE (accepted **main**) offers adjacent competencies such as game-loop timing, sensor-noise awareness, and ADC sampling labs; it does **not** ship an exact CH08 module ID (gunnchOS3k 2026). **LAB-IO-001** is therefore a **publication-owned** commodity lab. Do not invent a WAIKE lab ID for it.

Prerequisites. A computer or phone you may use for learning; optional local media file you already own.

Safety and privacy (non-negotiable).

- Do **not** capture other people’s faces, voices, or private spaces without explicit permission.
- Prefer the offline fixture, a public-domain file, or a permission dialog you **deny**.
- Do not enable always-on microphone or camera recording for this lab.
- No rooting, no unsafe thermal abuse to force glitches.

Time estimate. About 40–75 minutes including write-up.

8.7.1.1 Prediction

Write one sentence: if something feels wrong, which pipeline do you expect to notice first—display, audio, camera permission, or another sensor?

8.7.1.2 Route A — Fixture only (baseline)

Open `labs/LAB-IO-001/fixtures/pipeline_map.md`. Fill the capture → process → present table using the offline story. No live sensing required.

8.7.1.3 Route B — Local media you already own

Play a short local video or audio file. While it plays, trigger a lightweight UI animation (notification shade, settings pane). Note what you **observe** (banner appeared; audio continued) versus what you only **infer** (GPU overloaded).

8.7.1.4 Route C — Permission deny (optional)

Open an app’s camera or microphone prompt in a context where you are allowed to experiment, then **Deny** or cancel. Record whether the app offers a non-sensing path. Do not grant capture in shared spaces if bystanders could be recorded.

8.7.1.5 Evidence (minimum)

- one filled pipeline map,
- permission/consent notes,
- observation-vs-inference table,
- teach-back paragraph (Section 11 style).

8.7.1.6 Limits (say them out loud)

- UI timestamps are not photon-to-retina measurements.
- Denying permission is a valid outcome, not a failed lab.
- One session is a story under one set of conditions—not a device grade.
- Quartet wearable sensing remains PHYSICAL_PENDING; do not invent EVT numbers (“Gunuchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Completion means a claim is supported by an artifact—not that a camera preview opened.

8.8 8. Build it

Use the same pipeline story at the depth that matches your pathway.

8.8.1 Explorer

Draw Figure 8.1 from memory with five boxes. Label which boxes your LAB-IO-001 route actually touched.

8.8.2 Operator

Using only on-screen cues, decide whether a failure looks more like (a) permission/policy, (b) presentation/compositor, or (c) audio routing. Write why—without claiming root cause beyond evidence.

8.8.3 Builder

Create a one-page markdown map: arrows for capture and present; a dashed box labeled “permission gate.” Keep secrets and faces out of any attached screenshots.

8.8.4 Engineer

Propose how you would measure uneven presentation or audio glitches with commodity tools (for example, browser or OS performance views where available) (MDN Web Docs 2026b). Mark each proposed metric as **measured**, **inferred**, or **out of reach** on your device. Do not invent hitch thresholds.

8.8.5 Researcher

Design a repeated comparison: same device, same clip, with and without a concurrent animation. Pre-register what you will count as an observation. State limits: logging overhead, subjective feel ratings, and the absence of a pinned multimedia-systems survey citation for deadline physics in this edition’s claim plan.

Educators can facilitate teach-backs and adapt LAB-IO-001 for classrooms that forbid any camera use—fixture-only is enough.

8.9 9. Secure and include it

8.9.1 Security and privacy

Cameras and microphones are privileged sensors. Relevant ideas here:

- **permissions** and consent before capture (“Media Capture and Streams” 2025),
- **minimize retention**—labs should not keep other people’s media,
- **indicator honesty**—when capture is active, the person should be able to tell,
- **least privilege**—an app that needs a photo does not need always-on microphone,
- **trust boundaries**—a preview surface can be spoofed in malicious UI; teach skepticism without turning this chapter into a full adversarial playbook.

Platform-specific OS permission encyclopedias remain an open source need for later tightening; this chapter uses the W3C media-capture permission model as a representative, citable web path—not as a claim that every OS dialog matches it.

8.9.2 Accessibility

Do not ship experiences that only work for one sense. WCAG 2.2 provides dated guidance for alternatives to purely visual information, time-based media considerations, and motion sensitivity concerns that affect animation-heavy interfaces (W3C 2024). Captions, transcripts, non-motion alternatives, and keyboard-operable controls are part of Stability Contract thinking—not optional polish.

8.9.3 Equity

Not every learner has a high-refresh display, multiple cameras, or quiet space for audio labs. Fixture-first routes and deny-permission routes keep the learning intact when hardware, policy, or shared classrooms constrain capture. Designing only for ideal studio conditions silently excludes people.

8.10 10. Career lens

No table promises employment. Roles vary by organization. LAB-IO-001 artifacts resemble early professional evidence in miniature.

Domain	Example role	Professional artifact	Lab resemblance
Display / UI graphics	Graphics or UI engineer	Frame/composition notes	Presentation map + feel notes
Audio	Audio software engineer	Capture/render path write-up	Audio pipeline card + glitch hypotheses
Camera	Imaging / ISP engineer	Pipeline block diagram	Camera path without unauthorized capture
Sensors	Embedded sensing engineer	Sampling and calibration notes	Sensor vs interpretation separation
GPU / media	Media performance engineer	Trace bundle	Measured vs inferred timing labels
Privacy	Privacy engineer	Permission and retention review	Consent notes in portfolio
Accessibility	Accessibility specialist	WCAG-informed review	Non-visual alternative check
UX	Interaction designer	Motion and feedback spec	Banner-vs-video contention observation

When Quartet wearables appear in later sensing chapters, they remain research/learning spines—not shipping SKU ads—until EVT evidence exists (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

8.11 11. Check understanding

Answer with reasoning. Prefer short paragraphs.

1. Why can a video look fine while audio glitches, or the reverse?
2. What is the difference between a sensor reading and the meaning an app attaches to it?
3. Why is a denied camera permission not proof that the lens hardware failed?
4. How does sampling turn a continuous sound into something a computer can store?
5. Why might a notification animation change how smooth a video *feels* even if the file is unchanged?

6. Name two Stability Contract conditions for a media experience that are not “pretty pixels.”
7. What evidence would you need before blaming the GPU for uneven motion?
8. **Teach-back.** Explain capture → process → present to a family member **without** using the words *compositor*, *ISP*, *GPU*, or *buffer*. Then introduce those four terms one at a time, tied to something already understood.

Educator note: successful teach-backs show both directions (present and capture) and at least one permission or equity constraint.

8.12 References

Selected authoritative sources for this chapter’s general technical explanations live in the working Full31 bibliography (`publication/full131/WORKING_BIBLIOGRAPHY.bib`) and the book reference pool. Project-specific repository evidence is cited by claim ID where labeled.

Inline citations used in this chapter include Tanenbaum and Bos (2022), Patterson and Hennesy (2020b), “Media Capture and Streams” (2025), W3C (2024), “Vulkan Overview” (2026), MDN Web Docs (2026b), “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026), and gunnchOS3k (2026).

8.13

12. Glossary links

Terms introduced or relied on as formal vocabulary in this chapter should resolve in the living glossary registry. This section lists them for linking—not as a dump of free-standing essays.

Term	Role in this chapter
Display pipeline	Path to visible frames
Frame	One timed image update
Audio pipeline	Capture/render path for sound
Camera pipeline	Optics → sensor → processing → buffers
Sensor	Transducer from physical quantity to signal/data
Sampling	Discrete measurement of a continuous phenomenon
Compositor	Combines layers for presentation
GPU	Parallel processor often used for graphics/media
Permission	Policy gate for camera/microphone access
Stability Contract	Experience depends on hidden conditions staying acceptable
Representation	Digital media standing in for—not identical to—the world

Deeper entries and “not the same as” warnings live in the glossary network.

8.14 Figure references (embedded above; accessibility metadata)

All figures below are **conceptual** or **illustrative** as labeled. Source preference: editable SVG in the publication repository.

8.14.1 FIG-CH08-001 — Media sensorium map

- **Caption.** World sensing processing presentation perception.
- **Alt text.** Left-to-right conceptual map from physical world through capture and processing to display/speakers and human perception.
- **Text equivalent / reading order.** (1) Physical world → (2) Capture (cameras, mics, sensors) → (3) Process (ISP/codecs/apps/compositor/GPU) → (4) Present (frames, speakers) → (5) Human perception.
- **Status.** Conceptual.
- **Source.** Publication-owned original.

8.14.2 FIG-CH08-002 — Presentation timing and feel

- **Caption.** Illustrative steady versus uneven presentation feel.
- **Alt text.** Two timelines of frames; dashed boxes mark delayed or missing updates without numeric thresholds.
- **Status.** Illustrative teaching aid; not product benchmarks; frame-timing cite present without hitch thresholds (CLM-CH08-001 · SOURCE_IDENTIFIED).
- **Source.** Publication-owned original.

8.14.3 FIG-CH08-003 — Sampling continuous to discrete

- **Caption.** Continuous phenomenon compared with discrete samples.
- **Alt text.** Side-by-side conceptual plate: continuous wave versus stem samples labeled as pixels, PCM, or IMU readings.
- **Status.** Conceptual.
- **Source.** Publication-owned original.

8.15 Claim footnotes used in this chapter

Claim ID	Approved gist	Classification
CLM-CH08-002	Cameras/mics sample the world; digital media are representations	SOURCE_IDENTIFIED (w3c-mediacapture-streams-20251009)
CLM-CH08-003	Multiple media pipelines contend for CPU/GPU/memory/power	SOURCE_IDENTIFIED (tanenbaum-bos)
CLM-CH08-004	Wearable/camera Quartet sensing EVT remains PHYSICAL_PENDING	PHYSICAL_PENDING (src-hardware-quartet)

Claim ID	Approved gist	Classification
CLM-CH08-001	Displays present timed frames; missed deadlines can appear as hitching/tearing (qualitative)	SOURCE_IDENTIFIED (mdn-requestanimationframe, whatwg-html); no invented hitch thresholds

General statements about pipelines and sampling as teaching vocabulary are not rewritten as repository claims. Numbers in figures are illustrative unless a learner labels them measured.

End of Chapter 8 working manuscript draft.

Chapter 9

Chapter 9 — Power, Batteries, Thermals, and Mechanical Design

Status: WORKING_DRAFT_COMPLETE · **Chapter ID:** CH09

Author: Edmund Gunn, Jr.

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (human validation deferred until the full manuscript draft exists; this chapter is a working draft, not publication-ready).

9.1 1. The moment

You are on battery in a warm room. The same app that felt fine an hour ago—plugged in, cooler air, quieter fan—now hitches when you scroll, dims the display, or takes a beat longer before the next frame. Nothing in the icon tray says “software forgot how.” The code path you care about is still the code path. What changed is the **budget**: how much energy the device can deliver, how much heat it can shed, and what the operating system is willing to spend to keep the machine safe and usable.

Part II opened the device as electricity, compute, memory, and media. This chapter adds the constraint layer those chapters already hinted at: power, batteries, thermals, and mechanical design. Performance is not a permanent property of an app. It is what remains after energy and heat policies have taken their cut (The Linux Kernel Documentation 2026a).

The Device Quartet—Student 14.5-inch, Handheld Hybrid, DS-XL Coder, and Edge IO Wearables—appear here only as **research form-factor analogies**. Their watt and °C curves are **PHYSICAL_PENDING**; no shipping-SKU marketing language and no invented EVT telemetry belong in this chapter (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

9.2 2. What you notice

Before naming governors and heat sinks, notice the human contract that broke.

You expected the familiar app to feel familiar. Instead you notice hitching, delayed taps, a dimmer screen, a warmer case, a fan that works harder, or a battery percentage that falls faster than your intuition predicted. Sometimes the OS surfaces a battery or temperature cue. Sometimes it does not, and the experience just... softens.

Performance collapses when energy and thermal budgets tighten—even if the “same” software is running.

That sentence is the chapter’s first systems skill. The software did not necessarily regress. The **available performance** changed because power and thermal policy reduced clocks, limited radios, dimmed pixels, or deferred work (The Linux Kernel Documentation 2026a). Chapter 3 taught you to separate feel from blame; Chapter 6 and Chapter 7 taught competing work for CPU and memory. Here the competing force is the budget itself.

Optional commodity comparison (no specialized hardware): run a light local task while plugged in, then the same task on battery after the device has been warm for a while. Record only what you can see—battery mode, temperature warnings if any, qualitative smoothness—and label guesses as inference. Do not invent watt or °C numbers.

9.3 3. Exploded ecosystem

Energy does not appear inside the SoC by magic. It enters, converts, feeds loads, becomes heat, and leaves—or it accumulates until policy intervenes. Figure 9.1 is the first-minute map: energy in → conversion → loads → heat out, with a throttle feedback path. It is **conceptual**, not a measured board layout.

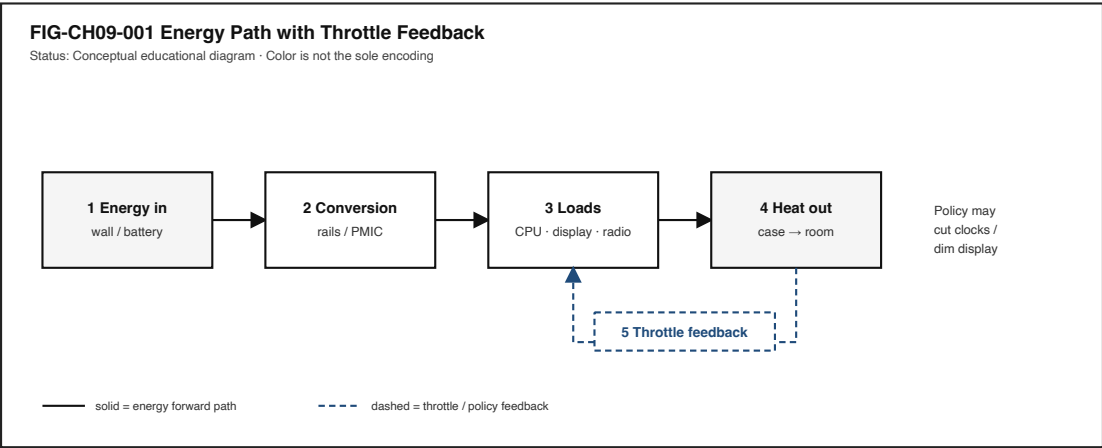


Figure 9.1: Energy path from source through conversion and loads to heat, with throttle feedback.

Walk the layers in ordinary language.

9.3.1 Human and room

Ambient temperature, clothing, lap vs desk, sunlight through a window—all change how easily heat leaves the case. The person’s body heat and the room are part of the thermal story, not decoration around it.

9.3.2 Energy sources

A wall adapter (or dock) and a **battery** are different sources. Plugged-in operation can often sustain higher continuous work. Battery operation draws from finite stored chemical energy with rate and aging constraints that everyday users experience as “how long until I need a charger,” not as an ideal constant-voltage lab supply. Formal cell and pack safety requirements for portable lithium systems are published in standards such as IEC 62133-2; household and commercial battery safety requirements appear in UL 2054 (International Electrotechnical Commission 2017; UL Standards & Engagement 2021). Those standards address safe operation and testing—not permission to abuse cells in a classroom.

9.3.3 Conversion and distribution

Regulators, power-management ICs, and rails convert and distribute energy to compute, display, radios, storage, audio, and sensors. Conversion is never perfectly efficient; some energy becomes heat in the conversion path itself.

9.3.4 Loads

CPU, GPU, display backlight or emissive panel, radios, storage, and sensors are the main spenders. Peak bursts can look fine for a moment; sustained feel is constrained by energy delivery and heat removal together (Patterson and Hennessy 2020b).

9.3.5 Sensors and policy

Temperature, current, and battery state-of-charge estimates inform OS and firmware **policy**. The policy may reduce clocks, limit frame rates, dim the display, or request shutdown. Linux documents CPU performance scaling as a first-class power-management concern (The Linux Kernel Documentation 2026a).

9.3.6 Mechanical enclosure

The case, hinges, seals, vents, materials, and button placement are **mechanical design**. They protect internals, shape heat paths, determine whether controls stay reachable, and affect repairability and durability. Qualitative mechanical discussion belongs here; measured Quartet enclosure EVT data does not (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Figure 9.2 opens the enclosure roles conceptually: protection, heat path, and human interface.

9.4 4. Follow the signal

Here the “signal” is energy and control, not a tap packet. Read the sequence as a logical story of budgets tightening—not as a claim that every device executes identical steps.

1. **Intent and workload.** You ask the device to do sustained interactive work (scroll, edit, render, sync).
2. **Energy request.** Loads draw current from the active source (adapter and/or battery).
3. **Conversion.** Power electronics deliver usable rails; some energy becomes heat in conversion.

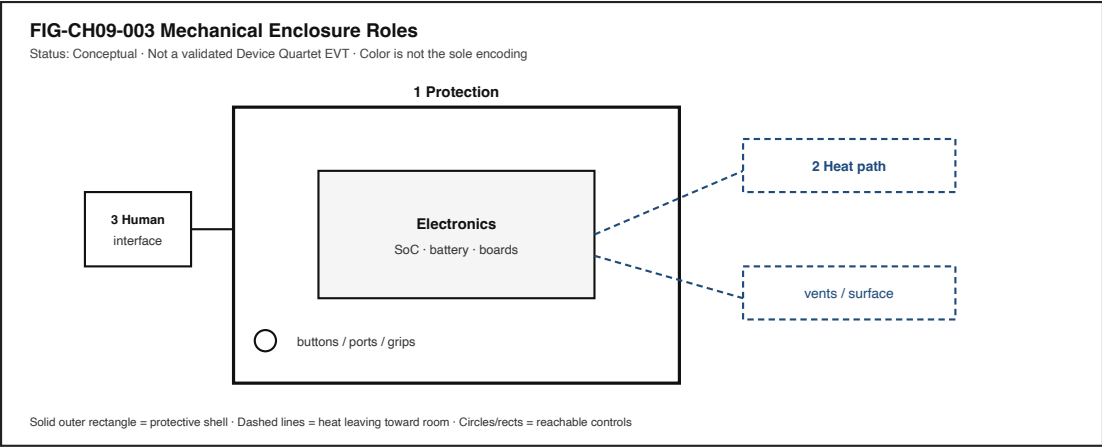


Figure 9.2: Exploded conceptual enclosure showing protection, heat path, and human-interface roles.

- 4. **Useful work.** Compute and I/O progress; displays and radios spend energy for the experience you notice.
- 5. **Heat generation.** Electrical work that is not stored or transmitted as useful signal becomes heat in silicon, boards, and batteries.
- 6. **Heat path.** Conduction, convection, and (sometimes) radiation move heat toward the case and room—or fail to keep up.
- 7. **Sensing.** Thermal and power sensors report state to firmware/OS policy.
- 8. **Budget check.** Policy compares demand to what is safe and sustainable on the current source and temperature.
- 9. **Throttle or dim (if needed).** Clocks drop, work is deferred, brightness falls, radios limit—**throttle** as deliberate protection (The Linux Kernel Documentation 2026a).
- 10. **Human perceives change.** Hitch, warmth, dimming, or battery cliff becomes the felt experience.
- 11. **Optional recovery.** Cooler ambient, lighter load, or returning to wall power can restore headroom—if nothing else is wrong.

9.4.1 Alternate paths (honesty rule)

Path	Everyday example	What changes
Plugged-in, cool	Desk use with charger	Higher sustained budget often available
Battery, cool	Travel use	Finite energy; rate limits still apply
Battery, warm	Warm room, heavy local app	Thermal policy may cut performance first
Low battery mode	OS power-saver	Explicit policy reduces spend
Display-heavy	Bright outdoor screen	Display can dominate energy share
Radio-heavy	Weak signal retries	Radios can dominate without “CPU busy”

Figure 9.3 contrasts on-charger vs on-battery feel as an **illustrative** teaching plate—no invented watts.

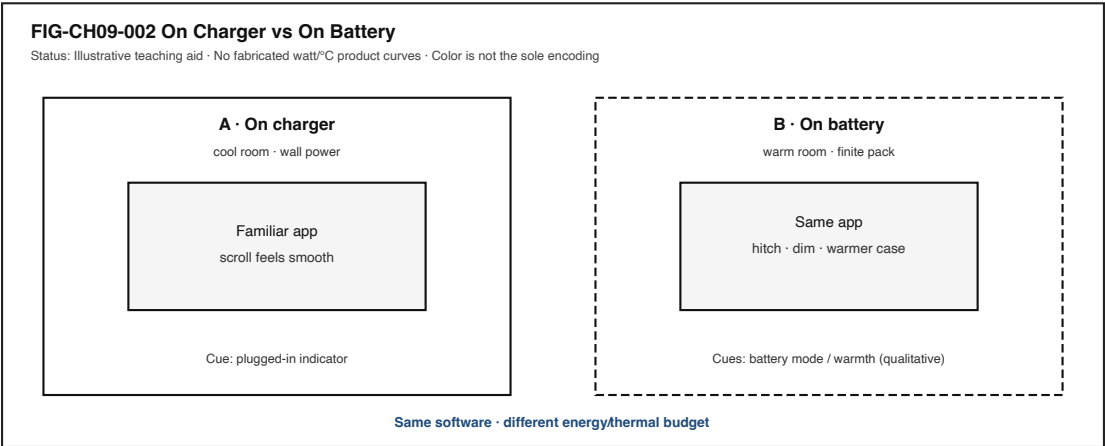


Figure 9.3: Side-by-side on-charger versus on-battery experience with shared app and different budgets.

9.5 5. Component cards

Component cards answer: What is it? What does it do for the person? What fails when it misbehaves?

9.5.1 Power budget

Plain definition. The sustainable energy delivery rate a design can keep without violating safety or thermal limits.

Experience benefit. Interactive work stays smooth when demand fits the budget.

Failure symptom. Hitching, dimming, unexpected slowdown, or shutdown under sustained load.

9.5.2 Battery

Plain definition. Stored chemical energy with capacity, rate, aging, and environmental constraints—not an infinite ideal source.

Experience benefit. Untethered use for a finite time.

Failure symptom. Rapid drain, unexpected power-off, swollen pack (stop using; seek proper disposal), or charging refusal. Do not open, puncture, crush, or heat cells as a “lab.”

9.5.3 Thermal limit

Plain definition. Heat must leave the system; excess heat forces reduced performance or protective shutdown.

Experience benefit. The device stays safe to hold and durable over time.

Failure symptom. Hot case, fan noise, throttle, thermal warnings, or shutdown.

9.5.4 Throttle

Plain definition. Deliberate reduction of performance to respect power, thermal, or safety limits.

Experience benefit. The machine survives the workload instead of damaging itself or cutting power abruptly.

Failure symptom. From the seat it can look like “the app got worse” even when policy is working as designed (The Linux Kernel Documentation 2026a).

9.5.5 Mechanical design

Plain definition. Enclosure, materials, hinges, seals, vents, and control placement that shape durability, heat, accessibility, and repair.

Experience benefit. The device stays usable, serviceable, and comfortable enough to hold.

Failure symptom. Flex that threatens boards, blocked vents, unreachable controls, or heat trapped against skin.

9.5.6 Energy vs peak performance

Plain definition. Short bursts can look fast; sustained feel is constrained by energy and heat together (Patterson and Hennessy 2020b).

Experience benefit. Honest expectations: demos at peak are not the same as all-day use.

Failure symptom. Benchmarks that look great for seconds, then collapse under continuous interactive load.

9.6 6. Stability contract

The **Stability Contract** returns with energy and heat as first-class conditions:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds—including power delivery and thermal headroom.

For this chapter, a sustainably good interactive experience may require all of the following to stay “good enough” at once:

- energy source adequate for the demanded load (adapter and/or battery state),
- conversion and rails intact,
- power budget not already exhausted by background work,
- thermal path able to shed heat into the room,
- policy still permitting needed clocks and display brightness,
- mechanical integrity preserved (no unsafe flex, blocked vents, or compromised packs),
- status cues available enough that the person can understand dimming or slowdown when platforms expose them,
- no unsafe lab or field procedure that defeats protections.

Three separations matter:

1. The app can be “the same binary” while **available performance** has already fallen.
2. The network can look fine while **thermal throttle** is the real villain (link back to CE-3 / Chapter 3 local-lag thinking).
3. A warm case is **evidence of heat**, not automatically a measured °C root-cause report—observe, then infer carefully.

Device Quartet watt/°C Stability Contract numbers remain **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Commodity-device observation is enough for this chapter’s lab.

9.7 7. Try it

9.7.1 LAB-PWR-001 — Budget Collapse Observation

Observable question. What visible battery-mode and thermal cues appear when I compare a light local workload with a heavier local workload on a device I already own—without unsafe heating or battery abuse?

WAIKE alignment note. WAIKE (accepted main, SHA e97e74fc9bfb44b1cdc26b272dc4848264f15f) includes adjacent labs such as `HARDWARE_ENGINEERING / lab_power_budget` and `EMBEDDED_PROTOTYPING / lab_ep_sleep_mode`. Those are **neighbors**, not renamed IDs. **LAB-PWR-001** is a **publication-owned commodity observation lab**.

Prerequisites. A phone, tablet, or laptop you may use for learning; ability to see battery percentage / power mode; optional OS battery or thermal status pages where present.

Safety (mandatory).

- Do **not** open, puncture, crush, overcharge, freeze, microwave, or externally heat batteries or packs.
- Do **not** block vents with blankets to “force throttle,” run devices in ovens, or defeat thermal protections.
- Stop if the device warns, smells unusual, swells, or becomes painfully hot.
- Prefer ambient room conditions; no outdoor asphalt “bake tests.”
- Do not capture passwords, tokens, or private content in screenshots.

Time estimate. About 45–75 minutes including write-up.

No-specialized-hardware route. Required. Simulation/fixture fallback is provided under `labs/LAB-PWR-001/fixtures/`.

9.7.1.1 Prediction

Write one sentence: do you expect the heavier local workload to change smoothness, warmth, battery drop, or OS cues first?

9.7.1.2 Route A — Commodity device (baseline)

1. Note device class, OS name, plugged vs battery, and approximate room comfort (cool / comfortable / warm)—no invented °C.
2. Run a **light** local task for a few minutes (static document, idle desktop, or the fixture “light” scenario).

3. Run a **heavier** local task (local video encode preview, large local spreadsheet recalc, or local photo filter)—still without network dependence if you can avoid it.
4. Record visible cues: battery mode, brightness changes, fan behavior if any, stutter, OS warnings.
5. Label each row **observed** vs **inferred**.

9.7.1.3 Route B — OS status pages (optional)

Where the platform exposes battery or energy pages, note qualitative fields only. Do not claim vendor-private telemetry you cannot see.

9.7.1.4 Route C — Offline fixture (required fallback)

Open `labs/LAB-PWR-001/fixtures/budget_collapse_card.md` and complete the observation table using the labeled **fixture** / **illustrative** scenarios when a personal device is unavailable or shared-lab policy forbids sustained loads.

9.7.1.5 Evidence (minimum)

- one scrubbed screenshot or written status list,
- a two-condition observation table (light vs heavier),
- one paragraph separating observation from inference,
- explicit safety confirmation: no battery abuse, no forced overheating.

9.7.1.6 Limits (say them out loud)

- Feeling warmer knowing a precise junction temperature.
- One session is not a product thermal qualification.
- Device Quartet EVT curves are **PHYSICAL_PENDING**; your commodity notes do not fill that gap (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).
- OS cues are platform-specific.

9.7.1.7 Portfolio output

A small folder with README (question, method, limits), observation table, one evidence artifact, reflection, and a teach-back paragraph for a nontechnical reader.

9.8 8. Build it

Use the energy story at the depth that matches your pathway.

9.8.1 Explorer

Draw the path wall/battery → conversion → one load you care about (display or CPU) → heat → possible throttle. Use ordinary words.

9.8.2 Operator

Compare plugged-in vs battery notes from LAB-PWR-001. List confounders: brightness, background apps, room warmth, network retries.

9.8.3 Builder

Build a one-page **power path map** (paper or SVG) with nodes and a feedback arrow for throttle. Mark every number either absent or labeled illustrative—never invent watts.

9.8.4 Engineer

Propose a diagnosis tree: “feels slow on battery” → check power mode → check thermal cues → check competing processes → check display brightness → only then suspect application regression. Cite where OS performance scaling concepts enter the tree (The Linux Kernel Documentation 2026a).

9.8.5 Researcher

Write a **PHYSICAL_PENDING** measurement plan for a future Device Quartet thermal/battery campaign: instruments, ambient controls, workload definition, stop criteria, and what would still be out of scope. Do not fabricate results (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Educators can run Route C fixtures when shared devices cannot sustain loads, and can emphasize the safety non-goals as the first learning outcome.

9.9 9. Secure and include it

9.9.1 Safety

Portable batteries are energy-dense. Classroom and field rules:

- no teardown, puncture, crush, or improvised heating,
- respect manufacturer charging accessories when required,
- stop on swelling, odor, or thermal alarms,
- dispose of damaged packs through proper channels—not general trash experiments.

IEC 62133-2 states safety requirements and tests for portable sealed secondary lithium cells and batteries under intended use and reasonably foreseeable misuse (International Electrotechnical Commission 2017). UL 2054 covers household and commercial batteries with requirements intended to reduce fire and explosion risk in product use and related handling (UL Standards & Engagement 2021). Citing those standards is not a license to reproduce abusive tests outside accredited labs.

9.9.2 Security

Power and thermal side channels exist in advanced threat models; this chapter does not teach exploitation. Practical posture: keep firmware/OS updates that include battery and thermal fixes; do not disable safety services “for benchmarks.”

9.9.3 Privacy

Battery and location-correlated telemetry can become sensitive when logged. LAB-PWR-001 artifacts must scrub identifiers and avoid capturing personal screen content.

9.9.4 Accessibility

Thermal and battery status must be perceivable: not color-only icons, not only a subtle case-warmth cue. Dimming and low-power modes should remain navigable with keyboard, switch, or screen-reader paths where the platform supports them. Mechanical design that hides critical controls behind heat-swollen gaps or unreachable flaps fails inclusion as well as ergonomics.

9.9.5 Equity

Charger access, device age, and hot classrooms are not evenly distributed. Designs and lessons that assume unlimited wall power and cool offices silently exclude learners. Local-first honesty about throttle and dimming matters more when batteries and rooms are constrained.

9.10 10. Career lens

Energy and mechanics cross many ownership domains. No table promises employment; roles vary by organization.

Layer	Example role	Professional artifact	Lab resemblance
Power architecture	Power electronics / PMIC engineer	Rail budget and efficiency notes	Builder power-path map
Battery systems	Battery systems engineer	Pack safety and BMS requirements	Safety non-goals checklist; standards awareness
Thermal	Thermal engineer	Heat-path analysis (measured in real programs)	Qualitative heat-path narrative; PHYSICAL_PENDING plan
Mechanical / ID	Mechanical or industrial designer	Enclosure and DFM package	Enclosure-roles sketch from Figure 9.2
Reliability	Reliability engineer	Stress and lifetime test plan	Stop criteria and abuse prohibitions
OS power	Systems / OS engineer	Governor and power-policy changes	Operator cue log; (The Linux Kernel Documentation 2026a) reading
Performance	Performance engineer	Throttle-aware profiles	Light vs heavy observation table
Field / IT	Field or support engineer	Triage: power → thermal → app	Diagnosis tree from Section 8
Accessibility	Accessibility specialist	Status-cue perceivability review	Color-independent cue notes

Portfolio evidence from LAB-PWR-001 looks like early professional habit: prediction, observation/inference split, safety limits, and refusal to invent product curves.

9.11 11. Check understanding

Answer with reasoning. Prefer short paragraphs over one-word guesses.

1. Why might the same app hitch on battery in a warm room after feeling fine on a charger?
2. What is the difference between peak burst performance and sustained interactive feel?
3. Why is “the CPU percentage is high” incomplete evidence that software alone regressed?
4. What does throttle protect, and how can correct throttle still feel like failure to a person?
5. Name two mechanical design choices that can affect heat or accessibility without changing application code.
6. Why are Device Quartet watt/°C curves labeled PHYSICAL_PENDING in this book?
7. List three lab actions that are explicitly forbidden in LAB-PWR-001 and why.
8. **Teach-back.** Explain to a family member—without saying *governor*, *junction*, or *PMIC*—why a warm phone on battery can feel slower. Then introduce those three terms one at a time, tied to something already understood.

Educator note: success is causal sequence (energy → heat → policy → feel) plus at least one safety boundary, not a parts catalog.

9.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Project-specific Device Quartet physical evidence remains **PHYSICAL_PENDING** and is cited via (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026), separately from external literature.

Inline citations used in this chapter include The Linux Kernel Documentation (2026a), Patterson and Hennessy (2020b), International Electrotechnical Commission (2017), UL Standards & Engagement (2021), and “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026).

Omitted as cited technical claims (evidence gaps remaining in the chapter claim plan): detailed battery electrochemistry textbook pinning for “non-ideal voltage source” formalization, and a pinned mechanical/industrial-design textbook edition for mechanical theory. Prose above treats those topics qualitatively or via verified safety standards only—no invented ISBNs.

9.13 12. Glossary links

Terms introduced or relied on as formal vocabulary in this chapter should resolve in the living glossary registry as candidates mature. This section lists them for linking—not as a dump of free-standing encyclopedia entries.

Term	Role in this chapter
Power budget	Sustainable energy delivery rate for the experience
Battery	Stored chemical energy with capacity/rate/aging constraints
Thermal limit	Heat constraint that forces reduced performance or shutdown

Term	Role in this chapter
Throttle	Deliberate performance reduction to respect limits
Mechanical design	Enclosure/materials/controls shaping durability, heat, UX
Energy vs peak performance	Bursts possible; sustained feel constrained by energy/heat
Stability Contract	Experience depends on hidden conditions staying acceptable
Conversion / regulation	Turning source energy into usable rails (with losses)
Heat path	How heat moves from silicon toward case and room
Status cue	Battery/thermal signal the person can perceive

Deeper entries and “not the same as” warnings live in the glossary network. Prefer following a link when stuck over inventing a private definition.

9.14 Figure references (embedded above; accessibility metadata)

All figures below are **conceptual** or **illustrative** unless a future revision cites validated hardware evidence. Device Quartet measured curves remain PHYSICAL_PENDING.

9.14.1 FIG-CH09-001 — Energy Path with Throttle Feedback

- **Caption.** Energy in → conversion → loads → heat out, with throttle feedback.
- **Alt text.** Flow diagram from energy source through conversion and loads to heat, plus a feedback arrow labeled throttle.
- **Text equivalent / reading order.** (1) Energy in → (2) Conversion → (3) Loads → (4) Heat out → (5) Throttle feedback to loads/policy.
- **Status.** Conceptual educational diagram.
- **Source.** Publication-owned original.

9.14.2 FIG-CH09-002 — On Charger vs On Battery

- **Caption.** Compare-and-choose plate for feel on charger versus on battery without invented watts.
- **Alt text.** Two columns sharing the same app; left plugged-in/cool, right battery/warm with hitch and dim cues.
- **Status.** Illustrative teaching aid.
- **Source.** Publication-owned original.

9.14.3 FIG-CH09-003 — Enclosure Roles

- **Caption.** Mechanical enclosure roles: protection, heat path, and human interface.

- **Alt text.** Exploded conceptual enclosure with three labeled roles.
- **Status.** Conceptual; not a validated EVT.
- **Source.** Publication-owned original.

9.15 Claim footnotes used in this chapter

Claim ID	Approved gist	Classification
CLM-CH09-001	Interactive devices operate under finite power/thermal budgets that can reduce available performance	general_technical · SOURCE_IDENTIFIED via The Linux Kernel Documentation (2026a)
CLM-CH09-002	Batteries as non-ideal finite sources	SOURCE_IDENTIFIED (iec-62133-2, ul-2054); safety posture only
CLM-CH09-003	Mechanical design affects thermal/durability/ally/repairability	ILLUSTRATIVE_ONLY (no pinned industrial-design textbook this wave)
CLM-CH09-004	Device Quartet thermal/battery EVT curves	PHYSICAL_PENDING via “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026)

General statements about heat needing a path out of a closed system are treated as ordinary physical reasoning and are not rewritten as repository claims. Any future numeric watt/°C figures must carry **illustrative**, **measured**, or **inferred** labels—and Quartet measured figures stay blocked until PHYSICAL_PENDING clears.

Chapter 10

Chapter 10 — Ports, Buses, Boards, Packaging, and Manufacturing

Status: draft · **Chapter ID:** CH10

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

10.1 1. The moment

You plug in a cable. Or you seat a card. Or you notice the seam lines on a phone—the thin edges where plastic, glass, and metal meet around a port you use every day.

Nothing about that moment feels like a lecture on manufacturing. It feels like a habit: find the connector, feel the orientation, push until it seats, wait for a light, a sound, a battery icon, or a polite refusal. Sometimes the accessory works immediately. Sometimes the host says “accessory not supported.” Sometimes nothing happens, and you wiggle the cable even though you know wiggling is not a diagnosis.

From your seat, it feels like one action: *connect*.

Underneath that feeling, several kinds of agreement have to hold at once. A **port** presents a physical face and often a power path. A **bus** may share wires and rules among more than two chips. A **board**—usually a printed circuit board (PCB)—organizes parts and traces so those agreements can be manufactured again and again. **Packaging** encloses chips and modules so they survive heat, handling, and everyday drops. **Manufacturing** and design-for-manufacturing (DFM) constraints decide which beautiful diagrams can become reproducible products.

This chapter is the close of Part II’s hardware path: not a connector encyclopedia, and not a factory tour with invented yield percentages. It follows the teaching model of this book—**human experience** → **system** → **component**—until ports and buses stop looking like magic sockets and start looking like contracts you can name, inspect carefully, and refuse to overclaim.

The governing question:

When I plug something in, what must already agree—electrically, mechanically, and in protocol—for the experience to continue?

10.2 2. What you notice

Before names like *protocol* or *DFM* enter, notice the human contract you already expect.

You expect the connector to fit without force. You expect orientation cues—shape, icons, or feel—to matter. You expect power delivery (when the cable is for charging) not to surprise you with heat that feels unsafe. You expect data (when the cable is for files or displays) either to appear soon enough to trust, or to fail in a way a person can understand. You expect the same port not to destroy an accessory when versions differ. You also expect the device’s seams and enclosure to keep dust and fingers away from parts that should stay sealed.

Those expectations are not decorations around hardware. They *are* the product, from the person’s point of view.

A successful plug-in is a human perception produced by stacked agreements: mechanical seat, electrical integrity, protocol meaning, and packaging that still protects the board.

Notice the split timelines. Mechanical seating can succeed while protocol negotiation is still pending. A host can report “charging” while a data path stays idle. A cable can look identical to last year’s cable and still implement different roles. Commodity labels on packaging are marketing summaries; they are not laboratory measurements of signal integrity.

Optional comparison, available on almost any device you already own: plug a familiar cable until it seats, then unplug and reseal with the same care. Then try a second familiar path (headphones jack if present, a different USB-style cable you already own, a charger brick you already trust). Do not force anything. The point is not to stress-test hardware. The point is to notice that “connected” is a family of experiences—power, data, both, or neither—and that outside observation alone rarely proves *why*.

10.3 3. Exploded ecosystem

A plug-in is not a single object. It is a path through an ecosystem. **FIG-CH10-001** is the first-minute map: chips and modules on a board shared buses ports the external world of cables and accessories. Treat it as **Representative educational architecture**, not a claim that any sealed phone or laptop looks exactly like the diagram inside.

Walk the layers in ordinary language, then keep the same layers when vocabulary deepens.

10.3.1 Human

You form intent: charge this device, copy a file, attach a keyboard, dock a display. Hands find a cable. Eyes check icons. Later, skin and eyes judge heat, light, and whether the right function arrived.

10.3.2 Port (the face you can see)

A **port** is a user- or system-facing interconnect for power and/or data. It includes connector geometry, contact materials, and often nearby protection and detection parts. The port is the experience’s front door—not the whole house (Patterson and Hennessy 2020b).

10.3.3 Cable / accessory packaging

The cable jacket, connector shell, strain relief, and accessory enclosure are packaging you can see. They shape durability, grip, and how hard it is to plug in upside down. They do not, by themselves, define the protocol that travels inside.

10.3.4 Board (PCB)

Inside the sealed host, a **PCB** organizes components and copper traces. Layout choices affect noise, timing margins, heat spreading, cost, and whether a design can be assembled reliably (Patterson and Hennessy 2020b). You usually cannot see that layout without opening the device—and this chapter’s labs do **not** ask you to open sealed devices.

10.3.5 Bus and protocol-on-a-wire

A **bus** is a shared interconnect with electrical conventions *and* communication rules. Connectors alone are not the whole story: the same physical pins can carry different meanings depending on negotiation and mode (Patterson and Hennessy 2020b). “Protocol on a wire” names that agreement layer without pretending one chapter can catalog every standard.

10.3.6 Packaging (chip / module / product)

Packaging here means how chips, modules, and whole devices are enclosed and interfaced mechanically and thermally—not product marketing packages on a store shelf. Chip packages connect silicon to boards; product packages protect boards from the world Chapter 9 already framed mechanically.

10.3.7 Manufacturing / DFM

Manufacturing turns designs into reproducible physical goods. **DFM** is the set of constraints and practices that keep assembly, test, and reliability within intended bounds. Qualitative literacy belongs here: clearance, fiducials, test access, thermal paths, and process capability matter. **Invented factory yield statistics do not.** Device Quartet board, EVT, and manufacturing evidence for gunnchOS learning form factors remains **PHYSICAL_PENDING** where not documented (CLM-0003; CLM-CH10-004).

10.3.8 System software (preview)

Firmware and operating-system drivers eventually interpret enumeration, power roles, and device classes. Part III will deepen boot and trust. Here, keep the honesty rule: a host UI message is an interpretation, not a complete bus trace.

FIG-CH10-004 later sketches packaging/manufacturing stages as a teaching sequence—not as Quartet EVT evidence.

10.4 4. Follow the signal

FIG-CH10-002 shows a numbered path: host request → bus transaction → device response, with failure branches. Read it as a logical story, not as a claim that every commodity cable executes exactly these steps with no overlap.

1. **Intent and seating.** A person aligns the connector and seats it. Mechanical incomplete seating can look identical to a protocol failure from the outside.
2. **Detection / presence.** Host-side circuits may notice attachment (or not). Absence of a UI change is an observation, not a root cause.
3. **Power domain agreement (when applicable).** Voltage, current limits, and role (source vs sink) must stay within intended bounds. Unsafe electrical experiments are forbidden in this book’s labs.
4. **Electrical link.** Contacts and traces must provide a usable path for the signals the protocol expects. Noise, damage, and contamination can break that path without changing the plastic shape of the connector.
5. **Protocol negotiation.** Devices exchange identity, capabilities, or mode selection according to agreed rules. A bus typically combines electrical conventions and a communication protocol (Patterson and Hennessy 2020b).
6. **Transaction.** The host requests; the device responds—or times out. **FIG-CH10-003** separates the electrical layer from the protocol layer on one interconnect so readers stop treating “the wire” as one idea.
7. **Software interpretation.** Drivers and services translate bus events into charging icons, mount dialogs, or error strings (Tanenbaum and Bos 2022).
8. **Human feedback.** Light, sound, haptics, or on-screen text close the loop—or fail to.

10.4.1 Alternate paths (the honesty rule)

Not every port carries a shared multi-device bus. Some links are point-to-point. Not every cable carries data. Some “connections” are power-only. Some accessories authenticate or refuse roles. Teaching those forks prevents the encyclopedia trap: listing every brand name instead of tracing one honest path.

10.4.2 Failure branch without drama

When something fails, prefer failure *domains* over confident blame:

- Mechanical seat / cable damage (still only a hypothesis until inspected safely)
- Power-role mismatch
- Protocol / driver mismatch
- Host policy (“accessory not supported”)
- Board or packaging damage you cannot see without opening the device

Outside observation rarely distinguishes those cleanly. That limitation is a feature of literacy, not a bug in the reader.

10.5 5. Component cards

For each object: plain language, analogy, technical function, constraints, common symptoms. Analogies are labeled as analogies.

10.5.1 Port

- **Plain language.** The opening and connector face where power and/or data enter or leave.
- **Analogy (labeled).** Like a doorway with a specific keyhole shape—useful, but not the hallway behind it.
- **Technical function.** Provides mechanical mating, contact paths, and often sensing/protection near the edge of a product (Patterson and Hennessy 2020b).
- **Constraints.** Wear, contamination, orientation, rated cycles, and nearby mechanical stress.
- **Symptoms.** Intermittent connect, needs reseating, works only at an angle, host never notices attachment.

10.5.2 Bus

- **Plain language.** A shared set of wires (or lanes) plus rules for meaningful exchange.
- **Analogy (labeled).** Like a shared conversation channel with turn-taking rules—not merely the microphone hardware.
- **Technical function.** Combines electrical conventions with a communication protocol so more than two parties can coordinate, or so two parties can reuse a standardized contract (Patterson and Hennessy 2020b).
- **Constraints.** Timing, contention, address space, topology, and electromagnetic limits (survey depth only here).
- **Symptoms.** Partial function (charge works, data does not), enumeration loops, “unknown device,” flaky accessories that work on another host.

10.5.3 Board (PCB)

- **Plain language.** The physical substrate that holds parts and connects them with copper.
- **Analogy (labeled).** Like a city’s street grid and building lots—layout shapes traffic even when buildings look similar.
- **Technical function.** Places components, routes signals and power, and provides mechanical/thermal structure for assembly (Patterson and Hennessy 2020b).
- **Constraints.** Layer count, clearances, impedance control (when required), heat, test access, and DFM rules.
- **Symptoms (usually invisible from outside).** Intermittent behavior after drops, heat-localized failures, manufacturing escapes—claim only with evidence; do not invent.

10.5.4 Packaging

- **Plain language.** How chips, modules, and devices are enclosed and interfaced.
- **Analogy (labeled).** Like a protective case that also has to let heat and connectors through in the right places.
- **Technical function.** Mechanical protection, pinout to the board, thermal paths, and sometimes shielding.
- **Constraints.** Size, warpage, moisture sensitivity, rework limits, and accessibility of ports/controls.
- **Symptoms.** Cracked shells, pushed-in ports, overheating under ordinary load (observe safely; do not induce heat).

10.5.5 Manufacturing / DFM

- **Plain language.** The processes and design constraints that make a design reproducible.
- **Analogy (labeled).** Like a kitchen recipe that must work for many cooks—not a single perfect plate in a demo.
- **Technical function.** Assembly, inspection, test, and process controls that keep variation within intended bounds (Patterson and Hennessy 2020b).
- **Constraints.** Tooling, tolerances, supply parts, test coverage, cost, and schedule.
- **Symptoms (organizational, not lab numbers).** Field failures clustering on one revision, hard-to-assemble features, missing test points. **Do not invent yield percentages.** Quartet fabrication evidence stays **PHYSICAL_PENDING** (CLM-0003; CLM-CH10-004).

10.5.6 Protocol on a wire

- **Plain language.** Agreed rules that turn electrical activity into meaningful messages.
- **Analogy (labeled).** Like grammar shared by speakers—voltage alone is not vocabulary.
- **Technical function.** Framing, addressing, commands, errors, and often mode negotiation atop the electrical layer (Patterson and Hennessy 2020b).
- **Constraints.** Compatibility windows, versioning, authentication policies, and host stack support (Tanenbaum and Bos 2022).
- **Symptoms.** Connected physically but silent logically; works on one OS version; accessory lights up but never mounts.

10.6 6. Stability contract

A plug-in experience continues only while multiple hidden conditions stay within acceptable bounds.

Interconnects deliver power and/or data with agreed protocols; failures should be detectable enough for a person or operator to notice; packaging should protect enough for intended use. Those statements are **qualitative**. They are not a promise that every commodity accessory meets every standard in every environment, and they are not a table of factory yields.

For the ordinary “I plugged it in and it worked” feeling, conditions like these must hold together:

1. **Mechanical seat** within the connector’s intended engagement.
2. **Electrical integrity** good enough for the protocol’s needs (contacts, cable, board path).
3. **Power agreement** inside safe, intended roles and limits.
4. **Protocol agreement** sufficient to select a mode and exchange meaning.
5. **Software mediation** that translates bus reality into honest-enough user feedback.
6. **Packaging integrity** that keeps strain, debris, and handling from silently breaking the path.
7. **Thermal / mechanical environment** still inside the product’s intended use (Chapter 9 adjacency).

A system can remain *physically* plugged while the human experience has already failed: charging icon absent, data silent, error vague, heat worrying. Conversely, a host can look “connected” while a deeper transaction is still pending. Stability is concurrent conditions—not a single green LED.

Honesty bound for this edition: Device Quartet PCB, EVT, and manufacturing measurements are not claimed as completed physical evidence (CLM-0003; CLM-CH10-004). Commodity observation in **LAB-BUS-001** produces *your* evidence for *your* cable and host—not a universal score and not a yield statistic.

10.7 7. Try it

10.7.1 LAB-BUS-001 — Name the Interconnect

Goal. Diagram a commodity cable or accessory path as host port bus/protocol device, using only safe observation—or an offline fixture if you have no hardware.

WAIKE alignment note. WAIKE accepted **main** includes adjacent digital_rc labs such as hardware bus-protocol and PCB rule-checking practice, plus embedded I²C/SPI neighbors. Those are competency adjacencies. They are **not** renamed as publication lab IDs. **LAB-BUS-001** is publication-owned.

Safety (hard stops).

- Do not open sealed devices.
- Do not cut, strip, or probe cables.
- Do not force connectors or use damaged/wet cables.
- No soldering, no bench supplies, no battery abuse, no intentional overheating.
- No Device Quartet fabrication required or requested.

Routes.

- **Commodity route.** One undamaged cable/accessory you already own + host you already use.
- **Offline fixture route.** Use the lab’s commodity path card and worksheet without hardware.

Explorer baseline (about 40 minutes).

1. Predict the path layers before you plug (or before you read the fixture card).
2. Seat the cable normally once—or study the fixture diagram.
3. Record observations only: seat feel, lights, on-screen text, coarse wall-clock timing.
4. Draw a labeled interconnect map with at least: human → port → (power and/or data) → bus/protocol (named or “unknown shared rules”) → device function.
5. Fill an observation-vs-inference table. Every causal guess needs a note about extra evidence required.

Operator extension. Compare two familiar paths (for example charge-only vs charge+data on cables you already trust). Do not conclude root cause from UI text alone.

Builder extension. Turn your map into a clean one-page teach-back another student could follow without opening any device.

Engineer extension. List which failure domains your observations *cannot* distinguish, and what non-destructive next evidence would be required (host logs if available without elevating privilege unsafely; alternate known-good cable; different host). Still no disassembly.

Researcher extension. State a hypothesis about one flaky accessory behavior, name variables you could ethically vary, and list limitations. Explicitly forbid inventing manufacturing yields or Quartet EVT numbers.

Evidence to keep. Interconnect path diagram; observation-vs-inference table; scrubbed notes; teach-back paragraph. Prefer synthetic descriptions over photos of other people’s devices.

10.8 8. Build it

Extend LAB-BUS-001 without turning Part II into a parts catalog.

10.8.1 Explorer

Build a pocket card: five layers (port, bus/protocol, board, packaging, manufacturing literacy) with one plain sentence each.

10.8.2 Operator

Build a “flaky accessory” checklist that starts with mechanical seat and host message text, and ends with “needs more evidence”—never with a fake certainty.

10.8.3 Builder

Build a labeled diagram (paper or digital) of host bus/port device for one commodity path. Annotate what is visible vs sealed. Optional: add a second diagram for a point-to-point link vs a shared bus, still at survey depth.

10.8.4 Engineer

Build a one-page DFM/packaging constraint list *qualitative only*: test access, connector strain relief, thermal keep-outs, assembly clearances. Mark each item “general literacy” vs “would need product-specific evidence.” Cite textbook framing for interconnect and organization concepts rather than inventing process capability indices (Patterson and Hennessy 2020b).

10.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to make yet—for example, “this revision’s port failures are manufacturing escapes.” Specify what PCB/EVT/test artifacts would be required, and keep Quartet physical claims **PHYSICAL_PENDING** (CLM-0003; CLM-CH10-004).

Educators can facilitate teach-backs from Section 11 and adapt LAB-BUS-001 for classrooms that must stay on the offline fixture route.

10.9 9. Secure and include it

10.9.1 Security

Ports are physical attack surface as well as convenience. An open port can accept malicious accessories, unexpected power roles, or unwanted data paths. Survey literacy here foreshadows later trust chapters: treat unknown accessories with the same caution you would give unknown files. Prefer host policies and user consent over blind trust in a connector shape. Classic

protection thinking still applies when physical interfaces mediate access to information systems (Tanenbaum and Bos 2022).

10.9.2 Privacy

Device names, serials, and accessory logs can become identifiers. LAB-BUS-001 artifacts must not capture account tokens or photos of classmates’ screens without consent. Scrub before portfolio save.

10.9.3 Accessibility

Physical connectors and controls are accessibility surfaces. Orientation cues should not depend on color alone; force and dexterity requirements exclude some users; alternate input paths (wireless, assistive switches) matter when a port is hard to use. Document seating steps in words, not only pictures. Where digital status text accompanies connection state, clear language helps more than color-only icons.

10.9.4 Equity

Accessory costs, proprietary connector eras, and “buy our cable” lock-in shape who can participate fully. Teaching interconnect literacy includes naming those cost barriers without pretending every classroom can stock every adapter.

10.9.5 Safety

Power and batteries remain Chapter 9’s hard boundary: no abuse, no puncture, no improvised heating. Manufacturing literacy does not require operating industrial equipment in a reading chapter.

10.9.6 Ethics

Do not claim factory yields, sealed teardowns you did not perform, or Quartet EVT results you do not have. Overclaiming manufacturing success is still a form of false evidence.

10.10 10. Career lens

One cable crosses many ownership domains. No table promises employment; roles vary by organization. LAB-BUS-001 artifacts resemble early professional evidence in miniature: labeled diagrams, observation discipline, and explicit uncertainty.

Role lens	Typical artifacts	Review questions
Hardware / board design	Schematics, PCB layouts, constraint sets	Can this be assembled and tested?
Signal / interconnect	Interface diagrams, timing budgets (when evidenced)	Electrical vs protocol ownership clear?
Manufacturing / test	Work instructions, ICT/FCT plans, DFM checklists	What escapes would we catch?
Embedded bring-up	Board bring-up notes, bus traces when authorized	What failed first: power, clock, or protocol?

Role lens	Typical artifacts	Review questions
Reliability / quality	Failure analysis with measured evidence	Are we guessing from outside symptoms?
Industrial / product design	Enclosure, port placement, strain relief	Can diverse hands use this safely?

Portfolio hint: a scrubbed interconnect map plus an observation-vs-inference table is more honest than a vibes-based “the cable is bad” claim.

10.11 11. Check understanding

Concept. In one sentence each, define *port*, *bus*, and *PCB* so that none of them swallows the other two.

System tracing. Trace a familiar charge-or-sync cable from hand to host feedback in numbered steps. Mark which steps you observed and which you inferred.

Misconception check. Why is “the connector is the bus” incomplete? What layer does that sentence erase?

Misconception check. Why must this chapter refuse invented manufacturing yield numbers even when discussing DFM?

Teach-it-back. Explain to a newcomer—using only LAB-BUS-001 vocabulary—why a cable can seat perfectly and still fail to transfer files.

Researcher prompt. What evidence would convert a PHYSICAL_PENDING Quartet board/EVT claim into a documented physical claim? What remains out of scope for a commodity classroom lab?

10.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Project-specific Device Quartet fabrication status remains in `evidence/claim_registry.yaml` (for example, CLM-0003) and in the chapter claim plan (CLM-CH10-004), separately from external literature.

Inline citations used in this chapter include Patterson and Hennessy (2020b) and Tanenbaum and Bos (2022).

10.13 12. Glossary links

Candidate terms introduced or reinforced here (see also chapter glossary candidates; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Port	Interconnect face for power and/or data
Bus	Shared interconnect with electrical + protocol rules
PCB	Board organizing components and traces
Packaging	Enclosure/interface of chips, modules, or devices
DFM	Design-for-manufacturing constraints and practices
Protocol on a wire	Agreed rules for meaningful exchange over a physical interconnect
Stability contract	Concurrent conditions that keep the plug-in experience alive

Related earlier chapters: signals and logic (CH05), mechanical/thermal adjacency (CH09). Related later chapters: firmware/boot/trust (CH11), productization (CH29).

10.14 Figure references (planned embeds; accessibility metadata)

All four figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured evidence. No fabricated telemetry.

10.14.1 FIG-CH10-001 — Board buses ports world

- **Type.** System map.
- **Reader should notice.** Experience-first left-to-right (or layered) path from chips/modules through buses and ports to cables/accessories.
- **Truth class.** Conceptual / Representative educational architecture.
- **Alt text requirement.** List nodes in reading order; state conceptual truth class; do not imply Quartet EVT.

10.14.2 FIG-CH10-002 — Host request → bus transaction → device response

- **Type.** Sequence.
- **Reader should notice.** Numbered steps plus a failure branch; optional software-interpretation step.
- **Truth class.** Illustrative.
- **Alt text requirement.** Enumerate steps in order; label failure branch; state illustrative truth class.

10.14.3 FIG-CH10-003 — Electrical layer vs protocol layer

- **Type.** Comparative.
- **Reader should notice.** Same interconnect drawn once as contacts/energy path and once as rules/messages.

- **Truth class.** Conceptual.
- **Alt text requirement.** Name both layers; state that connectors alone are incomplete.

10.14.4 FIG-CH10-004 — Packaging / manufacturing stages (conceptual)

- **Type.** Exploded / staged sequence.
 - **Reader should notice.** Design → board assembly ideas → package/protect → test/ship literacy—not Quartet factory data.
 - **Truth class.** Conceptual.
 - **Alt text requirement.** List stages; explicitly deny invented yields; mark PHYSICAL_PENDING for Quartet fabrication claims.
-

10.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH10-001 / CLM-CH10-002 / CLM-CH10-003.** General interconnect, bus-as-electrical-plus-protocol, and qualitative PCB/DFM literacy—framed with textbook survey depth (Patterson and Hennessy 2020b).
- **CLM-CH10-004 / CLM-0003.** Device Quartet board/EVT/manufacturing evidence remains **PHYSICAL_PENDING** where not documented. No shipping-SKU language; no fabricated measurements.

Chapter 11

Chapter 11 — Firmware, Boot, and Trust

Status: draft · Chapter ID: CH11

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (no Gate 3 PASS claimed)

11.1 1. The moment

You press the power button—or wake a phone that was only sleeping. A logo appears. Lights blink. A spinner turns. Then, if you are lucky, a lock screen or desktop. If you are unlucky: an endless logo, a recovery menu you did not ask for, or a warning that software could not be verified.

From the seat it feels like one event: *booting*.

Underneath that feeling, early software called **firmware** initializes hardware that Part II treated as ports, buses, boards, power, and compute. A **boot sequence** hands capability and—when the platform is designed for it—trust checks from stage to stage until an operating system can present something a person recognizes. None of that is the app you will open later. The lock screen is not the same thing as “secure boot.” A marketing checkbox is not a measured attestation result.

This chapter opens Part III: making hardware useful by naming the first software abstractions that sit between raw silicon and a usable computer—and by separating **trust intent**, **user authentication**, and **authorization** before deeper security chapters arrive.

The governing question:

When I press power, what ordered handoffs must succeed—and what would honest failure look like—before I can trust this device enough to type a password?

11.2 2. What you notice

Before words like *root of trust* or *measured boot* enter, notice the human contract you already expect.

You expect the device to become interactive in a bounded time—or to fail in a way a person can understand. You expect logos and spinners to mean “still working,” not “silently dead.” You expect update banners and recovery screens to be readable, not logo-only mysteries. You expect that entering a PIN or password proves *you* to the device—not that every piece of software that ran before the lock screen was authentic. You also expect that a feature named “secure boot” on a settings page or a sticker is a claim about **policy intent**, not automatic proof that *your* unit measured and attested a known-good chain.

Those expectations are the product, from the person’s point of view.

A successful boot is a human perception produced by ordered handoffs: firmware init, bootloader load, OS start, and only then the familiar UI—plus trust checks that may pass, fail visibly, or (on weaker designs) be absent.

Notice the split timelines. Hardware can power on while storage is still probing. A logo can freeze while a verification step waits. A lock screen can appear after a chain that never verified anything cryptographic at all. Cold boot, warm wake from sleep, and “restart after update” can feel similar while using different paths underneath.

Optional comparison on a device you already own: cold boot once (power fully off, then on), then wake from sleep if the device supports it. Record only what you can see—lights, logos, banners, lock screen, wall-clock time bands if you watch a clock. Do not flash firmware. Do not enter recovery menus you do not already know how to exit safely. Label every causal guess as inference.

11.3 3. Exploded ecosystem

Boot is not a single object. It is a path through an ecosystem. **FIG-CH11-001** is the first-minute map: power/reset → firmware → bootloader → kernel → userspace → lock screen or recovery. Treat it as **Representative educational architecture**, not a claim that every phone or laptop implements identical stages with identical names.

Walk the layers in ordinary language, then keep the same layers when vocabulary deepens.

11.3.1 Human

You form intent: start working, check a message, recover a broken update. Hands press power. Eyes watch logos. Later, fingers meet a lock screen or a recovery choice. Accessibility matters already: if failure is logo-only, some readers are excluded before any app exists.

11.3.2 Firmware

Firmware is software that runs early to initialize and configure hardware so later software can treat devices as usable abstractions (Tanenbaum and Bos 2022; Saltzer and Kaashoek 2009). It may live in flash, ROM, or vendor-partitioned storage. It is not “the whole computer,” and it is not the app. It is the first cooperating software story after electricity and clocks become usable.

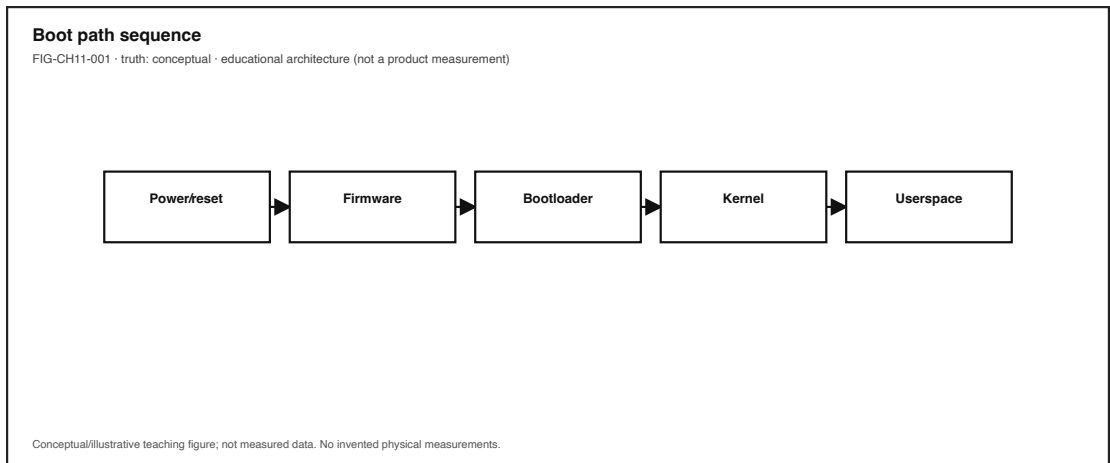


Figure 11.1: Power → firmware → bootloader → kernel → lock screen. Conceptual educational sequence; not universal timings.

11.3.3 Root of trust

A **root of trust** is a hardware and/or firmware foundation that later verification steps depend on. If that foundation is weak or bypassed by design, later “secure” labels inherit the weakness. Teaching the idea does not require inventing a measured root for the reader’s sealed device.

11.3.4 Bootloader

A **bootloader** loads and starts an operating-system image (or another stage). It is a handoff program—not the OS, and not proof that the image is authentic unless a separate policy checks signatures or measurements (Tanenbaum and Bos 2022).

11.3.5 Operating system kernel and userspace

The kernel takes control of privileged hardware mediation; userspace eventually presents the lock screen, desktop, or launcher. Chapter 12 deepens processes and scheduling. Here, keep the honesty rule: the familiar UI is late in the chain.

11.3.6 Trust policy vs user proof

Secure boot (policy intent) tries to constrain which software may run during boot (UEFI Forum, n.d.). **Authentication** at the lock screen asks who the user is. **Authorization** decides what that authenticated identity may do afterward. Collapsing those three into one word—“security”—is how marketing language and literacy diverge. CE-5 adjacency (AI, identity, privacy, and trust) owns richer consent and AI×trust synthesis; this chapter only needs the boot-time distinction clear.

11.3.7 Updates and recovery (preview)

Platforms often provide alternate paths when primary images fail verification or when an update leaves the device needing an honest recovery UI. Naming the path is literacy. Inventing vendor-specific brick rates or Quartet EVT recovery timings is not. Device Quartet boot and firmware

behavior remains **PHYSICAL_PENDING** research form-factor context only (CLM-CH11-005).

FIG-CH11-002 later sketches root-of-trust → verified stages as policy intent—not a product badge for any reader’s unit.

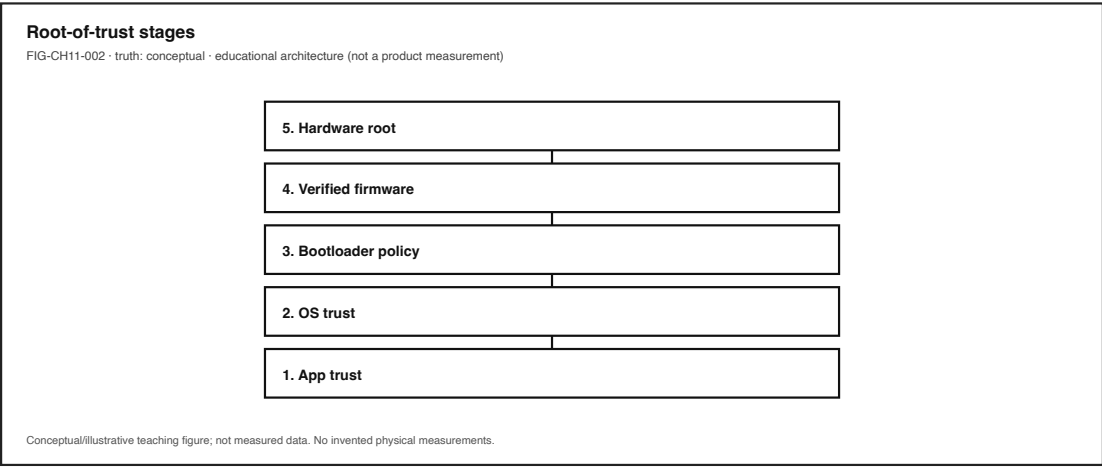


Figure 11.2: Root of trust → verified stages as policy intent—not a product badge.

11.4 4. Follow the signal

FIG-CH11-001 shows a numbered path. Read it as a logical story, not as a universal SoC bring-up with invented millisecond budgets.

1. **Power / reset.** Energy and clocks become available enough for early code to run (Chapter 5 and Chapter 9 adjacency). Incomplete power is an observation domain, not a root-cause claim from outside.
2. **Firmware init.** Early software probes memory controllers, storage, display, and input paths enough for later stages—or fails before anything readable appears (Tanenbaum and Bos 2022).
3. **Root-of-trust decision point (when present).** A foundation component may hold keys, measurements, or immutable code that later checks depend on (Saltzer and Kaashoek 2009). Absence of this step on some devices is itself a fact pattern—not a moral judgment about the owner.
4. **Bootloader handoff.** Control transfers to a program that can find and load an OS image (Tanenbaum and Bos 2022).
5. **Secure-boot policy check (when enabled).** Signature or authentication-info checks may allow, deny, or divert the next image according to platform policy (UEFI Forum, n.d.). A settings toggle labeled “secure boot” does not, by itself, prove your unit’s configuration matched a lab measurement.
6. **Optional measurement recording.** Some platforms record digests of boot components into registers or logs for later **attestation**—evidence about what ran, distinct from locking a policy and distinct from a PIN entry (Trusted Computing Group 2023).
7. **Kernel start → userspace.** Privileged software takes over resource mediation; services start; the lock screen or desktop becomes possible.

8. **Human feedback.** Logo ends; lock screen, desktop, recovery UI, or an unverified-software warning closes the loop—or fails to.

11.4.1 Alternate paths (the honesty rule)

Not every boot is a cold power-on. Sleep/wake may skip much of the chain. Recovery environments may load different images on purpose. Some embedded devices never show a lock screen. Teaching those forks prevents the encyclopedia trap: listing every vendor bootloader name instead of tracing one honest path.

11.4.2 Failure branch without drama

When something fails, prefer failure *domains* over confident blame:

- Power / incomplete init (still a hypothesis until evidenced)
- Storage or image not found
- Verification / policy deny
- Update mid-state requiring recovery affordance (qualitative Stability Contract concern; see Section 6)
- User-visible UI crash after a successful-enough boot

Outside observation rarely distinguishes those cleanly. That limitation is literacy.

11.5 5. Component cards

For each object: plain language, analogy, technical function, constraints, common symptoms. Analogies are labeled as analogies.

11.5.1 Firmware

- **Plain language.** Early software that initializes hardware and often participates in boot trust.
- **Analogy (labeled).** Like stage crew before the curtain—necessary, mostly invisible, not the play.
- **Technical function.** Configures devices and prepares abstractions later software can use (Tanenbaum and Bos 2022; Saltzer and Kaashoek 2009).
- **Constraints.** Flash size, update risk, vendor lock regions, platform diversity (do not universalize one SoC sequence).
- **Symptoms.** No display, endless logo, partial bring-up (fans/lights without UI).

11.5.2 Boot sequence

- **Plain language.** Ordered handoff from power/reset through firmware and bootloader into an OS.
- **Analogy (labeled).** Like a relay race—dropping the baton at any handoff ends the experience.
- **Technical function.** Sequences capability and optional trust checks before userspace (Tanenbaum and Bos 2022).
- **Constraints.** Timing expectations are product-specific; this book does not invent them.
- **Symptoms.** Hang at a recognizable stage; reboot loops; recovery entry.

11.5.3 Bootloader

- **Plain language.** Program that loads and starts an OS image.
- **Analogy (labeled).** Like a ferry that only carries what it is given—or what policy allows.
- **Technical function.** Locates, loads, and transfers control to the next image (Tanenbaum and Bos 2022).
- **Constraints.** Storage layout, signing policy, rollback rules (vendor-specific).
- **Symptoms.** “No bootable device,” wrong-slot boot, recovery menu.

11.5.4 Root of trust

- **Plain language.** Foundation whose integrity later verification depends on.
- **Analogy (labeled).** Like the first trustworthiness of a measuring stick—everything measured inherits its limits.
- **Technical function.** Anchors keys, immutable code, or measurement facilities for later stages (Saltzer and Kaashoek 2009; Trusted Computing Group 2023).
- **Constraints.** Physical access models, supply-chain assumptions, design threats out of Explorer scope.
- **Symptoms.** Usually invisible; learner sees downstream verify failures, not the root itself.

11.5.5 Secure boot (intent)

- **Plain language.** Boot-time policy intending to run only authorized software images.
- **Analogy (labeled).** Like a door policy for which costumes may enter the stage—not a name badge for the audience.
- **Technical function.** Constrains executable images via platform authentication/signing mechanisms (UEFI Forum, n.d.).
- **Constraints.** Key management, policy misconfiguration, update compatibility; **feature name measured guarantee for your device.**
- **Symptoms.** Unverified software warnings; refusal to boot unsigned images; surprising “secure boot off” after a repair.

11.5.6 Measured boot and attestation (concepts)

- **Plain language.** Recording what ran (measurement) and producing evidence about that state (attestation).
- **Analogy (labeled).** Like a sealed flight recorder versus a locked cockpit door—related aviation ideas, different jobs.
- **Technical function.** Measurement logs / platform configuration registers support attestation uses distinct from UI authentication (Trusted Computing Group 2023).
- **Constraints.** Evidence must be requested and interpreted; slogans are not attestations. No fabricated quotes or PCR values in this chapter.
- **Symptoms.** Remote “device not trusted” at work (when an enterprise system actually checks evidence); local PIN success despite never attesting anything.

11.5.7 Recovery / fail-safe path

- **Plain language.** Alternate boot environment when primary images fail verification or update.
- **Analogy (labeled).** Like a clearly marked emergency exit—useful only if a person can find and read it.

- **Technical function.** Loads a known recovery image or menu so the device can become honest again instead of silently unusable.
 - **Constraints.** Requires readable messaging and reachable controls; equity: not every learner has a spare device.
 - **Symptoms.** Recovery menus, “update failed” banners, limited function until repair completes.
-

11.6 6. Stability contract

A boot experience continues only while multiple hidden conditions stay within acceptable bounds.

Definition used throughout this book: a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For the ordinary “I pressed power and got a trustworthy-enough computer” feeling, conditions like these must hold together (qualitative; no invented numeric budgets):

1. **Firmware completes enough init** for storage, display, and input paths the experience needs.
2. **Boot chain reaches** an intended OS **or** an honest recovery UI—not a silent brick from the learner’s view.
3. **Trust policy matches learner expectation**, or failure is visible rather than silent.
4. **Update state** does not leave the device mid-change without a recovery affordance the person can understand (CLM-CH11-006 · SOURCE_IDENTIFIED via (“A/b (Seamless) System Updates” 2026) as a representative A/B seamless-update example; not universal firmware law).
5. **Accessibility:** boot and recovery messages have readable or alternate paths where the platform allows—not logo-only dead ends when text is possible.

A system can remain *electrically on* while the human experience has already failed: endless logo, inaccessible verify step, lock screen that appears after an unverified chain the person never consented to trust. Conversely, a lock screen can look “secure” while secure-boot policy was never enabled. Stability is concurrent conditions—not a single spinner.

Honesty bound for this edition: Device Quartet boot/firmware measurements are not claimed as completed physical evidence (CLM-CH11-005). Commodity observation in **LAB-BOOT-OBS-001** produces *your* evidence for *your* device—not a universal secure-boot score and not an attestation quote.

CE-5’s stability sketch (answer usability, identity continuity, authorization clarity, accessible trust) is **adjacent** for later identity and AI chapters. This chapter’s contract stays on power-on → trustworthy-enough computer.

11.7 7. Try it

11.7.1 LAB-BOOT-OBS-001 — Observe Boot and Wake (publication-owned, IMPLEMENTED_DIGITAL)

Goal. Observe cold boot vs wake (and any already-visible update/recovery banners) on a commodity device you own—or complete the offline fixture route—without flashing, unlocking, or bypassing anything.

WAIKE alignment note. WAIKE accepted `main` includes adjacent embedded bring-up and QEMU neighbors (`EMBEDDED_PROTOTYPING` / `HARDWARE_ENGINEERING` `digital_rc` adjacency at SHA `e97e74fc9bfb44b1cdc26b272dc4848264f15fe0`). Those are competency adjacencies. They are **not** renamed as publication lab IDs. **LAB-BOOT-OBS-001** is publication-owned. **LAB-TRUST-001** (CE-5) is trust/consent adjacency only—do not treat it as this chapter’s lab. Forward chip-to-cloud depth to Chapter 23.

Safety (hard stops).

- Do not open sealed devices.
- Do not flash, unlock, jailbreak, or bypass secure boot / recovery protections.
- Do not enter recovery modes you do not already know how to exit safely.
- No Device Quartet fabrication or EVT timing claims.
- Redact account identifiers, photos of others’ screens, and secrets from artifacts.

Routes.

- **Commodity route.** One phone or laptop you already own and may power cycle safely.
- **Offline fixture route.** Use a provided boot-symptom card and worksheet without hardware (equity default).

Explorer baseline (about 40–50 minutes).

1. Predict stages before power: firmware → bootloader → OS → lock screen (or “unknown”).
2. Cold boot once—or study the fixture card.
3. If safe and already familiar, wake from sleep once and compare observations.
4. Record observations only: lights, logos, banners, lock screen, coarse wall-clock bands.
5. Draw a labeled boot-chain diagram for one device class (phone or laptop).
6. Fill an observation-vs-inference table. Every causal guess needs a note about extra evidence required.
7. Teach-back in one sentence: why the lock screen is not secure boot.

Operator extension. Note any update-pending or recovery banner text already visible. Classify symptoms into domains (power/init, policy/verify, update/recovery, post-boot UI)—without claiming root cause.

Builder extension. Clean one-page diagram another student could follow without opening any device. Annotate visible vs sealed stages.

Engineer extension. Separate *secure-boot policy intent* from *attestation evidence*. List what evidence would be required before claiming “this device attested a known-good boot” (UEFI Forum, n.d.; Trusted Computing Group 2023). Still no flashing.

Researcher extension. State a hypothesis about one boot failure mode (for example, update-related recovery entry), name ethical variables, and list limitations. Explicitly forbid inventing Quartet timings or citing CLM-CH11-006 until sources exist.

Evidence to keep. Boot-chain diagram; observation-vs-inference table; scrubbed notes; teach-back paragraph. Prefer synthetic descriptions over photos of classmates’ devices.

Optional Operator post-boot health path: **LAB-CMS-001** (CE-3 adjacency) after a slow or failed boot narrative—do not duplicate CMS as Ch11 core.

11.8 8. Build it

Extend LAB-BOOT-OBS-001 without turning Part III into a firmware flashing course.

11.8.1 Explorer

Build a pocket card: firmware vs bootloader vs OS vs app, with one plain sentence each, plus “lock screen secure boot.”

11.8.2 Operator

Build a boot-symptom checklist that starts with observed logos/banners and ends with “needs more evidence”—never with fake certainty about root cause.

11.8.3 Builder

Build a labeled diagram (paper or digital) of power → firmware → bootloader → kernel → lock screen for one owned device class. Annotate where secure-boot policy *might* sit without claiming your unit’s configuration. Optional: second small diagram for sleep/wake as a shortened path.

11.8.4 Engineer

Build a one-page evidence plan: what would convert “marketing secure boot” into a justified trust claim for *one* device (policy state, measurement logs, attestation quote, update authenticity). Mark each row “observable in lab,” “needs enterprise tooling,” or “out of scope.” Cite standards vocabulary rather than inventing PCR values (UEFI Forum, n.d.; Trusted Computing Group 2023).

11.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to assert yet—for example, Quartet-specific interrupted-update EVT measurements—keeping Quartet boot claims **PHYSICAL PENDING** (CLM-CH11-005) while citing only living vendor/OS update docs already identified for CLM-CH11-006.

Educators can facilitate Section 11 teach-backs and keep classrooms on the offline fixture route when admin rights or spare devices are unavailable.

11.9 9. Secure and include it

11.9.1 Security — authentication vs authorization vs boot authenticity

Keep three ideas separate; CE-5 adjacency reinforces them for identity and AI contexts, and Chapter 23 deepens chip-to-cloud security.

Idea	Plain question	Boot-chapter example
Authentication	Who is acting?	Unlocking with PIN, password, or biometrics
Authorization	What may they do once identified?	After unlock, may this account change firmware settings?
Software authenticity / secure boot	May this image run?	Policy constraining bootloader/OS images (UEFI Forum, n.d.)
Attestation	What evidence shows what ran?	Measurement/attestation mechanisms distinct from UI login (Trusted Computing Group 2023)

A person can authenticate successfully on a device that never enforced secure boot. A platform can refuse an unsigned image (authenticity policy) while still leaving authorization decisions to later OS policy. Classic protection and naming discipline still apply: names and boundaries matter (Saltzer and Kaashoek 2009; Tanenbaum and Bos 2022).

No exploit, unlock, or bypass guidance belongs here. Prefer update authenticity literacy and least privilege over cleverness.

11.9.2 Privacy

Boot logs, serials, and recovery screenshots can become identifiers. LAB-BOOT-OBS-001 artifacts must not capture account tokens or photos of other people’s devices without consent. Scrub before portfolio save. **LAB-TRUST-001** consent-card craft remains available as adjacent practice—not a requirement to complete this chapter.

11.9.3 Accessibility

Boot and recovery are accessibility surfaces. Logo-only failure states exclude readers who need text. Color-only “error red” is insufficient. Keyboard or assistive paths to recovery choices matter when platforms provide them. Document steps in words, not only screenshots.

11.9.4 Equity

Not every learner has a spare device, admin rights, or a laptop that shows firmware menus. Observation-only commodity routes and offline fixtures are the equity default. Recovery literacy must not assume a second phone in the backpack.

11.9.5 Safety

No flashing. No forced recovery experiments. No intentional bricking. Power and battery hard stops from Chapter 9 still apply.

11.9.6 Ethics

Do not claim measured attestation results you did not obtain. Do not treat a feature name as proof. Do not invent Quartet EVT boot timings. Overclaiming trust is still false evidence.

11.10 10. Career lens

One power press crosses many ownership domains. No table promises employment; roles vary by organization. LAB-BOOT-OBS-001 artifacts resemble early professional evidence in miniature: labeled diagrams, observation discipline, and explicit uncertainty.

Role lens	Typical artifacts	Review questions
Firmware / embedded bring-up	Bring-up notes, flash maps (when authorized)	What initialized first—clocks, memory, or storage?
Bootloader / platform software	Signed image pipelines, rollback policy	What fails closed vs fails open?
Kernel / driver	Initcall order, device readiness	Which devices must exist before userspace?
Security engineer	Secure-boot policy, attestation design	Intent vs evidence distinguished?
SRE / fleet ops	Update rings, recovery runbooks	Is failure readable at human scale?
Accessibility / support	Recovery copy, alternate paths	Can someone exit recovery without vision-only cues?

Portfolio hint: a scrubbed boot-chain diagram plus “lock screen secure boot attestation” teach-back is more honest than a vibes-based “my phone is secure” claim.

11.11 11. Check understanding

Concept. In one sentence each, define *firmware*, *bootloader*, and *secure boot (intent)* so that none of them swallows the other two.

System tracing. Trace a familiar cold boot from power press to lock screen in numbered steps. Mark which steps you observed and which you inferred.

Misconception check. Why is “I typed my PIN, so secure boot worked” incomplete? Which ideas did that sentence collapse?

Misconception check. Why must authentication and authorization stay distinct after the lock screen appears?

Teach-it-back. Explain to a newcomer—using only LAB-BOOT-OBS-001 vocabulary—why a logo can spin forever without proving whether verification failed.

Researcher prompt. What evidence would convert a PHYSICAL_PENDING Quartet boot/firmware claim into a documented physical claim? What remains out of scope for a commodity classroom lab? What would be required before promoting interrupted-update failure modes out of SOURCE_NEEDED (CLM-CH11-006 omitted)?

11.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`), including representative A/B update recovery docs (“A/b (Seamless) System Updates” 2026). Project-specific Device Quartet boot/firmware status remains **PHYSICAL_PENDING** (CLM-CH11-005).

Inline citations used in this chapter include Tanenbaum and Bos (2022), Saltzer and Kaashoek (2009), UEFI Forum (n.d.), and Trusted Computing Group (2023).

CE-5 preproduction (`publication/preproduction/ce-05/`) is trust/identity/privacy adjacency; it does not replace this chapter’s boot literacy. Full-book human validation remains deferred until the full manuscript draft exists (`publication/full131/VALIDATION_SEQUENCE_DECISION.md`). This chapter does **not** claim Gate 3 PASS.

11.13 12. Glossary links

Candidate terms introduced or reinforced here (see also chapter glossary candidates; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Firmware	Early software that initializes hardware and may participate in boot trust
Boot sequence	Ordered handoff from power/reset to firmware to bootloader to OS
Bootloader	Program that loads an OS image and transfers control
Root of trust	Foundation whose integrity later verification depends on
Secure boot	Boot-time policy intending to run only authorized images
Measured boot	Recording measurements of boot components for later evidence
Attestation	Producing evidence about a device’s software state
Recovery mode	Alternate boot path when primary images fail
Authentication	Establishing who is acting (e.g., lock-screen proof)
Authorization	Deciding what an authenticated identity may do

Term	Plain link
Stability contract	Concurrent conditions that keep the boot experience alive

Related earlier chapters: system lens (CH01), signals/power adjacency (CH05, CH09), ports/boards handoff (CH10). Related later chapters: operating systems (CH12), cybersecurity chip-to-cloud (CH23), privacy/identity/ethics (CH24). LAB-TRUST-001 adjacency: CE-5 / CH23 forward link only.

11.14 Figure references (embedded; registered SVG + a11y)

All four figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured evidence. No fabricated telemetry or attestation quotes.

11.14.1 FIG-CH11-001 — Power → firmware → bootloader → kernel → lock screen

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Sequence diagram.
- **Reader should notice.** Ordered handoff plus optional recovery branch; lock screen is late.
- **Truth class.** Conceptual.
- **Alt text requirement.** List stages in order; name recovery branch; state conceptual truth class; deny universal timings.

11.14.2 FIG-CH11-002 — Root of trust → verified stages

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** System map.
- **Reader should notice.** Policy intent layers—not a product badge for the reader’s device.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name root and stages; state that presence of a marketing name is not measured proof.

11.14.3 FIG-CH11-003 — Lock screen vs secure boot

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Comparative layers.
- **Reader should notice.** User authentication vs software authenticity policy; authorization as a third idea.
- **Truth class.** Illustrative.
- **Alt text requirement.** Name both columns (and authz callout); forbid color-only encoding.

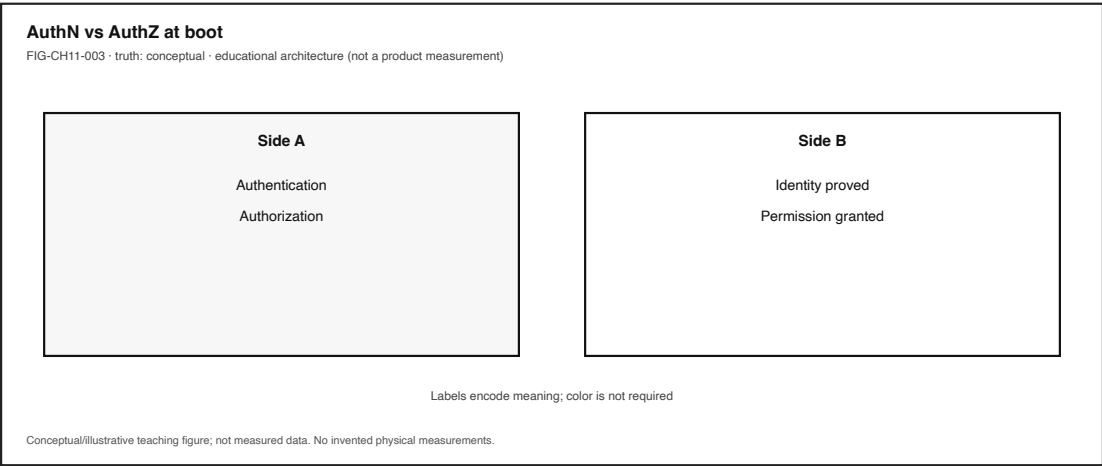


Figure 11.3: Lock screen authentication vs secure-boot authenticity policy (authorization call-out).

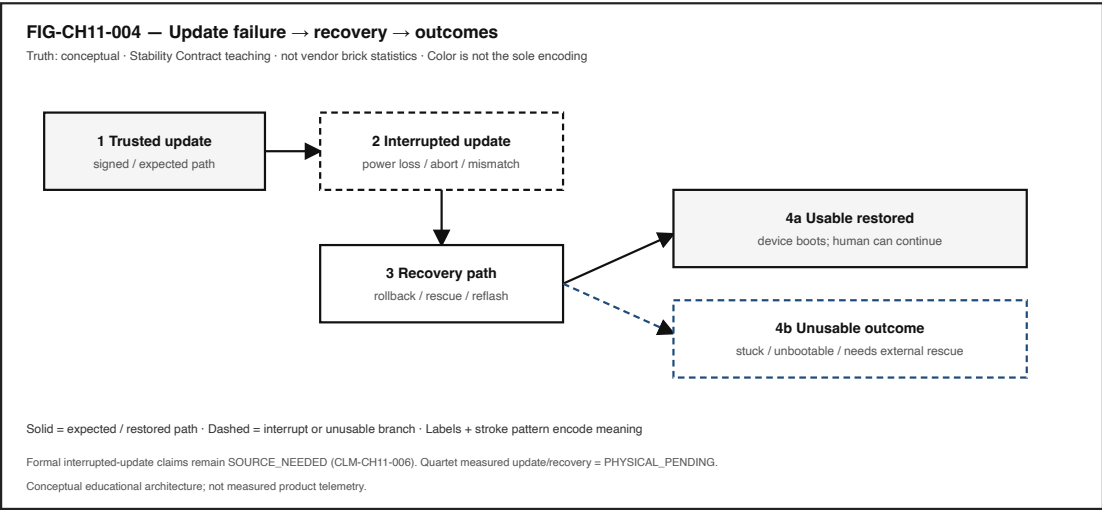


Figure 11.4: Update failure → recovery → outcomes. Conceptual Stability Contract teaching; not vendor brick rates.

11.14.4 FIG-CH11-004 — Update failure → recovery → outcomes

- **Production status.** `embedded` (registered SVG + accessibility sidecar).
 - **Type.** Failure map.
 - **Reader should notice.** Readable recovery vs unusable outcomes as Stability Contract teaching—not vendor brick statistics.
 - **Truth class.** Conceptual.
 - **Alt text requirement.** List branches; mark CLM-CH11-006 omitted / `SOURCE_NEEDED` for formal interrupted-update claims; mark Quartet `PHYSICAL_PENDING`.
-

11.15 Claim footnotes used in this chapter

- **CLM-CH11-001.** Firmware initializes hardware for later abstractions—textbook framing (Tanenbaum and Bos 2022; Saltzer and Kaashoek 2009).
- **CLM-CH11-002.** Boot as ordered handoff—not a single mysterious OS appearance (Tanenbaum and Bos 2022).
- **CLM-CH11-003.** Secure boot as policy intent; feature name `measured guarantee` for a reader’s device (UEFI Forum, n.d.).
- **CLM-CH11-004.** Attestation / `measured boot` distinct from UI lock screens (Trusted Computing Group 2023).
- **CLM-CH11-005.** Device Quartet boot/firmware behavior `PHYSICAL_PENDING`; research form factors only.
- **CLM-CH11-006.** Failed/interrupted firmware updates can leave recovery or unusable states (`SOURCE_IDENTIFIED` via `android-ab-ota`). Living docs; representative example only.

Chapter 12

Chapter 12 — Operating Systems, Processes, Threads, and Scheduling

Status: `working_draft` · **Chapter ID:** CH12

Author: Edmund Gunn, Jr.

Inheritance: CE-3 OS abstractions deepened; primary Try-it lab **LAB-CMS-001** (link, do not duplicate)

Gate posture: `GATE_3_IN_PROGRESS` - `READER_EVIDENCE_PENDING` (this chapter does **not** claim Gate 3 PASS)

12.1 1. The moment

You open two apps. One keeps playing audio while you scroll another. Or one hung window freezes the whole UI while a clock somewhere still ticks. From the seat it feels like multitasking magic—or “the phone froze.” Underneath, an **operating system** is isolating address spaces, scheduling runnable work, and mediating devices so hardware can be shared without every program owning the machine alone.

Chapter 6 already named instruction streams and “more cores always smoother.” Chapter 7 deepened memory and storage waits. This chapter stays with the **OS mediation story**: processes, threads, context switches, and the scheduler—without an encyclopedia dump of every ISA quirk or every vendor scheduling algorithm.

The governing question is ordinary and honest:

When two apps seem to “run at once,” what is the OS actually doing—and how is that different from the OS doing my app’s work for me?

12.2 2. What you notice

Before jargon, notice the human contract you already expect from a shared machine.

You expect more than one activity to make progress without every other window becoming dead. You expect a hung editor not to silently rewrite another app’s files. You expect music to keep playing while you scroll a document—until something fails the Stability Contract. You expect that “the computer is busy” and “the UI froze” are related but not identical stories. You expect that closing one icon does not always stop every helper process the OS still tracks.

Those expectations are not decorations. They *are* the product from the person’s point of view.

Multitasking feel is a perception produced by OS abstractions that share processors while keeping programs from freely overwriting each other.

What you may notice, without opening a monitor yet:

- audio continuing while another window scrolls,
- one app beachballing while another still redraws,
- a sudden full-UI freeze that feels like “the whole OS died,”
- recovery after you force-quit one process,
- fans or warmth when many apps compete.

What those symptoms do **not** prove by themselves: a single root cause named “bad scheduling,” or that the OS replaced your app’s computation. High activity and poor feel can travel together or apart. A frozen UI can be scheduling starvation, priority quirks, lock contention, waiting on I/O, or a busy main thread—diagnosis needs evidence (Tanenbaum and Bos 2022; The Linux Kernel Documentation 2026b).

Optional seat comparison: start a familiar local editor and a media player. Scroll the editor while media plays. Then intentionally open enough mild local load that something stutters. Write one sentence about what still progressed and what stalled *before* you look at any graph. That comparison is a teaching experience, not a universal benchmark—and not a Device Quartet EVT result.

12.3 3. Exploded ecosystem

Shared-device feel is not a single object. It is a path through an inside-the-device ecosystem. Figure 12.1 is the first-minute map for this chapter: app icons → processes → threads → scheduler → CPU cores. Treat it as **conceptual / Representative educational architecture**—not a claim that any specific manufactured revision looks exactly like the diagram.

The Device Quartet used elsewhere in this series—Student 14.5-inch, Handheld Hybrid, DS-XL Coder, and Edge IO Wearables—are research form factors and learning benchmarks. Physical fabrication and EVT scheduler/load traces remain **PHYSICAL_PENDING**; do not treat comparison-matrix core counts or thermal-scheduler stories as shipping product facts (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Walk the layers in ordinary language.

12.3.1 Human

You form intent: scroll this page, keep that song playing, save that file. Eyes and hands judge whether multiple activities stay present enough to trust.

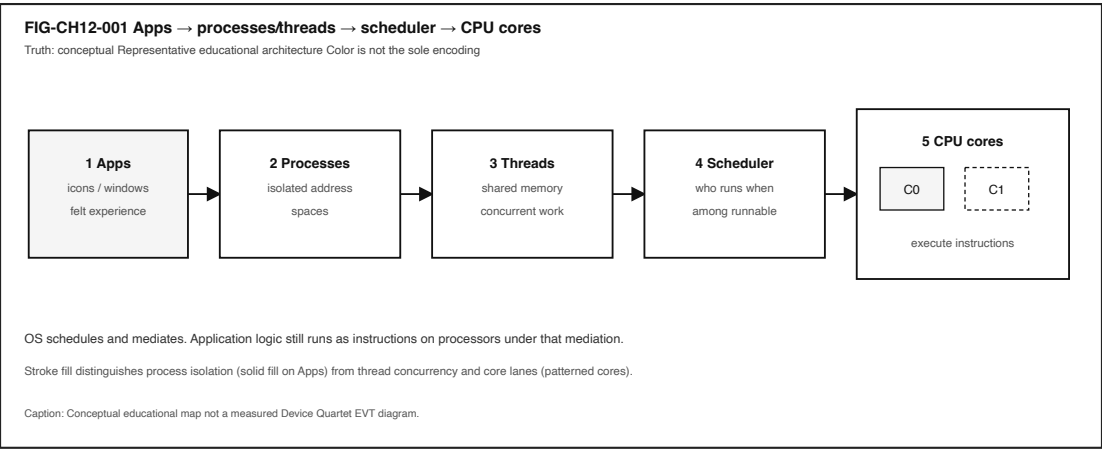


Figure 12.1: Conceptual map from apps through processes and threads to the OS scheduler and CPU cores.

12.3.2 Application

Your apps hold state and code. Icons and windows are the human surface. Underneath, software prepares work that will become **instructions** on processors—still the app’s logic, not a mystical OS substitute (Tanenbaum and Bos 2022; Silberschatz, Galvin, and Gagne 2018).

12.3.3 Process

A **process** is an OS abstraction for a running program instance: typically its own address space, resource handles, and scheduling identity (Tanenbaum and Bos 2022; Silberschatz, Galvin, and Gagne 2018). The icon on your dock is not identical to the process table entry—but for Explorer teach-back, “app icon process” is the useful first correction.

12.3.4 Thread

A **thread** is a unit of concurrent execution inside a process, usually sharing that process’s address space (Tanenbaum and Bos 2022; Silberschatz, Galvin, and Gagne 2018). A UI thread, a save thread, and a decoder thread can all be “the app” from the seat while remaining distinct to the scheduler.

12.3.5 Scheduler and OS mediation

A **scheduler** decides which runnable entity obtains processor time next (The Linux Kernel Documentation 2026b; Tanenbaum and Bos 2022). The OS mediates hardware and provides services. It does **not** replace application computation: your logic still runs as instructions under that mediation (Tanenbaum and Bos 2022).

12.3.6 CPU cores

Cores execute the chosen instruction streams. Extra cores can help when independent work is ready; they do not guarantee lower latency for every interaction (Patterson and Hennessy 2020b).

12.3.7 Adjacent layers (named, not encyclopedized)

Memory isolation, storage I/O, locks, interrupts, and power/thermal policy sit beside scheduling. Chapters 6–7 and 9 named those neighbors; CE-6’s Stability Contract reminds us that concurrent conditions fail together even when a connectivity icon stays polite.

12.4 4. Follow the signal

Follow one ordinary shared-device moment—say, scrolling a document while audio keeps playing—as a logical story. Platforms differ; the teaching sequence is orientation, not brand loyalty.

1. **Intent becomes software work.** Scroll and audio decode become work items inside one or more apps.
2. **Processes and threads exist.** The OS already tracks process boundaries and thread contexts for those apps (Silberschatz, Galvin, and Gagne 2018).
3. **Work becomes runnable.** Threads that are ready to execute become **runnable** if they are not waiting on I/O, locks, or other events (Tanenbaum and Bos 2022).
4. **Scheduler chooses.** The OS scheduler selects among runnable entities on available cores (The Linux Kernel Documentation 2026b). Interactive work can still be delayed under contention.
5. **Instructions execute.** On a chosen core, the CPU advances that thread’s instruction stream. The OS mediated the turn; the app’s code still runs (Tanenbaum and Bos 2022).
6. **Context switches happen.** When the OS moves the processor from one runnable entity to another, it **saves and restores CPU state**—a **context switch** (Tanenbaum and Bos 2022; Silberschatz, Galvin, and Gagne 2018). Figure 12.2 shows that swimlane as a conceptual teaching sequence, not a traced kernel log.
7. **Waits appear.** A thread may stop being runnable while it waits for memory, storage, a lock, or another thread. Busy and waiting can both feel like “frozen” from the seat.
8. **Feedback returns.** Pixels update; audio buffers refill; or the UI stops responding. Your nervous system judges the combined timeline.

12.4.1 Concurrency without fairy tales

Figure 12.3 separates **concurrency** (structuring overlapping work over time) from **parallelism** (simultaneous progress on multiple cores). The distinction is an **illustrative** teaching aid—not a measured speedup curve (Patterson and Hennessy 2020b; Tanenbaum and Bos 2022).

In plain language for this book:

- **Concurrency** lets a system make progress on more than one task by structuring overlapping work—even on a single core via interleaving.
- **Parallelism** means simultaneous execution when hardware and ready independent work allow.
- **More cores can improve throughput** for parallelizable work but **do not guarantee lower latency** for every click, scroll, or save (Patterson and Hennessy 2020b).

A practical seat-test: if the UI thread must finish drawing before it can handle the next input, a second core cannot teleport that dependency away. If audio decode and document layout truly need independent work, multiple cores may help—until locks, memory, or storage become the new line at the door.

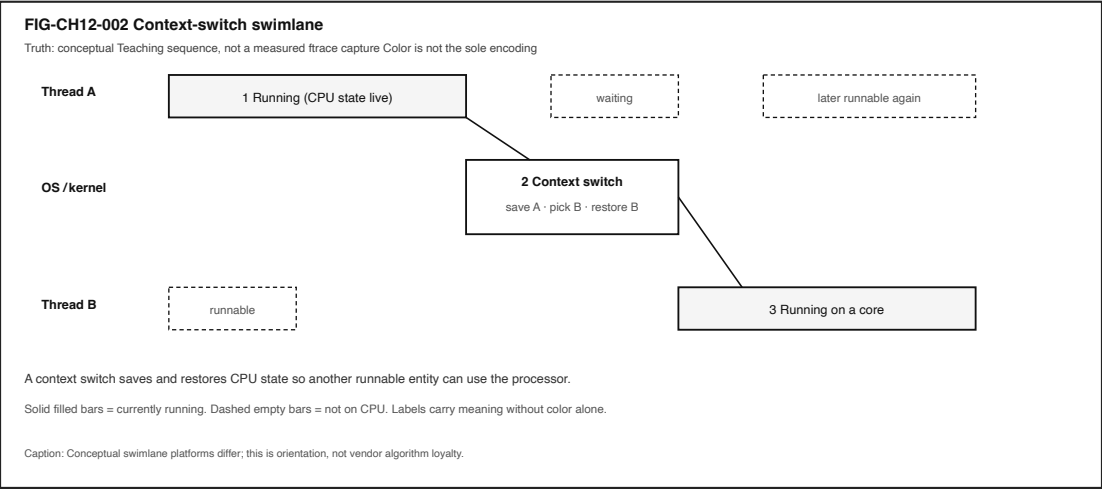


Figure 12.2: Conceptual context-switch swimlane from Thread A through an OS save/restore step to Thread B.

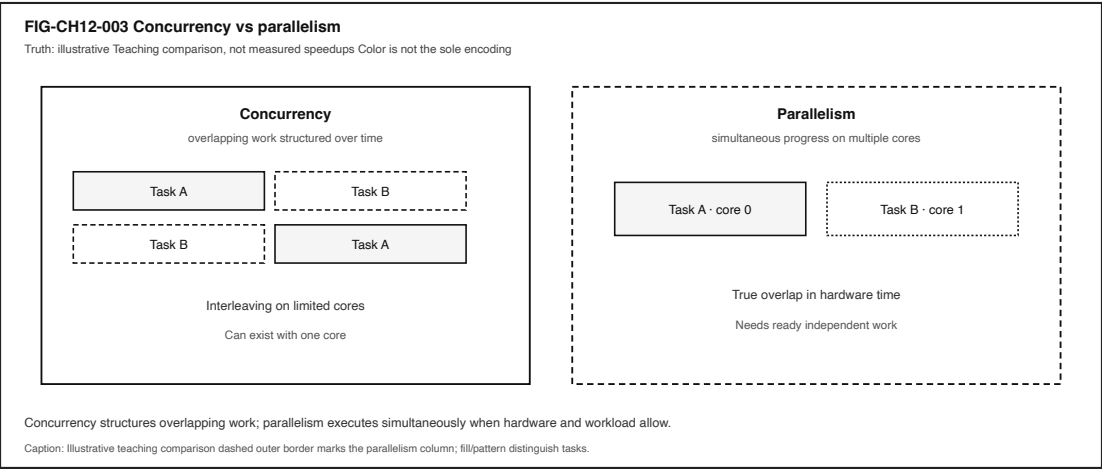


Figure 12.3: Illustrative comparison of concurrency interleaving versus parallel multi-core execution.

12.4.2 Honesty branches

- Audio can continue while a UI thread starves.
 - A monitor can show modest average CPU while the interactive thread keeps missing its turn.
 - Force-quitting a window may leave helper processes running.
 - “Frozen” can mean not scheduled, waiting, deadlocked, or busy in a useless loop—evidence decides which inference is fair (Tanenbaum and Bos 2022; The Linux Kernel Documentation 2026b).
-

12.5 5. Component cards

These cards are a first toolkit—not a bill of materials for every kernel.

12.5.1 Operating system

Plain definition. Software that mediates hardware resources and provides abstractions to programs (Tanenbaum and Bos 2022; Silberschatz, Galvin, and Gagne 2018).

Experience benefit. Many programs can share one machine without each owning the silicon outright.

Failure symptom. Mediation that starves interactive work, or a crash that takes more than one app with it.

12.5.2 Process

Plain definition. OS abstraction for a running program instance with isolation boundaries (Tanenbaum and Bos 2022).

Experience benefit. One buggy app is less likely to freely rewrite another’s memory.

Failure symptom. Too many competing processes; unclear which process owns the lag; assuming the icon *is* the process.

12.5.3 Thread

Plain definition. Schedulable execution context that shares a process address space (Silberschatz, Galvin, and Gagne 2018).

Experience benefit. Background saves and foreground UI can progress as separate units of work.

Failure symptom. A stuck UI thread; lock contention that freezes feel while other threads still “look busy.”

12.5.4 Scheduler

Plain definition. OS mechanism that allocates processor time among runnable entities (The Linux Kernel Documentation 2026b; Tanenbaum and Bos 2022).

Experience benefit. Interactive work can share the machine with background tasks.

Failure symptom. Foreground starvation; fair-looking averages that still feel awful in the seat. Do not universalize one vendor algorithm.

12.5.5 Context switch

Plain definition. Saving and restoring CPU state when switching among runnable entities (Tanenbaum and Bos 2022).

Experience benefit. The processor can serve more than one thread over time.

Failure symptom. Thrashy switching under pathological load (qualitative caution—measure before claiming).

12.5.6 Process isolation

Plain definition. Protection so one process cannot freely read or write another’s memory (Tanenbaum and Bos 2022; Silberschatz, Galvin, and Gagne 2018).

Experience benefit. Safety and privacy boundaries that make multitasking socially tolerable.

Failure symptom. Treating “apps share a CPU” as “apps share all secrets.”

12.5.7 System call

Plain definition. Controlled entry from user programs into kernel services (Tanenbaum and Bos 2022).

Experience benefit. Programs can request I/O, process creation, and other services without owning the hardware.

Failure symptom. Blocking too long in a service path that the UI thread needed.

12.5.8 Concurrency vs parallelism

Plain definition. Concurrency structures overlapping work; parallelism executes simultaneously on multiple cores when possible (Patterson and Hennessy 2020b).

Experience benefit. Language that prevents buying the wrong upgrade story.

Failure symptom. “I bought more cores and my typing still stutters”—often a serial, waiting, or lock path.

12.6 6. Stability contract

The **Stability Contract** is a signature idea across this book:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For Chapter 12, a smooth shared-device interactive experience may require all of the following to stay “good enough” at once—qualitative bounds only; no invented numeric budgets (CE-3 / CE-6 inheritance):

- the target process is scheduled enough to make progress,
- the UI/thread is not starved indefinitely under normal load,

- memory isolation holds (no silent cross-process corruption as a learner assumption),
- runnable work can obtain CPU time when needed (The Linux Kernel Documentation 2026b),
- needed waits (I/O, locks) do not erase the interactive path without visible symptoms,
- power/thermal policy does not collapse available CPU without any seat-visible clue,
- the person can still tell progress from failure.

A device can remain **powered on and connected** while the **human experience has already failed**.

Three separations matter here:

1. **OS schedules** versus **OS does my app’s work** — mediation is not substitution (Tanenbaum and Bos 2022).
2. **Concurrency** versus **parallelism** — structure versus simultaneous hardware progress (Patterson and Hennessy 2020b).
3. **Frozen UI** versus **CPU busy** — scheduling, locks, and waits need evidence (The Linux Kernel Documentation 2026b).

Commodity observations you collect in **LAB-CMS-001** are *your* evidence for *your* device and session—not universal EVT curves, and not Gate 3 reader validation. Figure 12.4 shows a classroom **n=1** before/during fixture snapshot inherited from that lab’s teaching transcript—not a product SLO and not Device Quartet EVT (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

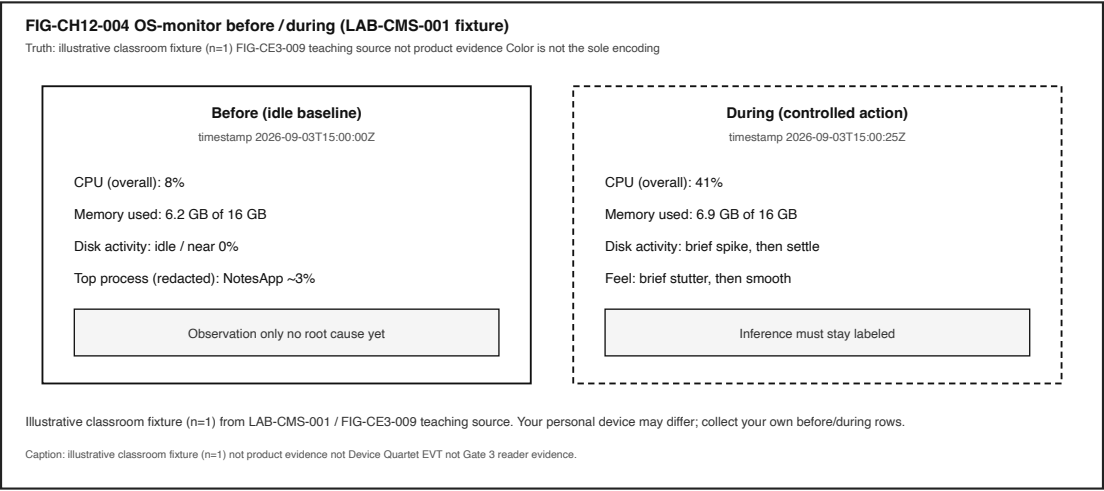


Figure 12.4: Before/during OS-monitor fixture snapshots for classroom teaching (n=1).

Device Quartet scheduler behavior under load remains **PHYSICAL_PENDING**—no fabricated thermal/scheduler traces in this chapter.

12.7 7. Try it

12.7.1 LAB-CMS-001 — Make Local Slowness Visible

Observable question. When two apps share one machine—or one hung window freezes the UI—what evidence can I gather—using only commodity tools—to separate **process**, **thread**, and **scheduler** behavior from a vague “the CPU is dying” story?

This chapter’s Try It **inherits and links** the publication-owned CE-3 lab rather than inventing a duplicate LAB-SCHED-001 package. Follow the full lab packet at `labs/LAB-CMS-001/` (README, routes, fixtures, portfolio templates). Focus your write-up on the **process** / **thread** / **scheduler** columns of the diagnosis; Chapters 6–7 reuse the same lab for CPU and memory/storage emphasis.

WAIKE alignment note. WAIKE accepted **main** (audit SHA in the CH12 packet) hosts adjacent labs on observability and OS users, but there is **no** exact WAIKE module ID for LAB-CMS-001. Do not mint one. Optional method craft for observation vs inference also sits in **LAB-TAP-001** (CH02)—use it for labeling habits, not as a substitute OS lab.

Prerequisites. A computer you may use for learning; built-in OS monitor (Task Manager / Activity Monitor / `top` or equivalent); a familiar local app.

Safety.

- Do not capture personal document contents; redact filenames before sharing.
- Do not install “optimizer” tools, disable security software, or load kernel modules.
- Use **mild** load only. **Stop** if the device warns about heat or becomes uncomfortably hot.
- No Device Quartet hardware is required. No invented EVT numbers.
- No privilege-escalation recipes.

Time estimate. About 45 minutes for Explorer + Operator baseline.

12.7.1.1 Prediction

Before measuring, write which OS behavior you expect to dominate during your controlled action: process contention, thread/UI-thread stalls, scheduling latency, or memory/storage side effects that only *look* like “CPU is busy.”

12.7.1.2 Route A — Commodity OS monitor (baseline)

1. Record **before** CPU, memory, and disk/storage activity; note which processes appear for your apps.
2. Perform one controlled local action (open a medium document, scroll, or save) while optional second media plays.
3. Record **during** readings, wall-clock feel, and whether another activity kept progressing.
4. Fill observation vs inference columns in the lab portfolio table.

12.7.1.3 Route B — Safe CLI snapshot (optional)

```
python3 labs/LAB-CMS-001/local_app/safe_snapshot.py
```

Read-only sampling where the OS exposes coarse stats. If a metric is unavailable, the script reports **unavailable**—never invent hardware claims.

12.7.1.4 Route C — Fixture fallback

Use `labs/LAB-CMS-001/fixtures/` when monitors are inaccessible or you must avoid personal screenshots. Fixtures teach concepts; they are not claims about your personal device. Figure 12.4 reproduces the LAB-CMS-001 before/during monitor teaching fixture for classroom reading. A separate CE measured-plate figure remains blocked pending qualifying evidence—do not treat this classroom plate as that blocked asset.

12.7.1.5 Evidence (minimum)

- observation table,
- two snapshots or fixture IDs with timestamps,
- teach-back: “OS schedules; apps still compute,” plus one sentence on process vs app icon.

12.7.1.6 Interpretation labels

Observation (allowed)	Inference (must label)
CPU % rose from A to B	“CPU-bound root cause”
UI froze; another process still drew	“Scheduler bug for sure”
Fans spun; device felt warmer	“Thermal throttle at X °C”
Scroll stuttered under extra load	“Need N more cores”

12.7.1.7 Limits (say them out loud)

- OS monitors are coarse; they are not silicon performance-counter labs.
- One run is not a benchmark.
- This lab does not close Gate 3 and does not validate Device Quartet EVT scheduler claims (**PHYSICAL_PENDING**).

12.8 8. Build it

Use the same shared-device story at the depth that matches your pathway. Prefer commodity tools and LAB-CMS-001 artifacts. A proposed stretch idea **LAB-SCHED-001** remains namespaced-only—do not mint it as a **WAIKE** ID.

12.8.1 Explorer

Draw Figure 12.1 from memory with four labels: process, thread, scheduler, CPU. Teach a nontechnical person: “The OS decides who runs; the app’s work still happens as instructions.”

12.8.2 Operator

Capture before/during monitor snapshots under idle vs mild multi-app load. Mark each row observation or inference. Note which processes appeared for each app icon.

12.8.3 Builder

Change **one** variable (extra documents, a background export, a second media app, fewer tabs). Re-run the observation table. Produce a **process/thread map** for one experience: which processes, which likely threads (UI vs background), what you did *not* measure.

12.8.4 Engineer

Order a diagnosis plan: connectivity rule-out (if the workload is local) → which process owns the feel → CPU activity vs waiting → locks/priority hypotheses → qualitative thermal/power. State what additional evidence would be required before claiming causation. Separate concurrency from parallelism using Figure 12.3 language without inventing speedup curves.

12.8.5 Researcher

Propose a small scheduling hypothesis: for example, “Adding mild background CPU load increases scroll hitch rate more than adding idle tabs.” Define variables, planned runs, confounders (thermal state, battery mode, other processes), and uncertainty. Report that Device Quartet physical scheduler claims remain **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Do not invent vendor algorithm universals or Gate 3 PASS language.

Educators can facilitate misconception probes from Section 11 and use LAB-CMS-001 fixtures for learners without admin rights.

12.9 9. Secure and include it

12.9.1 Security

Process isolation is an intent you can already name at survey depth: separate processes limit how much damage one buggy or malicious program can do to another’s memory and credentials (Tanenbaum and Bos 2022; Silberschatz, Galvin, and Gagne 2018). Scheduling and privilege boundaries are part of that story. This chapter does **not** teach exploitation or privilege-escalation recipes—only the literacy that “apps share a CPU” is not the same claim as “apps share all secrets.”

12.9.2 Privacy

Monitor screenshots can leak filenames, thumbnails, account names, and open document titles. Redact before sharing. Prefer fixtures when portfolio evidence must leave the device. LAB-CMS-001 forbids capturing personal document contents as a requirement.

12.9.3 Accessibility

Activity visualizations and color-coded monitor graphs are not equally readable. Prefer labeled columns, patterns, and text tables. Keyboard-reachable OS monitors matter; fixture transcripts exist when GUIs are hostile. Dictate observation tables if needed. Do not make “see the red spike” the only teaching channel—Figure 12.1 through Figure 12.4 encode meaning with shape, order, and stroke pattern as well as text.

12.9.4 Equity

Not every learner has a new multi-core laptop, admin rights, or a quiet thermal environment. Commodity routes and offline fixtures are the baseline. Device Quartet form factors are learning analogies here, not admission tickets (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Avoid shaming older hardware; teach diagnosis that works with the machine in front of you.

Shared family or school machines raise another equity edge: “just open Activity Monitor” may require permissions a learner does not hold. Fixture transcripts and verbal observation tables keep the pathway open without forcing privilege escalation. The Stability Contract fails for people first; pedagogy should not add a second failure by assuming identical tools.

12.10 10. Career lens

Literacy from this chapter shows up in several neighboring roles—without employment promises:

- **ROLE-KERNEL — Kernel / OS Engineer** — processes, threads, schedulers, and context-switch thinking; living docs such as Linux CPU scheduler documentation as examples, not universals (The Linux Kernel Documentation 2026b).
- **ROLE-APP — Application Developer** — UI threads vs background work; avoiding main-thread blocks that feel like “the OS froze.”
- **ROLE-SRE — SRE-adjacent local diagnosis** — separate scheduling starvation, waiting, and busy-work using monitors as evidence, not folklore.
- **ROLE-EMBEDDED — Embedded Engineer** — who runs when under tight latency budgets; interrupt-time vs process-time neighbors (adjacency only).
- **Educator / mentor** — teach observation vs inference so people stop buying the wrong upgrade story.

Day-one habits transfer across those roles: write a prediction before you open a monitor; label every causal sentence; refuse brochure numbers without a method. Career growth here means better questions: “Is this not scheduled, waiting, or busy without progress?”—not a guaranteed job title.

12.11 11. Check understanding

12.11.1 Misconception probes

1. **“The operating system runs my app’s logic for me.”**
Counter: the OS schedules and mediates; instructions still execute on processors (Tanenbaum and Bos 2022).
2. **“An app icon is the same thing as a process.”**
Counter: icons are the surface; processes are OS abstractions that may outlive or outnumber windows (Silberschatz, Galvin, and Gagne 2018).
3. **“If the clock widget still ticks, a frozen window cannot be a scheduling problem.”**
Counter: other processes can keep running while one UI thread or process is stuck waiting or not being scheduled (Tanenbaum and Bos 2022; Silberschatz, Galvin, and Gagne 2018).
4. **“Concurrency and parallelism are synonyms.”**
Counter: concurrency structures overlapping work; parallelism is simultaneous hardware progress (Patterson and Hennessy 2020b).
5. **“A frozen UI always means the CPU is saturated.”**
Counter: scheduling, locks, waits, and sampling artifacts confuse percent alone—label inferences (Tanenbaum and Bos 2022; The Linux Kernel Documentation 2026b).

6. “Device Quartet scheduler traces are measured shipping specs.”
Counter: research form factors; physical EVT remains **PHYSICAL_PENDING**
 (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

12.11.2 Teach-back (say it out loud)

Apps become processes and threads. The OS decides who runs when. Context switches move the CPU among runnable work. Apps still compute as instructions. Isolation keeps programs from freely rewriting each other. A healthy network icon does not prove local scheduling is fine.

12.11.3 Pathway self-check

Pathway	You can...
Explorer	Explain process vs app icon; teach-back “OS schedules while apps still compute.”
Operator	Use commodity monitors or fixtures to observe CPU/memory during controlled action (LAB-CMS-001).
Builder	Produce a process/thread map for one experience.
Engineer	Reason about concurrency vs parallelism and evidence needed for CPU-bound claims.
Researcher	Design a small scheduling hypothesis with labeled uncertainty; keep Quartet claims PHYSICAL_PENDING .

12.12 References

Inline citations used in this chapter include Tanenbaum and Bos (2022), Silberschatz, Galvin, and Gagne (2018), The Linux Kernel Documentation (2026b), Patterson and Hennessy (2020b), and “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026).

CE-3 preproduction packets ([publication/preproduction/ce-03/](#)) supply the inherited claim, lab, and stability wording this chapter adapts. Full bibliography entries live in the active Quarto project bibliography (including [publication/full131/WORKING_BIBLIOGRAPHY.bib](#)).

12.13 12. Glossary links

Term	Plain link
operating system	Software that mediates hardware and provides abstractions
process	OS abstraction for a running program instance with isolation

Term	Plain link
thread	Schedulable execution context sharing a process address space
scheduler	OS decision of what runs when among runnable entities
context switch	Saving/restoring CPU state when switching runnable entities
process isolation	Protection against unrestricted cross-process memory access
system call	Controlled request from user mode into kernel services
concurrency vs parallelism	Overlapping structure vs simultaneous multi-core progress
Stability Contract	Experience depends on hidden conditions staying acceptable

See also Chapter 2 (process/thread/scheduler naming), Chapter 6 (CPU / parallel work), Chapter 7 (memory/storage waits), Chapter 9 (power/thermal), and Chapter 13 (files/persistence neighbors).

12.14 Figure references (embedded above; accessibility metadata)

12.14.1 FIG-CH12-001 — Apps → processes/threads → scheduler → cores

- **File:** figures/architecture/fig-ch12-001-apps-to-scheduler.svg
- **Truth:** conceptual
- **A1ly:** figures/preproduction/accessibility/fig-ch12-001.yaml

12.14.2 FIG-CH12-002 — Context-switch swimlane

- **File:** figures/architecture/fig-ch12-002-context-switch.svg
- **Truth:** conceptual
- **A1ly:** figures/preproduction/accessibility/fig-ch12-002.yaml

12.14.3 FIG-CH12-003 — Concurrency vs parallelism

- **File:** figures/architecture/fig-ch12-003-concurrency-vs-parallelism.svg
- **Truth:** illustrative
- **A1ly:** figures/preproduction/accessibility/fig-ch12-003.yaml

12.14.4 FIG-CH12-004 — OS-monitor before/during (fixture)

- **File:** figures/architecture/fig-ch12-004-monitor-snapshots.svg
- **Truth:** illustrative classroom fixture (n=1) from LAB-CMS-001 before/during monitor readings (not product evidence; separate CE measured-plate asset remains blocked)

- **A11y:** figures/accessibility/fig-ch12-004.yaml

Related CE-3 maps (optional cross-read, not required embeds): figures/preproduction/ce-03/fig-ce-fig-ce3-003.svg.

12.15 Claim footnotes used in this chapter

Claim	Status handling in prose
CLM-CH12-001 — processes/threads are OS abstractions; scheduler allocates CPU time	Taught with Tanenbaum and Bos (2022), Silberschatz, Galvin, and Gagne (2018), The Linux Kernel Documentation (2026b) (SOURCE_IDENTIFIED)
CLM-CH12-002 — OS mediates; apps still compute as instructions	Taught with Tanenbaum and Bos (2022) (CE-3 wording boundary)
CLM-CH12-003 — more cores help throughput, not guaranteed latency	Taught with Patterson and Hennessy (2020b); no invented speedup curves
CLM-CH12-004 — frozen UI needs evidence (scheduling/locks/waits)	Taught with Tanenbaum and Bos (2022), The Linux Kernel Documentation (2026b) + LAB-CMS-001 observation/inference
CLM-CH12-005 — Quartet scheduler-under-load	PHYSICAL_PENDING with “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026)

Explicit non-claims: Gate 3 PASS; measured Device Quartet scheduler EVT; invented vendor algorithm universals; WAIKE lab ID LAB-SCHED-001 as if published.

Chapter 13

Chapter 13 — Files, Databases, and Data Lifecycles

Status: draft · **Chapter ID:** CH13

Author: Edmund Gunn, Jr.

Manuscript: working full-manuscript draft · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does not claim Gate 3 completion).

13.1 1. The moment

You hit Save. Or you close a tab. Or you walk away assuming the document will be there tomorrow the way you left it.

Later the file is missing, an older version returns, a “recovered document” dialog appears, or two copies of the same cloud note disagree. From the seat it feels like storage is broken—or like the computer betrayed a promise you thought was simple.

Underneath that feeling, several persistence stories collide: working memory that forgets when power leaves, filesystems that organize durable bytes, application buffers that may still be holding the truth you thought you saved, optional cloud replicas that sync on their own schedule, and policies about how long data is kept or removed.

This chapter’s governing question:

When I save, reopen, share, or delete, what durable-state contracts and lifecycle stages must hold—and why is “saved” not the same as “truth forever”?

Part III is about making hardware useful. Chapters nearby treat boot, trust, and the operating system’s handoff. Here the useful abstraction is **durable state**: named files, structured stores, and the human lifecycle from create to use to retain to share to delete or redact—without turning the chapter into forensics, undelete cookbooks, or invented product benchmarks.

Device Quartet storage endurance and performance numbers remain **PHYSICAL_PENDING** (CLM-CH13-006). Commodity observation is enough for the labs this chapter inherits.

13.2 2. What you notice

Before names like *filesystem* or *durability*, notice the human contracts you already expect.

You expect Save to mean the work will reopen. You expect Quit not to erase what you believed was durable. You expect “in the cloud” to mean another place holds a usable copy—not that every device instantly shares one perfect version. You expect Delete to reduce how available something is, at least for ordinary use. You also expect failure to be visible: a permission error, a conflict banner, a recovered-draft dialog—not silent wrong content that looks complete.

Those expectations *are* the product from the person’s point of view.

A persistence experience is produced by layered contracts: working memory is not durable storage; UI Save is not always proof bits reached stable media; replicas can disagree without the local filesystem being “broken.”

Notice the split timelines. What is on screen can still live only in RAM. What the app calls saved may still sit in a buffer. What one device shows may lag another replica. Optional comparison on a device you already own: type a short throwaway note in a local editor, save, quit, reopen—then compare that to an unsaved buffer you deliberately leave open. You are not collecting a product benchmark. You are noticing that “still visible” and “durable across quit” are different experiences (CLM-CH13-001) (Tanenbaum and Bos 2022).

A second optional notice: if you already use a cloud document, open the same note on two sessions you control and watch for conflict or version cues—**without** treating a conflict banner as proof that your disk failed. Label what you observe versus what you infer.

13.3 3. Exploded ecosystem

Persistence is not one object. It is a path through an ecosystem. **FIG-CH13-002** is the first-minute durability stack: app buffer → filesystem → device media → optional cloud replica. Treat it as **Representative educational architecture**, not a claim that every phone or laptop wires exactly like the diagram.

Walk the layers in ordinary language.

13.3.1 Human

You form intent: keep this draft, finish tomorrow, share with a teammate, remove something that should not linger. Hands hit Save. Eyes check filenames and conflict banners. Later you judge whether the reopen matched the promise.

13.3.2 Application buffer / working set

Editors and apps keep a live working copy in memory. That copy can feel complete while still depending on process lifetime. **RAM is not a substitute for durable storage** under ordinary power-off and quit conditions (CLM-CH13-001) (Tanenbaum and Bos 2022).

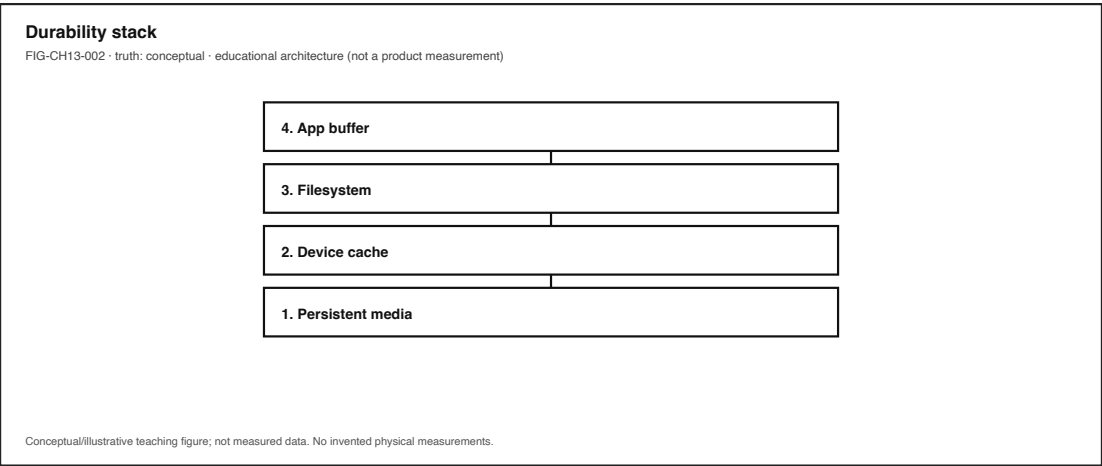


Figure 13.1: Durability stack: app buffer → filesystem → device media → optional cloud replica. Conceptual.

13.3.3 File and filesystem

A **file** is the ordinary named durable byte-sequence abstraction on personal devices. A **filesystem** organizes files and directories with metadata and durability rules the OS exposes to apps (Tanenbaum and Bos 2022). From the seat, “Documents” is a place. Underneath, it is a contract about names, permissions, and when writes become durable enough to survive crashes—within the OS’s stated assumptions.

13.3.4 Caches and write-back

Operating systems and devices use caches and buffers for speed. **Write-back** behavior can delay when data reaches stable media. The Save button is a human signal; it is not always instantaneous proof that bits are on non-volatile media (CLM-CH13-002) (Tanenbaum and Bos 2022; Saltzer and Kaashoek 2009). Systems design treats naming, buffering, and failure as first-class concerns rather than UI decorations (Saltzer and Kaashoek 2009).

13.3.5 Device media

Flash, disk, and related controllers hold durable bytes—until wear, full volumes, permission failures, or hardware faults intervene. Quartet-specific endurance curves stay **PHYSICAL_PENDING** (CLM-CH13-006). Classroom work uses commodity devices and fixtures, not invented EVT telemetry.

13.3.6 Structured stores (qualitative)

Many apps also keep state in **databases** or other structured stores: tables, indexes, and concurrency rules sitting above raw files. This draft treats that as a **responsibility split**—structure and concurrent access versus a single file blob—anchored to official concurrency docs (CLM-CH13-003 · **SOURCE_IDENTIFIED** via (“Concurrency Control” 2026)) without inventing textbook ACID slogans. **FIG-CH13-004** sketches file versus database responsibilities as a teaching plate, not as product endorsement.

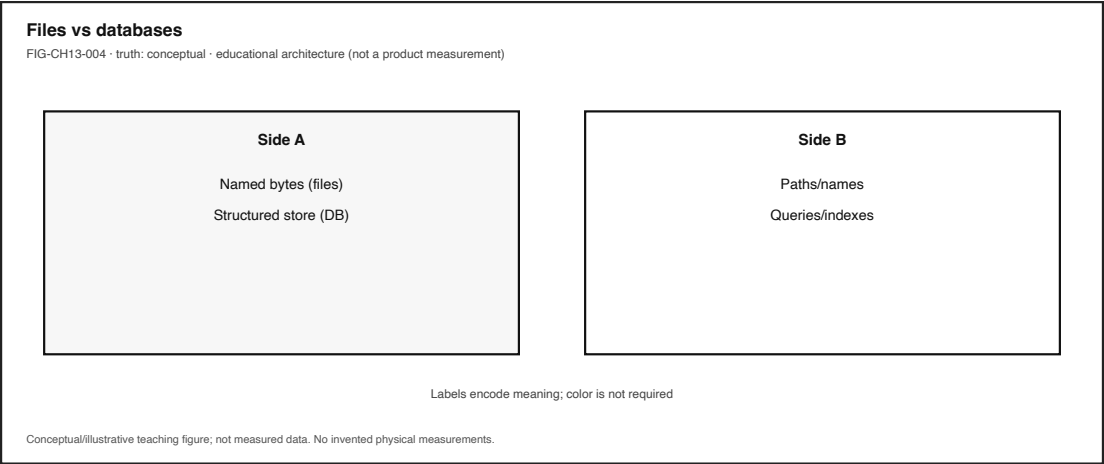


Figure 13.2: File vs database responsibilities. Teaching split; not product endorsement.

13.3.7 Sync and replicas (qualitative)

Optional cloud sync adds copies that update over time. From the seat, a conflict can feel like “storage broke.” A more careful reading: multiple durable states can exist at once, and disagreement is a distributed-state symptom to observe—not automatic proof that the local filesystem failed. Specific sync theorems and vendor conflict algorithms remain SOURCE_NEEDED (CLM-CH13-004 omitted as a sourced claim); this chapter keeps the honesty rule: label conflict UI as observation.

13.3.8 Lifecycle policy

Beyond mechanism sits **lifecycle**: create/collect → use → retain → share → delete/redact. **FIG-CH13-001** is the lifecycle arc. Policy language (“we delete after...”) and mechanism (what the UI removes) can diverge; later sections stay qualitative and refuse recoverability cookbooks.

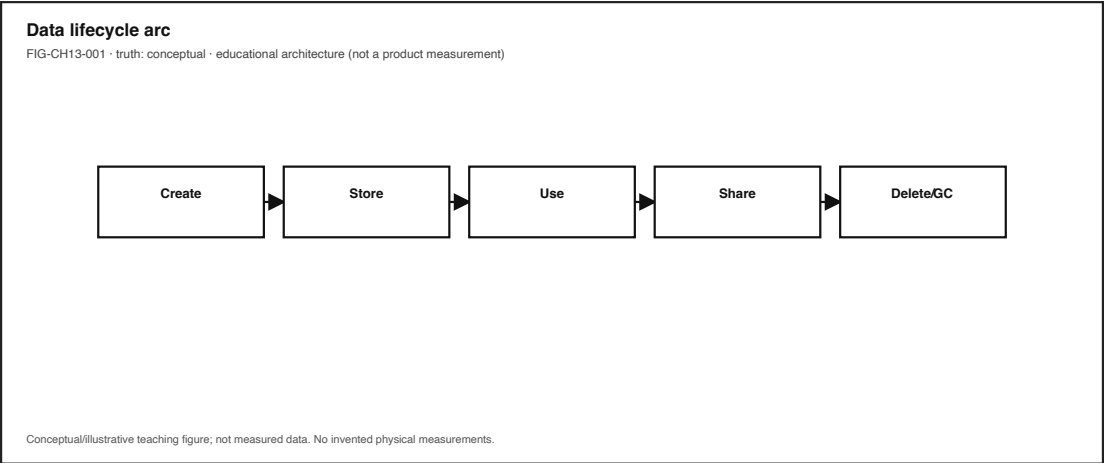


Figure 13.3: Data lifecycle: create/collect → use → retain → share → delete/redact. Conceptual.

13.3.9 System software

The OS mediates process lifetime, file APIs, permissions, and often sync agents. Apps still own when they call durable write paths. Mis-attribution is easy: blaming “the cloud” for a local unsaved buffer, or blaming “the disk” for a replica conflict.

13.4 4. Follow the signal

Here the “signal” is durable state moving through time—not a tap packet. **FIG-CH13-003** contrasts a Save click with a later durability point such as flush/fsync conceptually. Read it as a logical story, not as a claim that every app executes identical steps.

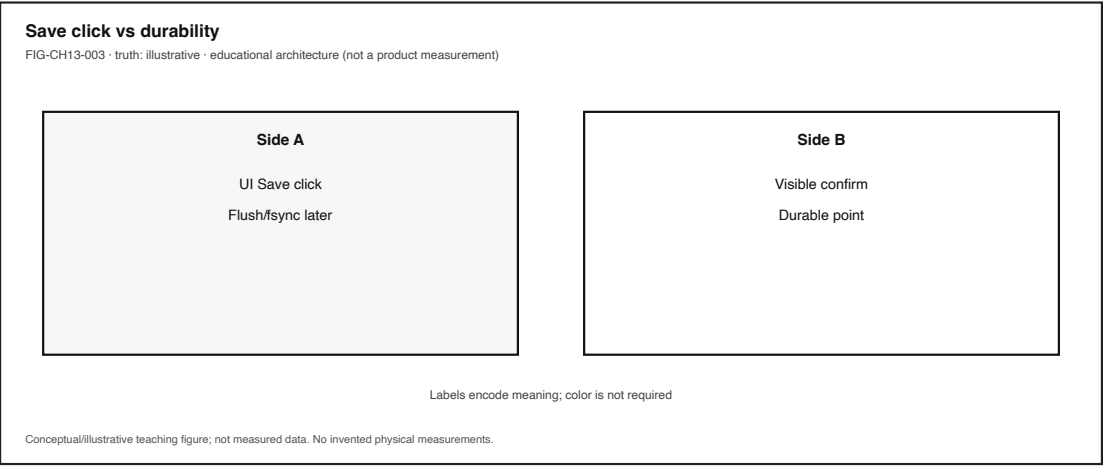


Figure 13.4: Save click vs later durability point (flush/fsync conceptually). Illustrative; no invented latencies.

1. **Intent.** You decide content should survive quit, crash, or tomorrow’s reopen.
2. **Edit in working memory.** Keystrokes update an in-memory buffer; the screen looks complete.
3. **Save action.** UI Save (or autosave) requests persistence.
4. **App write path.** The application issues write APIs toward a file or structured store.
5. **OS buffering.** Caches and write-back may hold data for performance before stable media acknowledges it (CLM-CH13-002) (Tanenbaum and Bos 2022; Saltzer and Kaashoek 2009).
6. **Durable media.** Bytes reach device storage—or fail (full volume, permission denied, I/O error).
7. **Optional replica.** A sync agent may upload, download, or merge copies on another device or service.
8. **Human reopen / share / delete.** Later experience tests whether the contract matched expectations.
9. **Feedback.** Success toast, conflict banner, recovered-draft dialog, or silent surprise closes the loop—or fails to.

13.4.1 Alternate paths (honesty rule)

Path	Everyday example	What changes
Local file, explicit Save	Notes app → Save → quit → reopen	Durability depends on write completion, not screen content
Autosave buffer	Recovered document after crash	Partial durability; timing uncertain from outside
Unsaved tab	Close without Save	Working memory ends with the process
Cloud conflict cue	Two devices edited offline	Replica disagreement symptom—not proven local FS failure
Delete in UI	Trash / remove	Ordinary availability may change; global unrecoverability is a separate, often unverified claim

Do not invent flush latency budgets. Do not treat one successful reopen as proof that every future autosave is durable.

13.5 5. Component cards

Component cards answer: What is it? What does it do for the person? What fails when it misbehaves?

13.5.1 File

Plain definition. A named durable sequence of bytes managed through a filesystem (Tanenbaum and Bos 2022).

Experience benefit. Work can outlive the process that created it.

Failure symptom. Missing file, permission denied, truncated content, or reopen that does not match what was on screen.

13.5.2 Filesystem

Plain definition. Software organizing files, directories, and metadata on storage with durability and permission rules (Tanenbaum and Bos 2022).

Experience benefit. Names, folders, and ordinary Save/Open paths stay navigable.

Failure symptom. Corrupt directory views, unexpected “file in use,” or space/permission failures surfaced as app errors.

13.5.3 Cache vs durable store

Plain definition. Fast temporary copies can present a complete picture before data is durable on stable media (CLM-CH13-002) (Tanenbaum and Bos 2022; Saltzer and Kaashoek 2009).

Experience benefit. Snappy editing and Saves that feel instant.

Failure symptom. Power loss or crash after a green checkmark still loses work; “saved” and “on media” diverge.

13.5.4 Durability

Plain definition. A promise that committed data survives expected crashes or power loss—within stated assumptions (Tanenbaum and Bos 2022; Saltzer and Kaashoek 2009).

Experience benefit. Reopen tomorrow matches the last honest commit.

Failure symptom. Lost edits, recovered partial drafts, or silent rollback to an older version.

13.5.5 Database (intro, qualitative)

Plain definition. A structured durable store with query and concurrency responsibilities beyond a single opaque file blob.

Experience benefit. Apps can keep related records consistent enough for everyday use without the person managing dozens of raw files.

Failure symptom. App-level “database error,” stuck sync of structured state, or migrations that block open—without this chapter claiming a sourced ACID textbook definition (CLM-CH13-003 omitted).

FIG-CH13-004 keeps the teaching split: files as named byte sequences; databases as structured stores with their own failure modes.

13.5.6 Consistency model (intro, qualitative)

Plain definition. Rules for when readers see writers’ updates—especially across replicas or sync.

Experience benefit. Shared documents feel coherent enough for the task.

Failure symptom. Conflict banners, divergent copies, or “old version returned.” Formal distributed consistency citations stay SOURCE_NEEDED (CLM-CH13-004 omitted).

13.5.7 Data lifecycle

Plain definition. Stages of data over time: create/collect → use → retain → share → delete/redact.

Experience benefit. People can map where a note goes and who can see it.

Failure symptom. Retention longer than expected, sharing broader than expected, or delete that does not match disclosed expectations—discussed qualitatively only.

13.5.8 Deletion / redaction (qualitative)

Plain definition. Removing or limiting ordinary availability of data; policy language and mechanism can differ.

Experience benefit. A person can reduce how present something is in everyday apps.

Failure symptom. Trash restores, synced copies elsewhere, or backups that retain content. This chapter states **uncertainty**—not a recoverability procedure (CLM-CH13-005 omitted as a sourced claim; no undelete cookbook).

13.6 6. Stability contract

The **Stability Contract** returns with persistence as a first-class condition:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds—including durable writes, coherent enough reads, visible or safely handled sync disagreement, and retention/deletion behavior that matches disclosed expectations.

For this chapter, a sustainably honest save/reopen/share experience may require all of the following to stay “good enough” at once:

- writes needed for the experience reach a durable store within a delay the person can accept (qualitative—no invented millisecond budgets),
- reads return coherent enough state for the task,
- app buffers and OS caches are not mistaken for completed durability,
- sync or replica conflicts are visible or resolved safely enough that the person is not silently misled,
- deletion and retention policies match what was disclosed—at least for ordinary use,
- status cues (saved, conflict, offline, permission denied) remain perceivable as text, not color-only icons,
- no lab asks for forensics, undelete exploits, or paid-cloud requirements.

Three separations matter:

1. **On screen durable.** RAM and dirty buffers can look complete (Tanenbaum and Bos 2022).
2. **Save UI instantaneous stable media.** Write-back can delay durability (CLM-CH13-002) (Tanenbaum and Bos 2022; Saltzer and Kaashoek 2009).
3. **Conflict UI proof the filesystem is broken.** Treat replica disagreement as an observation pending evidence (CLM-CH13-004 qualitative only).

Device Quartet storage Stability Contract numbers remain **PHYSICAL_PENDING** (CLM-CH13-006).

13.7 7. Try it

13.7.1 LAB-CMS-001 — Experience B adjacency (save / quit / reopen)

Observable question. After a controlled Save (or deliberate non-Save), quit, and reopen on a device I already own—or via offline fixtures—what content returned, and what cues appeared—without claiming root cause beyond observation?

Inheritance note. **LAB-CMS-001** (*Make Local Slowness Visible*) already ships Experience B as an optional persistence check beside Experience A’s local-lag path. This chapter **inherits** that adjacency; it does not rename the lab. Prefer the published routes and fixtures under `labs/LAB-CMS-001/`.

WAIKE alignment note. WAIKE accepted main (SHA e97e74fc9bfb44b1cdc26b272dc4848264f15f) includes adjacent digital_rc labs such as GENERAL_IT / lab_storage and GENERAL_IT / lab_backup. Those are **neighbors**, not renamed publication IDs. Proposed **LAB-DATA-LIFE-001** remains publication-owned ideation only until implemented.

Prerequisites. A commodity computer or phone you may use for learning; or LAB-CMS-001 offline fixtures when shared-lab policy or equity requires them.

Safety and privacy (mandatory).

- Do **not** capture personal document contents, passwords, tokens, or private photos.
- Redact filenames, account names, and thumbnails before any portfolio leave-device.
- Do not require a paid cloud account; fixture transcripts satisfy completion.
- No undelete tools, disk imaging, forensic recovery steps, or attempts to bypass trash/retention.
- Mild local edits only; stop on thermal warnings.

Time estimate. About 20–40 minutes for Experience B alone; longer if combined with Experience A from LAB-CMS-001.

13.7.1.1 Prediction

Write one sentence: after Save → quit → reopen, do you expect full content, partial recovery UI, or loss—and why?

13.7.1.2 Route A — Commodity local editor (baseline)

1. Create a short throwaway file with a unique phrase you will recognize.
2. Save explicitly. Note any UI cue (“Saved”, checkmark, path).
3. Quit the app fully (not only hide a window).
4. Reopen the file. Record whether the phrase returned.
5. Optional second trial: type a second phrase, **do not** Save, quit, reopen. Compare.
6. Label each row **observed** vs **inferred**.

13.7.1.3 Route B — Optional cloud conflict screenshot (equity-safe)

Only if you already have access without new paid accounts: open the same cloud note in two sessions you control and look for conflict or version cues. Capture at most one scrubbed screenshot. If you lack access, skip—fixtures and Route A remain enough.

13.7.1.4 Route C — Offline fixture (required fallback)

Use LAB-CMS-001 fixtures / Experience B instructions when a personal device is unavailable. Treat fixture numbers and transcripts as **illustrative**, not measurements of your hardware.

13.7.1.5 Lifecycle adjacency — LAB-TRUST-001

For retain/share/delete language, **LAB-TRUST-001** already includes a consent/trust card and data-lifecycle vocabulary. Do not reassign that lab. Optionally complete one lifecycle row (retain / share / delete-redact) on a throwaway example after Experience B—still with redaction rules.

13.7.1.6 Evidence (minimum)

- observation table (Save vs non-Save, or fixture IDs),
- one scrubbed screenshot or written status list,
- one paragraph separating observation from inference,
- explicit statement: no forensic recovery attempted.

13.7.1.7 Limits (say them out loud)

- One reopen does not prove all future autosaves.
- Conflict UI is not a filesystem root-cause report.
- “Deleted” in a UI is not a sourced claim of global unrecoverability.
- Quartet storage benches stay `PHYSICAL_PENDING`.

13.7.1.8 Portfolio output

Follow LAB-CMS-001 portfolio norms: README (question, method, limits), observation table, teach-back (RAM vs durable file), and redaction confirmation.

13.8 8. Build it

Use the persistence story at the depth that matches your pathway.

13.8.1 Explorer

Draw a personal **data lifecycle** postcard: create → use → retain → share → delete/redact for one throwaway note. Use ordinary words. Mark which stages you observed in Try it.

13.8.2 Operator

From LAB-CMS-001 Experience B notes, build a symptom checklist: missing content, recovered-draft dialog, conflict banner, permission denied. Pair each with “needs more evidence”—never with fake certainty.

13.8.3 Builder

Design a one-page **durability checklist** for a tiny app: when is content only in memory; when do you request a durable write; what UI cue will you show if write fails; what will you refuse to claim about fsync timing without measurement? Keep flush language conceptual (Tanenbaum and Bos 2022; Saltzer and Kaashoek 2009).

13.8.4 Engineer

Compare **file vs database responsibilities** for one use case (for example, a homework tracker): what belongs in a named file, what needs structured records and concurrent updates. Keep the comparison qualitative; cite concurrency docs without inventing ACID slogans beyond published definitions (CLM-CH13-003 · SOURCE_IDENTIFIED). Sketch against **FIG-CH13-004**’s teaching split.

13.8.5 Researcher

State uncertainty about deletion across replicas in one careful paragraph: what a UI delete can mean, what would still be unknown without primary policy and storage documents, and why this book refuses recoverability steps. Keep Quartet endurance claims PHYSICAL_PENDING (CLM-CH13-006).

Educators can run Route C fixtures when devices cannot quit/reopen freely, and can treat redaction as a first learning outcome—not an afterthought.

13.9 9. Secure and include it

13.9.1 Security

Files and databases are access-control surfaces. Ordinary posture: least privilege for shared folders, care with sync of sensitive notes, and refusal to disable security tools “to make Save faster.” This chapter does not teach unauthorized access, bypass, or recovery exploits.

13.9.2 Privacy

Lifecycle honesty matters: collect, use, retain, share, and delete/redact are human stakes, not only storage jargon. Discuss deletion as **policy plus mechanism**—qualitatively. Media-sanitization guidance (CLM-CH13-005 · SOURCE_IDENTIFIED via (“Guidelines for Media Sanitization” 2014)) supports “UI delete globally unrecoverable” without giving recovery exploit steps. Portfolio artifacts must scrub identifiers; do not sync lab evidence to cloud accounts as a requirement.

13.9.3 Accessibility

Sync, save, and error status must be perceivable as text (or equivalent), not color-only dots. Conflict and offline banners should remain reachable with keyboard and screen-reader paths where the platform supports them. Filenames and paths in teaching materials need text equivalents, not screenshot-only instructions.

13.9.4 Equity

Do not require paid cloud seats. Offline fixtures and local editors are first-class. Learners on shared or low-storage devices must be able to complete Experience B without buying upgrades. Backup and sync literacy includes naming cost barriers without pretending every classroom can stock every service.

13.9.5 Safety and ethics

No forensics labs. No undelete cookbooks. No fabricated durability latency tables. No shipping-SKU claims about Quartet storage. Overclaiming “deleted forever” or “always synced” is still a form of false evidence.

13.10 10. Career lens

Persistence work crosses many ownership domains. No table promises employment; roles vary by organization. Artifacts from LAB-CMS-001 Experience B resemble early professional habit: prediction, observation/inference split, and explicit uncertainty.

Layer	Example role	Professional artifact	Lab resemblance
Application durability	Backend / ROLE-BACKEND	Save/flush design notes; failure UX	Builder durability checklist
Filesystems / OS	Systems / OS engineer	FS and caching behavior notes	Save vs buffer separation; (Tanenbaum and Bos 2022)
Data platform	Data engineer	Pipeline retain/delete policy map	Explorer lifecycle postcard
Database operations	DBA-adjacent practitioner	Backup/restore runbooks (authorized)	Engineer file-vs-DB split (qualitative)
Sync / cloud	Cloud or mobile engineer	Conflict-resolution UX + limits	Operator conflict-symptom checklist
Privacy	Privacy engineer (see CH24)	Retention and deletion disclosures	Qualitative delete uncertainty note
Support / IT	Field or support engineer	Triage: unsaved buffer → disk → sync	Experience B observation table
Accessibility	Accessibility specialist	Status-cue perceivability review	Text status for save/sync/errors

Portfolio hint: a scrubbed reopen table plus a lifecycle postcard is more honest than vibes-based “the cloud ate my homework.”

13.11 11. Check understanding

Answer with reasoning. Prefer short paragraphs over one-word guesses.

1. Why can content on screen disappear after quit even though you “saw it finished”?
2. Why is a Save checkmark incomplete evidence that bits are on stable media?
3. What is the difference between a **file** and a **filesystem** in one careful sentence each?
4. Why might a cloud conflict banner appear even when the local disk is healthy? What must you *not* claim without more evidence?
5. Name the lifecycle stages create → use → retain → share → delete/redact and give one human stake for delete/redact that stays qualitative.
6. Why does this chapter refuse undelete or forensic recovery steps in labs?
7. Why are Device Quartet storage endurance claims labeled PHYSICAL_PENDING?
8. **Teach-back.** Explain to a family member—without saying *write-back*, *inode*, or *replica*—why Save and “still on screen” are different promises. Then introduce those three terms one at a time, tied to something already understood.

Educator note: success is causal sequence (buffer → write → durable media → optional sync → human reopen) plus at least one privacy/uncertainty boundary, not a product catalog.

13.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Project-specific Device Quartet storage evidence remains **PHYSICAL_PENDING** (CLM-CH13-006) and is tracked in the chapter claim plan, separately from external literature.

Inline citations used in this chapter include Tanenbaum and Bos (2022) and Saltzer and Kaashoek (2009).

Omitted as cited technical claims (SOURCE_NEEDED remaining in the chapter claim plan):

- **CLM-CH13-003** — databases add structure/concurrency/recovery relative to raw files (SOURCE_IDENTIFIED via `postgresql-mvcc`; no invented textbook ISBN).
- **CLM-CH13-004** — cloud sync conflicts as distributed-state problems (needs distributed-systems chapter and/or official sync docs).
- **CLM-CH13-005** — deletion as policy plus garbage collection plus replicas (SOURCE_IDENTIFIED via `nist-sp800-88r1`; no recovery exploit steps).

Prose above treats those topics **qualitatively** or omits them as sourced claims.

13.13 12. Glossary links

Terms introduced or relied on as formal vocabulary in this chapter should resolve in the living glossary registry as candidates mature. This section lists them for linking—not as a dump of free-standing encyclopedia entries.

Term	Role in this chapter
File	Named durable byte sequence via a filesystem
Filesystem	Organization of files/directories with metadata and durability rules
Database	Structured durable store beyond a single file blob (intro)
Durability	Committed data survives expected failures within assumptions
Cache vs durable store	Fast copies can precede stable-media durability
Consistency model (intro)	When readers see writers’ updates across replicas/sync
Data lifecycle	Create/use/retain/share/delete-redact over time
Deletion / redaction	Limiting availability; policy and mechanism can differ

Term	Role in this chapter
Stability Contract	Experience depends on hidden conditions staying acceptable
Write-back / buffer	Delayed durability behind a fast Save feel

Deeper entries and “not the same as” warnings live in the glossary network. Prefer following a link when stuck over inventing a private definition.

Related earlier chapters: memory and storage adjacency (CH07), OS abstractions (CH12). Related later chapters: networked services (CH14–CH15), privacy depth (CH24).

13.14 Figure references (embedded; registered SVG + ally)

All four figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured evidence. No fabricated telemetry. Device Quartet storage curves remain PHYSICAL_PENDING.

13.14.1 FIG-CH13-001 — Data lifecycle (create → use → retain → share → delete/redact)

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Lifecycle diagram.
- **Reader should notice.** Ordered stages with human stakes at delete/redact; policy vs mechanism note.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name all five stages in order; state conceptual truth class; no recoverability instructions.

13.14.2 FIG-CH13-002 — App buffer → filesystem → device media → optional cloud replica

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Comparative layers.
- **Reader should notice.** Left-to-right durability stack; RAM/buffer distinct from durable media.
- **Truth class.** Conceptual / Representative educational architecture.
- **Alt text requirement.** List layers in reading order; deny Quartet EVT implication.

13.14.3 FIG-CH13-003 — Save click vs durability point

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Sequence diagram.
- **Reader should notice.** Save UI precedes possible flush/fsync durability; failure branch for incomplete write.
- **Truth class.** Illustrative.
- **Alt text requirement.** Enumerate steps; label illustrative; no invented latency numbers.

13.14.4 FIG-CH13-004 — File vs database responsibilities

- **Production status.** `embedded` (registered SVG + accessibility sidecar).
- **Type.** System map / comparative.
- **Reader should notice.** Named bytes vs structured store responsibilities; both can fail.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name both columns; state that formal DB textbook pinning is still `SOURCE_NEEDED` for stronger claims.

13.15 Claim footnotes used in this chapter

Claim ID	Approved gist	Classification
CLM-CH13-001	Files are ordinary persistence on personal devices; RAM is not a durable substitute	<code>general_technical</code> · <code>SOURCE_IDENTIFIED</code> via Tanenbaum and Bos (2022)
CLM-CH13-002	Write-back caches/buffers can delay durability; Save UI proof on stable media	<code>general_technical</code> · <code>SOURCE_IDENTIFIED</code> via Tanenbaum and Bos (2022), Saltzer and Kaashoek (2009)
CLM-CH13-003	Databases add structure/concurrency/recovery vs raw files	<code>SOURCE_IDENTIFIED</code> (<code>postgresql-mvcc</code>); qualitative responsibility split
CLM-CH13-004	Cloud sync conflicts are distributed-state symptoms, not proof FS broke	<code>SOURCE_NEEDED</code> — reframed qualitatively ; observation vs inference only
CLM-CH13-005	Deletion is often policy + GC + replicas; UI delete global unrecoverable	<code>SOURCE_IDENTIFIED</code> (<code>nist-sp800-88r1</code>); no recovery steps
CLM-CH13-006	Quartet storage endurance/performance	<code>PHYSICAL_PENDING</code>

General teaching statements that working memory is volatile relative to durable files are tied to CLM-CH13-001 and the cited OS/systems texts. Any future numeric durability or endurance figures must carry **illustrative**, **measured**, or **inferred** labels—and Quartet measured figures stay blocked until `PHYSICAL_PENDING` clears.

Chapter 14

Chapter 14 — Applications, APIs, Runtimes, and User Interfaces

Status: draft · **Chapter ID:** CH14

Author: Edmund Gunn, Jr.

Manuscript: working full-manuscript draft · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (not Gate 3 PASS; this chapter does not edit Gate 3 packets)

14.1 1. The moment

You launch an app you already use. The skeleton appears almost at once: icon, title bar, navigation chrome, maybe a gray tile where a list should live. Something about the surface says *I am here*.

Then you wait.

The list stays empty. A spinner turns. Yesterday's content sits politely while a refresh never quite finishes. From your seat it still feels like *one app*. Underneath, a **user interface**, a language **runtime**, libraries and frameworks, local OS services, and—often—remote **APIs** are cooperating, stalling, or failing on different clocks.

This chapter's promise is practical:

After this chapter, you can explode an ordinary app into UI, runtime, libraries, and APIs—and explain why chrome-ready is not the same event as work-complete.

Chapter 1 / Concept Edition CE-1 already taught the system lens: the screen is not the whole machine, and chrome can arrive before content (Saltzer and Kaashoek 2009). Chapter 2 already proved a method: follow one interaction through the stack with **LAB-TAP-001**. This chapter stays in Part III's job—make OS services usable to humans—by naming the *application stack* those earlier chapters opened without replaying their labs as duplicates.

14.2 2. What you notice

Before jargon like *event loop* or *API version*, notice the human contracts you already expect.

You expect the open or tap to be recognized—finger, trackpad, keyboard, switch, or voice. You expect the shell of the experience to appear soon enough that the product feels awake. You expect usable content to arrive, or a failure you can understand. You expect the interface not to seize up while something invisible works. You expect “connected” status—when it matters—to relate to the *service this feature needs*, not merely to a radio icon. You also expect battery and heat to stay within reason for something so ordinary.

Those expectations are not decorations around software. From the person’s point of view, they *are* the product.

Chrome visible is not the same event as content usable—and neither is the same as “all APIs finished.”

A title bar can look finished while a handler is still waiting. A skeleton can look “alive” while a local cache miss or a remote 500 sits unseen. Stale content can look complete while a versioned API call never returns. Your nervous system may stitch those timelines into “the app opened” or into “something feels wrong even though the screen is there” (CLM-CH14-003).

Optional comparison on a device you already own: open a note, photo, or file that already lives locally. Then open something that must fetch fresh remote content. The first often finishes inside the device. The second may add waiting that has nothing to do with how hard you pressed the glass. CE-1’s readiness story and Chapter 2’s local-versus-remote fork are the adjacent methods; here we keep the same honesty without inventing launch-time budgets.

Accessibility belongs in this noticing, not later as a disclaimer. If the only “busy” cue is a colored spinner, someone who does not see that color—or someone using a screen reader that never hears a ready announcement—experiences a different application than the designer imagined. Inclusive readiness is part of the human contract; WCAG 2.2 states that obligation in two dated Recommendation editions that this book cites separately rather than collapsing onto an undated shortcut (W3C 2023, 2024).

14.3 3. Exploded ecosystem

An ordinary app experience is not a single binary floating alone. It is a path through cooperating parts: people, UI, application logic, runtime, libraries, OS services, and optional remote APIs (CLM-CH14-001) (Saltzer and Kaashoek 2009; WHATWG 2026b). **FIG-CH14-001** is the first-minute stack map: UI → app logic → runtime → libraries → OS services → optional remote API. Treat it as **Representative educational architecture**—not a measured teardown of any brand’s process list.

Walk the layers in ordinary language. Keep the same layers when vocabulary deepens.

14.3.1 Human

You form intent: refresh this list, send this note, open this portal. Muscles, voice, or assistive controls issue the action. Later, eyes, ears, and hands judge whether the result matches what

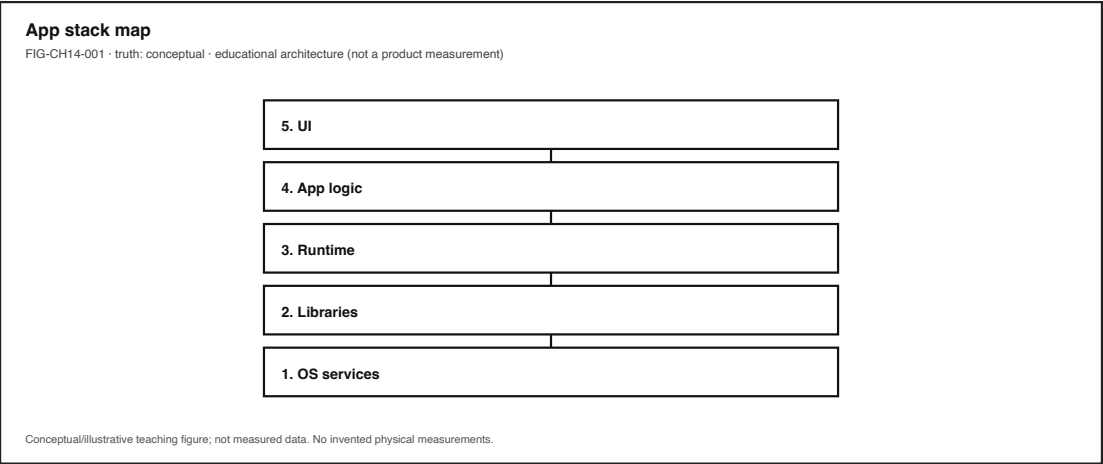


Figure 14.1: UI → app logic → runtime → libraries → OS services → optional remote API. Conceptual stack.

you meant.

14.3.2 User interface (UI)

The presentation and interaction surface—visual layout, spoken status, assistive-technology (AT) trees, chrome, content, and failure text. The UI is a cooperating part, not the whole story (WHATWG 2026b).

14.3.3 Application logic

Handlers, state, and feature code that decide what the interaction *means*. This is where “Refresh” becomes a sequence of local reads and optional remote calls—not where pixels are painted by magic.

14.3.4 Runtime

The language or platform environment that schedules work: garbage collection, timers, an **event loop** or equivalent dispatcher, and the rules for when UI work may run (WHATWG 2026b). Chrome can render from one scheduled path while content waits on another.

14.3.5 Libraries and frameworks

Reusable code that shapes how apps are built—UI toolkits, networking helpers, serialization, routing. They are opinionated neighbors of the runtime, not synonyms for “the OS.”

14.3.6 OS services (local APIs)

Permissions, storage, notifications, clipboard, sensors, and other host-provided contracts the app calls without leaving the device. Local does not mean free of failure: a denied permission is still a broken Stability Contract for the feature that needed it.

14.3.7 Optional remote APIs

When the feature needs fresh off-device work, a network path and a remote service contract enter. Packet and path literacy live in Part IV and in **LAB-PKT-001**; here, keep the teaching word **optional** honest.

14.3.8 Human-usable result

Content you can act on—or a clear failure. The ecosystem succeeds only when the person can finish the job they came to do.

Equity belongs in the map: always-online assumptions exclude learners on metered plans, shared devices, captive portals, or intermittent links. Low-cost devices remain first-class learning instruments. Device Quartet form factors may appear later as research/learning spines; Quartet UI latency budgets remain **PHYSICAL_PENDING** (CLM-CH14-005)—this chapter does not invent millisecond product laws.

14.4 4. Follow the signal

Everyday interactive software commonly waits for inputs, dispatches handlers, updates remembered **state**, and presents outputs (WHATWG 2026b, 2026a). **FIG-CH14-002** shows one teaching path for a feature after chrome is already visible: event → handler → API call → state update → render / AT feedback. Read it as a logical story, not as a claim that every platform uses one identical event-loop implementation or that steps never overlap.

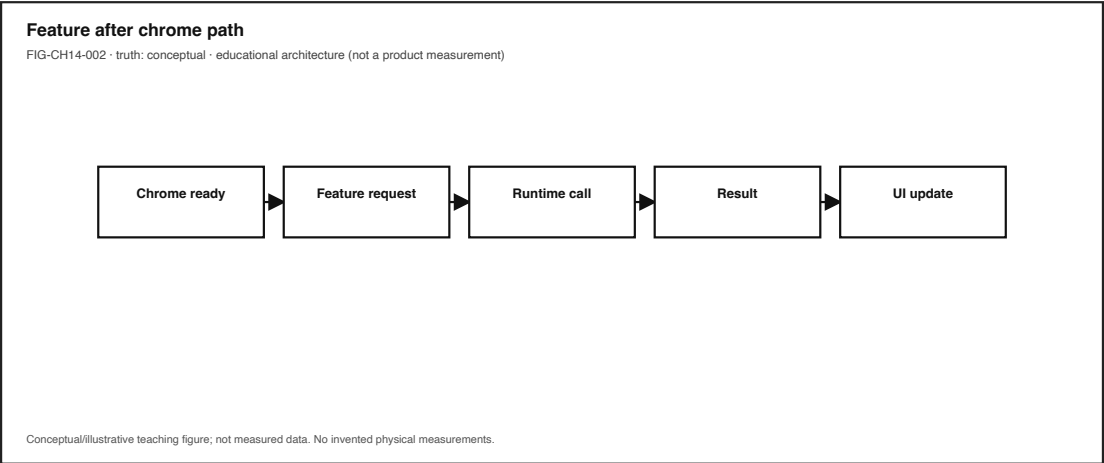


Figure 14.2: Event → handler → API → state → render/AT feedback. Conceptual sequence; steps may overlap.

A useful reading of the chrome-before-content moment inside an already-running app:

1. **Input arrives** — tap, key, switch, or voice becomes an event the software can interpret.
2. **Handler runs** — application logic decides what work is needed.
3. **Local API path** — storage, permission, or OS service may be consulted first.
4. **Optional remote API path** — if needed, a request may leave the device toward a versioned service contract.

5. **State updates** — success, failure, or partial data is remembered.
6. **Outputs update** — pixels, sound, haptics, and AT announcements reach the human.
7. **Human judgment** — ready enough to use, still busy, or failed.

FIG-CH14-003 separates **local API** failure domains from **remote API** failure domains. Same call metaphor (“ask for data”); different latency, authorization, and blame stories. Association DNS route auth this API version.

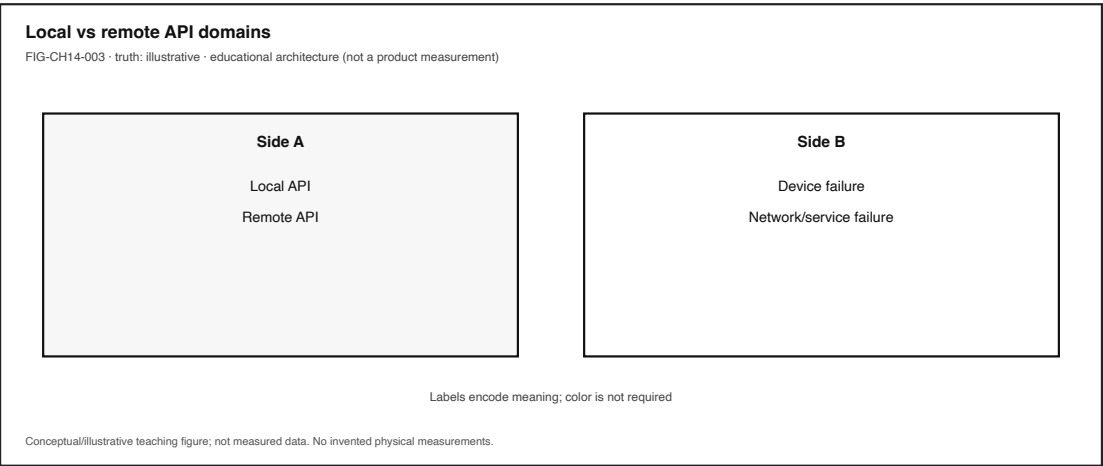


Figure 14.3: Local API vs remote API failure domains. Illustrative teaching comparison.

14.4.1 Alternate paths (the honesty rule)

- Opening a local draft may finish without any remote call.
- A messaging chrome may show while media attachments still travel.
- A portal may show a healthy connectivity icon while the application API still returns 401 or 503.

Illustrative teaching point (not a measured universal product fact): a connectivity indicator is related to, but not identical with, application API success.

Chapter 2’s cross-layer sequence remains the canonical method for *one tap through the whole stack*. This chapter zooms the middle of that story—application, runtime, and API contracts—without replacing **LAB-TAP-001**.

14.5 5. Component cards

These cards are orientation tools—not a complete bill of materials. Use them to name parts when something feels wrong.

14.5.1 User interface

What it is. The surface the person can see or hear: chrome, content, status text, spoken updates, AT tree.

Why it matters. It can look finished while handlers and APIs are incomplete.

Common misconception. “If I can see the screen, the work is done.”

Failure hint. Forever spinner; empty skeleton; missing ready announcement.

14.5.2 Application / feature logic

What it is. The code that interprets events and chooses local or remote work (WHATWG 2026b).

Why it matters. Chrome and content often travel different code paths.

Common misconception. “The app” is a single indivisible object.

Failure hint. Crash after chrome; frozen shell; endless redirect that still shows a title.

14.5.3 Runtime / event loop

What it is. The environment that schedules timers, I/O callbacks, and UI work (WHATWG 2026b).

Why it matters. A hung UI thread can make a healthy network look broken.

Common misconception. Slow feel always means “bad Wi-Fi.”

Failure hint. Whole UI seizes; other apps on the device still feel fine—or everything feels laggy (then look beyond one app).

14.5.4 Library / framework

What it is. Reusable structure that shapes routing, rendering, and networking helpers.

Why it matters. Framework defaults become Stability Contract defaults for users.

Common misconception. Framework name equals understanding of failure domains.

Failure hint. Errors wrap so deeply that the person only sees a generic toast.

14.5.5 API (contract)

What it is. An interface contract for invoking functionality across a boundary—library, OS service, or remote service (Saltzer and Kaashoek 2009).

Why it matters. Breaking changes and versioning change what users can trust (CLM-CH14-002) (Preston-Werner, n.d.).

Common misconception. “API” always means a public HTTPS JSON endpoint.

Failure hint. Local permission denied; remote 404 after a silent version bump; client built against yesterday’s contract.

14.5.6 Local vs remote API

What it is. Same call metaphor; different failure domains and latency.

Why it matters. Offline success and online failure often split here.

Common misconception. Connected icon proves *this* feature’s API.

Failure hint. Other local features work; one remote feature fails—or the reverse.

14.5.7 Accessibility API / inclusive status

What it is. Platform and web contracts that expose name, role, state, and updates to assistive technologies—not optional decoration (CLM-CH14-004) (W3C 2023, 2024).

Why it matters. Busy/ready must be communicable beyond color alone.

Common misconception. Accessibility is a polish pass after “real” engineering.

Failure hint. Spinner only; no text status; keyboard path cannot reach the control.

14.6 6. Stability contract

The **Stability Contract** is a signature idea across this book:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

Chapter 14 needs the application-stack lens, not invented numeric budgets. For the chrome-before-content / API-waiting anchor, concurrent conditions typically include:

- input events reach handlers (touch, keyboard, switch, voice),
- runtime scheduled; UI thread not hung,
- required local APIs available and authorized,
- required remote APIs available, authorized, and version-compatible *when* the feature needs them,
- render and AT feedback reach the human,
- power/thermal headroom sufficient that needed work is not silently deferred beyond human patience (qualitative only).

Three separations matter:

1. The interface can look **alive** after the usable result has already failed—or before it is ready.
2. A connectivity indicator can look healthy while *this* application API is unreachable (**illustrative** operator lesson, not a measured RF law).
3. Local API dependencies are not optional for on-device features; network API dependencies are optional branches for some features and mandatory for others—name which kind you are diagnosing.

When the contract breaks, people lose time, trust, and sometimes access to school, work, or care tasks. Mis-naming the failure domain—“the phone is broken” versus “the service returned 503” versus “permission denied”—leads to wrong fixes and unfair blame. Quartet-specific UI latency numbers stay **PHYSICAL_PENDING** until measured evidence exists (CLM-CH14-005).

14.7 7. Try it

Prefer inheriting existing CE labs. Do not invent CE/WAIKE lab IDs. The chapter’s observation craft comes from three real publication labs with different jobs.

14.7.1 LAB-SYS-001 — Name the system behind a familiar “open” (CE-1 adjacency)

Observable question. When chrome appears before a feature finishes, which cooperating parts—**UI**, **runtime**, **libraries**, and **APIs**—might still be working, and how do I avoid collapsing them into one vague “the app”?

Why this lab here. CE-1 / Chapter 1 adjacency: explode “the app” into cooperating parts and teach chrome versus content without claiming Quartet timings. Full procedure lives under labs/LAB-SYS-001/; do not duplicate the entire write-up in this chapter.

Evidence habit. Readiness observation table, labeled ecosystem map, observation-versus-inference paragraph, scrubbed notes.

14.7.2 LAB-TAP-001 — Trace one interaction (CH02 method adjacency)

Observable question. How much of a tap-to-response path can I directly observe on a device I already own?

Why this lab here. Chapter 2’s canonical method: follow one interaction with timestamps and honest limits. Use it when CH14 asks you to watch a *feature* after chrome is already up—handler timing, local work, optional remote wait—without rewriting Chapter 2. Full procedure lives under labs/LAB-TAP-001/.

Safety. Do not capture passwords, tokens, private chats, or classmate personal information. Prefer fixtures and public demos.

14.7.3 LAB-PKT-001 — When the API path leaves the device (CE-4 adjacency)

Observable question. When a connected action leaves the device, what path and access segments can I name—and what remains inference?

Why this lab here. When CH14’s optional remote API branch is taken, Part IV-style path literacy matters. **LAB-PKT-001** is the publication-owned lab for that adjacency; full procedure lives under labs/LAB-PKT-001/. Keep Wi-Fi / cellular / Internet / cloud vocabulary distinct—do not collapse them into synonyms.

14.7.4 Shared interpretation rule

Allowed as observation	Inference until more evidence
Chrome appeared at time T1	“Network is bad”
Content usable/failed at T2	“Runtime is hung”
API status code / error text (if visible)	“Server is down forever”
Permission dialog shown/denied	“Storage is dying”

Learner timings on commodity devices or fixtures are **your** observations—not publication benchmarks and not Quartet EVT results.

14.7.5 Optional operator extension (not a new lab ID)

On a non-sensitive public page or fixture HAR, browser DevTools Network/Performance panels can name phases without inventing a new CE ID (MDN Web Docs 2026b). Redact URLs that contain secrets. Proposed future idea LAB-API-OBS-001 remains namespaced opportunity only—do not treat it as implemented.

14.8 8. Build it

Use the same app story at the depth that matches your pathway. Your artifact is an **application-stack map** plus an API failure-domain note for one familiar feature.

14.8.1 Explorer

Draw or list two columns: visible UI cues versus guessed hidden parts (3 each: runtime, library, local API, optional remote API). Write one short teach-back paragraph explaining why chrome-ready is not work-complete—without drowning a nontechnical listener in jargon.

14.8.2 Operator

Repeat one feature once online and once offline/airplane (if safe), or use a lab fixture’s offline toggle. Fill an observation table. Assign a failure-domain guess (UI, runtime, local API, remote API) and mark it as inference. Note connectivity-icon state as a separate observation from usability. Inherit craft from **LAB-SYS-001** and **LAB-TAP-001**.

14.8.3 Builder

Map one feature to the APIs it likely calls: at least one local OS/web API and, if applicable, one remote endpoint *name or category* (not a secret URL). Document one tradeoff (example: richer live data versus offline usability). Prefer fixture HAR over production accounts.

14.8.4 Engineer

Reason about API versioning as a Stability Contract issue: what breaks for users when a contract changes incompatibly (Preston-Werner, n.d.; Saltzer and Kaashoek 2009). Write a short diagnosis plan that separates observation, interpretation, and causal claim. Optionally use Performance/Network panels only on a non-sensitive page; scrub secrets (MDN Web Docs 2026b).

14.8.5 Researcher

State a testable hypothesis such as: “For feature X, offline mode causes content failure while chrome still appears.” Define a small number of runs, environment notes, and explicit limits. Document what evidence would be required to upgrade an illustrative claim to a measured claim—Quartet UI latency remains **PHYSICAL_PENDING** until then. Do not publish student timings as universal product laws.

14.8.6 Educator

Facilitate a ten-to-fifteen-minute misconception probe: “If the spinner is spinning, the app must be downloading.” Adapt **LAB-SYS-001** with fixture fallback when networks or personal apps are unavailable. Point advanced students to **LAB-TAP-001** for method and **LAB-PKT-001** when the story leaves the device.

14.9 9. Secure and include it

14.9.1 Security

Chapter 14 teaches structure, not offensive technique. Features can involve sessions, tokens, and privileged APIs. Treat **identity/session** as a first-class failure domain: an expired login can look like a “broken app.” API keys and secrets never belong in screenshots or portfolio HARs. Prefer fixtures and public demos over production accounts with sensitive data.

14.9.2 Privacy

Screenshots, HAR files, and logs are evidence and risk. Do not save passwords, tokens, private message bodies, or classmates’ identifying details. Classroom mode should default to fixture content. Redact before sharing portfolio packets.

14.9.3 Accessibility

Accessibility APIs are part of the real application stack, not optional decoration (CLM-CH14-004). “Use the app” includes keyboard, switch, voice, and assistive pointer paths as first-class actions. Busy versus ready should be communicable beyond color alone. This book cites WCAG 2.2 via the dated Recommendation keys @wcag22-20231005 (5 October 2023) and @wcag22-20241212 (12 December 2024)—not the undated /TR/WCAG22/ latest shortcut as a sole canonical URL (W3C 2023, 2024). Labs accept audio notes and structured text tables instead of screenshots when needed.

14.9.4 Equity

Do not assume a high-end phone, unlimited data, or Device Quartet hardware. Offline/fixture fallback remains mandatory for inherited labs. “Bars look fine but the API fails” is also an equity story: shared networks, captive portals, throttled plans. Designing only for ideal always-online office conditions silently excludes many learners and workers.

14.9.5 Safety and ethics

No packet capture of others’ traffic. Do not disable required safety features for your context. Observation versus inference columns are ethical hygiene, not bureaucracy: overclaiming root cause can mis-blame people and platforms.

14.10 10. Career lens

One ordinary feature crosses many ownership domains. No table promises employment; roles vary by organization. Completing Chapter 14 or its inherited labs does not qualify anyone for a job. Your artifacts resemble early professional evidence in miniature.

Layer / concern	Example role family	Lab resemblance
Perceived readiness / chrome vs content	UX / product design (ROLE-UX)	Annotated chrome-vs-usable observations (LAB-SYS-001)
UI event instrumentation	Frontend engineering (ROLE-FRONTEND)	DevTools / tap timing craft (LAB-TAP-001)
Feature handlers and local APIs	Application development (ROLE-APP)	Feature→API map
Remote contracts and versioning	Backend / cloud (ROLE-BACKEND)	Failure-domain notes; SemVer literacy
Inclusive status / AT	Accessibility / HCI (ROLE-HCI)	Non-color busy/ready cues; WCAG-dated cites
Path when the call leaves the device	Network / connectivity operations	LAB-PKT-001 path vocabulary

Layer / concern	Example role family	Lab resemblance
Facilitation	Technical educator / mentorship	Probe questions for “spinner = downloading” misconceptions

When Device Quartet form factors appear in later chapters, they remain research/learning spines—not mascots and not fabricated shipping SKUs (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Commodity devices remain first-class citizens for Chapter 14 evidence.

14.11 11. Check understanding

Answer with reasoning. Prefer short paragraphs over one-word guesses.

1. Why might an app’s chrome appear quickly while the usable list stays empty?
2. Name at least four cooperating parts inside everyday language’s word “app.”
3. How can a local API failure look different from a remote API failure while the UI chrome still looks healthy?
4. Why is a connectivity indicator insufficient proof that a specific application API is healthy?
5. What does it mean to say an API is a *contract*, and how can versioning affect users’ Stability Contract?
6. Why are accessibility APIs part of the application stack rather than optional decoration?
7. Which inherited lab would you reach for to (a) map chrome vs content, (b) time one interaction, (c) follow a path that leaves the device?
8. **Teach-back.** Explain to a family member why chrome-ready is not work-complete **without** using the words *runtime*, *API*, or *event loop*. Then introduce those three terms one at a time, tying each to something already understood.

Educator note: successful teach-backs show UI versus hidden cooperating parts and at least one alternate path (local-only versus remote API), not memorized vocabulary lists. Gate 3 chapter-prototype reader evidence for Chapter 2 remains historical process; this chapter does not claim completed full-manuscript human validation and does not modify `publication/gates/gate-3/`.

14.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the working bibliography. Project-specific repository evidence is cited with keys such as “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026) and remains distinct from external literature.

Inline citations used in this chapter include Saltzer and Kaashoek (2009), WHATWG (2026b), WHATWG (2026a), Preston-Werner (n.d.), MDN Web Docs (2026b), W3C (2023), W3C (2024), and “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026).

Claim plan keys for CH14: CLM-CH14-001 through CLM-CH14-005 (`publication/full131/chapters/c`

14.13 12. Glossary links

Terms introduced or relied on as formal vocabulary in this chapter should resolve in the living glossary registry as entries mature. Candidates below are for linking—not a dump of free-standing encyclopedia definitions.

Term	Role in this chapter
Application	User-facing program assembling UI + logic + dependencies
User interface	Presentation and interaction surface (visual, voice, AT)
Runtime	Language/platform environment executing app code
Event loop	Dispatcher of UI/input/timer events to handlers
API	Contract for calling into a library, OS service, or remote service
Library / framework	Reusable code that shapes how apps are built
Local vs remote API	Same call metaphor; different failure domains and latency
Chrome vs content	Shell visible versus body usable
Accessibility API	Inclusive status and AT exposure as first-class stack
Failure domain	Bucket for where trouble may live
Stability Contract	Experience depends on hidden conditions staying acceptable
Semantic versioning	Compatibility vocabulary for contract changes
Device Quartet	Research form-factor learning laboratory (PHYSICAL_PENDING)

Deeper entries, analogies labeled as analogies, and “not the same as” warnings live in the glossary network. Prefer following a link when stuck over inventing a private definition.

14.14 Figure references (embedded; registered SVG + ally)

All figures below are **conceptual** or **illustrative** unless a future revision cites a specific validated hardware release. Source: editable SVG in the publication repository (**figures/full131/**). Production status: **embedded**.

14.14.1 FIG-CH14-001 — UI → logic → runtime → libraries → OS → optional remote API

- **Provisional ID:** FIG-CH14-001
- **Figure type:** exploded_diagram
- **Truth classification:** conceptual

- **Pedagogical purpose:** UI to app logic to runtime to libraries to OS to optional remote API
- **Alt / reading order:** Labeled stack layers from human-facing UI down through runtime and libraries to OS services, with a dashed optional remote API branch
- **Color-independent encoding:** Labels + solid vs dashed stroke for optional remote branch
- **Edition scope:** full_edition

14.14.2 FIG-CH14-002 — Event → handler → API → state → render

- **Provisional ID:** FIG-CH14-002
- **Figure type:** sequence_diagram
- **Truth classification:** conceptual
- **Pedagogical purpose:** Event to handler to API to state to render
- **Alt / reading order:** Numbered steps Event, Handler, API call, State update, Render/AT feedback; steps may overlap
- **Color-independent encoding:** Numbers + arrows
- **Edition scope:** full_edition

14.14.3 FIG-CH14-003 — Local API vs remote API failure domains

- **Provisional ID:** FIG-CH14-003
- **Figure type:** comparative_layers
- **Truth classification:** illustrative
- **Pedagogical purpose:** Local API vs remote API failure domains
- **Alt / reading order:** Two labeled columns (Local API, Remote API) with distinct failure examples; join at human-usable or failed
- **Color-independent encoding:** Column labels + text status words (not color alone)
- **Edition scope:** full_edition

14.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH14-001** — App means UI + runtime + libraries + OS services + optional remote APIs (SOURCE_IDENTIFIED).
- **CLM-CH14-002** — APIs are contracts; versioning affects Stability Contracts (SOURCE_IDENTIFIED).
- **CLM-CH14-003** — UI chrome can render before application work or remote fetches complete (SOURCE_IDENTIFIED).
- **CLM-CH14-004** — Accessibility APIs are part of the real stack (SOURCE_IDENTIFIED; dual dated WCAG keys).
- **CLM-CH14-005** — Quartet UI latency budgets (PHYSICAL_PENDING).

Chapter 15

Chapter 15 — Containers, Virtualization, Cloud, and Edge Computing

Status: draft · **Chapter ID:** CH15

Author: Edmund Gunn, Jr.

Inheritance: CE-4 edge/cloud *placement* + access-network disambiguation; CE-3 local resource multiplexing

Gate posture: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS)

15.1 1. The moment

A classroom lab says: *run this in a container*. Or your school Chromebook opens the same assignment as a website that “just works,” while a local install never appears. Or a sync spinner sits on “waiting for network” even though the Wi-Fi or cellular icon still looks healthy.

From the seat it feels like computers in the sky—or like the radio failed. Underneath, two different stories are mixed together. **Virtual machines** and **containers** multiplex hardware so many workloads can share one machine with different isolation boundaries (Tanenbaum and Bos 2022; Open Container Initiative 2026). Separately, services are **placed** near you (**edge**) or farther away in shared datacenter pools (**cloud**) (Mell and Grance 2011; Iorga et al. 2018). And separately again, your **access network** may be Wi-Fi or cellular—on-ramps that are *not* synonyms for cloud, edge, or “the Internet” (IEEE 2020; 3GPP 2026).

This chapter inherits Concept Edition CE-4’s hard rule: **placement** **access radio**. Part IV will deepen packets and radios; here the governing question is:

When something runs “in a container” or “in the cloud,” what is being shared, where is the work placed—and why is that not the same question as “Am I on Wi-Fi or cellular?”

15.2 2. What you notice

Before names like *hypervisor* or *orchestration* enter, notice the human contract that broke or held.

You expected the lab image to start, the browser app to feel local, or a sync to finish. Instead you notice a pull that takes forever, a permission prompt you do not understand, a green connectivity icon with a stalled spinner, or a Chromebook workflow that never needed an install. Sometimes the failure is “cannot reach service.” Sometimes it is “machine is busy.” Sometimes it is “wrong image version.” Those feel different once you stop treating “online” as one word.

A usable remote or packaged experience is a human perception produced by isolation boundaries, scheduled capacity, a reachable path when placement is remote, and configuration that matches what the app expects—not by radio bars alone.

Notice the split timelines. A container can start while the remote API it needs is unreachable. Cloud placement can be healthy while your café Wi-Fi is captive-portal blocked. Cellular can be fine while a distant region is overloaded. Outside observation (icon lit, page opened, spinner spinning) is evidence of *symptoms*, not automatic proof of *cause*. Label guesses as inference.

Optional commodity comparison (no paid cloud account required): open one familiar task three ways if available—(1) a local app you already have, (2) the same task via a browser URL, (3) a documented container/lab image if your classroom provides one. Record only what you can see: install vs URL vs container wording, access mode (Wi-Fi / cellular / unknown / fixture), and whether work felt nearby or far. Do not invent latency milliseconds.

15.3 3. Exploded ecosystem

A packaged or remote experience is not a single object. It is a path through an ecosystem. **FIG-CH15-001** compares two common multiplexing stories: hardware → hypervisor → virtual machines versus shared kernel → containers. **FIG-CH15-002** separates the **access network** from **edge vs cloud placement**. Treat both as **Representative educational architecture**, not a claim that any sealed phone or school Chromebook looks exactly like the diagram inside.

Walk the layers in ordinary language.

15.3.1 Human

You form intent: finish a lab, sync a document, open a class site. Hands and eyes judge whether the experience started, stalled, or recovered.

15.3.2 Local device and OS

Your laptop, phone, or Chromebook still has CPU, memory, storage, and an OS (Chapter 6–7 / CE-3 adjacency). Local lag can look like “network” when it is really resource pressure—especially if many containers or VMs compete on one host (optional stretch toward LAB-CMS-001).

15.3.3 Isolation / packaging layer

A **hypervisor** hosts **virtual machines** that present machine-like interfaces. A **container** typically packages a process group that shares a host kernel with namespace and resource controls

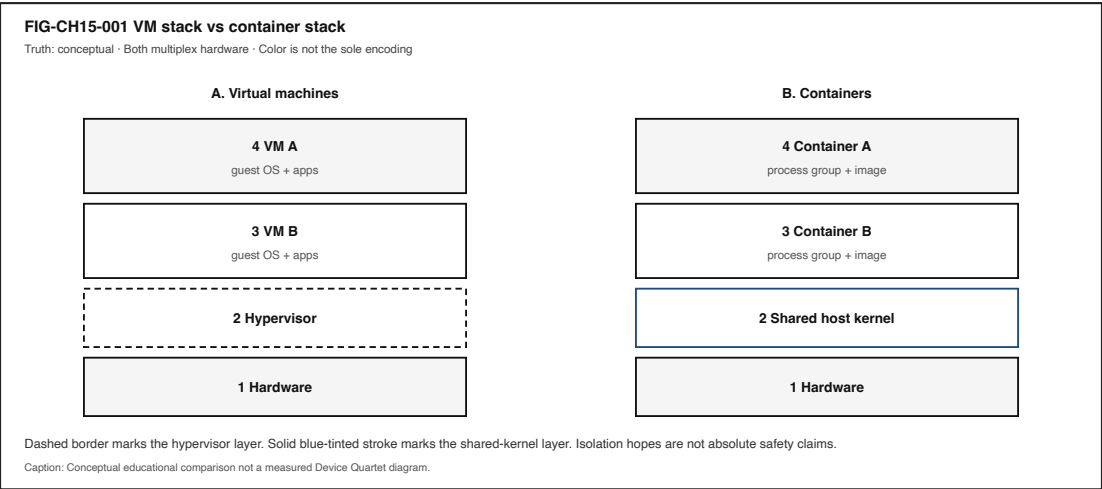


Figure 15.1: Comparative stacks: hardware to hypervisor and VMs versus hardware to shared kernel and containers.

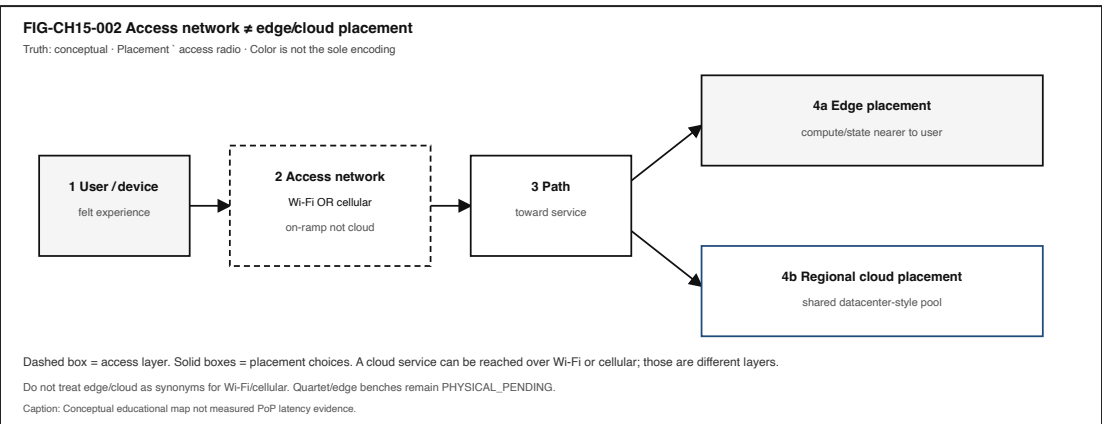


Figure 15.2: User and device to access network (Wi-Fi or cellular) to path, then edge or regional cloud placement.

(Tanenbaum and Bos 2022; Open Container Initiative 2026). An **image** is the filesystem-plus-metadata package used to start that work. Isolation reduces—but does not magically erase—shared-hardware risk (CLM-CH15-003).

15.3.4 Access network (not placement)

Wi-Fi is a wireless LAN access technology family (IEEE 2020). **Cellular** attachment follows operator architectures such as the 5G System family (3GPP 2026). Either can be an on-ramp toward a remote service. Neither *is* the cloud. Neither *is* the edge. A service can sit in a cloud region while your phone uses Wi-Fi—or cellular—to reach it (CLM-CH15-004).

15.3.5 Path / Internet (survey)

Packets still travel paths CE-4 and Chapter 16 deepen. Here, keep only the literacy you need: remote placement implies a reachable path, naming, and transport progress—or an honest offline/fixture story.

15.3.6 Edge placement

Edge means compute or state closer to users or devices than a distant regional pool—fog-adjacent framing in NIST’s conceptual model—not “Wi-Fi itself” and not a brand synonym for “fast Internet” (Iorga et al. 2018).

15.3.7 Cloud placement

Cloud means on-demand network access to a shared pool of configurable computing resources, with essential characteristics and service/deployment models as NIST defines them—not “any website” and not “the radio icon” (Mell and Grance 2011).

15.3.8 Orchestration (intro only)

Orchestration names automation that places and heals many containerized services. Survey depth only: enough to recognize the word; not a Kubernetes certification chapter.

Engineer depth (one distinction): the **control plane** decides and coordinates desired placement/state (schedulers, desired-state controllers, heal policies), while the **data plane** carries and executes the actual workload/data path (running containers, packet forwarding to a service, storage I/O). Failures in the two domains can present differently—for example, a healthy data-plane container with a broken control-plane reconcile loop, or a healthy control-plane “desired state” while the data path is unreachable.

15.3.9 Multitenancy

Many customers may share underlying hardware with isolation hopes. That hope is a design goal and a risk surface—not a promise of absolute safety.

Device Quartet / Edge IO placement benches remain **PHYSICAL_PENDING** (CLM-CH15-005). Research form factors may appear as analogies only; no shipping-SKU language and no invented PoP latency tables (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

15.4 4. Follow the signal

FIG-CH15-003 shows the same human task packaged three ways: local install, container/image, and cloud URL. Read it as a logical story, not as proof that every vendor product uses identical steps.

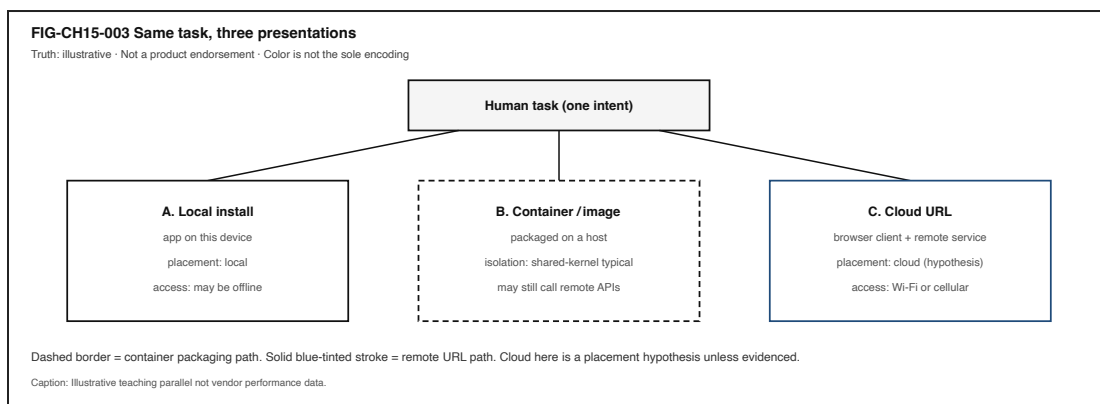


Figure 15.3: Same human task as local install, container/image, or cloud URL.

1. **Intent.** A person asks to run, open, or sync something.
2. **Local packaging decision.** Is the work a native install, a container/image start, a VM, or a browser client talking to a remote service?
3. **Isolation boundary.** VM-style machine isolation or container-style shared-kernel isolation (qualitative contrast) (Tanenbaum and Bos 2022; Open Container Initiative 2026).
4. **Capacity scheduling.** Host or cluster must have enough CPU/memory/storage for the start to succeed.
5. **Access attachment (if remote).** Wi-Fi association or cellular attachment—or a fixture path when live networks are unavailable (IEEE 2020; 3GPP 2026).
6. **Path toward placement.** Packets must make progress toward the edge or cloud dependency when the experience needs one.
7. **Placement executes.** Edge-near or cloud-far work runs (or times out). Placement is *where* computation/state live relative to the user (Mell and Grance 2011; Iorga et al. 2018).
8. **Response and human feedback.** UI success, stall, or error closes the loop—or fails to.

15.4.1 Alternate paths (the honesty rule)

Not every app is containerized. Not every remote URL is “cloud” in the NIST sense. Not every green icon means Internet scope. Not every edge story is measurable from a classroom seat—**observation vs inference** remains mandatory.

15.4.2 Failure domains without drama

Prefer failure *domains* over confident blame:

- Local install / image / config mismatch
- Host resource pressure (CPU/memory/storage)
- Access network (Wi-Fi vs cellular vs captive portal)—still not “the cloud broke”
- Path / naming / transport
- Remote placement unhealthy or wrong region
- Human/task expectation (“opened” “fully synced”)

Outside observation rarely distinguishes those cleanly. That limitation is literacy.

15.5 5. Component cards

For each object: plain language, analogy, technical function, constraints, common symptoms. Analogies are labeled as analogies.

15.5.1 Virtual machine (VM)

- **Plain language.** A computer-like instance hosted on shared hardware.
- **Analogy (labeled).** Like renting a whole apartment inside a building—more walls, more overhead.
- **Technical function.** Presents a machine interface via hypervisor-mediated isolation (Tanenbaum and Bos 2022).
- **Constraints.** Heavier than typical containers; still shares underlying hardware risk.
- **Symptoms.** Slow boot of a full guest; host thrash when too many VMs compete.

15.5.2 Hypervisor

- **Plain language.** Software that hosts virtual machines.
- **Analogy (labeled).** Like a building manager allocating apartments—not the tenants’ furniture.
- **Technical function.** Multiplexes hardware and mediates guest machine interfaces (Tanenbaum and Bos 2022).
- **Constraints.** Configuration and capacity limits; misconfig is a failure domain.
- **Symptoms.** Guests will not start; host reports resource exhaustion.

15.5.3 Container

- **Plain language.** Packaged process group that typically shares the host kernel with isolation controls.
- **Analogy (labeled).** Like a standardized shipping crate on a shared ship—lighter walls than a whole apartment, still not “alone on the ocean.”
- **Technical function.** Runtime configuration and lifecycle as described by container runtime practice/specs such as OCI (Open Container Initiative 2026).
- **Constraints.** Shared-kernel trust assumptions; image supply-chain and privilege boundaries matter.
- **Symptoms.** Image pull failures; permission denials; “works on my machine” until the host differs.

15.5.4 Image / packaging

- **Plain language.** Filesystem-plus-metadata used to start containers (or related packaged workloads).
- **Analogy (labeled).** Like a recipe card plus ingredients list—not the meal already eaten.
- **Technical function.** Makes starts reproducible across hosts when tags/digests match expectations (Open Container Initiative 2026).
- **Constraints.** Version drift; large pulls on metered links; unsigned or untrusted images.
- **Symptoms.** Wrong API behavior after an unnoticed tag move; pull stalls that feel like “Wi-Fi died.”

15.5.5 Cloud (placement)

- **Plain language.** Remote shared datacenter-style resources accessed over a network—**placement**, not radio.
- **Analogy (labeled).** Like a regional workshop you send work to—not the road you drive to get there.
- **Technical function.** On-demand access to a shared pool with NIST essential characteristics and models (Mell and Grance 2011).
- **Constraints.** Path dependency; tenancy; cost; region choice; provider policy.
- **Symptoms.** Reachable Wi-Fi/cellular with unhealthy remote service—or the reverse.

15.5.6 Edge (placement)

- **Plain language.** Compute/state closer to users or devices than a distant region—**still not Wi-Fi itself**.
- **Analogy (labeled).** Like a neighborhood workshop instead of a far warehouse—still a *place to run work*, not the sidewalk.
- **Technical function.** Distributes compute/storage/networking nearer to users/things in fog/edge-adjacent framing (Iorga et al. 2018).
- **Constraints.** Capacity, physical presence, continuity when the device moves; many consumer apps hide true placement.
- **Symptoms.** Feels snappy nearby until the dependency still phones home to a far region (inference unless evidenced).

15.5.7 Multitenancy

- **Plain language.** Multiple customers sharing underlying hardware with isolation hopes.
- **Analogy (labeled).** Like many tenants in one building—walls help; they are not magic force fields.
- **Technical function.** Economic sharing plus isolation mechanisms (VM and/or container layers) (Tanenbaum and Bos 2022).
- **Constraints.** Isolation is incomplete in principle; wording must avoid absolute safety claims (CLM-CH15-003).
- **Symptoms.** Noisy-neighbor performance; policy limits; incident blur across tenants (usually invisible to end users).

15.5.8 Orchestration (intro)

- **Plain language.** Automation that places and heals many services.
 - **Analogy (labeled).** Like a dispatcher assigning trucks—not the cargo inside one crate.
 - **Technical function.** Declares desired state for many containerized workloads (survey depth only).
 - **Control plane vs data plane.** Control plane = decide/coordinate desired placement/state; data plane = carry/execute the workload/data path. Their failure modes can diverge.
 - **Constraints.** Control-plane dependency; mis-scheduled capacity still fails the human experience.
 - **Symptoms.** “Service restarted” loops; region failover that still feels down until DNS/path catch up.
-

15.6 6. Stability contract

Definition (book-wide): a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

Chapter lens: a containerized or remote experience continues only while isolation boundaries hold enough for the task, scheduled capacity is available on the host or cloud, the network path to placement is reachable when remote, and the image/config version matches the expected API—and while the *access* story (Wi-Fi, cellular, or fixture) is not confused with the *placement* story (edge vs cloud).

A system can remain technically “connected” (radio icon lit; container “Up”) while the human experience has already failed.

15.6.1 Concurrent conditions (qualitative; no invented numeric budgets)

Condition	Why it matters
Isolation boundary adequate	Shared hosts must not silently break the task’s trust/assumptions
Capacity available	CPU/memory/storage/scheduling headroom to start and run
Image/config match	Wrong digest/tag → wrong API behavior that looks like “network”
Access attachment meaningful	Wi-Fi or cellular must exist <i>for the needed path</i> —or use fixtures
Path to placement reachable	Remote edge/cloud dependencies need progress, not only association
Placement healthy enough	Edge/cloud service must accept and complete the work
Continuity across changes	Switching Wi-Fi cellular must not be misread as “cloud moved”

15.6.2 Failure domains

1. **On-device / host** — local CPU/memory pressure; container engine stuck; wrong runtime.
2. **Packaging** — image pull, tag drift, missing dependencies.
3. **Access network** — Wi-Fi congestion/captive portal; cellular policy/metering.
4. **Path** — DNS, routing, filtering (Chapter 16 depth).
5. **Placement / service** — edge/cloud outage, overload, region mismatch.
6. **Human/task** — assuming “container started” means “assignment submitted.”

15.6.3 Measurements we can seek (when available)

- Browser Resource Timing phases for URL-based tasks (LAB-PKT-001 Route A)
- OS text labels for Wi-Fi vs cellular (never color alone)
- Fixture path parse proving observation vs inference (LAB-PKT-001 Route B)
- Qualitative host resource notes if many local containers/VMs (LAB-CMS-001 stretch only)

15.6.4 Measurements we cannot invent

- Authoritative edge PoP geolocation for arbitrary consumer apps
 - Quartet/Edge IO placement bench numbers (**PHYSICAL_PENDING**, CLM-CH15-005)
 - Universal millisecond SLOs for classroom sync
 - Absolute “containers are secure” claims
-

15.7 7. Try it

Primary lab: **LAB-PKT-001** — Trace one connected action across path and access, with an explicit **placement hypothesis** (local / edge / cloud / unknown) kept separate from **access mode** (Wi-Fi / cellular / fixture).

15.7.1 Baseline (all pathways)

1. Predict whether a stall would look like latency, reliability, or throughput—and whether you expect the failure domain to be access, path, or placement.
2. Run Route A (browser/CLI) **or** Route B (fixture)—Route B is mandatory-accessible when live networks are unsafe or unavailable.
3. Fill observation vs inference columns. Do not capture secrets, tokens, or other users’ traffic.
4. On your path diagram, label **access** and **placement** as different layers.

15.7.2 Pathway notes

Pathway	LAB-PKT-001 emphasis
Explorer	Teach-back sentence: cloud Wi-Fi; container VM.
Operator	Name whether symptoms look local packaging vs remote placement.
Builder	Produce the labeled path + placement diagram artifact.

Pathway	LAB-PKT-001 emphasis
Engineer	Separate DNS / connect / waiting / transfer when timing is visible.
Researcher	List what evidence would confirm an edge vs cloud claim you are <i>not</i> allowed to assert from icons alone.
Educator	Prefer fixture route for mixed classrooms; require text status labels.

Optional stretch only: **LAB-CMS-001** if many local containers/VMs make the host feel slow—keep that as *local resource pressure*, not as proof the cloud failed.

Placement literacy is an **INLINE_ACTIVITY** folded into LAB-PKT-001 above (access vs placement columns). The namespaced idea LAB-PLACE-001 is **not** a shipped **labs/** package—do not treat it as a **FULL_LAB** ID.

15.8 8. Build it

Extend LAB-PKT-001 without turning Part III into a cloud certification course.

15.8.1 Explorer

Build a pocket card with four sentences: VM, container, cloud (placement), edge (placement). Each sentence must refuse one synonym trap (especially cloud Wi-Fi).

15.8.2 Operator

Build a “green icon, bad experience” checklist that starts with access mode text, then path symptoms, then placement hypothesis, and ends with “needs more evidence.”

15.8.3 Builder

Build a labeled diagram: human → device → access network → path → edge **or** cloud placement for one familiar task. Mark observed vs inferred nodes.

15.8.4 Engineer

Build a one-page qualitative isolation comparison (VM vs container): boundary layer, typical sharing, failure modes. Cite textbook + runtime-spec survey depth—no exploit steps (Tanenbaum and Bos 2022; Open Container Initiative 2026).

15.8.5 Researcher

Build an evidence plan for a claim you cannot make yet—for example, “this app’s sync runs at a metro edge PoP.” Specify what measurements would be required, and keep Quartet/project benches **PHYSICAL_PENDING** (CLM-CH15-005).

Educators can facilitate Section 11 teach-backs and keep paid cloud accounts optional forever for the baseline.

15.9 9. Secure and include it

15.9.1 Security

Isolation and multitenancy reduce—but do not eliminate—shared-hardware and shared-infrastructure risk (Tanenbaum and Bos 2022). Teach boundaries and least privilege at concept level. **Out of scope:** container-escape tutorials, exploit development, unauthorized scanning, or bypassing access controls. Prefer official images, understand that “running in a container” is not a security certificate, and treat unknown registries as supply-chain risk surfaces.

15.9.2 Privacy

Screenshots of container logs, cloud consoles, or network panels may reveal emails, hostnames, tokens, or message previews—redact before portfolio save. Placement in cloud/edge implies third-party processing adjacency (forward to CE-5 / later privacy chapters without dumping full law here).

15.9.3 Accessibility

Status UIs that rely on color-only bars or waves exclude readers. LAB-PKT-001 requires text labels for access mode. Provide text-first diagrams for VM vs container and for placement. Fixture Route B keeps learners without personal hotspots inside the evidence path.

15.9.4 Equity

Do not require paid cloud accounts for baseline literacy. Café Wi-Fi, metered cellular, and school-filtered networks are first-class conditions—not edge cases. “Just deploy to the cloud” can exclude learners by cost and policy; local fixtures and public documentation URLs are valid evidence routes.

15.9.5 Safety

No unauthorized radio transmission labs; no social-engineering classmates’ credentials “for practice.”

15.9.6 Ethics

Do not claim Gate 3 PASS, Quartet placement benchmarks, or absolute isolation safety. Overclaiming “the cloud” when you only observed Wi-Fi association is still false evidence.

15.10 10. Career lens

One packaged or remote experience crosses many ownership domains. No table promises employment; roles vary by organization. LAB-PKT-001 artifacts resemble early professional evidence in miniature: labeled diagrams, observation discipline, and explicit uncertainty.

Role lens	Typical artifacts	Review questions
Cloud / backend	Service diagrams, deployment notes	Is placement stated separately from access?
Platform / containers	Image provenance, runtime config	What isolation boundary is assumed?
SRE / reliability	Incident timelines, dependency maps	Access vs path vs placement failure domain?
Network / wireless (adjacency)	Path traces, access-mode labels	Did someone confuse radio with cloud?
Embedded / edge	On-device vs offload decisions	What must stay local for latency/privacy?
Support / operator	Ticket notes with observation tables	Are inferences labeled as inferences?
Security (awareness)	Threat sketches without exploits	Multitenancy residual risk named?
Educator / facilitator	Fixture-first lab plans	Can every learner complete without paid cloud?

Portfolio hint: a scrubbed path+placement diagram with observation vs inference beats a vibes-based “the cloud is down” claim. Completing LAB-PKT-001 does **not** qualify anyone for CCNA or cloud certifications.

WAIKE adjacency (not equivalence): `CLOUD_DEVOPS / lab_cloud_cost / lab_slo_budget` at accepted-main SHA `e97e74fc9bfb44b1cdc26b272dc4848264f15fe0`—adjacent only; no exact CH15 module ID.

15.11 11. Check understanding

Concept. In one sentence each, define *virtual machine*, *container*, and *cloud (placement)* so that none of them swallows the other two—and none equals Wi-Fi.

System tracing. Trace a familiar browser-based task from intent to response. Mark access network and placement as different steps. Label observed vs inferred.

Misconception check. Why can a service be “in the cloud” while the user is on cellular—and why is that not a contradiction?

Misconception check. Why is “containers are always safer than VMs” (or the reverse) an overclaim this chapter refuses?

Teach-it-back. Explain to a newcomer—using only LAB-PKT-001 vocabulary—why “Wi-Fi connected” “Internet usable” “cloud service healthy.”

Researcher prompt. What evidence would convert a PHYSICAL_PENDING Quartet/edge placement claim into a documented physical claim? What remains out of scope for a commodity classroom lab?

15.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (book/references/references.bib). Project-specific Device Quartet / Edge IO placement status remains **PHYSICAL_PENDING** (CLM-CH15-005), separately from external literature.

Inline citations used in this chapter include Tanenbaum and Bos (2022), Open Container Initiative (2026), Mell and Grance (2011), Iorga et al. (2018), IEEE (2020), and 3GPP (2026).

CE inheritance: publication/preproduction/ce-04/ (placement access radio) and CE-3 local multiplexing adjacency. Gate posture remains **GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING**.

15.13 12. Glossary links

Candidate terms introduced or reinforced here (see also chapter glossary candidates; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Virtual machine	Computer-like instance hosted via hypervisor isolation
Hypervisor	Software layer that hosts virtual machines
Container	Packaged process group typically sharing a host kernel with isolation controls
Image	Filesystem-plus-metadata package used to start containerized work
Cloud (placement)	Network-accessible shared datacenter computing resources—not a radio
Edge (placement)	Compute/state closer to users/devices than a distant region—not Wi-Fi itself
Multitenancy	Multiple customers sharing underlying hardware with isolation hopes
Orchestration (intro)	Automation placing/healing many services
Access network	Wi-Fi or cellular on-ramp—distinct from placement
Stability contract	Concurrent conditions that keep the packaged/remote experience alive

Related earlier chapters: local compute multiplexing (CH06 / CE-3), systems boot/trust adjacency (CH11–CH14). Related later chapters: packets/routing depth (CH16), radio/access depth (CH17–CH18), latency/QoE (CH20), network security/privacy (CH23–CH25).

15.14 Figure references (working embeds; accessibility metadata)

All three figures are **conceptual** / **illustrative teaching aids**. No fabricated telemetry. No Quartet EVT measurements.

15.14.1 FIG-CH15-001 — Hardware → hypervisor/VM vs shared kernel → containers

- **Type.** Comparative layers.
- **Reader should notice.** VMs and containers both multiplex hardware but isolate at different layers.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name both stacks in reading order; state conceptual truth class; color not sole cue.

15.14.2 FIG-CH15-002 — User → access network → edge vs regional cloud

- **Type.** System map.
- **Reader should notice.** Access (Wi-Fi/cellular) is a different layer from placement (edge/cloud).
- **Truth class.** Conceptual.
- **Alt text requirement.** Separate access from placement labels; deny synonym collapse.

15.14.3 FIG-CH15-003 — Same app: local vs container vs cloud URL

- **Type.** Illustrative.
 - **Reader should notice.** One human task, three packaging/placement presentations.
 - **Truth class.** Illustrative.
 - **Alt text requirement.** Enumerate three parallel paths; label illustrative; no product endorsement.
-

15.15 Claim footnotes used in this chapter

- **CLM-CH15-001.** VMs and containers multiplex hardware at different typical isolation layers—framed with OS textbook + OCI runtime survey depth (Tanenbaum and Bos 2022; Open Container Initiative 2026).
- **CLM-CH15-002.** Edge vs cloud is placement relative to users/constraints—not synonyms for Wi-Fi or cellular (Mell and Grance 2011; Iorga et al. 2018).
- **CLM-CH15-003.** Isolation reduces shared-hardware risk without absolute safety claims (Tanenbaum and Bos 2022; Open Container Initiative 2026).
- **CLM-CH15-004.** Cloud placement and Wi-Fi/cellular access are different layers (Mell and Grance 2011; IEEE 2020; 3GPP 2026).
- **CLM-CH15-005.** gunnchOS/Quartet edge placement benchmarks remain **PHYSICAL_PENDING** / **PROJECT_EVIDENCE_NEEDED**.

Chapter 16

Chapter 16 — Packets, Protocols, Routing, and the Internet

Status: draft · Chapter ID: CH16

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

Gate posture: GATE_3_IN_PROGRESS – READER_EVIDENCE_PENDING (not Gate 3 PASS)

16.1 1. The moment

You send a short message. The UI shows **sent**. Then nothing: no delivered check, no reply, a spinner that will not end. You refresh a page you trust. The browser stalls on “looking up...” or hangs on connect. From your seat it feels like one verdict: *the Internet is broken*.

Underneath that verdict, several different things may have failed—or may still be fine. Your **device** may still be awake. Your **LAN** may still reach a printer. A **DNS** lookup for one name may have failed while another service answers. A **route** toward a destination may be missing even though Wi-Fi waves still decorate the status bar. The remote **service** may be unhealthy while your access radio looks perfect.

This chapter is Part IV’s packet-path spine. It inherits Concept Edition CE-4’s connectivity story and deepens packets, protocols, routing, and Internet scopes—without collapsing Wi-Fi, cellular, cloud, and “the Internet” into synonyms, and without claiming Gate 3 reader evidence that does not yet exist.

The governing question:

When everyday connected work stalls, what must I keep distinct—device, LAN, Internet, DNS, and service—so I can follow packets under protocols instead of blaming “the Internet”?

16.2 2. What you notice

Before words like *TTL* or *TCP*, notice the human contracts you already enforce with frustration.

You expect “sent” to mean progress toward another person—not only a local queue. You expect a refresh to either show content or fail with a readable reason. You expect bars and waves to mean something useful. You expect toggling Wi-Fi or walking nearer a window to sometimes help, and you treat that superstition as data even when it is not a root cause.

Those expectations are the product from the person’s point of view.

A connected experience is a human perception produced by concurrent conditions: local readiness, access attachment, name resolution when names are required, reachable routes, transport progress, and a healthy-enough remote service.

Notice the split timelines. A messenger can stamp **sent** while packets never leave the LAN. A browser can show a green lock on a previous tab while a new hostname never resolves. Cellular can rescue a café Wi-Fi stall without proving that “cellular is the Internet.” Chat can work while a large upload crawls—latency, reliability, and throughput are different failure families (Kurose and Ross 2020).

Optional notice on a device you already own (safe, commodity only): open a page you are allowed to load, watch whether failure language mentions lookup, connect, or waiting—or use the lab fixture if live networks are unavailable or untrusted. Do not capture other people’s traffic. Do not chase signal by climbing or distracted walking.

16.3 3. Exploded ecosystem

A stalled sync is not one object. It is a path through an ecosystem. **FIG-CH16-001** is the first-minute map: **device (local)**, **LAN**, and **Internet** as nested reachability scopes—not synonyms (CLM-CH16-002).

Walk the layers in ordinary language, then keep the same layers when vocabulary deepens.

16.3.1 Human

You form intent: send, sync, refresh, join a call. Eyes read checkmarks and spinners. Later you judge whether the other person *got it*.

16.3.2 Device (local scope)

Apps, OS network stack, and NIC/radio on *this* machine. Local drafts, local caches, and on-device failures live here. Local success is not Internet success.

16.3.3 Access on-ramp (not “the Internet”)

Wi-Fi is a wireless LAN access technology family (IEEE 2020). **Cellular** is a mobile-operator access architecture family at survey depth (3GPP 2026). Both can be on-ramps toward Internet-scoped work. Neither *is* the Internet (CLM-CH16-005). Icons report attachment moods; they do not certify remote services.

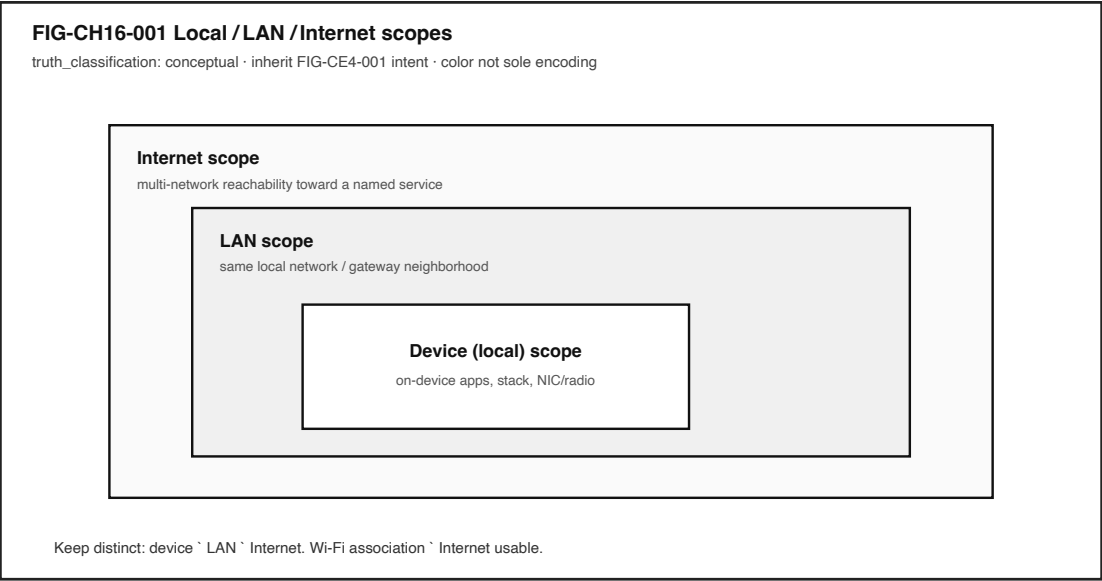


Figure 16.1: Conceptual nested scopes: device, LAN, and Internet.

16.3.4 LAN scope

Neighbors reachable under the same local network / gateway neighborhood—printers, captive portals, private address spaces commonly used on home and campus nets (Rekhter et al. 1996). A **LAN** may include **wired Ethernet** segments and/or wireless LAN attachment; those are local on-ramps and link technologies, not synonyms for Internet scope (Kurose and Ross 2020; IEEE 2020). A device can be “on Wi-Fi” or “on Ethernet” and still fail Internet-scoped work.

16.3.5 DNS (dependency, not the whole path)

The Domain Name System maps human-readable names to addresses (Mockapetris 1987a, 1987b). When a name is required, DNS sits on the critical path. DNS failure can look like “Internet down” while other paths still work (CLM-CH16-004).

16.3.6 Internet path / routing

Across networks, **routing** chooses where next toward a destination. Packets carry addresses; routers and gateways forward (Postel 1981; Kurose and Ross 2020). Internet scope means multi-network reachability—not “any radio is lit.”

16.3.7 Transport and protocols

A **protocol** is an agreed set of rules. **Transport** (survey: TCP, UDP, and modern options such as QUIC) decides how end-to-end conversation reliability is *attempted* over an unreliable datagram substrate (Eddy 2022; Postel 1980; Iyengar and Thomson 2021). Routing decides *where next*; transport decides *how the conversation tries to complete* (CLM-CH16-003).

16.3.8 Remote service (placement)

Edge-near or cloud-far placement is about where compute and state live—not about which radio you used. Service health is distinct from path health. Chapter 15 adjacency covers placement depth; here, keep the label honest: without provider evidence, “the cloud region failed” is inference.

16.3.9 System software preview

Browsers and OS tools can expose request phases useful for classroom observation (MDN Web Docs 2026c, 2026a). Those are software-visible timings—not kernel traces and not RF drive-tests (**PHYSICAL_PENDING** for Quartet/project path measurements; CLM-CH16-006).

16.4 4. Follow the signal

FIG-CH16-002 follows one connected action as encapsulation “sticky notes” across hops. Read it as a logical story, not as a claim that every messenger executes identical steps with measured timings.

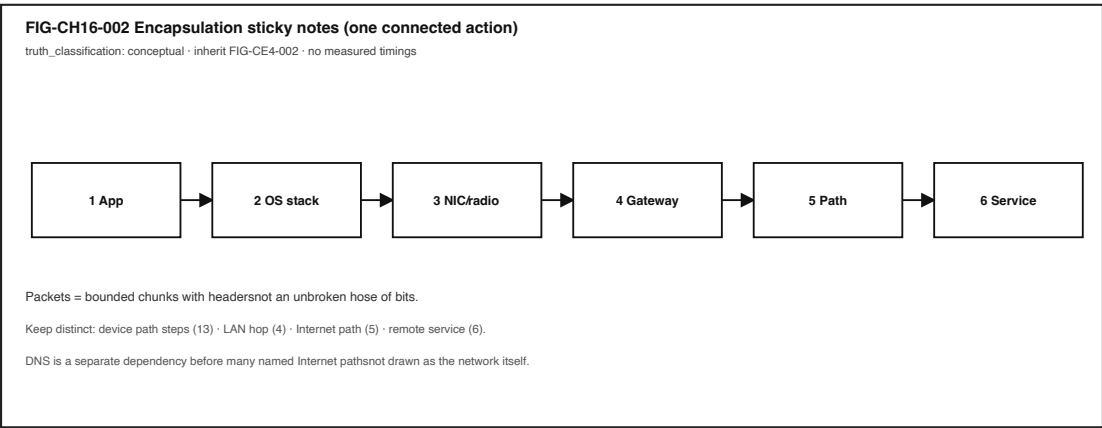


Figure 16.2: Encapsulation sequence from app through gateway and path to service.

1. **Intent.** You tap send or refresh. The UI may optimistically show progress.
2. **Local enqueue.** The app and OS prepare a request on the **device**. Failure here is not “the Internet.”
3. **Name resolution (when a hostname is used).** DNS must yield an address (Mockapetris 1987a, 1987b). **FIG-CH16-003** shows how DNS failure can strand a named path while other work continues.
4. **Packetization / encapsulation.** Data travels as **packets**—bounded chunks with headers—not an unbroken hose of bits (CLM-CH16-001) (Postel 1981). Each layer wraps payload with its headers (encapsulation).
5. **Access + LAN hop.** Frames leave via Wi-Fi or cellular attachment toward a gateway. Access Internet.
6. **Routing across Internet scope.** Each hop chooses a next hop toward the destination address (Postel 1981; Kurose and Ross 2020).

7. **Transport conversation.** TCP (and kin) attempt sequenced, acknowledgment-based recovery over IP’s best-effort delivery (Eddy 2022). UDP offers a minimal message service without that machinery (Postel 1980).
8. **Service response.** A remote service accepts, rejects, or times out. Placement (edge/cloud) is a hypothesis until evidenced.
9. **Human feedback.** Checkmarks, errors, or eternal spinners close the loop—or fail to.

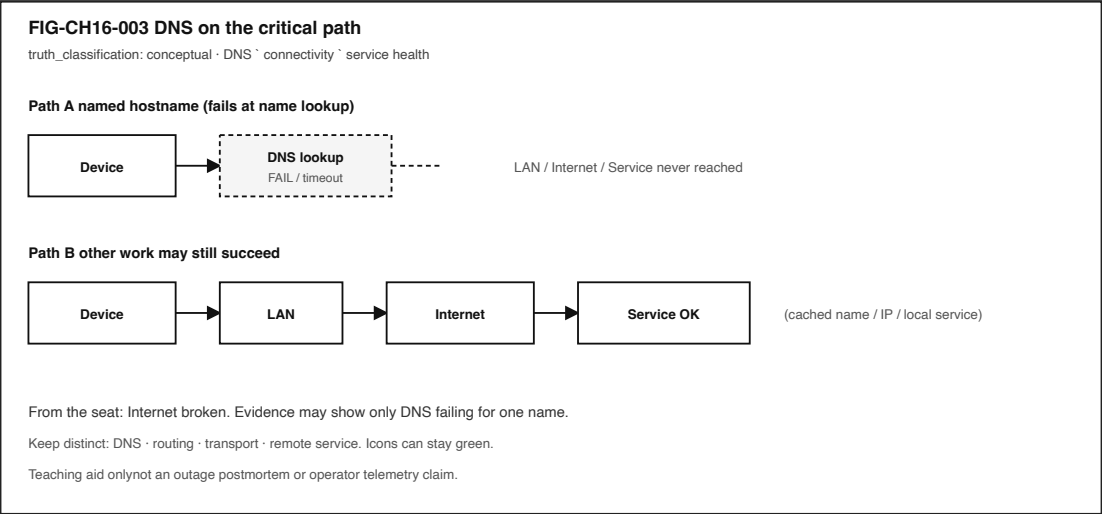


Figure 16.3: DNS failure can look like Internet failure while other paths work.

16.4.1 Failure domains without drama

Prefer labeled domains over confident blame:

Domain	Example observation	Forbidden leap
Device	App frozen; airplane mode on	“ISP is down”
Access	Wi-Fi associated; captive portal page	“DNS is broken” without lookup evidence
DNS	Browser DNS phase fails for one name	“Entire Internet is down”
Routing / reachability	Connect timeouts after DNS OK	“Server deleted my account”
Transport / retries	Repeated reconnects	Tower congestion from bars alone
Service	HTTP 5xx / app error after connect	“My Wi-Fi is stupid”

Outside observation rarely separates these cleanly. That limitation is literacy, not a reader failure.

16.5 5. Component cards

For each object: plain language, analogy, technical function, constraints, common symptoms. Analogies are labeled as analogies.

16.5.1 Packet

- **Plain language.** A bounded chunk of network data with headers that layers use to forward and deliver.
- **Analogy (labeled).** Like a labeled envelope—not a continuous fire hose.
- **Technical function.** Classic IPv4 datagram model includes source/destination addresses, protocol, and TTL among other fields (Postel 1981). IPv6 is a parallel modern network-layer standard (Deering and Hinden 2017).
- **Constraints.** Size limits, loss, reordering, and TTL expiry.
- **Symptoms.** Partial loads, stalls, “connection reset,” or silence after “sent.”

16.5.2 Encapsulation

- **Plain language.** Wrapping payload with layer-specific headers (and sometimes trailers) on the way out; peeling them on the way in.
- **Analogy (labeled).** Sticky notes added at each desk in a mailroom.
- **Technical function.** Lets protocols cooperate without each layer rewriting the whole application message (Kurose and Ross 2020).
- **Constraints.** Overhead, mismatch if a middlebox expects different headers.
- **Symptoms.** Works on one network path and fails on another that handles headers differently.

16.5.3 Addressing

- **Plain language.** Identifiers used to deliver packets (for example, IP addresses).
- **Analogy (labeled).** Street addresses for envelopes—not the road itself.
- **Technical function.** Network-layer delivery targets in IP (Postel 1981; Deering and Hinden 2017). Private IPv4 ranges commonly appear on LANs (Rekhter et al. 1996).
- **Constraints.** Ambiguity behind NAT; wrong address → wrong place.
- **Symptoms.** Reaches the wrong host; never leaves private scope when Internet scope was intended.

16.5.4 Routing

- **Plain language.** Choosing next hops toward a destination across networks.
- **Analogy (labeled).** Intersection decisions—not the conversation once you arrive.
- **Technical function.** Control-plane processes and protocols compute and distribute reachability/path information so forwarding tables stay useful (Postel 1981; Kurose and Ross 2020).
- **Forwarding vs routing.** **Forwarding** is the per-packet/data-plane action that uses an existing forwarding table; **routing** is the control-plane work that builds and updates that reachability information. Everyday speech often says “routing” for both—keep the split when diagnosing.
- **Constraints.** Misconfiguration, filtering, missing default route, policy blocks.
- **Symptoms.** DNS succeeds; connect hangs; LAN peers work; Internet peers do not.

16.5.5 Protocol

- **Plain language.** Shared rules so two parties can communicate meaningfully.
- **Analogy (labeled).** A script both sides agree to follow.
- **Technical function.** Defines formats and behaviors at each layer—from IP to TCP to application protocols (Postel 1981; Eddy 2022).
- **Constraints.** Version mismatch; middleboxes; incomplete implementations.
- **Symptoms.** “Protocol error,” failed negotiation, mysterious hangs after connect.

16.5.6 Transport (intro)

- **Plain language.** End-to-end conversation behaviors above the internetwork layer—reliability attempts, ports, streams vs messages.
- **Analogy (labeled).** How carefully you confirm that spoken sentences arrived—not which highway you drove.
- **Technical function.** TCP provides a connection-oriented byte-stream service with sequenced data and acknowledgment-based recovery (Eddy 2022). UDP provides a minimal message-oriented service (Postel 1980). QUIC is an optional modern survey pointer (Iyengar and Thomson 2021).
- **Constraints.** Cannot invent a lossless physical medium; timeouts still feel like failure.
- **Symptoms.** Retries, resets, one-way audio, “connected” with no progress.

16.5.7 DNS

- **Plain language.** A system mapping names people use to addresses machines need (Mockapetris 1987a, 1987b).
- **Analogy (labeled).** A phone book lookup before dialing—not the phone call itself.
- **Technical function.** Critical-path dependency for many named Internet requests (Kurose and Ross 2020).
- **Constraints.** Stale caches, broken resolvers, filtering, captive portals impersonating “success.”
- **Symptoms.** “Server not found” while IP-literal or cached services still work (CLM-CH16-004).

16.5.8 Local vs LAN vs Internet

- **Plain language.** Distinct reachability scopes (CLM-CH16-002).
- **Analogy (labeled).** Room / building / city—not three words for “online.”
- **Technical function.** Teaching structure for diagnosis; aligned with host/internet layering intuition (Braden 1989).
- **Constraints.** NAT and policy blur edges; still refuse synonym collapse.
- **Symptoms.** Printer works, web fails; guest Wi-Fi associates, Internet blocked.

16.6 6. Stability contract

The **Stability Contract** returns:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For this chapter’s anchor—send/sync/refresh that remains *human-usable*—keep conditions **qualitative**. Do not invent millisecond budgets (forbidden until measured; CE-4/CH16 posture).

Concurrent conditions (chapter lens):

1. **Device local stack** can enqueue and process network work.
2. **Access attachment** exists for the *needed* path (Wi-Fi or cellular as on-ramps—not synonyms for Internet) (IEEE 2020; 3GPP 2026).
3. **Auth / captive gates** cleared when they apply.
4. **DNS succeeds** when names are required (Mockapetris 1987a, 1987b).
5. **Route exists** toward the destination scope required (Postel 1981).
6. **Transport progresses** or fails visibly (Eddy 2022).
7. **Latency / loss** stay acceptable *for the human task* (qualitative).
8. **Remote service** healthy enough to complete the action.

A radio icon can stay positive while the human experience has already failed. Conversely, a page can load from cache while live Internet scope is gone. Stability is concurrent conditions—not a single green glyph.

Honesty bound: Device Quartet / project-specific path measurements remain **PHYSICAL_PENDING** (CLM-CH16-006). **LAB-PKT-001** produces *your* classroom evidence for *your* route—or fixture honesty banners—not a universal SLO.

16.7 7. Try it

16.7.1 LAB-PKT-001 — Trace One Connected Action Across Path and Access

Goal. When a connected action feels stuck, observe what you can on a commodity device—or parse fixtures—and separate **device / LAN / Internet / DNS / service**, plus access mode and latency vs reliability vs throughput.

WAIKE alignment note. WAIKE accepted **main** includes adjacent **COMPUTER_NETWORKING** labs such as **lab_datapath** and **DNS** neighbors. Those are competency adjacencies. They are **not** renamed as publication lab IDs. **LAB-PKT-001** is publication-owned.

Safety (hard stops).

- Do **not** capture other users’ traffic.
- Do **not** run unauthorized scanning or packet capture on networks you do not administer.
- Do **not** store passwords, tokens, private message bodies, or banking amounts.
- Prefer demo pages, public documentation URLs, or fixtures.
- On public/untrusted Wi-Fi classrooms, prefer **Route B**.
- No radio transmission experiments; no captive-portal term bypass.

Routes.

- **Route A — live commodity (browser or CLI).** **IMPLEMENTED_DIGITAL** with **EXTERNAL_DEPENDENCY** when live DNS/Internet is used. Browser Network / Resource Timing phases are allowed classroom observations (MDN Web Docs 2026a, 2026c). Optional CLI: `python3 labs/LAB-PKT-001/cli/path_inspect.py --fixture` and, only if policy allows, `--dns-check example.com`.

- **Route B — fixture fallback (mandatory accessible path).** `FIXTURE_VALIDATED`.
Use `fixtures/sample_path_trace.json`, `sample_timing_table.csv`, and `sample_observation`.
Mark rows as **not** your device measurements.

Explorer baseline (about 45–60 minutes).

1. Predict failure family: latency / reliability / throughput; name access if known (Wi-Fi / cellular / unknown / fixture).
2. Complete Route A **or** Route B.
3. Fill an observation-vs-inference table. Keep **device, LAN, Internet, DNS, and service** in separate columns or labels.
4. Draw a path diagram with access on-ramp and edge/cloud placement *hypothesis* labeled as hypothesis.
5. Teach-back: Wi-Fi Internet cellular cloud.

Operator. Change one condition on Route A, or answer fixture parse questions (TTL, ether-type, scopes) on Route B.

Builder. Clean diagram another student could follow without admin rights.

Engineer. Ordered hypotheses: access → DNS → reachability → transport/retries → remote placement. Separate DNS vs routing vs transport evidence.

Researcher. N 3 runs with confounders listed; no published benchmark claim from classroom n=1.

Evidence to keep. Timing/observation table; path diagram; screenshot **or** fixture note; reflection; teach-back. Bare **PASS** is not evidence.

FIG-CH16-004 shows the phase vocabulary learners record—measured as *classroom learning*, not as a product scoreboard.

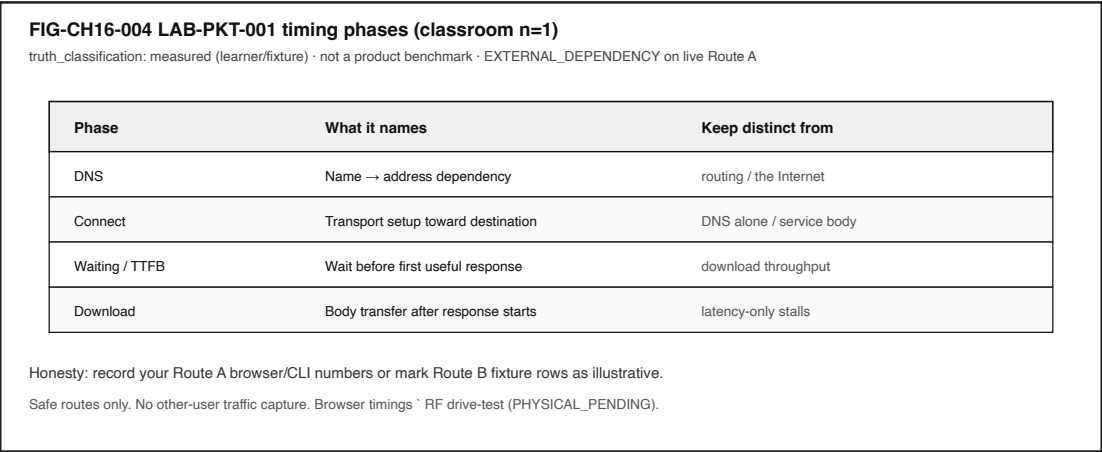


Figure 16.4: Classroom timing phase vocabulary for LAB-PKT-001.

16.8 8. Build it

Extend LAB-PKT-001 without turning Part IV into a certification dump.

16.8.1 Explorer

Build a pocket card with five labels—**device**, **LAN**, **Internet**, **DNS**, **service**—and one plain sentence each that refuses synonym collapse.

16.8.2 Operator

Build a “stuck sync” checklist that starts with access mode text (not color alone), then DNS evidence, then connect/waiting phases, and ends with “needs more evidence”—never with fake certainty about towers or cloud regions.

16.8.3 Builder

Build a labeled path diagram (paper or digital) for one experience: human → device → access on-ramp → LAN gateway → Internet path → service. Annotate which boxes you **observed** vs **inferred**. Optional: parse fixture JSON fields (TTL, ethertype) into diagram callouts.

16.8.4 Engineer

Build a one-page differential: DNS failure vs routing failure vs transport stall vs service error, each with *minimum evidence* required. Cite standards for packet/transport/DNS vocabulary rather than inventing vendor SLOs (Postel 1981; Eddy 2022; Mockapetris 1987a).

16.8.5 Researcher

Build an evidence plan for a claim you are **not** allowed to make yet—for example, “Quartet RF path latency is X ms.” Keep CLM-CH16-006 **PHYSICAL_PENDING**. Specify what drive-test or accepted-main artifacts would be required.

Educators: facilitate teach-backs from Section 11; default classrooms without stable broadband to Route B.

16.9 9. Secure and include it

16.9.1 Security

Open Wi-Fi is an eavesdropping and captive-portal phishing surface at awareness level—not a cracking tutorial. Prefer encrypted sessions in transit at concept depth (TLS adjacency) (Rescorla 2018). “Wi-Fi on” is not a security state. Out of scope for this chapter and lab: exploit development, WPA cracking, unauthorized interception, evil-twin how-tos, or bypassing access controls.

16.9.2 Privacy

Network screenshots may reveal hostnames, emails, locations, or message previews—redact. Timing tables store action labels and phase names, not payloads. Cellular vs Wi-Fi choice can imply location patterns; record access as a category, not a GPS trail.

16.9.3 Accessibility

Status UIs often rely on color icons (bars/waves). Lab writeups require **text** status. Captive portals frequently break screen readers and keyboard flows—document as real exclusion. Provide text-first path diagrams. Fixture Route B is mandatory so learners without personal hotspots can complete evidence.

16.9.4 Equity

Assuming always-on unlimited home broadband excludes many readers. Café/library Wi-Fi and metered cellular are first-class conditions—not edge cases. “Just use cellular” can impose cost. Do not require Device Quartet hardware.

16.9.5 Safety

No climbing for signal; no distracted walking-and-testing; no illegal antenna operation; no social-engineering classmates’ credentials “for the lab.”

16.9.6 Ethics

Connectivity failures are often system and policy issues, not learner moral failures. Distinguishing access technologies prevents false blame when the failure is DNS, routing, or remote service health.

16.10 10. Career lens

One stalled message crosses many ownership domains. No table promises employment; roles vary by organization. Completing LAB-PKT-001 does not grant CCNA or cloud certifications.

Role lens	Typical artifacts	Review questions
Network engineer	Path diagrams, scope labels, reachability notes	LAN vs Internet ownership clear?
ROLE-BACKEND / service owner	API error classes, placement hypotheses	Path vs service failure distinguished?
SRE / reliability	Incident timelines, dependency maps	DNS/routing/transport separated?
Support / operator	Ticket notes, observation-vs-inference	Did we ask for unauthorized captures?
Wireless (survey entry)	Access-mode labels only at CE/CH16 depth	Did we synonymize Wi-Fi with Internet?
Educator / facilitator	Fixture-first rubrics	Can offline learners still pass with evidence?

Portfolio hint: a scrubbed path diagram plus a DNS-vs-routing differential is more honest than “the Internet was down.”

16.11 11. Check understanding

Concept. In one sentence each, define *packet*, *routing*, and *DNS* so that none of them swallows the other two.

Scopes. Give an everyday example where **device**, **LAN**, and **Internet** disagree about “working.”

System tracing. Trace a refresh that fails “looking up host” in numbered steps. Mark observed vs inferred. Where does DNS sit relative to routing and service?

Misconception check. Why is “Wi-Fi connected means the Internet works” incomplete (CLM-CH16-005)?

Misconception check. Why can DNS failure look like total Internet failure while a LAN printer still works (CLM-CH16-004)?

Teach-it-back. Using only LAB-PKT-001 vocabulary, explain to a newcomer why a message can show **sent** while the peer never receives it—without capturing anyone’s traffic.

Researcher prompt. What evidence would convert CLM-CH16-006 from PHYSICAL_PENDING into a documented physical claim? What remains out of scope for a commodity classroom lab?

16.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`), with CE-4 verified register inheritance (`publication/preproduction/ce-04/SOURCE_REGISTER.md`). Project-specific Quartet path measurements remain **PHYSICAL_PENDING** (CLM-CH16-006). WAIKE adjacency uses accepted-main SHA `e97e74fc9bfb44b1cdc26b272dc4848264f15fe0` as competency neighbors only—not as a LAB-PKT-001 module ID.

Inline citation keys used in this chapter include Postel (1981), Postel (1980), Deering and Hinden (2017), Eddy (2022), Iyengar and Thomson (2021), Mockapetris (1987a), Mockapetris (1987b), Braden (1989), Rekhter et al. (1996), Rescorla (2018), IEEE (2020), 3GPP (2026), Kurose and Ross (2020), MDN Web Docs (2026c), and MDN Web Docs (2026a).

16.13 12. Glossary links

Candidate terms introduced or reinforced here (see also `publication/full31/chapters/ch16/GLOSSARY_CA`) do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Packet	Bounded network data unit with headers
Encapsulation	Adding layer-specific headers around payload
Addressing	Identifiers used to deliver packets
Routing	Selecting paths / next hops for packets
Protocol	Shared rules enabling communication

Term	Plain link
Transport layer	End-to-end conversation behaviors above internetworking
DNS	Name-to-address dependency often on the critical path
Local vs LAN vs Internet	Distinct reachability scopes—not synonyms
Stability contract	Concurrent conditions that keep the experience alive

Related earlier chapters: systems tracing method (CH02), cloud/edge placement adjacency (CH15). Related later chapters: Wi-Fi/cellular depth (CH17–CH18), continuity and QoE (CH19–CH20), network attack surfaces (CH23+). CE inheritance: CE-4 primary; CE-6 connected usable.

16.14 Figure references (embeds above; accessibility metadata)

All figures are teaching aids. No fabricated operator telemetry. No Gate 3 PASS claim.

16.14.1 FIG-CH16-001 — Local / LAN / Internet scopes

- **Type.** Comparative layers (inherit FIG-CE4-001 intent).
- **Reader should notice.** Nested scopes; Wi-Fi association Internet usable.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name device, LAN, Internet; state conceptual truth class.

16.14.2 FIG-CH16-002 — Encapsulation sticky notes

- **Type.** Sequence (inherit FIG-CE4-002 intent).
- **Reader should notice.** Headers add outbound; service is not the access radio.
- **Truth class.** Conceptual.
- **Alt text requirement.** Enumerate steps 1–6; deny measured timings.

16.14.3 FIG-CH16-003 — DNS on the critical path

- **Type.** System map.
- **Reader should notice.** Named path can fail at DNS while other work continues.
- **Truth class.** Conceptual.
- **Alt text requirement.** Contrast Path A vs Path B; DNS routing service.

16.14.4 FIG-CH16-004 — LAB-PKT-001 timing phases

- **Type.** Measured classroom / fixture table vocabulary.
- **Reader should notice.** Phase names; honesty banner for n=1 learning.
- **Truth class.** Measured (learner or illustrative fixture)—not a published benchmark.
- **Alt text requirement.** List phases; state EXTERNAL_DEPENDENCY / PHYSICAL_PENDING limits.

16.15 Claim footnotes used in this chapter

- **CLM-CH16-001.** Packets are bounded chunks with headers—not an unbroken hose (Postel 1981).
- **CLM-CH16-002.** Local, LAN, and Internet are different reachability scopes (Postel 1981; Braden 1989 teaching structure).
- **CLM-CH16-003.** Routing decides where next; transport decides how conversation reliability is attempted (Eddy 2022).
- **CLM-CH16-004.** DNS failure can look like Internet failure while other paths work (Mockapetris 1987a, 1987b; Kurose and Ross 2020).
- **CLM-CH16-005.** Wi-Fi and cellular are access on-ramps, not synonyms for the Internet (IEEE 2020; 3GPP 2026; Postel 1981).
- **CLM-CH16-006.** Project-specific path measurements on Quartet remain **PHYSICAL_PENDING**.

Chapter 17

Chapter 17 — Wi-Fi, Cellular, 5G, and the Road to 6G

Status: draft · Chapter ID: CH17

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

17.1 1. The moment

You leave a building. The Wi-Fi icon disappears—or stays, weakly—and a cellular icon takes over. A call continues, or it does not. A chat that felt instant now stalls. Sometimes a **5G** badge appears while the experience still feels bad. From your seat it feels like one story: *I have signal, so it should be fine.*

Underneath that feeling, different **radio access technologies (RATs)** are competing for ownership of the last hop. Wi-Fi is a local wireless LAN family; cellular is an operator mobile access family; neither is “the Internet,” and neither is “the cloud” (IEEE 2020; 3GPP 2026; Kurose and Ross 2020). A generation label on the status bar is not a guarantee that your app’s latency or reliability class is what marketing implied (CLM-CH17-001; CLM-CH17-002).

This chapter is Part IV access-network literacy—not a full RF course, not a 6G product brochure, and not a claim that Device Quartet drive-test numbers exist yet (CLM-CH17-005 remains **PHYSICAL_PENDING**). It follows the book’s arc—**human experience** → **system** → **component**—until bars, badges, and “online” stop being synonyms.

The governing question:

When my access icon changes—or stays lit while the experience fails—what actually changed: the radio path, the operator/core path, or only my story about the icon?

17.2 2. What you notice

Before names like *handover* or *5G System* enter, notice the human contract you already expect.

You expect a call or message to keep working when you walk. You expect “Wi-Fi” and “cellular” icons to mean something different. You expect a **5G** badge to mean *better*—faster, more reliable, somehow fairer. You also expect “connected” to mean the thing you are trying to do can finish.

Those expectations are not decorations around radios. They *are* the product, from the person’s point of view.

A lit access icon is a partial status report, not a completed Stability Contract for the human task.

Notice the split timelines. Association or attachment can succeed while a chat still waits. A 5G icon can stay lit while a handover or Wi-Fi offload briefly breaks the experience (CLM-CH17-004). Moving outdoors can improve bars and still leave a captive portal, DNS stall, or remote service outage untouched—those failures belong to other layers Chapter 16 and CE-4 already separated.

Optional comparison, available on a phone or laptop you already own (or via fixtures if you have no plan): note the **text label** of the active access (Wi-Fi / cellular / unknown), try one ordinary send or sync, then change **one** condition—walk toward a door, toggle Wi-Fi once, or use the lab fixture route. Do not climb for signal. Do not transmit on unlicensed equipment you are not authorized to operate. The point is not a drive test. The point is to notice that icons and usable outcomes can disagree.

17.3 3. Exploded ecosystem

An everyday walk is not one radio. It is a path through an ecosystem. **FIG-CH17-001** is the first-minute map: device Wi-Fi access point path **versus** cellular radio access network (RAN) path operator or campus backhaul Internet path edge/cloud service. Treat it as **Representative educational architecture**, not a claim that your phone’s sealed internals look exactly like the diagram.

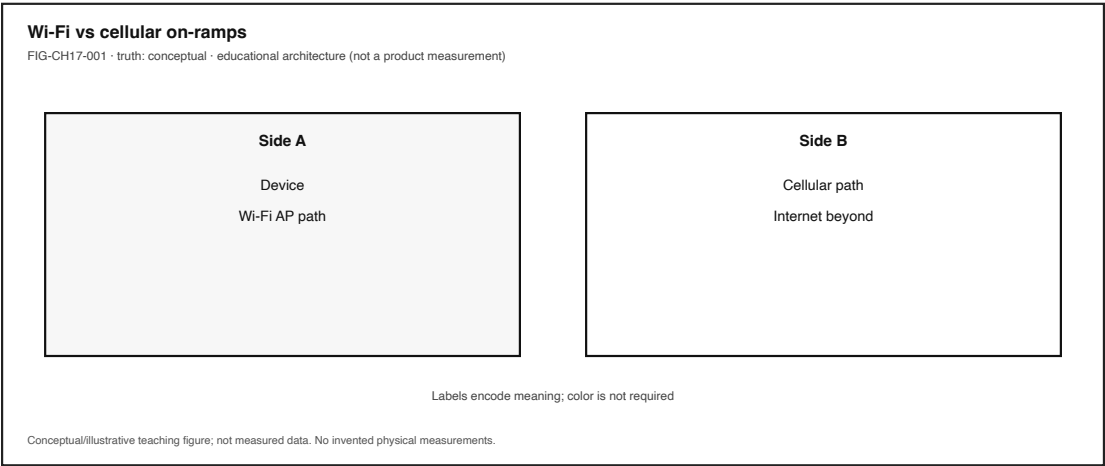


Figure 17.1: Wi-Fi AP path vs cellular RAN/core path vs Internet onward path. Conceptual; Wi-Fi cellular Internet.

Walk the layers in ordinary language, then keep the same layers when vocabulary deepens.

17.3.1 Human

You form intent: keep the call, finish the send, stay on the map. Eyes read icons. Ears and hands judge whether the experience held.

17.3.2 Device radios and policy

Commodity devices often carry **both** Wi-Fi and cellular radios. OS and modem policy choose which path carries which traffic, when to prefer Wi-Fi, and when to fall back. Dual-path does not mean dual success.

17.3.3 Wi-Fi access (local wireless LAN)

Wi-Fi names consumer and enterprise wireless LAN technologies built on the IEEE 802.11 family (IEEE 2020). An access point (AP) or mesh node is a local on-ramp. Wi-Fi can provide LAN reachability without guaranteeing Internet reachability—guest isolation and captive portals are everyday counterexamples (Kurose and Ross 2020).

17.3.4 Cellular access (operator mobile)

Cellular names wide-area mobile access operated as a service: cells, base stations, operator identity, and mobility procedures across coverage areas (3GPP 2026; Kurose and Ross 2020). Generations (2G→5G at survey depth) are standards stories, not bar-count synonyms.

17.3.5 Radio access network and core (cellular path)

On the cellular side, the **radio access network** sits between the device and a **core** that authenticates subscribers and connects onward toward Internet or operator services (3GPP 2026). You rarely see that split. You feel attachment, mobility, and policy.

17.3.6 Internet path and placement (not access)

Packets that leave the access network still need addressing, routing, and a place where the service runs—edge-near or cloud-far. Those are Chapter 15–16 concerns. Collapsing them into “5G” or “Wi-Fi” is the Part IV misconception this chapter refuses (CLM-CH17-001).

17.3.7 Icons, bars, and badges

Status UI summarizes radio and attachment state for humans. It is an interpretation layer—not a QoE meter, not a latency class certificate, and not proof that a particular 5G feature set is active for your app (CLM-CH17-002).

FIG-CH17-002 later places cellular generations through 5G and marks **6G as roadmap**—research and standards direction, not a deployed consumer fact in this manuscript (CLM-CH17-003 · SOURCE_IDENTIFIED via (2023a)).

17.4 4. Follow the signal

FIG-CH17-003 shows a walk: indoor Wi-Fi association → approach exit → Wi-Fi weakens → cellular attachment or handover/offload → app traffic continues or stalls. Read it as a logical story, not as a measured drive test of your city.

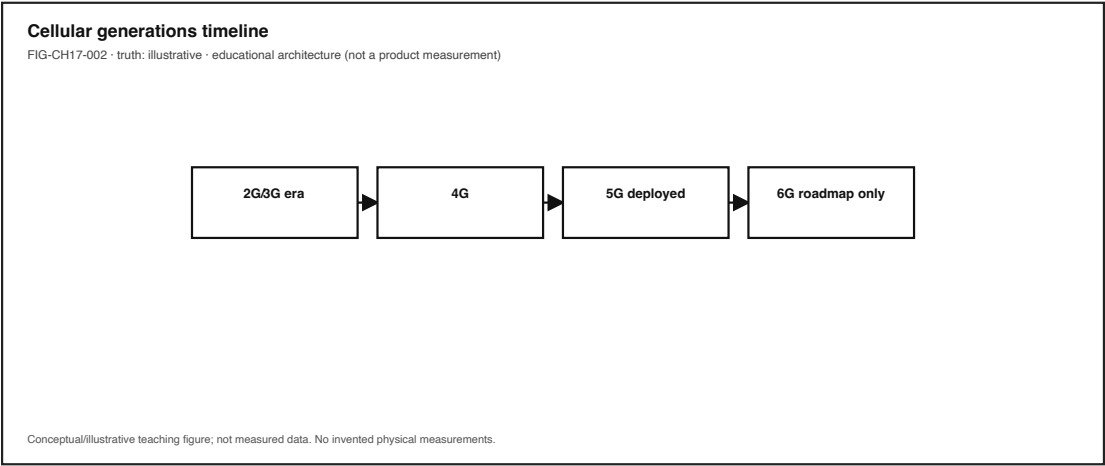


Figure 17.2: Cellular generations through 5G with 6G labeled roadmap—not a consumer deployment fact.

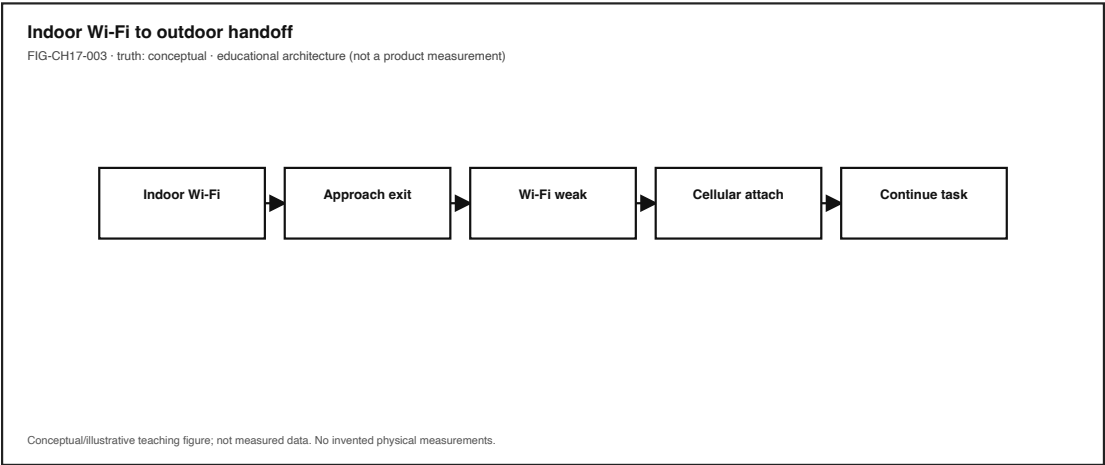


Figure 17.3: Wi-Fi to cellular transition during a walk. Conceptual teaching sequence; not a drive test.

1. **Intent.** A person starts or continues a task that needs a path.
2. **Access choice.** The device uses Wi-Fi, cellular, or switches between them according to policy and conditions (Kurose and Ross 2020).
3. **Local wireless LAN path (when Wi-Fi).** Association with an AP; LAN forwarding; optional captive/auth gates before Internet scope (IEEE 2020; Kurose and Ross 2020).
4. **Cellular path (when cellular).** Attachment to a cell/RAN; authentication and session context in the operator system; onward toward the Internet or operator service (3GPP 2026).
5. **Mobility event.** Handover between cells, or offload between Wi-Fi and cellular, tries to preserve service. Transient failures can occur while icons remain lit (CLM-CH17-004).
6. **Beyond-access path.** DNS (if used), routing, transport, and remote placement still have to succeed—CE-4 / CH16 adjacency.
7. **Human feedback.** Call audio, checkmarks, maps, or “waiting for network” close the loop—or fail to.

17.4.1 Alternate paths (the honesty rule)

Not every “5G” badge implies the same radio band, core feature set, or network slice. Not every Wi-Fi SSID provides Internet. Not every stall is radio: CPU, storage, app logic, or remote overload can impersonate “bad signal.” Teaching those forks prevents the encyclopedia trap—listing every generation marketing name instead of tracing one honest walk.

17.4.2 Failure branch without drama

When something fails, prefer failure *domains* over confident blame:

- Access attachment / association
- Mobility (handover / offload) interruption
- Captive portal / policy block
- Beyond-access path (DNS, routing, transport)
- Remote service placement
- On-device stack or app logic

Outside observation rarely distinguishes those cleanly. That limitation is literacy, not a bug in the reader.

17.5 5. Component cards

For each object: plain language, analogy, technical function, constraints, common symptoms. Analogies are labeled as analogies.

17.5.1 Wi-Fi (IEEE 802.11 family)

- **Plain language.** Local wireless LAN access—usually your building, café, or campus on-ramp.
- **Analogy (labeled).** Like a neighborhood side street onto the larger road system—not the highway itself.
- **Technical function.** Wireless LAN medium access and PHY/MAC families standardized under IEEE 802.11 (IEEE 2020).

- **Constraints.** Shared medium, coverage of APs, authentication/captive gates, interference (qualitative; no invented SNR tables here).
- **Symptoms.** Associated but no Internet; works near AP only; captive portal loop.

17.5.2 Cellular access

- **Plain language.** Operator mobile wide-area radio access you subscribe to (or roam on).
- **Analogy (labeled).** Like a municipal transit system with stations (cells) and rules for transferring—not the destination city.
- **Technical function.** Mobile attachment, mobility, and operator-mediated connectivity toward services and the Internet (3GPP 2026; Kurose and Ross 2020).
- **Constraints.** Coverage, operator policy, plan/capability, device bands (survey depth only).
- **Symptoms.** Bars with no usable data; works outdoors not indoors; generation badge without better feel.

17.5.3 Radio access technology (RAT)

- **Plain language.** A family of radio methods for attaching a device to a network.
- **Analogy (labeled).** Like different vehicle types that can use roads—bike path vs bus lane—not the map of the city.
- **Technical function.** Names the access *kind* (e.g., Wi-Fi vs cellular generations) so readers stop treating “wireless” as one thing (Kurose and Ross 2020).
- **Constraints.** Device support, regulatory bands, operator/AP deployment.
- **Symptoms.** Confusion when dual-radio devices switch without explaining why the experience changed.

17.5.4 5G system (survey)

- **Plain language.** Fifth-generation mobile system as specified in the 3GPP architecture family—at teaching survey depth.
- **Analogy (labeled).** Like a building code for a generation of transit—not a promise that every trip will be on-time.
- **Technical function.** 3GPP describes a 5G System architecture (access and core concerns) distinct from IEEE 802.11 Wi-Fi (3GPP 2026).
- **Constraints.** Deployment choices vary by operator and region; icon feature proof (CLM-CH17-002).
- **Symptoms.** “I have 5G” used as a diagnosis for stalls that are DNS, congestion beyond the RAN, or app-side.

17.5.5 6G roadmap

- **Plain language.** Research and standards *direction* after 5G—not a present consumer network fact in this book.
- **Analogy (labeled).** Like a published transit expansion plan—useful for orientation, not a ticket you can punch today.
- **Technical function.** Teaching humility: roadmap deployment. This manuscript does **not** invent 6G radio parameters, commercial availability dates, or performance guarantees (CLM-CH17-003 · SOURCE_IDENTIFIED via IMT-2030 framework (2023a)).
- **Constraints.** Any consumer “6G” marketing claim needs dated primary evidence before it becomes fact language.
- **Symptoms.** Treating “road to 6G” as “my phone already has 6G bars.”

17.5.6 Handover / mobility (intro)

- **Plain language.** Changing cells or APs while trying to keep service.
- **Analogy (labeled).** Like transferring trains without dropping your luggage—sometimes the transfer itself is the delay.
- **Technical function.** Mobility procedures aim to preserve sessions across radio changes; offload moves traffic between Wi-Fi and cellular (3GPP 2026; Kurose and Ross 2020).
- **Constraints.** Timing, coverage overlap, policy; transient failure possible (CLM-CH17-004).
- **Symptoms.** Brief mute, stalled send, then recovery—while the icon never “looked” offline.

17.5.7 Bars vs experience

- **Plain language.** Signal icons are not quality of experience.
 - **Analogy (labeled).** Like a fuel light that does not tell you whether the road ahead is closed.
 - **Technical function.** Separates radio/attachment summary UI from task success (Kurose and Ross 2020).
 - **Constraints.** Accessibility: do not teach color-only legends; use text labels.
 - **Symptoms.** Full bars, failed task; empty-feeling bars, task still works on another path.
-

17.6 6. Stability contract

A connected walk continues only while multiple hidden conditions stay within acceptable bounds.

Access technologies attach devices; backhaul and core paths carry traffic onward; mobility events must not interrupt beyond what the human task can tolerate; icons must not be mistaken for contract success. Those statements are **qualitative**. They are not a table of invented latency budgets, and they are not a promise that every operator’s “5G” means the same thing.

For the ordinary “I walked outside and it kept working” feeling, conditions like these must hold together:

1. **Chosen access** remains associated/attached as the task requires (Wi-Fi, cellular, or an intentional switch).
2. **Backhaul / core / onward path** remains usable toward the service—not merely a lit RAT icon.
3. **Mobility events** (handover, offload) stay within interruption the experience can absorb (CLM-CH17-004).
4. **Beyond-access dependencies** (DNS, routing, transport, remote placement) still succeed when the task needs them.
5. **Device policy and radios** agree on which path owns the traffic.
6. **Human-visible status** is honest enough not to train false confidence (icon QoE).
7. **Plan / authorization / captive gates** do not silently strand Internet scope.

A system can remain *radio-connected* while the human experience has already failed. Conversely, an icon can look weak while a dual-path device still completes the task on another RAT. Stability is concurrent conditions—not a single badge.

Honesty bound for this edition: Any gnnchOS / Device Quartet cellular or Wi-Fi drive-test numbers remain **PHYSICAL_PENDING** (CLM-CH17-005). Commodity observation in **LAB-PKT-001** produces *your* evidence for *your* path—or fixture honesty—not a universal

RF score. **6G stays roadmap language** under IMT-2030 framework cite (CLM-CH17-003 · SOURCE_IDENTIFIED).

17.7 7. Try it

17.7.1 LAB-PKT-001 — Trace One Connected Action Across Path and Access

Goal. Label a commodity (or fixture) path with device / LAN / Internet scopes **and** access network as Wi-Fi / cellular / unknown / fixture—practicing **Wi-Fi cellular Internet cloud**.

CE / WAIKE alignment note. This lab inherits CE-4 preproduction and publication lab ownership. WAIKE WIRELESS_6G / `wireless_dsp_6g` are **adjacent** competencies only—do not treat this chapter as that full course, and do not invent WAIKE lab IDs.

Safety (hard stops).

- No unlicensed RF transmitters, SDRs used as TXers, jammers, evil-twin APs, or cracking tutorials.
- No unauthorized scanning or packet capture on networks you do not administer.
- Do not capture other users' traffic; scrub PII, tokens, and message bodies from artifacts.
- Do not climb for signal; avoid distracted walk-and-test near traffic.
- No Device Quartet RF field campaign required or claimed.
- Learners without cellular plans: use **fixture Route B** (equity default).

Routes.

- **Route A — Commodity.** Browser or CLI path inspect on a device you already use; label access from OS text (not color alone); change one condition carefully or stop.
- **Route B — Fixture (mandatory accessible path).** Use `labs/LAB-PKT-001/fixtures/` and the CLI `--fixture` quiz; record that rows are not your radio measurements.

Explorer baseline (about 45–60 minutes).

1. Predict failure family (latency / reliability / throughput) and access mode before running.
2. Run Route A demo or Route B fixtures per `labs/LAB-PKT-001/README.md`.
3. Record observations only: icon/text labels, UI phrases, coarse timing phases if visible.
4. Draw a path diagram with access labeled Wi-Fi XOR cellular XOR unknown/fixture.
5. Fill observation-vs-inference; teach-back the four-way distinction.

Operator extension. Log icon/text vs usable send across one access change (doorway walk or Wi-Fi toggle)—or fixture comparison rows. Do not claim tower congestion without evidence.

Builder extension. Diagram a dual-path device (Wi-Fi + cellular) with policy choice as a labeled fork—no RF math required.

Engineer extension. Place “5G” at survey depth: cite architecture literacy via 3GPP TS 23.501 family without pinning undownloaded clause numbers (3GPP 2026). List which failure domains your observations cannot distinguish.

Researcher extension. Separate roadmap claims from deployed facts with dates. For 6G, state explicitly: **not deployed consumer fact here**; cite IMT-2030 framework (2023a) (CLM-CH17-003). Forbid inventing Quartet drive-test numbers (CLM-CH17-005).

Proposed observation-only stretch (not a separate shipping lab ID). LAB-ACCESS-OBS-001 in the chapter packet remains a **proposed** name for a walk-test icon-vs-outcome log with fixture alternative—still **no RF TX**.

Evidence to keep. Path diagram; timing/observation table; scrubbed notes or fixture proof; teach-back paragraph.

17.8 8. Build it

Extend LAB-PKT-001 without turning Part IV into a spectrum encyclopedia.

17.8.1 Explorer

Build a pocket card: Wi-Fi / cellular / Internet / cloud—one plain sentence each that forbids synonym collapse.

17.8.2 Operator

Build an “icon vs outcome” checklist: access text label → task result → next evidence needed. End with “needs more evidence,” never with fake tower certainty.

17.8.3 Builder

Build a dual-path diagram: human → device policy → Wi-Fi AP path and cellular RAN/core path → Internet → service placement hypothesis. Mark visible vs sealed.

17.8.4 Engineer

Build a one-page generation survey: 2G→5G as standards evolution bullets at teaching depth, plus a clearly labeled **6G = roadmap** box with no invented specs (3GPP 2026). Mark CLM-CH17-003 as bounded.

17.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to make yet—for example, “operator X’s 5G icon guarantees URLLC-class latency for app Y,” or “consumer 6G is live in region Z.” Specify dated standards/operator evidence required; keep Quartet measurements **PHYSICAL_PENDING**.

Educators can facilitate teach-backs from Section 11 and keep classrooms on Route B when plans, privacy, or public Wi-Fi make Route A unsafe.

17.9 9. Secure and include it

17.9.1 Security

Access networks are attack and abuse surface: rogue APs, captive-portal phishing, and unauthorized radio transmission. This book’s labs forbid jamming, cracking, and unlicensed TX experiments. Prefer known networks, fixture routes, and user consent over “interesting” radio

mischief. Survey literacy here foreshadows later trust and privacy chapters without becoming an offensive wireless lab.

17.9.2 Privacy

Access mode and timing logs can imply location patterns. LAB-PKT-001 artifacts record access as a category, not a GPS trail. Redact accounts, message previews, and classmate screens.

17.9.3 Accessibility

Status icons must not be taught by color alone—require text legends (Wi-Fi / cellular / 5G badge names as text). Fixture Route B is mandatory for learners without stable broadband or personal cellular plans. Document walk-test steps in words; do not require climbing or hazardous mobility.

17.9.4 Equity

Cellular plans, indoor coverage, and café Wi-Fi quality shape who can participate. Teaching access literacy includes naming those barriers and shipping a fixture path so literacy is not gated on a subscription.

17.9.5 Safety

No RF transmitter kits in classroom procedures. No distracted street testing. Physical and spectrum safety are part of professional wireless culture, not optional footnotes.

17.9.6 Ethics

Do not claim deployed consumer 6G, invent drive-test dB tables, or present Quartet radio results you do not have. Overclaiming coverage is still false evidence.

17.10 10. Career lens

One doorway walk crosses many ownership domains. No table promises employment; roles vary by organization. LAB-PKT-001 artifacts resemble early professional evidence in miniature: labeled access, observation discipline, and explicit uncertainty.

Role lens	Typical artifacts	Review questions
Wireless engineer (ROLE-WIRELESS)	Link/access observations, RF plans when authorized	Access vs beyond-access ownership clear?
Network engineer (ROLE-NET)	Path timing, reachability notes	Icon failure or path/DNS failure?
Mobile / systems (operator adjacency)	Mobility/attach incident notes	Handover/offload vs core/policy?
SRE (ROLE-SRE)	Connected vs usable evidence	Did the service placement fail while RAT looked fine?
Security (ROLE-SEC)	Threat notes for rogue AP / portal abuse	Did we forbid unauthorized TX and capture?

Role lens	Typical artifacts	Review questions
Accessibility (ROLE-A11Y)	Non-color status legends, fixture routes	Can learners without plans complete the lab?

Portfolio hint: a scrubbed access-labeled path diagram plus observation-vs-inference beats a vibes-based “5G is broken” claim.

17.11 11. Check understanding

Concept. In one sentence each, define *Wi-Fi*, *cellular*, and *Internet* so that none of them swallows the other two—and add why *cloud* is still a fourth idea.

System tracing. Trace a doorway walk from indoor Wi-Fi to outdoor cellular in numbered steps. Mark which steps you observed and which you inferred.

Misconception check. Why is “I have 5G, so latency must be fine” incomplete? What does the icon fail to prove (CLM-CH17-002)?

Misconception check. Why must this chapter refuse deployed-consumer 6G language without dated primary evidence (CLM-CH17-003)?

Teach-it-back. Explain to a newcomer—using only LAB-PKT-001 vocabulary—why a phone can show bars and still fail a send during a Wi-Fi cellular transition.

Researcher prompt. What evidence would convert a PHYSICAL_PENDING Quartet Wi-Fi/cellular measurement claim into a documented physical claim? What remains out of scope for a no-TX classroom lab?

17.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Project-specific Device Quartet RF status remains in `evidence/claim_registry.yaml` and in the chapter claim plan (CLM-CH17-005), separately from external literature.

Inline citations used in this chapter include IEEE (2020), 3GPP (2026), Kurose and Ross (2020), and (2023a). Claim CLM-CH17-003 (6G/IMT-2030 roadmap) is `SOURCE_IDENTIFIED`—still bounded as not deployed consumer fact.

17.13 12. Glossary links

Candidate terms introduced or reinforced here (see also chapter glossary candidates; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Wi-Fi	Wireless LAN access based on IEEE 802.11 technologies
Cellular	Mobile wide-area radio access operated as a service
Radio access technology (RAT)	Family of radio methods attaching devices to networks
5G	Fifth-generation 3GPP mobile system (survey sense)
6G roadmap	Next-generation research/standards direction—not a present consumer fact here
Handover	Transfer of a connection between cells or access points
Radio access network	Radio portion of a mobile network between device and core
Bars vs experience	Signal icons are not QoE
Stability contract	Concurrent conditions that keep the access experience alive

Related earlier chapters: packets/Internet (CH16), cloud/edge placement (CH15), CE-4 survey inheritance. Related later chapters: spectrum/radio depth (CH18), NTN continuity (CH19), latency/reliability/QoE (CH20).

17.14 Figure references (embedded; registered SVG + a11y)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured evidence. No fabricated telemetry or drive-test plots.

17.14.1 FIG-CH17-001 — Wi-Fi AP path vs cellular RAN path vs Internet

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Comparative layers.
- **Reader should notice.** Access technologies are on-ramps; Internet path and cloud/edge placement sit beyond; Wi-Fi cellular Internet.
- **Truth class.** Conceptual / Representative educational architecture.
- **Alt text requirement.** Name Wi-Fi AP path, cellular RAN/core path, Internet onward path, and service placement; state conceptual truth class; color not sole encoding.

17.14.2 FIG-CH17-002 — Generations survey to 5G to 6G roadmap

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Timeline.
- **Reader should notice.** 5G as deployed-generation survey language vs **6G clearly labeled roadmap** (not consumer deployment fact).
- **Truth class.** Illustrative.

- **Alt text requirement.** Enumerate generation labels in order; explicitly mark 6G as roadmap; deny invented 6G specs.

17.14.3 FIG-CH17-003 — Wi-Fi to cellular transition during a walk

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Sequence.
- **Reader should notice.** Numbered mobility/offload steps plus a failure branch where icons stay lit.
- **Truth class.** Conceptual.
- **Alt text requirement.** Enumerate steps; label transient failure while icon lit; state non-measured teaching sequence.

17.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH17-001.** Wi-Fi and cellular are different access technologies; neither is the Internet—framed with IEEE 802.11, 3GPP TS 23.501, and textbook survey depth (IEEE 2020; 3GPP 2026; Kurose and Ross 2020).
- **CLM-CH17-002.** 5G denotes a 3GPP system generation; icon presence does not prove a specific latency/reliability class for the user’s app (3GPP 2026).
- **CLM-CH17-003.** 6G/IMT-2030 is future/roadmap; do not claim deployed consumer 6G as present fact. Status: SOURCE_IDENTIFIED (itu-r-m2160-2023).
- **CLM-CH17-004.** Handover and offload can cause transient experience failures while icons stay lit (3GPP 2026; Kurose and Ross 2020).
- **CLM-CH17-005.** Any gunnchOS/Quartet cellular/Wi-Fi drive-test numbers are PHYSICAL_PENDING.

Chapter 18

Chapter 18 — Spectrum, Antennas, Beams, MIMO, and Radio Conditions

Status: draft · **Chapter ID:** CH18

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

18.1 1. The moment

You stand by a window and a stuttering video suddenly smooths out. In a kitchen, a call freezes when someone starts the microwave. You rotate the phone in your hand and the bars rearrange themselves—without opening Settings, without changing the app, without “fixing the Internet.”

From your seat, it feels like superstition: windows help, kitchens hurt, orientation is magic.

Underneath, several physical and regulatory conditions are moving at once. **Spectrum** is shared and constrained. **Antennas** couple circuits to waves with patterns that favor some directions over others. Obstacles and distance contribute to **path loss**. Unwanted energy becomes **interference**. Modern systems may use **MIMO** and **beamforming**—spatial techniques that textbooks and standards discuss at survey depth—while the icon on screen still says little more than “connected” (Kurose and Ross 2020; IEEE 2020).

This chapter is Part IV’s radio-conditions lens: not a drive-test report, not an antenna datasheet with invented dB gains, and not permission to transmit experimentally. It follows the book’s arc—**human experience** → **system** → **component**—until “bad Wi-Fi” stops being a single blame word and becomes a set of concurrent conditions you can name without overclaiming.

The governing question:

Why can radio conditions change experience while connection icons stay lit—and what am I still not allowed to invent as a measurement?

18.2 2. What you notice

Before names like *MIMO* or *delay spread* enter, notice the human contract you already expect.

You expect wireless to keep working when you walk across a room. You expect “more bars” to mean “better experience,” even though you have already lived through counterexamples. You expect a lit Wi-Fi or cellular icon to mean the service you care about is usable. You expect turning the device, covering it with a hand, or standing near glass or metal to somehow matter—even when you cannot see a radio wave.

Those expectations are not decorations around networking. They *are* the product, from the person’s point of view.

A usable wireless moment is a perception produced by spectrum access, antennas and orientation, time-varying channel conditions, and higher-layer recovery—not by an icon alone.

Notice the split timelines. Association or attachment can succeed while throughput collapses. A microwave can degrade a kitchen link while the laptop still shows connected. A window seat can improve one path while another room stays contested. Commodity bars and megabit summaries are marketing-friendly compressions; they are not laboratory measurements of received power, noise, or spatial streams (Kurose and Ross 2020).

Optional comparison, available on almost any device you already own: pick one familiar online action (a short video, a sync, a call preview). Do it once where you usually sit. Move once—near a window, or one room farther from the access point you already use—and repeat. Do not climb, do not open walls, do not attach improvised antennas, and do not transmit with anything you do not already legally operate as an end-user device. The point is to notice that “connected” and “usable” can diverge, and that outside observation alone rarely proves *why* in RF units.

18.3 3. Exploded ecosystem

A wireless moment is not a single object. It is a path through an ecosystem. **FIG-CH18-001** is the first-minute map: device obstacles/body antennas shared spectrum access point or cell site beyond. Treat it as **Representative educational architecture**, not a claim that your sealed phone’s antenna layout matches the cartoon (CLM-CH18-005).

Walk the layers in ordinary language, then keep the same layers when vocabulary deepens.

18.3.1 Human

You form intent: watch, talk, sync, navigate. Hands rotate the device. Body and furniture sit in the path. Eyes and ears judge smoothness, stalls, and whether the UI’s confidence matches lived experience.

18.3.2 Device radios and antennas

Inside the sealed host, radios tune to allowed channels and antennas couple energy to and from space. Antenna **patterns** matter: some directions are favored; body blockage and grip can change what the device “hears” without any settings change (IEEE 2020). This chapter does **not** invent device-specific gain or sensitivity numbers.

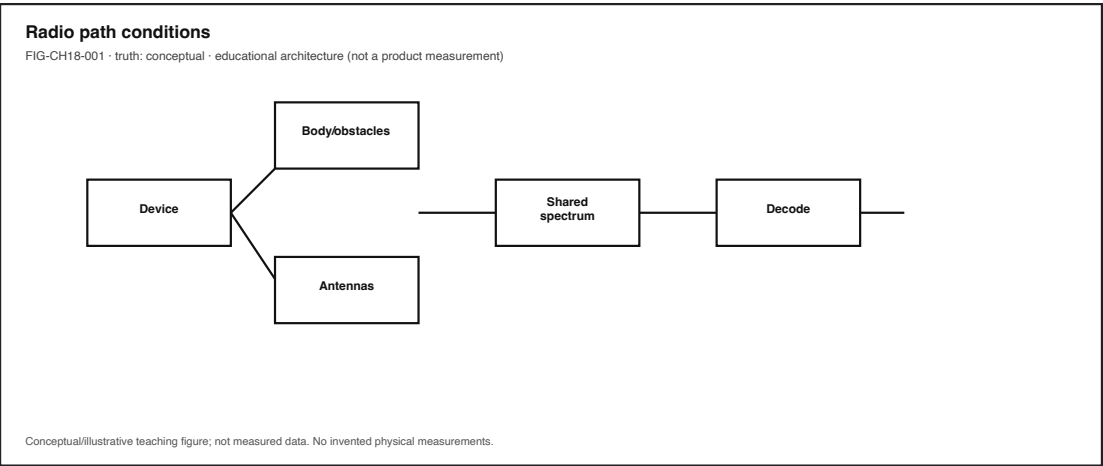


Figure 18.1: Device obstacles antennas shared spectrum AP/cell. Conceptual educational map.

18.3.3 Obstacles and geometry

Walls, glass, people, appliances, and distance shape how much useful energy arrives. Teaching language here is qualitative **path loss** and blockage—not a fabricated link budget for your apartment.

18.3.4 Spectrum vs channel

Spectrum is the regulated frequency resource wireless systems are allowed to use. A **channel** is a specific slice of that resource (plus the access rules for sharing it)—not a synonym for “all spectrum,” and not a synonym for “the Internet.” Coexistence and contention happen *on channels* inside allocated spectrum. Wi-Fi local access and cellular operator access remain distinct on-ramps from earlier Part IV chapters (Kurose and Ross 2020; IEEE 2020).

18.3.5 Access network edge

A home or campus **access point**, or a cellular radio access node, terminates the air interface and forwards toward local or Internet paths. Bars report a compressed story about that edge—not a full channel sounding.

18.3.6 Network and service beyond

Packets, DNS, transport retries, and edge/cloud placement still apply (CH16–CH17 adjacency; **LAB-PKT-001**). Radio conditions can dominate the feel, but they are not the only failure domain.

18.3.7 Measurement honesty overlay

FIG-CH18-004 marks Device Quartet / research RF measurements as **PHYSICAL_PENDING**. `WAIKE lab_fspl_budget` and `lab_delay_spread` are competency adjacencies—math toys or lab neighbors—not publication proof of your device’s dB path (CLM-CH18-005).

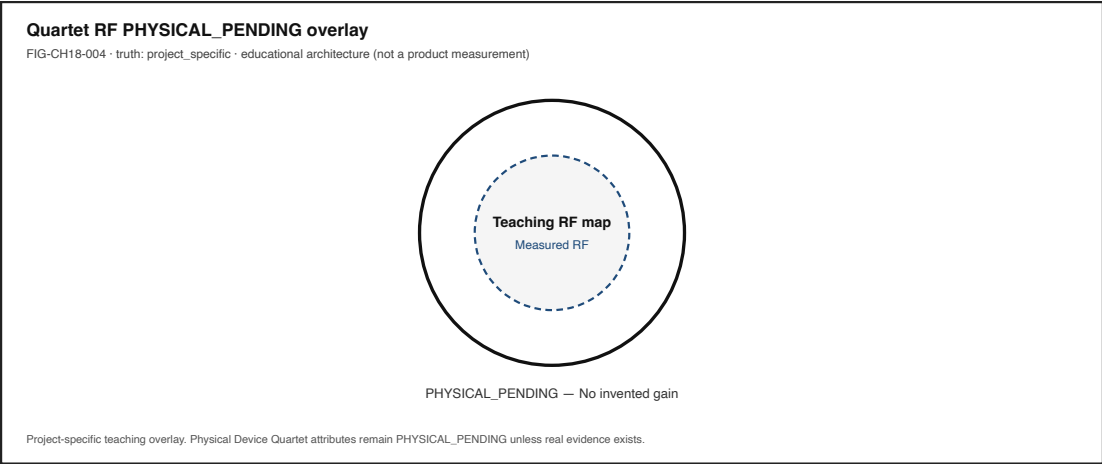


Figure 18.2: Quartet / research antenna placement with PHYSICAL_PENDING overlay. Project teaching aid.

18.4 4. Follow the signal

FIG-CH18-002 and the MIMO metaphor in **FIG-CH18-003** support this sequence. Read the steps as a logical story, not as a claim that every commodity chipset exposes every step to the UI.

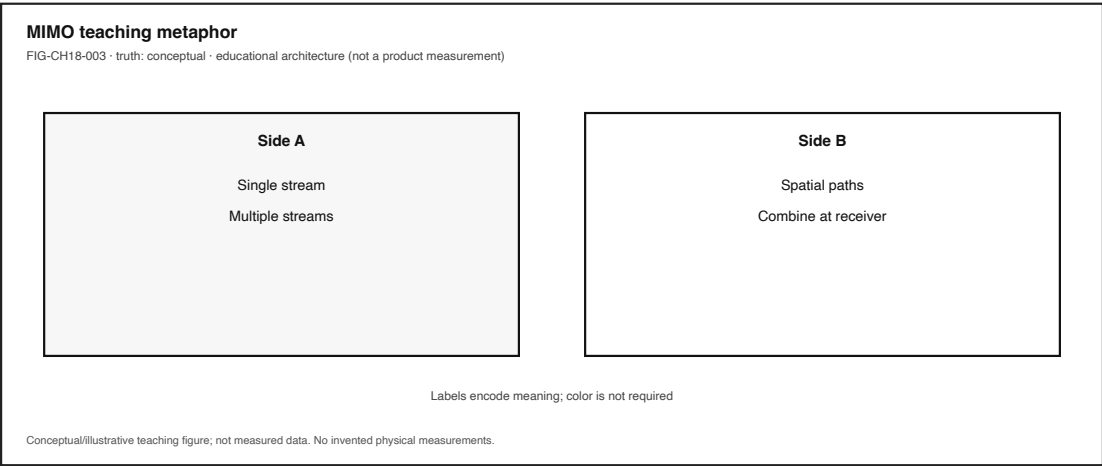


Figure 18.3: MIMO spatial streams metaphor. Illustrative teaching image—not a chipset claim.

1. **Intent.** A person starts an experience that needs bits across a radio hop.
2. **Spectrum / channel selection.** The device and network use allowed frequencies and channel-access rules appropriate to the technology family (IEEE 2020; Kurose and Ross 2020).
3. **Radiate / receive via antennas.** Energy couples through antenna structures; orientation and nearby bodies change coupling qualitatively.
4. **Propagate.** Distance, obstacles, and reflections shape what arrives—path loss and multipath as survey ideas, not invented delay-spread microseconds for your room.
5. **Interfere or contend.** Other transmitters, appliances, or dense neighbor networks can

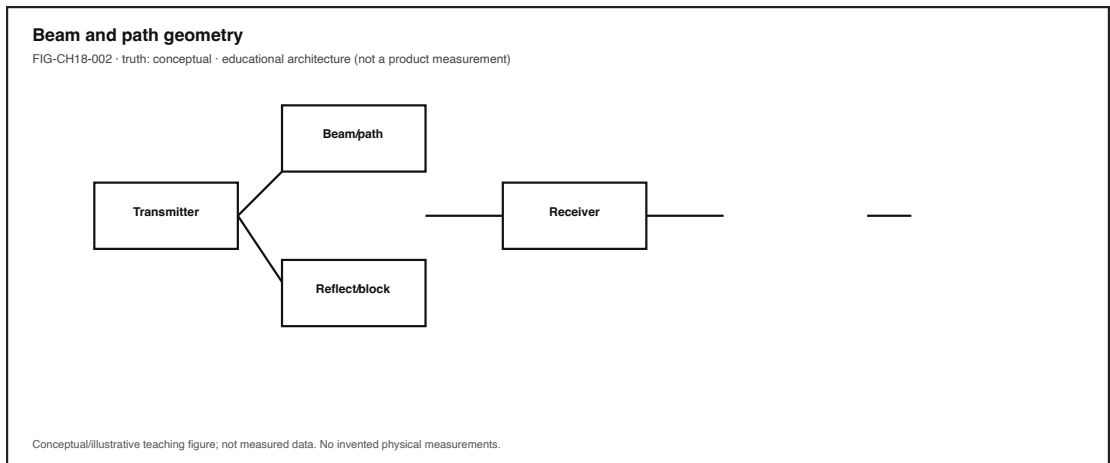


Figure 18.4: Omni vs beam pattern intuition. Conceptual; no invented dBi product scales.

degrade the desired link while association remains (IEEE 2020).

6. **Spatial processing (when present).** MIMO and beamforming are spatial techniques: multiple antennas and steered sensitivity/energy. Teaching depth here is vocabulary and honesty—**do not invent device-specific dB gains** (CLM-CH18-002) (IEEE 2020).
7. **Demodulate and recover.** Radios and link adaptation try to keep a usable bit pipe; retries may hide loss until the human timeline breaks.
8. **Forward beyond the air.** Frames become packets on a path toward a service; CH16/CH17 own that continuation.
9. **Human feedback.** Smooth video, frozen tiles, rising bars, or a still-green icon with a dead experience.

18.4.1 Alternate paths (the honesty rule)

Not every stall is radio. DNS, captive portals, server overload, CPU/thermal limits, and application logic can mimic “bad signal.” Not every improvement near a window is “more MIMO.” Teaching those forks prevents the encyclopedia trap: listing every band name instead of tracing one honest experience.

18.4.2 Failure branch without drama

When something fails, prefer failure *domains* over confident RF blame:

- Attachment/association present but experience failed (icon usable)
- Geometry / blockage / orientation (hypothesis until evidenced)
- Interference or contention (hypothesis; no unauthorized sniffing)
- Throughput vs latency vs reliability mismatch (reuse LAB-PKT-001 language)
- Path/service beyond the radio hop

Outside observation rarely yields calibrated dBm root cause. That limitation is literacy, not a bug in the reader (CLM-CH18-003).

18.5 5. Component cards

For each object: plain language, analogy, technical function, constraints, common symptoms. Analogies are labeled as analogies. No invented antenna gain or receiver sensitivity figures.

18.5.1 Spectrum

- **Plain language.** The range of radio frequencies allocated and used for communication.
- **Analogy (labeled).** Like the whole road network that regulators open under rules—not an infinite private highway.
- **Technical function.** Provides the frequency resource wireless systems are allowed to use (IEEE 2020; Kurose and Ross 2020).
- **Constraints.** Regulation, band plans, power limits, coexistence, and policy—not “use any frequency you can tune.”
- **Symptoms.** Crowded bands, DFS/weather-radar adjacency (awareness only), café density that feels “busy” without proving a spectrogram.

18.5.2 Channel

- **Plain language.** A specific frequency slice (and its sharing rules) inside allocated spectrum.
- **Analogy (labeled).** Like one lane with a speed limit and merge rules—not the entire road network.
- **Technical function.** Channelization and channel-access procedures let stations share spectrum without treating “spectrum” and “this channel” as the same word (IEEE 2020; Kurose and Ross 2020).
- **Constraints.** Width, numbering, DFS/availability, and contention; UI “channel” labels are compressed.
- **Symptoms.** Works on one AP channel setting and stalls on another; dense SSIDs fighting the same slice.

18.5.3 Antenna

- **Plain language.** The transducer between circuits and radiated waves; patterns matter.
- **Analogy (labeled).** Like a megaphone and ear shape combined—direction and blockage change what is loud, without rewriting the song.
- **Technical function.** Couples guided signals to propagating waves (and back); placement and orientation affect link behavior (IEEE 2020).
- **Constraints.** Size, detuning near hands/metal, pattern nulls, product industrial design. **No invented dBi/dBm claims here.**
- **Symptoms.** Hold-angle changes; case/hand coverage changes; one orientation works better for a call.

18.5.4 Path loss (intro)

- **Plain language.** Tendency for received usefulness to drop with distance and obstacles.
- **Analogy (labeled).** Like shouting across rooms—walls and distance quiet the message even when you still “have a voice.”
- **Technical function.** Names why geometry matters before any formula is memorized (Kurose and Ross 2020).
- **Constraints.** Environment-specific; classroom estimates are not Quartet EVT.

- **Symptoms.** Works by the window; fails in the elevator lobby; improves when you move one room closer.

18.5.5 Interference

- **Plain language.** Unwanted energy that degrades a desired link.
- **Analogy (labeled).** Like trying to talk while a blender runs—presence of “connection” does not mean presence of clarity.
- **Technical function.** Captures contention and energy that raise errors or force slower link adaptation (IEEE 2020).
- **Constraints.** Diagnosis without authorization is observation-only; no hostile jamming labs.
- **Symptoms.** Kitchen-appliance correlation; dense-apartment evening slowdowns; icon stays lit while apps stall (CLM-CH18-003).

18.5.6 MIMO (survey)

- **Plain language.** Using multiple antennas to send and/or receive spatial streams.
- **Analogy (labeled).** Like several coordinated conversation lanes in space—not merely “more bars.”
- **Technical function.** Spatial multiplexing / diversity ideas at survey depth inside modern WLANs and cellular systems (IEEE 2020). **SISO** (single-input single-output) is the single-antenna baseline; MIMO adds multiple antennas—without guaranteeing better experience from a marketing “NxN” label alone.
- **Constraints.** Channel conditions, device antenna count, and implementation limits; **no fake stream-gain tables**.
- **Symptoms.** Marketing “NxN” labels that do not explain a frozen call by themselves.

18.5.7 Beamforming / beams (survey)

- **Plain language.** Preferentially directing transmit energy or receive sensitivity in space.
- **Analogy (labeled).** Like turning a flashlight toward someone instead of lighting the whole room equally.
- **Technical function.** Spatial focusing techniques discussed in wireless standards families at overview depth (IEEE 2020).
- **Constraints.** Tracking mobility, calibration, and environment; survey literacy measured array factor for Quartet hardware (**PHYSICAL_PENDING**).
- **Symptoms.** Performance that seems direction-sensitive beyond simple distance.

18.5.8 Radio condition

- **Plain language.** The time-varying state of the wireless channel affecting links.
 - **Analogy (labeled).** Like weather for bits—changing under your feet while the map app still says you are “on the road.”
 - **Technical function.** Bundles path loss, interference, multipath, and mobility into the lived stability of the hop (Kurose and Ross 2020).
 - **Constraints.** Commodity UIs expose little; calibrated campaigns stay **PHYSICAL_PENDING** for project hardware (CLM-CH18-005).
 - **Symptoms.** Same SSID, different rooms, different feel; walk-induced glitches; recovery without changing settings.
-

18.6 6. Stability contract

A wireless experience continues only while multiple hidden conditions stay within acceptable bounds.

For the ordinary “this call/video/sync still works here” feeling, conditions like these must hold together—**qualitatively**, with **no invented numeric budgets**:

1. **Spectrum access** lawful and usable for the needed link (channel available enough for the task).
2. **Antenna coupling / orientation / body blockage** still inside what the link can tolerate.
3. **Path loss and geometry** not dominating the needed rate.
4. **Interference / contention** not dominating.
5. **Spatial features (MIMO/beams), when relied upon**, still tracking well enough for the moment (survey depth).
6. **Mobility** not exceeding what tracking and handoff/roaming can hide from the human timeline.
7. **Beyond-the-air path** still healthy enough (DNS, routing, service)—or the failure is not “radio only.”

A system can remain *technically* associated while the human experience has already failed: green icon, frozen tiles, rising spinner. Conversely, bars can look mediocre while a low-rate chat still succeeds. Stability is concurrent conditions—not a single glyph (CLM-CH18-001; CLM-CH18-003).

Honesty bound for this edition: Quartet / research RF measurements remain **PHYSICAL_PENDING**. Optional WAIKE FSPL fixtures are illustrative adjacency only—label them illustrative if used (CLM-CH18-005). Commodity observation in **LAB-RADIO-OBS-001** produces *your* experience log for *your* place—not a calibrated drive test and not a product antenna score.

18.7 7. Try it

18.7.1 LAB-RADIO-OBS-001 — Observe Radio Conditions Without Transmitting

Goal. Change one geometric condition relative to an access path you already use (or study an offline fixture card), record experience changes, and keep observation separate from RF inference—**no transmitters, no spectrum analyzers required, no unauthorized sniffing**.

Inheritance. Symptom vocabulary and path/access labeling continue **LAB-PKT-001**. This lab zooms into radio-condition *feel* without requiring packet tooling.

WAIKE alignment note. WAIKE accepted `main` includes adjacent digital_rc labs such as `lab_fspl_budget` and `lab_delay_spread`, plus course pointer **WIRELESS_6G**. Those are competency adjacencies. They are **not** renamed as publication lab IDs. **LAB-RADIO-OBS-001** is publication-owned.

Safety (hard stops).

- Legal end-user spectrum use only; operate only devices you already own and are allowed to use.

- No unauthorized transmission, no jamming, no amplifier experiments, no homemade transmitters.
- No spectrum sniffing or packet capture on networks you do not administer (prefer fixture / UI observation).
- No climbing for signal stunts; no opening sealed devices; no improvised external antennas.
- No Device Quartet RF campaign required or claimed.

Routes.

- **Commodity route.** Phone or laptop you already own + a familiar Wi-Fi or cellular path you already use.
- **Offline fixture route.** Use the lab's observation card and worksheet without live RF claims.

Explorer baseline (about 40 minutes).

1. Predict: will moving toward a window (or one room closer) change smoothness, stalls, or only icons?
2. Run one familiar action once at your usual seat—or read the fixture card.
3. Change **one** condition (window vs interior seat, or phone orientation). Do not change apps mid-comparison if you can avoid it.
4. Record observations only: UI text, coarse wall-clock feel, icon state, whether the experience became usable.
5. Fill an observation-vs-inference table. Every dBm-style or “interference from X” guess needs a note about extra evidence required.
6. Explicitly write: **no antenna gain/sensitivity numbers invented.**

Operator extension. Compare two times of day or two rooms. Still no root-cause certainty from bars alone.

Builder extension. Draw a labeled diagram: human → device/antenna → obstacles → AP or cell → beyond. Mark which nodes you observed vs inferred.

Engineer extension. List which failure domains your observations *cannot* distinguish (radio vs DNS vs server vs CPU). Name non-destructive next evidence (LAB-PKT-001 timing table; alternate known network; fixture). Still no unauthorized RF instruments.

Researcher extension. Draft a measurement plan marked **PHYSICAL_PENDING**: what calibrated instruments, permissions, and repeat counts would be required to convert a qualitative stall into a documented RF claim. Forbid publishing invented drive-test numbers.

Evidence to keep. Observation-vs-inference table; radio-condition diagram; scrubbed notes; teach-back paragraph. Prefer synthetic descriptions over photos of classmates' networks.

18.8 8. Build it

Extend LAB-RADIO-OBS-001 without turning Part IV into an illegal RF hobby kit.

18.8.1 Explorer

Build a pocket card: spectrum, channel, antenna, path loss, interference, SISO/MIMO/beams (survey)—one plain sentence each, plus “icon usable.”

18.8.2 Operator

Build a “kitchen stall” checklist that starts with observation columns (time, place, icon, experience) and ends with “needs more evidence”—never with a fake spectrogram.

18.8.3 Builder

Build a labeled diagram (paper or digital) of device—obstacle—AP/cell for one familiar place. Annotate **illustrative** vs **measured**. Optional: add a clearly labeled illustrative FSPL worksheet line—and mark it illustrative, not measured (CLM-CH18-005).

18.8.4 Engineer

Build a one-page MIMO/beamforming vocabulary card at survey depth citing standards/textbook framing rather than inventing array gains (IEEE 2020; Kurose and Ross 2020). Mark each bullet “general literacy” vs “needs product evidence.”

18.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to make yet—for example, “Quartet antenna placement improves uplink by N dB.” Specify permissions, instruments, and ethics. Keep the claim **PHYSICAL_PENDING** (CLM-CH18-005).

Educators can facilitate teach-backs from Section 11 and keep classrooms on the offline fixture route when live networks are unsafe or inequitable.

18.9 9. Secure and include it

18.9.1 Security

Radio links are shared mediums: eavesdropping risk on open networks, evil-twin awareness, and “connected” “safe.” Survey literacy foreshadows later trust chapters. Do not teach cracking, jamming, or unauthorized interception. Learner labs must not include unauthorized transmission or spectrum sniffing that violates law or policy (CLM-CH18-004) (Gunn 2026a).

18.9.2 Privacy

Screenshots of Wi-Fi names, cell status, maps, or message previews can reveal location and identity. LAB-RADIO-OBS-001 artifacts must scrub SSIDs that identify home addresses when sharing portfolios, and must not capture account tokens.

18.9.3 Accessibility

Bars and heatmaps often encode status with color alone. Lab writeups must include text status (“usable / stalled / icon-only”). Fixture routes keep learners without personal hotspots inside the learning loop. Motion-only “wave” animations are not a substitute for a text diagram. Prefer patterns and labels over color-only encodings (W3C 2023).

18.9.4 Equity

Assuming always-on home mesh Wi-Fi excludes many readers. Café, library, and metered cellular conditions are first-class. “Just buy a better router” is not a universal fix; literacy includes naming cost barriers. No SDR required for this chapter’s labs.

18.9.5 Safety

No climbing, no illegal antennas, no battery or device abuse, no distraction hazards while walking-and-testing. Physical RF campaigns for Quartet hardware stay out of classroom scope until evidence exists.

18.9.6 Ethics

Do not invent antenna gains, sensitivity floors, drive-test plots, or Quartet RF results. Overclaiming radio performance is still false evidence.

18.10 10. Career lens

One stuttering call crosses many ownership domains. No table promises employment; roles vary by organization. LAB-RADIO-OBS-001 artifacts resemble early professional evidence in miniature: observation discipline, labeled diagrams, and explicit uncertainty.

Role lens	Typical artifacts	Review questions
Wireless / RF engineer	Link notes, authorized measurements	Are we guessing from icons?
Antenna engineer	Pattern/placement reviews (when evidenced)	Orientation and body effects named?
Wireless systems researcher	Channel models, measurement plans	PHYSICAL_PENDING labeled honestly?
Network engineer	Path/access diagrams	Radio vs beyond-the-air owned clearly?
IT support / operator	Ticket notes with observation tables	Did we separate feel from root cause?
Accessibility / equity advocate	Non-color status, fixture routes	Who cannot take the live path?

Portfolio hint: a scrubbed observation-vs-inference table plus a teach-back that icons usable is more honest than a vibes-based “the spectrum is bad” claim.

18.11 11. Check understanding

Concept. In one sentence each, define *spectrum*, *channel*, and *antenna* so that none of them swallows the other two.

System tracing. Trace a familiar stall from hand/orientation to human feedback in numbered steps. Mark which steps you observed and which you inferred.

Misconception check. Why is “Wi-Fi connected means radio conditions are fine” incomplete?

Misconception check. Why must this chapter refuse invented antenna gain or Quartet drive-test numbers even when discussing MIMO and beams?

Teach-it-back. Explain to a newcomer—using only LAB-RADIO-OBS-001 vocabulary—why a microwave-adjacent stall can happen while the icon stays lit.

Researcher prompt. What evidence would convert a PHYSICAL_PENDING Quartet RF claim into a documented physical claim? What remains out of scope for a commodity classroom lab?

18.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Project-specific Device Quartet / research RF status remains **PHYSICAL_PENDING** in the chapter claim plan (CLM-CH18-005), separately from external literature.

Inline citations used in this chapter include IEEE (2020), Kurose and Ross (2020), Gunn (2026a), and W3C (2023).

18.13 12. Glossary links

Candidate terms introduced or reinforced here (see also chapter glossary candidates; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Spectrum	Frequency resources allocated and used for radio communication
Channel	Specific frequency slice and sharing rules inside allocated spectrum
Antenna	Transducer between circuits and radiated waves; patterns matter
Path loss	Tendency for link usefulness to drop with distance/obstacles
Interference	Unwanted energy degrading a desired link
SISO / MIMO	Single-antenna baseline vs multiple-antenna spatial techniques (survey)
Beamforming	Preferential spatial focusing of energy or sensitivity (survey)
Radio condition	Time-varying wireless channel state affecting experience
Stability contract	Concurrent conditions that keep the wireless experience alive

Related earlier chapters: packets/path (CH16), Wi-Fi/cellular access (CH17), signals adjacency (CH05). Related later chapters: NTN continuity (CH19), latency/reliability/QoE (CH20).

18.14 Figure references (embedded; registered SVG + a11y)

All four figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured evidence. No fabricated RF telemetry. No invented gain plots.

18.14.1 FIG-CH18-001 — Device—obstacle—AP path loss cartoon

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Conceptual system map.
- **Reader should notice.** Experience-first path from person/device through obstacles to AP/cell, with spectrum sharedness visible.
- **Truth class.** Conceptual / Representative educational architecture.
- **Alt text requirement.** List nodes in reading order; state conceptual truth class; deny calibrated path-loss numbers.

18.14.2 FIG-CH18-002 — Omni vs beam pattern intuition

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Conceptual comparison.
- **Reader should notice.** Broad coverage vs preferential direction—qualitative only.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name both patterns; forbid invented dBi scales as product truth.

18.14.3 FIG-CH18-003 — MIMO spatial streams metaphor

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Illustrative metaphor.
- **Reader should notice.** Multiple spatial lanes as teaching image—not a chipset claim.
- **Truth class.** Illustrative.
- **Alt text requirement.** Label metaphor explicitly; state illustrative truth class.

18.14.4 FIG-CH18-004 — Quartet antenna placement with PHYSICAL_PENDING overlay

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Project-specific teaching overlay.
- **Reader should notice.** Research form-factor discussion without measured RF evidence.
- **Truth class.** Project-specific; **PHYSICAL_PENDING**.
- **Alt text requirement.** State PHYSICAL_PENDING explicitly; deny drive-test numbers.

18.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH18-001.** Radio links depend on spectrum, antennas, and time-varying conditions—not only on “Wi-Fi on” (IEEE 2020; Kurose and Ross 2020).
- **CLM-CH18-002.** MIMO/beamforming are spatial techniques; no invented device-specific dB gains (IEEE 2020).
- **CLM-CH18-003.** Interference and blockage can degrade experience while icons remain lit (IEEE 2020; Kurose and Ross 2020).
- **CLM-CH18-004.** Labs forbid unauthorized transmission and unlawful sniffing (Gunn 2026a).
- **CLM-CH18-005.** Quartet / research RF measurements remain **PHYSICAL_PENDING**; WAIKE FSPL adjacency is not publication RF proof.

Chapter 19

Chapter 19 — NTN and Service Continuity Across Ground, Air, and Space

Status: draft · **Chapter ID:** CH19

Author: Edmund Gunn, Jr.

Manuscript: working full-manuscript draft · human validation pending · not publication-ready

19.1 1. The moment

You are somewhere the usual story breaks.

On a flight, the cabin mode flips and the familiar cellular bars go quiet. In a rural stretch, the map still draws roads while the phone reports no usable service. During an outage, a neighbor’s device still shows “connected” somewhere—but your message will not send. Then a satellite-messaging feature, an emergency-text path, or a marketing banner about space Internet appears. From the seat it feels like a simple question: *do I still have service?*

Underneath, the question is not one switch. It is a path-class change. A **terrestrial network**—ground-based access and core infrastructure—may be unavailable, degraded, or forbidden by policy (for example, airplane radio modes). A **non-terrestrial network (NTN)** path—space or airborne platforms such as satellites or high-altitude systems—may offer a different capability class entirely: sometimes short messages, sometimes constrained data, sometimes broadband-class service in specific products. Marketing language can blur those classes into one glowing icon (3GPP 2026) (CLM-CH19-001; CLM-CH19-004 · **SOURCE_NEEDED** for operator capability-class documents).

This chapter’s governing question:

When the ground path fails or changes, what actually continues for a human—and how do we refuse to confuse an icon with experience continuity across ground, air, and space?

Part IV already separated Wi-Fi, cellular, packets, and cloud placement. Concept Edition continuity language named the problem without dumping every NTN detail into CE-4/CE-6. Here we deepen **service continuity** as experience continuity across path classes—not as an always-on badge—and we stay humble about deployed capabilities and project evidence.

19.2 2. What you notice

Before vocabulary like *LEO*, *GEO*, or *multi-connectivity*, notice the human contracts you already expect.

You expect “connected” to mean the task you care about can finish—or fail with an honest reason. You expect a messaging-only satellite feature not to behave like home broadband. You expect handovers across networks not to silently drop the draft you were writing. You expect rural and flight stories to be teachable without requiring travel or unauthorized radio experiments. You expect marketing screenshots not to substitute for official capability descriptions.

Those expectations are not decorations. They *are* continuity, from the person’s point of view.

Service continuity is about experience across changes—not merely keeping an icon lit (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.; “Reference Guide to Quality of Experience Assessment Methodologies” 2016; 3GPP 2026) (CLM-CH19-002).

Notice the split timelines. A radio indicator can remain green while an application stalls. A satellite feature can accept a short text while blocking a video call. A coverage map can look complete at city scale while a valley remains a **coverage gap**—a region or time without usable terrestrial service. Optional comparison on devices you already own (no flights required): open a familiar messaging or sync action on Wi-Fi, then switch to cellular or airplane mode and watch how status text, send state, and human-visible progress diverge. Record observations separately from causal guesses. You are practicing continuity literacy, not collecting operator latency numbers.

19.3 3. Exploded ecosystem

A continuity moment is not a single radio. It is a path through cooperating layers. **FIG-CH19-001** is the first-minute map: human task → device → terrestrial and/or NTN path classes → core/service → human-visible outcome. Treat it as **Representative educational architecture**, not a claim that any one operator’s sky looks exactly like the diagram.

19.3.1 Human

You form intent: send a status to family, keep a document syncing, reach emergency services, finish a class upload. Eyes and ears judge whether the experience continued.

19.3.2 Device and local stack

Radios, modem firmware, OS connectivity managers, and apps interpret path availability. Icons summarize; they do not measure your task.

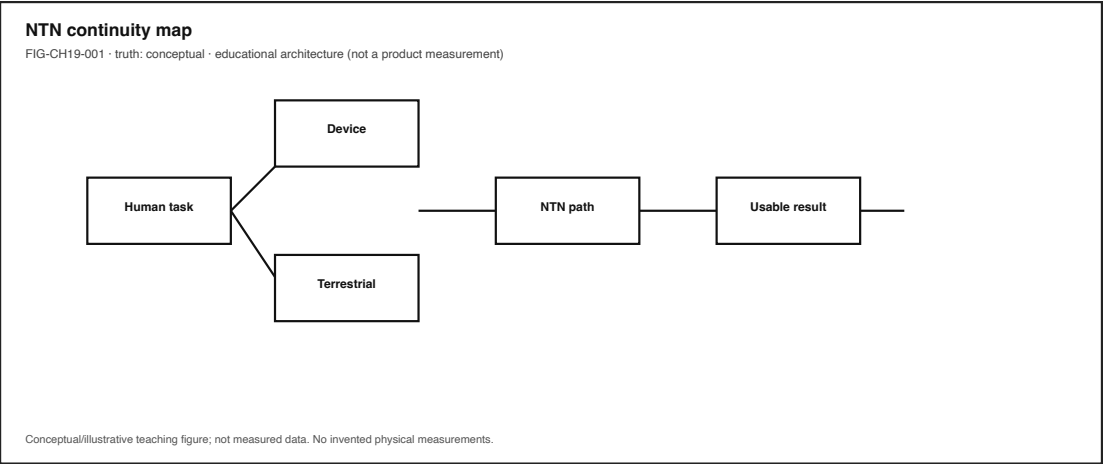


Figure 19.1: Ground / air / space path classes to a device. Conceptual educational architecture.

19.3.3 Terrestrial path class

Towers, small cells, backhaul, and mobile core functions implement ground access. Wi-Fi may provide a local on-ramp; cellular may provide a mobile on-ramp; neither is “the Internet,” and neither is automatically identical to an NTN path (3GPP 2026) (CLM-CH19-001).

19.3.4 Air / space path class (NTN)

Satellites, high-altitude platform systems (HAPS), and related non-ground platforms can extend reach. NTN adds path classes; it does not automatically deliver the same delay, bandwidth, or application behavior as terrestrial 5G service (3GPP 2026) (CLM-CH19-001). Exact NTN specification clause IDs beyond the 5G system-architecture frame remain a sourcing follow-up (see claim footnotes).

19.3.5 Core, edge, and service placement

Even when a radio path exists, the service you need may live nearby (edge) or far (cloud). Continuity can fail in the radio, the transport path, or the service—CE-6’s connected-but-unusable lesson still applies.

19.3.6 Operator policy and capability class

What a product *may* do (messaging-only, emergency text, constrained data, broadband-class) is a capability claim. Verify it from official operator or standards documentation before teaching it as fact (CLM-CH19-004 · **SOURCE_NEEDED**). Do not invent product feature matrices in this chapter.

19.3.7 Evidence boundary (project)

WAIKE accepted **main** hosts a synthetic polar NTN teaching case study under a Graham Land / 7GC path. That fixture is for local teaching validation only. It is **not** field-validated twin evidence. Project NTN demo/twin claims remain **PHYSICAL_PENDING** (CLM-CH19-005) (gunnchOS3k 2026).

19.4 4. Follow the signal

Follow one human-visible action across a path change—without inventing sky telemetry.

1. **Intent forms.** You tap send, sync, or join.
2. **Local stack checks path class.** The device reports terrestrial availability, NTN/satellite feature availability, or neither.
3. **Capability gate.** The active path either supports the required class (for example short message vs bulk upload) or refuses.
4. **Delay and reliability regime.** Propagation and system delays differ by path class. Orbit *classes* (for example low Earth orbit vs geostationary) imply different qualitative delay regimes; **do not invent product latency numbers** here (CLM-CH19-003 · **SOURCE_IDENTIFIED** via (2023b; Kurose and Ross 2020)). Networking survey texts discuss satellite-link delay as a path-class property distinct from terrestrial access—useful orientation only, not an operator guarantee (Kurose and Ross 2020).
5. **Handover / multi-path behavior.** The stack may stay on one path, fail over, or use more than one path (**multi-path continuity** / multi-connectivity ideas at survey depth).
6. **Human-visible state.** Progress, errors, and drafts either survive or silently break—**FIG-CH19-002** contrasts continuity of experience vs icon-lit status.

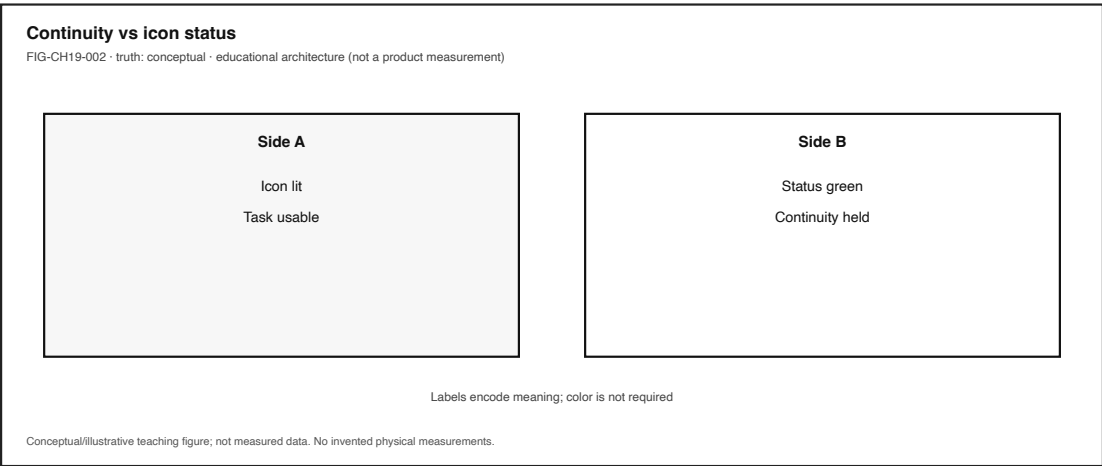


Figure 19.2: Service continuity vs icon-lit status. Conceptual; color not sole encoding.

19.4.1 Alternate paths (the honesty rule)

If you cannot observe an NTN radio, you can still practice the continuity method on terrestrial path changes (Wi-Fi cellular offline) using commodity devices or fixtures. That is adjacency to **LAB-PKT-001** and **LAB-CE06-001**, not a fake satellite measurement.

19.4.2 Failure branch without drama

Common honest failures: no usable path for the required capability; path exists but delay/reliability outside task tolerance; handover drops application state; marketing implied broadband while the official class is messaging-only. None of these require inventing orbit milliseconds.

19.5 5. Component cards

19.5.1 Non-terrestrial network (NTN)

- **Role.** Network components using space or airborne platforms.
- **Plain contract.** Extends path options beyond ground infrastructure.
- **Misread.** “NTN” means automatic full Internet everywhere.
- **Evidence note.** Architecture vocabulary via 5G system references (3GPP 2026); product claims need official docs (CLM-CH19-001; CLM-CH19-004 · SOURCE_NEEDED).

19.5.2 Terrestrial network

- **Role.** Ground-based access and core infrastructure.
- **Plain contract.** The everyday cellular/Wi-Fi-adjacent world of towers, backhaul, and local wireless.
- **Misread.** Treating terrestrial and NTN as interchangeable synonyms for “bars.”

19.5.3 Service continuity

- **Role.** Keeping the human experience working across path/access changes.
- **Plain contract.** Experience continuity icon continuity (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.; “Reference Guide to Quality of Experience Assessment Methodologies” 2016) (CLM-CH19-002).
- **Misread.** A lit satellite glyph proves the user’s task still works.

19.5.4 Coverage gap

- **Role.** Space/time without usable terrestrial service.
- **Plain contract.** Gaps motivate alternate path classes; they do not specify which capability arrives.
- **Misread.** A map without hatching means no human has ever lost service there.

19.5.5 Delay regime

- **Role.** Propagation and system delay classes that differ sharply across terrestrial vs orbit classes.
- **Plain contract.** Compare regimes qualitatively; refuse invented product latency tables (CLM-CH19-003 · SOURCE_IDENTIFIED).
- **Misread.** One “satellite latency” number fits all orbits and operators.
- **Figure.** **FIG-CH19-003** may show comparative bars only when labeled illustrative—never as measured product data.

19.5.6 Multi-path continuity

- **Role.** Using more than one access path to sustain experience.
 - **Plain contract.** Extra paths help only if capability, delay, and state survival match the task.
 - **Misread.** More radios automatically mean seamless broadband.
-

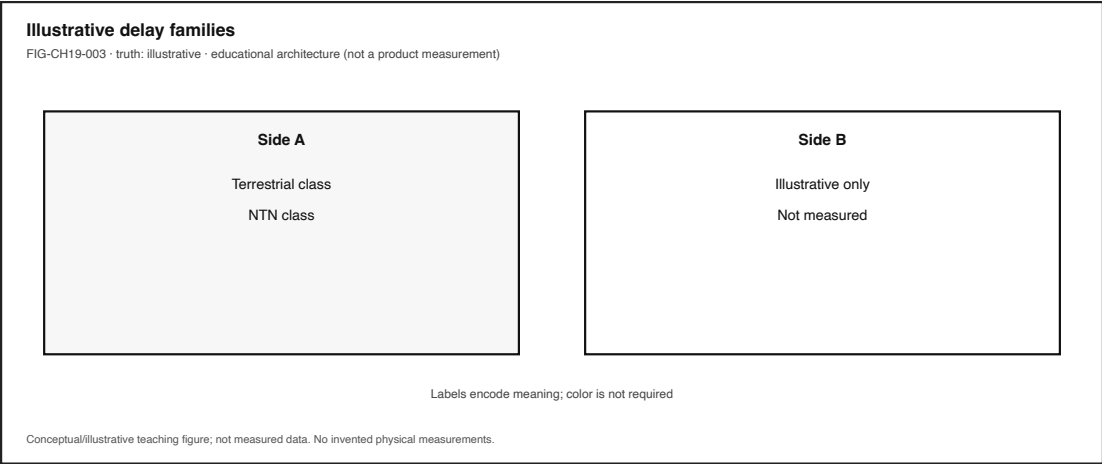


Figure 19.3: Delay-regime comparison. Illustrative bars only—not operator measurements.

19.6 6. Stability contract

Definition retained from the book: a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

Chapter-focused concurrent conditions (qualitative; no invented numeric budgets):

1. **At least one usable path class** is available for the *required* capability (messaging vs broadband-class, and so on).
2. **Delay and reliability** of the active path stay within the task’s tolerance—judged by human-visible completion, not by icon color alone (“Reference Guide to Quality of Experience Assessment Methodologies” 2016).
3. **Handover across domains** does not silently drop human-visible state (drafts, sessions, progress).
4. **Capability class matches expectation**—marketing language has been checked against official descriptions (CLM-CH19-004 · SOURCE_NEEDED).

A system can remain *icon-connected* while the human experience has already failed: send pending forever, sync looping, call setup impossible, emergency text succeeding while apps stall. Conversely, a constrained NTN path can preserve a narrow life-critical or messaging experience while “full Internet” remains unavailable. Stability is concurrent conditions—not a single sky emoji.

Honesty bound for this edition: Do not invent orbit delay numbers or operator capability matrices. Project NTN twin/demo evidence remains **PHYSICAL_PENDING** (CLM-CH19-005) (gunnchOS3k 2026). Commodity continuity labs produce *your* observations on *your* devices or fixtures—not a satellite field trial.

19.7 7. Try it

19.7.1 Continuity inheritance — LAB-PKT-001 and LAB-CE06-001

Goal. Practice continuity literacy on path and usability changes you can ethically observe—then extend the same worksheet logic toward public NTN/satellite *feature classification* using official documents only.

WAIKE alignment note. WAIKE accepted **main** maps advanced wireless coursework adjacently (**WIRELESS_6G**); it does **not** provide an exact NTN lab ID. Do not invent one. Publication continuity work inherits **LAB-PKT-001** (path/access framing) and **LAB-CE06-001** (connected-but-unusable diagnosis). The NTN document-only capability-class route below is an **INLINE_ACTIVITY** (namespaced idea LAB-CONT-001 is **not** a shipped **labs/** package and must not be minted as a WAIKE course ID).

Safety (hard stops).

- No unauthorized satellite ground-station activity, no unlicensed transmitters, no aircraft-system interference.
- No Device Quartet or specialized RF TX required.
- No travel required; rural/flight stories may use fixtures, public docs, and terrestrial path-change analogs.
- Redact secrets, tokens, precise locations, and classmate PII from portfolio artifacts.

Routes.

- **Commodity route (terrestrial continuity analog).** Reproduce a connected-but-unusable or path-change moment on a phone/laptop you already own; follow LAB-CE06-001 / LAB-PKT-001 observation rules.
- **Fixture route.** Use lab fixtures when metered data, travel, or live networks are unavailable.
- **Document-only capability-class route (NTN-facing, no flight).** Pick one public consumer satellite/NTN feature. Using **official operator or standards documentation only**, classify: messaging-only, emergency-limited, constrained data, broadband-class, or “insufficient official evidence.” Record quotes/links; do not invent capabilities (CLM-CH19-004 · SOURCE_NEEDED until docs are pinned per feature).

Explorer baseline (about 45–60 minutes).

1. Predict whether your chosen task fails for path absence, delay/reliability, service placement, or capability-class mismatch.
2. Run one terrestrial continuity observation (LAB-PKT-001 or LAB-CE06-001 route) **or** complete the document-only capability-class card.
3. Fill observation-vs-inference columns. Icon state is an observation; “the satellite is 40 ms away” is an inference you are **not** allowed to invent (CLM-CH19-003 · SOURCE_IDENTIFIED; qualitative only).
4. Write a five-sentence teach-back: NTN is an additional path class; continuity is experience; icons are not proof.

Operator extension. Compare two official feature descriptions (two products or two modes). Note where marketing pages and support docs disagree; prefer the more specific official capability statement.

Builder extension. Produce a one-page continuity checklist: path class → capability class → human-visible state survivors → evidence still needed.

Engineer extension. Sketch qualitative delay-regime comparison (terrestrial vs LEO-class vs GEO-class) with **no numeric product claims**; cite NTN/orbit-class sources without inventing product milliseconds (CLM-CH19-003 · SOURCE_IDENTIFIED).

Researcher extension. Read the WAIKE synthetic polar NTN case-study boundary: teaching fixture, local validation needed, **PHYSICAL_PENDING** for field twin claims (CLM-CH19-005) (gunnchOS3k 2026). Write what evidence would be required to change that label—and what still would not be proven.

Evidence to keep. Observation-vs-inference table; capability-class card with official-doc citations; scrubbed notes; teach-back paragraph.

19.8 8. Build it

Extend the Try-it routes without turning Part IV into a satellite product catalog.

19.8.1 Explorer

Build a pocket card: four lines—(1) terrestrial path, (2) NTN path, (3) icon experience, (4) capability class must be verified.

19.8.2 Operator

Build a “feature claim” checklist that starts with official docs and ends with “insufficient evidence”—never with a guessed broadband promise.

19.8.3 Builder

Build a labeled continuity diagram for one task: human → device → path class(es) → service → outcome. Annotate what is observed vs inferred. Optional second diagram: messaging-only vs broadband-class as different contracts on the same sky metaphor.

19.8.4 Engineer

Build a qualitative delay-regime brief: define LEO-class vs GEO-class as orbit *regimes* affecting propagation scale, cite (2023b; Kurose and Ross 2020), and leave product milliseconds blank (CLM-CH19-003). Pair with 5G system-architecture literacy (3GPP 2026) without collapsing NTN into terrestrial 5G.

19.8.5 Researcher

Build an evidence plan for promoting CLM-CH19-005: what physical/field measurements, ethics approvals, and replication packages would be required beyond a synthetic teaching fixture (gunnchOS3k 2026). Explicitly forbid treating simulation screenshots as twin validation.

Educators can facilitate Section 11 teach-backs and keep classrooms on fixture/document routes when travel or RF gear is unavailable.

19.9 9. Secure and include it

19.9.1 Security

NTN and satellite features expand the trust boundary: new radios, new intermediaries, new failure modes for spoofed status. Prefer official provisioning and user consent. Do not run unauthorized ground-station or uplink experiments as “labs.”

19.9.2 Privacy

Location, flight paths, and message metadata can become sensitive identifiers. Portfolio artifacts must scrub precise coordinates, account identifiers, and private message content.

19.9.3 Accessibility

Continuity status must not depend on color-only icons. Provide text equivalents for path/capability state. Document checklists in words so screen-reader users and print users can follow the same method (W3C 2024).

19.9.4 Equity

Rural gaps, flight constraints, and device/plan pricing shape who receives which capability class. Teach continuity without requiring students to buy satellite plans or travel to polar field sites. Fixtures and official docs are first-class.

19.9.5 Safety

No interference with aviation systems, no unlicensed transmission, no improvised RF hardware. Continuity literacy is observational and documentary in this chapter.

19.9.6 Ethics

Do not invent operator capabilities, orbit delay tables, or field-validated NTN twin results. Overclaiming sky coverage is still a form of false evidence. Keep PHYSICAL_PENDING and SOURCE_NEEDED labels visible where they belong.

19.10 10. Career lens

Roles that touch this chapter’s problems (publication career registry IDs where they exist; otherwise descriptive). **No employment guarantees.**

Layer / concern	Role lens	Owns (example)	Classroom analogue
Radio / NTN path literacy	Wireless engineer (ROLE-WIRELESS)	Radio-performance analysis with honest limits	Weak-signal / path-class discussion; no fake drive tests
Path & timing diagnosis	Network engineer (ROLE-NET)	Packet/latency analysis on observable paths	LAB-PKT-001 timing table on commodity/fixture routes

Layer / concern	Role lens	Owns (example)	Classroom analogue
Experience under change	SRE (ROLE-SRE)	Service reliability evidence; connected vs usable	LAB-CE06-001 continuity diagnosis
Mobile core / architecture	Mobile core engineer (descriptive)	Core functions that survive access changes	Architecture teach-back from TS 23.501 survey depth (3GPP 2026)
Space systems	Satellite systems engineer (descriptive)	Orbit/link budgets in real programs	Qualitative regime literacy only here; no invented budgets
Policy / spectrum	Policy & spectrum roles (descriptive)	Rules that constrain who may transmit and where	Document-only capability and regulation reading

Portfolio signal: a scrubbed continuity checklist plus an official-doc capability-class card beats a screenshot of a satellite marketing banner.

19.11 11. Check understanding

1. In one sentence, why is NTN not automatically identical to terrestrial 5G service?
2. What is the difference between an icon staying lit and service continuity?
3. Name two concurrent Stability Contract conditions for a cross-domain continuity moment.
4. Why does this chapter refuse to publish invented LEO/GEO product latency numbers?
5. What evidence class is required before teaching a consumer satellite feature as “broad-band”?
6. What does **PHYSICAL_PENDING** mean for the WAIKE polar NTN case study?
7. Which existing labs inherit continuity practice without inventing an NTN flight lab?
8. Teach-back (Explorer): explain to a nontechnical person why a satellite-messaging feature can be valuable without being home Internet.

Researcher prompt. What primary sources would close CLM-CH19-003 and CLM-CH19-004, and what still could not be claimed without field measurements?

19.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`), including 3GPP TS 23.501 family architecture vocabulary (3GPP 2026) and ITU-T QoE vocabulary/assessment guidance (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.; “Reference Guide to Quality of Experience Assessment Methodologies” 2016). Project-specific NTN twin status remains **PHYSICAL_PENDING** and is cited via (gunnchOS3k 2026), separately from external literature. Orbit-delay product numbers and operator capability-class documents remain **SOURCE_NEEDED** where marked.

19.13 12. Glossary links

Term	Plain link
Non-terrestrial network (NTN)	Network components using space or airborne platforms
Terrestrial network	Ground-based access/core infrastructure
Service continuity	Keeping usable experience across path/access changes
Coverage gap	Absence of usable terrestrial service in space/time
Delay regime	Qualitative delay class tied to path/orbit regime
Multi-path continuity	Using more than one access path to sustain experience
LEO / GEO (qualitative)	Low Earth orbit vs geostationary orbit <i>regimes</i> —not product latency figures
Stability contract	Concurrent conditions that keep the experience alive

Related earlier chapters: packets/Internet path literacy (CH16), Wi-Fi/cellular/5G survey (CH17), spectrum/radio conditions (CH18), CE-4/CE-6 continuity. Related later chapters: sensing and twin honesty (CH20+), publication/operations ethics (Part VI).

19.14 Figure references (embedded; registered SVG + a11y)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured evidence. No fabricated satellite telemetry.

19.14.1 FIG-CH19-001 — Ground / air / space path classes to a device

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** System map.
- **Reader should notice.** Distinct terrestrial vs NTN path classes feeding one human task.
- **Truth class.** Conceptual / Representative educational architecture.
- **Alt text requirement.** Name human, device, terrestrial path, NTN path, service, outcome; state conceptual truth class.

19.14.2 FIG-CH19-002 — Continuity vs icon-lit

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Comparative layers.
- **Reader should notice.** Icon can remain lit while human-visible task fails (or narrow messaging succeeds).
- **Truth class.** Conceptual.
- **Alt text requirement.** Contrast icon state vs task outcome; color not sole encoding.

19.14.3 FIG-CH19-003 — Delay regime comparison (illustrative)

- **Production status.** `embedded` (registered SVG + accessibility sidecar).
- **Type.** Illustrative comparative bars.
- **Reader should notice.** Regime classes differ; bars are not product measurements.
- **Truth class.** Illustrative.
- **Alt text requirement.** State illustrative only; forbid reading numbers as operator guarantees; note CLM-CH19-003 `SOURCE_IDENTIFIED` qualitative-only.

19.15 Claim footnotes used in this chapter

Claim ID	Text (short)	Status
CLM-CH19-001	NTN adds non-ground path classes; not automatically identical to terrestrial 5G	<code>SOURCE_IDENTIFIED</code> (<code>threegpp-ts23501</code>)
CLM-CH19-002	Service continuity is experience across changes—not merely an icon	<code>SOURCE_IDENTIFIED</code> (<code>itu-t-p10-g100</code> , <code>itu-t-g1011</code> , <code>threegpp-ts23501</code>)
CLM-CH19-003	Satellite delay regimes differ by orbit class; do not invent product latency numbers	<code>SOURCE_IDENTIFIED</code> (<code>threegpp-tr38821</code> , <code>kurose-ross-8</code>); qualitative only
CLM-CH19-004	Marketing satellite connectivity may mean messaging-only or limited modes—verify capability class	<code>SOURCE_NEEDED</code> (official operator docs per feature)
CLM-CH19-005	Project NTN twin/demo remains <code>PHYSICAL_PENDING</code> ; WAIKE polar NTN case study is synthetic teaching fixture only	<code>PHYSICAL_PENDING</code> (<code>src-waike @</code> <code>e97e74fc9bfb44b1cdc26b272dc4848264f</code>)

Chapter 20

Chapter 20 — Latency, Reliability, QoE, and the Stability Contract

Status: draft · **Chapter ID:** CH20

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS; EMIT fixtures and illustrative portfolios are teaching infrastructure, not human reader evidence).

Part IV has already named paths, packets, radios, and services. This chapter closes the connect-everything arc by refusing a single-metric story. It inherits the Concept Edition CE-6 Stability Contract package and the publication-owned capstone **LAB-CE06-001**, then expands them for full-book depth: latency, reliability, throughput, QoE, QoS, and the discipline of evidence.

20.1 1. The moment

Everything looks connected. The icon says online. The Wi-Fi name is familiar. Cellular bars are present. You tap send, submit, refresh, or sync—and the experience stalls, flickers, retries, or never finishes. Sometimes a toast claims success while the remote effect never arrives. Sometimes ping looks fine and the app still feels awful.

From the seat: contradiction.

Underneath: concurrent hidden conditions. A **connectivity indicator** answers a narrow question—“is some association or reachability path believed present?” It does not answer whether *this person’s* intended action completed with acceptable delay, correctness, and feedback. **Quality of Experience (QoE)** is the human-facing question. **Quality of Service (QoS)** is related service-characteristic language. **Ping** is one probe among many—not a full contract (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.).

The governing question for this chapter:

When everything looks connected but the experience fails, which concurrent conditions broke—and what evidence do I actually have?

This is Part IV’s synthesis close and the full-book expansion of CE-6. It is not a second one-tap lecture, not a carrier-grade MOS campaign, and not a table of invented product SLOs.

20.2 2. What you notice

Before naming latency budgets or reliability mathematics, notice the human contract that broke.

You expected a familiar action to finish. Instead you notice a persistent spinner, a partial local update without remote confirmation, a reconnect banner that never resolves, choppy media while “connected” stays green, or an assistive path that never announces completion. A classmate on a different device or network may finish the same action while you do not. That divergence is part of the technical story—not a character flaw in the user.

Connected indicators can remain green while the human experience has already failed.

That distinction is the chapter’s first systems skill (CLM-CH20-004; CE-6 signature). Status chrome is an observation about *status chrome*. Usable completion is a different observation. Collapsing them produces confident wrong blame: “the Wi-Fi,” “the cloud,” “the phone,” as if those were synonyms for one failure.

Optional commodity notice (no specialized gear): attempt one familiar send/submit/sync on a device you already own. Write two columns—*status shown* and *action outcome for a human*—before you invent a cause. If live reproduction is unsafe, offline, or metered, use the LAB-CE06-001 fixture route and mark rows **fixture**. Fixture timings are illustrative teaching data, not your measured evidence and not Gate validation.

20.3 3. Exploded ecosystem

A stalled send is not a single object. It is a path through an ecosystem. **FIG-CH20-001** is the first-minute map: human experience at the center, with concurrent spokes—not a single ordered villain chain. Treat it as **Representative educational architecture**, not a claim that every app fails the same way.

Walk the layers in ordinary language.

20.3.1 Human and context

Intent, expectation, attention, and situation set what “acceptable” means *for this person now*. A classroom quiz submit and a casual chat send do not share one universal delay tolerance. Acceptable bounds are context-dependent; inventing universal millisecond tables as gunnchOS truth is forbidden (CLM-CH20-005).

20.3.2 Input and interaction path

Taps, keys, voice, or assistive technology must be recognized and delivered into the software path. SC-01 in the CE-6 contract list. A silent AT path can fail the experience even when

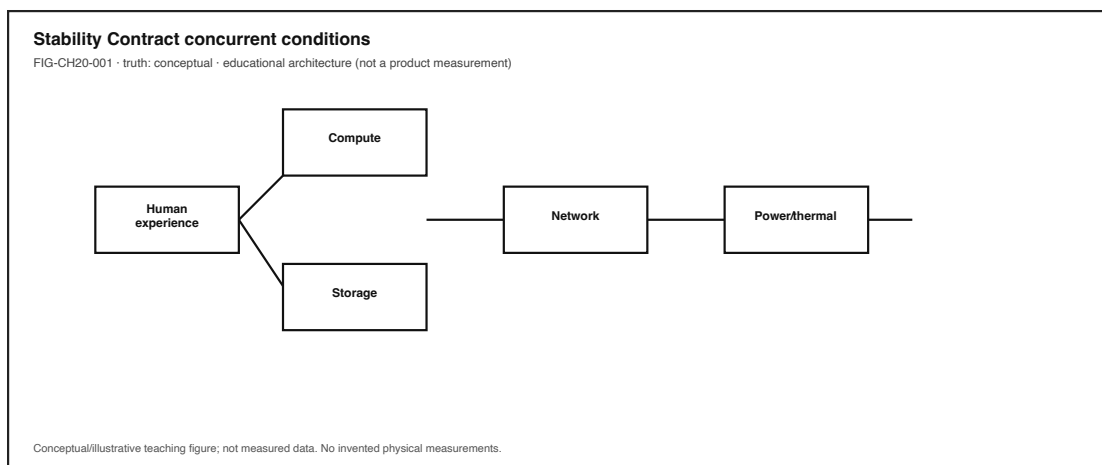


Figure 20.1: Stability Contract concurrent conditions around human experience. Conceptual hub-and-spoke.

sighted UI chrome looks busy (W3C 2024).

20.3.3 Application / code / event loop

Handlers must be scheduled and must not hang. SC-02. Browser and app event-loop behavior matters when the UI thread is blocked (WHATWG 2026b). Local code can stall while the radio still shows associated.

20.3.4 Memory, storage, and local state

Working memory and storage responsiveness condition saves, attachments, and caches. SC-03 / SC-04. Hitch after a large attachment can be storage pressure without any network villain (Patterson and Hennessy 2020b).

20.3.5 Network path (if remote work is required)

Link association, DNS, routing, transport, and retries. SC-05. Part IV already separated Wi-Fi, cellular, Internet path, and cloud placement—do not collapse them into “the network” as one synonym. A usable local association does not entail a usable end-to-end path.

20.3.6 Remote service / authorization

Service health, sessions, scopes, and API contracts. SC-06. HTTP 5xx, 401, or “success UI without durable effect” are service-domain symptoms—not proof that the phone radio failed.

20.3.7 Render / display feedback

Coherent UI feedback must reach the human. SC-07. Work can finish on a server while frames arrive late, or a spinner can outlive the request.

20.3.8 Power / thermal policy

Battery and thermal modes can collapse performance mid-experience. SC-08. Chapter 9’s budget lesson returns as a contract condition (The Linux Kernel Documentation 2026a).

20.3.9 Trust / permissions

Identity, scopes, and secure storage must allow the action. SC-09. Permission denial is not “Wi-Fi.”

20.3.10 Accessibility / alternate path

Equivalent feedback along assistive or non-pointer paths. SC-10. Accessibility is a Stability Contract condition, not a garnish (W3C 2024).

20.3.11 Total delay and variability

SC-11: delay and variability remain acceptable *to this person in this context*. Total delay is an aggregate of the above—not a license to invent product SLOs.

FIG-CH20-002 separates latency vs reliability vs throughput symptom families so readers stop treating one green probe as all three.

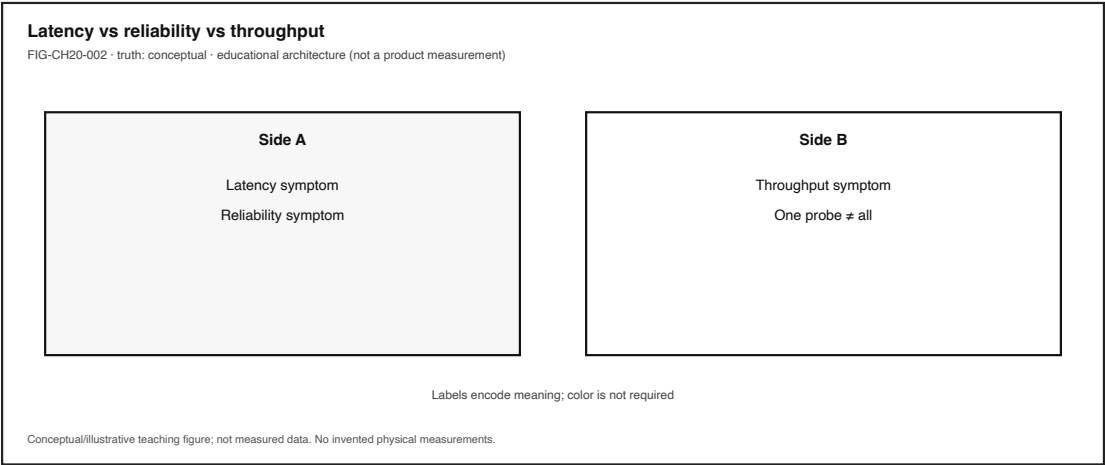


Figure 20.2: Latency vs reliability vs throughput symptom families. Conceptual; do not collapse into one probe.

20.4 4. Follow the signal

Here the “signal” is the human action’s fate across layers—not a single ICMP echo. Read the sequence as a logical diagnosis story. Alternate paths exist; open questions stay labeled undetermined.

1. **Intent.** A person chooses send / submit / sync / stream.
2. **Input delivery.** The OS and app receive the intent (or do not).
3. **Local work.** Scheduling, memory, storage, and UI update begin.

- 4. **Optional remote request.** If required, a network transaction starts—DNS, TLS, transport, retries.
- 5. **Service decision.** Authorize, accept, reject, timeout, or partially apply.
- 6. **Return path.** Response, error, or silence travels back.
- 7. **Render / announce.** Visual, haptic, audio, or AT feedback reaches (or fails to reach) the human.
- 8. **Human judgment.** Finished, stuck, wrong, or “works for them / not for me.”

20.4.1 Metric families without collapse

Family	Everyday question	Typical mis-use
Latency	How long until a useful response?	Treating one RTT sample as the whole experience
Reliability	Does the action complete correctly often enough over time?	Calling a single success “reliable”
Throughput	How much useful data per time?	Assuming high throughput cures latency or correctness
QoS language	What service characteristics are in play?	Treating KPI compliance as guaranteed delight
QoE	How delightful or annoying is the experience for the user?	Equating ping with QoE (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.)
Ping / probe	What did <i>this</i> probe return?	Promoting one probe into a Stability Contract

Latency, reliability, and throughput are different failure/success families; collapsing them misleads diagnosis (CLM-CH20-001) (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023; “Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.). Software quality models discuss characteristics such as performance efficiency and reliability as related but distinct lenses—useful vocabulary, not a product certification claim for this book (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023).

20.4.2 Evidence hierarchy (climb only as far as tools and ethics allow)

FIG-CH20-003 and CE-6’s evidence ladder teach the same honesty:

- 1. Illustrative teaching aid (labeled)
- 2. Commodity observation (status UI, wall-clock, visible stalls)
- 3. Browser / OS instrumentation where exposed (for example Performance API timings) (MDN Web Docs 2026b, 2026c)
- 4. Correlated multi-signal inspection (still not proof)
- 5. Controlled comparison under ethical constraints
- 6. Standards-aligned subjective / objective QoE methods (ITU-T G.1011 context)—**not** required for classroom completion (“Reference Guide to Quality of Experience Assessment Methodologies” 2016)

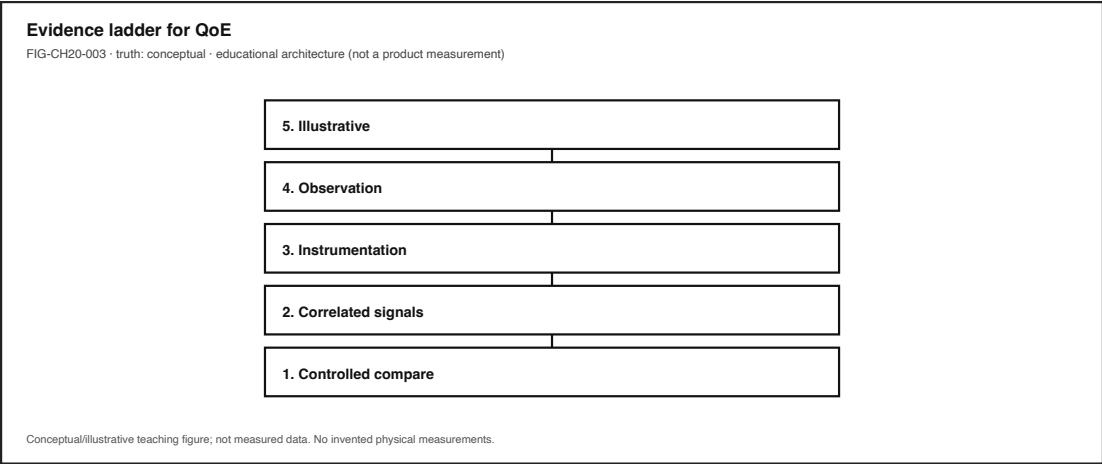


Figure 20.3: QoE vs QoS vs ping. Conceptual non-entailment teaching.

Learner labs sit mid-ladder. Carrier-grade MOS campaigns and invented scores do not belong in Explorer portfolios.

20.4.3 Failure branch without drama

Prefer failure *domains* over confident blame: input, compute/schedule, memory/storage, network path, service/backend, render/display, power/thermal, trust/permissions, equity/access. Outside observation rarely distinguishes them cleanly. That limitation is literacy.

20.5 5. Component cards

For each idea: plain language, analogy (labeled), technical function, constraints, common symptoms.

20.5.1 Latency

- **Plain language.** Delay until a useful result is available—often many segments, not one number.
- **Analogy (labeled).** Like waiting for a reply in a conversation with several people passing the message—total wait is not “the last person’s fault” by default.
- **Technical function.** Names time-to-useful-response across input, compute, path, service, and render segments.
- **Constraints.** Context-dependent acceptability; clock granularity; instrumentation coverage gaps.
- **Symptoms.** Spinner longevity, late first paint of result, stalled progress with partial UI.

20.5.2 Reliability

- **Plain language.** Completing correctly often enough over repeated attempts and time.
- **Analogy (labeled).** Like a bridge that must carry traffic many days—not a single photo of a car that made it once.

- **Technical function.** Correct completion under intended conditions; related to software quality “reliability” language without claiming ISO certification (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023).
- **Constraints.** One lab run reliability study; fixture scripts fleet evidence.
- **Symptoms.** Intermittent success, silent failure, success toast without durable effect.

20.5.3 Throughput

- **Plain language.** Useful data volume per unit time.
- **Analogy (labeled).** Like how many boxes fit through a door per minute—not how long until the first box arrives.
- **Technical function.** Capacity of a path or transfer for bulk usefulness.
- **Constraints.** High throughput can coexist with awful interactive latency or failed reliability.
- **Symptoms.** Slow large uploads/downloads while tiny control messages still feel “stuck” for other reasons.

20.5.4 Quality of Experience (QoE)

- **Plain language.** Human-facing acceptability—delight or annoyance of the application or service experience, in ITU vocabulary terms (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.).
- **Analogy (labeled).** Like whether a journey felt usable—not only whether the odometer moved.
- **Technical function.** Centers the person’s outcome; related to but not identical with QoS.
- **Constraints.** Formal subjective methods are higher on the evidence ladder (“Reference Guide to Quality of Experience Assessment Methodologies” 2016); classroom MOS invention is forbidden.
- **Symptoms.** “Connected but unusable,” peer divergence, abandon/retry behavior.

20.5.5 Quality of Service (QoS)

- **Plain language.** Service-characteristic language about properties that bear on satisfying user needs (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.).
- **Analogy (labeled).** Like published transit schedule metrics—related to rider satisfaction, not identical to it.
- **Technical function.** Gives operators shared vocabulary for path and service characteristics.
- **Constraints.** Meeting a KPI does not entail QoE without evidence (CLM-CH20-002).
- **Symptoms.** Operators citing loss, delay, availability numbers while users still abandon.

20.5.6 Ping / probe

- **Plain language.** One measurement sample from one probing method.
- **Analogy (labeled).** Like tapping a pipe once—useful, incomplete.
- **Technical function.** Cheap reachability or RTT hint when ethically available.
- **Constraints.** Not SC-01...SC-11; not QoE; not reliability.
- **Symptoms.** “Ping is fine” beside a broken submit.

20.5.7 Stability Contract (teaching model)

- **Plain language.** A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.
- **Analogy (labeled).** Like a mobile that stays upright only while many guy-wires stay tensioned—any one wire can drop the experience.
- **Technical function.** Publication teaching model for concurrent conditions (not an ITU term; do not attribute the phrase to ITU) (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.).
- **Constraints.** Qualitative where unmeasured; no fake SLOs (CLM-CH20-003; CLM-CH20-005).
- **Symptoms.** Green chrome + failed human outcome; single-metric overconfidence.

20.6 6. Stability contract

Definition (publication teaching model): a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

Signature distinction: a system can remain technically **connected** while the human experience has already **failed**.

This section is the deepest Stability Contract synthesis in the book’s Part IV close. It inherits CE-6’s formal teaching model and concurrent-condition table, then binds them to latency/reliability/QoE literacy without inventing numeric product budgets.

20.6.1 How to teach and apply it (eight moves)

1. **Name the human experience** — what success looks like for a person.
2. **List concurrent conditions** — SC-01...SC-11 style; not a single villain metric.
3. **Separate status chrome from experience outcomes.**
4. **Assign symptoms to failure domains** with evidence gates.
5. **Label every datum** observed / inferred / illustrative / fixture.
6. **Climb the evidence hierarchy** only as far as tools and ethics allow (“Reference Guide to Quality of Experience Assessment Methodologies” 2016; MDN Web Docs 2026b).
7. **State tradeoffs and who is excluded** when conditions are tuned.
8. **Close with teach-back** — can another person use the model?

20.6.2 Concurrent conditions (qualitative anchor)

For a successful send/submit/sync experience, conditions such as the following must remain *good enough together* (CE-6 inheritance):

ID	Hidden condition
SC-01	Input / intent recognized and delivered
SC-02	Application scheduled; handler not hung
SC-03	Memory / working state available
SC-04	Storage responsive if persistence required
SC-05	Network path usable <i>if</i> remote work is required
SC-06	Remote service available and authorized <i>if</i> required
SC-07	Coherent render / UI feedback reaches the human

ID	Hidden condition
SC-08	Power / thermal state not collapsing performance
SC-09	Trust / permissions allow the action
SC-10	Accessibility / alternate path provides equivalent feedback
SC-11	Total delay and variability acceptable <i>in this context</i>

Numeric humility rule: use qualitative language where no measured threshold exists. Do **not** invent performance budgets, MOS scores, or gunnchOS product SLOs (CLM-CH20-005). Any numeric bands in teaching figures prior to learner measurement are **illustrative** only.

20.6.3 CE-6 + LAB-CE06-001 linkage

- **CE-6 preproduction** (`publication/preproduction/ce-06/`) is the primary inheritance package: Stability Contract model, experience map, claim boundaries, and figure intents.
- **LAB-CE06-001** is the publication-owned EMIT capstone (Explain → Measure → Improve → Teach). Status on accepted infrastructure: **FIXTURE_VALIDATED**. That status means validators, blank portfolio templates, and illustrative fixtures exist and pass structural checks. It does **not** mean Gate 3 PASS. It does **not** mean illustrative example portfolios are human evidence. It does **not** mean fixture timings are product SLOs.

Honesty bound for this edition: Device Quartet QoE benches remain **PHYSICAL_PENDING** (CLM-CH20-006). Commodity observation in LAB-CE06-001 produces *your* evidence for *your* chosen experience—or fixture-labeled teaching practice—not a universal score.

20.7 7. Try it

20.7.1 LAB-CE06-001 — Explain, Measure, Improve, and Teach

Goal. Using the book’s full model, gather ethical commodity evidence to explain a real experience, measure what is actually observable, propose one bounded improvement, and teach the Stability Contract for that experience to someone else.

WAIKE alignment note. WAIKE accepted `main` (SHA `e97e74fc9bfb44b1cdc26b272dc4848264f15f`) includes adjacent competencies such as networking, observability, ethics-ladder evidence language, and SLO-budget *neighbors*. Those are adjacencies. They are **not** renamed as publication lab IDs. There is **no** exact WAIKE module literally named Stability Contract / EMIT / TECHNOLOGY_LANDSCAPE_CAPSTONE. **LAB-CE06-001** is publication-owned.

Safety (hard stops).

- No passwords, tokens, private messages, health data, or classmate PII in portfolios.
- No rooting, jailbreaking, unauthorized scanning, or attacking systems.
- Prefer local demos, benign public endpoints, or supplied fixtures; heed metered-data warnings.
- No Device Quartet / specialized RF / EVT hardware required or requested.

Routes.

- **Route A — Notebook + status UI.** Reproduce a connected-but-unusable send/submit/sync (or allowed substitute) once on a commodity device. Record OS connectivity status *separately* from whether the action finished for a human.
- **Route B — Local stall demo (optional).** Open `labs/LAB-CE06-001/browser/index.html`; compare local UI updates vs stalled remote path; label timings observed vs inferred.
- **Route F — Fixture fallback (mandatory offline path).** Use `fixtures/sample_observation.md`, `fixtures/sample_result_table.csv`, and optionally study `fixtures/illustrative_example/` (**ILLUSTRATIVE ONLY — not human evidence**). Mark fixture-derived rows as `fixture`.

Explorer baseline.

1. Predict which failure domain will dominate before measuring.
2. Complete EMIT: Explain → Measure → Improve → Teach.
3. List 4 Stability Contract conditions; mark observed vs guessed.
4. Fill observation vs inference; no causal claim without naming extra evidence needed.
5. Produce teach-back a peer or family member could use.

Operator extension. Add 3 inspection artifacts and one comparison (for example local-only vs remote, or two network classes you already have). Keep LAB-PKT-001 adjacency in mind: predict metric family (latency vs reliability vs throughput symptoms) before reading instrumentation.

Builder extension. One reusable checklist or helper plus a tradeoff note (who might be excluded if you “optimize” only for your device).

Engineer extension. Diagnosis tree; two claims placed on the evidence hierarchy; explicit instrumentation limits (Performance API is software timing, not touch-to-photon or RF truth) (MDN Web Docs 2026b).

Researcher extension. Falsifiable hypothesis; 3 confounders; **no invented statistics or MOS**. State what a G.1011-aligned study would require that this lab does not provide (“Reference Guide to Quality of Experience Assessment Methodologies” 2016).

Educator facilitation. Use `rubric.yaml`; keep classrooms on Route F when live capture is inequitable or unsafe.

Evidence to keep. Portfolio field set under `labs/LAB-CE06-001/portfolio/`; result table; scrubbed notes; teach-back. Validator: `validate_portfolio.py`. Completion means artifact-backed claims—not a bare exit code and not the string PASS.

20.8 8. Build it

Extend LAB-CE06-001 without turning Part IV into a fake SLO catalog.

20.8.1 Explorer

Build a pocket card: *status chrome usable outcome*, plus five contract conditions in plain sentences.

20.8.2 Operator

Build a two-column inspection sheet: status observations | experience outcomes. Add a third column only for inferences, each with “evidence still needed.”

20.8.3 Builder

Build a one-page Improve plan: one bounded change, predicted failure-domain effect, and **qualitative** success criteria you could observe. Optional: a tiny local helper that timestamps visible UI state transitions—never a claim of product latency budgets.

20.8.4 Engineer

Build a metric-family decision tree (latency / reliability / throughput / QoE question) that ends in “needs more evidence” more often than in fake certainty. Cite QoS/QoE vocabulary honestly (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.). Optional adjacency: reuse LAB-PKT-001 prediction discipline on the same experience.

20.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to make yet—for example, a MOS score or Quartet QoE bench. Specify what subjective/objective methodology and physical evidence would be required; keep Quartet claims **PHYSICAL_PENDING** (CLM-CH20-006). Peer-reviewed empirical QoE studies remain **SOURCE_NEEDED** for later depth—do not invent DOIs.

Educators can facilitate Section 11 teach-backs and keep Route F as a first-class equitable path—not a lesser path.

20.9 9. Secure and include it

20.9.1 Security

Traces, screenshots, HAR exports, and retry logs can leak tokens, message contents, and identifiers. Prefer fixtures and redaction. Do not capture others’ screens without consent. Unauthorized scanning is out of scope.

20.9.2 Privacy

Portfolio artifacts must scrub account names, emails, message previews, and location breadcrumbs. Metered-link learners should not be forced onto expensive live captures—Route F exists for equity and privacy together.

20.9.3 Accessibility

SC-10 is a contract condition. Document whether completion was announced to assistive technology; do not rely on color-only status. WCAG 2.2 informs teaching intent here—not a claim that this book or any gunnchOS product is certified (W3C 2024). Timing tasks allow extended time; avoid flicker-heavy demos.

20.9.4 Equity

“Works on the author’s laptop on fast Wi-Fi” is not a Stability Contract. Weaker devices, metered cellular, shared computers, offline contexts, and assistive paths change which concurrent conditions hold. Name who is excluded when “optimization” assumes one privileged setup. Device Quartet form factors may appear as optional research analogies only—never required lab hardware (PHYSICAL_PENDING).

20.9.5 Safety

Commodity devices only. No RF transmit experiments, no battery abuse, no jailbreaks.

20.9.6 Ethics

Do not present illustrative fixture numbers as measured human evidence. Do not present `FIXTURE_VALIDATED` as Gate 3 PASS. Do not invent SLOs, MOS scores, or Quartet EVT curves. Overclaiming measurement is still false evidence.

20.10 10. Career lens

One stalled submit crosses many ownership domains. No table promises employment; roles vary by organization. LAB-CE06-001 artifacts resemble early professional evidence in miniature: labeled observations, failure-domain shortlists, and explicit uncertainty.

Role lens (registry IDs where present)	Typical artifacts	Review questions
SRE (<code>ROLE-SRE</code>)	Service reliability evidence; connected usable notes	Did we confuse reachability with completion?
Performance engineer (<code>ROLE-PERF</code>)	Trace/profile; segmented delay hypotheses	Which segments are observed vs inferred?
Network engineer (<code>ROLE-NET</code>)	Path/latency analysis with labeled probes	Is ping being over-promoted?
HCI (<code>ROLE-HCI</code>)	Interaction study notes; expectation vs outcome	What did the person think “done” meant?
Accessibility (<code>ROLE-A11Y</code>)	AT pathway writeup	Was equivalent feedback present?
Frontend / backend (<code>ROLE-FRONTEND</code> / <code>ROLE-BACKEND</code>)	UI vs API evidence split	Local toast vs durable effect?
Educator / facilitator (descriptive)	Rubric scores; misconception prompts	Did learners label fixtures correctly?

Portfolio hint: a scrubbed result table with observation/inference/**fixture** labels is more honest than a vibes-based “the network is bad” claim.

20.11 11. Check understanding

Concept. In one sentence each, distinguish *latency*, *reliability*, and *throughput* so that none swallows the other two.

Concept. In one sentence each, distinguish *QoE*, *QoS*, and *ping*.

System tracing. Trace a familiar send/submit/sync from intent to human feedback in numbered steps. Mark observed vs inferred. Name 4 Stability Contract conditions that had to hold together.

Misconception check. Why can ping be fine while the app feels awful?

Misconception check. Why must this chapter refuse invented universal millisecond budgets and MOS scores even when discussing QoE?

Evidence ethics. What is the difference between LAB-CE06-001 `FIXTURE_VALIDATED` infrastructure and Gate 3 human reader validation? Why are `fixtures/illustrative_example/` portfolios not human evidence?

Teach-it-back. Explain to a newcomer—using only LAB-CE06-001 vocabulary—why a green connected icon is not a Stability Contract.

Researcher prompt. What additional evidence would be required to move a claim from commodity observation toward ITU-T G.1011-aligned assessment—and what remains out of scope for this classroom lab (“Reference Guide to Quality of Experience Assessment Methodologies” 2016)?

20.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). CE-6 chapter-local proposals and the full31 working bibliography informed key promotion of `itu-t-g1011`. Project-specific Device Quartet QoE status remains **PHYSICAL_PENDING** in the chapter claim plan (CLM-CH20-006). Gate 3 reader evidence remains pending; this working draft is not publication-ready.

Inline citations used in this chapter include “Vocabulary for Performance, Quality of Service and Quality of Experience” (n.d.), “Reference Guide to Quality of Experience Assessment Methodologies” (2016), “Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” (2023), MDN Web Docs (2026b), MDN Web Docs (2026c), W3C (2024), WHATWG (2026b), Patterson and Hennessy (2020b), and The Linux Kernel Documentation (2026a).

Primary inheritance (link, prefer over duplication): `publication/preproduction/ce-06/` and `labs/LAB-CE06-001/`.

20.13 12. Glossary links

Candidate terms introduced or reinforced here (see also chapter glossary candidates; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Latency	Delay until a useful result is available (often multi-segment)
Reliability	Correct completion often enough over time
Throughput	Useful data volume per unit time
QoE	Human-facing quality of the experience
QoS	Service-characteristic language related to, not identical with, QoE
Ping / probe	One sample from one probing method—not the full contract
Stability Contract	Concurrent hidden conditions that keep an experience alive (book teaching model)
Connected usable	Status chrome can succeed while human outcomes fail
Evidence hierarchy	Climb from illustrative aids toward stronger methods without faking rungs
EMIT	Explain → Measure → Improve → Teach (LAB-CE06-001 spine)
Observation vs inference	What happened vs what may explain it

Related earlier chapters: experience-first path and observation craft (CH02/CH03 adjacency), power/thermal budgets (CH09), packets and metric families (Part IV / LAB-PKT-001). Related later chapters: equity as contract condition (CH25), evidence practice (CH27), capstone synthesis (CH31).

20.14 Figure references (embedded; registered SVG + a11y)

All four figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured learner or lab evidence. No fabricated telemetry. No product SLO curves.

20.14.1 FIG-CH20-001 — Stability Contract concurrent conditions

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Conceptual / hub-and-spoke (inherit CE-6 FIG-CE06-001 intent).
- **Reader should notice.** Multiple concurrent conditions; any one can break the experience.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name center (human experience) and each spoke; state conceptual truth class; deny numeric product budgets.

20.14.2 FIG-CH20-002 — Latency vs reliability vs throughput

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Comparative layers.
- **Reader should notice.** Three symptom families that must not collapse into one green probe.

- **Truth class.** Conceptual.
- **Alt text requirement.** Name all three families and one everyday question each.

20.14.3 FIG-CH20-003 — QoE vs QoS vs ping

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Conceptual comparative.
- **Reader should notice.** Human-facing QoE service QoS language one probe.
- **Truth class.** Conceptual.
- **Alt text requirement.** State the non-entailment: KPI compliance does not guarantee delight without evidence.

20.14.4 FIG-CH20-004 — Learner portfolio timings labeled n=1

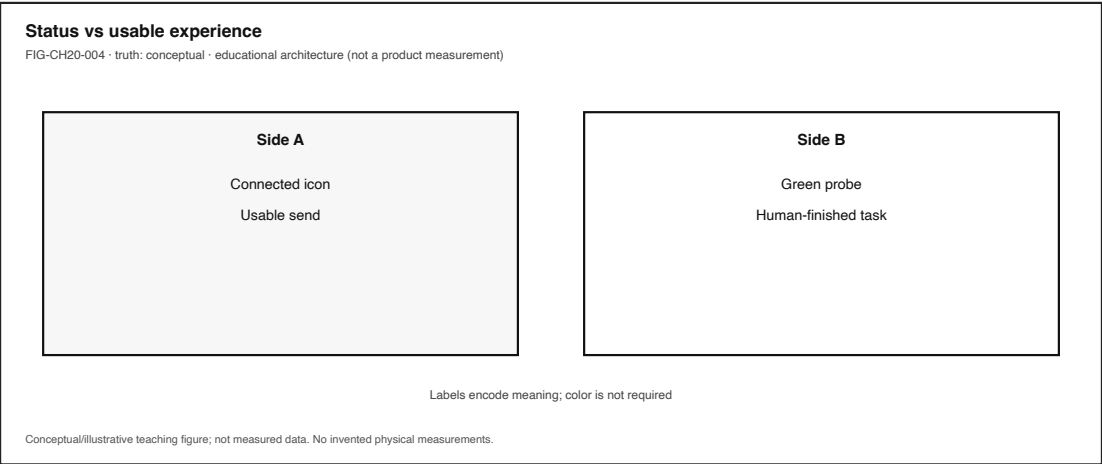


Figure 20.4: Learner portfolio timings labeled n=1. Illustrative unless filled with learner-owned data.

- **Production status.** embedded (registered SVG + accessibility sidecar).
- **Type.** Measured-only when filled with learner data; otherwise illustrative placeholder.
- **Reader should notice.** n=1 classroom evidence; fixture rows labeled **fixture**; not Gate 3 human validation.
- **Truth class.** Measured *only* for learner-owned labeled timings; illustrative otherwise.
- **Alt text requirement.** State sample size / fixture labels; forbid reading illustrative bands as product SLOs.

20.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH20-001.** Latency, reliability, and throughput are different families—framed with ISO/IEC 25010 vocabulary and ITU QoS/QoE terms without certification claims (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023; “Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.).

- **CLM-CH20-002.** QoE is human-facing; QoS is related service language; ping is one probe (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.; “Reference Guide to Quality of Experience Assessment Methodologies” 2016; MDN Web Docs 2026b).
- **CLM-CH20-003.** Stability Contract teaching definition—publication model, not an ITU phrase (“Reference Guide to Quality of Experience Assessment Methodologies” 2016; “Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023).
- **CLM-CH20-004.** Connected indicators can remain green while experience failed—illustrative teaching distinction with CE-6 inheritance.
- **CLM-CH20-005.** No invented universal millisecond budgets or MOS scores as gunnchOS product truth—publication-internal honesty rule.
- **CLM-CH20-006.** Quartet QoE benches **PHYSICAL_PENDING**; learner timings are n=1 classroom evidence when measured and labeled.

Chapter 21

Chapter 21 — Data, Machine Learning, and Generative AI

Status: draft · **Chapter ID:** CH21

Author: Edmund Gunn, Jr.

Manuscript: working full-manuscript draft · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter never claims Gate 3 completion; fixtures are teaching infrastructure, not human reader evidence).

Part V opens the intelligence, security, and responsibility arc. This chapter inherits Concept Edition **CE-5** AI slices—data, model, inference, generative systems, uncertainty, and local-versus-cloud deployment—then expands them for full-book depth. The publication-owned lab **LAB-TRUST-001** is the primary Try/Build surface. Deeper identity, encryption, and privacy-lifecycle chapters follow (CH23–CH24); here the job is to keep AI features readable as systems under a Stability Contract, not as minds.

21.1 1. The moment

You ask a practical question of an on-device or in-browser assistant. The reply arrives quickly. The sentences are smooth. The tone sounds certain—or it hedges, or it refuses, or it changes style after an app update you barely noticed.

From the seat: an answer that feels like conversation.

Underneath: **data** conditioned a **model**; that model’s stored parameters were applied at **inference** to new inputs; an **output** was rendered into the interface you can see or hear. Fluency is a property of the generated surface. Correctness is a separate question. Logging, retention, and whether the path stayed local or crossed a network can change what left the device even when the screen still looks like “just chat” (Goodfellow, Bengio, and Courville 2016) (CLM-CH21-001).

The governing question for this chapter:

When an assistant answer appears, what path produced it—and where do uncertainty, logging, and deployment choice change the experience?

This is not a product review of any named assistant, not a course in training large models from scratch, and not permission to treat fluent text as proof that a system “understands.”

21.2 2. What you notice

Before naming parameters or cloud regions, notice what broke or felt strange in human terms.

You expected a usable answer or a clear refusal. Instead you notice a confident-sounding wrong claim, a hedge that never names what is uncertain, a different tone after an update, a permission or privacy prompt, a spinner that lasts longer than the sentence that finally appears, or an assistive path that never announces the reply. A classmate on another device may get a different boundary: local path on theirs, remote path on yours, or the reverse.

Fluent output can fail the experience even when the interface looks successful.

That distinction is the chapter’s first systems skill (CE-5 signature, carried into CH21). An HTTP 200, a green “online” badge, or a polished paragraph is an observation about *delivery and surface*. Usefulness, honesty about uncertainty, and privacy posture are different observations. Collapsing them produces confident wrong blame: “the AI lied,” “the AI knows me,” or “the cloud is smart,” as if one cartoon agent owned data, model, network, and logging at once.

Optional commodity notice (no specialized gear): ask one non-sensitive practical question of an assistant you already use—or open the LAB-TRUST-001 fixture transcripts if live use is unsafe, offline, or inequitable. Write two columns—*what appeared* and *what you would need in order to trust it for a real decision*—before you invent a cause. Mark fixture rows **fixture**. Fixture text is illustrative teaching data, not your measured product evidence and not Gate validation.

21.3 3. Exploded ecosystem

An assistant answer is not a single object. It is a path through an ecosystem. **FIG-CH21-001** is the first-minute map: human experience at the center, with cooperating layers—not a cartoon brain. Treat it as **representative educational architecture**, not a claim that every product wires the same stack.

Walk the layers in ordinary language.

21.3.1 Human and context

Intent, stakes, language, and situation set what “good enough” means *for this person now*. A homework hint and a medical-adjacent guess do not share one acceptable uncertainty bound. Acceptable bounds are context-dependent; inventing universal accuracy percentages as gunnchOS product truth is forbidden.

21.3.2 Input and interaction path

Keys, voice, images, or assistive technology must be recognized and delivered into the software path. A silent assistive failure can break the experience while sighted chrome looks busy (W3C 2024).

21.3.3 Application / prompt assembly

The app builds a request: user text, UI state, optional retrieved documents, account claims, and policy flags. Code choices here decide what the model ever “sees” as input—still not a mind, still a pipeline.

21.3.4 Model parameters and runtime

Stored parameters shape how inputs map to outputs. Deep-learning teaching vocabulary treats learning as fitting parameters and inference as applying a trained model to new examples (Goodfellow, Bengio, and Courville 2016). Runtime availability, version pinning, and quantization choices (foreshadowed in CH22) condition latency and energy.

21.3.5 Local vs remote placement

Inference may run on-device, on a nearby edge box, or on remote computers. That deployment choice changes privacy exposure, dependency, update control, and often delay. **FIG-CH21-002** keeps those boundaries visible.

21.3.6 Network and service (when remote)

DNS, TLS, API gateways, regional endpoints, and retries. A network stall can look like “the assistant is thinking” even when no inference has started. Part IV’s connected-versus-usable lesson still applies.

21.3.7 Logging, retention, and side effects

Prompts, outputs, and metadata may be retained, shared with vendors, or queued for review. Side effects can outlive the on-screen reply. Disclosure posture is part of the experience, not an appendix.

21.3.8 Render / announce

Text, speech, or UI actions must reach the human—including keyboard and assistive paths. Color-only “confidence” meters fail accessibility teaching rules (W3C 2024).

21.3.9 Society and equity

Who can complete the lab without paid APIs or GPUs? Who is harmed by fluent errors presented as fact? NIST AI RMF 1.0 frames organizational risk management for AI systems without treating “trust” as a binary property of AI itself (Tabassi 2023) (CLM-CH21-002).

21.4 4. Follow the signal

Here the “signal” is the fate of one practical question—from input to human judgment—not a mystical spark of understanding. Read the sequence as a logical path. Alternate product paths exist; open questions stay labeled undetermined.

1. **Intent.** A person asks, dictates, pastes, or uploads.
2. **Input capture.** Sensors, IME, or files deliver bits into the app.

- 3. **Context assembly.** Prompt text, optional retrieval, account/session claims, and UI state combine.
- 4. **Policy gate.** Session valid? Feature allowed? Consent cover this purpose?
- 5. **Inference placement.** Local runtime or remote API applies model parameters to the assembled input (Goodfellow, Bengio, and Courville 2016).
- 6. **Output generation.** Predicted tokens, classifications, rankings, or tool-call proposals return.
- 7. **Side effects.** Logs, analytics, abuse review, or fine-tune queues may record more than the screen shows.
- 8. **Render / announce.** Visual, audio, or assistive feedback reaches (or fails to reach) the human.
- 9. **Human judgment.** Useful, wrong-but-fluent, hedged, refused, or “works for them / not for me.”

21.4.1 Generative systems without mythology

Machine learning is the broader practice of fitting model parameters from data and applying them at inference—classification, ranking, detection, and generation all sit under that umbrella when they learn from examples (Goodfellow, Bengio, and Courville 2016). **Generative AI**, in this book’s systems view, means systems that produce new text, images, code, or similar media by predicting likely continuations under a model and decoding procedure. Generative systems are one family of ML applications, not a synonym for all machine learning. Prediction of likely tokens is not comprehension. The pedagogical firewall is deliberate: speak of inputs, parameters, inference, and outputs—not beliefs or intentions (CLM-CH21-001 wording boundary).

21.4.2 Local versus cloud as a path fork

Path class	Everyday question	Typical mis-read
Local / on-device	Did sensitive content stay on this device?	Treating a UI badge as proof of zero telemetry
Remote / cloud	Which org received the prompt, and under what retention?	Treating TLS as “private forever at every endpoint”
Hybrid	Which stages left the device?	Collapsing retrieval, inference, and logging into one word—“AI”

Illustrative comparison tables for latency, energy, privacy, and control are **teaching aids** unless a measurement bundle is attached (CLM-CH21-005). Do not promote classroom wall-clock samples into Device Quartet product benchmarks (**PHYSICAL_PENDING** when project SKUs are mentioned).

21.4.3 Uncertainty families without collapse

FIG-CH21-003 separates three families readers must not collapse:

Family	Everyday question	Typical mis-use
Fluency	Does the output sound coherent?	Treating polish as truth
Correctness	Is the claim accurate for this task?	One lucky correct answer as “reliable”
Evaluation evidence	What labeled checks support usefulness claims?	Inventing metrics or citing fixtures as human validation

21.5 5. Component cards

Plain-language cards for the objects that cooperate. Each card names constraints and human-visible failure symptoms. None of these components is a person.

21.5.1 Training and input data

What it is. Examples and records used to fit a model, plus the live inputs that enter inference (prompts, files, sensor snippets, retrieved passages).

Constraints. Quality, coverage, consent, and retention bounds. Missing or skewed data produce systematic failures that look like “personality.”

Failure symptoms. Confident wrong answers on underrepresented topics; leakage of sensitive training-like content; retrieval that surfaces the wrong document.

21.5.2 Model parameters

What they are. Stored numbers (and associated architecture/code) that shape how inputs map to outputs (Goodfellow, Bengio, and Courville 2016).

Constraints. Version consistency, license/policy limits, size versus device memory, update authenticity.

Failure symptoms. Sudden style or quality shifts after an update; offline feature when weights are missing; incompatible runtime after an OS upgrade.

21.5.3 Inference runtime / output path

What it is. The engine that applies parameters to new inputs and returns predictions or generations.

Constraints. Latency and energy budgets for interactive use; thermal/power policy on commodity devices; decoding settings that change verbosity and risk.

Failure symptoms. Spinners with no output; truncated answers; “connected” UI while the runtime is stalled or unreachable.

21.5.4 Retrieval / tool adapters (optional)

What they are. Components that fetch documents or call tools before or after generation.

Constraints. Access control on corpora; redaction; tool permission scopes.

Failure symptoms. Answers that cite absent sources; leaked patron or classmate data in a shared corpus demo; tool actions the user did not authorize.

21.5.5 Logging and disclosure store

What it is. Records of prompts, outputs, metadata, and consent state.

Constraints. Purpose limitation, retention clocks, export/delete pathways, accessible disclosure language.

Failure symptoms. History resurfacing after a delete request; portfolio screenshots that still contain identifiers; disclosure text the learner cannot operate with keyboard or assistive technology.

21.5.6 Identity and authorization adjacency

What they are. Session proof and allow/deny decisions that gate the AI feature. Full depth lives in later chapters; CH21 only needs the UX-linked split: signed-in entitled to every action.

Failure symptoms. Feature visible but silently blocked; mystery re-auth loops; verify steps that exclude assistive paths.

21.6 6. Stability contract

Definition (publication teaching model): a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For assistant answers in this chapter, the contract is not “the model is trustworthy.” It is a set of concurrent conditions on data, runtime, deployment honesty, and human-catchable uncertainty—aligned with CE-5 teaching bounds and NIST AI RMF’s organizational risk-management posture (Tabassi 2023).

Bound (teaching ID)	Within bounds means	Out of bounds (human-visible)
SC-CH21-Q Answer usability	Output is actionable or clearly refused/hedged	Fluent wrong answer treated as fact
SC-CH21-U Uncertainty honesty	Uncertainty is visible enough for the stakes	Confident fabrication; unexplained refusal storms
SC-CH21-D Data fitness	Inputs/retrieval quality within acceptable bounds	Garbage-in presented as expert-out
SC-CH21-M Model/runtime consistency	Version and availability match what the UI implies	Silent model swap; offline while chrome claims ready
SC-CH21-L Path locality disclosure	Learner can tell local vs remote at a usable level	Unexpected off-device send
SC-CH21-T Latency/energy budget	Interactive delay acceptable in context	Spinners that erase trust faster than the sentence helps
SC-CH21-P Disclosure/logging posture	Retention/share match consent card	Prompt history reused beyond disclosure
SC-CH21-A Accessible trust path	Answer and disclosure work with AT / keyboard	Vision-only confidence chrome; unspoken reply

Bound (teaching ID)	Within bounds means	Out of bounds (human-visible)
SC-CH21-E Equity of completion	Fixtures allow completion without paid APIs/GPUs	Lab gated on cloud credits

Observation versus inference rule (lab and prose):

- **Observation** — on-screen text, network-required Y/N, fixture versus live, wall-clock if measured and labeled **n=1**
- **Interpretation** — “probably cloud latency,” “probably retrieval miss”
- **Causal claim** — needs extra evidence (config, logs, controlled comparison)

Connected-versus-usable instances for this chapter: API returns 200 while the answer is wrong and authoritative-looking; model loaded while output never reaches the accessibility tree; TLS established while the endpoint operator can still read plaintext after decryption.

Non-claims: no numeric product SLOs for gunnchOS AI features; no assertion that meeting this teaching contract equals legal compliance; no labeling of AI as inherently inside or outside trust—only systems and practices are.

21.7 7. Try it

Primary lab: LAB-TRUST-001 — Compare local versus remote AI paths and write a consent/trust card.

Publication ownership. LAB-TRUST-001 is a book-side lab design inheriting CE-5 themes. It is **not** a WAIKE course module ID (CLM-CH21-004). Status tags on the lab package include `IMPLEMENTED_DIGITAL` and `FIXTURE_VALIDATED` for synthetic Route L / Route C transcripts—teaching infrastructure, never Gate 3 human validation.

WAIKE adjacency (not exact map). At audited WAIKE SHA `e97e74fc9bfb44b1cdc26b272dc4848` the `digital_rc` package **AI_ML_EDGE** hosts file-backed labs on scoring/inference, edge budgets, and RAG redaction. Catalog/track pointers such as `ai_ml_data` are related program language, not interchangeable IDs. Cite adjacency honestly; do not invent a WAIKE module named LAB-TRUST-001 or CH21 (gunnchOS3k 2026) (CLM-CH21-003).

21.7.1 Procedure spine (Explorer baseline)

1. **Predict** which path exposes more data leaving the device, and which Stability Contract bound (locality, privacy lifecycle, uncertainty honesty) is most likely to break first.
2. **Route L** — local / offline-capable simulator or fixture transcript.
3. **Route C** — remote assistant path (live optional) or fixture transcript.
4. **Compare** with observations only: what appeared, network required Y/N, disclosed retention cues.
5. **Consent/trust card** — audience, purpose, data classes, retention, opt-out, AI disclosure.
6. **Dual ledger** — human-trust feeling versus technical-trust controls/evidence.
7. **Uncertainty note** — fluent-but-wrong / unverifiable / none observed *with checks named*.

Safety: use only non-sensitive prompts; never paste real secrets, health records, or private messages; redact identifiers before portfolio share; no exploit steps or credential harvesting.

21.7.2 Pathway extensions

Operator. Locate permission, account, and privacy surfaces; separate symptoms that look like identity, authorization, network, or model quality.

Builder. Add one redaction rule for a sample prompt/log line; document utility-versus-exposure tradeoff.

Engineer. Sketch trust boundaries crossed by one prompt; compare local versus remote for latency, energy, privacy, and control—label illustrative rows unless measured.

Researcher. State an evaluation question with labeled limitations; do not invent peer-reviewed metrics. Accessible ML evaluation methods literature remains **SOURCE_NEEDED** for later depth (`peer_eval_methods` in the chapter source register)—do not invent DOIs.

Educator. Keep Route Paper / fixtures first-class for equity; facilitate “feeling versus control” teach-backs.

Evidence to keep. Comparison table, consent/trust card, dual ledger, uncertainty note, scrubbed reflection, teach-back. Completion means artifact-backed claims—not a vibes-based “the AI is trustworthy” slogan.

21.8 8. Build it

Extend LAB-TRUST-001 without turning Part V into a fake benchmark catalog.

21.8.1 Explorer

Build a pocket card: *fluency* *correctness*, plus the data→model→inference chain in one plain sentence.

21.8.2 Operator

Build a three-column inspection sheet: observations | inferences | evidence still needed. Add one row for “where did this prompt go?”

21.8.3 Builder

Build a one-page disclosure note for a toy retrieval path: what is collected, why, how long, who can see it, how to opt out, and how AI assistance is named. Optional: a tiny redaction helper on synthetic log lines—never live capture of others’ private data.

21.8.4 Engineer

Build a local-versus-cloud decision tree that ends in “needs more evidence” whenever UI copy claims locality without proof. List energy and latency as first-class tradeoffs alongside privacy and control (CLM-CH21-005 illustrative discipline).

21.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to make yet—for example, a published accuracy number for a named assistant on a homework corpus. Specify what labeled evaluation

and confounder control would require; keep peer-eval depth **SOURCE_NEEDED**. A future publication-owned toy eval worksheet (labeling observed versus inferred model errors) remains **proposed** until implemented—do not invent a WAIKE module id for it.

Educators can facilitate Section 11 teach-backs and keep fixture routes as equitable defaults—not lesser paths.

21.9 9. Secure and include it

21.9.1 Security (UX-linked, not a scare-list)

Prompts, pasted documents, and retrieved files are attack surface because they enter model context. Permission dialogs and update badges are trust boundaries learners can see. Classical protection principles such as least privilege and psychological acceptability still connect controls to usable experience (Saltzer and Schroeder 1975). Stay at concepts and symptoms: no exploit recipes, no credential harvesting, no unauthorized scanning.

21.9.2 Privacy

Teach lifecycle stages with decision owners: collect → use → retain → share → delete/redact. Portfolio artifacts must scrub account names, emails, and message previews. Metered-link and offline learners should not be forced onto live cloud assistants—LAB-TRUST-001 fixtures exist for equity and privacy together.

21.9.3 Accessibility

Accessible disclosure language and operable verify/answer paths are Stability Contract conditions, not garnish. Do not rely on color-only confidence meters; provide text hedges. Timing tasks allow extended time; avoid seizure-risk “thinking” animations. WCAG 2.2 informs teaching intent—not a claim that this book or any gunnchOS product is certified (W3C 2024).

21.9.4 Equity

Local GPUs and paid APIs are unevenly available. Fixtures and recorded transcripts are required for fair completion (SC-CH21-E). Language and dialect gaps in generative outputs are quality and equity issues, not user failure. Device Quartet form factors may appear only as optional future learning-lab context—**PHYSICAL_PENDING** for product-specific AI benchmarks; never required lab hardware.

21.9.5 Safety

Commodity devices only. Non-sensitive prompts only. No attempts to extract others’ private data “for the lab.”

21.9.6 Ethics and responsible use

Do not anthropomorphize models as moral agents; humans remain accountable for deployment and reliance. Prefer organizational risk-management language from NIST AI RMF over slogans that AI is inherently safe or unsafe (Tabassi 2023). Do not present **FIXTURE_VALIDATED** as Gate validation. Do not invent evaluation scores. Overclaiming measurement is still false evidence.

21.10 10. Career lens

One assistant answer crosses many ownership domains. No table promises employment; roles vary by organization. LAB-TRUST-001 artifacts resemble early professional evidence in miniature: labeled observations, disclosure notes, and explicit uncertainty.

Role family	Typical artifacts	Review questions
Applied ML / AI engineer	Eval note with limitations; model-card style summary	What was measured vs inferred?
Privacy / responsible AI analyst	Data-path + retention note; consent card	Does disclosure match logging?
Security engineer	Trust-boundary sketch on one prompt	Which UX symptom maps to which control?
SRE / platform	Runtime version and dependency notes	Did we confuse uptime with answer usability?
Accessibility specialist	AT pathway writeup for answer + disclosure	Was equivalent feedback present?
Educator / facilitator	Misconception checklist: fluency correctness	Did learners label fixtures correctly?

Portfolio hint: a scrubbed consent card plus an uncertainty note beats a vibes-based claim that “the model understands the user.”

21.11 11. Check understanding

Concept. In one sentence, frame a familiar AI feature as data conditioning a model applied at inference to produce outputs—without saying the system understands.

Concept. In one sentence each, distinguish *fluency*, *correctness*, and *evaluation evidence*.

System tracing. Trace one practical question from input to human judgment in numbered steps. Mark observed versus inferred. Name 4 Stability Contract conditions that had to hold together.

Misconception check. Why can a polished paragraph still fail the experience?

Misconception check. Why must this chapter refuse to treat AI as inherently trustworthy or untrustworthy?

Deployment check. Name one privacy, one latency/energy, and one control tradeoff between local and remote inference—label illustrative unless you measured.

Evidence ethics. What is the difference between LAB-TRUST-001 `FIXTURE_VALIDATED` infrastructure and Gate 3 human reader validation? Why must `WAIKE AI_ML_EDGE` adjacency never be renamed as LAB-TRUST-001?

Teach-it-back. Explain to a newcomer—using only CE-5 / LAB-TRUST-001 vocabulary—why fluency is not correctness.

Researcher prompt. What additional evidence would be required to move from a classroom uncertainty note toward a labeled evaluation study—and what remains **SOURCE_NEEDED** for this chapter?

21.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Deep Learning (Goodfellow et al.) supplies parameters/inference vocabulary; NIST AI RMF 1.0 supplies organizational risk-management language without binary trust slogans (Goodfellow, Bengio, and Courville 2016; Tabassi 2023). The textbook key `russell_norvig_aima` remains **NEEDS_PRIMARY_VERIFICATION** (edition/ISBN deferred) and is **omitted** from inline citation in this working draft until primary shelf verification closes.

Project-specific WAIKE adjacency uses audited SHA `e97e74fc9bfb44b1cdc26b272dc4848264f15fe0` via `@src-waike` (CLM-CH21-003). LAB-TRUST-001 evidence lives under `labs/LAB-TRUST-001/` on the publication repository (CLM-CH21-004). Peer-reviewed accessible evaluation-methods depth remains **SOURCE_NEEDED**. Gate 3 reader evidence remains pending; this working draft is not publication-ready.

Inline citations used in this chapter include Goodfellow, Bengio, and Courville (2016), Tabassi (2023), gunnchOS3k (2026), Saltzer and Schroeder (1975), and W3C (2024).

Primary inheritance (link, prefer over duplication): Concept Edition CE-5 under the publication preproduction tree, and `labs/LAB-TRUST-001/`.

21.13 12. Glossary links

Candidate terms introduced or reinforced here (see also chapter glossary candidates; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Training and input data	Examples/records used to fit a model or answer a question
Machine learning	Broader practice of fitting models from data and applying them at inference (not only generative systems)
Model parameters	Stored numbers shaping inputs→outputs; not a mind
Inference / output	Running a model on new inputs to produce a prediction or generation
Generative AI (systems view)	One ML family: systems that produce new text/images/code by predicting likely continuations (not all ML)
Uncertainty and errors	Outputs can be wrong or incomplete even when fluent
Local vs cloud AI	Whether inference runs on-device or on remote computers

Term	Plain link
Evaluation under uncertainty	Checking usefulness with labeled observations and limits
Fluency correctness	Coherent surface text is not proof of accuracy
Stability Contract	Concurrent hidden conditions that keep an experience alive (book teaching model)
Observation vs inference	What happened vs what may explain it
Consent / trust card	Disclosure artifact: audience, purpose, data classes, retention, opt-out, AI notice
Dual ledger	Human-trust feeling recorded beside technical controls and evidence

Related earlier chapters: experience-first observation craft (CH02), memory/storage lifecycles (CH07), apps/APIs (CH14), placement and cloud/edge (CH15). Related later chapters: acceleration and edge budgets (CH22), cybersecurity (CH23), privacy/identity/ethics (CH24), evidence practice (CH27), capstone synthesis (CH31).

21.14 Figure references (planned embeds; accessibility metadata)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured learner or lab evidence. No fabricated telemetry. No product accuracy curves.

21.14.1 FIG-CH21-001 — Data → model → inference → output

- **Type.** Conceptual flowchart (inherit CE-5 FIG-CE5-001 intent).
- **Reader should notice.** Data conditions a model; parameters apply at inference; output is not understanding.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name each stage in order; state conceptual truth class; forbid mind metaphors as literal claims.

21.14.2 FIG-CH21-002 — Local vs cloud AI boundaries

- **Type.** Layered comparison (inherit CE-5 FIG-CE5-002 intent).
- **Reader should notice.** Same question, different privacy/latency/control boundaries.
- **Truth class.** Illustrative unless a measurement bundle is attached.
- **Alt text requirement.** Name both paths and at least one differing bound (privacy, latency, or control); deny product SKU timings without evidence.

21.14.3 FIG-CH21-003 — Fluency vs correctness vs evaluation evidence

- **Type.** Conceptual hierarchy / separation.
- **Reader should notice.** Three families that must not collapse into “the AI was right.”
- **Truth class.** Conceptual.

- **Alt text requirement.** Name all three families and one everyday question each; state that fixtures are not human validation.
-

21.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH21-001.** Accessible systems teaching can frame many AI features as data conditioning a model applied at inference—cited via (Goodfellow, Bengio, and Courville 2016); `russell_norvig_aima` omitted pending primary verification. Prohibited wording: the model understands/knows.
- **CLM-CH21-002.** NIST AI RMF 1.0 provides voluntary organizational guidance for managing AI risks without treating trust as a binary property of AI itself (Tabassi 2023).
- **CLM-CH21-003.** WAIKE `digital_rc AI_ML_EDGE` hosts adjacent file-backed labs at audited SHA `e97e74fc9bfb44b1cdc26b272dc4848264f15fe0` (gunnchOS3k 2026)—adjacency only; no invented WAIKE module LAB-TRUST-001 / CH21.
- **CLM-CH21-004.** Publication LAB-TRUST-001 is a book-side lab inheriting CE-5 themes; not a WAIKE course ID.
- **CLM-CH21-005.** Illustrative local-vs-cloud comparison tables are teaching aids unless a measurement bundle is attached—no measured gunnchOS SKU latency without evidence.

Chapter 22

Chapter 22 — Edge AI, Sensors, and Embodied Interaction

Status: draft · **Chapter ID:** CH22

Author: Edmund Gunn, Jr.

Manuscript: working full-manuscript draft · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS; fixtures and illustrative budgets are teaching infrastructure, not human reader evidence).

Part V opens the intelligence arc. Chapter 21 names data, models, and generative outputs. This chapter asks what changes when sensing and inference move *near the body*—on a phone, a laptop, or a wearable learning form factor—so a gesture, glance, or haptic cue can close a loop without waiting on a distant API. Concept Edition CE-5 already framed local-vs-cloud latency and privacy tradeoffs; here we deepen **embodied interaction** without claiming shipping Device Quartet AI performance.

22.1 1. The moment

You raise a hand, glance at a camera preview, or feel a short haptic pulse meant to confirm an assistive action. Sometimes the feedback arrives while you are still mid-gesture—offline, private, immediate. Sometimes nothing happens: the preview is black because a permission was denied; the model stalls while the device thermally throttles; a sleeve occludes the sensor; the cloud path is unreachable and there is no honest local fallback.

From the seat: either the body and the machine felt *together*, or the loop broke in a way a green “AI ready” badge cannot explain.

Underneath, the question is not “did the model understand me?” It is a path: physical signal → features → on-device or edge inference → actuator or UI feedback under human timing (Goodfellow, Bengio, and Courville 2016) (CLM-CH22-001). Running inference near the sensor can reduce round-trip dependency compared with remote APIs—at the cost of device resource budgets. That is a systems tradeoff, not a product FPS or milliwatt claim for Edge IO Wearables.

This chapter’s governing question:

When sensing plus on-device inference tries to close an embodied loop, which concurrent conditions keep the experience alive—and which privacy, budget, and fallback failures make the body and the machine fall out of sync?

22.2 2. What you notice

Before vocabulary like *quantization*, *NPU*, or *sensor fusion*, notice the human contracts you already expect.

You expect a gesture-triggered assist to work offline when the network is gone—or to fail with an honest reason. You expect a camera or microphone prompt to mean something: consent before continuous capture, not silent recording of classmates (“Media Capture and Streams” 2025). You expect a haptic or spoken cue soon enough that it still feels like a reply to *this* motion. You expect occlusion (hand over lens, sleeve over IMU), low light, and battery saver modes to produce recognizable symptoms—not mysterious “AI broken” blame. You expect portfolio screenshots not to contain other people’s faces, voices, or private spaces.

Those expectations *are* embodied interaction literacy.

Embodied features fail as human timing and consent problems before they fail as model trivia.

Optional commodity notice (no specialized wearables): on a device you already own, open a familiar camera or motion-assisted feature you are allowed to use. Write two columns—*permission/status shown* and *body-visible outcome*—then stop. If live capture is unsafe, unethical, or unavailable, use a fixture route (Section 7) and mark rows **fixture**. You are practicing observation vs inference, not collecting Device Quartet benchmarks.

22.3 3. Exploded ecosystem

An embodied assist is not “the model.” It is a path through cooperating layers. **FIG-CH22-001** is the first-minute map: sense → feature → edge inference → haptic/UI feedback. Treat it as **Representative educational architecture**, not a wiring diagram for any shipping wearable SKU.

22.3.1 Human and context

Intent, attention, posture, clothing, and lighting set what “usable sensing” means *now*. A classroom demos gesture and a private assistive cue do not share one universal latency tolerance.

22.3.2 Sensors and local capture path

Cameras, microphones, IMUs, touch, and proximity convert physical events into samples. CH08 already taught sensor pipelines; here we *use* them as inputs to an embodied loop, not re-teach the catalog. On the web platform, camera and microphone access is mediated capture under a permission model—not unmediated truth about the room (“Media Capture and Streams” 2025).

22.3.3 Features and on-device / edge inference

Raw samples become features; a model’s parameters are applied at inference time to produce outputs—an accessible systems framing, not a claim that networks “understand” intent (Goodfellow, Bengio, and Courville 2016) (CLM-CH22-001). Placement may be fully on-device, on a nearby edge node, or hybrid. CE-5’s local-vs-cloud contrast still applies: locality can improve latency and shrink what leaves the device, while cloud offload can buy capacity at privacy and round-trip cost (illustrative teaching contrast unless a measurement bundle is attached).

22.3.4 Actuators, haptics, and UI feedback

Motors, speakers, displays, and assistive announcements close the loop. Feedback that arrives after the gesture’s meaning has passed is a failed embodiment even if the logits were correct.

22.3.5 Budgets and thermal/power managers

Latency, energy, memory, and heat bound what the local path may do. **FIG-CH22-002** may show qualitative bands only when labeled illustrative—never as measured Edge IO Wearables EVT.

22.3.6 Policy, identity, and privacy boundary

Continuous sensing expands what may be exposed beyond the visible UI: faces in the background, overheard speech, precise motion patterns. Organizational AI risk language such as NIST AI RMF remains voluntary guidance for trustworthy-system characteristics—not a certificate that “edge AI is safe” (Tabassi 2023).

22.3.7 Evidence boundary (Device Quartet)

Edge IO Wearables appear here as the primary Device Quartet *learning* form factor for embodied edge themes. Physical fabrication and measured AI/sensing benchmarks remain **PHYSICAL_PENDING** (CLM-CH22-002) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). **FIG-CH22-003** must carry that badge whenever the Wearables context is drawn.

22.4 4. Follow the signal

Follow one human-visible embodied action without inventing wearable telemetry.

1. **Intent forms.** You gesture, glance, speak a wake phrase you are allowed to use, or wait for a haptic confirm.
2. **Permission / consent gate.** The stack asks—or should ask—whether this app may use the sensor *now* (“Media Capture and Streams” 2025).
3. **Sense.** Samples arrive with noise, occlusion, or adequate quality.
4. **Feature.** Local code turns samples into a representation the model expects.
5. **Edge / on-device inference.** Parameters apply near the device, reducing remote round-trip dependency when the local budget holds (Goodfellow, Bengio, and Courville 2016) (CLM-CH22-001).
6. **Actuate / feedback.** Haptic, audio, or UI state answers under human timing—**FIG-CH22-001**.

7. **Fallback.** If the local model is unavailable, thermally throttled, or out of memory, an honest degraded path (cloud offload with disclosure, offline refusal, or simpler heuristic) must exist—or the experience fails openly.

22.4.1 Alternate paths (the honesty rule)

If you lack a wearable or NPU, you can still practice the loop on commodity camera/mic permission flows, fixture occlusion worksheets, and CE-5 **LAB-TRUST-001** local-vs-remote path comparison. That is adjacency—not a fake Edge IO Wearables measurement (CLM-CH22-002) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

22.4.2 Failure branch without drama

Common honest failures: permission denied; sensor occluded; thermal/power budget exceeded; model file missing; cloud fallback unreachable without disclosure; feedback too late for the gesture. None of these require inventing milliwatts or FPS.

22.5 5. Component cards

22.5.1 Embodied sensing

- **Role.** Capturing physical-world signals for interaction.
- **Plain contract.** Turns motion, light, sound, or touch into samples the rest of the loop can use.
- **Misread.** Treating the sensor reading as ground truth about people or intent.
- **Failure symptoms.** Black preview, silent mic, flat IMU under occlusion, noisy samples in motion.

22.5.2 Edge / on-device inference

- **Role.** Running models near the sensor or actuator.
- **Plain contract.** Can reduce remote round-trip dependency when device budgets allow (Goodfellow, Bengio, and Courville 2016) (CLM-CH22-001).
- **Misread.** “On-device” means unlimited intelligence with zero energy cost.
- **Failure symptoms.** Stalls, thermal throttle, out-of-memory, missing model artifact.

22.5.3 Edge resource budget

- **Role.** Latency, energy, memory, and thermal limits that bound experience.
- **Plain contract.** Embodiment lives inside a budget envelope—**FIG-CH22-002**.
- **Misread.** One universal millisecond table fits every assistive cue.
- **Evidence note.** Measured Device Quartet budgets remain **PHYSICAL_PENDING** (CLM-CH22-002).

22.5.4 Embodied interaction loop

- **Role.** Sense → infer → actuate/feedback under human timing.
- **Plain contract.** Correct logits with late haptics still feel broken.
- **Misread.** Collapsing the loop into “the AI decided.”

22.5.5 Sensing privacy boundary

- **Role.** What continuous capture may expose beyond the visible UI.
- **Plain contract.** Background faces, overheard speech, and motion patterns can leave the intended subject.
- **Misread.** A local model automatically makes capture ethical.
- **Citation posture.** Camera/microphone permission mediation is cited (CLM-CH22-004 · SOURCE_IDENTIFIED via (“Media Capture and Streams” 2025; “Permissions” 2025)). Do not invent IMU-fusion ISO/IEEE designations.

22.5.6 Edge / cloud fallback

- **Role.** Degraded path when local inference cannot complete.
- **Plain contract.** Disclose offload; refuse silently inventing success.
- **Misread.** Fallback is free capacity with identical privacy.

22.6 6. Stability contract

Definition retained from the book: a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

Chapter-focused concurrent conditions (qualitative; no invented numeric Device Quartet budgets):

1. **Sensor signal quality** stays within usable bounds (occlusion, lighting, contact).
2. **On-device compute / thermal / power budget** can finish inference in time for the gesture.
3. **Permission and privacy boundary** for continuous sensing remains respected—consent before capture; no unauthorized recording of others (“Media Capture and Streams” 2025).
4. **Haptic / UI feedback latency** remains acceptable for embodiment (human timing, not logo timing).
5. **Fallback** exists when the edge model is unavailable—cloud with disclosure, offline refusal, or simpler local heuristic.

A device can remain “AI enabled” in settings while the embodied experience has already failed: permission denied, model thermally stalled, haptic late, or cloud path silently substituted without consent. Stability is concurrent conditions—not a single neural badge.

Honesty bound for this edition: Do not invent Edge IO Wearables FPS, milliwatts, or fusion accuracy. Physical AI/sensing evidence remains **PHYSICAL_PENDING** (CLM-CH22-002) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Commodity and fixture labs produce *your* observations—not shipping-product EVT, and not Gate 3 validation.

22.7 7. Try it

22.7.1 Local-path adjacency — LAB-TRUST-001

Goal. Rehearse locality, privacy exposure, and consent using the publication-owned CE-5 lab, then extend the same observation discipline toward embodied sensing *without* unauthorized capture.

WAIKE alignment note. WAIKE accepted main maps edge-budget and embedded latency labs adjacently (AI_ML_EDGE, lab_quantize_budget, EMBEDDED_PROTOTYPING, catalog edge_ai_embedded). Those are competency neighbors—not CH22 module IDs (CLM-CH22-003) (gunnchOS3k 2026). Do not invent edge_ai_ch22 or claim LAB-TRUST-001 is a WAIKE course ID.

Safety (hard stops).

- No unauthorized camera/mic capture of other people, classrooms, or private spaces.
- No covert recording, no credential harvesting, no exploit steps, no keyloggers or clipboard monitors.
- Use only prompts and fixtures that contain non-sensitive content; never paste secrets, health records, or private messages.
- Redact faces, voices, account IDs, and precise locations before portfolio share.
- No Device Quartet hardware required; no invented wearable measurements.

Routes.

- **LAB-TRUST-001 commodity/fixture route.** Complete Route L vs Route C comparison and one consent/trust card per that lab’s contract (browser, local simulator, or supplied fixtures). Mark live cloud attempts optional; fixtures are first-class for equity.
- **Proposed sensing worksheet (LAB-CH22-SENSE-001 — proposed, not a shipped WAIKE ID).** Using *your own* device or a still-image/fixture only: (1) record the permission dialog outcome (allow / deny / not prompted); (2) simulate occlusion with a lens cover or fixture note; (3) write observation-vs-inference rows for “feature unavailable.” Do **not** aim the camera at other people.

Explorer baseline (about 45–60 minutes).

1. Predict which Stability Contract bound breaks first under denial, occlusion, or offline mode.
2. Run LAB-TRUST-001 local vs remote comparison **or** the proposed sensing permission/occlusion card.
3. Fill observation-vs-inference columns. “Permission denied” is an observation; “the NPU is 3 TOPS” is an inference you are **not** allowed to invent here.
4. Write a five-sentence teach-back: sense → feature → edge infer → feedback; privacy boundary; PHYSICAL_PENDING for Wearables.

Operator extension. Compare symptoms: permission deny vs occlusion vs airplane mode. Note which UI text is honest.

Builder extension. Sketch a privacy-preserving sampling config for a *toy* gesture detector: sample rate intent, on-device-only checkbox, retention “discard after inference,” and an explicit “no cloud upload” line. Fixture-only is fine.

Engineer extension. Draft a qualitative edge budget worksheet (latency / energy / memory / thermal) with blank measured cells labeled **PHYSICAL_PENDING** for Quartet hardware (CLM-CH22-002). Cite CE-5 illustrative local-vs-cloud contrast as teaching-only unless you attach your own measurements.

Researcher extension. Propose a measurement plan that would be required to promote Wearables AI/sensing claims out of PHYSICAL_PENDING: instruments, ethics/consent protocol, stop criteria, and what still would not be proven by a classroom n=1 (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Reference WAIKE adjacent labs only as curriculum neighbors (CLM-CH22-003) (gunnchOS3k 2026).

Evidence to keep. Scrubbed observation table; consent/trust card; teach-back paragraph; budget worksheet with honest blanks.

22.8 8. Build it

Extend Try-it without turning Part V into a wearable product catalog.

22.8.1 Explorer

Build a pocket card: four lines—(1) sense, (2) on-device infer, (3) feedback under human timing, (4) permission before capture.

22.8.2 Operator

Build a symptom card: permission / occlusion / thermal / offline / late haptic—each with one human-visible clue and one next check.

22.8.3 Builder

Build a labeled loop diagram for one assistive action: human → sensor → feature → edge model → actuator. Annotate what is observed vs inferred. Optional second diagram: local-only vs disclosed cloud fallback.

22.8.4 Engineer

Build an edge budget brief: define qualitative bands, list what primary MCU-NN or official edge-ML documentation would be required to deepen `edge_ml_sys_refs` (still `SOURCE_NEEDED` for promotion), and leave product milliwatts blank (Goodfellow, Bengio, and Courville 2016).

22.8.5 Researcher

Build an evidence plan for CLM-CH22-002: fabrication status, ethics for continuous sensing studies, replication package, and explicit forbid on treating concept renders as EVT (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Separately note that sensing-privacy standards promotion (omitted CLM-CH22-004) still needs primary designation—do not invent ISO numbers.

Educators can facilitate Section 11 teach-backs and keep classrooms on fixture / self-capture-only / no-camera routes whenever shared spaces make live sensing unsafe.

22.9 9. Secure and include it

22.9.1 Security

Edge models and sensor daemons expand the trust boundary: new binaries, new permissions, new failure modes for spoofed “local success.” Prefer least privilege and fail-safe defaults in design language; keep this chapter at concepts and UX symptoms—no offensive capture tooling (Tabassi 2023).

22.9.2 Privacy

Continuous sensing is a privacy surface. Local inference reduces some egress but does not erase on-device retention risk. Portfolio artifacts must scrub bystander faces, voices, and precise locations. Unauthorized capture of others is out of scope for every lab route.

22.9.3 Accessibility

Embodied feedback must not depend on vision-only or color-only cues. Provide haptic and text alternatives; describe permission and occlusion states in words so screen-reader and print users follow the same method (W3C 2024).

22.9.4 Equity

Not every learner owns a wearable, NPU laptop, or quiet lab. Commodity phones, fixture transcripts, and offline worksheets are first-class. Edge IO Wearables are learning context with **PHYSICAL_PENDING** status—not an admission ticket (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

22.9.5 Safety

No covert recording, no interfering with others’ devices, no improvised body-worn electrical experiments in this chapter’s labs. Embodied literacy is observational, consensual, and fixture-friendly.

22.9.6 Ethics

Do not invent Wearables AI benchmarks, fake privacy-standard citations, or Gate 3 PASS language. Overclaiming on-device magic is still a form of false evidence. Keep PHYSICAL_PENDING and omitted SOURCE_NEEDED labels visible where they belong.

22.10 10. Career lens

Roles that touch this chapter’s problems. **No employment guarantees.**

Layer / concern	Role lens	Portfolio evidence (classroom analogue)
On-device / edge ML	Embedded / edge ML engineer	Edge budget worksheet with honest blanks; LAB-TRUST-001 locality card
Sensing systems	Sensor systems engineer	Occlusion / permission / noise symptom card (self or fixture only)
Human factors / safety	Safety / HF engineer	Embodied timing risk note—no product claims
Privacy engineering	Privacy engineer (descriptive)	Consent card + retention “discard after inference” sketch

Layer / concern	Role lens	Portfolio evidence (classroom analogue)
Responsible AI	Trustworthy-AI / GRC adjacency	NIST AI RMF-informed risk note for continuous sensing (Tabassi 2023)
Accessibility	Accessibility specialist	Non-visual feedback checklist (W3C 2024)

Portfolio signal: a scrubbed permission/occlusion card plus an honest PHYSICAL_PENDING measurement plan beats a marketing screenshot of a glowing wearable.

22.11 11. Check understanding

1. In one sentence, why can on-device inference reduce round-trip dependency *and* still fail the human experience?
2. Name the sense → feature → infer → feedback path and one failure at each hop.
3. What does **PHYSICAL_PENDING** mean for Device Quartet Edge IO Wearables in this chapter?
4. Why is LAB-TRUST-001 adjacent rather than a WAIKE “CH22 module”?
5. What is the difference between a sensing privacy *boundary* and a claim that local AI is automatically ethical?
6. Name two Stability Contract conditions specific to embodied loops.
7. What hard stop applies to camera/mic labs in shared spaces?
8. Teach-back (Explorer): explain to a nontechnical person why a late haptic can make a “correct” model feel broken.

Researcher prompt. What primary sources would be required to (a) promote Wearables sensing/AI claims out of PHYSICAL_PENDING and (b) close the omitted sensing-privacy standards gap—without inventing designations?

22.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`), including systems framing for models and inference (Goodfellow, Bengio, and Courville 2016), platform-mediated capture permissions (“Media Capture and Streams” 2025; “Permissions” 2025), voluntary AI risk-management guidance (Tabassi 2023), and accessibility feedback requirements (W3C 2024). Project-specific Device Quartet and WAIKE adjacency evidence is cited via (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026) and (gunnchOS3k 2026), separately from external literature. IMU-fusion *standards* designations remain out of claim scope this wave (CLM-CH22-004 resolved for camera/mic permission mediation only). Official MCU-NN / edge-ML system references (`edge_ml_sys_refs`) remain SOURCE_NEEDED for later depth.

22.13 12. Glossary links

Term	Plain link
Embodied sensing	Capturing physical-world signals for interaction
Edge / on-device inference	Running models near the sensor/actuator
Edge resource budget	Latency, energy, memory, thermal limits that bound experience
Embodied interaction loop	Sense → infer → actuate/feedback under human timing
Sensing privacy boundary	What continuous capture may expose beyond the visible UI
Edge / cloud fallback	Degraded path when local inference cannot complete
Stability contract	Concurrent conditions that keep the experience alive
PHYSICAL_PENDING	Claim awaiting physical fabrication or measured validation

Related earlier chapters: sensors/IO (CH08), power/thermal budgets (CH09), cloud/edge placement (CH15), data/ML framing (CH21), CE-5 local-vs-cloud trust. Related later chapters: chip-to-cloud security (CH23), privacy/responsibility deepening (CH24), twin honesty (CH29/CH31).

22.14 Figure references (planned embeds; draft-blocked until SVG + a11y land)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured evidence. No fabricated wearable telemetry.

22.14.1 FIG-CH22-001 — Sense → feature → edge inference → haptic/UI feedback

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Sequence.
- **Reader should notice.** Embodied loop hops and where human timing attaches.
- **Truth class.** Conceptual / Representative educational architecture.
- **Alt text requirement.** Name sense, feature, inference, feedback; state conceptual truth class; color never sole cue.

22.14.2 FIG-CH22-002 — Edge resource budget bands (illustrative)

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Budget.
- **Reader should notice.** Qualitative latency/energy/memory/thermal bands.
- **Truth class.** Illustrative.
- **Alt text requirement.** State illustrative only; forbid reading bands as Device Quartet EVT.

22.14.3 FIG-CH22-003 — Edge IO Wearables learning context

- **Production status.** draft-blocked (no SVG embed in this draft); **PHYSICAL_PENDING** badge required.
- **Type.** Ecosystem / form-factor context.
- **Truth class.** Conceptual with qualification **PHYSICAL_PENDING** (CLM-CH22-002).
- **Alt text requirement.** Name Wearables as research/learning form factor; state physical fabrication and measured AI/sensing pending; color never sole cue.

22.15 Claim footnotes used in this chapter

Claim ID	Text (short)	Status
CLM-CH22-001	On-device/edge inference can reduce round-trip dependency vs remote APIs, at device budget cost	SOURCE_IDENTIFIED (goodfellow_deep_learning)
CLM-CH22-002	Device Quartet Edge IO Wearables remain research/learning form factors; physical AI/sensing benchmarks pending	PHYSICAL_PENDING (src-hardware-quartet)
CLM-CH22-003	WAIKE AI_ML_EDGE / lab_quantize_budget / EMBEDDED_PROTOTYPING are adjacent, not CH22 module IDs	SOURCE_IDENTIFIED (src-waike @e97e74fc9bfb44b1cdc26b272dc48482
CLM-CH22-004	Camera/mic permission mediation (no invented IMU-fusion ISO)	SOURCE_IDENTIFIED (w3c-mediacapture-streams-202510w3c-permissions-20251006)

Chapter 23

Chapter 23 — Cybersecurity from Chip to Cloud

Status: `working_draft` · **Chapter ID:** CH23

Author: Edmund Gunn, Jr.

Inheritance: CE-5 identity / crypto-goal / UX-linked attack-surface slices; CH11 firmware/trust adjacency (link, do not duplicate)

Primary Try-it lab: LAB-TRUST-001 (publication-owned; link, do not duplicate)

Gate posture: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS)

23.1 1. The moment

You sign in. The app looks connected. Then one of two ordinary things happens: an action you can see is refused—“you don’t have permission”—or a “verify it’s you” step interrupts a flow that felt already open. The glass still shows the product. The network icon may still look fine. Trust, from your seat, has already hitched.

From the outside it is tempting to collapse everything into one complaint: “security is broken,” or “the password worked so why can’t I do this?” Those complaints name real frustration. They also mix three different ideas: **identity** (who or what you claim to be), **authentication** (how believable that claim is for the risk), and **authorization** (what an authenticated actor may do). Login success is not a blank check (Temoshok et al. 2025).

This chapter’s governing question is ordinary and honest:

When a login succeeds but an action is denied—or a verify step appears while the app still looks connected—what trust conditions from chip to cloud still have to hold?

Part V starts security literacy here on purpose. Later chapters deepen privacy (CH24), responsibility and research ethics (CH27), and portfolio evidence (CH31). Chapter 11 already named firmware and early trust adjacency. This chapter does **not** replay a scare-list of vulnerabilities, and it does **not** teach attack recipes. It follows **principles and UX symptoms** along one chip-to-cloud path of conditions.

If you remember only one sentence after the first reading, remember this: **AuthN AuthZ—being recognized is not the same as being allowed.**

23.2 2. What you notice

Before jargon, notice the human contract you already enforce with frustration.

You expect a successful sign-in to open a session that matches the risk of what you are about to do. You expect some actions to remain blocked even after login when your role does not include them. You expect a “verify it’s you” step to feel annoying but purposeful—not random theater. You expect recovery after lockout to remain possible without a single inaccessible channel. You expect that “connected” and “usable trust” can diverge.

Those expectations are not decorations. They *are* the product from the person’s point of view.

Security feel is a perception produced by layered trust conditions—not by a single padlock or a single password field.

What you may notice, without opening a debugger:

- login succeeds, then a specific button or page is denied,
- MFA or step-up verify interrupts a familiar flow,
- lockout after repeated attempts,
- a session that suddenly asks you to sign in again,
- recovery that only works through one channel (SMS-only, vision-only CAPTCHA, or a help desk you cannot reach).

What those symptoms do **not** prove by themselves: that the network failed, that “HTTPS means private forever,” that the cloud is evil, or that you should follow an exploit recipe to “test security.” High connectivity and broken trust can travel together. Diagnosis needs labeled observation versus inference—never unauthorized scanning of systems you do not own.

Optional seat comparison (safe): open a familiar account on a device you may use for learning. Note one action that works and one that is denied or asks for extra verify—*before* you invent a causal story. Prefer a personal sandbox or school account. Do not capture other people’s credentials, tokens, or private messages.

23.3 3. Exploded ecosystem

Usable trust is not a single object. It is a path of cooperating conditions. Figure 23.1 is the first-minute map for this chapter: chip/boot → firmware/OS → app/session → network → cloud/policy, with human UX symptoms attached to the same flows people actually use. Treat it as **conceptual**—not a claim that any manufactured revision wires trust the same way, and not a measured Device Quartet secure-boot capture. Chip/firmware measured secure-boot validation for research form factors remains **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026; “Gunnchos-Device-Os Accepted Main (CE-3 Audit)” 2026).

Walk the layers in ordinary language. Keep the same layers when vocabulary deepens. Do **not** treat this as a vulnerability encyclopedia.

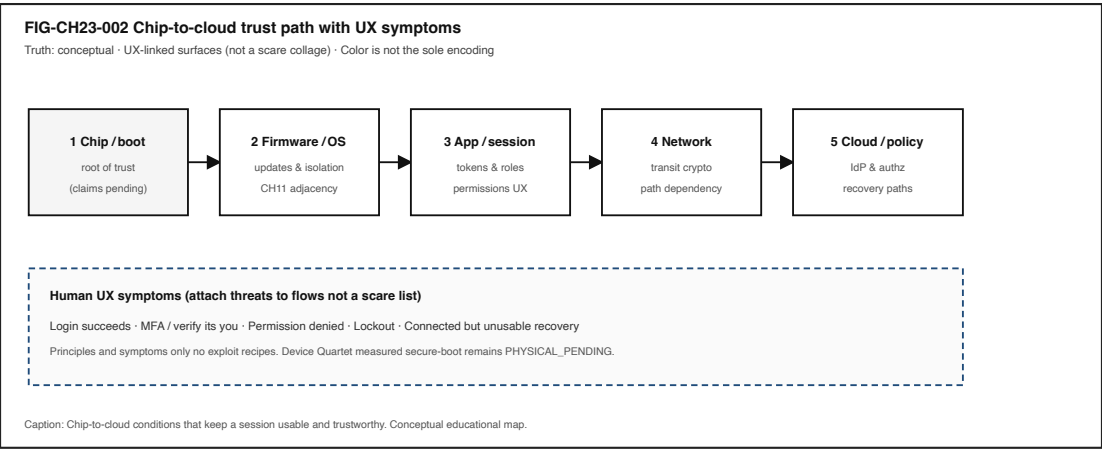


Figure 23.1: Conceptual chip-to-cloud trust path with human UX symptoms attached.

23.3.1 Human

You form intent: open a document, change a setting, approve a payment, ask an assistant. Eyes and hands judge whether the session still feels like *your* session—and whether denials and verifies are explainable.

23.3.2 Chip / boot (conceptual)

Early hardware and boot stages can act as a **root of trust** for later software. This book names the idea without claiming a certified shipping secure-boot story for Device Quartet or gunnchOS products. Measured validation stays **PHYSICAL_PENDING** where product fact would be required (“Gunnchos-Device-Os Accepted Main (CE-3 Audit)” 2026; “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Chapter 11 holds firmware adjacency in more depth—link, do not duplicate.

23.3.3 Firmware / OS

Updates, isolation, and privilege boundaries shape what apps can claim about the machine underneath. Failures here often surface later as mysterious re-auth loops, broken sessions, or “the device no longer trusts itself”—not as a labeled “firmware” dialog.

23.3.4 App / session

Tokens, cookies, roles, and permission prompts live where people can see them. Attack surfaces attach to the same flows: prompt boxes, permission dialogs, session tokens, update badges—UX-linked, not a scare collage (CE-5 inheritance via FIG-CE5-007 → FIG-CH23-002).

23.3.5 Network

Transit protections matter for data in motion. A healthy path does not prove the endpoint is honest, and a padlock icon does not prove lifelong privacy. See Figure 23.3.

23.3.6 Cloud / identity / policy

Identity providers, authorization services, logging, and recovery policies often sit off-device. The person experiences allow/deny, MFA, and restore—not the policy YAML.

23.3.7 Society / recovery equity

Who can complete verify and recovery—and who is excluded by SMS-only, CAPTCHA-only, or unpaid support—belongs inside security literacy, not as an appendix afterthought.

23.4 4. Follow the signal

Read the following as a logical story for one ordinary protected action—not as a claim that every product executes exactly one step at a time.

1. **Intent.** You decide to perform an action that matters (read, write, share, pay, administer).
2. **Identity claim.** The session asserts who or what is acting—a person, a device, a bot, a service account (Temoshok et al. 2025).
3. **Authentication.** The system checks whether that claim is believable enough *for this risk*—password, passkey, MFA, hardware-backed proof, or a prior session assurance that has aged (Temoshok et al. 2025).
4. **Authorization.** Separately, a policy decides whether *this* authenticated actor may do *this* action on *this* object. AuthN success does not skip AuthZ (Temoshok et al. 2025).
5. **Crypto boundary (as needed).** Data may be protected in transit and/or at rest; endpoints that decrypt still matter (Figure 23.3).
6. **Effect or denial.** The UI allows, denies, or steps up verification. The person notices the outcome.
7. **Logging / retention (often invisible).** Systems may record the attempt for audit or support—privacy lifecycle territory deepened in CH24.
8. **Recovery path.** Lockout, lost-factor, or incident restore must return *usable* trust without theater alone.

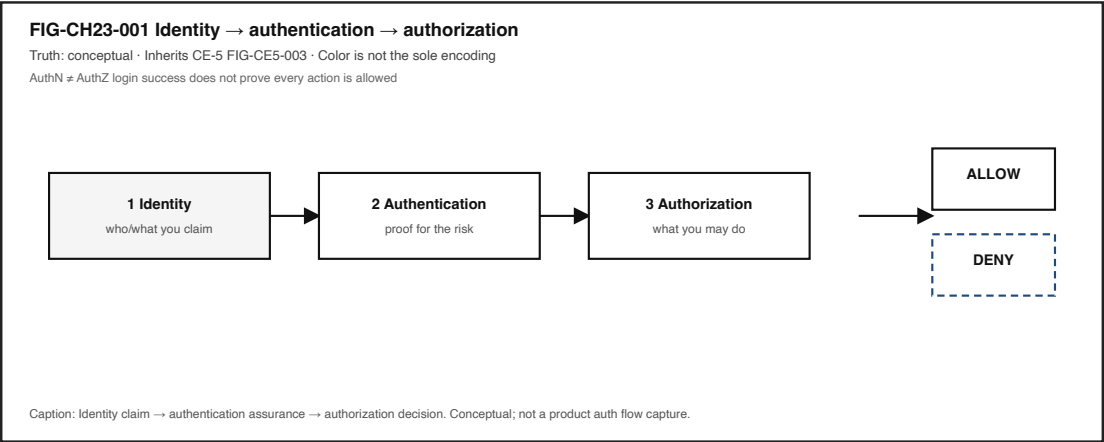


Figure 23.2: Identity → authentication → authorization decision ladder.

Failure branches are part of honesty:

- **AuthN fails** → cannot prove the claim (wrong factor, expired session, inaccessible MFA).
- **AuthZ fails** → claim is accepted, action still denied (role/attribute mismatch)—the chapter’s anchor moment.
- **Crypto boundary misunderstood** → padlock present while endpoint fully exposes content to the unlocked session.
- **Recovery fails** → technically “secure” lockout that strands a person without an accessible path.

Figure 23.2 is the decision ladder you should be able to teach back without jargon inflation. Protection design principles—least privilege, fail-safe defaults, psychological acceptability—remain useful for connecting controls to usable experience (Saltzer and Schroeder 1975). This chapter cites those principles as **design constraints**, not as a vulnerability trivia dump.

23.5 5. Component cards

Each card is plain language + constraints + failure symptoms. None of these cards is a how-to for breaking systems.

23.5.1 Identity

What it is. A claim about who or what an actor is—user account, device identity, service principal, bot.

Constraints. Claims need a namespace and lifecycle (create, change, retire). Identity alone never implies permission.

Failure symptoms. Wrong account selected; ghost accounts; “signed in as someone else”; confusing account switchers.

23.5.2 Authentication (AuthN)

What it is. Checking whether an identity claim is believable enough for the risk of the next actions (Temoshok et al. 2025).

Constraints. Assurance should match risk. Extra friction without purpose is theater. Accessible alternatives matter when factors exclude people.

Failure symptoms. Endless verify loops; MFA channel unreachable; session fixation *as a UX story* (mystery re-auth)—without teaching exploitation steps.

23.5.3 Authorization (AuthZ)

What it is. Deciding what an authenticated actor may do (Temoshok et al. 2025).

Constraints. Least privilege; fail-safe defaults (Saltzer and Schroeder 1975). Visible buttons that silently fail are a UX and equity problem.

Failure symptoms. Login works, action denied; feature visible but blocked; admin tools exposed to every desk role by mistake (design smell—not an invite to escalate privileges).

23.5.4 Session / recovery

What it is. Continuity of trust across time, and paths back after lockout or incident.

Constraints. Recovery must remain usable and accessible. Incident restore should return usable trust—not security theater alone (CE-5 / CH23 Stability Contract).

Failure symptoms. Locked out with no second channel; restore that never re-enables legitimate work.

23.5.5 Encryption boundary

What it is. Goals for unreadability without keys in transit and/or at rest—plus honest endpoint limits (Figure 23.3; CE-5 FIG-CE5-004 inheritance).

Constraints. Encryption often supports **confidentiality**, but **integrity** (detecting unwanted change) and **availability** (keeping usable access/recovery paths alive) are separate goals—the CIA trio is not a padlock icon. Encryption does not alone guarantee correct UX, honest UI, safe endpoint, or human trust.

Failure symptoms. “We use encryption” used as a conversation-stopper while unlocked endpoints still expose content; integrity or recovery failures blamed on “the network” alone.

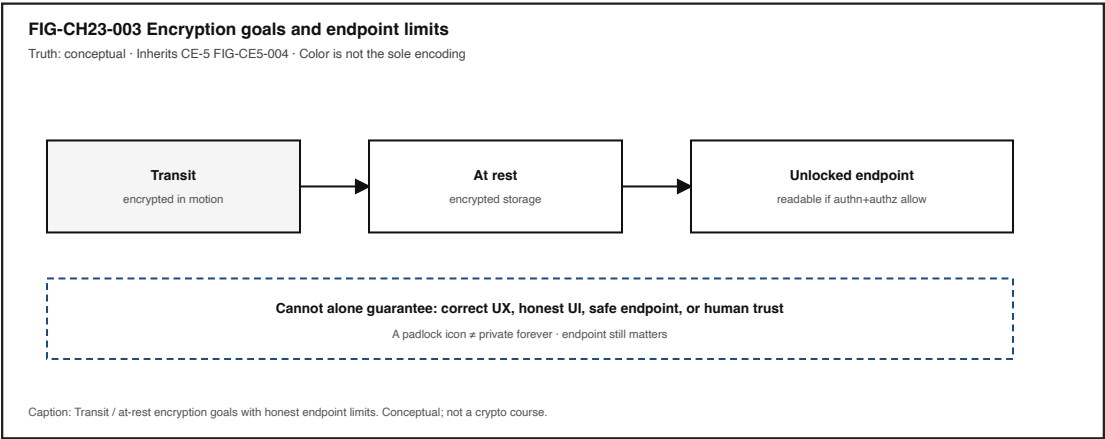


Figure 23.3: Encryption transit / at-rest goals with unlocked-endpoint limits.

23.5.6 Chip-to-cloud trust path

What it is. Boot/firmware/OS/app/network/cloud controls treated as one chain of conditions (Figure 23.1).

Constraints. Do not overclaim measured secure boot for shipping research hardware while evidence is **PHYSICAL_PENDING** (“Gunnchos-Device-Os Accepted Main (CE-3 Audit)” 2026; “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Failure symptoms. Connected glass with broken trust; update badges that people cannot interpret; local vs cloud path opacity (deeper in CE-5 / CH21 adjacency).

23.6 6. Stability contract

The **Stability Contract** is a signature idea across this book:

A user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For Chapter 23, a usable trusted session may require all of the following to stay “good enough” at once—qualitative bounds only; no invented numeric SLOs:

- authentication assurance appropriate to risk (Temoshok et al. 2025),
- authorization decisions consistent with least privilege (Saltzer and Schroeder 1975),
- crypto boundaries for transit/at-rest goals, with endpoint limits stated honestly,
- session and recovery paths remain usable and accessible,
- incident restore returns usable trust without security theater alone,
- the person can still tell allow, deny, and verify apart from “the network died.”

A system can remain **powered on and connected** while the **human experience of trust has already failed**.

Three separations matter here:

1. **Authentication** versus **authorization** — AuthN AuthZ (Temoshok et al. 2025).
2. **Encryption in transit/at rest** versus **endpoint exposure** — padlock private forever; confidentiality integrity availability (Figure 23.3).
3. **Felt safety** versus **technical trust evidence** — feelings matter; they are not the same as controls and artifacts (CE-5 dual-ledger adjacency).

Commodity observations you collect with **LAB-TRUST-001** are *your* evidence for *your* routes and fixtures—not universal product certifications, and not Gate 3 reader validation. Device Quartet measured secure-boot claims remain **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

23.7 7. Try it

23.7.1 LAB-TRUST-001 — Authn/authz literacy + trust restore adjacency (link)

Observable question. When the same practical question (or the same protected action) is answered or mediated by a local path versus a remote path, what can I observe about identity, privacy, and trust—without treating a model as a person who knows, and without building attacks?

This chapter’s Try It **inherits and links** the publication-owned CE-5 lab rather than inventing a duplicate LAB-AUTH-001 package. Follow the full lab packet at `labs/LAB-TRUST-001/` (README, routes, fixtures, portfolio templates, A11Y_PRIVACY_SAFETY.md). Focus your write-up on the **AuthN vs AuthZ** columns and on which Stability Contract bounds stayed in bounds / out of bounds / unknown.

WAIKE alignment note. WAIKE accepted `main` (audit SHA `e97e74fc9bfb44b1cdc26b272dc484826` in the CH23 packet) hosts adjacent CYBERSECURITY competencies such as `lab_iam_rbac` and `lab_incident_playbook`, plus `SOFTWARE_BUILDER lab_authz—adjacent neighbors only` (gunnchOS3k 2026). There is **no** exact WAIKE module ID for LAB-TRUST-001 or CH23. Do not mint one.

Prerequisites. A computer or phone you may use for learning; modern browser optional; Python optional for local route; fixture route always available.

Safety (non-negotiable).

- Principles and UX symptoms only—**no** exploit steps, scanning, phishing kits, credential harvesting, or capture of others’ private data.
- Use only non-sensitive prompts; never paste real SSNs, health records, passwords, tokens, or private messages.
- Redact account identifiers before portfolio share.
- Prefer fixtures when live cloud accounts would expose personal history.
- No Device Quartet hardware is required. No invented EVT secure-boot numbers.
- Stop if a vendor warns about account lock risk; use the fixture path instead.

Time estimate. About 50 minutes for Explorer + Operator baseline.

23.7.1.1 Prediction

Before comparing routes, predict which path exposes more data leaving the device, and which bound is most likely to break first: locality, privacy lifecycle, identity continuity, authorization clarity, or uncertainty honesty.

23.7.1.2 Route L — Local / offline-capable (or fixture)

Complete Route L via `labs/LAB-TRUST-001/local_app/trust_sim.py`, the browser worksheet, or `fixtures/route_l_transcript.md`. Record observations only.

23.7.1.3 Route C — Cloud / remote (optional live) or fixture

Complete Route C live only if allowed and safe; otherwise use `fixtures/route_c_transcript.md`. Equity of completion requires the fixture path.

23.7.1.4 Route paper — No-device

Use printed/fixture materials when devices or bandwidth are unavailable.

23.7.1.5 Evidence (minimum)

- comparison table (observation vs inference columns),
- consent/trust card,
- dual-ledger note (human-trust feeling + technical-trust control),
- uncertainty note,
- teach-back sentence that includes **AuthN** **AuthZ**.

23.7.1.6 Pathway notes

Pathway	LAB-TRUST-001 emphasis for CH23
Explorer	Name identity vs authentication vs authorization in ordinary language.
Operator	Map lockouts, MFA prompts, and permission denials to likely domains.
Builder	Prefer the authz / consent card artifacts; prepare for Build it matrix.

Pathway	LAB-TRUST-001 emphasis for CH23
Engineer	Mark component-boundary least privilege / fail-safe defaults on your sketch.
Researcher	State what evidence would support a trust claim without overclaiming.
Educator	Prefer fixture route; forbid offensive content; require text status labels.

LAB-TRUST-001 is `FIXTURE_VALIDATED` as a publication lab design on its agent line—that is **not** Gate 3 human validation.

23.8 8. Build it

Extend LAB-TRUST-001 without turning Part V into an offensive security course.

23.8.1 Explorer

Build a pocket card with three sentences: Identity, Authentication, Authorization. Each sentence must refuse one synonym trap (especially “login = permission”).

23.8.2 Operator

Build a symptom → likely domain checklist: lockout, MFA prompt, permission denial, padlock-with-bad-feeling, recovery dead-end. End every row with “needs more evidence.”

23.8.3 Builder

Fill an **authz matrix** for a toy service with three actors—desk user, reader-only guest, automation bot—and three actions—read, write, admin. Mark allow/deny. Document one least-privilege choice and one fail-safe default (Saltzer and Schroeder 1975). Do **not** implement privilege-escalation tests against live systems you do not own.

23.8.4 Engineer

Sketch one component boundary (app identity provider, or app data store). Annotate AuthN check, AuthZ check, and crypto boundary (transit / at rest / endpoint). Apply least privilege and fail-safe defaults as labeled constraints—not as attack surface hunting.

23.8.5 Researcher

State what evidence would support a trust claim (“this session restores usable trust after lock-out”) including method, confounders, and uncertainty. Keep Device Quartet / gunnchOS measured secure-boot claims **PHYSICAL_PENDING** (“Gunnchos-Device-Os Accepted Main (CE-3 Audit)” 2026; “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Do not invent Gate 3 PASS language.

23.8.6 Educator

Adapt the authz matrix to a classroom with fixture-only completion. Provide keyboard / screen-reader notes for verify steps. Ban real credential capture as a learning activity.

23.9 9. Secure and include it

23.9.1 Security

Teach security as UX-linked conditions: permission dialogs (least privilege + psychological acceptability), verify steps (authentication assurance), feature blocked (authorization), strange login notices (session/account recovery UX), update badges (supply trust as literacy)—not a detached scare-list (Saltzer and Schroeder 1975). Out of scope for this book’s classroom path: exploit development, credential stuffing how-tos, privilege-escalation recipes, live social-engineering drills against third parties, jailbreak catalogs as entertainment, or binary reverse engineering.

23.9.2 Privacy

Consent and lifecycle (collect → use → retain → share → delete/redact) sit beside AuthN/AuthZ. LAB-TRUST-001’s consent/trust card is the adjacent artifact; CH24 deepens privacy without treating this chapter as a law course.

23.9.3 Accessibility

Auth and recovery paths must work with assistive technology, keyboard, and low bandwidth where feasible. SMS-only, CAPTCHA-only, or vision-only verify can exclude people—pair every “harder AuthN” discussion with accessible alternatives. Figures Figure 23.2 through Figure 23.3 encode meaning with shape, order, labels, and stroke pattern—not color alone. Automated checkers do not certify WCAG conformance.

23.9.4 Equity

Not every learner has GPU credits, paid API keys, admin rights, or a private phone number for SMS MFA. Fixture routes are the baseline for fair completion. Device Quartet form factors are conceptual analogies here, not admission tickets (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). Avoid shaming older hardware or unpaid accounts; teach diagnosis that works with the machine and routes in front of you.

Shared family or school machines raise another equity edge: “just open the security settings” may require permissions a learner does not hold. Fixtures and verbal observation tables keep the pathway open without forcing privilege escalation. The Stability Contract fails for people first; pedagogy should not add a second failure by assuming identical tools—or by teaching attacks.

23.10 10. Career lens

Literacy from this chapter shows up in several neighboring roles—without employment promises:

Role family	Portfolio evidence (safety-first)
Security engineer	UX-linked threat-model worksheet (principles + symptoms; no exploits)
IAM engineer	AuthN/AuthZ decision card; authz matrix for desk/reader/bot
SRE / incident responder	Restore-usable-trust playbook adjacency (detect → contain → recover as literacy)
Privacy-adjacent builder	Consent/trust card + redaction notes from LAB-TRUST-001
Educator / mentor	Misconception drills: login authorization; padlock private forever

WAIKE CYBERSECURITY labs such as `lab_iam_rbac` and `lab_incident_playbook` are **competency neighbors**, not renamed book lab IDs (gunnchOS3k 2026). Completing artifacts does **not** guarantee employment.

Day-one habits transfer: write a prediction before you open a live account; label AuthN vs AuthZ; refuse brochure trust claims without a method; never confuse “I can attack it” with “I understand it.”

23.11 11. Check understanding

23.11.1 Misconception probes

1. **“Login success means I am authorized for every action.”**
Counter: AuthN AuthZ; authorization is a separate decision (Temoshok et al. 2025).
2. **“HTTPS / a padlock means private forever.”**
Counter: transit protection is not the whole story; unlocked endpoints still matter (Figure 23.3).
3. **“Security is a scare-list of vulnerabilities detached from UX.”**
Counter: this chapter attaches risks to flows people notice—permissions, verify, deny, recovery (Saltzer and Schroeder 1975).
4. **“Encryption alone guarantees human trust.”**
Counter: encryption cannot alone guarantee correct UX, honest UI, or safe endpoints (Figure 23.3).
5. **“LAB-TRUST-001 is an official WAIKE module ID.”**
Counter: publication-owned lab; WAIKE links are adjacent only (gunnchOS3k 2026).
6. **“Device Quartet / gunnchOS measured secure boot is a shipping proven fact in this chapter.”**
Counter: **PHYSICAL_PENDING** / project evidence needed (“Gunnchos-Device-Os Accepted Main (CE-3 Audit)” 2026; “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).
7. **“Gate 3 has PASSED for this manuscript.”**
Counter: `GATE_3_IN_PROGRESS` - `READER_EVIDENCE_PENDING`. Do not fabricate reader evidence.

23.11.2 Teach-back (say it out loud)

Identity is a claim. Authentication checks the claim for the risk. Authorization decides what you may do. Login can succeed while an action is denied. Encryption helps in transit and at rest, but endpoints still matter. Trust runs from chip to cloud as conditions—not as a single icon. We study principles and symptoms, not attack recipes.

23.11.3 Pathway self-check

Pathway	You can...
Explorer	Name identity vs authentication vs authorization in ordinary language.
Operator	Map lockouts, MFA prompts, and permission denials to likely domains.
Builder	Fill an authz matrix (desk/reader/bot) for a toy service.
Engineer	Apply least privilege / fail-safe defaults to a component boundary sketch.
Researcher	State what evidence would support a trust claim without overclaiming.
Educator	Facilitate AuthN AuthZ; keep labs fixture-safe and non-offensive.

23.12 References

Inline citations used in this chapter include Saltzer and Schroeder (1975), Temoshok et al. (2025), gunnchOS3k (2026), “Gunnchos-Device-Os Accepted Main (CE-3 Audit)” (2026), and “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026).

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Project-specific measured chip/boot claims remain **PHYSICAL_PENDING** and are cited separately from external literature. Living documents such as OWASP materials are **not** silently invented here; pin retrieval dates if promoted later (`owasp_living` remains **SOURCE_NEEDED** in the packet).

23.13 12. Glossary links

Term	Plain link
Attack surface (UX-linked)	Places misuse can enter through the same flows people use
Identity	A claim about who/what an actor is
Authentication	Checking an identity claim is believable enough for the risk

Term	Plain link
Authorization	Deciding what an authenticated actor may do
Encryption goals and limits	Unreadability without keys; endpoints still matter
Chip-to-cloud trust path	Boot/firmware/OS/app/network/cloud controls as one chain of conditions
Protection design principles	Least privilege, fail-safe defaults, psychological acceptability, etc.

Related earlier chapters: CH11 firmware/trust adjacency; CE-5 Concept Edition slices for identity/crypto/attack-surface. Related later chapters: privacy depth (CH24), responsibility (CH27), research portfolio honesty (CH31).

23.14 Figure references (embedded above; accessibility metadata)

- **FIG-CH23-001** — figures/architecture/fig-ch23-001-identity-authn-authz.svg (inherits FIG-CE5-003)
 - **A11y:** figures/preproduction/accessibility/fig-ch23-001.yaml
- **FIG-CH23-002** — figures/ecosystem/fig-ch23-002-chip-to-cloud-trust.svg (inherits FIG-CE5-007 pattern)
 - **A11y:** figures/preproduction/accessibility/fig-ch23-002.yaml
- **FIG-CH23-003** — figures/architecture/fig-ch23-003-encryption-boundaries.svg (inherits FIG-CE5-004)
 - **A11y:** figures/preproduction/accessibility/fig-ch23-003.yaml

Related CE-5 maps (optional cross-read, not required duplicates): figures/preproduction/ce-05/fig-fig-ce5-004.svg, fig-ce5-007.svg.

23.15 Claim footnotes used in this chapter

Claim ID	Teaching use	Evidence posture
CLM-CH23-001 — Saltzer & Schroeder principles as design constraints	least privilege / fail-safe / psychological acceptability	SOURCE_IDENTIFIED via Saltzer and Schroeder (1975)
CLM-CH23-002 — Identity / AuthN / AuthZ separation	AuthN AuthZ	SOURCE_IDENTIFIED via Temoshok et al. (2025)
CLM-CH23-003 — WAIKE CYBERSECURITY labs as adjacent neighbors	do not invent CH23 WAIKE IDs	SOURCE_IDENTIFIED via gunnchOS3k (2026)

Claim ID	Teaching use	Evidence posture
CLM-CH23-004 — Measured chip/firmware secure-boot product fact	research form-factor honesty	PHYSICAL_PENDING with “Gunnchos-Device-Os Accepted Main (CE-3 Audit)” (2026) / “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026)

Chapter 24

Chapter 24 — Privacy, Identity, Safety, Accessibility, and Ethics

Status: draft · **Chapter ID:** CH24

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS; fixtures and illustrative portfolios are teaching infrastructure, not human reader evidence).

Part V has already named models, inference paths, and chip-to-cloud security language. This chapter refuses the appendix treatment: privacy, identity, safety, accessibility, and ethics are concurrent conditions of usable technology. It inherits Concept Edition **CE-5** (privacy/lifecycle/responsible-use) and publication lab **LAB-TRUST-001**, with CE-6 adjacency for accessibility-as-contract-condition—without duplicating CH21’s ML depth or CH23’s full attack-surface tour.

Two non-collapse rules govern the prose:

1. **Privacy security.** Encryption, authentication, and least privilege can hold while collection, retention, sharing, or deletion still fail the person.
2. **Accessibility convenience.** A path that is faster for one body/tool is not automatically usable for another; alternate routes are success conditions, not polish.
3. **Safety censorship.** Safety limits aim to reduce harm from fluent or actionable outputs; they are not the same as viewpoint-censorship claims, and over-refusal is a distinct failure mode to name honestly.

24.1 1. The moment

You need to finish something ordinary: reset access after a lockout, accept a permission or privacy notice, recover an account, or complete a verify step so a familiar feature will run. The app looks modern. A classmate finishes in seconds. Your path stalls—CAPTCHA that never announces, SMS you cannot receive, a privacy wall of text you cannot act on, or a recovery flow that exists only as a vision-only selfie.

From the seat: exclusion dressed as “just a security step,” or surprise reuse of something you thought you deleted.

Underneath: concurrent conditions. A **permission notice** is not proof of informed control. An **authenticated session** is not authorization for every button on the screen. A **padlock** is not a privacy lifecycle. An **accessible path** is not a nice-to-have theme.

The governing question for this chapter:

When a permission, recovery, or accessibility path decides whether someone can finish a task others complete easily—which conditions broke, and what evidence do I actually have?

This is CE-5’s responsible-use spine expanded for full-book depth. It is not a scare-list, not legal advice, and not a claim that citing WCAG or NIST certifies any product in this book.

24.2 2. What you notice

Before naming Solove harms or assurance levels, notice the human contract that broke.

You expected a familiar task to remain *doable*. Instead you notice a consent prompt you cannot parse into actions, a “verify it’s you” loop that excludes your assistive path, a fluent assistant answer that sounds finished while you still do not know where the prompt went, or a delete request that leaves history resurfacing later. A peer on a different device, network, or body may finish while you do not. That divergence is part of the technical story—not a character flaw in the user.

Privacy symptoms and security symptoms can diverge. HTTPS can succeed while retention exceeds disclosure. Login can succeed while authorization silently blocks. Status chrome can say “secured” while the person cannot recover access with their tools (Saltzer and Schroeder 1975).

Accessibility failure is experience failure. If the only recovery path requires a modality you cannot use, the Stability Contract already broke—even if the backend “worked” for other users (W3C 2024).

Optional commodity notice (no specialized gear): open one familiar app’s privacy/permission surface *or* an account-recovery entry point you already own. Write two columns—*what the UI claimed* and *what you could actually do*—before inventing a cause. If live capture is unsafe, inequitable, or metered, use LAB-TRUST-001 fixtures and mark rows **fixture**. Fixture transcripts are illustrative teaching data, not your measured evidence and not Gate validation.

24.3 3. Exploded ecosystem

A stuck recovery or opaque consent is not a single object. It is a path through an ecosystem. **FIG-CH24-001** is the first-minute map: human experience at the center, with concurrent spokes for collect → use → retain → share → delete/redact. Treat it as **Representative educational architecture**, not a claim that every app shares one vendor topology.

Walk the layers in ordinary language.

24.3.1 Human and context

Intent, stigma of being locked out, literacy load of legal text, and who is watching over the shoulder set what “private enough” and “usable enough” mean *for this person now*.

24.3.2 Interaction and alternate path

Keyboard, pointer, voice, switch, or screen reader must deliver intent and receive equivalent feedback. Accessibility is a path through the ecosystem—not a post-hoc theme (W3C 2023, 2024). Cite both dated WCAG 2.2 Recommendation editions when referring to the standard family; do not silently collapse dates into one undated /TR/WCAG22/ shortcut.

24.3.3 Application / policy / consent UI

Handlers assemble prompts, show disclosures, and gate features. Psychological acceptability of controls matters as much as their existence (Saltzer and Schroeder 1975).

24.3.4 Identity provider and session state

Claims about who you are (or which service is speaking) unlock experiences. Proofing and authentication are not the same as authorization of a specific action (Temoshok et al. 2025).

24.3.5 Data stores and logs

History, backups, analytics queues, moderation review, and fine-tune candidates are where life-cycle decisions become durable. Delete UI without residual-copy honesty is a privacy failure even when disks are encrypted.

24.3.6 Inference / remote services (when AI is in play)

Local vs cloud path changes who can observe prompts and outputs (CE-5 adjacency). Fluency is not correctness; disclosure modes belong in the consent story (gunnchOS3k 2026).

24.3.7 Network and crypto boundary

Transit protections can be real while endpoints still process plaintext for the service. Padlock private forever.

24.3.8 Organizational / societal layer

Vendors, reviewers, regulators, and equity of private compute shape who can complete labs and who is surveilled by default. Fixtures exist so unpaid API credits do not gate Explorer completion.

FIG-CH24-002 separates accessible recovery/auth paths from blocked ones so readers stop treating “security worked” as “everyone could finish.”

24.4 4. Follow the signal

Here the “signal” is a person trying to keep usable control of access and data—not a single TLS handshake. Read the sequence as a logical journey. Alternate paths exist; open questions stay

labeled undetermined.

- 1. **Intent.** A person tries to use, recover, consent, refuse, or delete.
- 2. **Input delivery.** OS and app receive the intent—including along assistive paths—or do not.
- 3. **Identity claim.** Session, token, or proof step asserts who/what is acting (Temoshok et al. 2025).
- 4. **Authorization / policy gate.** Role, attribute, risk, or feature flag allows or denies.
- 5. **Data movement.** Collect / use for the stated purpose; optional off-device send.
- 6. **Side effects.** Retain, share with vendors/reviewers, or queue for later training—disclosed or not.
- 7. **Feedback.** Visual, haptic, audio, or AT announcement of success, denial, or uncertainty.
- 8. **Human judgment.** Finished, locked out, surprised by reuse, or “works for them / not for me.”

24.4.1 Lifecycle stages without collapse

Stage	Everyday question	Typical mis-use
Collect	What entered the prompt, form, sensor, or log?	Treating “I typed it” as voluntary forever
Use	What purpose is happening now?	Purpose creep without new disclosure
Retain	What still exists after the screen clears?	Equating logout with deletion
Share	Who else can see copies?	Assuming encryption forbids all sharing
Delete/redact	What residual copies remain?	Calling UI “deleted” without residual honesty

Solove’s taxonomy supplies peer-reviewed vocabulary for naming privacy *harms* carefully; do not invent page numbers in this draft (Solove 2006). Harm language is not a substitute for lifecycle controls.

24.4.2 Evidence hierarchy (climb only as far as tools and ethics allow)

- 1. Illustrative teaching aid (labeled)
- 2. Commodity observation (UI claims, wall-clock, what you could/couldn’t do)
- 3. Config / disclosure text you can actually read
- 4. Correlated multi-signal inspection (still not proof)
- 5. Controlled comparison under ethical constraints (LAB-TRUST-001 Route L vs Route C)
- 6. Formal assurance or conformance assessment—**not** required for classroom completion and **not** claimed for this book (Temoshok et al. 2025; W3C 2023, 2024)

24.4.3 Failure branch without drama

Prefer failure *domains* over confident blame: disclosure/consent, identity proofing, authentication session, authorization, retention/share, accessibility path, safety filter UX, network/crypto boundary. Outside observation rarely distinguishes them cleanly. That limitation is literacy.

24.5 5. Component cards

For each idea: plain language, analogy (labeled), technical function, constraints, common symptoms.

24.5.1 Privacy / data lifecycle

- **Plain language.** Rules and practices for collect, use, retain, share, delete/redact.
- **Analogy (labeled).** Like a library checkout slip that must say what was borrowed, for how long, and who else may see it—not merely that the building has a lock.
- **Technical function.** Names data fate across systems that security controls alone do not settle.
- **Constraints.** Residual copies; vendor subprocessors; backup lag; UI delete cryptographic erase.
- **Symptoms.** Surprise recall of old prompts; “deleted” history resurfacing; consent text that cannot be acted on.

24.5.2 Identity (human systems)

- **Plain language.** Claims about people or services that unlock experiences.
- **Analogy (labeled).** Like a name badge that is not the same thing as a key to every room.
- **Technical function.** Separates identity proofing / authentication from authorization decisions (Temoshok et al. 2025).
- **Constraints.** Assurance levels are context-dependent; this book does not certify products.
- **Symptoms.** Mystery re-auth loops; recovery that excludes AT; login success with silent feature deny.

24.5.3 Authentication vs authorization

- **Plain language.** Authn: proving a claim. Authz: deciding what that claim may do.
- **Analogy (labeled).** Like showing ID at the door versus being allowed into the lab fridge.
- **Technical function.** Prevents collapsing “signed in” with “permitted.”
- **Constraints.** Visible buttons can still be unauthorized; error copy may lie by omission.
- **Symptoms.** Feature visible but blocked; admin UI for non-admins; session valid while scope insufficient.

24.5.4 Safety constraints

- **Plain language.** Limits that keep harmful outcomes from being easy to enact—especially fluent outputs that look actionable.
- **Analogy (labeled).** Like a power tool’s guard: it does not make the tool “moral”; it changes what is easy.
- **Technical function.** Bounds deployment and reliance practices; NIST AI RMF vocabulary is useful without declaring AI itself good or bad (Tabassi 2023).
- **Constraints.** Safety censorship: filters can over-refuse without becoming a free-speech slogan; fluency can still mislead; humans remain accountable for reliance.
- **Symptoms.** Confident wrong answer treated as fact; unexplained refusal storms; tool actions without disclosure.

24.5.5 Accessibility path

- **Plain language.** Alternate routes so the experience succeeds for diverse bodies and tools.
- **Analogy (labeled).** Like a building that needs a ramp *and* stairs—speed on the stairs is not the ramp.
- **Technical function.** Equivalent perception, operable controls, understandable feedback, robust AT names (W3C 2024).
- **Constraints.** Automated checkers do not certify conformance; dated WCAG editions must not be silently overwritten (W3C 2023, 2024).
- **Symptoms.** CAPTCHA-only verify; focus traps; status by color alone; completion never announced.

24.5.6 Ethics ladder

- **Plain language.** Observation before inference; disclosure and responsible use before confident blame.
- **Analogy (labeled).** Like a lab notebook: write what you saw before what you guess.
- **Technical function.** Keeps portfolios and reviews from laundering speculation as evidence (gunnchOS3k 2026).
- **Constraints.** Feeling safe technical control; disclosure modes are practices, not magic.
- **Symptoms.** “The AI knows”; fixture numbers presented as human evidence; Gate status inflated.

24.5.7 Consent / trust card

- **Plain language.** A compact artifact: audience, purpose, data classes, retention, opt-out, AI disclosure.
 - **Analogy (labeled).** Like a nutrition label for a data path—incomplete if any required field is missing.
 - **Technical function.** Forces lifecycle and disclosure decisions into inspectable form (LAB-TRUST-001 / WAIKE lab_consent_disclosure adjacency).
 - **Constraints.** A filled card is pedagogy, not legal compliance.
 - **Symptoms.** Vague “improve the product” checkboxes; no retention bound; no opt-out path.
-

24.6 6. Stability contract

Definition (publication teaching model): a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

Signature distinction for this chapter: a system can remain technically **secured** or **online** while the human experience has already **failed** on privacy, recovery, or accessibility.

24.6.1 Concurrent conditions (qualitative anchor)

For a successful permission / recovery / trust-usable experience, conditions such as the following must remain *good enough together* (CE-5 inheritance + CH24 packet):

ID	Hidden condition
SC-CH24-C	Consent/disclosure understandable and actionable
SC-CH24-I	Identity recovery usable without excluding assistive paths
SC-CH24-Z	Authorization matches what the UI implies
SC-CH24-P	Data lifecycle controls (retain/delete/redact) operable as disclosed
SC-CH24-A	Accessibility path remains concurrent with other success conditions
SC-CH24-S	Safety constraints prevent harmful fluent outputs from being actionable blindly
SC-CH24-L	Path locality (local vs cloud) knowable at a usable level when AI is involved
SC-CH24-E	Equity of completion—fixtures available when live paths are gated by cost or hardware

Numeric / legal humility rule: this teaching contract is **not** GDPR/CCPA compliance, **not** WCAG certification of the book or any gunnchOS product, and **not** NIST product assurance. Cite standards for vocabulary and teaching intent only (Temoshok et al. 2025; W3C 2023, 2024).

24.6.2 CE-5 + LAB-TRUST-001 linkage

- **CE-5 preproduction** (publication/preproduction/ce-05/) is the primary inheritance package: lifecycle, local-vs-cloud trust, authn/authz, consent card, ethics ladder.
- **LAB-TRUST-001** is publication-owned. Status tags include **IMPLEMENTED_DIGITAL** and **FIXTURE_VALIDATED**. Those tags mean worksheets, validators, and synthetic Route L / Route C transcripts exist. They do **not** mean Gate 3 PASS. They do **not** mean fixtures are human evidence. They do **not** mean Device Quartet on-device measurements (**PHYSICAL_PENDING**).

WAIKE accepted main (SHA e97e74fc9bfb44b1cdc26b272dc4848264f15fe0) hosts adjacent COMM_PD_ETHICS labs (lab_consent_disclosure, lab_ai_disclosure_modes, lab_ethics_ladder, lab_accessibility_comm). Those are adjacencies—not renamed as CH24 course IDs (gunnchOS3k 2026).

24.7 7. Try it

24.7.1 LAB-TRUST-001 — Compare local vs remote AI paths and write a consent/trust card

Goal. Observe the same practical question on a local-capable path vs a remote path (or fixtures), then produce a consent/trust card and a dual-ledger note (one human-trust feeling + one technical-trust control)—without treating the model as a person who “knows.”

WAIKE alignment note. Adjacent only at audited SHA e97e74fc9bfb44b1cdc26b272dc4848264f15fe0. **LAB-TRUST-001** is publication-owned; do not invent a WAIKE ID with that name.

Safety (hard stops).

- Non-sensitive prompts only; no secrets, health records, private messages, or precise location.
- No exploit steps, credential harvesting, unauthorized scanning, or capture of others’ private data.
- Redact account identifiers before portfolio share.
- No Device Quartet / specialized hardware required.

Routes.

- **Route L — Local / offline-capable (or fixture).** Use on-device/local runtime if available; otherwise `fixtures/` labeled `FIXTURE`.
- **Route C — Cloud / browser assistant (or fixture).** Network required when live; otherwise supplied remote transcript.
- **Route F — Fixture fallback (mandatory equitable path).** Complete with synthetic transcripts when live paths are unavailable, unsafe, or costly.

Explorer baseline.

1. Predict which route requires network and which (claimed) sends prompt text off-device.
2. Fill observation columns only (time if measured by you; hedges; errors).
3. Complete consent/trust card: audience, purpose, data classes, retention, opt-out, AI disclosure.
4. Dual-ledger: one feeling + one control.
5. Uncertainty note: one fluent-but-wrong or unverifiable claim—or “none observed” with what was checked.
6. Mark fixture-derived rows as `fixture`.

Operator extension. Classify one friction symptom as identity, authorization, network, model quality, privacy disclosure, or accessibility.

Builder extension. Add one redaction rule for a sample log/prompt line; document utility vs exposure.

Engineer extension. Sketch trust boundaries crossed by Route C; name which SC-CH24 conditions you observed vs inferred.

Researcher extension. State a hypothesis about leakage or overrefusal; list 3 confounders; no invented statistics.

Educator facilitation. Prefer Route F when live capture is inequitable; run misconception drill from Section 11.

Evidence to keep. Comparison table, consent card, dual-ledger, scrubbed notes, teach-back. Completion means artifact-backed claims—not a bare exit code and not Gate PASS.

24.8 8. Build it

Extend LAB-TRUST-001 without turning Part V into a fake compliance certificate.

24.8.1 Explorer

Build a pocket card: *privacy security, ally convenience*, plus the five lifecycle stages in plain sentences.

24.8.2 Operator

Build a two-column sheet: UI claims | actions you could complete. Add a third column only for inferences, each with “evidence still needed.”

24.8.3 Builder

Build a consent-card template plus one redaction config for a toy log line—or a minimal authz matrix (desk / reader / bot) that shows authn authz.

24.8.4 Engineer

Build a recovery-path decision tree that ends in “inaccessible for AT / keyboard / SMS-unavailable users” as a first-class failure—not an afterthought. Cite identity guidance vocabulary honestly without product certification claims (Temoshok et al. 2025). Map at least one blocked path against WCAG-informed operable/perceivable failure language using a **dated** key (W3C 2024).

24.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to make yet—for example, “this app is WCAG 2.2 conformant” or “this IdP meets a NIST assurance level.” Specify what audit artifacts would be required; keep book posture non-certifying. Note both WCAG dated Recommendation editions exist and must not be silently merged (W3C 2023, 2024).

Educators can facilitate Section 11 teach-backs and keep Route F first-class.

24.9 9. Secure and include it

This section is the chapter’s core—not an appendix. Keep threats attached to user experience.

24.9.1 Security (UX-linked)

Permission dialogs, verify steps, and session emails are where Saltzer & Schroeder’s least privilege and psychological acceptability meet real screens (Saltzer and Schroeder 1975). Teach boundaries: phishing as experience failure; prompt/document text as untrusted context for models. Out of scope: exploit development, credential-stuffing how-tos, live social-engineering against third parties.

24.9.2 Privacy

Lifecycle stages with decision owners (Section 4). Security controls can succeed while privacy fails—name that split explicitly. Solove-style harm vocabulary may label issues carefully without invented pages (Solove 2006).

24.9.3 Identity

Separate proofing/authentication from authorization (Temoshok et al. 2025). Recovery must remain concurrent with accessibility conditions (SC-CH24-I + SC-CH24-A).

24.9.4 Accessibility

Accessibility is a Stability Contract condition. Figures and labs: alt text, text equivalents, reading order; color never sole cue. Auth flows: keyboard operable; screen-reader names for verify steps. Automated checkers do **not** certify WCAG conformance. When citing WCAG 2.2, keep dual dated keys: Recommendation 5 October 2023 and Recommendation 12 December 2024 (W3C 2023, 2024).

24.9.5 Equity

Private compute gap, language/dialect quality gaps, SMS/CAPTCHA/selfie exclusion, unpaid data labor myths, and API cost gates. Fixtures are equity infrastructure.

24.9.6 Safety

Prevent harmful fluent outputs from being treated as automatically actionable (Tabassi 2023). Trauma-aware optional examples; proportionate account-takeover stories.

24.9.7 Ethics

Observation before inference (**FIG-CH24-003** ethics ladder). Disclose AI assistance where learners present work. Do not anthropomorphize models as moral agents. Do not present **FIXTURE_VALIDATED** as Gate 3 PASS. Do not invent WAIKE ethics course IDs for CH24 (gun-nchOS3k 2026).

24.10 10. Career lens

One blocked recovery crosses many ownership domains. No table promises employment; roles vary by organization. LAB-TRUST-001 artifacts resemble early professional evidence in miniature: labeled observations, consent cards, and explicit uncertainty.

Role family	Typical artifacts	Review questions
Privacy engineer	Lifecycle map + consent card	Did retain/share/delete match disclosure?
Identity engineer	Authn vs authz notes; recovery path writeup	Was login confused with permission?
Accessibility engineer	WCAG-informed review note (dated key cited)	Was equivalent feedback present—not “nice UI”?
Trust & safety / ethics facilitator	Ethics ladder + AI disclosure mode note	Observation before inference?
Security engineer (UX-linked)	Least-privilege / acceptability note	Did controls exclude assistive paths?
Educator / facilitator	Rubric; misconception drill; Route F plan	Were fixtures labeled correctly?

Portfolio hint: a scrubbed consent card with **fixture** labels beats a vibes-based “it’s private because HTTPS” claim.

24.11 11. Check understanding

Concept. In one sentence, distinguish *privacy* from *security* so neither swallows the other.

Concept. In one sentence, distinguish *accessibility* from *convenience*.

Concept. In one sentence each, distinguish *authentication* from *authorization*.

System tracing. Trace a familiar permission, recovery, or delete request from intent to human feedback in numbered steps. Mark observed vs inferred. Name 4 SC-CH24 conditions that had to hold together.

Misconception check. Why can TLS succeed while a privacy lifecycle still fails?

Misconception check. Why must this chapter refuse collapsing WCAG 2.2 into a single undated citation key?

Evidence ethics. What is the difference between LAB-TRUST-001 FIXTURE_VALIDATED infrastructure and Gate 3 human reader validation?

Teach-it-back. Explain to a family member—using only LAB-TRUST-001 vocabulary—why a smooth AI answer can still be unsafe to act on, and what one control (technical or social) would make the experience more trustworthy for more people.

Researcher prompt. What additional evidence would be required to move from a classroom consent card toward a formal identity-assurance or accessibility-conformance claim—and what remains out of scope for this lab (Temoshok et al. 2025; W3C 2024)?

24.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). CE-5 chapter-local proposals and the full31 working bibliography informed key use of dual WCAG dated Recommendations, NIST SP 800-63-4, Solove (pages deferred), and WAIKE adjacency at SHA `e97e74fc9bfb44b1cdc26b272dc4848264f15fe0`. Safety/responsible-use vocabulary may reference NIST AI RMF without product certification claims. Gate 3 reader evidence remains pending; this working draft is not publication-ready.

Inline citations used in this chapter include W3C (2023), W3C (2024), Temoshok et al. (2025), Solove (2006), gunnchOS3k (2026), Saltzer and Schroeder (1975), and Tabassi (2023).

Primary inheritance (link, prefer over duplication): `publication/preproduction/ce-05/`, CE-6 accessibility-as-contract adjacency, and `labs/LAB-TRUST-001/`.

24.13 12. Glossary links

Candidate terms introduced or reinforced here (see also `publication/full31/chapters/ch24/GLOSSARY`) do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Privacy / data lifecycle	Collect → use → retain → share → delete/redact
Identity (human systems)	Claims about people/services that unlock experiences
Authentication	Proving an identity claim
Authorization	Deciding what a proven claim may do
Safety constraints	Limits that keep harmful outcomes from being easy to enact
Accessibility path	Alternate routes so experience succeeds for diverse bodies/tools
Ethics ladder	Observation before inference; disclosure and responsible use
WCAG as guidance	W3C accessibility guidelines; cite dated editions, not silent merges
Consent / trust card	Audience, purpose, classes, retention, opt-out, AI disclosure
Privacy security	Controls can hold while lifecycle still fails the person
Accessibility convenience	Faster for one path is not usable for all paths
Safety censorship	Harm-reduction limits viewpoint-censorship claims; over-refusal is a named failure
Stability Contract	Concurrent hidden conditions that keep an experience alive (book teaching model)

Related earlier chapters: CE-5 / CH21 inference and disclosure adjacency; CH23 security boundaries without replacing privacy. Related later chapters: equity deepening (CH25), evidence practice (CH27), responsibility synthesis (CH30/CH31).

24.14 Figure references (planned embeds; draft-blocked until SVG + a11y land)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured learner or lab evidence. No fabricated telemetry. No product certification claims.

24.14.1 FIG-CH24-001 — Privacy data lifecycle wheel with owners

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Lifecycle / wheel.
- **Reader should notice.** Collect → use → retain → share → delete/redact with decision owners.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name each stage and that owners differ by stage; color never sole cue; deny legal-compliance claim.

24.14.2 FIG-CH24-002 — Accessible vs blocked recovery/auth paths

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Comparative paths.
- **Reader should notice.** Security can “succeed” for one path while another body/tool is locked out.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name accessible path vs blocked path; state a11y convenience; deny WCAG certification of products.

24.14.3 FIG-CH24-003 — Ethics ladder (observation → inference → disclosure)

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Ladder.
- **Reader should notice.** Observation before inference; disclosure as a practice step.
- **Truth class.** Conceptual.
- **Alt text requirement.** Order the three rungs; state fixtures are not human evidence; deny Gate 3 PASS.

Chapter 25

Chapter 25 — Digital Equity: Who Benefits, Who Is Excluded, and What We Can Measure

Status: draft · **Chapter ID:** CH25

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS; EMIT fixtures and illustrative portfolios are teaching infrastructure, not human reader evidence).

Part V asks who is protected, who is served, and who is quietly left out. Chapter 20 already treated accessibility and equity as Stability Contract conditions. Chapter 24 widens privacy, identity, safety, and ethics. This chapter makes **digital equity** the primary measurement problem: who can complete an experience, who cannot, and which claims are observed versus assumed. It inherits CE-6 equity/accessibility seeds and LAB-CE06-001 portfolio fields—deepening measurement discipline without fabricating population studies (CLM-CH25-001).

25.1 1. The moment

Two peers attempt the same task. One finishes on a strong device and a stable link. The other stalls: cheaper hardware, metered cellular, shared computer time, denser language, or an assistive path that never announces “done.” From the author’s seat the product looks fine. From the second seat the experience never existed.

That divergence is not a personality flaw. It is a systems fact.

Digital equity, in this book’s teaching sense, means asking who can benefit from a technology experience and who is excluded—then separating what we measured from what we merely hoped. Marketing “inclusive for everyone” language is not evidence. A green connected icon is not evidence that *this person* completed *this action* under *these constraints*.

The governing question for this chapter:

Who can complete this experience—and who is excluded—and what evidence do we actually have?

This is not a census chapter, not a WCAG certification course, and not a Device Quartet marketing sheet. Official ICT access statistics exist and matter (ITU 2025); classroom portfolios still cannot invent national percentages. Accessibility standards inform intent (W3C 2024); labs do not certify products.

25.2 2. What you notice

Before naming “digital divide” numbers, notice the human contract that broke for one person and held for another.

You expected a familiar action to finish for both peers. Instead you notice: spinner longevity on one device only; success toast without durable effect on a weaker link; keyboard or screen-reader silence while the pointer path looks busy; a download that burns a metered plan; a disclosure wall written above the literacy the lab assumes; or a “required” specialized tool that one learner cannot afford.

Works-for-me is not a Stability Contract.

That distinction is the chapter’s first systems skill (CE-6 equity seed). Status chrome, author demos, and privileged lab setups answer narrow questions. Completion under declared constraints answers a different question. Collapsing them produces confident wrong inclusion claims: “everyone can do this,” “the lab is accessible,” “offline is just for beginners,” as if those were measured.

Optional commodity notice (no specialized gear): pick one familiar send/submit/sync or lab step. Attempt it once under your usual setup, then once under a *declared* constraint you already have access to—phone-only, throttled/metered link, keyboard-only, or fixture fallback. Write three columns before you invent a cause: *who completed*, *what differed in the visible experience*, *what remains unobserved*. If live comparison is unsafe, inequitable, or metered-expensive, use LAB-CE06-001 Route F and mark rows **fixture**. Fixture rows are teaching practice, not human equity evidence and not Gate validation.

25.3 3. Exploded ecosystem

Exclusion is rarely a single broken widget. It is a path through an ecosystem where one privileged configuration can hide concurrent failures. **FIG-CH25-001** is the first-minute map: same task, divergent completion across constraints. Treat it as **Representative educational architecture**, not a claim that every product fails the same way.

Walk the layers in ordinary language.

25.3.1 Human and context

Intent, time available, language, literacy, caregiving load, and classroom power shape what “acceptable” means *for this person now*. A peer who finishes quickly may simply have been granted a softer contract.

25.3.2 Device and form factor

Screen size, CPU/memory headroom, battery state, and input modalities change which paths are usable. Device Quartet form factors may appear as learning lenses only; multi-form-factor equity benches remain **PHYSICAL_PENDING** (CLM-CH25-004) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

25.3.3 Cost and ownership

Shared devices, prepaid data, no paid cloud account, and no specialized lab kit are first-class conditions—not excuses. Offline/fixture completion is an **equity feature**, not a lesser path (CE-6).

25.3.4 Network path

Association can look fine while bandwidth, latency, captive portals, or intermittent campus Wi-Fi break completion. Metered cellular changes which “just refresh” advice is ethical.

25.3.5 Application / service assumptions

Pointer-first UI, English-dense disclosures, DevTools-only diagnosis, always-on broadband, and “bring your laptop” are design choices with exclusion consequences.

25.3.6 Accessibility / alternate path

Equivalent feedback along keyboard, switch, captions, or screen-reader paths is a Stability Contract condition (SC-10), not a garnish (W3C 2024).

25.3.7 Evidence and governance

How teams label observations, what they publish as “inclusive,” and what privacy they demand from comparative notes all shape who can safely participate in measurement itself.

FIG-CH25-003 groups exclusion mechanism cards—device, cost, bandwidth, accessibility, language/literacy, policy—so readers stop treating “Wi-Fi” as a synonym for every exclusion.

25.4 4. Follow the signal

Here the “signal” is the human task’s fate across privileged and constrained seats—not a single ping. Read the sequence as a logical equity-diagnosis story. Alternate paths exist; open questions stay labeled undetermined.

1. **Declare the task.** What counts as completion for a person (not for a status icon)?
2. **Declare the route.** Strong-device / strong-link; phone-first; low-bandwidth; assistive; fixture/offline.
3. **Observe seat A.** Record status chrome *and* human outcome separately.
4. **Observe seat B (or constrained self-run).** Same task; different declared constraints.
5. **Diff the experience.** What diverged: device class, connectivity class, assistive-path status, cost/time pressure, language load?
6. **Assign exclusion mechanisms** without over-causal certainty.

- 7. **Label every datum.** Observed / inferred / illustrative / fixture / population-claim (blocked unless sourced).
- 8. **Decide the next honest move.** Measure more, qualify the claim, or redesign a constrained completion route.

25.4.1 Evidence labels for equity claims

FIG-CH25-002 and CE-6’s evidence ladder teach the same honesty:

Label	Means	Equity misuse to refuse
Observed	You saw/heard/recorded it under declared conditions	Promoting one seat into “everyone”
Inferred	A plausible explanation still needing evidence	Treating inference as measured inclusion
Illustrative	Teaching aid, not learner or population evidence	Pasting into “our users can...”
Fixture	Supplied lab data for offline/equity routes	Claiming fixture timings as peer study results
Population-claim	National/global statistical assertion	Inventing census percentages (CLM-CH25-003)

Official ICT access statistics are the right class of source for population claims when you later cite **specific tables** from a verified edition (ITU 2025). Until those table citations are in hand, keep population claims blocked in portfolios. Classroom n=1 or n=2 peer notes are *local observations*, not digital-divide epidemiology.

25.4.2 Failure branch without humiliation

Prefer exclusion *mechanisms* over blaming individuals: device class, cost, bandwidth/metering, skill/tooling assumptions, language/literacy load, disability/AT path gaps, policy/account requirements. Outside observation rarely isolates one mechanism cleanly. That limitation is literacy.

25.5 5. Component cards

For each idea: plain language, analogy (labeled), technical function, constraints, common symptoms.

25.5.1 Digital equity

- **Plain language.** Who can benefit from a technology experience and who is excluded.
- **Analogy (labeled).** Like asking whether a bridge works for walkers, wheelchairs, and night travelers—not only for the designer’s car at noon.
- **Technical function.** Centers completion and exclusion as first-class systems questions alongside latency and security.
- **Constraints.** Not a synonym for “be nice”; not automatic proof from marketing copy.
- **Symptoms.** Peer divergence; works-for-me blindness; inclusive claims without labeled evidence.

25.5.2 Exclusion mechanism

- **Plain language.** A concrete barrier family—device, cost, bandwidth, skill/tooling, language, disability/AT path, policy.
- **Analogy (labeled).** Like locked doors with different keys—naming the lock beats shouting “access problem.”
- **Technical function.** Gives diagnosis vocabulary so Improve plans target a barrier, not a stereotype.
- **Constraints.** Multiple mechanisms often co-occur; one lab run mechanism census.
- **Symptoms.** Task impossible without paid API, laptop, DevTools, fluent English disclosure, or pointer-only UI.

25.5.3 Measurable inclusion signals

- **Plain language.** Observations you can collect without inventing population statistics.
- **Analogy (labeled).** Like a checklist of whether the door opened for *these* people *today*—not a national housing survey.
- **Technical function.** Operationalizes equity as recordable fields: completion yes/no, device/connectivity class, assistive-path status, route used, evidence labels.
- **Constraints.** Local signals national ICT indicators (ITU 2025).
- **Symptoms.** Portfolios full of vibes (“pretty inclusive”) with no completion rows.

25.5.4 Constrained completion route

- **Plain language.** A path that works without specialized hardware or paid APIs—often phone-first, low-bandwidth, or fixture/offline.
- **Analogy (labeled).** Like a marked accessible entrance—not a service elevator treated as second-class.
- **Technical function.** Makes equity actionable in labs: Route F / fixture fallback is first-class (CE-6; CLM-CH25-002 adjacency to WAIKE phone-first / low-cost intent) (gunn-chOS3k 2026; “Waike-Research-Ops Accepted Main Curriculum Evidence” 2026).
- **Constraints.** A route’s existence is not WCAG certification (W3C 2024).
- **Symptoms.** “Optional offline” framed as failure; labs that cannot finish without paid cloud.

25.5.5 Honest evidence labeling

- **Plain language.** Distinguishing observed / inferred / illustrative / fixture / population-claim so equity language cannot launder uncertainty.
- **Analogy (labeled).** Like food labels—ingredients listed beat “probably healthy.”
- **Technical function.** Extends observation-vs-inference craft into inclusion claims (CLM-CH25-001).
- **Constraints.** Labels do not replace ethics; redaction still required for peer comparisons.
- **Symptoms.** Fixture examples cited as human validation; Gate infrastructure confused with Gate PASS.

25.5.6 Accessibility path (contract condition)

- **Plain language.** Equivalent feedback and completion along non-pointer / assistive routes.
- **Analogy (labeled).** Like captions that carry the same punchline—not a silent film for some seats.

- **Technical function.** SC-10 in the Stability Contract list; WCAG 2.2 informs teaching intent only (W3C 2024).
- **Constraints.** Intent product certification; Explorer path must work without DevTools (CE-6).
- **Symptoms.** Color-only status; completion never announced to AT; timing tasks with no extended-time option.

25.6 6. Stability contract

Definition (publication teaching model): a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

Signature distinction for this chapter: a system can remain “fine on the author’s setup” while the human experience has already **failed for someone else**.

25.6.1 Equity-specific contract conditions (chapter focus)

Inherit CE-6 concurrent conditions (SC-01...SC-11), then emphasize these measurable inclusion conditions for CH25:

Condition	Honest test question
Task completion under constrained device/network/assistive conditions	Did a declared constrained route finish for a person?
Cost and bandwidth barriers within declared lab routes	Was paid cloud / specialized kit / heavy download required?
Language/literacy of disclosures usable	Could a newcomer act on the disclosure without decoding jargon walls?
Evidence labels prevent overclaiming “inclusive for all”	Is every inclusion sentence tagged observed / inferred / illustrative / fixture?
Accessibility path equivalent feedback	Was completion perceivable without pointer-only cues (W3C 2024)?
Privacy of comparative evidence	Are peer identifiers redacted before share?

Numeric humility rule: do **not** invent inclusion percentages, “accessibility scores,” or Device Quartet marketing curves. Official statistics belong only with verified table citations (ITU 2025). Qualitative language is required where unmeasured.

25.6.2 CE-6 + LAB-CE06-001 linkage

- **CE-6 preproduction** (publication/preproduction/ce-06/), especially SECURITY_EQUITY_ACCESSIBILITY and the Stability Contract model, is the primary inheritance package: equity as contract condition, offline/fixture as equity feature, phone-first Explorer routes, and Improve-plan “who benefits / who is left out?” checks.
- **LAB-CE06-001** supplies publication-owned EMIT portfolio fields including portfolio/equity_social. Status on accepted infrastructure: **FIXTURE_VALIDATED**. That means structural templates and validators exist. It does **not** mean Gate 3 PASS. It does **not** mean illustrative EMIT examples are human equity evidence (CLM-CH25-001).

Honesty bound for this edition: Device Quartet multi-form-factor equity measurements remain **PHYSICAL_PENDING** (CLM-CH25-004) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). WAIKE **ACCESSIBILITY_AND_LOW_COST.md** is adjacent intent evidence—not ally certification and not checklist-complete proof (CLM-CH25-002) (gunnchOS3k 2026; “Waike-Research-Ops Accepted Main Curriculum Evidence” 2026).

25.7 7. Try it

25.7.1 LAB-CE06-001 — Equity fields inside EMIT (inherit)

Goal. Reuse the publication-owned EMIT capstone to practice equity measurement: explain who completed, measure what is observable under declared constraints, propose one inclusion-minded Improve, and teach the difference between observed exclusion and assumed universality.

Proposed pair lab. LAB-CH25-PAIR-001 (paired constrained-route worksheet) remains **proposed** in the chapter packet—do not invent a WAIKE ID for it. Until authored, complete equity practice inside LAB-CE06-001.

WAIKE alignment note. WAIKE accepted main (SHA e97e74fc9bfb44b1cdc26b272dc4848264f15f) includes adjacent docs and labs (**ACCESSIBILITY_AND_LOW_COST.md**, **COMM_PD_ETHICS / lab_accessibility_comm**, capstone portfolio culture). Those are agencies. There is **no** exact WAIKE module named **DIGITAL_EQUITY / CH25**.

Safety (hard stops).

- No passwords, tokens, private messages, health data, or classmate PII in portfolios; redact peer comparisons.
- No rooting, jailbreaking, unauthorized scanning, social-engineering classmates, or attacking systems.
- Prefer owned devices, benign demos, or supplied fixtures; heed metered-data warnings.
- No Device Quartet / specialized RF / EVT hardware required or requested.

Routes.

- **Route A — Dual-seat notice.** Same task; two constraint classes you already can access (or self + one constrained re-run). Log device class, connectivity class, assistive-path status, completion yes/no.
- **Route B — Status vs outcome.** Keep CH20 discipline: OS/connectivity chrome *separate* from human completion.
- **Route F — Fixture fallback (mandatory offline/equity path).** Use LAB-CE06-001 fixtures; mark **fixture**. Study illustrative examples only as **ILLUSTRATIVE ONLY — not human evidence**.

Explorer baseline.

1. Name who succeeded/failed and what differed in the visible experience.
2. Fill **equity_societal_impact.md**: assumption → who might be excluded → mitigation idea.
3. Label every row observed / inferred / fixture.
4. Refuse any invented population percentage.
5. Produce teach-back a peer could use: “works on my machine inclusive.”

Operator extension. Log device class, connectivity class, and assistive-path status as observations for 2 declared routes; add one comparison table.

Builder extension. Add or document one low-bandwidth / no-specialized-hardware completion route for a toy step; note tradeoffs.

Engineer extension. Propose one measurable inclusion metric with explicit limitations (no fake benchmarks); place it on the evidence hierarchy.

Researcher extension. Design a small comparative study plan separating observation from inference; list confounders; cite what official ICT statistics would require that this lab does not provide (ITU 2025). Keep Quartet claims PHYSICAL_PENDING.

Educator facilitation. Prefer Route F when live dual-seat capture is inequitable or unsafe; treat fixture completion as first-class.

Evidence to keep. Equity portfolio fields; scrubbed comparison notes; teach-back. Validator discipline from LAB-CE06-001 applies. Completion means artifact-backed claims—not a bare exit code and not the string PASS.

25.8 8. Build it

Extend equity practice without turning Part V into invented benchmarks.

25.8.1 Explorer

Build a pocket card: *works-for-me inclusive*, plus five exclusion mechanism names in plain sentences.

25.8.2 Operator

Build a three-column sheet: constraint class | completion observation | evidence label. No fourth column for vibes.

25.8.3 Builder

Build a one-page constrained-route stub: phone-first or fixture steps, predicted who benefits / who is still left out, and qualitative success criteria you could observe without specialized kit. Align with WAIKE phone-first / low-cost *intent* without claiming certification (gunnchOS3k 2026).

25.8.4 Engineer

Build an inclusion-metric proposal that ends in limitations: sample size, selection bias, AT coverage gaps, and “needs official statistics / specific ITU table citation before population language” (ITU 2025). Optional: map metric to SC-10 / cost / bandwidth conditions.

25.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to make yet—for example, “this lab is accessible to all students nationwide” or “Device Quartet proves inclusive hardware.” Specify what physical or statistical evidence would be required; keep Quartet **PHYSICAL_PENDING** (CLM-CH25-004). Do not invent DOIs or census numbers.

Educators can facilitate Section 11 teach-backs and keep Route F as a first-class equitable path—not a lesser path.

25.9 9. Secure and include it

25.9.1 Security

Comparative screenshots, HAR exports, and retry logs can leak tokens and identifiers. Prefer fixtures and redaction. Do not capture others' screens without consent. Unauthorized scanning is out of scope.

25.9.2 Privacy

Peer equity comparisons are sensitive. Scrub names, faces, handle strings, message previews, and precise location. Distinguish measurement-for-learning from surveillance. Metered-link learners must not be forced into expensive live captures—Route F exists for equity and privacy together.

25.9.3 Accessibility

Document whether completion was announced along assistive paths; do not rely on color-only status. WCAG 2.2 informs teaching intent—not a claim that this book, LAB-CE06-001, or any gunnchOS product is certified (W3C 2024). Timing tasks allow extended time; avoid flicker-heavy demos. Explorer diagnosis must work without DevTools (CE-6).

25.9.4 Equity (primary depth)

Strong-device / strong-link authors can falsely conclude the contract is fine. Metered data, shared devices, older hardware, intermittent connectivity, language load, and missing AT equivalence change acceptable bounds. Offline/fixture fallback is an equity feature. Improve plans that boost an author's metrics while excluding people fail the inclusion test even if a spinner got shorter. Name who benefits and who is left out when “optimization” assumes one privileged setup. Device Quartet analogies stay optional and non-required (PHYSICAL_PENDING).

25.9.5 Safety

Commodity devices only. No RF transmit experiments, no battery abuse, no jailbreaks, no social-engineering classmates “to test equity.”

25.9.6 Ethics

Do not present illustrative fixture numbers as measured human evidence. Do not present `FIXTURE_VALIDATED` as Gate 3 PASS. Do not invent inclusion percentages, ally certification badges, or Quartet EVT inclusivity curves. Overclaiming measurement is still false evidence—and it harms the people your claim pretended to include.

25.10 10. Career lens

Equity work crosses product, research, policy, and education. No table promises employment; roles vary by organization. LAB-CE06-001 equity fields resemble early professional evidence in miniature: labeled observations, constrained-route notes, and explicit uncertainty.

Role lens	Typical artifacts	Review questions
Digital equity researcher / policy analyst	Exclusion measurement plan; sourced statistical citations	Are population claims table-cited or blocked?
Product inclusion lead	Constrained-route completion evidence; Improve tradeoff notes	Did “ship faster” erase a route?
Accessibility specialist	AT pathway writeup; SC-10 evidence	Equivalent feedback—or color-only chrome?
SRE / performance	Connected usable notes under multiple constraint classes	Whose latency budget was optimized?
Educator / facilitator	No-device / fixture adaptation note; misconception prompts	Is Route F treated as first-class?
Trust & safety / privacy	Redaction checklist for comparative studies	Did measurement become surveillance?

Portfolio hint: a scrubbed two-seat completion table with evidence labels is more honest than a vibes-based “our product is inclusive” claim.

25.11 11. Check understanding

Concept. In one sentence, define *digital equity* so that it requires asking who is excluded—not only who is delighted.

Concept. Name three *exclusion mechanisms* and one measurable signal you could record for each without inventing population statistics.

System tracing. Trace the same task for two constraint classes in numbered steps. Mark observed vs inferred. Name 4 Stability Contract / inclusion conditions that had to hold together.

Misconception check. Why is “it works on my laptop on fast Wi-Fi” not evidence that a lab is inclusive?

Misconception check. Why must this chapter refuse invented census percentages even while pointing readers at ITU Facts and Figures (ITU 2025)?

Evidence ethics. What is the difference between LAB-CE06-001 FIXTURE_VALIDATED infrastructure and Gate 3 human reader validation? Why are illustrative EMIT portfolios not human equity evidence?

Teach-it-back. Explain to a newcomer why a constrained completion route (fixture/offline/phone-first) is an equity feature, not a lesser path.

Researcher prompt. What additional evidence would be required to move from a two-seat classroom observation to a responsible population-level ICT access claim—and what remains out of scope for this lab (ITU 2025)?

25.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). CE-6 chapter-local proposals informed promotion of `itu-facts-figures-2025` for CH25. Project-specific Device Quartet equity status remains **PHYSICAL_PENDING** in the chapter claim plan (CLM-CH25-004). Gate 3 reader evidence remains pending; this working draft is not publication-ready.

Inline citations used in this chapter include ITU (2025), W3C (2024), `gunnchOS3k` (2026), “Waike-Research-Ops Accepted Main Curriculum Evidence” (2026), and “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026).

Primary inheritance (link, prefer over duplication): `publication/preproduction/ce-06/` (especially `SECURITY_EQUITY_ACCESSIBILITY.md`, `STABILITY_CONTRACT.md`) and `labs/LAB-CE06-001/` (especially `portfolio/equity_societal_impact.md`).

25.13 12. Glossary links

Candidate terms introduced or reinforced here (see also chapter glossary candidates; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Digital equity	Who can benefit from a technology experience and who is excluded
Exclusion mechanism	Device, cost, bandwidth, skill, language, disability, or policy barrier
Measurable inclusion signals	Observations collectable without inventing population stats
Constrained completion route	Path that works without specialized hardware or paid APIs
Honest evidence labeling	Observed / inferred / illustrative / fixture / population-claim distinctions
Accessibility path	Equivalent feedback/completion along assistive or non-pointer routes
Stability Contract	Concurrent hidden conditions that keep an experience alive (book teaching model)
Works-for-me inclusive	Author-seat success does not entail completion for others
Fixture as equity feature	Offline/supplied-data routes that keep constrained learners included
Observation vs inference	What happened vs what may explain it

Related earlier chapters: experience-first observation craft (CH02/CH03), QoE and Stability Contract depth (CH20), privacy/identity/safety/accessibility ethics (CH24), CE-6 Concept Edition seeds. Related later chapters: evidence practice (CH27), careers and contribution (CH30), EMIT / responsibility capstone (CH31).

25.14 Figure references (planned embeds; draft-blocked until SVG + a11y land)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured learner or lab evidence. No fabricated telemetry. No invented inclusion percentages. No Device Quartet marketing curves.

25.14.1 FIG-CH25-001 — Same task; divergent completion

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Comparative (planned).
- **Reader should notice.** Same task; divergent completion across constraints.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name both seats and at least one differing constraint; state conceptual truth class; deny population statistics.

25.14.2 FIG-CH25-002 — Evidence labels for equity claims

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Hierarchy (planned).
- **Reader should notice.** Observed / inferred / illustrative / fixture / population-claim ladder.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name each label and one misuse to refuse; color never sole cue.

25.14.3 FIG-CH25-003 — Exclusion mechanism cards

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Map / card set (planned).
- **Reader should notice.** Device, cost, bandwidth, accessibility, language (and related) mechanism families.
- **Truth class.** Illustrative.
- **Alt text requirement.** List mechanism card titles in text; deny that the set is exhaustive census evidence.

25.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH25-001.** CE-6 / LAB-CE06-001 plant equity and accessibility as Stability Contract conditions; CH25 deepens measurement without fabricating population studies—do not treat illustrative EMIT examples as human equity evidence.
- **CLM-CH25-002.** WAIKE ACCESSIBILITY_AND_LOW_COST.md documents phone-first / low-cost intent (adjacency); checklist completeness and a11y certification must not be over-claimed (gunnchOS3k 2026; “Waikē-Research-Ops Accepted Main Curriculum Evidence” 2026).

- **CLM-CH25-003.** Primary digital-divide / ICT access statistics for national or global claims require verified official sources; cite specific ITU Facts and Figures tables before quoting numbers—do not invent census percentages (ITU 2025).
- **CLM-CH25-004.** Device Quartet multi-form-factor equity measurements are **PHYSICAL_PENDING**; form factors may be learning lenses only—not shipping inclusivity marketing (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Chapter 26

Chapter 26 — Software Development and Version Control

Status: draft · **Chapter ID:** CH26

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS; fixtures and illustrative worksheets are teaching infrastructure, not human reader evidence).

Part VI opens the build-prove-contribute arc. Earlier chapters taught you to notice experiences, name cooperating layers, and keep evidence honest. This chapter turns that literacy into a small, reviewable change loop: edit, inspect, commit, show what differed—and recover without mythology.

26.1 1. The moment

You change one behavior.

Maybe you fix a typo in a README, rename a confusing label, tighten a checklist, or adjust a tiny function so a test passes. The file looks different on your screen. Then someone asks the adult question: *what exactly changed, and can I trust that story later?*

From the seat: either confidence or fog.

Underneath: **version control**—recording changes so history can be inspected and recovered (Git Project 2026) (CLM-CH26-001). **Version control backup:** a commit history helps you inspect and carefully recover *tracked* states, but it is not a backup policy—untracked files, whole-machine loss, and off-site copies remain separate concerns. A **commit** is a snapshot with a message and identity metadata. A **diff** is the readable difference. A **branch** is a parallel line of work that must later integrate carefully. None of those words are slogans about “being a developer.” They are tools for making a change *reviewable*.

The governing question for this chapter:

After I change one behavior, can I show what differed, leave a clear message, and recover from a mistake without rewriting history myths?

This is Part VI's software-pathway opener (CH01→CH02→...→CH26→CH27). It is not a full Git encyclopedia, not an employment promise, and not a Device Quartet shipping claim.

26.2 2. What you notice

Before naming remotes or rebase strategies, notice the human contracts you already expect.

You expect to know whether your work is saved as history or still only local edits. You expect a teammate—or your future self—to see *what* changed without replaying your whole afternoon. You expect a mistake to be recoverable without panic folklore (“just force it”). You expect secrets (tokens, passwords, private keys) never to become permanent history. You expect a clear message to beat a vague “fixes.”

Those expectations *are* the development loop from the person's point of view.

Status and diff are observations about change; “I’m done” is a claim that needs evidence.

Optional commodity notice (no paid IDE required): open any small text project you already may edit—or a fixture worksheet if you prefer offline. Make one harmless change. Before you “save forever,” write two columns: *what I think I changed* and *what status/diff actually lists*. If tools are unavailable, use a printed before/after pair and mark the row **fixture**. Fixture worksheets are teaching aids, not Gate 3 human validation.

26.3 3. Exploded ecosystem

A single commit is not a lone object. It is a path through an ecosystem. **FIG-CH26-001** is the first-minute map: edit → status → diff → commit → review. Treat it as **Representative educational architecture**, not a claim that every team uses identical tools or hosting vendors.

26.3.1 Human and intent

You decide what “better” means for this change: clearer text, safer behavior, a passing check. Intent sets the review question.

26.3.2 Working tree

Files as they exist on disk right now—including uncommitted edits. The working tree can disagree with history (Git Project 2026).

26.3.3 Staging / index (when used)

The optional “what I intend to include next” set. Confusion here produces commits that omit the file you thought you saved or include a file you meant to leave out.

26.3.4 Committed history

Snapshots linked as history. **FIG-CH26-002** separates working tree vs committed history so readers stop treating “I edited a file” as “it is in history.”

26.3.5 Message and identity metadata

Who recorded the snapshot, when, and what they claimed it meant. Messages are part of the evidence surface.

26.3.6 Branch / integration path

Parallel lines of work and the careful combine later. Integration is a social and technical event—not automatic correctness.

26.3.7 Review surface

Humans inspect diffs before integration. Review is literacy, not theater.

26.3.8 Build-test loop

Change → verify → evidence before claiming done. Chapter 27 deepens testing and observability; here the loop is the minimum honesty check.

26.3.9 Secrets boundary

Credentials and tokens must not enter history. **FIG-CH26-003** makes the refuse-line visible.

26.3.10 Project evidence neighbors (adjacency only)

WAIKE’s `SOFTWARE_BUILDER` package is an **adjacent** builder competency neighbor—not a CH26 module ID and not an invented course label (gunnchOS3k 2026) (CLM-CH26-002). DS-XL Coder may appear later as a learn-to-build lens only; any measured local-build hardware claim remains **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026) (CLM-CH26-003).

26.4 4. Follow the signal

Here the “signal” is one reviewable change traveling from intent into inspectable history.

1. **Intent.** You choose one small behavior to change.
2. **Edit.** The working tree diverges from the last snapshot.
3. **Status.** You ask what the tool believes is modified, untracked, or staged (Git Project 2026).
4. **Diff.** You read the actual line-level (or file-level) difference before freezing it (Git Project 2026).
5. **Secrets scan (human).** You refuse tokens, passwords, and private keys—even in “temporary” files.
6. **Commit.** You record a snapshot with a clear message (Git Project 2026).
7. **Verify.** You run the relevant check, test, or checklist for *this* change.
8. **Review.** You (or a peer) inspect the diff as if you did not write it.

9. **Integrate carefully.** Branch work combines without pretending history myths fix mistakes.
10. **Evidence keep.** You can point to a SHA, a message, and a scrubbed artifact—not a vibe.

26.4.1 Alternate paths stay labeled

GUI clients, accessible front-ends, and offline fixture worksheets can complete the same literacy when a CLI is unavailable. The pedagogy is the loop, not a brand of hosting. Open questions (exact team policy, required review tools) stay undetermined until evidence exists.

26.4.2 Version numbers when APIs are public

When a change crosses a public API boundary, **Semantic Versioning** offers MAJOR.MINOR.PATCH compatibility vocabulary: incompatible API changes, backward-compatible additions, and backward-compatible fixes (Preston-Werner, n.d.). That vocabulary is optional depth here—use it when you claim compatibility, not as decoration on every typo fix.

26.5 5. Component cards

For each idea: plain language, analogy (labeled), technical function, constraints, common symptoms.

26.5.1 Version control

- **Plain language.** Recording changes so history can be inspected and recovered—not a synonym for backup.
- **Analogy (labeled).** Like keeping dated drafts of an essay instead of one overwritten file—so you can see what changed between versions. That is not the same as storing a full off-site copy of the machine.
- **Technical function.** Stores snapshots and relationships so diffs and recovery of tracked history are possible (Git Project 2026).
- **Constraints.** Tool literacy required; policies differ by team; untracked files and disk failure are outside VCS alone; history that includes secrets is hard to truly un-share.
- **Symptoms.** “I thought I saved it,” “I can’t show what changed,” “we lost the good version,” “Git is my only backup.”

26.5.2 Commit

- **Plain language.** A snapshot of change with message and identity metadata.
- **Analogy (labeled).** Like sealing a labeled envelope of today’s draft—not the whole library rebuild.
- **Technical function.** Creates a recoverable history node with a message (Git Project 2026).
- **Constraints.** A commit is not automatic proof of correctness; huge unrelated dumps hurt review.
- **Symptoms.** Empty messages, kitchen-sink commits, “fixed stuff” with no readable scope.

26.5.3 Diff

- **Plain language.** The readable difference between states.
- **Analogy (labeled).** Like a red-pen markup showing additions and deletions.
- **Technical function.** Surfaces what will enter (or already entered) history (Git Project 2026).
- **Constraints.** Binary files may not show useful text diffs; generated noise can hide intent.
- **Symptoms.** Surprising files in the change set; “I didn’t mean to include that.”

26.5.4 Branch / integration

- **Plain language.** Parallel lines of work later combined carefully.
- **Analogy (labeled).** Like two co-authors editing copies, then merging with a checklist—not silently overwriting each other.
- **Technical function.** Names lines of development and integration points (Git Project 2026).
- **Constraints.** Integration conflicts need human judgment; rewriting shared history has collaboration costs.
- **Symptoms.** Lost work after a careless overwrite; fear of “touching main.”

26.5.5 Code review

- **Plain language.** Human inspection of diffs before integration.
- **Analogy (labeled).** Like a second reader checking a proof before publication.
- **Technical function.** Catches intent mismatches, missing tests, and secrets before they spread.
- **Constraints.** Review without time or norms becomes rubber-stamping.
- **Symptoms.** “LGTM” with no diff read; surprises after merge.

26.5.6 Secrets hygiene

- **Plain language.** Credentials and tokens must not enter history.
- **Analogy (labeled).** Like not laminating your house key into the scrapbook—you cannot truly take it back from every copy.
- **Technical function.** Keeps sensitive material out of commits, logs, and portfolio screenshots.
- **Constraints.** Once pushed widely, assume exposure; rotation and revocation become necessary.
- **Symptoms.** Keys in README samples; `.env` committed “just once.”

26.5.7 Build-test loop

- **Plain language.** Change → verify → evidence before claiming done.
 - **Analogy (labeled).** Like tasting the soup after you salt it—before serving guests.
 - **Technical function.** Ties commits to observable checks appropriate to the change size.
 - **Constraints.** No test suite does not excuse skipping a checklist; fixtures → production proof.
 - **Symptoms.** “It worked on my machine” with no artifact; green CI misunderstood as full correctness.
-

26.6 6. Stability contract

A reviewable change experience exists only while several conditions remain within acceptable bounds.

Condition	Plain meaning	Failure symptom
Working-tree understandability	Status/diff are readable for this change	Surprise files; unknown dirty state
History integrity practices	Collaboration-appropriate honesty about shared history	Lost peer work; panic rewrites
Secrets never committed	Tokens/passwords/keys stay out	Credential exposure in history
Build/test loop runnable	Commodity hardware or fixtures can verify	“Done” with no check path
Message clarity	Future readers can see intent	Undebuggable archaeology
Accessible tool path	CLI <i>or</i> GUI/fixture route exists	Equity exclusion by tool assumption
Observation vs inference	What the tool showed vs what you guess	Confident wrong blame

A repository can be “technically present” while the human experience has already failed: nobody can tell what changed, secrets leaked, or verification never ran.

26.7 7. Try it

Inherited adjacency. Builder pathway habits from Concept Edition packages and WAIKE SOFTWARE_BUILDER remain **adjacent**—link, do not invent a CH26 WAIKE lab ID (gunnchOS3k 2026) (CLM-CH26-002).

INLINE_ACTIVITY (chapter-native). The safe commit/review worksheet below *is* the learning activity. The namespaced idea LAB-CH26-GIT-001 is **not** a shipped **labs/** package and must not be minted as a WAIKE course code. Fixture / offline routes remain first-class.

Living example without marketing. This publication repository itself is a real Git history you can read as a learner: status, diff, commit messages, and SHAs are ordinary evidence surfaces. Reading history is not permission to invent product claims.

26.7.1 Explorer baseline (about 40–50 minutes)

1. Predict, in ordinary language, what “commit,” “branch,” and “diff” mean before touching tools.
2. Make one tiny safe edit (or use a fixture before/after pair).
3. Run or simulate **status**, then **diff**—write what you observe (Git Project 2026).
4. Draft a clear commit message *before* recording (even on paper for fixtures).
5. Fill observation vs inference: what the tool showed vs what you assume about “done.”
6. Teach-back: explain the loop to a family member without saying “rebase.”

26.7.2 Operator extension

Read a status/diff and say what will change **before** running a mutating command. Practice stopping when a surprise file appears.

26.7.3 Builder extension

Make a small change with a test note or checklist evidence. Prefer PR-sized scope: one intent, one message, one verification note.

26.7.4 Engineer extension

Propose a reviewable change with rationale: why this diff, what could break, what evidence would convince a reviewer.

26.7.5 Researcher extension

Document reproducibility of the change environment at a SHA (tool versions you actually used; unknowns labeled). Do not invent Git behavior beyond official docs (Git Project 2026).

26.7.6 Safety / privacy / accessibility

- No exploit steps, credential harvesting, or capture of others' private data.
- Redact identifiers before portfolio share.
- Provide fixture / no-specialized-hardware completion routes.
- Prefer accessible CLI *or* GUI paths; do not assume a paid IDE (W3C 2024).

26.8 8. Build it

Extend the Try-it loop by one honest notch—not by boiling the ocean.

1. **Pick one behavior** with a human-visible outcome (label clarity, checklist item, tiny logic fix).
2. **Write the review question** you want a stranger to answer from your diff alone.
3. **Keep the change small** enough that a peer can finish the review in one sitting.
4. **Attach evidence:** scrubbed status/diff excerpt, message text, and pass/fail note for your check.
5. **Optional compatibility note:** if you touch a public API, state whether SemVer MAJOR/MINOR/PATCH thinking applies—or explicitly say “not a public API change” (Preston-Werner, n.d.).
6. **Refuse scope creep:** new features, refactors, and secret cleanup are separate commits when possible.

Success looks like: another learner can restate your intent from the message + diff without asking you to narrate your afternoon.

Failure modes to name: kitchen-sink commits; “WIP” forever; screenshots that leak tokens; claiming employment readiness from one worksheet.

26.9 9. Secure and include it

26.9.1 Secrets

Never commit credentials, API tokens, private keys, or live customer data. Treat “I’ll remove it in the next commit” as insufficient once history is shared. Portfolio packets get redaction passes.

26.9.2 Inclusive tooling

Do not assume a paid IDE, a particular OS, or unbroken high-speed networking. Offline fixtures and GUI clients are first-class equity routes. Color must never be the sole cue in status diagrams (W3C 2024).

26.9.3 Collaboration ethics

Rewriting shared history without agreement can erase peers’ work. Prefer honest recovery practices appropriate to your collaboration setting. “Force it” is not a teaching goal here.

26.9.4 Device and hardware honesty

DS-XL Coder and other Device Quartet form factors remain **PHYSICAL_PENDING** for measured local-build claims—learning lenses only, not shipping-product marketing (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026) (CLM-CH26-003).

26.9.5 Evidence ethics

Fixtures and illustrative worksheets are not human Gate 3 validation. Do not present proposed labs as shipped. Do not invent WAIKE IDs.

26.10 10. Career lens

Version control literacy shows up across software roles. **Completing this chapter’s artifacts does not guarantee employment, promotion, or a job offer.** Portfolio evidence demonstrates that you can produce reviewable change; hiring decisions belong to organizations and remain outside this book’s promises.

Role family	Portfolio evidence (miniature)	Review questions
Software engineer	Small PR-sized change + test/checklist note	Is the diff reviewable in one sitting?
Release engineer	SHA-pinned change evidence	Can someone reproduce <i>which</i> snapshot?
Tech lead / reviewer	Review checklist artifact	Did review catch secrets and scope?
Educator / mentor	Misconception log + teach-back notes	Can learners separate status from “done”?

Prefer a scrubbed diff + clear message over a slogan résumé line about “passionate coding.”

26.11 11. Check understanding

Concept. In one sentence each, explain *commit*, *branch*, and *diff* so none swallows the other two.

Concept. Why is the working tree not the same thing as committed history?

System tracing. Trace one small change from intent → status → diff → commit → review in numbered steps. Mark observed vs inferred.

Misconception check. Why is “I’ll delete the secret in the next commit” not good enough once history is shared?

Misconception check. Why does a clear commit message matter to someone who was not in the room?

Evidence ethics. What is the difference between a proposed publication git worksheet, an adjacent WAIKE SOFTWARE_BUILDER package, and Gate 3 human reader validation?

Career boundary. Why is portfolio evidence not an employment promise?

Teach-it-back. Explain to a newcomer—without saying “rebase” or “detached HEAD”—how to show what changed after editing one file.

Researcher prompt. What metadata would you record to make a change environment reproducible at a SHA, and what remains out of scope without primary sources (Git Project 2026)?

26.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Official Git documentation is the preferred primary reference for core VCS concepts (CLM-CH26-001) (Git Project 2026). Semantic Versioning is cited only where compatibility vocabulary is claimed (Preston-Werner, n.d.). Project-specific adjacency uses repository evidence keys such as `gunnchOS3k` (2026) and “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026) with **PHYSICAL_PENDING** where required (CLM-CH26-002, CLM-CH26-003).

Inline citations used in this chapter include Git Project (2026), Preston-Werner (n.d.), `gunnchOS3k` (2026), “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026), and W3C (2024).

Primary packet (link, prefer over duplication): `publication/full31/chapters/ch26/`. Gate 3 reader evidence remains pending; this working draft is not publication-ready and does not modify `publication/gates/gate-3/`.

26.13 12. Glossary links

Candidate terms introduced or reinforced here (see also `GLOSSARY_CANDIDATES.yaml`; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Version control	Recording changes so history can be inspected and recovered; backup
Commit	Snapshot of change with message and identity metadata
Diff	Readable difference between states
Branch / integration	Parallel lines of work later combined carefully
Code review	Human inspection of diffs before integration
Secrets hygiene	Credentials/tokens must not enter history
Build-test loop	Change → verify → evidence before claiming done
Working tree	Files on disk now, including uncommitted edits
SHA	Identifier for a specific snapshot in history
Semantic versioning	MAJOR.MINOR.PATCH compatibility vocabulary for public APIs
Stability Contract	Concurrent conditions that keep a reviewable-change experience alive
Observation vs inference	What the tool showed vs what may explain it

Related earlier chapters: experience-first method (CH02), applications/APIs and SemVer adjacency (CH14), containers/cloud placement (CH15). Related later chapters: testing and evidence (CH27), product design (CH29), career maps (CH30), capstone (CH31).

26.14 Figure references (planned embeds; draft-blocked until SVG + a11y land)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured learner evidence. No fabricated telemetry. Color never sole cue.

26.14.1 FIG-CH26-001 — Edit → status → diff → commit → review

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Sequence.
- **Reader should notice.** The reviewable-change loop as ordered literacy, not vendor marketing.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name each stage in order; deny that every team hosts identically.

26.14.2 FIG-CH26-002 — Working tree vs committed history

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Comparison.
- **Reader should notice.** Edited files are not automatically history.
- **Truth class.** Conceptual.
- **Alt text requirement.** Contrast “on disk now” vs “recorded snapshot”; shape + label encoding.

26.14.3 FIG-CH26-003 — Secrets must not enter the repository

- **Production status.** draft-blocked (no SVG embed in this draft).
 - **Type.** Boundary.
 - **Reader should notice.** The refuse-line for credentials/tokens/keys.
 - **Truth class.** Conceptual.
 - **Alt text requirement.** State the boundary explicitly; color never sole cue.
-

26.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH26-001.** Git official documentation is the preferred primary reference for core VCS concepts (Git Project 2026).
- **CLM-CH26-002.** WAIKE SOFTWARE_BUILDER is adjacent builder competency—not a CH26 module ID (gunnchOS3k 2026).
- **CLM-CH26-003.** DS-XL Coder learn-to-build form factor remains **PHYSICAL_PENDING** for measured local-build claims (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Chapter 27

Chapter 27 — Testing, Observability, and Evidence

Status: draft · **Chapter ID:** CH27

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS; fixtures, simulations, and illustrative portfolios are teaching infrastructure, not human reader evidence).

Part VI asks builders to prove what they claim. Chapter 20 already taught concurrent conditions and an evidence hierarchy. Chapter 26 already covered reviewable change history. This chapter names the remaining honesty problem: **a green test, a green dashboard, and a green simulation can all be true while the human experience is still false—or unmeasured.** Simulation is not measurement. A fixture is not a reader study. An OpenTelemetry signal vocabulary is not a fake product SLO.

The signature distinction for this chapter:

Simulation measurement. Tests check claimed behavior under deliberate cases. Observability lets you ask new questions of a running system. Evidence is what remains after you label what was observed, what was inferred, and what was only illustrated.

27.1 1. The moment

A feature “passes” locally. Unit tests are green. The staging checklist is checked. Status dashboards look calm—until a classmate, a customer, or you on a slower link see stalls, silent failures, or a success toast that never finished the remote work.

From the seat: contradiction between **test theater** and **lived outcome**.

Underneath: mismatched oracles, missing signals, and over-promoted simulations. A test that asserts “HTTP 200” does not assert “the person finished with acceptable delay and feedback.” A dashboard panel that plots a synthetic load profile does not measure your classroom device. A

fixture portfolio that trains observation/inference labeling is teaching infrastructure—not Gate 3 human validation (CLM-CH27-004).

The governing question:

When tests and dashboards look fine while people still suffer, what was tested, what was observed, and what was only simulated or inferred?

This chapter expands CE-6’s evidence hierarchy and observation-vs-inference craft into full-book testing and observability literacy. It is not an observability product manual, not a carrier QoE campaign, and not a license to invent benchmarks as human evidence (“Reference Guide to Quality of Experience Assessment Methodologies” 2016).

27.2 2. What you notice

Before naming traces or assertion libraries, notice the human contract that broke.

You expected a change to be safe because something “passed.” Instead you notice: local green / remote red; green metrics beside abandoned actions; a replay or demo that works while live traffic does not; a log flood that still fails to answer “did this person finish?”; or a portfolio that copies fixture timings as if they were measured on your device.

A test result is an observation about the test.

A log line or metric sample is an observation about a signal.

A causal story is an inference until more evidence arrives.

Collapsing those three produces confident wrong blame: “tests prove it works,” “the dashboard proves users are fine,” “the simulation proves the design.” Each may be useful; none automatically becomes the others.

Optional commodity notice (no specialized gear): pick one recent change you made—or one familiar send/submit/sync. Write three columns—*test or checklist outcome*, *signal or status observed*, *human outcome*—before you invent a cause. If live capture is unsafe, offline, or metered, use LAB-CE06-001 Route F and mark rows **fixture**. Fixture timings are illustrative teaching data, not your measured evidence and not Gate validation.

27.3 3. Exploded ecosystem

Evidence about an experience is not a single object. It is a path through layers that produce tests, signals, and claims. **FIG-CH27-001** is the evidence hierarchy ladder: illustrative aid → commodity observation → instrumentation → correlated multi-signal inspection → controlled comparison → standards-aligned QoE methods. Treat the ladder as **Representative educational architecture**. Learner labs sit mid-ladder; they are not ITU-T G.1011 assessment campaigns (CLM-CH27-001) (“Reference Guide to Quality of Experience Assessment Methodologies” 2016).

Walk the layers in ordinary language.

27.3.1 Human experience and claim

What success means for a person—finished, correct, announced, acceptable. Every test oracle and every dashboard tile should eventually answer a human question, or admit it does not.

27.3.2 Deliberate cases (tests)

Checks run under chosen inputs and environments. They answer narrow questions well and overclaim when treated as the whole world.

27.3.3 Oracles

How we decide pass/fail. An oracle that matches HTTP codes but ignores usable completion is a mismatched oracle—not “proof the experience works.”

27.3.4 Runtime signals

Traces, metrics, and logs (and related events) as observability vocabulary. OpenTelemetry’s conceptual documentation describes these as distinct **signals**—useful shared language, not a version pin or mandatory install for Explorer readers (“Signals” 2026).

27.3.5 Aggregation and views

Dashboards, alerts, and summaries. A green panel is an observation about the panel’s query—not automatically about every user’s session.

27.3.6 Simulation / fixture / replay

Teaching and engineering tools that approximate behavior. Powerful for practice and regression; dishonest when silently upgraded to field measurement.

27.3.7 Inference and narrative

Stories that explain observations. Allowed—and required to stay labeled until extra evidence arrives.

27.3.8 Redaction and sharing

Secrets and PII in traces/logs/screenshots. Portfolio literacy includes what you must remove before share.

FIG-CH27-002 separates *test pass* from *usable experience* so readers stop treating a green suite as QoE.

27.4 4. Follow the signal

Here the “signal” is the chain from human claim → test or observation → labeled inference. Read it as a diagnosis story. Alternate paths exist; open questions stay undetermined.

1. **Human claim.** “Send finishes with coherent feedback for this person in this context.”
2. **Oracle choice.** What would count as evidence for that claim—not merely for a proxy?

- 3. **Deliberate case.** A test, checklist, or lab route exercises a bounded situation.
- 4. **Runtime observation (if live).** Status UI, commodity timing, log/metric/trace excerpt—ethically captured (“Signals” 2026).
- 5. **Label.** Observed / inferred / illustrative / fixture / simulated.
- 6. **Inference gate.** Causal story only with named extra evidence still needed.
- 7. **Share decision.** Redact before portfolio or incident notes leave the machine.
- 8. **Honesty close.** State what the artifact proves—and what it does not.

27.4.1 Signal families without collapse

Family	Everyday question	Typical mis-use
Test result	Did this deliberate case meet its oracle?	Treating the suite as the whole user population
Metric	What did this numeric signal sample or aggregate show?	Promoting one KPI into a Stability Contract
Log	What event text was recorded?	Reading a log flood as root-cause certainty
Trace	What spans/path did this request (or demo) follow?	Claiming complete visibility from one partial trace (“Signals” 2026)
Simulation / fixture	What did the teaching or synthetic setup show?	Silent upgrade to measured human evidence
Inference	What may explain the observations?	Writing inference in the observation column

FIG-CH27-003 is the observation → inference gate: signals enter; labels are required; causal claims need extra evidence.

27.4.2 Evidence hierarchy (climb only as far as tools and ethics allow)

Inherited from CE-6 / CH20 and restated here for testing/observability depth (CLM-CH27-001) (“Reference Guide to Quality of Experience Assessment Methodologies” 2016):

- 1. Illustrative teaching aid (labeled)
- 2. Commodity observation (status UI, wall-clock, visible stalls)
- 3. Browser / OS / app instrumentation where exposed
- 4. Correlated multi-signal inspection (still not proof)
- 5. Controlled comparison under ethical constraints
- 6. Standards-aligned subjective / objective QoE methods—**not** required for classroom completion

Simulation and fixtures belong on the lower rungs unless independently measured and labeled otherwise. That is the chapter’s non-negotiable honesty rule.

27.4.3 Adjacent practice (not invented CH27 WAIKE modules)

WAIKE accepted main (SHA e97e74fc9bfb44b1cdc26b272dc4848264f15fe0) includes adjacent CLOUD_DEVOPS labs such as lab_slo_budget and lab_incident_runbook, plus DATA_DASHBOARDS debug/freshness labs. Those are **adjacent** competencies—not renamed as publication lab IDs and not an invented “observability course for CH27” (CLM-CH27-002) (“Waikes-Research-Ops Accepted Main Curriculum Evidence” 2026).

27.5 5. Component cards

For each idea: plain language, analogy (labeled), technical function, constraints, common symptoms.

27.5.1 Testing

- **Plain language.** Checking claimed behavior with deliberate cases.
- **Analogy (labeled).** Like a fire drill—useful rehearsal, not proof every real fire will go the same way.
- **Technical function.** Produces pass/fail under chosen oracles and environments.
- **Constraints.** Coverage gaps; oracle mismatch; local production; one run reliability study.
- **Symptoms.** Green suite beside failed human outcomes; flaky cases treated as “random” without evidence.

27.5.2 Test oracle

- **Plain language.** How we know a test outcome matches the experience claim.
- **Analogy (labeled).** Like a referee’s rulebook—if the rulebook measures the wrong game, the whistle misleads.
- **Technical function.** Binds assertions to the human question (or admits it only binds a proxy).
- **Constraints.** Proxies (status codes, unit purity) can be necessary and still insufficient for QoE.
- **Symptoms.** “All tests passed” while send stalls for users.

27.5.3 Observability

- **Plain language.** Ability to ask new questions of a running system via signals.
- **Analogy (labeled).** Like being able to inspect a living city with multiple instrument types—not only replaying yesterday’s drill.
- **Technical function.** Traces, metrics, logs (and related signals) as inspectable evidence channels (“Signals” 2026).
- **Constraints.** Instrumentation coverage gaps; cost; privacy; Explorer pathway does not require installing OpenTelemetry.
- **Symptoms.** Dashboards green while the question you need was never instrumented.

27.5.4 Evidence hierarchy

- **Plain language.** From illustrative aid to stronger measurement methods without faking rungs.
- **Analogy (labeled).** Like climbing a ladder—skipping rungs by renaming simulation as measurement is falling with style.
- **Technical function.** Pedagogical ancestor from CE-6; aligns classroom honesty with G.1011-aware vocabulary without claiming lab = formal QoE study (CLM-CH27-001) (“Reference Guide to Quality of Experience Assessment Methodologies” 2016).
- **Constraints.** Tools, ethics, and access limit how far a reader can climb.
- **Symptoms.** Fixture numbers pasted as “our benchmark.”

27.5.5 Observation vs inference

- **Plain language.** What happened vs what may explain it.
- **Analogy (labeled).** Like a notebook that separates “saw the light blink” from “the pump must be broken.”
- **Technical function.** Portfolio and incident discipline; LAB-CE06-001 field split.
- **Constraints.** Outside observation rarely distinguishes failure domains cleanly.
- **Symptoms.** Causal certainty in the observation column.

27.5.6 Evidence redaction

- **Plain language.** Removing secrets/PII before sharing artifacts.
- **Analogy (labeled).** Like blanking account numbers on a photocopy before you pin it on a board.
- **Technical function.** Makes observability artifacts shareable without leaking tokens, messages, or identifiers.
- **Constraints.** Over-redaction can destroy needed evidence; under-redaction harms people.
- **Symptoms.** Screenshots with emails, auth headers, or chat previews in portfolios.

27.6 6. Stability contract

Definition (publication teaching model, inherited): a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

For this chapter, add evidence conditions to that contract:

ID	Evidence condition
EV-01	Signals available at the depth the claim requires
EV-02	Test oracles match the human experience claims (or admit proxy limits)
EV-03	Observation / inference / fixture / simulation boundaries enforced in artifacts
EV-04	PII / secrets redacted before portfolio or external share
EV-05	Simulations and fixtures never silently upgraded to measured human evidence
EV-06	Uncertainty and instrumentation limits stated explicitly

Signature distinctions:

- Connected usable (CH20 / CE-6).
- **Test-pass experience-pass.**
- **Simulation measurement.**
- **FIXTURE_VALIDATED Gate 3 PASS (CLM-CH27-004).**

27.6.1 How to teach and apply it (eight moves)

1. **Name the human experience claim** in one sentence.
2. **Name the oracle or signal** that could support it.
3. **Separate** test outcomes, runtime observations, and inferences.
4. **Place the datum on the evidence hierarchy** (“Reference Guide to Quality of Experience Assessment Methodologies” 2016).
5. **Label** fixture / simulated / illustrative when applicable.
6. **Redact** before share.
7. **State limits**—what commodity tools cannot see.
8. **Close with teach-back**—can another person apply the same labels?

27.6.2 LAB-CE06-001 linkage

LAB-CE06-001 remains the publication-owned EMIT practice surface for evidence fields (observations, inferences, measurements, `evidence_limitations`, and related portfolio templates). Status on accepted infrastructure: **FIXTURE_VALIDATED**. The lab’s `validate_portfolio.py` checks package structure and forbids treating fixtures as Gate 3 human PASS. That status means validators, blank templates, and illustrative fixtures exist and pass structural checks. It does **not** mean Gate 3 PASS. It does **not** mean illustrative example portfolios are human evidence (CLM-CH27-004).

27.7 7. Try it

27.7.1 LAB-CE06-001 — evidence practice (inherit)

Goal. Practice observation vs inference, evidence limitations, and honest labeling on a real or fixture experience—without inventing benchmarks.

Routes (summary; full procedure in lab README).

- **Route A — Notebook + status UI.** Reproduce a connected-but-unusable (or allowed) experience once; separate status chrome from human outcome.
- **Route B — Local stall demo.** Optional `browser/index.html`; label timings observed vs inferred.
- **Route F — Fixture fallback.** Use supplied fixtures; mark `fixture`; study `fixtures/illustrative_example/` only as **ILLUSTRATIVE — not human evidence**.

Explorer baseline for CH27 emphasis.

1. Predict whether failure will show up first in a *test oracle gap*, a *missing signal*, or a *mis-labeled inference*.
2. Fill observation vs inference; no causal claim without naming extra evidence needed.
3. Place two claims on the evidence hierarchy (for example: commodity status UI vs “would need G.1011-aligned study”).
4. Redact any accidental identifiers.
5. Produce teach-back: explain simulation measurement to a peer.

Operator extension. Add one metric/log/trace *concept* note using OpenTelemetry signal vocabulary—without requiring a full OTel install (“Signals” 2026). Label what you could and could not observe.

Builder extension. Add one assertion or checklist item whose oracle names the human outcome (not only a proxy), plus one observability note on the change.

Engineer extension. Design a minimal signal set for one failure domain; list coverage gaps; cite signal concepts without invented version pins (CLM-CH27-003) (“Signals” 2026).

Researcher extension. State what would be required to move a classroom claim toward standards-aligned QoE methods—and what this lab does not provide (“Reference Guide to Quality of Experience Assessment Methodologies” 2016). No invented statistics.

Educator facilitation. Keep Route F first-class. Reject portfolios that paste fixture numbers as personal measurements.

27.7.2 Proposed worksheet (not a WAIKE ID)

proposed: LAB-CH27-SIGNAL-001 — redacted fixture-trace labeling worksheet (observation / inference / redact / hierarchy rung). Until published as a lab package, complete the same discipline inside LAB-CE06-001 portfolio fields.

WAIKE alignment note. Adjacent only: CLOUD_DEVOPS lab_slo_budget / lab_incident_runbook and DATA_DASHBOARDS debug/freshness labs at SHA e97e74fc9bfb44b1cdc26. No exact WAIKE module equals CH27 (CLM-CH27-002) (“Waive-Research-Ops Accepted Main Curriculum Evidence” 2026).

27.8 8. Build it

Extend evidence discipline without shipping a fake benchmark catalog.

27.8.1 Explorer

Build a pocket card: *test result* *human outcome* *causal story*, plus the line **simulation measurement**.

27.8.2 Operator

Build a three-column sheet: *test/checklist* | *signal observed* | *inference (evidence still needed)*. Add a fourth mark for **fixture** / **simulated** when applicable.

27.8.3 Builder

Build one change note that includes: (1) one assertion or manual check tied to a human-visible outcome; (2) one observability note (what signal would reveal regression); (3) redaction checklist for any screenshot or log snippet.

27.8.4 Engineer

Build a minimal signal map for one failure domain (which questions need metrics vs logs vs traces) using OpenTelemetry conceptual vocabulary without version theater (“Signals” 2026). End more branches in “needs more evidence” than in fake certainty.

27.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to make yet—for example, a MOS score or product SLO from classroom fixtures. Specify what methodology and physical/human evidence would be required; keep Gate 3 and Quartet claims honest and pending where applicable (“Reference Guide to Quality of Experience Assessment Methodologies” 2016).

Educators can facilitate Section 11 teach-backs and treat Route F as equitable completion—not a lesser path.

27.9 9. Secure and include it

27.9.1 Security

Traces, logs, HAR exports, and test dumps can leak tokens, session cookies, and internal hosts. Prefer fixtures and redaction. Unauthorized scanning and credential harvesting are out of scope.

27.9.2 Privacy

Portfolio artifacts must scrub account names, emails, message previews, and precise location. Metered-link learners should not be forced onto expensive live captures—Route F exists for equity and privacy together.

27.9.3 Accessibility

Evidence tooling must have text equivalents. Do not rely on color-only dashboard states. Timing and labeling tasks allow extended time. WCAG 2.2 informs teaching intent—not a claim that this book or any product is certified (W3C 2024).

27.9.4 Equity

“Works in CI on a fast runner” is not a Stability Contract for humans on weaker devices, metered cellular, shared computers, or assistive paths. Fixture routes keep learning accessible without requiring production access. Name who is excluded when “proof” assumes one privileged setup.

27.9.5 Safety

Commodity devices only. No exploit steps, no capture of others’ private data, no jailbreaks.

27.9.6 Ethics

Do not present illustrative fixture numbers as measured human evidence. Do not present `FIXTURE_VALIDATED` as Gate 3 PASS. Do not invent observability product benchmarks or OTel version pins. **Overclaiming measurement is still false evidence.**

27.10 10. Career lens

Testing and observability cross ownership domains. No table promises employment; roles vary by organization. LAB-CE06-001 artifacts resemble early professional evidence in miniature:

labeled observations, oracle notes, and explicit uncertainty.

Role lens	Typical artifacts	Review questions
SRE / observability	Signal map; redacted trace note	Did we confuse dashboard green with user completion?
QA / test engineer	Oracle vs experience mismatch writeup	What human claim does this assertion actually bind?
Security-minded developer	Log redaction checklist	What would leak if this screenshot were shared?
Performance engineer	Segmented delay hypotheses with labels	Which segments are observed vs inferred?
Data / dashboard practitioner	Debug/freshness notes (WAIKE-adjacent habits)	Is a synthetic series being read as field truth?
Educator / facilitator	Rubric scores; misconception prompts	Did learners label fixtures and simulations correctly?

Portfolio hint: a scrubbed result table with observation / inference / **fixture** labels is more honest than a vibes-based “tests passed so we’re done” claim.

27.11 11. Check understanding

Concept. In one sentence each, distinguish *testing*, *observability*, and *evidence hierarchy*.

Concept. In one sentence, explain why **simulation** **measurement**.

System tracing. Trace a familiar change or send/submit from human claim → oracle/test → signal → labeled inference. Mark observed vs inferred. Name 2 evidence conditions (EV-01...EV-06) that had to hold.

Misconception check. Why can a full green test suite coexist with a broken human experience?

Misconception check. Why must OpenTelemetry be cited as conceptual signal vocabulary here without invented version pins (CLM-CH27-003)?

Evidence ethics. What is the difference between LAB-CE06-001 **FIXTURE_VALIDATED** infrastructure and Gate 3 human reader validation? Why are illustrative fixture portfolios not human evidence (CLM-CH27-004)?

Teach-it-back. Explain to a newcomer—using LAB-CE06-001 vocabulary—why a green dashboard panel is not automatically a Stability Contract.

Researcher prompt. What additional evidence would be required to move a claim from commodity observation toward ITU-T G.1011-aligned assessment—and what remains out of scope for this classroom lab (“Reference Guide to Quality of Experience Assessment Methodologies” 2016)?

27.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). CE-6 / CH20 inheritance supplies the evidence hierarchy teaching model; OpenTelemetry conceptual docs supply signal vocabulary; WAIKE adjacency is repository-audited, not invented.

Inline citations used in this chapter include “Signals” (2026), “Reference Guide to Quality of Experience Assessment Methodologies” (2016), “Waikē-Research-Ops Accepted Main Curriculum Evidence” (2026), and W3C (2024).

Primary inheritance (link, prefer over duplication): `publication/preproduction/ce-06/publication/full31/chapters/ch27/`, and `labs/LAB-CE06-001/`.

Project-specific honesty: Gate 3 reader evidence remains pending; this working draft is not publication-ready. No Gate 3 files were modified for this draft.

27.13 12. Glossary links

Candidate terms introduced or reinforced here (see also `publication/full31/chapters/ch27/GLOSSARY`) do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Testing	Checking claimed behavior with deliberate cases
Test oracle	How we know a test outcome matches the experience claim
Observability	Ability to ask new questions of a running system via signals
Evidence hierarchy	From illustrative aid to stronger measurement methods
Observation vs inference	What happened vs what may explain it
Evidence redaction	Removing secrets/PII before sharing artifacts
Simulation measurement	Synthetic/teaching setups are not automatically field evidence
Signal (OTel sense)	Conceptual family such as traces, metrics, logs (“Signals” 2026)
FIXTURE_VALIDATED	Lab package structurally validated—not Gate 3 human PASS

Related earlier chapters: observation craft (CH02/CH03), Stability Contract and evidence ladder (CH20 / CE-6), trust and uncertainty (CH23/CH24). Related later chapters: simulation and reproducible research (CH28), portfolio proof (CH30), EMIT capstone expansion (CH31).

27.14 Figure references (planned embeds; draft-blocked until SVG + a11y land)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured learner or lab evidence. No fabricated telemetry. No product SLO curves. No fake benchmarks as human evidence.

27.14.1 FIG-CH27-001 — Evidence hierarchy ladder

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Conceptual hierarchy (inherit CE-6 evidence ladder intent).
- **Reader should notice.** Rungs from illustrative aid toward stronger methods; learner labs mid-ladder.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name each rung; deny that classroom fixtures equal G.1011 campaigns.

27.14.2 FIG-CH27-002 — Test pass vs usable experience

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Comparative.
- **Reader should notice.** Green suite can diverge from human completion.
- **Truth class.** Conceptual.
- **Alt text requirement.** State non-entailment: test pass does not guarantee usable experience.

27.14.3 FIG-CH27-003 — Signal → observation → inference gate

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Sequence.
- **Reader should notice.** Labels required before causal claims; fixtures marked.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name the gate; forbid reading simulation as measurement.

27.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH27-001.** CE-6 evidence hierarchy is the pedagogical ancestor; learner labs are not ITU-T G.1011 campaigns (“Reference Guide to Quality of Experience Assessment Methodologies” 2016).
- **CLM-CH27-002.** WAIKE CLOUD_DEVOPS / DATA_DASHBOARDS labs are adjacent only at audited SHA e97e74fc9bfb44b1cdc26b272dc4848264f15fe0—do not invent a CH27 observability course ID (“Waike-Research-Ops Accepted Main Curriculum Evidence” 2026).
- **CLM-CH27-003.** Cite OpenTelemetry signals conceptually via @otel-signals; no fake version pins (“Signals” 2026).
- **CLM-CH27-004.** LAB-CE06-001 FIXTURE_VALIDATED supports evidence practice; fixtures are not Gate 3 human validation (publication paths under labs/LAB-CE06-001/ @ accepted-main evidence closure).

Chapter 28

Chapter 28 — Digital Twins, Simulation, and Reproducible Research

Status: draft · **Chapter ID:** CH28

Author: Edmund Gunn, Jr.

Inheritance: CE-6 measurement and limitation habits; WAIKE `reproducible_research` catalog adjacency

Gate posture: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS; fixtures and illustrative portfolios are never human reader validation)

Part VI asks you to build, prove, and contribute. Chapter 27 named testing, observability, and evidence. This chapter refuses two quiet upgrades that break research honesty: treating any computational stand-in as if it were the measured world, and treating a one-machine result as if peers could regenerate it without pinned inputs. The governing skill is **bounded stand-ins plus declared reproduction**.

28.1 1. The moment

A simulation says the design is fine. The team ships confidence. Then the real device path, network path, or classroom fixture disagrees—timeouts, thermal throttling, a sensor that never arrives, a peer who cannot regenerate the table you published in the shared notebook.

Or the opposite seat: everything “worked on my laptop.” No SHA. No seed. No fixture ID. A classmate opens the same repo a week later and gets a different plot. Nobody lied on purpose. The environment was never declared.

From the seat: contradiction between a model story and a lived outcome—or between a claimed result and a failed regeneration.

Underneath: a **simulation** is a computational stand-in for *selected* real behaviors. A **digital twin**, in the teaching sense used here, is not “any model with a fancy name.” It is a maintained pairing between a model and a real or planned system, with update rules and **validity bounds**—where the stand-in is trusted, and where it is not. Standards language for manufacturing digital-

twin frameworks exists; this book cites ISO 23247-1 as an overview of that framework family without inventing clause-level fidelity scores or product twin grades (ISO 2021).

The governing question for this chapter:

When a stand-in says “fine” while the world (or a peer’s regeneration) says otherwise—what kind of stand-in were we using, what bounds did we declare, and what SHA, fixture, and seed would make the claim reproducible?

This is not a CAD marketing chapter, not a Device Quartet product-twin validation report, and not permission to promote teaching fixtures into physical proof.

28.2 2. What you notice

Before twin vocabulary or reproducibility cards, notice the human contract that broke.

You expected the model’s green light to predict the experience you can feel—or you expected a classmate to regenerate your table from what you shared. Instead you notice: a demo that only works on one machine; a plot that changes when someone else runs it; a “digital twin” slide that never states how often the model is updated from the real system; a fixture CSV treated as if a sensor farm measured it; assistive or commodity paths that never appear in the simulated story.

A simulation can be useful and still wrong for the decision you are making. A twin is not an ordinary static model with better branding. A fixture is a teaching or measurement stand-in unless labeled otherwise.

Notice the evidence split. Outside observation: the simulation printed “pass,” the spinner finished, the notebook rendered a table. Inference: “therefore the physical unit will pass,” or “therefore anyone can reproduce this.” Those inferences need declared bounds, inputs, and environment—or they stay labeled guesses (CLM-CH28-004).

Optional commodity notice (no specialized twin platform required): pick one small computational experiment you already have—a script, a notebook cell, or a CE lab fixture route. Write three columns before you upgrade language: (1) what the stand-in claimed, (2) what a human could observe without trusting the claim, (3) what is still unpinned (code SHA, data fixture ID, seed, OS/runtime). If you use LAB-CE06-001 fixture timings, mark rows **fixture**. Fixture outputs are illustrative teaching data unless a later measured label applies—they never become Gate evidence by themselves.

28.3 3. Exploded ecosystem

A disputed simulation or a failed reproduction is not a single object. It is a path through an ecosystem. **FIG-CH28-001** separates the simulated/twin story from the measured world. Treat it as **Representative educational architecture**, not a claim that every lab or plant looks like the diagram.

Walk the layers in ordinary language.

28.3.1 Human and decision context

Someone is trying to learn, design, debug, or publish. Acceptable risk depends on the decision: classroom teach-back is not plant commissioning. Validity bounds are decision-relative; inventing universal twin “fidelity percentages” as gunnchOS truth is forbidden.

28.3.2 Question and claim

What is being asked of the stand-in? Predict latency? Check a thermal envelope? Regenerate a table? Vague questions produce vague upgrades from illustrative to “proven.”

28.3.3 Model / simulation

Code and assumptions that approximate selected behaviors. Useful when the selected behaviors match the decision. Dangerous when the omitted behaviors are exactly the ones that break the human experience.

28.3.4 Twin pairing and update path (when claimed)

If you say **digital twin**, you imply a maintained relationship to a real or planned system—not a one-shot offline sketch. Manufacturing-oriented twin frameworks discuss overview principles for that relationship; cite the standard family’s overview carefully and do not invent unimplemented update rates or accuracy grades (ISO 2021). Teaching twins of Device Quartet research form factors remain conceptual / **PHYSICAL_PENDING** until measured validation exists (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026) (CLM-CH28-003).

28.3.5 Inputs, fixtures, and seeds

Data files, synthetic traces, random seeds, and lab fixtures. Deterministic teaching aids are allowed—and must be labeled. CE lab fixtures across the publication are illustrative or deterministic teaching aids unless explicitly labeled measured (CLM-CH28-004).

28.3.6 Compute and runtime environment

Language version, libraries, OS, container image digest, hardware class. “Works on my laptop” is an environment claim, not a result claim.

28.3.7 Observation and instrumentation

Logs, plots, status chrome, assistive announcements. Observation vs inference remains CE-6 / CH27 craft: what happened is not automatically why it happened (Gunn 2026b).

28.3.8 Peer regeneration path

Another person (or future you) with the declared inputs. Reproducible research, in the teaching sense here, means others can regenerate results from declared inputs and environment—not that every paper in the wild already meets a guide we have not yet pinned (**repro_research_guide** remains **SOURCE_NEEDED**; do not invent a peer-reviewed citation).

28.3.9 Society / equity / access

Who can run the stand-in? Who is excluded by specialized software, paid twin platforms, or unlabeled personal data inside a “demo”? Commodity and fixture routes are equity conditions, not optional garnish.

FIG-CH28-003 is the validity-bounds annulus: trusted inside a declared ring; unlabeled outside.

28.4 4. Follow the signal

Here the “signal” is the fate of a claim that travels from human question through stand-in to peer regeneration—not a single radio packet. Read the sequence as a logical honesty path. Alternate routes exist; open questions stay undetermined.

- 1. **Intent.** A person asks a decision-shaped question (will this stall? can a peer regenerate this table?).
- 2. **Stand-in choice.** Simulation, twin-paired model, fixture, or live measurement—named honestly.
- 3. **Bounds declaration.** Where the stand-in is trusted; where it is not.
- 4. **Input pin.** Code SHA, fixture ID, seed, config hashes—as applicable.
- 5. **Execution.** Compute runs; outputs appear.
- 6. **Labeling.** Illustrative / fixture / commodity observation / measured—chosen to match evidence.
- 7. **Comparison (optional).** Live path or second environment—if ethically and practically available.
- 8. **Peer attempt.** Another seat regenerates from the card—or fails, which is also evidence.
- 9. **Human judgment.** Decision allowed, deferred, or rejected; language upgraded only as far as evidence reaches.

28.4.1 Stand-in families without collapse

Family	Everyday question	Typical mis-use
Ordinary model	What simplified story helps me think?	Calling every model a twin
Simulation	What happens under selected assumed behaviors?	Treating selected behaviors as the whole world
Digital twin (teaching sense)	What maintained pairing + update/validity rules bind model to system?	Inventing fidelity scores; skipping update rules (ISO 2021)
Fixture	What deterministic stand-in keeps the lab fair?	Promoting fixture rows to physical proof
Measured run	What did this instrumented attempt observe?	One n=1 run as universal product truth
Reproducible package	Can a peer regenerate from declared pins?	Sharing plots without SHA/fixture/seed

A digital twin is not an ordinary model. The difference is the maintained pairing and the declared bounds—not the marketing adjective (CLM-CH28-002).

28.4.2 Evidence hierarchy (climb only as far as tools and ethics allow)

1. Illustrative teaching aid (labeled)
2. Fixture / deterministic lab stand-in (labeled **fixture**)
3. Commodity observation on a device you already own
4. Instrumented local run with pinned SHA and seeds
5. Peer regeneration from the reproducibility card
6. Standards-aligned twin or measurement practice in a real domain—**not** required for classroom completion; manufacturing twin framework overview is vocabulary, not a classroom certification (ISO 2021)

Learner labs sit mid-ladder. Invented twin fidelity dashboards and fake Gate validation do not belong in Explorer portfolios.

28.5 5. Component cards

For each idea: plain language, analogy (labeled), technical function, constraints, common symptoms.

28.5.1 Simulation

- **Plain language.** A computational stand-in for selected real behaviors.
- **Analogy (labeled).** Like a flight-training scenario that practices some weather—not a claim that every real storm was flown.
- **Technical function.** Explores consequences under declared assumptions.
- **Constraints.** Omitted physics, policy, or human paths can be exactly what fails later.
- **Symptoms.** Green sim / failed live path; overconfident “proved in simulation” language.

28.5.2 Digital twin (teaching sense)

- **Plain language.** A maintained model paired to a real or planned system with update and validity rules.
- **Analogy (labeled).** Like a living map that must be redrawn when the city changes—not a postcard from last summer.
- **Technical function.** Names pairing, update expectations, and bounds; manufacturing twin framework overviews exist in standards literature (ISO 2021).
- **Constraints.** No invented fidelity percentages; Quartet product twins **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).
- **Symptoms.** “Twin” slides with no update story; twin language for a one-shot CAD export.

28.5.3 Validity bounds

- **Plain language.** Where the model is trusted—and where it is not.
- **Analogy (labeled).** Like a trail map marked “maintained to the ranger station; beyond that, unmarked.”
- **Technical function.** Prevents silent promotion of stand-in outputs into decisions they cannot support.

- **Constraints.** Bounds are decision- and context-relative; universal numeric twin grades are out of scope here.
- **Symptoms.** Arguments that never state the out-of-bounds region.

28.5.4 Fixture

- **Plain language.** A deterministic teaching or measurement stand-in when live systems vary.
- **Analogy (labeled).** Like a practice exam with a published answer key—useful, not the live contest.
- **Technical function.** Keeps labs fair and accessible across networks and hardware.
- **Constraints.** Fixtures = human evidence; fixtures = physical proof unless a measured label is earned (CLM-CH28-004).
- **Symptoms.** Portfolio claiming “measured on Device Quartet” from a CSV in the repo.

28.5.5 Reproducible research (teaching sense)

- **Plain language.** Others can regenerate results from declared inputs and environment.
- **Analogy (labeled).** Like a recipe that lists brand, batch, oven, and timer—not a photo of plated food alone.
- **Technical function.** Pins code, data, seeds, and runtime so regeneration is a testable claim.
- **Constraints.** A peer-reviewed reproducibility guide citation remains **SOURCE_NEEDED** for later depth; WAIKE offers an adjacent catalog track, not an exact CH28 twin module (gunnchOS3k 2026) (CLM-CH28-001).
- **Symptoms.** “Run this somehow” README; plots without SHA; environment drift blamed on classmates.

28.5.6 Reproducibility card

- **Plain language.** A short declared package: question, inputs, code SHA, fixture/seed, runtime, limitations, label.
- **Analogy (labeled).** Like a packing list taped to a box—so the next person is not guessing.
- **Technical function.** Makes regeneration attemptable; shared schema intent with CH31 (proposed).
- **Constraints.** Card completeness = Gate validation; card = product certification.
- **Symptoms.** Beautiful notebooks that omit the one pin a peer needed.

28.5.7 Observation vs inference

- **Plain language.** What happened versus what may explain it.
 - **Analogy (labeled).** Like hearing a thud—then guessing which shelf fell, before you look.
 - **Technical function.** Protects Stability Contract diagnosis inherited from CE-6 (Gunn 2026b; “Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023).
 - **Constraints.** Simulation pass is an observation about the simulation.
 - **Symptoms.** Causal language (“proves the device is fine”) from sim-only outputs.
-

28.6 6. Stability contract

Definition (book-wide): a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

Chapter lens: a research or design claim that depends on a stand-in remains honest only while model validity bounds stay explicit, seeds/fixtures/SHAs stay pinned for any claimed regeneration, simulated outputs keep observation/inference labels, and nobody silently promotes simulation or fixture results into physical proof.

A system can remain technically “green” in a simulator while the human experience—or a peer’s regeneration—has already failed.

28.6.1 Concurrent conditions (qualitative; no invented twin fidelity)

Condition	Why it matters
Stand-in type named	Simulation twin fixture measured
Validity bounds explicit	Decisions need an out-of-bounds region
Inputs pinned	SHA, fixture ID, seed, config as applicable
Runtime declared	Language/OS/image drift breaks regeneration
Labels honest	Illustrative / fixture / measured—chosen to match evidence
Update story (if twin claimed)	Pairing without update rules is ordinary model language
Peer path possible	Commodity/fixture routes so regeneration is not elite-only
Accessibility of artifacts	Notebooks and cards usable with assistive tech (W3C 2024)

28.6.2 Failure domains

1. **Model mismatch** — omitted behavior is the one that fails live.
2. **Twin overclaim** — “twin” without pairing/update/bounds.
3. **Fixture promotion** — teaching CSV treated as plant measurement.
4. **Environment drift** — unpinned runtime; “works on my machine.”
5. **Seed / data drift** — stochastic or swapped inputs without declaration.
6. **Human/process** — publishing plots without a regeneration card.

28.6.3 Measurements we can seek (when available)

- Code commit SHA and fixture identifiers on a reproducibility card
- Seed values for stochastic runs
- Fixture-route completion on LAB-CE06-001 with **fixture** labels

- Peer regeneration attempt notes (pass/fail + missing pin)

28.6.4 Measurements we cannot invent

- Device Quartet twin fidelity scores or measured product-twin validation (**PHYSICAL_PENDING**, CLM-CH28-003) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026)
- Universal numeric “twin accuracy” grades as gunnchOS truth
- Fake ISO clause citations beyond the verified 23247-1 overview metadata (ISO 2021)
- Peer-reviewed reproducibility-guide page cites while repro_research_guide is **SOURCE_NEEDED**

28.7 7. Try it

Primary inheritance: **LAB-CE06-001** — keep EMIT discipline and honest labels; use fixture routes when live paths are unsafe, offline, or unfair. Adjacent: existing CE lab fixtures with explicit illustrative/fixture labels. WAIKE reproducible_research is an **adjacent** catalog track pointer, not an exact CH28 twin module (gunnchOS3k 2026).

28.7.1 Baseline (all pathways)

1. Choose one small computational artifact (script, notebook cell, or CE fixture table).
2. Fill a draft reproducibility card: question, inputs, code SHA (or “unpinned”), fixture/seed, runtime, limitations, truth label.
3. Change exactly one pin (seed, fixture file, or declare a different SHA) and regenerate. Record observation vs inference.
4. Do not capture secrets, others’ private data, or real personal traces in simulated “people” datasets—use synthetic data only.

28.7.2 Pathway notes

Pathway	Emphasis
Explorer	Teach-back: simulation measured world; twin ordinary model.
Operator	Run a fixture route; label outputs illustrative vs measured.
Builder	Produce the filled card artifact with at least one real SHA or honest “unpinned.”
Engineer	State validity bounds and one failure mode the stand-in cannot see.
Researcher	Attempt peer regeneration instructions; list missing pins.

Pathway	Emphasis
Educator	Prefer fixture routes; require text labels, never color alone (W3C 2024).

Proposed-only (not live): LAB-CH28-REPRO-001 (pin SHA + fixture + seed; regenerate table) remains a packet opportunity name—do **not** treat it as an implemented publication lab ID yet.

28.8 8. Build it

Extend the try-it card into a shareable one-pager a peer could use without asking you informal questions.

28.8.1 Minimum build

1. **Question** — one decision-shaped sentence.
2. **Stand-in type** — simulation / teaching twin / fixture / measured (pick one primary).
3. **Validity bounds** — three bullets: trusted / untrusted / unknown.
4. **Pins** — code SHA, fixture ID, seed, runtime note.
5. **Truth label** — illustrative, fixture, commodity observation, or measured.
6. **Limitations** — what a peer must not infer.
7. **Accessibility note** — how a non-pointer or screen-reader path can read the card (W3C 2024).

28.8.2 Stretch (optional)

- Pair the card with a second environment regeneration (classmate machine or clean container) and attach a short diff of outcomes.
- Draft shared field names intended for CH31 portfolio reuse—schema proposal only; not a WAIKE course ID.
- If you discuss Device Quartet form factors, keep language conceptual / **PHYSICAL_PENDING**; never paste invented twin fidelity (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

28.8.3 What not to build

- Dashboards that imply measured twin accuracy without instruments
- Marketing “digital twin” badges for ordinary offline models
- Invented WAIKE DIGITAL_TWIN module IDs

28.9 9. Secure and include it

Security, equity, and accessibility are contract conditions—not an appendix.

28.9.1 Secure

- Simulated personal data must be synthetic. Do not record classmates’ messages, locations, biometrics, or credentials “for realism.”
- Pin artifacts without embedding secrets in notebooks or screenshots.
- Treat model outputs that influence safety-critical decisions as bounded advice, not silent authority.

28.9.2 Include

- Commodity computers and fixture routes are first-class—not consolation prizes. Equity means a peer without a twin vendor license can still practice regeneration.
- WAIKE adjacency for reproducible research culture is a track pointer, not a paywall story (gunnchOS3k 2026).
- Research notebooks and reproducibility cards need text structure, meaningful headings, and alternatives to color-only status (W3C 2024).

28.9.3 Failure symptoms to watch

- “Demo users” that are lightly anonymized real people
- Twin platforms required for a baseline Explorer outcome
- Cards that only make sense as a screenshot of a green UI

28.10 10. Career lens

Portfolio evidence beats vibes. Employment is **not** guaranteed by completing artifacts.

Role family	Portfolio evidence from this chapter
Research engineer	Reproducibility card with SHA, fixture/seed, limitations, peer regeneration note
Simulation / digital twin engineer	Validity-bounds worksheet; clear simulation-vs-twin wording; no invented fidelity
Metrologist / measurement scientist (adjacency)	Uncertainty and label note separating fixture, simulation, and measured

Role family	Portfolio evidence from this chapter
Educator / lab lead	Fixture-first lab adaptation with observation vs inference columns
Security-minded builder	Synthetic-data checklist for simulated people and redaction before share

Relate claims to quality-characteristic vocabulary only as literacy—not as ISO product certification (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023).

28.11 11. Check understanding

28.11.1 Misconceptions to retire

1. **“Any model is a digital twin.”** — Twin language requires maintained pairing, update expectations, and validity bounds—not branding.
2. **“Simulation pass proves the device.”** — Simulation pass proves something about the simulation under its assumptions.
3. **“Fixtures are measured twins.”** — Fixtures are teaching/deterministic stand-ins unless labeled measured (CLM-CH28-004).
4. **“My notebook is reproducible because I shared the file.”** — Without pins, peers are guessing the environment.
5. **“ISO twin standards let us invent fidelity scores.”** — Catalogue-verified overview citations are not a license to fabricate grades (ISO 2021).

28.11.2 Quick checks

- Name one difference between an ordinary model and a teaching-sense digital twin.
- Given a green simulation and a failed live path, what labels belong on each observation?
- What four pins belong on a minimal reproducibility card?
- Why is Device Quartet twin validation still **PHYSICAL_PENDING** here (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026)?

Teach-it-back. Explain to a newcomer—without saying “kernel,” “hypervisor,” or “ISO clause”—why a simulation can be useful and still not be a twin, and why a classmate needs your SHA and fixture ID.

Researcher prompt. What evidence would be required to move a teaching twin claim toward measured validation—and what remains out of scope while Quartet twin benches are **PHYSICAL_PENDING** and while `repro_research_guide` is **SOURCE_NEEDED**?

28.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). ISO 23247-1 overview metadata was verified for manufacturing digital-twin framework vocabulary; do not invent clause/page fidelity citations (ISO 2021). WAIKE adjacency uses the accepted-main audit SHA recorded for `reproducible_research` (gunnchOS3k 2026). Device Quartet twin validation remains **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026). A peer-reviewed reproducibility guide remains **SOURCE_NEEDED**.

Inline citations used in this chapter include ISO (2021), gunnchOS3k (2026), “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026), “Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” (2023), W3C (2024), and Gunn (2026b).

Primary inheritance (link, prefer over duplication): `publication/preproduction/ce-06/labs/LAB-CE06-001/`, and `publication/full31/chapters/ch28/`.

28.13 12. Glossary links

Candidate terms introduced or reinforced here (see also chapter glossary candidates; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Simulation	Computational stand-in for selected real behaviors
Digital twin (teaching sense)	Maintained model paired to a real or planned system with update/validity rules
Validity bounds	Where the model is trusted—and where it is not
Fixture	Deterministic teaching/measurement stand-in when live systems vary
Reproducible research	Others can regenerate results from declared inputs and environment
Reproducibility card	Declared pins + limitations + truth label for regeneration
Observation vs inference	What happened vs what may explain it
Stability Contract	Concurrent hidden conditions that keep an experience (or honest claim) alive
PHYSICAL_PENDING	Project evidence not yet measured for claimed physical twin validation
SOURCE_NEEDED	Required external source not yet pinned with verified metadata

Related earlier chapters: measurement and Stability Contract habits (CH20 / CE-6), evidence practice (CH27). Related later chapters: embodied/physical contribution honesty (CH29), portfolio and EMIT synthesis (CH31).

28.14 Figure references (planned embeds; draft-blocked until SVG + a11y land)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured learner or lab evidence. No fabricated twin fidelity. No product accuracy curves.

28.14.1 FIG-CH28-001 — Simulation/twin vs measured world

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Conceptual comparison.
- **Reader should notice.** Stand-in story and measured world are different evidence classes.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name both sides; state conceptual truth class; deny invented fidelity scores.

28.14.2 FIG-CH28-002 — Reproducibility card fields

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Conceptual checklist.
- **Reader should notice.** Question, pins (SHA/fixture/seed/runtime), label, limitations.
- **Truth class.** Conceptual.
- **Alt text requirement.** List required fields in reading order; color never sole cue.

28.14.3 FIG-CH28-003 — Validity bounds annulus

- **Production status.** draft-blocked (no SVG embed in this draft).
- **Type.** Conceptual boundary.
- **Reader should notice.** Trusted inside declared bounds; unlabeled outside.
- **Truth class.** Conceptual.
- **Alt text requirement.** Describe inner trusted region and outer unknown/untrusted region without numeric fake grades.

28.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH28-001.** WAIKE catalog `reproducible_research` exists as an adjacent track—not an exact CH28 twin module (gunnchOS3k 2026).
- **CLM-CH28-002.** Digital-twin standards vocabulary cites ISO 23247-1 overview meta-data; no fake ISO numbers; no invented fidelity (ISO 2021).
- **CLM-CH28-003.** Device Quartet digital twins as measured validation of physical units remain **PHYSICAL_PENDING** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).
- **CLM-CH28-004.** Publication fixtures across CE labs are illustrative/deterministic teaching aids unless labeled measured—fixtures are never silent physical proof.

Chapter 29

Chapter 29 — Designing a Complete Technology Product

Status: draft · **Chapter ID:** CH29

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS; EMIT fixtures and illustrative portfolios are teaching infrastructure, not human reader evidence).

Part VI asks you to build, prove, and contribute. Earlier chapters taught layers in isolation and in pairs: experience maps, stacks, evidence ladders, security/privacy/accessibility gates, and career artifacts. This chapter assembles those pieces into one **complete technology product** story—experience + stack + evidence + responsibility—without marketing language and without inventing shipping claims.

29.1 1. The moment

The demo works. The slide looks finished. The feature lights up on the presenter’s laptop on fast Wi-Fi. Applause. Then a classmate on a weaker phone, a metered cellular link, or an assistive path tries the same flow. The spinner never resolves. Privacy wording is vague. Accessibility feedback is missing. There is no evidence packet—only confidence.

From the seat: a product that “worked in the demo” failed the **Stability Contract** for real users.

Underneath: incomplete product design. UI chrome was treated as the product. Concurrent conditions, claim boundaries, and inclusion routes were optional garnish.

The governing question for this chapter:

What must a complete technology product keep true—experience, stack, evidence, security/inclusion, and honest claims—before anyone may say it is usable?

This is not a pitch deck chapter, not a Device Quartet SKU catalog, and not a PMI certification course. Product-management body-of-knowledge citations are not selected this wave

(CLM-CH29-003 · ILLUSTRATIVE_ONLY; do not invent PMI/ISBN cites). Systems and quality vocabulary come from standards and textbooks already in the bibliography (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023; Saltzer and Kaashoek 2009).

29.2 2. What you notice

Before naming architectures or gates, notice what broke for a person.

You expected a familiar action to finish with clear feedback, understandable privacy, and a path that works on more than the demo machine. Instead you notice: success toast without durable effect; “works here” divergence across devices; a permission prompt that never explains retention; an assistive path that never announces completion; a claim (“ready,” “secure,” “private”) with no evidence plan behind it.

A polished demo is not a Stability Contract.

That distinction is the chapter’s first product skill. Demo success is an observation about *one privileged setup*. Usable product experience is a different observation: concurrent conditions hold for the people you claim to serve, and claims stay inside the evidence you actually have.

Optional commodity notice (no specialized gear): pick one feature you already use or built in a prior lab. Write three columns—*demo claim*, *human outcome on your ordinary device*, *evidence you actually hold*. If you cannot reproduce safely, use fixture routes from prior labs (LAB-CE06-001 / LAB-TRUST-001) and mark rows **fixture**. Fixture rows are teaching practice, not Gate validation and not product certification.

29.3 3. Exploded ecosystem

A “complete product” is not a single screen. It is a stack of cooperating layers under a Stability Contract. **FIG-CH29-001** is the first-minute map: human experience at the center; surrounding rings for interaction, application/code, local resources, network path, services/identity, evidence/observability, and society (privacy, equity, accessibility). Treat it as **Representative educational architecture**, not a claim that every product shares one topology (Saltzer and Kaashoek 2009).

Walk the layers in ordinary language.

29.3.1 Human experience and context

Intent, expectation, attention, device class, bandwidth, language, and assistive needs set what “acceptable” means *for this person now*. Software product quality models name characteristics such as usability, performance efficiency, reliability, security, and accessibility-related concerns as related but distinct lenses—useful vocabulary, not a certification of this book or any gunnchOS artifact (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023).

29.3.2 Interaction and feedback path

Taps, keys, voice, or assistive technology must be recognized and must produce coherent feedback. A silent AT path can fail the product even when sighted chrome looks finished (W3C 2024).

29.3.3 Application / code / local state

Handlers, storage, and caches must keep the intended effect durable when the product claims durability. Local success without remote confirmation is a known failure family (CH20 adjacency).

29.3.4 Network and placement (if remote work is required)

Association, DNS, routing, transport, and cloud placement are different failure domains. Do not collapse them into “the network” as one synonym.

29.3.5 Identity, authorization, and privacy lifecycle

Who is authenticated, what is authorized, what is collected/retained/shared/deleted. Consent chrome without a lifecycle story is incomplete product design (CE-5 / CH23–CH24 adjacency) (Solove 2006; Temoshok et al. 2025).

29.3.6 Evidence and observability

What can be measured, logged (ethically), reproduced, and shown in a portfolio or design review before a ship claim. No evidence plan → no honest “ready.”

29.3.7 Society and responsibility

Who is excluded when “works on the author’s laptop” is the only gate. Equity and accessibility are product conditions, not appendices (W3C 2024).

Device Quartet form factors may appear only as research/learning laboratory lenses across commodity-like form factors. They are **not** commercial product SKUs on accepted evidence (CLM-CH29-001; PHYSICAL_PENDING) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

29.4 4. Follow the signal

Here the “signal” is the product claim’s fate across design gates—not a single demo click. Read the sequence as a logical assembly story. Alternate paths exist; open questions stay labeled undetermined.

1. **Name the human experience.** What does success look like for a person (not for a slide)?
2. **List concurrent conditions.** Which hidden conditions must hold together (Stability Contract)?
3. **Map the stack.** Interaction → code → local resources → path/service → identity/privacy → feedback.
4. **Assign failure domains.** For each fragile step, who owns diagnosis when it breaks?

- 5. **Write design gates.** Checks that must pass before “ship/usable” language (**FIG-CH29-002**).
- 6. **Attach an evidence plan.** What will be observed, measured, or fixture-labeled—and what remains unknown?
- 7. **Draw the claim boundary.** What may the product assert given current evidence? Use the one-pager evidence fields (**FIG-CH29-003**) and write allowed/forbidden wording—including **PHYSICAL_PENDING** badges—in the packet text, not as inventing measurements on the diagram.
- 8. **Name tradeoffs.** Latency, cost, privacy, power, inclusion—together, not one hero metric.
- 9. **Secure and include.** Threat, privacy, ally, equity routes integrated—not bolted on last.
- 10. **Portfolio handoff.** One-pager + evidence + inclusion routes someone else can review.

29.4.1 Product assembly without marketing collapse

Lens	Everyday question	Typical mis-use
Experience	Did the person’s intended action finish with usable feedback?	Treating demo applause as usability
Stack	Which layers must cooperate for that experience?	Calling UI “the product”
Evidence plan	What will be tested/measured before claims?	Shipping on vibes
Design gates	Which checks block “usable/ship” language?	Optional checklists after marketing copy
Claim boundary	What may we assert given evidence state?	Upgrading planned → implemented silently
Explicit tradeoffs	What did we sacrifice, and who is excluded?	Optimizing only the presenter laptop

Layered system design literacy—naming interfaces, dependencies, and failure containment—is older than any one product fashion; use it as discipline, not branding (Saltzer and Kaashoek 2009). Quality-characteristic language helps separate reliability, security, usability, and performance stories so one green demo cannot swallow the rest (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023).

29.4.2 Failure branch without drama

Prefer failure *domains* over confident blame: interaction/AT, compute/schedule, memory/storage, network path, service/backend, identity/authz, privacy lifecycle, render/feedback, power/thermal, equity/access. Outside observation rarely distinguishes them cleanly. That limitation belongs on the claim boundary board.

29.5 5. Component cards

For each idea: plain language, analogy (labeled), technical function, constraints, common symptoms.

29.5.1 Complete technology product

- **Plain language.** Experience + stack + evidence + responsibility—not UI alone.
- **Analogy (labeled).** Like a bridge that includes foundations, load rating, inspection records, and who may safely cross—not only a fresh coat of paint.
- **Technical function.** Assembles human outcome, cooperating layers, and honest claims into one design object.
- **Constraints.** Demo success product completeness; no Device Quartet marketing (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).
- **Symptoms.** “Works in the demo” divergence; missing evidence packet; chrome without durable effect.

29.5.2 Design gates

- **Plain language.** Checks that must pass before claiming ship/usable.
- **Analogy (labeled).** Like preflight checks—not a promise that the flight will be pleasant, but a refusal to roll without the list.
- **Technical function.** Binds experience acceptance, security/privacy/a11y, and evidence readiness into go/no-go language.
- **Constraints.** Gates are teaching/process tools here—not legal compliance certificates.
- **Symptoms.** Marketing language ahead of gate results; optional “we’ll fix a11y later.”

29.5.3 Evidence plan

- **Plain language.** What will be measured or tested before claims.
- **Analogy (labeled).** Like a lab notebook outline written *before* the experiment—not a story invented after the poster is printed.
- **Technical function.** Separates observed / inferred / illustrative / fixture / PHYSICAL_PENDING.
- **Constraints.** Fixture-validated lab infrastructure Gate 3 human validation; n=1 classroom runs fleet proof.
- **Symptoms.** Empty “metrics” slides; unlabeled illustrative numbers.

29.5.4 Explicit tradeoffs

- **Plain language.** Latency, cost, privacy, power, and inclusion named together.
- **Analogy (labeled).** Like packing a bag with a fixed volume—each choice displaces another.
- **Technical function.** Forces multi-objective honesty when tuning a product experience (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023).
- **Constraints.** One hero KPI can hide exclusion; write who loses when you optimize.
- **Symptoms.** “Faster” that breaks AT timing; “cheaper” that forces cloud-only with no fixture route.

29.5.5 Claim boundary

- **Plain language.** What the product may assert given evidence state.
- **Analogy (labeled).** Like a map legend that marks surveyed roads vs sketched trails.
- **Technical function.** Prevents silent upgrades (planned → implemented; simulated → deployed; alpha → shipping OS).

- **Constraints.** gunnchOS device OS documents claim boundaries and is not a finished shipping OS (`beta_ready` false per project audit language) (**CLM-CH29-002**) (“Gunnchos-Device-Os Accepted Main (CE-3 Audit)” 2026).
- **Symptoms.** “Production-ready” wording without evidence; Device Quartet described as shipping SKUs (**CLM-CH29-001**).

29.6 6. Stability contract

Definition (publication teaching model): a user experience exists only while multiple hidden technical conditions remain within acceptable bounds.

Signature distinction: a system can remain technically **demo-successful** while the human experience has already **failed** for the people you claim to serve.

This section binds Part VI product assembly to the CE-5/CE-6 responsibility + contract inheritance without inventing numeric product SLOs or marketing Device Quartet readiness.

29.6.1 How to teach and apply it (eight moves)

1. **Name the human experience** — what success looks like for a person.
2. **List concurrent conditions** — not a single villain metric.
3. **Separate demo chrome from experience outcomes.**
4. **Assign symptoms to failure domains** with evidence gates.
5. **Label every datum** observed / inferred / illustrative / fixture / `PHYSICAL_PENDING`.
6. **Climb the evidence hierarchy** only as far as tools and ethics allow.
7. **State tradeoffs and who is excluded** when conditions are tuned.
8. **Close with teach-back** — can another person use the one-pager and claim boundary?

29.6.2 Concurrent conditions (qualitative product anchor)

For a successful product experience under classroom/portfolio scope, conditions such as the following must remain *good enough together* (inherit CE-6 style; qualitative only):

ID	Hidden condition
SC-P01	Intended human action completable with coherent feedback
SC-P02	Interaction / AT path equivalent enough for claimed users
SC-P03	Local compute/storage responsive enough for claimed durability
SC-P04	Network/service path usable <i>if</i> remote work is required
SC-P05	Identity/authorization matches the risk of the action
SC-P06	Privacy lifecycle matches disclosed consent
SC-P07	Security posture matches claim boundary (no silent overclaim)
SC-P08	Evidence plan exists before “usable/ship” language

ID	Hidden condition
SC-P09	Explicit tradeoffs named (latency/cost/privacy/power/inclusion)
SC-P10	Equity route exists (fixture / low-bandwidth / no specialized hardware)

Numeric humility rule: use qualitative language where no measured threshold exists. Do **not** invent product SLOs, MOS scores, or Quartet EVT curves as gunnchOS truth.

29.6.3 CE + lab linkage (link — do not duplicate depth)

- **Whole-book ecosystem model** and **CE-5/CE-6** packages supply responsibility, trust, and Stability Contract inheritance (`publication/preproduction/ce-05/`, `publication/preproduction/ce-06/`).
- **LAB-CE06-001** and **LAB-TRUST-001** remain publication-owned prior artifacts learners may synthesize into a product one-pager. Fixture-validated status on agent branches is **not** Gate 3 PASS.
- **Device Quartet** remains learning-laboratory context only (**PHYSICAL_PENDING**) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

Honesty bound for this edition: shipping OS / finished product claims for gunnchOS device OS remain out of bounds (`beta_ready` false; claim-boundary teaching only) (**CLM-CH29-002**) (“Gunnchos-Device-Os Accepted Main (CE-3 Audit)” 2026).

29.7 7. Try it

29.7.1 **INLINE_ACTIVITY — Product one-pager + evidence/inclusion gates**

Goal. Assemble prior lab artifacts into a non-marketing **product one-pager** that names the experience, stack layers, Stability Contract conditions, design gates (including secure/include), evidence plan, claim boundary, and explicit tradeoffs.

Status. `INLINE_ACTIVITY` (chapter-native worksheet). The namespaced idea `LAB-CH29-ONEPAGER-001` is **not** a shipped `labs/` package—do not treat it as a `FULL_LAB` ID. Prefer inheriting real prior lab IDs (`LAB-CE06-001`, `LAB-TRUST-001`). Do **not** invent `WAIKE` course/lab IDs. `WAIKE` accepted main (SHA `e97e74fc9bfb44b1cdc26b272dc4848264f15fe0`) offers only **adjacent** culture (`capstones/`, `COMM_PD_ETHICS / lab_pd_capstone`, `CLOUD_DEVOPS`)—not an exact CH29 module (gunnchOS3k 2026).

Safety (hard stops).

- No passwords, tokens, private messages, health data, or classmate PII in portfolios.
- No rooting, jailbreaking, unauthorized scanning, or attacking systems.
- No Device Quartet / specialized RF / EVT hardware required or requested.
- Redact identifiers before any portfolio share.

Routes.

- **Route A — Synthesize prior labs.** Pull scrubbed artifacts from LAB-CE06-001 and/or LAB-TRUST-001 (or earlier Part labs you already completed). Build the one-pager fields below.
- **Route F — Fixture fallback (mandatory offline path).** Use illustrative prior-lab fixtures labeled *fixture*. Fixture synthesis is teaching practice, not human evidence and not Gate validation.

One-pager field set (minimum).

1. Human experience (one sentence) + who must be able to complete it
2. Stack layers (5–9 bullets) mapped to failure domains
3. 5 Stability Contract conditions (observed vs guessed)
4. Design gates checklist including security, privacy, accessibility, equity
5. Evidence plan (what measured / fixture / still unknown)
6. Claim boundary (allowed / forbidden wording; PHYSICAL_PENDING badges where needed)
7. Explicit tradeoffs (name at least two tensions and who might be excluded)
8. Career artifact note (TPM / systems designer / ally+privacy co-owner lens)—employment **not** guaranteed

Explorer baseline. Complete fields 1–7 in plain language; teach-back to a peer.

Operator extension. Add inspection notes: which gate would fail first on a weaker device or AT path.

Builder extension. Turn gates into a reusable checklist someone else can run.

Engineer extension. Failure-domain map with test/observability needs per domain; mark instrumentation limits.

Researcher extension. List unknowns and PHYSICAL_PENDING items explicitly; state what would move a claim up the evidence hierarchy—without inventing measurements.

Educator facilitation. Keep Route F first-class. Score honesty of labels over theatrical demo language.

Evidence to keep. One-pager document; scrubbed source artifacts; claim-boundary board; teach-back notes. Completion means artifact-backed claims—not the string PASS.

29.8 8. Build it

Extend the one-pager without turning Part VI into a fake product catalog.

29.8.1 Explorer

Build a pocket card: *demo usable product*, plus five contract conditions in plain sentences.

29.8.2 Operator

Build a two-column gate sheet: *claim on the slide* | *evidence we hold*. Add a third column only for inferences, each with “evidence still needed.”

29.8.3 Builder

Build the full one-pager template filled once for a familiar feature. Optional: a tiny local checklist file (markdown/YAML) that blocks “ready” wording unless evidence-plan and inclusion-route fields are non-empty—never a claim of commercial readiness.

29.8.4 Engineer

Build a failure-domain map (interaction, compute, storage, path, service, identity, privacy, ally/equity) with one test or observability need each. Cite quality-characteristic separation so reliability/security/usability do not collapse (“Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023).

29.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to make yet—for example, Device Quartet physical validation or production OS certification. Keep those **PHYSICAL PENDING** / claim-boundary limited (**CLM-CH29-001**, **CLM-CH29-002**). Do not invent PMI BoK ISBNs to fill **CLM-CH29-003**.

Educators can facilitate Section 11 teach-backs and keep Route F equitable—not lesser.

29.9 9. Secure and include it

FIG-CH29-002 places secure/include *inside* design gates, not in an appendix after the pitch.

29.9.1 Security

Threat routes belong in the gate list: spoofed “success,” session confusion, over-broad tokens in screenshots, supply of models/configs. Prefer fixtures and redaction. Unauthorized scanning is out of scope. Do not claim production MDM / certified secure boot for gunnchOS from this chapter’s evidence (“Gunnchos-Device-Os Accepted Main (CE-3 Audit)” 2026).

29.9.2 Privacy

Portfolio artifacts must scrub account names, emails, message previews, and location breadcrumbs. Name collect → use → retain → share → delete/redact at product level; a padlock icon is not a lifecycle (Solove 2006). Identity assurance language, when used, stays within standards vocabulary—not a claim that your classroom app meets a federal profile (Temoshok et al. 2025).

29.9.3 Accessibility

Equivalent feedback along assistive or non-pointer paths is a Stability Contract condition. WCAG 2.2 informs teaching intent—not a claim that this book or any gunnchOS product is certified (W3C 2024). Timing tasks allow extended time; avoid color-only status and flicker-heavy demos.

29.9.4 Equity

“Works on the author’s laptop on fast Wi-Fi” is not a product gate. Weaker devices, metered cellular, shared computers, offline contexts, and assistive paths change which concurrent conditions hold. Name who is excluded when optimization assumes one privileged setup. Device Quartet form factors are optional research analogies only—never required lab hardware (**PHYSICAL_PENDING**) (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).

29.9.5 Safety

Commodity devices only. No RF transmit experiments, no battery abuse, no jailbreaks.

29.9.6 Ethics

Do not present illustrative fixture numbers as measured human evidence. Do not present fixture-validated lab infrastructure as Gate 3 PASS. Do not invent SLOs, shipping SKUs, or finished-OS claims. Overclaiming measurement is still false evidence. **CLM-CH29-003** product-management BoK cites stay omitted until a verified edition/ISBN is selected.

29.10 10. Career lens

One “works in the demo” failure crosses many ownership domains. No table promises employment; roles vary by organization. The product one-pager resembles early professional evidence in miniature: labeled claims, gates, and explicit uncertainty.

Role family	Typical portfolio evidence	Review questions
TPM / product engineer	Product one-pager + evidence plan	Did we separate demo from usable outcomes?
Systems designer	Failure-domain map	Which layers own which breaks?
Accessibility + privacy co-owners	Inclusion/privacy gate checklist	Is a11y/privacy a gate or an appendix?
Security engineer (adjacency)	Threat routes on the gate list	What claims exceed evidence?
SRE / reliability adjacency	Concurrent-condition notes	Connected/demo-green usable?
Educator / facilitator	Rubric for honest labels	Did learners mark fixture vs observed?

Portfolio hint: a scrubbed one-pager with claim-boundary badges is more honest than a vibes-based “we’re ready to ship” paragraph. Employment is **not** guaranteed by completing artifacts.

29.11 11. Check understanding

Concept. In one sentence, define a *complete technology product* so that UI alone cannot satisfy the definition.

Concept. In one sentence each, distinguish *design gates*, *evidence plan*, and *claim boundary*.

System tracing. Trace a familiar feature from human intent to feedback in numbered steps. Mark observed vs inferred. Name 5 Stability Contract conditions that had to hold together for a non-demo user.

Misconception check. Why can a feature “work in the demo” while failing real users on weaker devices or AT paths?

Misconception check. Why must this chapter refuse Device Quartet shipping-SKU language and finished gunnchOS OS certification language?

Evidence ethics. What is the difference between `INLINE_ACTIVITY` fixture synthesis for the CH29 one-pager and Gate 3 human reader validation? Why are illustrative `EMIT` examples not human evidence?

Teach-it-back. Explain to a newcomer—using only this chapter’s vocabulary—why a polished demo is not a Stability Contract.

Researcher prompt. What additional evidence would be required to move a Device Quartet or production-OS claim out of `PHYSICAL_PENDING` / claim-boundary limits—and what remains out of scope for this classroom one-pager?

29.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Project-specific Device Quartet and device-OS statuses remain constrained by the chapter claim plan (**CLM-CH29-001** `PHYSICAL_PENDING`; **CLM-CH29-002** claim-boundary teaching). **CLM-CH29-003** (PM BoK) is omitted pending a verified edition/ISBN—no invented PMI cites. Gate 3 reader evidence remains pending; this working draft is not publication-ready.

Inline citations used in this chapter include “Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” (2023), Saltzer and Kaashoek (2009), W3C (2024), Solove (2006), Temoshok et al. (2025), “Gunnchos-Hardware-Industrial-Design Device Quartet Docs” (2026), “Gunnchos-Device-Os Accepted Main (CE-3 Audit)” (2026), and `gunnchOS3k` (2026).

Primary inheritance (link, prefer over duplication): `publication/preproduction/ce-05/`, `publication/preproduction/ce-06/`, `labs/LAB-CE06-001/`, `labs/LAB-TRUST-001/`.

29.13 12. Glossary links

Candidate terms introduced or reinforced here (see also `publication/full31/chapters/ch29/GLOSSARY`) do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Complete technology product	Experience + stack + evidence + responsibility, not UI alone

Term	Plain link
Design gates	Checks that must pass before claiming ship/usable
Evidence plan	What will be measured/tested before claims
Explicit tradeoffs	Latency, cost, privacy, power, inclusion named together
Claim boundary	What the product may assert given evidence state
Stability Contract	Concurrent hidden conditions that keep an experience alive (book teaching model)
Demo usable	Privileged demo success can coexist with failed real-user outcomes
PHYSICAL_PENDING	Physical/hardware validation not yet available—do not upgrade wording
Observation vs inference	What happened vs what may explain it

Related earlier chapters: ecosystem literacy (CH01/CH04), stacks and contracts (CH10/CH20), AI/trust and privacy/allly (CH21/CH23/CH24), evidence practice (CH26/CH27). Related later chapters: contribution/portfolio (CH30), capstone synthesis (CH31).

29.14 Figure references (planned embeds; accessibility metadata)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured learner or lab evidence. No fabricated telemetry. No product SLO curves. **PHYSICAL_PENDING** badges are mandatory where hardware/OS shipping claims would otherwise be implied.

29.14.1 FIG-CH29-001 — Complete product layers under Stability Contract

- **Type.** Conceptual ecosystem / layered rings (educational original).
- **Reader should notice.** Experience at center; stack + evidence + responsibility rings; not UI alone.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name center and each ring; state conceptual truth class; deny shipping-SKU reading; color never sole cue.

29.14.2 FIG-CH29-002 — Design gates including secure/include

- **Type.** Conceptual checklist flow.
- **Reader should notice.** Security, privacy, accessibility, and equity sit inside gates—not after marketing copy.
- **Truth class.** Conceptual.
- **Alt text requirement.** List gate families in reading order; state that gates are teaching tools, not legal certification.

29.14.3 FIG-CH29-003 — One-pager evidence fields (claim-boundary support)

- **Type.** Conceptual field stack (Experience → System boundary → Risks → Evidence → Limitations).
 - **Reader should notice.** Evidence and limitations fields must be filled before “ready/ship” language; claim-boundary / `PHYSICAL_PENDING` badges belong in the written one-pager, not as invented SVG measurements.
 - **Truth class.** Conceptual; qualification **`PHYSICAL_PENDING`** where hardware/OS shipping would be implied in accompanying text.
 - **Alt text requirement.** Name the five fields in reading order; state that the figure is a teaching template, not a shipping certificate.
-

29.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH29-001.** Device Quartet form factors are research/learning benchmarks, not commercial product SKUs—**`PHYSICAL_PENDING`** (“Gunnchos-Hardware-Industrial-Design Device Quartet Docs” 2026).
- **CLM-CH29-002.** gunnchOS device OS documents claim boundaries and is not a finished shipping OS (`beta_ready` false per CE-5 audit language)—claim-boundary teaching only (“Gunnchos-Device-Os Accepted Main (CE-3 Audit)” 2026).
- **CLM-CH29-003.** Product-management BoK citations not selected this wave—**`ILLUSTRATIVE_ONLY`** pedagogy/synthesis (no invented PMI/ISBN).

Chapter 30

Chapter 30 — Career Maps and Portfolio Proof

Status: draft · **Chapter ID:** CH30

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter does **not** claim Gate 3 PASS; CE lab portfolio folders, fixtures, and illustrative packets are teaching infrastructure, not human reader evidence).

Part VI has already asked you to build, test, and design. This chapter refuses a different failure mode: finishing labs while still having nothing a mentor, peer, or hiring-adjacent reviewer can *inspect*. It maps **role families** to **portfolio proof**, organizes that proof as an **index** with **claim boundaries**, and keeps the hard contract every Concept Edition career map already states: **employment_guarantee: false**. Completing artifacts does **not** guarantee employment, salary, placement, or title.

30.1 1. The moment

You finished the labs. Folders exist. Screenshots exist. Maybe a README you wrote at 1 a.m. Still, when someone asks “show me what you can do,” the packet falls apart. You open five unrelated capture files. Claims live in your head, not next to the artifacts. Limitations are missing. A mentor cannot tell which rows were live observation and which were fixtures. A hiring-adjacent reviewer cannot tell which skill you are claiming.

From the seat: finished work that is not yet *reviewable*.

Underneath: missing map between **interest**, **artifact**, **claim**, and **limitation**. Screenshots without claim boundaries are souvenirs, not portfolio proof.

The governing question for this chapter:

How do I assemble a coherent, reviewable portfolio packet from real lab work—without promising myself or anyone else a job?

This is Part VI’s career and evidence-organization chapter. It is not a placement service, not a salary guide, and not a labor-market forecast. Official occupational statistics remain **SOURCE_NEEDED** for this manuscript wave and are omitted rather than invented (CLM-CH30-004).

30.2 2. What you notice

Before naming role families or index schemas, notice the human symptoms of an unreviewable packet.

You have output but no front door: a reviewer does not know where to start. You have images but no claim: “I did networking” is not attached to a specific artifact with a specific limitation. You mixed live commodity runs with fixture rows and forgot to label them. You included a token, email, classmate name, or device serial you should have redacted. You treated a polished screenshot as stronger evidence than a labeled table. You assumed “career chapter” means “you will get hired.”

Labs can be complete while portfolio proof is still absent.

That distinction is the chapter’s first systems skill. Completion of a procedure is not the same as a claim supported by an inspectable artifact—the same honesty WAIKE portfolio culture asks for in adjacent teaching materials (gunnchOS3k 2026). Collapsing them produces credential theater: busy folders that cannot survive a five-minute review.

Optional commodity notice (no specialized gear): open one completed CE or full-book lab portfolio folder you already own. Write three columns—*artifact filename*, *one-sentence claim*, *limitation or fixture label*. If you cannot fill a cell, mark it **missing**. That incomplete table is itself useful evidence of what this chapter must fix.

30.3 3. Exploded ecosystem

A reviewable portfolio is not a single PDF. It is a path through an ecosystem of people, artifacts, and constraints. **FIG-CH30-001** is the first-minute geometry: **role family** → **artifact** → **review criteria**, with the employment non-guarantee as a hard outer boundary. Treat it as **Representative educational architecture**, not a claim that every organization hires the same way.

Walk the layers in ordinary language.

30.3.1 Human interest and context

You care about some kinds of work more than others. Interest is real; interest is not a job offer. Pathway language (Explorer → Researcher, plus Educator) changes how deep the packet must go, not whether employment is promised.

30.3.2 Role family (not job title promise)

A **role family** is a cluster of related work—UX, frontend, SRE, network, security, accessibility, research, education—not a guarantee that a specific title exists at a specific employer. Publica-

tion `careers/role_registry.yaml` and `CE CAREER_MAP.yaml` files are project-local scaffolds for that vocabulary (CLM-CH30-001; CLM-CH30-002).

30.3.3 Skill and chapter work

Skills are practiced across chapters and labs. A skill without an artifact is a wish. An artifact without a skill label is a orphan file.

30.3.4 Artifact / portfolio proof

Portfolio proof means artifacts that make skill inspectable: tables, diagrams, redacted logs, teach-backs, threat-model notes, accessibility pathway writeups. Learner analogues resemble professional artifacts in miniature; they do not replace professional experience.

30.3.5 Claim boundary

Every artifact needs a claim you are willing to defend and a limitation you are unwilling to hide. Fixture rows stay labeled **fixture**. Unreproduced ideas stay labeled unreproduced.

30.3.6 Review criteria

Review criteria answer what makes the packet convincing to a mentor or hiring-adjacent reader: clarity of claim, observation vs inference, redaction, pathway depth, and honesty about gaps.

30.3.7 Index and front door

A **portfolio index** is the map from artifact $\rightarrow$ claim $\rightarrow$ limitation (and optionally role family). Without an index, reviewers wander.

30.3.8 Sharing and accessibility surface

Formats must be openable without private accounts when possible. Color must not be the only cue. Captions and plain-language summaries matter as much as screenshots.

30.3.9 Employment non-guarantee boundary

Learning evidence is not a hired outcome. Every CE career map in this repository sets `employment_guarantee: false`; this chapter preserves that contract (CLM-CH30-002).

FIG-CH30-003 states the same rule visually: learning proof employment guarantee.

30.4 4. Follow the signal

Here the “signal” is the fate of one skill claim as it becomes reviewable—not a packet on the wire. Read the sequence as an assembly story. Alternate pathway depths exist; open questions stay labeled undetermined.

1. **Interest.** Name 2–3 role families adjacent to what you actually care about (Explorer outcome).

2. **Harvest.** List completed lab artifacts you already have (CE portfolio folders, teach-backs, result tables).
3. **Claim.** Attach one defensible claim to each candidate artifact.
4. **Bound.** Add limitations, fixture labels, and what you did *not* measure.
5. **Map.** Place each row under a role family and optional skill id from the publication careers scaffolds.
6. **Index.** Build the front-door portfolio index (Operator outcome).
7. **Polish one.** Choose one artifact to make mentor-readable with a limitations section (Builder outcome).
8. **Criteria pass.** Run a review-criteria checklist as if you were the mentor (Engineer outcome).
9. **Uncertainty.** Explicitly list unreproduced or SOURCE_NEEDED items (Researcher outcome).
10. **Share safely.** Redact, check accessibility of the packet format, then stop—do not invent placement claims.

30.4.1 Portfolio index fields (minimum)

Field	Everyday question	Failure if missing
Artifact id / path	What file or folder?	Reviewer cannot find the proof
Claim	What skill or behavior are you asserting?	Screenshot theater
Role family	Which work cluster does this evidence?	Random pile of files
Pathway depth	Explorer note vs Engineer measurement?	Mismatched expectations
Observation vs inference	What happened vs what might explain it?	Overclaim
Limitation / fixture	What is incomplete, simulated, or unreproduced?	Fake strength
Redaction status	Is PII/secret scrubbed?	Unsafe share
Reviewer ask	What should a mentor check first?	No front door

FIG-CH30-002 is this table as a visual form: portfolio index fields + claim boundary.

30.4.2 Evidence honesty ladder (portfolio edition)

1. Illustrative teaching aid (labeled; not your achievement)
 2. Fixture-completed lab row (valid completion route; still labeled **fixture**)
 3. Commodity live observation with scrubbed artifacts
 4. Repeated or compared runs with uncertainty named
 5. External review notes from a mentor/educator (human; not fabricated)
 6. Professional work samples under real employment norms—**outside** this book’s completion criteria
- Classroom portfolios live mid-ladder. Invented recommendation letters, fabricated Gate 3 reviews, and labor-market percentages without verified sources do not belong here.

30.5 5. Component cards

For each idea: plain language, analogy (labeled), technical function, constraints, common symptoms.

30.5.1 Role family

- **Plain language.** A cluster of related work—not a job promise.
- **Analogy (labeled).** Like saying “kitchen work” instead of “you are hired as head chef at Restaurant X.”
- **Technical function.** Groups ownership domains and typical artifacts so learners can aim practice without title cosplay.
- **Constraints.** Titles vary by organization; registry IDs are teaching scaffolds, not accredited credentials.
- **Symptoms.** “I am an SRE now” after one lab; refusing to name any adjacent family at all.

30.5.2 Portfolio proof

- **Plain language.** Artifacts that make skill inspectable.
- **Analogy (labeled).** Like a lab notebook page a second person can read—not a private memory of having been in the room.
- **Technical function.** Turns procedure completion into a claim supported by evidence (WAIKE-adjacent portfolio culture) (gunnchOS3k 2026).
- **Constraints.** Fixtures allowed when labeled; private data not allowed; screenshots alone are weak.
- **Symptoms.** Capture spam; claims only in chat; missing limitations.

30.5.3 Review criteria

- **Plain language.** What makes an artifact convincing to a careful reader.
- **Analogy (labeled).** Like a rubric a coach uses—not a guarantee the player makes the team.
- **Technical function.** Checks clarity, boundaries, redaction, pathway fit, and honesty.
- **Constraints.** Criteria are educational; they are not a company’s hiring bar.
- **Symptoms.** Pretty formatting with empty claims; hidden fixture origins.

30.5.4 Employment non-guarantee

- **Plain language.** Learning evidence hired outcome.
- **Analogy (labeled).** Like a practice recording proving you practiced—not that a venue booked you.
- **Technical function.** Preserves CE and careers registry contract: `employment_guarantee: false` on role entries and `CAREER_MAP` files (CLM-CH30-002).
- **Constraints.** Forbids salary, placement, and “guaranteed interview” language in chapter and checklist prose.
- **Symptoms.** Marketing tone; invented demand percentages; pressure framing.

30.5.5 Portfolio index

- **Plain language.** Map from artifact → claim → limitation (plus role family).
- **Analogy (labeled).** Like a table of contents that also admits what is unfinished.

- **Technical function.** Gives mentors a front door and learners an assembly checklist (CH30 `INLINE_ACTIVITY` portfolio index; namespaced idea `LAB-CH30-PORTFOLIO-001` is not a `FULL_LAB` package).
 - **Constraints.** Index quality depends on honest rows; cannot repair missing underlying labs.
 - **Symptoms.** Orphan files; duplicate claims; one screenshot mapped to five unrelated skills.
-

30.6 6. Stability contract

A reviewable portfolio experience exists only while several conditions hold together. Break any one and the packet stops being trustworthy—even if the zip file still downloads.

30.6.1 Portfolio Stability Contract (CH30)

1. **Artifacts are reviewable without private data.** No secrets, tokens, personal identifiers, or classmate PII in shared packets.
2. **Claim boundaries attach to each artifact.** Claim, limitation, and fixture/live labels travel with the file.
3. **No employment-guarantee language.** Not in README, index, teach-back, or career table.
4. **Pathway coverage is represented honestly.** Explorer depth is valid; do not fake Engineer measurements.
5. **Observation vs inference remains labeled.** Vibes are not causal claims.
6. **Inherited lab integrity rules still apply.** Do not relabel illustrative EMIT / fixture examples as human Gate evidence.
7. **Formats remain accessible enough to open.** Prefer plain text/Markdown tables; if you use color, add non-color cues.
8. **Uncertainty is visible.** `SOURCE_NEEDED` and unreproduced items are named, not deleted.

If status chrome says “labs complete” while these conditions fail, the human portfolio experience has already failed.

Device Quartet hardware is **not required** for this chapter.

30.7 7. Try it

Lab posture: Prefer inherit and assemble. Do not invent WAIKE course IDs. This chapter’s Try It is an **INLINE_ACTIVITY** (portfolio index assembly). The namespaced idea `LAB-CH30-PORTFOLIO-001` is **not** a shipped `labs/` package—complete the Try route using any CE lab `portfolio/` outputs you already have plus the careers scaffolds.

30.7.1 Route A — Commodity assemble (preferred)

1. Pick 2–3 role families from `careers/role_registry.yaml` or a CE `CAREER_MAP.yaml` that match your interest.

2. Collect at least three existing artifacts (result table, teach-back, diagram, redaction checklist, etc.).
3. Fill the portfolio index fields from §4 for each artifact.
4. Mark every fixture-derived row **fixture**.
5. Write one limitations paragraph for the whole packet.

30.7.2 Route F — Fixture / no-specialized-hardware

If you lack live lab outputs, use published illustrative portfolio summaries (for example LAB-CE06-001 **fixtures/illustrative_example/**) and label the entire packet **fixture-assembled**. Fixture assembly trains the index skill; it is not proof you ran the live lab, and it is not Gate 3 validation.

30.7.3 Operator checklist

- ☐ Front-door index exists
- ☐ Each row has claim + limitation
- ☐ Role families named without job promises
- ☐ Redaction status checked
- ☐ No salary / placement / guarantee language

Time box: 45–90 minutes for a first honest index. Deeper polish belongs in Build.

30.8 8. Build it

Builder outcome: produce **one** polished artifact with an explicit limitations section, then link it from the index.

30.8.1 Small extension options (pick one)

1. **Polished teach-back.** One page, ordinary language, claim boundary at the top, “what I still cannot show” at the bottom.
2. **Criteria-annotated index.** Add a column **mentor_check** with the first question a reviewer should ask for each row (Engineer flavor).
3. **Pathway remind.** Take one Explorer-only note and deepen it one rung (for example add observation/inference labels) without inventing measurements.
4. **Educator facilitation card.** Three misconception probes for peers who confuse portfolio proof with employment guarantees.

30.8.2 Done means

- The polished artifact opens without private accounts
- Limitations are impossible to miss
- Index row points to it
- README states **employment_guarantee: false** in plain language

Stop conditions: do not fabricate mentor quotes, recommendation letters, or labor-market charts.

30.9 9. Secure and include it

Security, privacy, accessibility, and equity are portfolio conditions—not an appendix.

30.9.1 Privacy and safety

Redact names, emails, phone numbers, exact locations, account IDs, tokens, API keys, and screenshots that show other people’s faces or work. Prefer synthetic or fixture identifiers when teaching. Do not capture others’ private data to “enrich” a portfolio.

30.9.2 Accessibility

Ship a text-first index. If you include diagrams, provide short alt text that states the teaching idea (role → artifact → criteria; index fields; proof hire). Do not rely on color alone to mark pass/fail rows. Allow extended time for assembly tasks; avoid flicker-heavy capture demos.

30.9.3 Equity

Fixture-based artifacts are **valid completion evidence** when labeled. Learners without specialized hardware, paid cloud accounts, or uninterrupted high-speed links must still be able to finish an honest packet. “Works on the author’s laptop with five prior internships’ worth of private repos” is not a Stability Contract for this chapter.

30.9.4 Ethics

Do not present illustrative fixture portfolios as human Gate 3 evidence. Do not present this chapter as a placement guarantee. Do not invent BLS or other occupational percentages while CLM-CH30-004 remains SOURCE_NEEDED. Overclaiming career outcomes is still false evidence.

30.10 10. Career lens

Career maps in this book are scaffolds for aiming practice. They are not offers. The publication careers registry and CE CAREER_MAP files encode that boundary explicitly (`employment_guarantee: false` on roles and maps).

Role family (illustrative)	Portfolio evidence that helps a reviewer	Review questions (not hiring bars)
Career coach / educator	Pathway map without guarantees; misconception probes	Did the learner avoid placement language?
Hiring-manager adjacency	Review-criteria checklist + index front door	Can I find claim + limitation in one minute?
Learner self-advocate	Portfolio index + teach-back	Is the best artifact obvious?
UX / HCI	Interaction notes; teach-back of a human moment	Observation vs opinion labeled?
Frontend / application	Scrubbed UI/event notes; local vs remote pair	Fixture rows marked?

Role family (illustrative)	Portfolio evidence that helps a reviewer	Review questions (not hiring bars)
SRE / backend / cloud	Failure-domain shortlist; connected usable notes	Overclaim on reliability?
Network / wireless	Timing or path notes without invented RF theater	Probe limits named?
Cybersecurity / trust	Threat-model lite + redaction checklist	Secrets absent?
Accessibility	Keyboard/switch/voice pathway writeup	Non-color status encoding?
Research	Uncertainty section; unreproduced list	SOURCE_NEEDED honest?

WAIKE adjacency for portfolio and capstone culture remains useful context, not an exact career-placement course map (gunnchOS3k 2026). Employment is **not** guaranteed by completing artifacts.

30.11 11. Check understanding

Concept. In one sentence each, define *role family*, *portfolio proof*, and *portfolio index* so that none swallows the other two.

Concept. What is a *claim boundary*, and why is a screenshot without one weak evidence?

System tracing. Trace one skill you care about from interest → artifact → claim → limitation → index row → reviewer question. Mark any step still **missing**.

Misconception check. Why can labs be “complete” while a mentor still cannot review your work?

Misconception check. Why must this chapter refuse employment-guarantee and invented job-growth percentages?

Evidence ethics. What is the difference between a fixture-assembled portfolio (valid labeled completion) and Gate 3 human reader validation?

Teach-it-back. Explain to a newcomer why a portfolio packet is closer to a lab notebook with a table of contents than to a job offer letter.

Researcher prompt. What additional verified sources would be required before citing occupational outlook statistics—and why does this draft omit them while CLM-CH30-004 is SOURCE_NEEDED?

30.12 References

Selected authoritative and project sources for this chapter’s explanations are listed in the bibliography (`book/references/references.bib`) and in publication-internal careers/CE scaffolds. Inline citation used for WAIKE adjacency: gunnchOS3k (2026).

Project evidence (link, prefer over duplication):

- `careers/role_registry.yaml`, `careers/skill_map.yaml`, `careers/chapter_role_map.md` (CLM-CH30-001)
- Concept Edition `CAREER_MAP.yaml` files under `publication/preproduction/ce-*/` with `employment_guarantee: false` (CLM-CH30-002)
- WAIKE adjacent portfolio/capstone patterns at audited SHA `e97e74fc9bfb44b1cdc26b272dc4848264` (CLM-CH30-003) (gunnchOS3k 2026)

Omitted on purpose: BLS Occupational Outlook Handbook (or equivalent) labor-market statistics (CLM-CH30-004 / `bls_or_equivalent`) — status `SOURCE_NEEDED`; do not invent percentages.

Gate 3 reader evidence remains pending; this working draft is not publication-ready.

30.13 12. Glossary links

Candidate terms introduced or reinforced here (see also `publication/full31/chapters/ch30/GLOSSARY_CA`; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
Role family	A cluster of related work, not a job promise
Portfolio proof	Artifacts that make skill inspectable
Review criteria	What makes an artifact convincing to a careful reader
Employment non-guarantee	Learning evidence hired outcome
Portfolio index	Map from artifact → claim → limitation
Claim boundary	Claim + limitation (+ fixture/live label) attached to evidence
Credential theater	Busy artifacts that cannot survive review
Observation vs inference	What happened vs what may explain it

Related earlier chapters: experience-first observation craft (CH02/CH03), evidence and observability practice (CH27), product design coherence (CH29). Related later chapter: capstone Explain → Measure → Improve → Teach (CH31). CE inheritance: all CE `CAREER_MAP.yaml` patterns and CE lab `portfolio/` outputs.

30.14 Figure references (planned embeds; accessibility metadata)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured learner evidence. No fabricated placement outcomes. No invented labor-market charts.

30.14.1 FIG-CH30-001 — Role family → artifact → review criteria

- **Type.** Conceptual map.

- **Reader should notice.** Role family connects to artifacts; artifacts are judged by review criteria; employment is outside the proof chain.
- **Truth class.** Conceptual.
- **Alt text requirement.** Describe the three-node map and state that it is educational, not a hiring pipeline guarantee; color never sole cue.

30.14.2 FIG-CH30-002 — Portfolio index fields + claim boundary

- **Type.** Checklist / form.
- **Reader should notice.** Index fields (artifact, claim, role, limitation, redaction) and the claim-boundary pair.
- **Truth class.** Conceptual.
- **Alt text requirement.** List the required fields in words; state checklist is teaching infrastructure.

30.14.3 FIG-CH30-003 — Learning proof employment guarantee

- **Type.** Boundary diagram.
- **Reader should notice.** Portfolio proof can be strong while employment remains un-garanteed.
- **Truth class.** Conceptual / policy boundary.
- **Alt text requirement.** State both sides of the inequality in words; forbid reading the figure as a placement promise.

Chapter 31

Chapter 31 — Capstone: Explain, Measure, Improve, and Teach the Ecosystem

Status: draft · **Chapter ID:** CH31

Author: Edmund Gunn, Jr.

Manuscript: working draft complete · human validation pending · not publication-ready

Gate note: GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING (this chapter never claims Gate 3 human validation; EMIT fixtures and illustrative portfolios are teaching infrastructure, not human reader evidence).

Part VI asks you to build, prove, and contribute. This chapter is the book’s closing practice: take one real accessible experience and complete **Explain** → **Measure** → **Improve** → **Teach** (**EMIT**) with a portfolio that names honest evidence labels. It inherits the Concept Edition CE-6 Stability Contract package and the publication-owned lab **LAB-CE06-001**, then expands them as a full-edition capstone—not a second latency lecture and not a certification ceremony (Gunn 2026b).

31.1 1. The moment

You sit down to *prove* what this book taught—not to re-diagnose connectivity from scratch. In front of you: one real accessible experience you already lived (or a LAB-CE06-001 fixture stand-in), a blank EMIT portfolio, and a peer who needs a teach-back they can actually use. Chapter 20 already named the connected usable contradiction and the formal Stability Contract vocabulary. This chapter’s job is different: finish **Explain** → **Measure** → **Improve** → **Teach** with honest evidence labels, without turning fixtures into Gate 3 human validation (Gunn 2026b).

From the seat: pressure to sound finished.

Underneath: a portfolio integrity problem. Can you name concurrent **Stability Contract** conditions, separate observation from inference, propose one bounded improvement, and teach the ecosystem—while admitting what you did not measure (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.)?

The governing question for this chapter:

Using the book’s full model, what evidence can I gather—on devices and tools I already have—to explain a real experience, measure what is actually observable, propose one improvement, and teach the ecosystem to someone else?

This is the CE-6 capstone spine expanded for the full book. It is not a second latency lecture, not a Device Quartet EVT campaign, not a carrier MOS study, and not permission to promote illustrative fixtures into human validation.

31.2 2. What you notice

Before filling fifteen portfolio fields, notice what “done” usually fakes.

You may notice a temptation to paste a green connectivity screenshot and call it root cause; to reuse CH20’s connected usable story without adding Measure/Improve/Teach artifacts; to treat a validator exit code as learning; or to write a teach-back only you understand. A peer on a weaker link, older device, metered plan, or keyboard-only path may need different evidence than the path that worked for you.

A completed EMIT packet is not the same event as a vibes-based postmortem.

That distinction is the capstone’s first systems skill. Status chrome vs usable completion remains true—and Chapter 20 / CE-6 already planted it. Here you practice *showing your work*: labeled observations, failure-domain shortlists, one bounded Improve plan, and a teach-back another person can run.

Inheritance, not duplication: if you already recorded a connected usable send/submit/sync in CH20 or LAB-CE06-001, reuse that observation set and advance it through Improve + Teach. If you need a fresh or offline start, follow `labs/LAB-CE06-001/` Route F fixtures and mark rows **fixture**. Fixture timings are illustrative teaching data, not your measured evidence and not Gate validation.

31.3 3. Exploded ecosystem

A stalled send is not a single object—and this capstone does not re-teach Part IV from zero. It is a path through an ecosystem you must *document*. **FIG-CH31-001** is the first-minute map: human experience at the center, with concurrent spokes—not a single ordered villain chain. Treat it as **representative educational architecture**, not a claim that every app fails the same way (Saltzer and Kaashoek 2009).

Walk the layers in ordinary language—the book’s central chain: **human experience** → **system** → **component** → **code** → **network** → **society**.

31.3.1 Human and context

Intent, expectation, attention, and situation set what “acceptable” means *for this person now*. A classroom quiz submit and a casual chat send do not share one universal delay tolerance.

31.3.2 Input and interaction path

Taps, keys, voice, or assistive technology must be recognized and delivered into the software path. A silent AT path can fail the experience even when sighted UI chrome looks busy (W3C 2024).

31.3.3 Application / code / event loop

Handlers must be scheduled and must not hang. Local code can stall while the radio still shows associated (WHATWG 2026b).

31.3.4 Memory, storage, and local state

Working memory and storage responsiveness condition saves, attachments, and caches (Patterson and Hennessy 2020b).

31.3.5 Network path (if remote work is required)

Link association, DNS, routing, transport, and retries. A usable local association does not entail a usable end-to-end path.

31.3.6 Remote service / authorization

Service health, sessions, scopes, and API contracts. HTTP errors or “success UI without durable effect” are service-domain symptoms—not proof that the phone radio failed.

31.3.7 Render / display feedback

Coherent UI feedback must reach the human. Work can finish on a server while frames arrive late.

31.3.8 Power / thermal, trust / permissions, accessibility path

Battery and thermal modes can collapse performance. Identity and scopes must allow the action. Equivalent feedback along assistive paths is a contract condition, not a garnish (W3C 2024).

31.3.9 Society / equity boundary

Who can complete this experience under real constraints—metered data, shared devices, older hardware, language load, assistive technology—belongs inside the ecosystem map, not only in a final disclaimer.

FIG-CH31-002 shows the EMIT cycle with evidence gates so readers stop treating one screenshot as Explain, Measure, Improve, and Teach all at once.

31.4 4. Follow the signal

Here the “signal” is the human action’s fate across layers—and the portfolio’s climb from notice to teach-back. Read the sequence as a logical diagnosis and evidence story. Alternate paths exist; open questions stay labeled undetermined.

1. **Intent.** A person chooses send / submit / sync / stream (or an allowed substitute experience).
2. **Input delivery.** The OS and app receive the intent (or do not).
3. **Local work.** Handlers, memory, and storage do local work (or stall).
4. **Optional remote path.** If required, packets leave, services answer, and authorization holds (or fails).
5. **Feedback.** Render / AT feedback reaches the human (or does not).
6. **Explain.** Name the ecosystem path and ownership domains without collapsing blame.
7. **Measure.** Collect commodity observations; label each row observation / inference / fixture.
8. **Improve.** Propose one bounded change plus the minimum evidence that would confirm it.
9. **Teach.** Produce a teach-back another person can use at Explorer depth.
10. **Limitations.** State what the packet does *not* prove—especially that illustrative fixtures are not human reader evidence.

FIG-CH31-003 is the firewall: fixture / illustrative teaching aids on one side; human-validated Gate evidence on the other. Crossing that wall is an integrity failure, not a shortcut.

31.5 5. Component cards

For the capstone, “components” are failure-domain cards plus portfolio field cards. Each card needs plain language, a constraint, and a failure symptom.

31.5.1 Failure-domain cards

Domain	Plain function	Constraint	Failure symptom
Input / interaction	Deliver human intent into the app	Devices and AT differ	Tap ignored; AT silence
Schedule / compute	Run handlers without hanging	CPU/thermal budgets	Spinner forever; UI freeze
Memory / storage	Hold working state and persist	Capacity / I/O latency	Hitch after attachment
Network path	Carry remote work when required	Association end-to-end path	Connected but no completion
Service / auth	Authorize and fulfill remote effect	Sessions expire; scopes matter	401/5xx; toast without effect
Render / feedback	Show coherent outcome	Frame timing; AT announcements	Done on server, stuck on screen
Power / thermal	Keep performance alive	Battery / thermal policy	Mid-action collapse
Trust / privacy	Protect credentials and traces	Least data; redaction	Leaked tokens in screenshots
Equity / access	Keep the experience completable for more people	Metered / shared / AT paths	Works-for-me only

31.5.2 Capstone portfolio field cards (required set)

LAB-CE06-001 requires these **capstone artifact fields**. Completing them is the chapter’s Build/Try proof—not a vibes-based exit.

Field	What it holds	Honesty rule
human_experience	What the person tried and felt	No invented drama
system_boundary	What is in / out of scope for this diagnosis	Do not blame outside the boundary without evidence
components	Parts that cooperated or failed	Name ownership domains
software_code_role	What local software had to do	Code stall “the network”
network_role	What remote path had to do <i>if required</i>	Association usable path
stability_contract	Concurrent conditions for success	Latency-only contracts fail the model
observations	What was directly seen/heard/logged	Timestamps, status strings, visible stalls
inferences	Candidate explanations	Mark “evidence still needed”
measurements	Commodity timings or fixture citations	Label fixture rows; no invented numbers
evidence_limitations	What the packet does not prove	Fixtures human Gate evidence
security_privacy_accessibility	Threats, scrub, AT path	No secrets in portfolios (Saltzer and Schroeder 1975)
equity_societal_impact	Who is included / excluded	“Works on my laptop” is not the contract
proposed_improvement	One bounded change + confirmation plan	Qualitative success criteria allowed
teach_back	Explanation another person can use	Audience-appropriate depth
portfolio_summary	EMIT index of the packet	No bare PASS as evidence

Quality-of-experience vocabulary stays human-facing; quality-of-service language stays service-characteristic; ping remains one probe (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.; “Reference Guide to Quality of Experience Assessment Methodologies” 2016; “Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” 2023).

31.6 6. Stability contract

The **Stability Contract** (book teaching model) says: a human experience succeeds only while concurrent hidden conditions remain acceptable. It is not an ITU phrase and not a product certification. CE-6 is the primary inheritance package; CH20 expands latency / reliability / QoE formalisms—this chapter uses the contract as the spine of the EMIT portfolio (Gunn 2026b; “Reference Guide to Quality of Experience Assessment Methodologies” 2016).

31.6.1 Concurrent conditions (teaching list)

ID	Condition
SC-01	Input / interaction path delivers intent
SC-02	Application handlers run without hanging
SC-03	Memory available for the working set
SC-04	Storage responsive if persistence required
SC-05	Network path usable <i>if</i> remote work is required
SC-06	Remote service available and authorized <i>if</i> required
SC-07	Coherent render / UI feedback reaches the human
SC-08	Power / thermal state not collapsing performance
SC-09	Trust / permissions allow the action
SC-10	Accessibility / alternate path provides equivalent feedback
SC-11	Total delay and variability acceptable <i>in this context</i>

Numeric humility rule: use qualitative language where no measured threshold exists. Do not invent performance budgets, MOS scores, or gunnchOS product SLOs.

31.6.2 CE-6 + LAB-CE06-001 linkage

- **CE-6 preproduction** (`publication/preproduction/ce-06/`) is the primary inheritance package: Stability Contract model, experience map, claim boundaries, and figure intents. Synthesize CE-1...CE-5; do not re-lecture one-tap sequencing as a new first lesson.
- **LAB-CE06-001** is the publication-owned EMIT capstone. Status on accepted infrastructure: **FIXTURE_VALIDATED**. That status means validators, blank portfolio templates, and illustrative fixtures exist and pass structural checks. It does **not** mean Gate 3 human validation. It does **not** mean illustrative example portfolios are human evidence. It does **not** mean fixture timings are product SLOs.

31.6.3 Evidence firewall (non-negotiable)

Illustrative EMIT examples under `labs/LAB-CE06-001/fixtures/illustrative_example/` are **teaching-only**. They must never be represented as human reader evidence, Gate responses, or completed full-manuscript validation. Observation inference measurement human validation.

31.7 7. Try it

31.7.1 LAB-CE06-001 — Explain, Measure, Improve, and Teach

Goal. Using the book’s full model, gather ethical commodity evidence to explain a real experience, measure what is actually observable, propose one bounded improvement, and teach the Stability Contract for that experience to someone else.

WAIKE alignment note. WAIKE accepted main (SHA `e97e74fc9bfb44b1cdc26b272dc4848264f15fe0`) includes adjacent materials such as `capstones/`, catalog `reproducible_research`, `teach/communicate` adjacency, SLO-budget neighbors, and datapath diagnosis neighbors (gunnchOS3k 2026; “Waike-Research-Ops Accepted Main Curriculum Evidence” 2026). Those are **adjacencies**.

They are not renamed as publication lab IDs. There is **no** exact WAIKE module literally named Stability Contract / EMIT / TECHNOLOGY_LANDSCAPE_CAPSTONE. **LAB-CE06-001** is publication-owned (`waike_map_class: no-map`).

Safety (hard stops).

- No passwords, tokens, private messages, health data, or classmate PII in portfolios.
- No rooting, jailbreaking, unauthorized scanning, or attacking systems.
- Prefer local demos, benign public endpoints, or supplied fixtures; heed metered-data warnings.
- No Device Quartet / specialized RF / EVT hardware required or requested.

Routes (inherit the lab packet; do not re-author CH20's latency drill here).

- **Route A — Capstone notebook.** Bring one prior connected usable observation (from CH20, CE-6, or a single new commodity attempt) into the fifteen-field portfolio. Keep OS connectivity status *separate* from whether the human action finished—then spend most of the time on Improve + Teach, not on re-discovering Part IV.
- **Route B — Local stall demo (optional).** Open `labs/LAB-CE06-001/browser/index.html`; compare local UI updates vs stalled remote path; label timings observed vs inferred.
- **Route F — Fixture fallback (mandatory offline path).** Use `fixtures/sample_observation`, `fixtures/sample_result_table.csv`, and optionally study `fixtures/illustrative_example/` (**ILLUSTRATIVE ONLY — not human evidence**). Mark fixture-derived rows as `fixture`.

Explorer baseline.

1. State which prior observation set you are inheriting (live, CH20, or fixture).
2. Complete EMIT: Explain → Measure → Improve → Teach.
3. List 4 Stability Contract conditions; mark observed vs guessed.
4. Fill all fifteen capstone artifact fields; keep observation vs inference separate.
5. Produce teach-back a peer or family member could use.

Operator extension. Add 3 inspection artifacts and one comparison (local-only vs remote, or two network classes you already have). Commodity Performance timing is software timing—not touch-to-photon or RF truth (MDN Web Docs 2026b).

Pathway extensions (inherit, do not re-author). Builder / Engineer / Researcher / Educator pathway bullets live in the LAB-CE06-001 packet and were taught with CH20's latency/QoE framing. For this capstone, grade the *Teach* artifact and fifteen-field integrity hardest—reuse those pathway prompts only as stretch depth, not as a second Part IV drill.

Evidence to keep (capstone bar). Complete fifteen-field packet under `labs/LAB-CE06-001/portfolio` plus result table, scrubbed notes, and a teach-back someone else can use. Run `validate_portfolio.py` as an integrity check. Completion means artifact-backed EMIT claims—not a bare exit code, not the string PASS, and not a copied CH20 diagnosis without Improve/Teach.

31.8 8. Build it

Extend LAB-CE06-001 without turning the capstone into a fake SLO catalog or a copied illustrative packet.

31.8.1 Explorer

Build a pocket card: *status chrome usable outcome*, plus five contract conditions in plain sentences. Fill **human_experience**, **stability_contract**, and **teach_back** first if time is short—then complete the remaining fields.

31.8.2 Operator

Build a two-column inspection sheet: status observations | experience outcomes. Add a third column only for inferences, each with “evidence still needed.” Populate **observations**, **inferences**, and **measurements** with honest labels.

31.8.3 Builder

Build a one-page Improve plan in **proposed_improvement**: one bounded change, predicted failure-domain effect, and **qualitative** success criteria you could observe. Document one intentional tradeoff (for example more logging vs privacy overhead) in **security_privacy_accessibility**.

31.8.4 Engineer

Build a metric-family / failure-domain decision tree that ends in “needs more evidence” more often than in fake certainty. Cite QoS/QoE vocabulary honestly (“Vocabulary for Performance, Quality of Service and Quality of Experience,” n.d.). Fill **system_boundary**, **components**, **software_code_role**, and **network_role** so ownership stays visible.

31.8.5 Researcher

Build an evidence plan for a claim you are *not* allowed to make yet—for example, a MOS score or Quartet QoE bench. Specify what subjective/objective methodology and physical evidence would be required; keep Quartet claims **PHYSICAL_PENDING**. Record the non-claim in **evidence_limitations** and summarize in **portfolio_summary**.

Educators can facilitate Section 11 teach-backs and keep Route F as a first-class equitable path—not a consolation prize.

31.9 9. Secure and include it

31.9.1 Security

Traces, screenshots, HAR exports, and retry logs can leak tokens, message contents, and identifiers. Prefer fixtures and redaction. Do not capture others’ screens without consent. Unauthorized scanning is out of scope (Saltzer and Schroeder 1975).

31.9.2 Privacy

Portfolio artifacts must scrub account names, emails, message previews, and location breadcrumbs. Distinguish observability for learning from surveillance. Taxonomy language for privacy harms helps keep “more data” from becoming an unexamined default (Solove 2006). Metered-link learners should not be forced onto expensive live captures—Route F exists for equity and privacy together.

31.9.3 Accessibility

SC-10 is a contract condition. Document whether completion was announced to assistive technology; do not rely on color-only status. WCAG 2.2 informs teaching intent here—not a claim that this book or any gunnchOS product is certified (W3C 2024). Timing tasks allow extended time; avoid flicker-heavy demos. Explorer path must work without DevTools.

31.9.4 Equity

“Works on the author’s laptop on fast Wi-Fi” is not a Stability Contract. Weaker devices, metered cellular, shared computers, offline contexts, and assistive paths change which concurrent conditions hold. Name who is excluded when “optimization” assumes one privileged setup. Fill `equity_societal_impact` with that analysis. Device Quartet form factors may appear as optional research analogies only—never required lab hardware (`PHYSICAL_PENDING`).

31.9.5 AI / uncertainty (CE-5 synthesis)

If the chosen experience includes an AI feature, treat model uncertainty, data leaving the device, and opaque failure modes as contract conditions—not magic. Risk-management vocabulary can inform framing without pretending the classroom lab is a full AI risk assessment (Tabassi 2023).

31.9.6 Ethics

Do not present illustrative fixture numbers as measured human evidence. Do not present `FIXTURE_VALIDATED` as Gate 3 human validation. Do not invent SLOs, MOS scores, or Quartet EVT curves. Overclaiming measurement is still false evidence.

31.10 10. Career lens

A finished EMIT packet is closer to professional evidence than a single stalled-submit anecdote. No table promises employment; roles vary by organization. What travels is labeled observation, an Improve proposal with tradeoffs, a teach-back another person can run, and explicit uncertainty—skills mentors and hiring panels can inspect without trusting vibes.

Role lens	Typical artifacts	Review questions
TPM / educator synthesis	Full EMIT portfolio + rubric	Are fixture rows labeled? Is teach-back transferable?
Mentor	Rubric-based feedback	Did feedback avoid PASS-as-learning theater?
Accessibility	AT pathway writeup inside the packet	Was equivalent completion feedback present?
Security / privacy	Redaction checklist; least-data note	Would this packet leak secrets?
SRE / reliability (inherited)	Connected usable notes from CH20/CE-6	Did Improve change a real failure domain?
Researcher	Limitations + next measurement plan	What would escalate the claim honestly?

Portfolio hint: a scrubbed fifteen-field packet with observation / inference / **fixture** labels beats a vibes-based “the network is bad” claim. Completing artifacts does **not** guarantee employment.

31.11 11. Check understanding

Concept. In one sentence, state the EMIT spine and why Teach is required portfolio proof—not optional fluff.

Concept. Name the fifteen capstone artifact fields without looking; then check yourself against Section 5.

System tracing. Take one experience already explained in your packet and show how Measure → Improve → Teach changes what you are allowed to claim. Mark observed vs inferred. Name 4 Stability Contract conditions that had to hold together.

Misconception check. Why is completing CH20’s connected usable diagnosis not the same as completing this capstone?

Misconception check. Why is latency not the only Stability Contract dimension?

Misconception check. Why must a screenshot alone never count as root-cause proof?

Evidence ethics. In a *capstone* packet, what is the difference between LAB-CE06-001 FIXTURE_VALIDATED infrastructure, a validator exit code, and Gate 3 human reader validation? Why can an illustrative portfolio never close human validation?

Teach-it-back. Explain to a newcomer—using only LAB-CE06-001 vocabulary—how to run EMIT on one experience, and which evidence labels belong on fixture rows.

Researcher prompt. For your Improve claim specifically, what additional evidence would be required to move from commodity observation toward ITU-T G.1011-aligned assessment—and what remains out of scope for this classroom capstone (“Reference Guide to Quality of Experience Assessment Methodologies” 2016)?

31.12 References

Selected authoritative sources for this chapter’s general technical explanations are listed in the bibliography (`book/references/references.bib`). Primary inheritance (link, prefer over duplication): `publication/preproduction/ce-06/` and `labs/LAB-CE06-001/`. WAIKE adjacency uses audited SHA `e97e74fc9bfb44b1cdc26b272dc4848264f15fe0` without inventing an EMIT module ID (gunnchOS3k 2026; “Waike-Research-Ops Accepted Main Curriculum Evidence” 2026). Project-specific Device Quartet form-factor claims remain **PHYSICAL_PENDING**. Gate 3 reader evidence remains pending; this working draft is not publication-ready.

Inline citations used in this chapter include Gunn (2026b), “Vocabulary for Performance, Quality of Service and Quality of Experience” (n.d.), “Reference Guide to Quality of Experience Assessment Methodologies” (2016), “Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model” (2023), W3C (2024), MDN Web Docs (2026b), WHATWG (2026b), Patterson and Hennessy (2020b), Saltzer and Kaashoek (2009), Saltzer and Schroeder (1975), Solove (2006), Tabassi (2023), “Signals”

(2026), gunnchOS3k (2026), and “Waike-Research-Ops Accepted Main Curriculum Evidence” (2026).

31.13 12. Glossary links

Candidate terms introduced or reinforced here (see also chapter glossary candidates; do not treat this list as an auto-merge into the live glossary):

Term	Plain link
EMIT spine	Explain → Measure → Improve → Teach (LAB-CE06-001 spine)
Stability Contract	Concurrent hidden conditions that keep an experience alive (book teaching model)
Teach-back	Explaining the ecosystem so another person can use the model
Evidence limitations	What the packet does not prove
Fixture-validated infrastructure	Deterministic lab validation human reader validation
Connected usable	Status chrome can succeed while human outcomes fail
Observation vs inference	What happened vs what may explain it
Evidence hierarchy	Climb from illustrative aids toward stronger methods without faking rungs
QoE	Human-facing quality of the experience
QoS	Service-characteristic language related to, not identical with, QoE
Capstone portfolio	Fifteen-field EMIT packet with integrity labels

Related earlier chapters: experience-first path (CH02), device bottlenecks (Part II synthesis), packets and paths (Part IV), Stability Contract formalism (CH20), equity as contract condition (CH25), evidence practice (CH27), career/portfolio maps (CH30). This chapter closes the loop by requiring you to **do** EMIT with honest labels.

31.14 Figure references (planned embeds; accessibility metadata)

All three figures are **conceptual / illustrative teaching aids** unless a future revision replaces them with measured learner or lab evidence. No fabricated telemetry. No product SLO curves. No human Gate evidence implied.

31.14.1 FIG-CH31-001 — Stability Contract concurrent conditions

- **Type.** Conceptual / hub-and-spoke (inherit CE-6 FIG-CE06-001 intent).

- **Reader should notice.** Multiple concurrent conditions; any one can break the experience.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name center (human experience) and each spoke; state conceptual truth class; deny numeric product budgets.

31.14.2 FIG-CH31-002 — EMIT cycle with evidence gates

- **Type.** Sequence.
- **Reader should notice.** Explain → Measure → Improve → Teach as ordered gates with required artifacts.
- **Truth class.** Conceptual.
- **Alt text requirement.** Name each EMIT stage and one required artifact; deny that fixtures alone close human validation.

31.14.3 FIG-CH31-003 — Fixture / illustrative vs human evidence wall

- **Type.** Boundary.
- **Reader should notice.** Teaching fixtures and illustrative examples must not cross into human reader evidence.
- **Truth class.** Illustrative (firewall teaching aid).
- **Alt text requirement.** State both sides of the wall; forbid reading illustrative packets as Gate evidence.

31.15 Claim footnotes used in this chapter (project-specific)

- **CLM-CH31-001.** LAB-CE06-001 implements EMIT with blank template, illustrative example, rubric, validator, and tests; status `FIXTURE_VALIDATED` — not Gate 3 human validation / not human-validated from fixtures alone.
- **CLM-CH31-002.** No exact WAIKE EMIT / Stability Contract / TECHNOLOGY_LANDSCAPE_CAPSTONE module at audited SHA; mappings are adjacent or no-map (gunnchOS3k 2026; “Waike-Research-Ops Accepted Main Curriculum Evidence” 2026).
- **CLM-CH31-003.** CE-6 preproduction is the primary inheritance package for CH31; link it rather than duplicating its depth (Gunn 2026b).
- **CLM-CH31-004.** Illustrative EMIT example fixtures must never be represented as human reader evidence or Gate responses (`ILLUSTRATIVE_ONLY`).

Chapter 32

Parts and chapters

Canonical full-book architecture: **31 chapters in six parts** (book/chapter_registry.yaml; architecture: specs/02_BOOK_ARCHITECTURE_AND_CHAPTER_REGISTRY.md).

Registry **status** values below are **structural** markers from the chapter registry (often still **outline** even when working prose exists). They are not Gate 3 outcomes and not publication-ready claims.

32.0.1 Part I — Read the experience

Chapter	Title	Registry status
CH01	Technology Is a System, Not a Screen	outline
CH02	Follow One Tap Through the Entire Stack	draft
CH03	Performance: Why Technology Feels Fast, Slow, Smooth, or Unstable	outline
CH04	The Device Quartet as a Learning Laboratory	outline

32.0.2 Part II — Open the device

Chapter	Title	Registry status
CH05	Electricity, Signals, Clocks, and Logic	outline
CH06	CPU, Instructions, and Parallel Work	outline
CH07	Memory, Cache, and Storage	outline
CH08	Graphics, Displays, Audio, Cameras, and Sensors	outline
CH09	Power, Batteries, Thermals, and Mechanical Design	outline
CH10	Ports, Buses, Boards, Packaging, and Manufacturing	outline

32.0.3 Part III — Make hardware useful

Chapter	Title	Registry status
CH11	Firmware, Boot, and Trust	outline
CH12	Operating Systems, Processes, Threads, and Scheduling	outline
CH13	Files, Databases, and Data Lifecycles	outline
CH14	Applications, APIs, Runtimes, and User Interfaces	outline
CH15	Containers, Virtualization, Cloud, and Edge Computing	outline

32.0.4 Part IV — Connect everything

Chapter	Title	Registry status
CH16	Packets, Protocols, Routing, and the Internet	outline
CH17	Wi-Fi, Cellular, 5G, and the Road to 6G	outline
CH18	Spectrum, Antennas, Beams, MIMO, and Radio Conditions	outline
CH19	NTN and Service Continuity Across Ground, Air, and Space	outline
CH20	Latency, Reliability, QoE, and the Stability Contract	outline

32.0.5 Part V — Intelligence, security, and responsibility

Chapter	Title	Registry status
CH21	Data, Machine Learning, and Generative AI	outline
CH22	Edge AI, Sensors, and Embodied Interaction	outline
CH23	Cybersecurity from Chip to Cloud	outline
CH24	Privacy, Identity, Safety, Accessibility, and Ethics	outline
CH25	Digital Equity: Who Benefits, Who Is Excluded, and What We Can Measure	outline

32.0.6 Part VI — Build, prove, and contribute

Chapter	Title	Registry status
CH26	Software Development and Version Control	outline
CH27	Testing, Observability, and Evidence	outline
CH28	Digital Twins, Simulation, and Reproducible Research	outline
CH29	Designing a Complete Technology Product	outline
CH30	Career Maps and Portfolio Proof	outline
CH31	Capstone: Explain, Measure, Improve, and Teach the Ecosystem	outline

32.1 Subject index (editorial)

A traditional back-of-book **subject index** remains a human/editorial deliverable. Automated keyword dumps are **not** claimed as a publication-grade subject index.

Glossary

Working definitions live in `glossary/glossary.yaml` (aliases in `glossary/aliases.yaml`, acronyms in `glossary/acronym_registry.yaml`). Book-wide canonical meanings for high-risk terms live in `book/terminology.yaml` (contradiction fence; chapter prose may vary). Misconception map: `publication/full31/quality/MISCONCEPTION_MATRIX.md`. Chapter “Glossary links” should point to defined term IDs rather than inventing conflicting definitions.

The table below is a **navigation index** of term IDs — not a substitute for the full YAML definitions, and not a publication-ready terminology freeze.

Term ID	Term	Introduced in
<code>accessibility-path</code>	Accessibility path	CH24
<code>api</code>	API	CH02
<code>attack-surface-ux-linked</code>	Attack surface (UX-linked)	CH23
<code>authentication</code>	Authentication	CH23
<code>authorization</code>	Authorization	CH23
<code>branch-integration</code>	Branch / integration	CH26
<code>build-test-loop</code>	Build-test loop	CH26
<code>cache</code>	Cache	CH02
<code>chip-to-cloud-trust-path</code>	Chip-to-cloud trust path	CH23
<code>claim-boundary</code>	Claim boundary	CH29
<code>cloud-computing</code>	Cloud computing	CH02
<code>code-review</code>	Code review	CH26
<code>commit</code>	Commit	CH26
<code>complete-technology-product</code>	Complete technology product	CH29
<code>constrained-completion-route</code>	Constrained completion route	CH25
<code>cpu</code>	CPU	CH02
<code>design-gates</code>	Design gates	CH29
<code>device-driver</code>	Device driver	CH02
<code>digital-equity</code>	Digital equity	CH25
<code>digital-twin-teaching-sense</code>	Digital twin (teaching sense)	CH28
<code>edge-on-device-inference</code>	Edge / on-device inference	CH22
<code>edge-computing</code>	Edge computing	CH02
<code>edge-resource-budget</code>	Edge resource budget	CH22
<code>edge-cloud-fallback</code>	Edge/cloud fallback	CH22
<code>embodied-interaction-loop</code>	Embodied interaction loop	CH22
<code>embodied-sensing</code>	Embodied sensing	CH22
<code>emit-spine</code>	EMIT spine	CH31
<code>employment-non-guarantee</code>	Employment non-guarantee	CH30
<code>encryption-goals-and-limits</code>	Encryption goals and limits	CH23

Term ID	Term	Introduced in
ethics-ladder	Ethics ladder	CH24
evaluation-under-uncertainty	Evaluation under uncertainty	CH21
event	Event	CH02
event-loop	Event loop	CH02
evidence-hierarchy	Evidence hierarchy	CH27
evidence-limitations	Evidence limitations	CH31
evidence-plan	Evidence plan	CH29
evidence-redaction	Evidence redaction	CH27
exclusion-mechanism	Exclusion mechanism	CH25
explicit-tradeoffs	Explicit tradeoffs	CH29
fixture	Fixture	CH28
fixture-validated-infrastructure	Fixture-validated infrastructure	CH31
frame	Frame	CH02
generative-ai-systems-view	Generative AI (systems view)	CH21
gpu	GPU	CH02
haptic-feedback	Haptic feedback	CH02
honest-evidence-labeling	Honest evidence labeling	CH25
identity	Identity	CH23
identity-in-human-systems	Identity in human systems	CH24
inference-output	Inference / output	CH21
interrupt	Interrupt	CH02
jitter	Jitter	CH02
kernel	Kernel	CH02
latency	Latency	CH02
local-vs-cloud-ai	Local vs cloud AI	CH21
measurable-inclusion-signals	Measurable inclusion signals	CH25
model-parameters	Model parameters	CH21
observability	Observability	CH27
observation-vs-inference	Observation vs inference	CH27
packet	Packet	CH02
portfolio-index	Portfolio index	CH30
portfolio-proof	Portfolio proof	CH30
privacy-data-lifecycle	Privacy / data lifecycle	CH24
process	Process	CH02
protection-design-principles	Protection design principles	CH23
protocol	Protocol	CH02
radio	Radio	CH02
ram	RAM	CH02
rendering	Rendering	CH02
reproducible-research	Reproducible research	CH28
review-criteria	Review criteria	CH30
role-family	Role family	CH30
router	Router	CH02
safety-constraints	Safety constraints	CH24
scheduler	Scheduler	CH02
secrets-hygiene	Secrets hygiene	CH26
sensing-privacy-boundary	Sensing privacy boundary	CH22
service	Service	CH02

Term ID	Term	Introduced in
simulation	Simulation	CH28
stability-contract	Stability Contract	CH02
stability-contract-capstone	Stability Contract (capstone use)	CH31
storage	Storage	CH02
teach-back	Teach-back	CH31
test-oracle	Test oracle	CH27
testing	Testing	CH27
thread	Thread	CH02
touch-digitizer	Touch digitizer	CH02
training-and-input-data	Training and input data	CH21
uncertainty-and-errors	Uncertainty and errors	CH21
validity-bounds	Validity bounds	CH28
version-control	Version control	CH26
wcag-as-guidance	WCAG as guidance	CH24

References

Citations resolve through the shared BibTeX file:

`book/references/references.bib`

Full31 working bibliography (additive; Gate 3 untouched): `publication/full131/WORKING_BIBLIOGRAPHY`
+ `WORKING_BIBLIOGRAPHY_INDEX.yaml`.

Chapter prose should use Quarto/Pandoc citation syntax (for example `@rfc791`). Do not paste unverifiable citation keys. Project-specific claims belong in the claim registry with explicit evidence status — not as invented endorsements.

For a compact standards-oriented navigation list, see Key standards.

Key standards

Working navigational list of bibliography entries classified `standards_specifications` in `publication/full131/WORKING_BIBLIOGRAPHY_INDEX.yaml` (25 keys). Cite with Quarto/Pandoc keys (for example `@rfc791`). This is **not** a complete standards corpus and **not** a claim of normative compliance (including WCAG).

Bib key	Title (when known)	Year
@ieee80211-2020	{IEEE} Standard for Information Technology—Telecommunications and Information Exchange between Systems Local and Metropolitan Area Networks—Specific Requirements Part 11: Wireless {LAN} Medium Access Control ({MAC}) and Physical Layer ({PHY}) Specifications	2020
@ieee80211-wg	(see <i>BibTeX / working bibliography</i>)	2026
@iso-iec-25010-2023	Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model	2023
@itu-t-g1011	Reference guide to quality of experience assessment methodologies	2016
@itu-t-p10-g100	Vocabulary for performance, quality of service and quality of experience	—
@nist-sp500-325	Fog Computing Conceptual Model	2018
@nist-sp800-145	The {NIST} Definition of Cloud Computing	2011
@nist_ai_rmf_100_1	(see <i>BibTeX / working bibliography</i>)	2023

Bib key	Title (when known)	Year
@nist_sp_800_63_4	Digital Identity Guidelines	2025
@rfc1034	Domain Names 2014	1987
	Concepts and Facilities	
@rfc1035	Domain Names 2014	1987
	Implementation and	
	Specification	
@rfc1122	Requirements for Internet	1989
	Hosts 2014 Communication	
	Layers	
@rfc1918	Address Allocation for	1996
	Private Internets	
@rfc3022	Traditional IP Network	2001
	Address Translator	
	(Traditional NAT)	
@rfc768	User Datagram Protocol	1980
@rfc791	Internet Protocol	1981
@rfc8200	Internet Protocol, Version 6	2017
	(IPv6) Specification	
@rfc8446	The Transport Layer Security	2018
	(TLS) Protocol Version 1.3	
@rfc9000	QUIC: A UDP-Based	2021
	Multiplexed and Secure	
	Transport	
@rfc9293	Transmission Control	2022
	Protocol (TCP)	
@threegpp-ts23501	System architecture for the	2026
	5G System (5GS)	
@w3c-pointerevents	Pointer Events	2026
@w3c-uievents	UI Events	2026
@wcag22-20231005	Web Content Accessibility	2023
	Guidelines ({WCAG}) 2.2	
@wcag22-20241212	Web Content Accessibility	2024
	Guidelines (WCAG) 2.2	

Canonical cite files: `book/references/references.bib` and the Full31 working bibliography under `publication/full31/`.

Lab index

Machine-readable registry: `labs/lab_registry.yaml`. Packages live under `labs/`.

Lab ID	Title	Chapter / module	Registry status
LAB-TAP-001	Trace and Measure One Tap	CH02	draft
LAB-SYS-001	Name the System Behind a Familiar “Open”	CH01	IMPLEMENTED_DIGITAL
LAB-CMS-001	Make Local Slowness Visible	CE-3	IMPLEMENTED_DIGITAL
LAB-PERF-001	Make Feel Visible	CH03	IMPLEMENTED_DIGITAL
LAB-PKT-001	Trace One Connected Action Across Path and Access	CE-4	IMPLEMENTED_DIGITAL
LAB-TRUST-001	Compare local vs remote AI paths and write a consent/trust card	CE-5	IMPLEMENTED_DIGITAL
LAB-CE06-001	Explain, Measure, Improve, and Teach One Real Technology Experience	CE-6	FIXTURE_VALIDATED
LAB-QUARTET-001	Constraint Contrast Without Quartet Hardware	CH04	unspecified
LAB-SIG-001	From Press to Logic	CH05	unspecified
LAB-PWR-001	Budget Collapse Observation	CH09	IMPLEMENTED_DIGITAL
LAB-IO-001	Map One Media Pipeline (Capture → Process → Present)	CH08	DRAFT_DIGITAL
LAB-BUS-001	Name the Interconnect	CH10	IMPLEMENTED_DIGITAL
LAB-RADIO-OBS-001	Observe Radio Conditions Without Transmitting	CH18	IMPLEMENTED_DIGITAL
LAB-BOOT-OBS-001	Observe Boot and Wake	CH11	IMPLEMENTED_DIGITAL

Fixture data is teaching material unless labeled as measured evidence. Synthetic fixtures must never be described as human reader validation or Gate 3 PASS.

Figure index

Canonical assets: `figures/` + `figures/figure_registry.yaml` (accessibility metadata enforced by validation scripts).

Chapter column is derived from `figure_id` (FIG-CHnn-...). Registry `chapter:` fields were repaired during quality convergence to match ID-derived chapters.

Chapter 2 prototype figures (FIG-CH02-001 ... FIG-CH02-007) are the historical Gate 3 reader-package set. Keep `PHYSICAL_PENDING` / `illustrative` / `conceptual` labels honest.

This index is a **working navigational inventory**, not a publication-final figure freeze.

Figure ID	Title	Chapter (from ID)	Status
FIG-CH01-001	First-minute ecosystem map	CH01	conceptual
FIG-CH01-002	Inputs to processing to state to outputs	CH01	conceptual
FIG-CH01-003	Why the app is not one thing	CH01	conceptual
FIG-CH01-004	Local-only vs network-dependent readiness	CH01	illustrative
FIG-CH01-005	Failure-domain map	CH01	illustrative
FIG-CH01-006	Exploded ecosystem preview	CH01	conceptual
FIG-CH01-007	Stability Contract preview	CH01	conceptual
FIG-CH02-001	One Tap: Human-to-System Overview	CH02	conceptual
FIG-CH02-002	Cross-Layer Tap Sequence	CH02	conceptual
FIG-CH02-003	Representative Device Exploded View	CH02	conceptual
FIG-CH02-004	Software Stack	CH02	conceptual
FIG-CH02-005	Local vs Network-Dependent Tap	CH02	conceptual
FIG-CH02-006	Tap Latency Budget	CH02	illustrative
FIG-CH02-007	Stability Contract	CH02	conceptual

Figure ID	Title	Chapter (from ID)	Status
FIG-CH03-001	Feel words to diagnostic axes	CH03	conceptual
FIG-CH03-002	Action to hitch causal flow	CH03	conceptual
FIG-CH03-003	Same average, different jitter	CH03	illustrative
FIG-CH03-004	Local vs network feel contributors	CH03	conceptual
FIG-CH04-001	Device Quartet as a Learning Laboratory	CH04	conceptual
FIG-CH04-002	Same Task Across Form Factors	CH04	illustrative
FIG-CH04-003	Constraint Matrix (Conceptual)	CH04	conceptual
FIG-CH05-001	Press to Logic Flow	CH05	conceptual
FIG-CH05-002	Analog versus Digital	CH05	conceptual
FIG-CH05-003	Clock Edges Sequencing Decisions	CH05	illustrative
FIG-CH05-004	Logic Building Blocks	CH05	conceptual
FIG-CH06-001	App → process/threads → scheduler → cores/accelerator	CH06	conceptual
FIG-CH06-002	Single-thread vs parallelizable work	CH06	illustrative
FIG-CH06-003	CPU-bound vs waiting-on-I/O symptoms	CH06	conceptual
FIG-CH07-001	Memory hierarchy — registers, cache, RAM, storage	CH07	conceptual
FIG-CH07-002	Open file path — storage to RAM to cache to CPU	CH07	conceptual
FIG-CH07-003	Healthy network icon with memory-pressure hitch	CH07	illustrative
FIG-CH08-001	World sensing processing presentation	CH08	conceptual
FIG-CH08-002	Presentation timing and feel (illustrative)	CH08	illustrative
FIG-CH08-003	Sampling: continuous → discrete	CH08	conceptual
FIG-CH09-001	Energy Path with Throttle Feedback	CH09	conceptual

Figure ID	Title	Chapter (from ID)	Status
FIG-CH09-002	On Charger vs On Battery	CH09	illustrative
FIG-CH09-003	Mechanical Enclosure Roles	CH09	conceptual
FIG-CH10-001	Board buses ports world	CH10	conceptual
FIG-CH10-002	Host request bus transaction response	CH10	illustrative
FIG-CH10-003	Electrical vs protocol layer	CH10	conceptual
FIG-CH10-004	Packaging manufacturing stages	CH10	conceptual
FIG-CH11-001	Boot path sequence	CH11	conceptual
FIG-CH11-002	Root-of-trust stages	CH11	conceptual
FIG-CH11-003	AuthN vs AuthZ at boot	CH11	conceptual
FIG-CH11-004	Update and recovery boundary	CH11	conceptual
FIG-CH12-001	Apps to processes, threads, scheduler, and CPU cores	CH12	conceptual
FIG-CH12-002	Context-switch swimlane	CH12	conceptual
FIG-CH12-003	Concurrency versus parallelism	CH12	illustrative
FIG-CH12-004	Monitor snapshots classroom n=1	CH12	illustrative
FIG-CH13-001	Data lifecycle arc	CH13	conceptual
FIG-CH13-002	Durability stack	CH13	conceptual
FIG-CH13-003	Save click vs durability	CH13	illustrative
FIG-CH13-004	Files vs databases	CH13	conceptual
FIG-CH14-001	App stack map	CH14	conceptual
FIG-CH14-002	Feature after chrome path	CH14	conceptual
FIG-CH14-003	Local vs remote API domains	CH14	illustrative
FIG-CH15-001	VM versus container stacks	CH15	conceptual
FIG-CH15-002	Access versus placement	CH15	conceptual
FIG-CH15-003	Same app three ways	CH15	illustrative
FIG-CH16-001	Network scopes	CH16	conceptual
FIG-CH16-002	Encapsulation	CH16	conceptual
FIG-CH16-003	DNS on the critical path	CH16	conceptual
FIG-CH16-004	Lab timing classroom n=1	CH16	illustrative

Figure ID	Title	Chapter (from ID)	Status
FIG-CH17-001	Wi-Fi vs cellular on-ramps	CH17	conceptual
FIG-CH17-002	Cellular generations timeline	CH17	illustrative
FIG-CH17-003	Indoor Wi-Fi to outdoor handoff	CH17	conceptual
FIG-CH18-001	Radio path conditions	CH18	conceptual
FIG-CH18-002	Beam and path geometry	CH18	conceptual
FIG-CH18-003	MIMO teaching metaphor	CH18	conceptual
FIG-CH18-004	Quartet RF PHYSICAL_PENDING overlay	CH18	project_specific
FIG-CH19-001	NTN continuity map	CH19	conceptual
FIG-CH19-002	Continuity vs icon status	CH19	conceptual
FIG-CH19-003	Illustrative delay families	CH19	illustrative
FIG-CH20-001	Stability Contract concurrent conditions	CH20	conceptual
FIG-CH20-002	Latency vs reliability vs throughput	CH20	conceptual
FIG-CH20-003	Evidence ladder for QoE	CH20	conceptual
FIG-CH20-004	Status vs usable experience	CH20	conceptual
FIG-CH21-001	Data to model to inference	CH21	conceptual
FIG-CH21-002	Local vs remote AI placement	CH21	conceptual
FIG-CH21-003	Model vs product vs claim families	CH21	conceptual
FIG-CH22-001	Sense to edge feedback	CH22	conceptual
FIG-CH22-002	Illustrative latency budget bands	CH22	illustrative
FIG-CH22-003	Wearables PHYSICAL_PENDING badge	CH22	project_specific
FIG-CH24-001	Privacy lifecycle wheel	CH24	conceptual
FIG-CH24-002	Accessible vs blocked recovery	CH24	conceptual
FIG-CH24-003	Ethics and inclusion ladder	CH24	conceptual
FIG-CH25-001	Same task divergent completion	CH25	conceptual

Figure ID	Title	Chapter (from ID)	Status
FIG-CH25-002	Equity evidence hierarchy	CH25	conceptual
FIG-CH25-003	Exclusion mechanism cards	CH25	conceptual
FIG-CH26-001	Edit to review sequence	CH26	conceptual
FIG-CH26-002	Working tree vs history	CH26	conceptual
FIG-CH26-003	Employment non-guarantee boundary	CH26	conceptual
FIG-CH27-001	Evidence hierarchy ladder	CH27	conceptual
FIG-CH27-002	Test pass vs usable experience	CH27	conceptual
FIG-CH27-003	Observation to inference gate	CH27	conceptual
FIG-CH28-001	Twin story vs measured world	CH28	conceptual
FIG-CH28-002	Reproducibility checklist path	CH28	conceptual
FIG-CH28-003	Validity bounds annulus	CH28	conceptual
FIG-CH29-001	Complete product ecosystem map	CH29	conceptual
FIG-CH29-002	Secure/include inside design gates	CH29	conceptual
FIG-CH29-003	One-pager evidence fields	CH29	conceptual
FIG-CH30-001	Role to artifact to review	CH30	conceptual
FIG-CH30-002	Portfolio checklist fields	CH30	conceptual
FIG-CH30-003	Learning proof vs employment	CH30	conceptual
FIG-CH31-001	Capstone concurrent-condition hub	CH31	conceptual
FIG-CH31-002	EMIT cycle with evidence gates	CH31	conceptual
FIG-CH31-003	Fixture vs human-evidence firewall	CH31	conceptual

Career and role map

Learner portfolio artifacts may **resemble** professional artifacts; they do **not** guarantee employment, salary, or licensure.

Machine-readable maps:

- careers/role_registry.yaml
- careers/skill_map.yaml
- careers/chapter_role_map.md (Chapter 2 prototype map)

Role ID	Title	Related chapters (registry)	Employment guarantee
ROLE-UX	UX Designer	CH02	false
ROLE-HCI	HCI Engineer	CH02	false
ROLE-EMBEDDED	Embedded Engineer	CH02	false
ROLE-FIRMWARE	Firmware Engineer	CH02	false
ROLE-DRIVER	Device-Driver Engineer	CH02	false
ROLE-KERNEL	Kernel / OS Engineer	CH02	false
ROLE-APP	Application Developer	CH02	false
ROLE-FRONTEND	Frontend Engineer	CH02	false
ROLE-BACKEND	Backend / Cloud Engineer	CH02	false
ROLE-SRE	SRE	CH02	false
ROLE-NET	Network Engineer	CH02	false
ROLE-WIRELESS	Wireless Engineer	CH02	false
ROLE-SEC	Cybersecurity Engineer	CH02	false
ROLE-PERF	Performance Engineer	CH02	false
ROLE-ARCH	Computer Architect	CH02	false
ROLE-A11Y	Accessibility Engineer	CH02	false
ROLE-RESEARCH	Researcher	CH02	false
ROLE-TPM	Technical Program Manager	CH02	false

As later chapters leave scaffold state, role links should stay evidence-based and free of invented endorsements.

Errata

Working-draft corrections are tracked as versioned errata — not silent rewrites of reader-facing truth without a record.

32.2 How to report

1. Prefer the GitHub issue form: `.github/ISSUE_TEMPLATE/errata.yml` (label `errata`).
2. Include edition/version (for example `WORKING FULL-MANUSCRIPT DRAFT`), chapter ID, severity, and a precise description.
3. Do not include passwords, tokens, private messages, device serials, or precise location.

32.3 Maintainer workflow

See `publication/full131/quality/ERRATA_WORKFLOW.md` (stub). Accepted fixes should land with a dated note; do not invent “validated by reviewers” language from errata alone.

Gate posture remains `GATE_3_IN_PROGRESS - READER_EVIDENCE_PENDING`. Errata intake is **not** Gate 3 PASS.

Acknowledgments

Acknowledgments for contributors, reviewers, and institutional partners will be completed before any publication-ready release.

Placeholder only — no endorsements. Do not invent reviewer names, institutional seals, blurbs, or “featured in” claims in this working draft.

2023a. Recommendation ITU-R M.2160-0 (11/2023). <https://www.itu.int/rec/R-REC-M.2160-0-202311-I/en>.

———. 2023b. 3GPP TR 38.821 (Release 16 family; portal lists versions under change control; ETSI TR 138 821 V16.2.0 cover 2023-03). <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3525>.

3GPP. 2026. “System Architecture for the 5G System (5GS).” 3GPP TS 23.501 (living specification family; revision not pinned). <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3144>.

“A/b (Seamless) System Updates.” 2026. AOSP documentation (living). <https://source.android.com/docs/core/ota/ab>.

Braden, R. 1989. “Requirements for Internet Hosts — Communication Layers.” IETF RFC 1122. <https://www.rfc-editor.org/rfc/rfc1122>.

“Concurrency Control.” 2026. PostgreSQL 18 documentation (living current docs). <https://www.postgresql.org/docs/current/mvcc.html>.

Deering, S., and R. Hinden. 2017. “Internet Protocol, Version 6 (IPv6) Specification.” IETF RFC 8200. <https://www.rfc-editor.org/rfc/rfc8200>.

Eddy, W. 2022. “Transmission Control Protocol (TCP).” IETF RFC 9293. <https://www.rfc-editor.org/rfc/rfc9293>.

Git Project. 2026. “Git - Reference.” Official Git documentation (living). <https://git-scm.com/docs>.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press. <https://www.deeplearningbook.org/>.

“Guidelines for Media Sanitization.” 2014. NIST Special Publication 800-88 Revision 1 (Final). <https://csrc.nist.gov/pubs/sp/800/88/r1/final>.

Gunn, Edmund, Jr. 2026a. “The Technology Landscape.” Publication repository. <https://github.com/gunnchOS3k/gunnchos-technology-landscape>.

———. 2026b. “The Technology Landscape.” Publication repository. <https://github.com/gunnchOS3k/gunnchos-technology-landscape>.

gunnchOS3k. 2026. “Waiko-Research-Ops Accepted Main Audit for CE-1 Adjacency.” GitHub repository. <https://github.com/gunnchOS3k/waiko-research-ops>.

“Gunnchos-Device-Os Accepted Main (CE-3 Audit).” 2026. Git repository. <https://github.com/gunnchOS3k/gunnchos-device-os>.

“Gunnchos-Hardware-Industrial-Design Device Quartet Docs.” 2026. GitHub repository. <https://github.com/gunnchOS3k/gunnchos-hardware-industrial-design>.

- Harris, Sarah, and David Harris. 2021. *Digital Design and Computer Architecture, RISC-v Edition*. 1st ed. Morgan Kaufmann.
- IEEE. 2020. “IEEE Standard for Information Technology—Telecommunications and Information Exchange Between Systems Local and Metropolitan Area Networks—Specific Requirements Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications.” IEEE Std 802.11-2020 listing. https://standards.ieee.org/standard/802_11-2020.html.
- International Electrotechnical Commission. 2017. “Secondary Cells and Batteries Containing Alkaline or Other Non-Acid Electrolytes — Safety Requirements for Portable Sealed Secondary Cells, and for Batteries Made from Them, for Use in Portable Applications — Part 2: Lithium Systems.” IEC 62133-2:2017 (amended by IEC 62133-2:2017/AMD1:2021; consolidated CSV ISBN 978-2-8322-5203-1). <https://webstore.iec.ch/en/publication/70017>.
- Iorga, Michaela, Larry Feldman, Robert Barton, Michael J. Martin, Ned Goren, and Charif Mahmoudi. 2018. “Fog Computing Conceptual Model.” NIST Special Publication 500-325. National Institute of Standards; Technology. <https://csrc.nist.gov/pubs/sp/500/325/final>.
- ISO. 2021. “Automation Systems and Integration — Digital Twin Framework for Manufacturing — Part 1: Overview and General Principles.” ISO 23247-1:2021 (Edition 1, published 2021-10). <https://www.iso.org/standard/75066.html>.
- ITU. 2025. “Measuring Digital Development: Facts and Figures 2025.” ITU-D ICT statistics publication. <https://www.itu.int/en/ITU-D/Statistics/Pages/facts/default.aspx>.
- Iyengar, J., and M. Thomson. 2021. “QUIC: A UDP-Based Multiplexed and Secure Transport.” IETF RFC 9000. <https://www.rfc-editor.org/rfc/rfc9000>.
- Kurose, James F., and Keith W. Ross. 2020. *Computer Networking: A Top-down Approach*. 8th ed. Pearson.
- MDN Web Docs. 2026a. “Network Monitor.” Mozilla Developer Network (living documentation). https://developer.mozilla.org/en-US/docs/Tools/Network_Monitor.
- . 2026b. “Performance API.” Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/API/Performance>.
- . 2026c. “Resource Timing API.” Mozilla Developer Network (living documentation). https://developer.mozilla.org/en-US/docs/Web/API/Performance_API/Resource_timing.
- “Media Capture and Streams.” 2025. W3C Candidate Recommendation Draft 09 October 2025. <https://www.w3.org/TR/mediacapture-streams/>.
- Mell, Peter, and Timothy Grance. 2011. “The NIST Definition of Cloud Computing.” NIST Special Publication 800-145. National Institute of Standards; Technology. <https://csrc.nist.gov/pubs/sp/800/145/final>.
- Mockapetris, P. 1987a. “Domain Names — Concepts and Facilities.” IETF RFC 1034. <https://www.rfc-editor.org/rfc/rfc1034>.
- . 1987b. “Domain Names — Implementation and Specification.” IETF RFC 1035. <https://www.rfc-editor.org/rfc/rfc1035>.
- Open Container Initiative. 2026. “Open Container Initiative Runtime Specification.” Living specification (main branch document). <https://raw.githubusercontent.com/opencontainers/runtime-spec/main/spec.md>.
- Patterson, David A., and John L. Hennessy. 2020a. *Computer Organization and Design RISC-v Edition: The Hardware/Software Interface*. 2nd ed. Morgan Kaufmann.
- . 2020b. *Computer Organization and Design: The Hardware/Software Interface*. 6th ed. Morgan Kaufmann.
- “Permissions.” 2025. W3C Working Draft 06 October 2025. <https://www.w3.org/TR/2025/WD-permissions-20251006/>.
- Postel, J. 1980. “User Datagram Protocol.” IETF RFC 768. <https://www.rfc-editor.org/rfc/rfc768>.
- . 1981. “Internet Protocol.” IETF RFC 791. <https://www.rfc-editor.org/rfc/rfc791>.

- Preston-Werner, Tom. n.d. “Semantic Versioning 2.0.0.” Specification (v2.0.0). <https://semver.org/spec/v2.0.0.html>.
- “Reference Guide to Quality of Experience Assessment Methodologies.” 2016. Recommendation ITU-T G.1011 (07/2016). <https://www.itu.int/rec/T-REC-G.1011>.
- Rekhter, Y., B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear. 1996. “Address Allocation for Private Internets.” IETF RFC 1918. <https://www.rfc-editor.org/rfc/rfc1918>.
- Rescorla, E. 2018. “The Transport Layer Security (TLS) Protocol Version 1.3.” IETF RFC 8446. <https://www.rfc-editor.org/rfc/rfc8446>.
- Saltzer, Jerome H., and M. Frans Kaashoek. 2009. *Principles of Computer System Design: An Introduction*. Morgan Kaufmann.
- Saltzer, Jerome H., and Michael D. Schroeder. 1975. “The Protection of Information in Computer Systems” 63 (9): 1278–308. <https://doi.org/10.1109/PROC.1975.9939>.
- “Signals.” 2026. OpenTelemetry documentation. <https://opentelemetry.io/docs/concepts/signals/>.
- Silberschatz, Abraham, Peter B. Galvin, and Greg Gagne. 2018. *Operating System Concepts*. 10th ed. Wiley.
- Solove, Daniel J. 2006. “A Taxonomy of Privacy.” *University of Pennsylvania Law Review* 154.
- “Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model.” 2023. ISO/IEC 25010:2023. <https://www.iso.org/standard/78176.html>.
- Tabassi, Elham. 2023. National Institute of Standards; Technology. <https://doi.org/10.6028/NIST.AI.100-1>.
- Tanenbaum, Andrew S., and Herbert Bos. 2022. *Modern Operating Systems*. 5th ed. Pearson.
- Temoshok, David et al. 2025. “Digital Identity Guidelines.” NIST SP 800-63-4. National Institute of Standards; Technology. <https://doi.org/10.6028/NIST.SP.800-63-4>.
- The Linux Kernel Documentation. 2026a. “CPU Performance Scaling.” kernel.org documentation. <https://docs.kernel.org/admin-guide/pm/cpufreq.html>.
- . 2026b. “CPU Scheduler.” kernel.org documentation. <https://docs.kernel.org/scheduler/>.
- . 2026c. “Introduction (Linux Input Subsystem Userspace API).” kernel.org documentation. <https://docs.kernel.org/input/input.html>.
- Trusted Computing Group. 2023. “TCG PC Client Platform Firmware Profile Specification.” Family “2.0,” Level 00, Version 1.06, Revision 52 (published 2023-12-04). https://trustedcomputinggroup.org/wp-content/uploads/TCG-PC-Client-Platform-Firmware-Profile-Version-1.06-Revision-52_pub-3.pdf.
- UEFI Forum. n.d. “Secure Boot and Driver Signing.” UEFI Specification Version 2.10, Chapter 32 (HTML). https://uefi.org/specs/UEFI/2.10/32_Secure_Boot_and_Driver_Signing.html.
- UL Standards & Engagement. 2021. “Household and Commercial Batteries.” UL 2054, Edition 3 (published 2021-11-17; revision 2022-03-10). <https://www.shopulstandards.com/ProductDetail.aspx?productId=UL2054>.
- “Vocabulary for Performance, Quality of Service and Quality of Experience.” n.d. Recommendation ITU-T P.10/G.100. <https://www.itu.int/rec/T-REC-P.10/en>.
- “Vulkan Overview.” 2026. Living documentation hub. <https://www.khronos.org/vulkan/>.
- W3C. 2023. “Web Content Accessibility Guidelines (WCAG) 2.2.” W3C Recommendation 5 October 2023. <https://www.w3.org/TR/2023/REC-WCAG22-20231005/>.
- . 2024. “Web Content Accessibility Guidelines (WCAG) 2.2.” W3C Recommendation 12 December 2024. <https://www.w3.org/TR/2024/REC-WCAG22-20241212/>.
- . 2026a. “Pointer Events.” W3C Recommendation. <https://www.w3.org/TR/pointerevents3/>.
- . 2026b. “UI Events.” W3C Working Draft. <https://www.w3.org/TR/uievents/>.

- “Waikere-research-ops Accepted Main Curriculum Evidence.” 2026. GitHub repository. <https://github.com/gunnchOS3k/waikere-research-ops>.
- WHATWG. 2026a. “DOM Standard.” Living Standard. <https://dom.spec.whatwg.org/>.
- . 2026b. “HTML Standard.” Living Standard. <https://html.spec.whatwg.org/>.
- “Window: requestAnimationFrame() Method.” 2026. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame>.